

■ AlliedEdge

Full Source Code Export

<https://github.com/Rayan-Mohammed-Rafeeq/AlliedEdge>

Total files included: **244**

Table of Contents

1. backend/Dockerfile
2. backend/pom.xml
3. backend/render.yaml
4. backend/src/main/java/org/AE/alliededge/AlliedEdgeApplication.java
5. backend/src/main/java/org/AE/alliededge/config/CloudinaryConfig.java
6. backend/src/main/java/org/AE/alliededge/config/CorsConfig.java
7. backend/src/main/java/org/AE/alliededge/config/CustomOAuth2UserService.java
8. backend/src/main/java/org/AE/alliededge/config/DataInitializer.java
9. backend/src/main/java/org/AE/alliededge/config/FileUploadConfig.java
10. backend/src/main/java/org/AE/alliededge/config/FlywayRepairConfig.java
11. backend/src/main/java/org/AE/alliededge/config/FlywayRepairRunner.java
12. backend/src/main/java/org/AE/alliededge/config/FrontendRedirectAuthenticationSuccessHandler.java
13. backend/src/main/java/org/AE/alliededge/config/GlobalModelAttributes.java
14. backend/src/main/java/org/AE/alliededge/config/HomeRedirectAuthenticationSuccessHandler.java
15. backend/src/main/java/org/AE/alliededge/config/JacksonConfig.java
16. backend/src/main/java/org/AE/alliededge/config/NoopCloudinaryServiceConfig.java
17. backend/src/main/java/org/AE/alliededge/config/SecurityConfig.java
18. backend/src/main/java/org/AE/alliededge/config/WebConfig.java
19. backend/src/main/java/org/AE/alliededge/config/WebSocketConfig.java
20. backend/src/main/java/org/AE/alliededge/controller/AdminController.java
21. backend/src/main/java/org/AE/alliededge/controller/AnnouncementController.java
22. backend/src/main/java/org/AE/alliededge/controller/AppErrorController.java
23. backend/src/main/java/org/AE/alliededge/controller/AuthController.java
24. backend/src/main/java/org/AE/alliededge/controller/AuthorPostController.java
25. backend/src/main/java/org/AE/alliededge/controller/ChatController.java
26. backend/src/main/java/org/AE/alliededge/controller/CommentController.java
27. backend/src/main/java/org/AE/alliededge/controller/CommentReplyController.java
28. backend/src/main/java/org/AE/alliededge/controller/CsrfController.java
29. backend/src/main/java/org/AE/alliededge/controller/FollowController.java
30. backend/src/main/java/org/AE/alliededge/controller/HealthController.java
31. backend/src/main/java/org/AE/alliededge/controller/HomeController.java
32. backend/src/main/java/org/AE/alliededge/controller/PostController.java
33. backend/src/main/java/org/AE/alliededge/controller/ProfileController.java
34. backend/src/main/java/org/AE/alliededge/controller/SetupUsernameController.java
35. backend/src/main/java/org/AE/alliededge/controller/UserController.java
36. backend/src/main/java/org/AE/alliededge/controller/WsDebugController.java
37. backend/src/main/java/org/AE/alliededge/dto/AvailabilityDto.java
38. backend/src/main/java/org/AE/alliededge/dto/ChatMessageDTO.java
39. backend/src/main/java/org/AE/alliededge/dto/ChatRoomDTO.java
40. backend/src/main/java/org/AE/alliededge/dto/CommentDto.java
41. backend/src/main/java/org/AE/alliededge/dto/EducationDto.java
42. backend/src/main/java/org/AE/alliededge/dto/ExperienceDto.java
43. backend/src/main/java/org/AE/alliededge/dto/LanguageDto.java
44. backend/src/main/java/org/AE/alliededge/dto/PostDetailDto.java
45. backend/src/main/java/org/AE/alliededge/dto/PostFeedItemDto.java
46. backend/src/main/java/org/AE/alliededge/dto/PostPageDto.java
47. backend/src/main/java/org/AE/alliededge/dto/PostStatsDto.java
48. backend/src/main/java/org/AE/alliededge/dto/ProfileResponseDto.java
49. backend/src/main/java/org/AE/alliededge/dto/ProfileSectionsDto.java
50. backend/src/main/java/org/AE/alliededge/dto/ProjectDto.java
51. backend/src/main/java/org/AE/alliededge/dto/PublicProfileUserDto.java
52. backend/src/main/java/org/AE/alliededge/dto/SendChatMessageRequest.java
53. backend/src/main/java/org/AE/alliededge/exception/ApiExceptionHandler.java
54. backend/src/main/java/org/AE/alliededge/exception/GlobalExceptionHandler.java
55. backend/src/main/java/org/AE/alliededge/exception/ResourceNotFoundException.java
56. backend/src/main/java/org/AE/alliededge/model/Announcement.java
57. backend/src/main/java/org/AE/alliededge/model/ChatMessage.java

58. backend/src/main/java/org/AE/alliededge/model/ChatRoom.java
59. backend/src/main/java/org/AE/alliededge/model/Comment.java
60. backend/src/main/java/org/AE/alliededge/model/CommentLike.java
61. backend/src/main/java/org/AE/alliededge/model/CommentReply.java
62. backend/src/main/java/org/AE/alliededge/model/CommentReplyLike.java
63. backend/src/main/java/org/AE/alliededge/model/Post.java
64. backend/src/main/java/org/AE/alliededge/model/PostLike.java
65. backend/src/main/java/org/AE/alliededge/model/PostView.java
66. backend/src/main/java/org/AE/alliededge/model/Project.java
67. backend/src/main/java/org/AE/alliededge/model/User.java
68. backend/src/main/java/org/AE/alliededge/model/UserFollow.java
69. backend/src/main/java/org/AE/alliededge/repository/AnnouncementRepository.java
70. backend/src/main/java/org/AE/alliededge/repository/ChatMessageRepository.java
71. backend/src/main/java/org/AE/alliededge/repository/ChatRoomRepository.java
72. backend/src/main/java/org/AE/alliededge/repository/CommentLikeRepository.java
73. backend/src/main/java/org/AE/alliededge/repository/CommentReplyLikeRepository.java
74. backend/src/main/java/org/AE/alliededge/repository/CommentReplyRepository.java
75. backend/src/main/java/org/AE/alliededge/repository/CommentRepository.java
76. backend/src/main/java/org/AE/alliededge/repository/PostLikeRepository.java
77. backend/src/main/java/org/AE/alliededge/repository/PostRepository.java
78. backend/src/main/java/org/AE/alliededge/repository/PostViewRepository.java
79. backend/src/main/java/org/AE/alliededge/repository/ProjectRepository.java
80. backend/src/main/java/org/AE/alliededge/repository/UserFollowRepository.java
81. backend/src/main/java/org/AE/alliededge/repository/UserRepository.java
82. backend/src/main/java/org/AE/alliededge/service/AnnouncementService.java
83. backend/src/main/java/org/AE/alliededge/service/AnnouncementUpdateRequest.java
84. backend/src/main/java/org/AE/alliededge/service/ChatService.java
85. backend/src/main/java/org/AE/alliededge/service/CloudinaryService.java
86. backend/src/main/java/org/AE/alliededge/service/CloudinaryServiceImpl.java
87. backend/src/main/java/org/AE/alliededge/service/CommentLikeService.java
88. backend/src/main/java/org/AE/alliededge/service/CommentReplyLikeService.java
89. backend/src/main/java/org/AE/alliededge/service/CommentReplyService.java
90. backend/src/main/java/org/AE/alliededge/service/CommentService.java
91. backend/src/main/java/org/AE/alliededge/service/ContentLinkifier.java
92. backend/src/main/java/org/AE/alliededge/service/CustomUserDetailsService.java
93. backend/src/main/java/org/AE/alliededge/service/JwtService.java
94. backend/src/main/java/org/AE/alliededge/service/PostRealtimePublisher.java
95. backend/src/main/java/org/AE/alliededge/service/PostService.java
96. backend/src/main/java/org/AE/alliededge/service/ProfileService.java
97. backend/src/main/java/org/AE/alliededge/service/ProfileServiceImpl.java
98. backend/src/main/java/org/AE/alliededge/service/ProjectService.java
99. backend/src/main/java/org/AE/alliededge/service/ResumeService.java
100. backend/src/main/java/org/AE/alliededge/service/UserFollowService.java
101. backend/src/main/java/org/AE/alliededge/service/UserService.java
102. backend/src/main/resources/application.properties
103. backend/src/main/resources/templates/error/404.html
104. backend/src/main/resources/templates/error/generic-error.html
105. backend/src/test/java/org/AE/alliededge/AlliedEdgeApplicationTests.java
106. backend/src/test/java/org/AE/alliededge/config/TestCloudinaryConfig.java
107. backend/src/test/java/org/AE/alliededge/controller/AnnouncementsPublicWebMvcTest.java
108. backend/src/test/java/org/AE/alliededge/controller/AuthControllerWebMvcTest.java
109. backend/src/test/java/org/AE/alliededge/controller/ChatMutualFollowSecurityTest.java
110. backend/src/test/java/org/AE/alliededge/controller/ChatPaginationWebMvcTest.java
111. backend/src/test/java/org/AE/alliededge/controller/PostDetailJsonTest.java
112. backend/src/test/java/org/AE/alliededge/controller/PostFeedCommentCountWebMvcTest.java
113. backend/src/test/java/org/AE/alliededge/controller/ProfileControllerTest.java
114. backend/src/test/java/org/AE/alliededge/controller/ProfileEducationPersistenceTest.java
115. backend/src/test/java/org/AE/alliededge/controller/ProfileResumeDeleteTest.java
116. backend/src/test/java/org/AE/alliededge/controller/PublicProfileByUsernameWebMvcTest.java

117. backend/src/test/java/org/AE/alliededge/controller/PublicProfileSecurityWebMvcTest.java
118. backend/src/test/java/org/AE/alliededge/controller/UserSearchExcludesSelfWebMvcTest.java
119. backend/src/test/java/org/AE/alliededge/PostSearchTest.java
120. backend/src/test/java/org/AE/alliededge/service/FakeCloudinaryService.java
121. docker-compose.yml
122. frontend/components.json
123. frontend/docs/google-login.md
124. frontend/eslint.config.js
125. frontend/index.html
126. frontend/main.ts
127. frontend/package.json
128. frontend/public/robots.txt
129. frontend/public/sitemap.xml
130. frontend/src/components/AnimatedBackground.tsx
131. frontend/src/components/AutoPlayMutedVideo.tsx
132. frontend/src/components/CommentCard.tsx
133. frontend/src/components/CreateCommentCard.tsx
134. frontend/src/components/CreatePostCard.tsx
135. frontend/src/components/DefaultAvatarFallback.tsx
136. frontend/src/components/EditProfileDialog.tsx
137. frontend/src/components/ImageCarousel.tsx
138. frontend/src/components/LogoDropdown.tsx
139. frontend/src/components/Navigation.tsx
140. frontend/src/components/PostCard.tsx
141. frontend/src/components/ProtectedRoute.tsx
142. frontend/src/components/RightSidebar.tsx
143. frontend/src/components/ui/accordion.tsx
144. frontend/src/components/ui/alert-dialog.tsx
145. frontend/src/components/ui/alert.tsx
146. frontend/src/components/ui/aspect-ratio.tsx
147. frontend/src/components/ui/avatar.tsx
148. frontend/src/components/ui/badge.tsx
149. frontend/src/components/ui/breadcrumb.tsx
150. frontend/src/components/ui/button.tsx
151. frontend/src/components/ui/calendar.tsx
152. frontend/src/components/ui/card.tsx
153. frontend/src/components/ui/carousel.tsx
154. frontend/src/components/ui/chart.tsx
155. frontend/src/components/ui/checkbox.tsx
156. frontend/src/components/ui/collapsible.tsx
157. frontend/src/components/ui/command.tsx
158. frontend/src/components/ui/context-menu.tsx
159. frontend/src/components/ui/dialog.tsx
160. frontend/src/components/ui/drawer.tsx
161. frontend/src/components/ui/dropdown-menu.tsx
162. frontend/src/components/ui/form.tsx
163. frontend/src/components/ui/hover-card.tsx
164. frontend/src/components/ui/index.ts
165. frontend/src/components/ui/input-otp.tsx
166. frontend/src/components/ui/input.tsx
167. frontend/src/components/ui/label.tsx
168. frontend/src/components/ui/menubar.tsx
169. frontend/src/components/ui/navigation-menu.tsx
170. frontend/src/components/ui/pagination.tsx
171. frontend/src/components/ui/popover.tsx
172. frontend/src/components/ui/progress.tsx
173. frontend/src/components/ui/radio-group.tsx
174. frontend/src/components/ui/resizable.tsx
175. frontend/src/components/ui/scroll-area.tsx

176. frontend/src/components/ui/select.tsx
177. frontend/src/components/ui/separator.tsx
178. frontend/src/components/ui/sheet.tsx
179. frontend/src/components/ui/sidebar.tsx
180. frontend/src/components/ui/skeleton.tsx
181. frontend/src/components/ui/slider.tsx
182. frontend/src/components/ui/sonner.tsx
183. frontend/src/components/ui/switch.tsx
184. frontend/src/components/ui/table.tsx
185. frontend/src/components/ui/tabs.tsx
186. frontend/src/components/ui/textarea.tsx
187. frontend/src/components/ui/toggle-group.tsx
188. frontend/src/components/ui/toggle.tsx
189. frontend/src/components/ui/tooltip.tsx
190. frontend/src/components/VerifiedBadge.tsx
191. frontend/src/contexts/AuthContext.tsx
192. frontend/src/contexts/ThemeContext.tsx
193. frontend/src/hooks/use-auth.ts
194. frontend/src/hooks/use-mobile.ts
195. frontend/src/hooks/usePostRealtime.ts
196. frontend/src/index.css
197. frontend/src/instrumentation.tsx
198. frontend/src/lib/api.ts
199. frontend/src/lib/chatRealtime.ts
200. frontend/src/lib/formatCount.ts
201. frontend/src/lib/postRealtime.ts
202. frontend/src/lib/utils.ts
203. frontend/src/main.tsx
204. frontend/src/pages/Admin.tsx
205. frontend/src/pages/AdminAnnouncements.tsx
206. frontend/src/pages/AdminDashboard.tsx
207. frontend/src/pages/AdminUserPosts.tsx
208. frontend/src/pages/AdminUsers.tsx
209. frontend/src/pages/AnnouncementCreate.tsx
210. frontend/src/pages/AnnouncementEdit.tsx
211. frontend/src/pages/Announcements.tsx
212. frontend/src/pages/Auth.tsx
213. frontend/src/pages/ChangeUsername.tsx
214. frontend/src/pages/ChangeUsernameError.tsx
215. frontend/src/pages/Chat.tsx
216. frontend/src/pages/ChatDashboard.tsx
217. frontend/src/pages/EditProfile.tsx
218. frontend/src/pages/ErrorMessage.tsx
219. frontend/src/pages/Feed.tsx
220. frontend/src/pages/Help.tsx
221. frontend/src/pages/Landing.tsx
222. frontend/src/pages/Login.tsx
223. frontend/src/pages/MyPosts.tsx
224. frontend/src/pages/NotFound.tsx
225. frontend/src/pages/PostDetail.tsx
226. frontend/src/pages/PostForm.tsx
227. frontend/src/pages/Privacy.tsx
228. frontend/src/pages/Profile.tsx
229. frontend/src/pages/Register.tsx
230. frontend/src/pages/SetupUsername.tsx
231. frontend/src/pages/Terms.tsx
232. frontend/src/pages/UserSearch.tsx
233. frontend/src/types/global.d.ts
234. frontend/src/types/sockjs-client.d.ts

- 235. frontend/src/vite-env.d.ts
- 236. frontend/sst-env.d.ts
- 237. frontend/tsconfig.app.json
- 238. frontend/tsconfig.json
- 239. frontend/tsconfig.node.json
- 240. frontend/vercel.json
- 241. frontend/vite.config.ts
- 242. frontend/vly-toolbar-readonly.tsx
- 243. LICENSE
- 244. README.md

■ backend/Dockerfile

```
# ----- BUILD STAGE -----
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app

# Copy pom.xml first for dependency caching
COPY pom.xml .
RUN mvn dependency:go-offline

# Copy source code
COPY src ./src

# Build the application
RUN mvn clean package -DskipTests

# ----- RUNTIME STAGE -----
FROM eclipse-temurin:17-jre

WORKDIR /app

# Copy only the jar from build stage
COPY --from=builder /app/target/*.jar app.jar

# Expose Spring Boot port
EXPOSE 8080

# Run the app
ENTRYPOINT ["java", "-jar", "app.jar"]
```

■ backend/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.7</version>
        <relativePath></relativePath> <!-- lookup parent from repository -->
    </parent>
    <groupId>org.AE</groupId>
    <artifactId>backend</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>AlliedEdge-Blog</name>
    <description>AlliedEdge-Blog</description>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
```

```

</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-database-postgresql</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
<!-- Cloudinary Java SDK v2 (HTTP5) -->
<dependency>
    <groupId>com.cloudinary</groupId>
    <artifactId>cloudinary-http5</artifactId>
    <version>2.0.0</version>
</dependency>
<dependency>
    <groupId>com.cloudinary</groupId>
    <artifactId>cloudinary-taglib</artifactId>
    <version>2.0.0</version>
</dependency>
<!-- dotenv for loading Cloudinary credentials from .env in dev -->
<dependency>
    <groupId>io.github.cdimascio</groupId>
    <artifactId>dotenv-java</artifactId>
    <version>2.2.4</version>
</dependency>
<!-- WebSocket for real-time chat -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
<!-- Nimbus JOSE JWT for ID token verification and signing -->
<dependency>
    <groupId>com.nimbusds</groupId>
    <artifactId>nimbus-jose-jwt</artifactId>

```



```

        <version>9.37.4</version>;
    </dependency>;
</dependency>;
    <groupId>me.paulschwarz</groupId>;
    <artifactId>spring-dotenv</artifactId>;
    <version>4.0.0</version>;
    </dependency>;
</dependencies>;
<build>;
    <plugins>;
        <plugin>;
            <groupId>org.springframework.boot</groupId>;
            <artifactId>spring-boot-maven-plugin</artifactId>;
        </plugin>;
    </plugins>;
</build>;
</project>;

```

■ backend/render.yaml

```

# Exported from Render on 2026-02-14T12:32:19Z
version: "1"
databases:
- name: alliededge-postgres
  databaseName: alliededge_prod
  user: alliededge_prod_user
  plan: free
  region: singapore
  ipAllowList:
  - source: 0.0.0.0/0
    description: everywhere
  postgresMajorVersion: "18"

```

■ backend/src/main/java/org/AE/alliededge/AlliedEdgeApplication.java

```

package org.AE.alliededge;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AlliedEdgeApplication {

    public static void main(String[] args) {
        SpringApplication.run(AlliedEdgeApplication.class, args);
    }

}

```

■ backend/src/main/java/org/AE/alliededge/config/CloudinaryConfig.java

```

package org.AE.alliededge.config;

import com.cloudinary.Cloudinary;
import io.github.cdimascio.dotenv.Dotenv;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;

import java.util.HashMap;
import java.util.Map;

@Configuration
public class CloudinaryConfig {

    @Bean

```

```

@ConditionalOnProperty(name = "clouddinary.enabled", havingValue = "true")
public Clouddinary clouddinary() {
    // Load from environment or .env file (dotenv will also see real env vars)
    Dotenv dotenv = Dotenv.configure()
        .ignoreIfMissing()
        .load();

    String cloudName = getenvOrDefault("CLOUDINARY_CLOUD_NAME", dotenv);
    String apiKey = getenvOrDefault("CLOUDINARY_API_KEY", dotenv);
    String apiSecret = getenvOrDefault("CLOUDINARY_API_SECRET", dotenv);

    // clouddinary.enabled gate should prevent missing credentials, but keep a clear message
    // if misconfigured.
    if (cloudName == null || apiKey == null || apiSecret == null) {
        throw new IllegalStateException(
            "Clouddinary is enabled but credentials are missing. Please set CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET");
    }

    Map<String, Object> config = new HashMap<>();
    config.put("cloud_name", cloudName);
    config.put("api_key", apiKey);
    config.put("api_secret", apiSecret);
    config.put("secure", true);

    return new Clouddinary(config);
}

private String getenvOrDefault(String key, Dotenv dotenv) {
    String fromEnv = System.getenv(key);
    if (fromEnv != null && !fromEnv.isBlank()) {
        return fromEnv;
    }
    String fromDotEnv = dotenv.get(key);
    if (fromDotEnv != null && !fromDotEnv.isBlank()) {
        return fromDotEnv;
    }
    return null;
}
}

```

■ backend/src/main/java/org/AE/alliededge/config/CorsConfig.java

```

package org.AE.alliededge.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.Arrays;
import java.util.List;

@Configuration
public class CorsConfig {

    /**
     * Comma-separated list of allowed origins.
     * Example: http://localhost:5173,http://127.0.0.1:5173
     */
    * Why this is needed now:
    * - With Thymeleaf, the browser loaded pages and called endpoints from the SAME origin as the backend,
    *   so cookies (JSESSIONID) were sent automatically and the browser didn't enforce CORS.
    * - With a React SPA on a DIFFERENT origin (http://localhost:5173), the browser blocks cross-origin
    *   requests unless the backend explicitly allows them via CORS.
    * - Because auth is session-based, the SPA must be allowed to send cookies; that requires
    *   allowCredentials(true) AND an explicit (non-"" ) allowed origin.
    */
}

```

```

@Value("${app.cors.allowed-origins:http://localhost:5173,http://127.0.0.1:5173}")
private String allowedOrigins;

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();

    List<String> origins = Arrays.stream(allowedOrigins.split("\\s*,\\s*"))
        .map(String::trim)
        .filter(s -> !s.isBlank())
        .distinct()
        .toList();

    // NOTE: When allowCredentials(true), you must NOT use "*" for allowed origins.
    config.setAllowedOrigins(origins);
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("*"));

    // Cookies (JSESSIONID) + CSRF cookie/header will be used by the SPA.
    // Without allowCredentials(true), browsers will omit cookies on cross-origin XHR/fetch.
    config.setAllowCredentials(true);

    // Useful for debugging / integrating CSRF with SPA.
    config.setExposedHeaders(List.of("Location", "Set-Cookie", "X-CSRF-TOKEN"));

    // Cache preflight responses in browsers.
    config.setMaxAge(3600L);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    // Apply to all paths so session login/logout and API calls work cross-origin.
    source.registerCorsConfiguration("/**", config);
    return source;
}
}

```

■ backend/src/main/java/org/AE/alliededge/config/CustomOAuth2UserService.java

```

package org.AE.alliededge.config;

import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.oauth2.client.userinfo.DefaultOAuth2UserService;
import org.springframework.security.oauth2.client.userinfo.OAuth2UserRequest;
import org.springframework.security.oauth2.client.userinfo.OAuth2UserService;
import org.springframework.security.oauth2.core.OAuth2AuthenticationException;
import org.springframework.security.oauth2.core.user.DefaultOAuth2User;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.stereotype.Service;

import java.util.*;

@Service
public class CustomOAuth2UserService implements OAuth2UserService<OAuth2UserRequest, OAuth2User> {

    private final UserRepository userRepository;

    public CustomOAuth2UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public OAuth2User loadUser(OAuth2UserRequest userRequest) throws OAuth2AuthenticationException {
        DefaultOAuth2UserService delegate = new DefaultOAuth2UserService();
        OAuth2User oAuth2User = delegate.loadUser(userRequest);

        // Extract email from Google's userinfo
        String email = (String) oAuth2User.getAttributes().get("email");
        if (email == null || email.isBlank()) {

```

```

        throw new OAuth2AuthenticationException("Email not found in Google profile");
    }

    // Determine role based on email
    boolean isSpecialAdminEmail = email.equalsIgnoreCase("rayanmohammedrafeeq@gmail.com")
        || email.equalsIgnoreCase("rakinmohammedrafeeq@gmail.com");

    // Determine display name from Google profile if available
    Object nameAttr = oauth2User.getAttributes().get("name");
    String displayName = nameAttr != null ? nameAttr.toString() : null;
    if (displayName == null || displayName.isBlank()) {
        displayName = email.substring(0, email.indexOf('@'));
    }

    // Create or update user in DB using email as canonical key
    User user = userRepository.findByEmail(email).orElseGet(User::new);

    // If the user exists and is banned, prevent login.
    if (user.getId() != null && user.isBanned()) {
        throw new OAuth2AuthenticationException("ACCOUNT_BANNED");
    }

    boolean isNew = (user.getId() == null);
    if (isNew) {
        user.setEmail(email);
        // Prefill username from email local-part on first login
        String emailLocalPart = email.substring(0, email.indexOf('@'));
        user.setUsername(emailLocalPart);
        user.setPassword("GOOGLE_LOGIN");
        user.setFirstLogin(true); // Set first login flag for new users
    }

    // Role assignment:
    // - Never overwrite an existing role for non-admin accounts (e.g. ROLE_AUTHOR stays ROLE_AUTHOR).
    // - Only force ROLE_ADMIN for special admin emails.
    // - If role is missing, default to ROLE_AUTHOR.
    if (isSpecialAdminEmail) {
        user.setRole("ROLE_ADMIN");
        user.setAdmin(true);
    } else {
        if (user.getRole() == null || user.getRole().isBlank()) {
            user.setRole("ROLE_AUTHOR");
        }
        user.setAdmin(false);
    }

    // Set display name
    if (isSpecialAdminEmail) {
        // Preserve existing custom display names if already set
        if (user.getDisplayName() == null || user.getDisplayName().isBlank()) {
            if (email.equalsIgnoreCase("rayanmohammedrafeeq@gmail.com")) {
                user.setDisplayName("Rayan Mohammed Rafeeq & Rakin Mohammed Rafeeq");
            } else if (email.equalsIgnoreCase("rakinmohammedrafeeq@gmail.com")) {
                user.setDisplayName("Rakin Mohammed Rafeeq & Rayan Mohammed Rafeeq");
            }
        }
    } else {
        user.setDisplayName(displayName);
    }

    userRepository.save(user);

    // Build authorities for Spring Security from stored role
    String storedRole = user.getRole();
    String authority = storedRole != null && storedRole.startsWith("ROLE_")
        ? storedRole
        : "ROLE_" + storedRole;
    Collection<GrantedAuthority> authorities = List.of(new SimpleGrantedAuthority(authority));

    // Return a DefaultOAuth2User with our mapped authorities and attributes
    Map<String, Object> attrs = new HashMap<>(oauth2User.getAttributes());

```

```

        // Expose both email (canonical principal name) and the app username handle
        attrs.put("email", email);
        attrs.put("appUsername", user.getUsername());

        // IMPORTANT: Use email as the principal name so authentication.getName()
        // returns the stable Google email across logins, even if username changes.
        return new DefaultOAuth2User(authorities, attrs, "email");
    }
}

```

■ backend/src/main/java/org/AE/alliededge/config/DataInitializer.java

```

package org.AE.alliededge.config;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class DataInitializer implements CommandLineRunner {
    @Override
    public void run(String... args) {
        // Admin seeding handled by Flyway migration V6__seed_admin_user.sql
    }
}

```

■ backend/src/main/java/org/AE/alliededge/config/FileUploadConfig.java

```

package org.AE.alliededge.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class FileUploadConfig implements WebMvcConfigurer {
    // This configuration class is now obsolete because all media files are served from Cloudinary.
    // It is kept as an empty shell to avoid bean resolution issues if referenced, but does nothing.
}

```

■ backend/src/main/java/org/AE/alliededge/config/FlywayRepairConfig.java

```

package org.AE.alliededge.config;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.boot.autoconfigure.flyway.FlywayMigrationStrategy;

/**
 * Repairs Flyway schema history (useful after editing an already applied migration)
 * BEFORE running migrate, preventing checksum mismatch failures at startup.
 *
 * NOTE: Remove this class once the repair has executed successfully (one-time fix).
 */
@Configuration
public class FlywayRepairConfig {
    private static final Logger log = LoggerFactory.getLogger(FlywayRepairConfig.class);

    @Bean
    public FlywayMigrationStrategy flywayMigrationStrategy() {
        return flyway -> {
            try {
                log.warn("[FlywayRepair] Executing flyway.repair() prior to migrate to resolve checksum mismatches.");
                flyway.repair();
                log.info("[FlywayRepair] Repair completed. Proceeding with migrate...");
                flyway.migrate();
            }

```

```

        log.info("[FlywayRepair] Migrate completed successfully.");
    } catch (Exception ex) {
        log.error("[FlywayRepair] Failed during Flyway repair/migrate sequence", ex);
        throw ex; // propagate so startup still fails if unrecoverable
    }
}
};
}
}

```

■ backend/src/main/java/org/AE/alliededge/config/FlywayRepairRunner.java

```

package org.AE.alliededge.config;

import org.flywaydb.core.Flyway;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.autoconfigure.condition.ConditionalOnBean;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.core.Ordered;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

/**
 * Temporary runner to automatically repair Flyway schema history on startup
 * in order to fix checksum mismatches (e.g. for migration version 12).
 *
 * IMPORTANT: Remove this class once the repair has successfully run once
 * and the application starts cleanly.
 */
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
@ConditionalOnProperty(value = "flyway.repair.enabled", havingValue = "true")
@ConditionalOnBean(Flyway.class)
public class FlywayRepairRunner implements CommandLineRunner {

    private static final Logger logger = LoggerFactory.getLogger(FlywayRepairRunner.class);

    private final Flyway flyway;

    public FlywayRepairRunner(Flyway flyway) {
        this.flyway = flyway;
    }

    @Override
    public void run(String... args) {
        logger.warn("Running Flyway.repair() at startup to fix migration metadata issues (temporary runner)");
        try {
            flyway.repair();
            logger.info("Flyway schema repair executed successfully.");
        } catch (Exception ex) {
            logger.error("Flyway repair failed", ex);
        }
    }
}

```

■ backend/src/main/java/org/AE/alliededge/config/FrontendRedirectAuthenticationSuccessHandler.java

```

package org.AE.alliededge.config;

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.Authentication;
import org.springframework.security.web.authentication.AuthenticationSuccessHandler;
import org.springframework.stereotype.Component;

```

```

import java.io.IOException;

/**
 * After a successful login (form or OAuth2), redirect the user back to the frontend.
 *
 * Frontend only needs to hit /oauth2/authorization/google. The backend completes
 * the OAuth2 flow and then redirects to the SPA.
 */
@Component
public class FrontendRedirectAuthenticationSuccessHandler implements AuthenticationSuccessHandler {

    private final String frontendRedirectUrl;

    public FrontendRedirectAuthenticationSuccessHandler(
        @Value("${app.frontend.redirect-url:http://localhost:5173/}") String frontendRedirectUrl
    ) {
        this.frontendRedirectUrl = frontendRedirectUrl;
    }

    @Override
    public void onAuthenticationSuccess(HttpServletRequest request,
                                       HttpServletResponse response,
                                       Authentication authentication) throws IOException, ServletException {
        // Ensure the session exists so Spring Security can store the Authentication
        request.getSession(true);

        // Redirect back to the SPA. From there you can call backend APIs with cookies (credentials: 'include').
        response.sendRedirect(frontendRedirectUrl);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/config/GlobalModelAttributes.java

```

package org.AE.alliededge.config;

import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.oauth2.client.authentication.OAuth2AuthenticationToken;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.stereotype.Component;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ModelAttribute;

import java.util.Collection;
import java.util.Map;
import java.util.Optional;

@ControllerAdvice
@Component
public class GlobalModelAttributes {

    private static final String DEFAULT_AVATAR_PATH = "/images/default-profile.jpg";

    private final UserRepository userRepository;

    public GlobalModelAttributes(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @ModelAttribute
    public void addCurrentUser(Model model, @AuthenticationPrincipal Object principal) {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
        if (authentication == null || !authentication.isAuthenticated()) {
            model.addAttribute("currentUser", null);
        }
    }
}

```

```

        model.addAttribute("isAdmin", false);
        model.addAttribute("isAuthenticated", false);
        model.addAttribute("currentAvatarUrl", DEFAULT_AVATAR_PATH);
        model.addAttribute("currentDisplayName", null);
        return;
    }

    // Spring Security will often use the literal string "anonymousUser" for unauthenticated requests
    if ("anonymousUser".equals(authentication.getPrincipal())) {
        model.addAttribute("currentUser", null);
        model.addAttribute("isAdmin", false);
        model.addAttribute("isAuthenticated", false);
        model.addAttribute("currentAvatarUrl", DEFAULT_AVATAR_PATH);
        model.addAttribute("currentDisplayName", null);
        return;
    }

    String email = null;

    // Username/password login
    if (principal instanceof UserDetails userDetails) {
        email = userDetails.getUsername();
    }

    // OAuth2 login (Google)
    else if (authentication instanceof OAuth2AuthenticationToken oauthToken) {
        OAuth2User oauthUser = oauthToken.getPrincipal();
        if (oauthUser != null) {
            Map<String, Object> attributes = oauthUser.getAttributes();
            Object emailAttr = attributes.get("email");
            if (emailAttr != null) {
                email = emailAttr.toString();
            }
        }
    }

    if (email == null || email.isBlank()) {
        model.addAttribute("currentUser", null);
        model.addAttribute("isAdmin", false);
        model.addAttribute("isAuthenticated", false);
        model.addAttribute("currentAvatarUrl", DEFAULT_AVATAR_PATH);
        model.addAttribute("currentDisplayName", null);
        return;
    }

    Optional<User> userOpt = userRepository.findByEmail(email);
    if (userOpt.isEmpty()) {
        model.addAttribute("currentUser", null);
        model.addAttribute("isAdmin", false);
        model.addAttribute("isAuthenticated", false);
        model.addAttribute("currentAvatarUrl", DEFAULT_AVATAR_PATH);
        model.addAttribute("currentDisplayName", null);
        return;
    }

    User user = userOpt.get();
    model.addAttribute("currentUser", user);
    model.addAttribute("isAuthenticated", true);

    // Determine admin flag either from user entity or from authorities
    boolean isAdmin = user.isAdmin();
    if (!isAdmin && authentication != null) {
        Collection<? extends GrantedAuthority> authorities = authentication.getAuthorities();
        if (authorities != null) {
            isAdmin = authorities.stream().anyMatch(a ->
                "ROLE_ADMIN".equals(a.getAuthority()) ||
                "ADMIN".equalsIgnoreCase(a.getAuthority()));
        }
    }
    model.addAttribute("isAdmin", isAdmin);

    // Avatar URL and display name helpers for templates

```



```

        String avatarUrl = user.getProfileImageUrl();
        if (avatarUrl == null || avatarUrl.isBlank()) {
            avatarUrl = DEFAULT_AVATAR_PATH;
        }
        model.addAttribute("currentAvatarUrl", avatarUrl);

        String displayName = user.getDisplayName();
        if (displayName == null || displayName.isBlank()) {
            displayName = user.getUsername();
        }
        model.addAttribute("currentDisplayName", displayName);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/config/HomeRedirectAuthenticationSuccessHandler.java

```

package org.AE.alliededge.config;

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.core.Authentication;
import org.springframework.security.web.authentication.AuthenticationSuccessHandler;
import org.springframework.stereotype.Component;

import java.io.IOException;

/**
 * Custom AuthenticationSuccessHandler that ALWAYS redirects to "/" after
 * a successful login (form or OAuth2).
 */
@Component
public class HomeRedirectAuthenticationSuccessHandler implements AuthenticationSuccessHandler {

    @Override
    public void onAuthenticationSuccess(HttpServletRequest request,
                                       HttpServletResponse response,
                                       Authentication authentication) throws IOException, ServletException {
        // Clear any saved request or previous attributes to avoid default redirect behaviour
        request.getSession(false);

        // Always redirect to feed page
        response.sendRedirect("/posts");
    }
}

```

■ backend/src/main/java/org/AE/alliededge/config/JacksonConfig.java

```

package org.AE.alliededge.config;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class JacksonConfig {
    /**
     * A shared ObjectMapper bean for safely serializing/deserializing profile JSON sections.
     */
    @Bean
    public ObjectMapper objectMapper() {
        ObjectMapper mapper = new ObjectMapper();
        mapper.findAndRegisterModules();
        mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
        return mapper;
    }
}

```

```
}
```

■ backend/src/main/java/org/AE/alliededge/config/NoopCloudinaryServiceConfig.java

```
package org.AE.alliededge.config;

import org.AE.alliededge.service.CloudinaryService;
import org.springframework.boot.autoconfigure.condition.ConditionalOnMissingBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.multipart.MultipartFile;

import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.InputStream;
import java.util.Map;

/**
 * Fallback CloudinaryService used when Cloudinary isn't configured.
 *
 * Why this exists:
 * - In production, missing CLOUDINARY_* env vars previously caused the app to fail at startup,
 *   which made Render mark the service unhealthy and Cloudflare return 521.
 * - Most endpoints (reading posts, auth status, etc.) don't require Cloudinary.
 *
 * With this fallback, the app can start and serve non-upload features. Any endpoint that
 * actually needs Cloudinary will fail fast with a clear error message.
 */
@Configuration
public class NoopCloudinaryServiceConfig {

    private static final String MESSAGE =
        "Cloudinary is not configured on this server. Set CLOUDINARY_CLOUD_NAME, " +
        "CLOUDINARY_API_KEY, and CLOUDINARY_API_SECRET (or enable Cloudinary) to use uploads.";

    @Bean
    @ConditionalOnMissingBean(CloudinaryService.class)
    public CloudinaryService cloudinaryService() {
        return new CloudinaryService() {
            private RuntimeException notConfigured() {
                return new IllegalStateException(MESSAGE);
            }

            @Override
            public String uploadFile(MultipartFile file, String folder) {
                throw notConfigured();
            }

            @Override
            public String uploadMedia(MultipartFile file) {
                throw notConfigured();
            }

            @Override
            public Map<String, String> uploadRawFile(MultipartFile file, String folder) {
                throw notConfigured();
            }

            @Override
            public String getRawUrlFromPublicId(String publicId) {
                // This helper can still return null when not configured.
                return null;
            }

            @Override
            public void streamResumeToResponse(String resumeUrl, String filename, HttpServletResponse response) throws
                throw notConfigured();
            }

            @Override

```

```

        public InputStream streamResume(String publicId) throws IOException {
            throw notConfigured();
        }

        @Override
        public void deleteMediaByUrl(String url) {
            throw notConfigured();
        }

        @Override
        public void deleteByPublicId(String publicId, String resourceType) {
            throw notConfigured();
        }

        @Override
        public String uploadVideo(MultipartFile file, String folder) throws IOException {
            throw notConfigured();
        }

        @Override
        public String extractPublicId(String url) {
            // Best-effort fallback (non-fatal); delete paths will still throw if called.
            return url;
        }

        @Override
        public void deleteVideo(String publicId) throws IOException {
            throw notConfigured();
        }
    };
}

```

■ backend/src/main/java/org/AE/alliededge/config/SecurityConfig.java

```

package org.AE.alliededge.config;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.MediaType;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.security.web.csrf.CookieCsrfTokenRepository;
import org.springframework.security.oauth2.core.OAuth2AuthenticationException;
import org.springframework.beans.factory.annotation.Value;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
// Configures Spring Security for authentication, authorization,
// OAuth2 login flow, and API access control for the application
public class SecurityConfig {

    private static final Logger log = LoggerFactory.getLogger(SecurityConfig.class);

    private final CustomOAuth2UserService customOAuth2UserService;
    private final FrontendRedirectAuthenticationSuccessHandler frontendRedirectAuthenticationSuccessHandler;
    private final CorsConfigurationSource corsConfigurationSource;

```

```

private final String frontendRedirectUrl;

public SecurityConfig(CustomOAuth2UserService customOAuth2UserService,
                    FrontendRedirectAuthenticationSuccessHandler frontendRedirectAuthenticationSuccessHandler,
                    CorsConfigurationSource corsConfigurationSource,
                    @Value("${app.frontend.redirect-url:http://localhost:5173/}") String frontendRedirectUrl) {
    this.customOAuth2UserService = customOAuth2UserService;
    this.frontendRedirectAuthenticationSuccessHandler = frontendRedirectAuthenticationSuccessHandler;
    this.corsConfigurationSource = corsConfigurationSource;
    this.frontendRedirectUrl = frontendRedirectUrl;
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        // Ensure Spring Security uses the SAME CORS config as the MVC layer.
        .cors(cors -> cors.configurationSource(corsConfigurationSource))

        // Session-based auth: Authentication is stored in HttpSession (JSESSIONID).
        .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED))

        // CSRF:
        // Enable CSRF with cookie-based tokens for the SPA. Frontend calls GET /api/csrf to obtain XSRF-TOKEN cookie.
        // Keep WS endpoints ignored.
        .csrf(csrf -> csrf
            .csrfTokenRepository(csrfTokenRepository())
            .ignoringRequestMatchers("/ws/**")
        )

        .exceptionHandling(ex -> ex
            // Keep API responses clean and not tied to OAuth provider wording.
            .authenticationEntryPoint((request, response, authException) -> {
                String uri = request.getRequestURI();
                if (uri != null && (uri.startsWith("/api/") || uri.startsWith("/ws/"))) {
                    response.setStatus(HttpStatus.UNAUTHORIZED);
                    response.setContentType(MediaType.APPLICATION_JSON_VALUE);
                    response.getWriter().write("{\"error\":\"unauthorized\"}");
                } else {
                    response.sendError(HttpStatus.UNAUTHORIZED);
                }
            })
            .accessDeniedHandler((request, response, accessDeniedException) -> {
                String uri = request.getRequestURI();
                if (uri != null && (uri.startsWith("/api/") || uri.startsWith("/ws/"))) {
                    response.setStatus(HttpStatus.FORBIDDEN);
                    response.setContentType(MediaType.APPLICATION_JSON_VALUE);
                    response.getWriter().write("{\"error\":\"forbidden\"}");
                } else {
                    response.sendError(HttpStatus.FORBIDDEN);
                }
            })
        )

        .authorizeHttpRequests(auth -> auth
            // Always allow CORS preflight.
            .requestMatchers(HttpMethod.OPTIONS, "**").permitAll()

            // WebSocket / SockJS endpoints (chat + posts). Must be public for handshake/transport.
            .requestMatchers("/ws/**").permitAll()

            // Public pages + static
            .requestMatchers(
                "/", "/home", "/register", "/login", "/oauth2/**", "/login/oauth2/**",
                "/css/**", "/js/**", "/images/**", "/favicon.ico", "/error"
            ).permitAll()

            // SPA session probes + CSRF bootstrap

```

```

        .requestMatchers("/api/auth/status").permitAll()
        .requestMatchers(HttpMethod.GET, "/api/csrf").permitAll()

        // --- API auth rules ---
        // Public reads
        .requestMatchers(HttpMethod.GET, "/api/**").permitAll()
        // Mutations require an authenticated session (role checks handled by @PreAuthorize where needed)
        .requestMatchers(HttpMethod.POST, "/api/**").authenticated()
        .requestMatchers(HttpMethod.PUT, "/api/**").authenticated()
        .requestMatchers(HttpMethod.PATCH, "/api/**").authenticated()
        .requestMatchers(HttpMethod.DELETE, "/api/**").authenticated()

        // Server-rendered MVC endpoints (keep existing behavior)
        .requestMatchers(HttpMethod.GET, "/posts", "/posts/**").permitAll()
        .requestMatchers(HttpMethod.POST, "/posts/*/like").authenticated()
        .requestMatchers(HttpMethod.POST, "/posts/*/comments").hasAnyRole("USER", "AUTHOR", "ADMIN")
        .requestMatchers("/author/**").hasAnyRole("AUTHOR", "ADMIN")
        .requestMatchers("/admin/**").hasRole("ADMIN")

        .anyRequest().permitAll()
    )

    .formLogin(form -> form
        .loginPage("/login")
        .successHandler(frontendRedirectAuthenticationSuccessHandler)
        .permitAll()
    )

    // Backend-owned login flow.
    // Handles Google OAuth2 login and redirects user to frontend after authentication
    .oauth2Login(oauth -> oauth
        .loginPage("/login")
        .userInfoEndpoint(userInfo -> userInfo.userService(customOAuth2UserService))
        .successHandler(frontendRedirectAuthenticationSuccessHandler)
        .failureHandler((request, response, exception) -> {
            String code = "login_failed";

            if (exception instanceof OAuth2AuthenticationException oae) {
                // Prefer error code when present; fall back to message.
                String errCode = (oae.getError() != null) ? oae.getError().getErrorCode() : null;
                String msg = oae.getMessage();
                String combined = ((errCode == null ? "" : errCode) + " " + (msg == null ? "" : msg)).toUpperCase();

                if (combined.contains("ACCOUNT_BANNED")) {
                    code = "banned";
                }

                log.warn("OAuth2 login failed: code={}, errCode={}, message={} ", code, errCode, msg);
            } else {
                log.warn("OAuth2 login failed: code={}, exceptionType={}, message={}", code,
                    exception == null ? null : exception.getClass().getName(),
                    exception == null ? null : exception.getMessage());
            }

            // Always redirect to SPA login (not backend /login)
            String base = (frontendRedirectUrl == null || frontendRedirectUrl.isBlank())
                ? "http://localhost:5173/"
                : frontendRedirectUrl;

            if (!base.endsWith("/")) base = base + "/";

            String redirectTo = base + "login?error=" + code;
            log.warn("OAuth2 failure redirect -> {}", redirectTo);

            response.sendRedirect(redirectTo);
        })
    )

    .logout(logout -> logout
        .logoutSuccessUrl("/")
        .permitAll()
    )

```

```

        );

        return http.build();
    }

    /**
     * CSRF cookie for SPA.
     *
     * Important: in production the frontend is on a different site (Vercel) than the backend (Render),
     * so the cookie must be SameSite=None; Secure, otherwise browsers won't persist it and the SPA
     * can't send the required X-XSRF-TOKEN header for unsafe requests.
     */
    @Bean
    public CookieCsrfTokenRepository csrfTokenRepository() {
        CookieCsrfTokenRepository repo = CookieCsrfTokenRepository.withHttpOnlyFalse();
        repo.setCookiePath("/");
        repo.setSecure(true);
        // Spring Security 6 supports SameSite via the cookie customizer.
        repo.setCookieCustomizer(cookie -> cookie.sameSite("None"));
        return repo;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/config/WebConfig.java

```

package org.AE.alliededge.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    // Local filesystem handlers for /uploads and /profile-uploads have been removed.
    // All media is now served directly from Cloudinary URLs, so no ResourceHandler is needed here.

}

```

■ backend/src/main/java/org/AE/alliededge/config/WebSocketConfig.java

```

package org.AE.alliededge.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        // Enable a simple in-memory message broker to send messages to clients
        registry.enableSimpleBroker("/topic");
        // Set application destination prefix for messages from clients
        registry.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        // Register /ws/chat endpoint with SockJS fallback
        registry.addEndpoint("/ws/chat")
            .setAllowedOriginPatterns(
                "https://alliededge.app",
                "http://localhost:5173",
            );
    }
}

```

```

        "http://127.0.0.1:5173"
    )
    .withSockJS();

    // Realtime post stats (likes/comments)
    registry.addEndpoint("/ws/posts")
        .setAllowedOriginPatterns(
            "https://alliededge.app",
            "http://localhost:5173",
            "http://127.0.0.1:5173"
        )
        .withSockJS();
    }
}

```

■ backend/src/main/java/org/AE/alliededge/controller/AdminController.java

```

package org.AE.alliededge.controller;

import jakarta.validation.Valid;
import org.AE.alliededge.exception.ResourceNotFoundException;
import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.AE.alliededge.service.PostService;
import org.AE.alliededge.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

import java.security.Principal;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/admin")
@PreAuthorize("hasRole('ADMIN')")
public class AdminController {
    private final PostService postService;
    private final UserService userService;
    private final org.AE.alliededge.repository.CommentRepository commentRepository;
    private final org.AE.alliededge.repository.PostLikeRepository postLikeRepository;

    @Autowired
    private UserRepository userRepository;

    public AdminController(PostService postService, UserService userService,
                           org.AE.alliededge.repository.CommentRepository commentRepository,
                           org.AE.alliededge.repository.PostLikeRepository postLikeRepository) {
        this.postService = postService;
        this.userService = userService;
        this.commentRepository = commentRepository;
        this.postLikeRepository = postLikeRepository;
    }

    @GetMapping
    public Map<String, Object> adminDashboard() {
        return Map.of("message", "admin" );
    }

    @GetMapping("/posts")
    public ResponseEntity<Map<String, Object>> showPostListDashboard(@RequestParam(defaultValue = "0") int
                                                                    @RequestParam(defaultValue = "10") int size,

```

```

        @RequestParam(value = "keyword", required = false)
        @RequestParam(value = "sortBy", required = false)

Pageable pageable = PageRequest.of(page, size);
Page<Post> postPage = postService.getAllPosts(keyword, sortBy, pageable);

Map<String, Object> body = new LinkedHashMap<>();
body.put("postPage", postPage);
body.put("keyword", keyword);
body.put("sortBy", (sortBy == null || sortBy.isBlank()) ? "newest" : sortBy);
return ResponseEntity.ok(body);
}

@PostMapping("/posts")
public ResponseEntity<?> savePost(@Valid @ModelAttribute("post") Post post,
                                BindingResult bindingResult,
                                Principal principal) {
    if (bindingResult.hasErrors()) {
        return ResponseEntity.badRequest().body(Map.of("message", "Validation failed", "errors", bindingResult.getA
    )
    if (principal == null) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
    }

    Post saved = postService.savePost(post, principal.getName());
    return ResponseEntity.status(HttpStatus.CREATED).body(saved);
}

@GetMapping("/posts/{id}")
public Post getPost(@PathVariable Long id) {
    return postService.findPostById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Post not found with Id:" + id));
}

@DeleteMapping("/posts/{id}")
public Map<String, Object> deletePost(@PathVariable Long id) {
    postService.deletePostById(id);
    return Map.of("message", "Deleted" );
}

// ===== USER MANAGEMENT ENDPOINTS =====

@GetMapping("/users")
public ResponseEntity<Map<String, Object>> listUsers(@RequestParam(required = false) String keyword,
                                                     @RequestParam(defaultValue = "0") int page,
                                                     @RequestParam(defaultValue = "10") int size) {

    if (page <= 0) page = 0;
    if (size <= 0) size = 10;
    if (size > 100) size = 100;

    Pageable pageable = PageRequest.of(page, size);
    Page<User> usersPage;

    String effectiveKeyword = (keyword == null) ? "" : keyword.trim();
    if (!effectiveKeyword.isEmpty()) {
        usersPage = userRepository.searchUsersPaged(effectiveKeyword, pageable);
    } else {
        usersPage = userRepository.findAllUsersPaged(pageable);
    }

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("users", usersPage.getContent());
    body.put("keyword", effectiveKeyword);
    body.put("totalUsers", usersPage.getTotalElements());
    body.put("page", usersPage.getNumber());
    body.put("size", usersPage.getSize());
    body.put("totalPages", usersPage.getTotalPages());
    body.put("first", usersPage.isFirst());
    body.put("last", usersPage.isLast());
    return ResponseEntity.ok(body);
}

@GetMapping("/users/{id}/posts")

```



```

public ResponseEntity<Map<String, Object>> viewUserPosts(@PathVariable Long id) {
    User user = userService.getUserById(id);
    if (user == null) {
        throw new ResourceNotFoundException("User not found with ID: " + id);
    }

    List<Post> userPosts = postService.findPostsByUserId(id);

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("user", user);
    body.put("posts", userPosts);
    body.put("totalPosts", userPosts.size());
    return ResponseEntity.ok(body);
}

@DeleteMapping("/posts/{postId}/user-post")
public Map<String, Object> deleteUserPost(@PathVariable Long postId,
                                           @RequestParam(required = false) Long userId) {
    postService.deletePostById(postId);
    return Map.of("message", "Post deleted successfully", "userId", userId);
}

@GetMapping("/post-stats")
public Map<String, Object> getPostStats() {
    long totalUsers = userRepository.count();
    long activeUsers = userRepository.countByRole("ROLE_AUTHOR");
    long totalComments = commentRepository.countAllComments();
    long totalLikes = postLikeRepository.countAllLikes();

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("totalUsers", totalUsers);
    body.put("activeUsers", activeUsers);
    body.put("totalComments", totalComments);
    body.put("totalLikes", totalLikes);
    return body;
}

@PostMapping("/users/{id}/ban")
public ResponseEntity<Map<String, Object>> banUser(@PathVariable Long id) {
    User target = userService.getUserById(id);
    if (target == null) {
        throw new ResourceNotFoundException("User not found with ID: " + id);
    }
    if (target.isAdmin() || "ROLE_ADMIN".equalsIgnoreCase(target.getRole())) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(Map.of("message", "Cannot ban an admin user"));
    }
    target.setBanned(true);
    userRepository.save(target);
    return ResponseEntity.ok(Map.of("message", "User banned", "id", target.getId(), "banned", true));
}

@PostMapping("/users/{id}/unban")
public ResponseEntity<Map<String, Object>> unbanUser(@PathVariable Long id) {
    User target = userService.getUserById(id);
    if (target == null) {
        throw new ResourceNotFoundException("User not found with ID: " + id);
    }
    target.setBanned(false);
    userRepository.save(target);
    return ResponseEntity.ok(Map.of("message", "User unbanned", "id", target.getId(), "banned", false));
}
}

```

■ backend/src/main/java/org/AE/alliededge/controller/AnnouncementController.java

```
package org.AE.alliededge.controller;
```

```
import org.AE.alliededge.model.Announcement;
```

```

import org.AE.alliededge.service.AnnouncementService;
import org.AE.alliededge.service.CloudinaryService;
import org.AE.alliededge.service.AnnouncementUpdateRequest;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api")
public class AnnouncementController {
    @Autowired
    private AnnouncementService announcementService;

    @Autowired
    private CloudinaryService cloudinaryService;

    // Public: List all visible announcements
    @GetMapping("/announcements")
    public List<Announcement> announcements() {
        return announcementService.findAllVisible();
    }

    // Admin: List all announcements
    @PreAuthorize("hasRole('ADMIN')")
    @GetMapping("/admin/announcements")
    public List<Announcement> adminAnnouncements() {
        return announcementService.findAll();
    }

    // Admin: Create new announcement (supports optional videoFile)
    @PreAuthorize("hasRole('ADMIN')")
    @PostMapping("/admin/announcements")
    public ResponseEntity<Announcement> createAnnouncement(
        @ModelAttribute Announcement announcement,
        @RequestParam(value = "imageFiles", required = false) List<MultipartFile> imageFiles,
        @RequestParam(value = "imageFiles[]", required = false) List<MultipartFile> imageFilesAlt,
        @RequestParam(value = "videoFile", required = false) MultipartFile videoFile) {
        try {
            if ((imageFiles == null || imageFiles.isEmpty()) && imageFilesAlt != null && !imageFilesAlt.isEmpty()) {
                imageFiles = imageFilesAlt;
            }

            if (announcement.getImageUrls() == null) {
                announcement.setImageUrls(new java.util.ArrayList<>());
            }

            if (imageFiles != null && !imageFiles.isEmpty()) {
                for (MultipartFile imageFile : imageFiles) {
                    if (imageFile == null || imageFile.isEmpty()) continue;
                    String contentType = imageFile.getContentType();
                    if (contentType == null || !contentType.toLowerCase().startsWith("image/")) {
                        throw new RuntimeException("Only image files are allowed");
                    }
                    String imageUrl = cloudinaryService.uploadMedia(imageFile);
                    announcement.getImageUrls().add(imageUrl);
                }
            }

            if (videoFile != null && !videoFile.isEmpty()) {
                String videoUrl = cloudinaryService.uploadVideo(videoFile, "announcements/videos");
                String publicId = cloudinaryService.extractPublicId(videoUrl);
                announcement.setVideoUrl(videoUrl);
                announcement.setVideoPublicId(publicId);
            }

            Announcement created = announcementService.create(announcement);

```

```

        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    } catch (Exception e) {
        throw new RuntimeException("Failed to create announcement: " + e.getMessage(), e);
    }
}

// Admin: Delete announcement
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/admin/announcements/{id}")
public Map<String, Object> deleteAnnouncement(@PathVariable Long id) {
    announcementService.delete(id);
    return Map.of("message", "Deleted");
}

// Admin: Toggle visibility
@PreAuthorize("hasRole('ADMIN')")
@PostMapping("/admin/announcements/{id}/toggle")
public Map<String, Object> toggleVisibility(@PathVariable Long id) {
    announcementService.toggleVisibility(id);
    return Map.of("message", "Toggled" );
}

// Admin: Toggle welcome announcement status
@PreAuthorize("hasRole('ADMIN')")
@PostMapping("/admin/announcements/{id}/toggle-welcome")
public Map<String, Object> toggleWelcomeAnnouncement(@PathVariable Long id) {
    announcementService.toggleWelcomeAnnouncement(id);
    return Map.of("message", "Toggled" );
}

// Admin: Get announcement by id
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/admin/announcements/{id}")
public Announcement getAnnouncement(@PathVariable Long id) {
    return announcementService.findById(id)
        .orElseThrow(() -> new RuntimeException("Announcement not found"));
}

// Admin: Update announcement (with optional video changes)
@PreAuthorize("hasRole('ADMIN')")
@PutMapping("/admin/announcements/{id}")
public Announcement editAnnouncement(
    @PathVariable Long id,
    @RequestParam(value = "title", required = false) String title,
    @RequestParam(value = "message", required = false) String message,
    @RequestParam(value = "visible", required = false) Boolean visible,
    @RequestParam(value = "isWelcomeAnnouncement", required = false) Boolean isWelcomeAnnouncement,
    @RequestParam(value = "welcomeAnnouncement", required = false) Boolean welcomeAnnouncement,
    @RequestParam(value = "imageFiles", required = false) List<MultipartFile> imageFiles,
    @RequestParam(value = "imageFilesAlt", required = false) List<MultipartFile> imageFilesAlt,
    @RequestParam(value = "removeImages", required = false) Boolean removeImages,
    @RequestParam(value = "videoFile", required = false) MultipartFile videoFile,
    @RequestParam(value = "removeVideo", required = false) Boolean removeVideo) {

    if ((imageFiles == null || imageFiles.isEmpty()) && imageFilesAlt != null && !imageFilesAlt.isEmpty())
        imageFiles = imageFilesAlt;
}

AnnouncementUpdateRequest req = new AnnouncementUpdateRequest();
req.setTitle(title);
req.setMessage(message);
req.setVisible(visible);
// accept both param names for compatibility with older forms
req.setIsWelcomeAnnouncement(isWelcomeAnnouncement != null ? isWelcomeAnnouncement : welcomeAnnouncement);
req.setImageFiles(imageFiles);
req.setRemoveImages(removeImages);
req.setVideoFile(videoFile);
req.setRemoveVideo(removeVideo);

return announcementService.update(id, req);
}

```

```

// Admin: Create announcement via create form equivalent (kept for compatibility)
@PreAuthorize("hasRole('ADMIN')")
@PostMapping("/announcements/create")
public ResponseEntity<Announcement> createAnnouncementForm(@ModelAttribute Announcement announcement,
                                                           @RequestParam(value = "imageFiles", required = false) List<MultipartFile> imageFiles,
                                                           @RequestParam(value = "imageFilesAlt", required = false) List<String> imageFilesAlt,
                                                           @RequestParam(value = "videoFile", required = false) MultipartFile videoFile) {
    try {
        if ((imageFiles == null || imageFiles.isEmpty()) && imageFilesAlt != null && !imageFilesAlt.isEmpty()) {
            imageFiles = imageFilesAlt;
        }

        if (announcement.getImageUrls() == null) {
            announcement.setImageUrls(new java.util.ArrayList<>());
        }

        if (imageFiles != null && !imageFiles.isEmpty()) {
            for (MultipartFile imageFile : imageFiles) {
                if (imageFile == null || imageFile.isEmpty()) continue;
                String contentType = imageFile.getContentType();
                if (contentType == null || !contentType.toLowerCase().startsWith("image/")) {
                    throw new RuntimeException("Only image files are allowed");
                }
                String imageUrl = cloudinaryService.uploadMedia(imageFile);
                announcement.getImageUrls().add(imageUrl);
            }
        }

        if (videoFile != null && !videoFile.isEmpty()) {
            String videoUrl = cloudinaryService.uploadVideo(videoFile, "announcements/videos");
            String publicId = cloudinaryService.extractPublicId(videoUrl);
            announcement.setVideoUrl(videoUrl);
            announcement.setVideoPublicId(publicId);
        }

        announcement.setCreatedAt(java.time.LocalDateTime.now());
        announcement.setVisible(true);
        Announcement created = announcementService.create(announcement);

        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    } catch (Exception e) {
        throw new RuntimeException("Failed to create announcement: " + e.getMessage(), e);
    }
}

// Admin: Delete video from announcement
@PreAuthorize("hasRole('ADMIN')")
@PostMapping("/admin/announcements/{id}/delete-video")
public Announcement deleteAnnouncementVideo(@PathVariable Long id) {
    try {
        Announcement announcement = announcementService.findById(id)
            .orElseThrow(() -> new RuntimeException("Announcement not found"));

        if (announcement.getVideoPublicId() != null) {
            cloudinaryService.deleteVideo(announcement.getVideoPublicId());
            announcement.setVideoUrl(null);
            announcement.setVideoPublicId(null);

            AnnouncementUpdateRequest req = new AnnouncementUpdateRequest();
            req.setRemoveVideo(true);
            return announcementService.update(id, req);
        }

        return announcement;
    } catch (Exception e) {
        throw new RuntimeException("Failed to delete video: " + e.getMessage(), e);
    }
}
}

```

■ backend/src/main/java/org/AE/alliededge/controller/AppErrorController.java

```
package org.AE.alliededge.controller;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.boot.web.servlet.error.ErrorController;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.LinkedHashMap;
import java.util.Map;

@RestController
@RequestMapping("/api")
public class AppErrorController implements ErrorController {

    @RequestMapping("/error")
    public ResponseEntity<Map<String, Object>> handleError(HttpServletRequest request) {
        Object statusObj = request.getAttribute(RequestDispatcher.ERROR_STATUS_CODE);
        Object messageObj = request.getAttribute(RequestDispatcher.ERROR_MESSAGE);
        int statusCode = 500;
        if (statusObj instanceof Integer) {
            statusCode = (Integer) statusObj;
        } else if (statusObj != null) {
            try {
                statusCode = Integer.parseInt(statusObj.toString());
            } catch (NumberFormatException ignored) {
            }
        }
        String path = (String) request.getAttribute(RequestDispatcher.ERROR_REQUEST_URI);

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("status", statusCode);
        body.put("path", path);
        body.put("errorMessage", messageObj != null ? messageObj.toString() : "Unexpected error");

        HttpStatus status;
        try {
            status = HttpStatus.valueOf(statusCode);
        } catch (Exception ex) {
            status = HttpStatus.INTERNAL_SERVER_ERROR;
        }

        return ResponseEntity.status(status).body(body);
    }
}
```

■ backend/src/main/java/org/AE/alliededge/controller/AuthController.java

```
package org.AE.alliededge.controller;

import org.AE.alliededge.model.User;
import org.AE.alliededge.service.UserService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.authentication.AnonymousAuthenticationToken;
import org.springframework.security.oauth2.core.user.OAuth2User;

import java.util.*;

import org.AE.alliededge.repository.UserRepository;

@RestController
```

```

@RequestMapping("/api")
public class AuthController {

    private final UserService userService;
    private final UserRepository userRepository;

    public AuthController(UserService userService, UserRepository userRepository) {
        this.userService = userService;
        this.userRepository = userRepository;
    }

    @GetMapping("/auth/status")
    public Map<String, Object> status() {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();

        boolean authenticated = auth != null
            && auth.isAuthenticated()
            && !(auth instanceof AnonymousAuthenticationToken);

        Map<String, Object> body = new HashMap<>();
        body.put("authenticated", authenticated);

        if (!authenticated) {
            return body;
        }

        String email = null;
        try {
            email = auth.getName();
        } catch (Exception ignored) {
            // Defensive: some principals can throw on getName
        }

        // In some authentication modes, getName() might not be the email.
        // Prefer a stable email from OAuth2 attributes when available.
        if (email == null || !email.contains("@")) {
            Object principal = auth.getPrincipal();
            if (principal instanceof OAuth2User oauth2User) {
                Object attrEmail = oauth2User.getAttributes().get("email");
                if (attrEmail != null) {
                    email = String.valueOf(attrEmail);
                }
            }
        }

        body.put("name", email);

        final String emailFinal = email;

        // Always emit a user payload when authenticated, even if DB lookup fails.
        // This prevents the SPA from treating the session as logged out.
        Map<String, Object> userPayload = new HashMap<>();
        userPayload.put("role", auth.getAuthorities().stream().findFirst().map(org.springframework.security.core.GrantedAuthority::getAuthority).orElse(""));

        if (emailFinal != null) {
            try {
                userRepository.findByEmail(emailFinal).ifPresentOrElse(u -> {
                    if (u.isBanned()) {
                        body.put("authenticated", false);
                        body.put("message", "Your account has been banned. Please contact support.");
                        userPayload.clear();
                        return;
                    }
                    userPayload.put("id", u.getId());
                    String display = u.getDisplayName();
                    String username = u.getUsername();
                    userPayload.put("name", (display != null && !display.isBlank()) ? display : username);
                    userPayload.put("email", u.getEmail());
                    userPayload.put("role", u.getRole());
                    userPayload.put("profileImageUrl", u.getProfileImageUrl());
                }, () -> {

```

```

        // Fallback if user isn't found in DB (shouldn't happen, but avoid 500)
        userPayload.put("email", emailFinal);
    });
} catch (Exception ex) {
    // Avoid failing the endpoint; include minimal info.
    userPayload.put("email", emailFinal);
}
}

body.put("user", userPayload);
return body;
}

@PostMapping("/auth/logout")
public Map<String, Object> logout(jakarta.servlet.http.HttpServletRequest request) {
    var session = request.getSession(false);
    if (session != null) {
        session.invalidate();
    }
    return Map.of("message", "ok");
}

@PostMapping("/auth/register")
public ResponseEntity<Map<String, Object>> registerUser(@ModelAttribute User user) {
    try {
        userService.registerUser(user);
        return ResponseEntity.status(HttpStatus.CREATED).body(Map.of("message", "Registered"));
    } catch (IllegalArgumentException ex) {
        return ResponseEntity.badRequest().body(Map.of("message", ex.getMessage()));
    }
}
}
}

```

■ backend/src/main/java/org/AE/alliededge/controller/AuthorPostController.java

```

package org.AE.alliededge.controller;

import jakarta.validation.Valid;
import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.PostRepository;
import org.AE.alliededge.service.PostService;
import org.AE.alliededge.service.UserService;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.util.Map;

@RestController
@RequestMapping("/api")
@PreAuthorize("hasAnyRole('AUTHOR', 'ADMIN')")
public class AuthorPostController {

    private final PostRepository postRepository;
    private final UserService userService;
    private final PostService postService;

    public AuthorPostController(PostRepository postRepository, UserService userService, PostService postService) {
        this.postRepository = postRepository;
        this.userService = userService;
        this.postService = postService;
    }

    @GetMapping("/author/posts")

```

```

public Page<Post> listAuthorPosts(@RequestParam(defaultValue = "0") int page,
                                   @RequestParam(defaultValue = "10") int size,
                                   @RequestParam(value = "keyword", required = false) String keyword,
                                   @RequestParam(value = "sortBy", required = false) String sortBy) {
    User current = userService.getCurrentUserOrThrow();
    Pageable pageable = PageRequest.of(page, size);
    return postService.getAuthorPosts(current.getId(), keyword, sortBy, pageable);
}

@PostMapping("/author/posts")
public Map<String, Object> createPost(@Valid @ModelAttribute("post") Post post,
                                       BindingResult bindingResult,
                                       @RequestParam(name = "images", required = false) java.util.List<MultipartFile> images,
                                       @RequestParam(name = "video", required = false) MultipartFile video) {
    User current = userService.getCurrentUserOrThrow();
    if (!current.isAuthor() && !current.isAdmin()) {
        throw new AccessDeniedException("Only authors can create posts");
    }

    if (bindingResult.hasErrors()) {
        return Map.of("message", "Validation failed", "errors", bindingResult.getAllErrors());
    }

    post.setId(null);
    post.setUser(current);
    Post saved = postService.savePost(post, current.getEmail());
    return Map.of("message", "Created", "post", saved);
}

@GetMapping("/author/posts/{id}")
public Post getPost(@PathVariable Long id) {
    User current = userService.getCurrentUserOrThrow();
    Post post = postRepository.findById(id).orElseThrow(() -> new IllegalArgumentException("Post not found"));
    enforceOwnership(current, post);
    return post;
}

@PutMapping("/author/posts/{id}")
public Map<String, Object> updatePost(@PathVariable Long id,
                                       @Valid @ModelAttribute("post") Post updated,
                                       BindingResult bindingResult,
                                       @RequestParam(name = "images", required = false) java.util.List<MultipartFile> images,
                                       @RequestParam(name = "video", required = false) MultipartFile video) {
    User current = userService.getCurrentUserOrThrow();
    Post existing = postRepository.findById(id).orElseThrow(() -> new IllegalArgumentException("Post not found"));
    enforceOwnership(current, existing);

    if (bindingResult.hasErrors()) {
        return Map.of("message", "Validation failed", "errors", bindingResult.getAllErrors());
    }

    existing.setTitle(updated.getTitle());
    existing.setContent(updated.getContent());

    Post saved = postRepository.save(existing);
    return Map.of("message", "Updated", "post", saved);
}

@DeleteMapping("/author/posts/{id}")
public Map<String, Object> deletePost(@PathVariable Long id) {
    User current = userService.getCurrentUserOrThrow();
    Post post = postRepository.findById(id).orElseThrow(() -> new IllegalArgumentException("Post not found"));
    enforceOwnership(current, post);
    postRepository.delete(post);
    return Map.of("message", "Deleted");
}

private void enforceOwnership(User current, Post post) {
    if (current.isAdmin()) {
        return;
    }
}

```



```

        if (!post.getUser().getId().equals(current.getId())) {
            throw new AccessDeniedException("You do not own this post");
        }
    }
}

```

■ backend/src/main/java/org/AE/alliededge/controller/ChatController.java

```

package org.AE.alliededge.controller;

import org.AE.alliededge.dto.ChatMessageDTO;
import org.AE.alliededge.dto.ChatRoomDTO;
import org.AE.alliededge.dto.SendChatMessageRequest;
import org.AE.alliededge.model.ChatMessage;
import org.AE.alliededge.model.ChatRoom;
import org.AE.alliededge.model.User;
import org.AE.alliededge.service.ChatService;
import org.AE.alliededge.service.UserFollowService;
import org.AE.alliededge.service.UserService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.messaging.handler.annotation.Payload;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import org.springframework.format.annotation.DateTimeFormat;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api")
public class ChatController {

    private final ChatService chatService;
    private final UserService userService;
    private final UserFollowService userFollowService;
    private final SimpMessagingTemplate messagingTemplate;

    public ChatController(ChatService chatService,
                          UserService userService,
                          UserFollowService userFollowService,
                          SimpMessagingTemplate messagingTemplate) {
        this.chatService = chatService;
        this.userService = userService;
        this.userFollowService = userFollowService;
        this.messagingTemplate = messagingTemplate;
    }

    /**
     * Chat dashboard - list all conversations for current user
     */
    @GetMapping("/chats")
    public ResponseEntity<?> chatDashboard(Authentication authentication) {
        if (authentication == null || !authentication.isAuthenticated()) {
            return ResponseEntity.status(401).body(Map.of("message", "Not authenticated"));
        }

        String principal = authentication.getName();
        User currentUser = userService.getUserByPrincipal(principal);

        if (currentUser == null) {
            return ResponseEntity.status(401).body(Map.of("message", "Not authenticated"));
        }
    }
}

```

```

List<ChatRoom> chatRooms = chatService.getChatRoomsForUser(currentUser);

List<ChatRoomDTO> chatRoomDTOs = new ArrayList<>();
for (ChatRoom room : chatRooms) {
    User otherUser = chatService.getOtherUser(room, currentUser);
    ChatMessage lastMessage = chatService.getLatestMessage(room);

    String lastMessageText = lastMessage != null ? lastMessage.getContent() : "No messages yet";

    ChatRoomDTO dto = new ChatRoomDTO(
        room.getId(),
        otherUser.getId(),
        otherUser.getUsername(),
        otherUser.getDisplayName() != null ? otherUser.getDisplayName() : otherUser.getUsername(),
        otherUser.getProfileImageUrl() != null ? otherUser.getProfileImageUrl()
            : "/images/default-profile.jpg",
        lastMessageText,
        lastMessage != null ? lastMessage.getTimestamp() : null);
    chatRoomDTOs.add(dto);
}

return ResponseEntity.ok(Map.of(
    "chatRooms", chatRoomDTOs,
    "currentUser", currentUser
));
}

/**
 * Open/get chat room + messages with a target user
 */
@GetMapping("/chats/with/{userId}")
public ResponseEntity<?> openChat(@PathVariable Long userId, Authentication authentication) {
    if (authentication == null || !authentication.isAuthenticated()) {
        return ResponseEntity.status(401).body(Map.of("message", "Not authenticated"));
    }

    String principal = authentication.getName();
    User currentUser = userService.getUserByPrincipal(principal);
    User targetUser = userService.getUserById(userId);

    if (currentUser == null || targetUser == null) {
        return ResponseEntity.status(404).body(Map.of("message", "User not found"));
    }

    boolean followsTarget = userFollowService.isFollowing(principal, userId);
    boolean mutual = userFollowService.isMutualFollowing(principal, userId);

    // LinkedIn-style: allow opening the thread UI, but lock sending unless mutual follow.
    if (!mutual) {
        String reason;
        if (!followsTarget) {
            reason = "FOLLOW_REQUIRED";
        } else {
            reason = "FOLLOW_BACK_REQUIRED";
        }

        return ResponseEntity.status(403).body(Map.of(
            "message", "Messaging is locked until you both follow each other",
            "canMessage", false,
            "reason", reason,
            "targetUser", targetUser,
            "currentUser", currentUser
        ));
    }

    ChatRoom chatRoom = chatService.getOrCreateChatRoom(currentUser, targetUser);
    List<ChatMessage> messages = chatService.getMessages(chatRoom.getId());

    List<ChatMessageDTO> messageDTOs = messages.stream()
        .map(msg -> new ChatMessageDTO(
            chatRoom.getId(),

```

```

        msg.getSender().getUsername(),
        msg.getSender().getDisplayName() != null ? msg.getSender().getDisplayName()
            : msg.getSender().getUsername(),
        msg.getContent(),
        msg.getTimestamp())
        .collect(Collectors.toList());

    return ResponseEntity.ok(Map.of(
        "canMessage", true,
        "chatRoom", chatRoom,
        "targetUser", targetUser,
        "currentUser", currentUser,
        "messages", messageDTOs
    ));
}

/**
 * Persisted send endpoint (REST).
 * This guarantees delivery (message saved to DB) even if WebSocket is blocked.
 */
@PostMapping("/chats/{roomId}/messages")
public ResponseEntity<?> sendMessageRest(@PathVariable Long roomId,
                                         @RequestBody SendChatMessageRequest request,
                                         Authentication authentication) {
    if (authentication == null || !authentication.isAuthenticated()) {
        return ResponseEntity.status(401).body(Map.of("message", "Not authenticated"));
    }

    String principal = authentication.getName();
    User sender = userService.getUserByPrincipal(principal);
    if (sender == null) {
        return ResponseEntity.status(401).body(Map.of("message", "Not authenticated"));
    }

    ChatRoom room = chatService.getChatRoom(roomId);
    if (room == null || !room.isParticipant(sender)) {
        return ResponseEntity.status(404).body(Map.of("message", "Chat room not found"));
    }

    User otherUser = chatService.getOtherUser(room, sender);
    if (otherUser == null) {
        return ResponseEntity.status(404).body(Map.of("message", "User not found"));
    }

    boolean mutual = userFollowService.isMutualFollowing(principal, otherUser.getId());
    if (!mutual) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(Map.of(
            "message", "Messaging is locked until you both follow each other",
            "canMessage", false,
            "reason", userFollowService.isFollowing(principal, otherUser.getId()) ? "FOLLOW_BACK_REQUIRED" : "FOLLOW_REQUIRED"
        ));
    }

    String content = request != null ? request.getContent() : null;
    if (content == null || content.trim().isEmpty()) {
        return ResponseEntity.badRequest().body(Map.of("message", "Message content is required"));
    }

    ChatMessage savedMessage = chatService.sendMessage(roomId, sender, content.trim());

    ChatMessageDTO response = new ChatMessageDTO(
        savedMessage.getChatRoom().getId(),
        sender.getUsername(),
        sender.getDisplayName() != null ? sender.getDisplayName() : sender.getUsername(),
        savedMessage.getContent(),
        savedMessage.getTimestamp());

    // Broadcast for realtime listeners.
    messagingTemplate.convertAndSend("/topic/chat/" + roomId, response);

    return ResponseEntity.ok(response);
}

```

```

}

/**
 * Handle incoming WebSocket chat messages
 */
@MessageMapping("/chat.send")
public void sendMessage(@Payload ChatMessageDTO messageDTO, Authentication authentication) {
    if (authentication == null || !authentication.isAuthenticated()) {
        return;
    }

    String principal = authentication.getName();
    User sender = userService.getUserByPrincipal(principal);

    if (sender == null) {
        return;
    }

    ChatRoom room = chatService.getChatRoom(messageDTO.getRoomId());
    if (room == null || !room.isParticipant(sender)) {
        return;
    }

    User otherUser = chatService.getOtherUser(room, sender);
    if (otherUser == null) {
        return;
    }

    // Server-side enforcement for WebSocket path (can't be bypassed by a custom client)
    boolean mutual = userFollowService.isMutualFollowing(principal, otherUser.getId());
    if (!mutual) {
        return;
    }

    ChatMessage savedMessage = chatService.sendMessage(
        messageDTO.getRoomId(),
        sender,
        messageDTO.getContent());

    ChatMessageDTO response = new ChatMessageDTO(
        savedMessage.getChatRoom().getId(),
        sender.getUsername(),
        sender.getDisplayName() != null ? sender.getDisplayName() : sender.getUsername(),
        savedMessage.getContent(),
        savedMessage.getTimestamp());

    messagingTemplate.convertAndSend("/topic/chat/" + messageDTO.getRoomId(), response);
}

/**
 * Paginated messages (newest-first). Use for infinite scroll.
 *
 * Query:
 * - size: page size (default 30)
 * - before: cursor timestamp (optional). If provided, returns messages strictly older than this timestamp.
 */
@GetMapping("/chats/{roomId}/messages")
public ResponseEntity<?> listMessagesPaged(@PathVariable Long roomId,
                                           @RequestParam(name = "size", required = false, defaultValue = "30") int size,
                                           @RequestParam(name = "before", required = false)
                                           @DateTimeFormat(iso = DateTimeFormat.ISO.DATE_TIME) LocalDateTime before,
                                           Authentication authentication) {
    if (authentication == null || !authentication.isAuthenticated()) {
        return ResponseEntity.status(401).body(Map.of("message", "Not authenticated"));
    }

    String principal = authentication.getName();
    User currentUser = userService.getUserByPrincipal(principal);
    if (currentUser == null) {
        return ResponseEntity.status(401).body(Map.of("message", "Not authenticated"));
    }
}

```

```

ChatRoom room = chatService.getChatRoom(roomId);
if (room == null || !room.isParticipant(currentUser)) {
    return ResponseEntity.status(404).body(Map.of("message", "Chat room not found"));
}

User otherUser = chatService.getOtherUser(room, currentUser);
if (otherUser == null) {
    return ResponseEntity.status(404).body(Map.of("message", "User not found"));
}

// Enforce mutual follow to read messages (same gate as sending/opening).
boolean mutual = userFollowService.isMutualFollowing(principal, otherUser.getId());
if (!mutual) {
    String reason = userFollowService.isFollowing(principal, otherUser.getId()) ? "FOLLOW_BACK_REQUIRED" : "FOLLOW_REQUIRED";
    return ResponseEntity.status(HttpStatus.FORBIDDEN).body(Map.of(
        "message", "Messaging is locked until you both follow each other",
        "canMessage", false,
        "reason", reason
    ));
}

// We intentionally fetch one extra record (size+1) so we can compute hasMore reliably.
int safeSize = Math.max(1, Math.min(size, 200));
int fetchSize = Math.min(201, safeSize + 1);

List<ChatMessage> fetched;
if (before != null) {
    fetched = chatService.getOlderMessages(roomId, before, fetchSize);
} else {
    fetched = chatService.getRecentMessages(roomId, fetchSize);
}

boolean hasMore = fetched.size() > safeSize;
List<ChatMessage> page = hasMore ? fetched.subList(0, safeSize) : fetched;

// Returned newest-first; frontend will reverse for display.
List<ChatMessageDTO> messageDTOs = page.stream()
    .map(msg -> new ChatMessageDTO(
        room.getId(),
        msg.getSender().getUsername(),
        msg.getSender().getDisplayName() != null ? msg.getSender().getDisplayName() : msg.getSender().getUsername(),
        msg.getContent(),
        msg.getTimestamp()))
    .collect(Collectors.toList());

return ResponseEntity.ok(Map.of(
    "messages", messageDTOs,
    "hasMore", hasMore
));
}
}

```

■ backend/src/main/java/org/AE/alliededge/controller/CommentController.java

```

package org.AE.alliededge.controller;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import org.AE.alliededge.model.Comment;
import org.AE.alliededge.service.CommentLikeService;
import org.AE.alliededge.service.CommentService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.security.Principal;
import java.util.Map;

@RestController
@RequestMapping("/api")

```

```

public class CommentController {

    private final CommentService commentService;
    private final CommentLikeService commentLikeService;

    public CommentController(CommentService commentService, CommentLikeService commentLikeService) {
        this.commentService = commentService;
        this.commentLikeService = commentLikeService;
    }

    public static class UpdateCommentRequest {
        @NotBlank
        public String content;

        public String getContent() {
            return content;
        }

        public void setContent(String content) {
            this.content = content;
        }
    }

    @PutMapping("/comments/{id}")
    public ResponseEntity<Map<String, Object>> updateComment(@PathVariable Long id,
                                                             @Valid @RequestBody UpdateCommentRequest body,
                                                             Principal principal) {
        if (principal == null) {
            return ResponseEntity.status(401).body(Map.of("message", "Not authenticated"));
        }

        Comment updated = commentService.updateComment(id, body.getContent(), principal.getName());
        Long postId = updated.getPost() != null ? updated.getPost().getId() : null;

        Map<String, Object> resp = new java.util.LinkedHashMap<>();
        resp.put("comment", updated);
        resp.put("postId", postId);
        return ResponseEntity.ok(resp);
    }

    @PostMapping("/comments/{id}/like")
    public ResponseEntity<Map<String, Object>> likeComment(@PathVariable Long id, Principal principal) {
        if (principal == null) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
        }

        boolean nowLiked = commentLikeService.toggleLike(id, principal.getName());

        // Fetch updated likes count via service-side denormalized field
        Comment updated = commentService.getById(id);

        Map<String, Object> body = new java.util.LinkedHashMap<>();
        body.put("id", updated.getId());
        body.put("likes", updated.getLikes());
        body.put("liked", nowLiked);
        return ResponseEntity.ok(body);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/controller/CommentReplyController.java

```

package org.AE.alliededge.controller;

import jakarta.validation.Valid;
import org.AE.alliededge.dto.CommentDto;
import org.AE.alliededge.model.CommentReply;
import org.AE.alliededge.service.CommentReplyLikeService;
import org.AE.alliededge.service.CommentReplyService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

```

```

import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import java.security.Principal;
import java.util.Map;

@RestController
@RequestMapping("/api")
public class CommentReplyController {

    private final CommentReplyService replyService;
    private final CommentReplyLikeService replyLikeService;

    public CommentReplyController(CommentReplyService replyService, CommentReplyLikeService replyLikeService) {
        this.replyService = replyService;
        this.replyLikeService = replyLikeService;
    }

    @PostMapping("/comments/{commentId}/replies")
    @PreAuthorize("isAuthenticated()")
    public ResponseEntity<?> addReply(@PathVariable Long commentId,
                                      @Valid @RequestBody CommentDto dto,
                                      Principal principal) {
        if (principal == null) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
        }

        CommentReply saved = replyService.addReply(commentId, principal.getName(), dto);

        org.AE.alliededge.dto.PostDetailDto.CommentReplyViewDto rd =
            new org.AE.alliededge.dto.PostDetailDto.CommentReplyViewDto();
        rd.setId(saved.getId());
        rd.setContent(saved.getContent());
        rd.setCreatedAt(saved.getCreatedAt());
        rd.setLikes(saved.getLikes());
        rd.setLikedByMe(false);

        if (saved.getUser() != null) {
            org.AE.alliededge.dto.PostDetailDto.AuthorDto a = new org.AE.alliededge.dto.PostDetailDto.AuthorDto();
            a.setId(saved.getUser().getId());
            a.setUsername(saved.getUser().getUsername());
            a.setDisplayName(saved.getUser().getDisplayName() != null && !saved.getUser().getDisplayName().isBlank()
                ? saved.getUser().getDisplayName()
                : saved.getUser().getUsername());
            a.setProfileImageUrl(saved.getUser().getProfileImageUrl());
            a.setAdmin(saved.getUser().isAdmin());
            rd.setAuthor(a);
        }

        return ResponseEntity.status(HttpStatus.CREATED).body(Map.of("reply", rd, "commentId", commentId));
    }

    @PostMapping("/replies/{replyId}/like")
    @PreAuthorize("isAuthenticated()")
    public ResponseEntity<?> likeReply(@PathVariable Long replyId, Principal principal) {
        if (principal == null) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
        }

        boolean nowLiked = replyLikeService.toggleLike(replyId, principal.getName());
        CommentReply updated = replyService.getById(replyId);

        Map<String, Object> body = new java.util.LinkedHashMap<>();
        body.put("id", updated.getId());
        body.put("likes", updated.getLikes());
        body.put("liked", nowLiked);
        return ResponseEntity.ok(body);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/controller/CsrfController.java

```
package org.AE.alliededge.controller;

import org.springframework.security.web.csrf.CsrfToken;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

/**
 * CSRF bootstrap endpoint for the SPA.
 *
 * Spring Security will create/populate the XSRF-TOKEN cookie (CookieCsrfTokenRepository)
 * when this endpoint is hit. The frontend can call it once on app start (or before the
 * first unsafe request) to ensure subsequent POST/PUT/DELETE requests include the
 * X-CSRF-TOKEN header.
 */
@RestController
@RequestMapping("/api")
public class CsrfController {

    @GetMapping("/csrf")
    public Map<String, Object> csrf(CsrfToken token) {
        return Map.of(
            "headerName", token.getHeaderName(),
            "parameterName", token.getParameterName(),
            "token", token.getToken()
        );
    }
}
```

■ backend/src/main/java/org/AE/alliededge/controller/FollowController.java

```
package org.AE.alliededge.controller;

import org.AE.alliededge.service.UserFollowService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api")
public class FollowController {

    private final UserFollowService userFollowService;

    public FollowController(UserFollowService userFollowService) {
        this.userFollowService = userFollowService;
    }

    @PostMapping("/users/{id}/follow")
    public ResponseEntity<String> followUser(@PathVariable Long id, Authentication authentication) {
        if (authentication == null || !authentication.isAuthenticated()) {
            return ResponseEntity.status(401).body("Unauthorized");
        }

        try {
            String email = authentication.getName();
            userFollowService.followUser(email, id);
            return ResponseEntity.ok("Followed successfully");
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }

    @PostMapping("/users/{id}/unfollow")
```



```

    public ResponseEntity<String> unfollowUser(@PathVariable Long id, Authentication authentication) {
        if (authentication == null || !authentication.isAuthenticated()) {
            return ResponseEntity.status(401).body("Unauthorized");
        }

        try {
            String email = authentication.getName();
            userFollowService.unfollowUser(email, id);
            return ResponseEntity.ok("Unfollowed successfully");
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }
}

```

■ backend/src/main/java/org/AE/alliededge/controller/HealthController.java

```

package org.AE.alliededge.controller;

import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

/**
 * Simple health endpoints.
 *
 * Why:
 * - Render/Cloudflare (or other monitors) may probe GET/HEAD /.
 * - Without a mapping, Spring can log NoResourceFoundException for "/".
 * - Having a fast 200 OK endpoint also makes it easier to confirm the service is reachable.
 */
@RestController
public class HealthController {

    @GetMapping(value = "/", produces = MediaType.TEXT_PLAIN_VALUE)
    public String root() {
        return "OK";
    }

    @GetMapping(value = "/api/health", produces = MediaType.APPLICATION_JSON_VALUE)
    public Map<String, Object> apiHealth() {
        return Map.of("status", "UP");
    }
}

```

■ backend/src/main/java/org/AE/alliededge/controller/HomeController.java

```

package org.AE.alliededge.controller;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api")
public class HomeController {
    // Note: Root path "/" mapping removed - handled by PostController.getPosts()
}

```

■ backend/src/main/java/org/AE/alliededge/controller/PostController.java

```

package org.AE.alliededge.controller;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.validation.Valid;

```

```

import org.AE.alliededge.dto.CommentDto;
import org.AE.alliededge.dto.PostDetailDto;
import org.AE.alliededge.dto.PostFeedItemDto;
import org.AE.alliededge.dto.PostPageDto;
import org.AE.alliededge.exception.ResourceNotFoundException;
import org.AE.alliededge.model.Announcement;
import org.AE.alliededge.model.Comment;
import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.AE.alliededge.service.AnnouncementService;
import org.AE.alliededge.service.CommentService;
import org.AE.alliededge.service.PostService;
import org.AE.alliededge.service.UserService;
import org.AE.alliededge.service.CloudinaryService;
import org.AE.alliededge.service.CommentLikeService;
import org.AE.alliededge.service.CommentReplyLikeService;
import org.AE.alliededge.service.PostRealtimePublisher;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import org.springframework.web.server.ResponseStatusException;

import java.io.IOException;
import java.security.Principal;
import java.util.*;
import org.springframework.data.domain.PageRequest;
import org.AE.alliededge.repository.CommentRepository;

// Handles all post-related operations including feed retrieval,
// post creation, updates, likes, comments, and real-time updates
@RestController
@RequestMapping("/api")
public class PostController {

    private final PostService postService;
    private final CommentService commentService;
    private final UserService userService;
    private final CloudinaryService cloudinaryService;
    private final AnnouncementService announcementService;
    private final UserRepository userRepository;
    private final CommentLikeService commentLikeService;
    private final CommentReplyLikeService commentReplyLikeService;
    private final CommentRepository commentRepository;
    private final PostRealtimePublisher postRealtimePublisher;

    // Admin emails that are not allowed to like posts
    private static final Set<String> ADMIN_LIKE_BLOCKED_EMAILS = Set.of(
        "rayanmohammedrafeeq@gmail.com",
        "rakinmohammedrafeeq@gmail.com");

    public PostController(PostService postService, CommentService commentService, UserService userService,
        CloudinaryService cloudinaryService, AnnouncementService announcementService,
        UserRepository userRepository, CommentLikeService commentLikeService,
        CommentReplyLikeService commentReplyLikeService,
        CommentRepository commentRepository,
        PostRealtimePublisher postRealtimePublisher) {
        this.postService = postService;
        this.commentService = commentService;
        this.userService = userService;
        this.cloudinaryService = cloudinaryService;
        this.announcementService = announcementService;
    }

```

```

        this.userRepository = userRepository;
        this.commentLikeService = commentLikeService;
        this.commentReplyLikeService = commentReplyLikeService;
        this.commentRepository = commentRepository;
        this.postRealtimePublisher = postRealtimePublisher;
    }

    // ■ Home Page / Feed
    // Fetches paginated posts feed with sorting, search, and user-specific metadata
    @GetMapping("/posts")
    public ResponseEntity<Map<String, Object>>> getPosts(
        @PageableDefault(size = 5, sort = "createdAt", direction = Sort.Direction.DESC) Pageable pageable,
        @RequestParam(value = "keyword", required = false) String keyword,
        @RequestParam(value = "sortBy", required = false, defaultValue = "newest") String sortBy,
        Principal principal,
        HttpServletRequest request) {

        // Determine sort strategy
        Sort sort;
        switch (sortBy) {
            case "oldest" -> sort = Sort.by(Sort.Direction.ASC, "createdAt");
            case "mostLiked" -> sort = Sort.by(Sort.Direction.DESC, "likes");
            case "mostViewed" -> sort = Sort.by(Sort.Direction.DESC, "views");
            case "newest" -> sort = Sort.by(Sort.Direction.DESC, "createdAt");
            default -> {
                sort = Sort.by(Sort.Direction.DESC, "createdAt");
                sortBy = "newest";
            }
        }

        Pageable sortedPageable = PageRequest.of(pageable.getPageNumber(), pageable.getPageSize(), sort);

        // Normalize keyword and use the unified search query (title/content/author).
        String kw = (keyword == null) ? null : keyword.trim();
        Page<Post> postPage = postService.getAllPosts(kw, sortBy, sortedPageable);

        // Pre-compute comment counts for this page (avoid LAZY collection pitfalls)
        Map<Long, Integer> commentCountsByPostId = new HashMap<>();
        List<Long> postIds = postPage.getContent().stream()
            .map(Post::getId)
            .filter(Objects::nonNull)
            .toList();
        if (!postIds.isEmpty()) {
            for (Object[] row : commentRepository.countByPostIds(postIds)) {
                if (row == null || row.length < 2) continue;
                Long postId = row[0] != null ? ((Number) row[0]).longValue() : null;
                Integer cnt = row[1] != null ? ((Number) row[1]).intValue() : 0;
                if (postId != null) commentCountsByPostId.put(postId, cnt);
            }
        }

        // Map entities -> stable DTOs (prevents entity graph recursion)
        List<PostFeedItemDto> postDtos = postPage.getContent().stream().map(p -> {
            PostFeedItemDto dto = new PostFeedItemDto();
            dto.setId(p.getId());
            dto.setTitle(p.getTitle());
            dto.setContent(makeLinksClickable(p.getContent()));
            dto.setCreatedAt(p.getCreatedAt());
            dto.setLikes(p.getLikes());
            dto.setViews(p.getViews());
            dto.setCommentCount(commentCountsByPostId.getOrDefault(p.getId(), 0));
            dto.setAuthorName(p.getAuthorName());
            dto.setAuthorIsAdmin(p.isAuthorIsAdmin());
            // Persist liked state for feeds:
            if (principal != null) {
                dto.setLikedByMe(postService.hasUserLiked(p, principal.getName()));
            } else {
                dto.setLikedByMe(null);
            }
        });

        // Include media URLs so the UI can render images/videos on the feed.

```

```

        dto.setImageUrls(p.getImageUrls());
        dto.setVideoUrl(p.getVideoUrl());

        User u = p.getUser();
        if (u != null) {
            dto.setAuthorId(u.getId());
            dto.setAuthorUsername(u.getUsername());
            dto.setAuthorEmail(u.getEmail());
            // Prefer display name if present, fall back to username
            dto.setAuthorDisplayName(u.getDisplayName() != null && !u.getDisplayName().isBlank()
                ? u.getDisplayName()
                : u.getUsername());
            dto.setAuthorProfileImageUrl(u.getProfileImageUrl());
            // If author_name was not populated on legacy data, fill with best effort.
            if (dto.getAuthorName() == null || dto.getAuthorName().isBlank()) {
                dto.setAuthorName(dto.getAuthorDisplayName());
            }
        }

        return dto;
    }).toList();

    PostPageDto safePage = new PostPageDto();
    safePage.setContent(postDtos);
    safePage.setPageNumber(postPage.getNumber());
    safePage.setPageSize(postPage.getSize());
    safePage.setTotalElements(postPage.getTotalElements());
    safePage.setTotalPages(postPage.getTotalPages());
    safePage.setFirst(postPage.isFirst());
    safePage.setLast(postPage.isLast());

    // latest announcements (kept from original home logic)
    List<Announcement> latestAnnouncements = announcementService.findAllVisible()
        .stream()
        .limit(2)
        .toList();

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("postPage", safePage);
    body.put("sortBy", sortBy);
    body.put("keyword", keyword);
    body.put("latestAnnouncements", latestAnnouncements);

    if (principal != null) {
        String userEmail = principal.getName();
        body.put("currentUserEmail", userEmail);

        // first login + welcome announcement
        boolean firstLogin = false;
        Announcement welcomeAnnouncement = null;
        Optional<User> optUser = userRepository.findByEmail(userEmail);
        if (optUser.isPresent()) {
            User user = optUser.get();
            if (user.isFirstLogin()) {
                firstLogin = true;
                welcomeAnnouncement = announcementService.getWelcomeAnnouncement().orElse(null);
                user.setFirstLogin(false);
                userRepository.save(user);
            }
        }
        body.put("firstLogin", firstLogin);
        body.put("welcomeAnnouncement", welcomeAnnouncement);

        Set<Long> likedPostIds = new HashSet<>();
        for (Post p : postPage.getContent()) {
            if (postService.hasUserLiked(p, userEmail)) {
                likedPostIds.add(p.getId());
            }
        }
        body.put("likedPostIds", likedPostIds);
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
    }

```

```

        boolean isAdmin = auth != null && auth.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .anyMatch("ROLE_ADMIN"::equals);
        body.put("isAdmin", isAdmin);
        body.put("isAuthenticated", true);
        body.put("isAdminEmailLikeBlocked", ADMIN_LIKE_BLOCKED_EMAILS.contains(userEmail));
    } else {
        body.put("isAdmin", false);
        body.put("isAuthenticated", false);
        body.put("isAdminEmailLikeBlocked", false);
        body.put("firstLogin", false);
        body.put("welcomeAnnouncement", null);
    }

    body.put("baseUrl", getBaseUrl(request));
    return ResponseEntity.ok(body);
}

// ■ Post Detail
@GetMapping("/posts/{id}")
public ResponseEntity<Map<String, Object>> getPost(@PathVariable Long id,
    @RequestParam(value = "sortComments", required = false, defaultValue = "Principal principal",
        HttpServletRequest request) {

    Post post = postService.findPostById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Post not found with Id: " + id));

    String userEmail = null;
    if (principal != null) {
        userEmail = principal.getName();
        post = postService.recordView(id, userEmail);
    }

    post.setContent(makeLinksClickable(post.getContent()));

    boolean liked = principal != null && postService.hasUserLiked(post, userEmail);
    boolean isAuthenticated = principal != null;
    boolean isAdminEmailLikeBlocked = isAuthenticated && ADMIN_LIKE_BLOCKED_EMAILS.contains(userEmail);

    Authentication auth = SecurityContextHolder.getContext().getAuthentication();
    boolean isAdmin = auth != null && auth.getAuthorities().stream()
        .map(GrantedAuthority::getAuthority)
        .anyMatch("ROLE_ADMIN"::equals);

    // Build stable DTO (avoid exposing entities directly)
    PostDetailDto dto = new PostDetailDto();
    dto.setId(post.getId());
    dto.setTitle(post.getTitle());
    dto.setContent(post.getContent());
    dto.setCreatedAt(post.getCreatedAt());
    dto.setLikes(post.getLikes());
    dto.setViews(post.getViews());
    dto.setImageUrls(post.getImageUrls());
    dto.setVideoUrl(post.getVideoUrl());

    if (post.getUser() != null) {
        PostDetailDto.AuthorDto author = new PostDetailDto.AuthorDto();
        author.setId(post.getUser().getId());
        author.setUsername(post.getUser().getUsername());
        author.setDisplayName(
            post.getUser().getDisplayName() != null && !post.getUser().getDisplayName().isBlank()
                ? post.getUser().getDisplayName()
                : post.getUser().getUsername()
        );
        author.setProfileImageUrl(post.getUser().getProfileImageUrl());
        author.setAdmin(post.getUser().isAdmin());
        dto.setAuthor(author);
    }

    List<PostDetailDto.CommentViewDto> commentDtos = new ArrayList<>();
    List<Comment> comments = post.getComments() != null ? new ArrayList<>(post.getComments()) : new Arr

```

```

// Sort comments
switch (sortComments == null ? "newest" : sortComments) {
    case "oldest" -> comments.sort(Comparator.comparing(Comment::getCreatedAt, Comparator.nullsLast(Comparator.naturalOrder())));
    case "mostReplied" -> comments.sort((a, b) -> Integer.compare(
        b.getReplies() != null ? b.getReplies().size() : 0,
        a.getReplies() != null ? a.getReplies().size() : 0
    ));
    case "newest" -> comments.sort(Comparator.comparing(Comment::getCreatedAt, Comparator.nullsLast(Comparator.naturalOrder())));
    default -> comments.sort(Comparator.comparing(Comment::getCreatedAt, Comparator.nullsLast(Comparator.naturalOrder())));
}

for (Comment c : comments) {
    PostDetailDto.CommentViewDto cd = new PostDetailDto.CommentViewDto();
    cd.setId(c.getId());
    cd.setContent(c.getContent());
    cd.setCreatedAt(c.getCreatedAt());
    cd.setLikes(c.getLikes());
    cd.setLikedByMe(principal != null && commentLikeService.hasUserLiked(c, userEmail));

    if (c.getUser() != null) {
        PostDetailDto.AuthorDto ca = new PostDetailDto.AuthorDto();
        ca.setId(c.getUser().getId());
        ca.setUsername(c.getUser().getUsername());
        ca.setDisplayName(
            c.getUser().getDisplayName() != null && !c.getUser().getDisplayName().isBlank()
                ? c.getUser().getDisplayName()
                : c.getUser().getUsername()
        );
        ca.setProfileImageUrl(c.getUser().getProfileImageUrl());
        ca.setAdmin(c.getUser().isAdmin());
        cd.setAuthor(ca);
    }

    List<PostDetailDto.CommentReplyViewDto> replyDtos = new ArrayList<>();
    if (c.getReplies() != null) {
        List<org.AE.alliededge.model.CommentReply> replies = new ArrayList<>(c.getReplies());
        replies.sort(Comparator.comparing(org.AE.alliededge.model.CommentReply::getCreatedAt,
            Comparator.nullsLast(Comparator.naturalOrder())));

        for (org.AE.alliededge.model.CommentReply r : replies) {
            PostDetailDto.CommentReplyViewDto rd = new PostDetailDto.CommentReplyViewDto();
            rd.setId(r.getId());
            rd.setContent(r.getContent());
            rd.setCreatedAt(r.getCreatedAt());
            rd.setLikes(r.getLikes());
            rd.setLikedByMe(principal != null && commentReplyLikeService.hasUserLiked(r, userEmail));

            if (r.getUser() != null) {
                PostDetailDto.AuthorDto ra = new PostDetailDto.AuthorDto();
                ra.setId(r.getUser().getId());
                ra.setUsername(r.getUser().getUsername());
                ra.setDisplayName(
                    r.getUser().getDisplayName() != null && !r.getUser().getDisplayName().isBlank()
                        ? r.getUser().getDisplayName()
                        : r.getUser().getUsername()
                );
                ra.setProfileImageUrl(r.getUser().getProfileImageUrl());
                ra.setAdmin(r.getUser().isAdmin());
                rd.setAuthor(ra);
            }
            replyDtos.add(rd);
        }
    }
    cd.setReplies(replyDtos);

    commentDtos.add(cd);
}
dto.setComments(commentDtos);

Map<String, Object> body = new LinkedHashMap<>();
body.put("post", dto);

```

```

        body.put("liked", liked);
        body.put("isAuthenticated", isAuthenticated);
        body.put("isAdminEmailLikeBlocked", isAdminEmailLikeBlocked);
        body.put("isAdmin", isAdmin);
        body.put("baseUrl", getBaseUrl(request));
        body.put("sortComments", sortComments);

        return ResponseEntity.ok(body);
    }

    // ■ Add Comment
    @PostMapping("/posts/{postId}/comments")
    @PreAuthorize("isAuthenticated()")
    public ResponseEntity<?> submitComment(@PathVariable Long postId,
                                           @Valid @RequestBody CommentDto commentDto,
                                           Principal principal) {
        if (principal == null) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
        }
        String userEmail = principal.getName();
        Comment saved = commentService.saveComment(postId, userEmail, commentDto);

        // Realtime: notify all subscribers that this post's counts changed.
        // Publishes real-time updates (likes/comments) to subscribed clients via WebSockets
        postRealtimePublisher.publishPostStats(postId);

        PostDetailDto.CommentViewDto cd = new PostDetailDto.CommentViewDto();
        cd.setId(saved.getId());
        cd.setContent(saved.getContent());
        cd.setCreatedAt(saved.getCreatedAt());
        if (saved.getUser() != null) {
            PostDetailDto.AuthorDto ca = new PostDetailDto.AuthorDto();
            ca.setId(saved.getUser().getId());
            ca.setUsername(saved.getUser().getUsername());
            ca.setDisplayName(
                saved.getUser().getDisplayName() != null && !saved.getUser().getDisplayName().isBlank()
                ? saved.getUser().getDisplayName()
                : saved.getUser().getUsername()
            );
            ca.setProfileImageUrl(saved.getUser().getProfileImageUrl());
            ca.setAdmin(saved.getUser().isAdmin());
            cd.setAuthor(ca);
        }

        return ResponseEntity.status(HttpStatus.CREATED).body(Map.of("comment", cd, "postId", postId));
    }

    // ■ Create Post (Admin media upload endpoint)
    // NOTE: AdminController owns POST /api/admin/posts (metadata-only create).
    // This endpoint is specifically for multipart media uploads.
    @PostMapping("/admin/posts/media")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Post> createPostAsAdmin(@ModelAttribute Post post,
                                                  @RequestParam(value = "imageFiles", required = false) List<MultipartFile> imageFiles,
                                                  @RequestParam(value = "videoFile", required = false) MultipartFile videoFile,
                                                  Principal principal) throws IOException {
        if (principal == null) {
            throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Not authenticated");
        }

        if (post.getImageUrls() == null) {
            post.setImageUrls(new ArrayList<>());
        }

        if (imageFiles != null && !imageFiles.isEmpty()) {
            for (MultipartFile imageFile : imageFiles) {
                if (imageFile == null || imageFile.isEmpty()) {
                    continue;
                }
                if (imageFile.getSize() > 20L * 1024 * 1024) {

```

```

        throw new ResponseStatusException(HttpStatus.PAYLOAD_TOO_LARGE,
            "Image file is too large. Max 20MB per image.");
    }
    String contentType = imageFile.getContentType();
    if (contentType == null || !contentType.toLowerCase().startsWith("image/")) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
            "Only image files are allowed for images");
    }
    String imageUrl = cloudinaryService.uploadMedia(imageFile);
    post.getImageUrls().add(imageUrl);
}

}

if (videoFile != null && !videoFile.isEmpty()) {
    if (videoFile.getSize() > 100L * 1024 * 1024) {
        throw new ResponseStatusException(HttpStatus.PAYLOAD_TOO_LARGE,
            "Video file is too large. Max 100MB.");
    }
    String videoUrl = cloudinaryService.uploadVideo(videoFile, "posts/videos");
    post.setVideoUrl(videoUrl);
}

String userEmail = principal.getName();
Post saved = postService.savePost(post, userEmail);
return ResponseEntity.status(HttpStatus.CREATED).body(saved);
}

// ■ Update Post (Admin endpoint kept, now JSON)
@PutMapping("/admin/posts/{id}/media")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<Post> updatePostAsAdmin(@PathVariable Long id,
        @ModelAttribute Post post,
        @RequestParam(value = "imageFiles", required = false) List<MultipartFile> imageFiles,
        @RequestParam(value = "videoFile", required = false) MultipartFile videoFile,
        @RequestParam(value = "deleteImageId", required = false) List<Long> deleteImageIds,
        @RequestParam(value = "deleteVideo", required = false) Boolean deleteVideo,
        Principal principal) throws IOException {

    post.setId(id);

    Post existing = postService.findPostById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Post not found with Id: " + id));

    assertCanModifyPost(existing, principal);

    List<String> currentImages = existing.getImageUrls() != null
        ? new ArrayList<>(existing.getImageUrls())
        : new ArrayList<>();

    if (deleteImageIds != null && !deleteImageIds.isEmpty()) {
        deleteImageIds.stream()
            .sorted((a, b) -> Long.compare(b, a))
            .forEach(idxLong -> {
                int idx = idxLong.intValue();
                if (idx >= 0 && idx < currentImages.size()) {
                    String urlToDelete = currentImages.get(idx);
                    cloudinaryService.deleteMediaByUrl(urlToDelete);
                    currentImages.remove(idx);
                }
            });
    }

    post.setImageUrls(currentImages);

    if (imageFiles != null && !imageFiles.isEmpty()) {
        for (MultipartFile imageFile : imageFiles) {
            if (imageFile == null || imageFile.isEmpty()) {
                continue;
            }
            if (imageFile.getSize() > 20L * 1024 * 1024) {
                throw new ResponseStatusException(HttpStatus.PAYLOAD_TOO_LARGE,

```



```

        "Image file is too large. Max 20MB per image.");
    }
    String contentType = imageFile.getContentType();
    if (contentType == null || !contentType.toLowerCase().startsWith("image/")) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
            "Only image files are allowed for images");
    }
    String imageUrl = cloudinaryService.uploadMedia(imageFile);
    post.getImageUrls().add(imageUrl);
}

String effectiveVideoUrl = existing.getVideoUrl();
String effectiveVideoPublicId = existing.getVideoPublicId();

if (Boolean.TRUE.equals(deleteVideo) && effectiveVideoUrl != null) {
    if (effectiveVideoPublicId != null && !effectiveVideoPublicId.isBlank()) {
        cloudinaryService.deleteVideoByPublicId(effectiveVideoPublicId);
    } else {
        cloudinaryService.deleteMediaByUrl(effectiveVideoUrl);
    }
    effectiveVideoUrl = null;
    effectiveVideoPublicId = null;
}

if (videoFile != null && !videoFile.isEmpty()) {
    if (videoFile.getSize() > 100L * 1024 * 1024) {
        throw new ResponseStatusException(HttpStatus.PAYLOAD_TOO_LARGE,
            "Video file is too large. Max 100MB.");
    }
    if (effectiveVideoUrl != null && !Boolean.TRUE.equals(deleteVideo)) {
        if (effectiveVideoPublicId != null && !effectiveVideoPublicId.isBlank()) {
            cloudinaryService.deleteVideoByPublicId(effectiveVideoPublicId);
        } else {
            cloudinaryService.deleteMediaByUrl(effectiveVideoUrl);
        }
    }

    String videoUrl = cloudinaryService.uploadVideo(videoFile, "posts/videos");
    post.setVideoUrl(videoUrl);
} else {
    post.setVideoUrl(effectiveVideoUrl);
    post.setVideoPublicId(effectiveVideoPublicId);
}

String userEmail = principal.getName();
Post saved = postService.savePost(post, userEmail);
return ResponseEntity.ok(saved);
}

// ■ Create Post (authenticated user)
@PostMapping("/posts")
@PreAuthorize("hasAnyAuthority('ROLE_USER', 'ROLE_AUTHOR', 'ROLE_ADMIN')")
public ResponseEntity<Post> createAuthorPost(@Valid @ModelAttribute Post post,
        @RequestParam(value = "imageFiles", required = false) List<MultipartFile> imageFiles,
        @RequestParam(value = "videoFile", required = false) MultipartFile videoFile,
        Principal principal) throws IOException {
    if (principal == null) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Not authenticated");
    }
    if (post.getImageUrls() == null) {
        post.setImageUrls(new ArrayList<>());
    }
    if (imageFiles != null && !imageFiles.isEmpty()) {
        for (MultipartFile imageFile : imageFiles) {
            if (imageFile == null || imageFile.isEmpty()) continue;
            if (imageFile.getSize() > 20L * 1024 * 1024) {
                throw new ResponseStatusException(HttpStatus.PAYLOAD_TOO_LARGE,
                    "Image file is too large. Max 20MB per image.");
            }
            String contentType = imageFile.getContentType();

```

```

        if (contentType == null || !contentType.toLowerCase().startsWith("image/")) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "Only image files are allowed for images");
        }
        String imageUrl = cloudinaryService.uploadMedia(imageFile);
        post.getImageUrls().add(imageUrl);
    }
}
if (videoFile != null &&& !videoFile.isEmpty()) {
    if (videoFile.getSize() > 100L * 1024 * 1024) {
        throw new ResponseStatusException(HttpStatus.PAYLOAD_TOO_LARGE,
            "Video file is too large. Max 100MB.");
    }
    String videoUrl = cloudinaryService.uploadVideo(videoFile, "posts/videos");
    post.setVideoUrl(videoUrl);
}
String userEmail = principal.getName();
Post saved = postService.savePost(post, userEmail);
return ResponseEntity.status(HttpStatus.CREATED).body(saved);
}

// ■ Update Post (owner/admin)
// The SPA calls PUT /api/posts/{id}. Keep /posts/{id} for backwards compatibility.
@PutMapping("/posts/{id}")
@PreAuthorize("hasAnyAuthority('ROLE_USER', 'ROLE_AUTHOR', 'ROLE_ADMIN')")
public ResponseEntity<Post> updatePost(@PathVariable Long id,
    @Valid @ModelAttribute Post post,
    @RequestParam(value = "imageFiles", required = false) List<MultipartFile> imageFiles,
    @RequestParam(value = "videoFile", required = false) MultipartFile videoFile,
    @RequestParam(value = "deleteImageId", required = false) List<Long> deleteImageIds,
    @RequestParam(value = "deleteVideo", required = false) Boolean deleteVideo,
    Principal principal) throws IOException {

    post.setId(id);
    post.setDeleteVideo(Boolean.TRUE.equals(deleteVideo));

    Post existing = postService.findPostById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Post not found with Id: " + id));

    // Enforce owner/admin (inside method) rather than role-gating the endpoint.
    assertCanModifyPost(existing, principal);

    // Start from existing media, then apply deletions/uploads.
    List<String> currentImages = existing.getImageUrls() != null
        ? new ArrayList<>(existing.getImageUrls())
        : new ArrayList<>();

    if (deleteImageIds != null &&& !deleteImageIds.isEmpty()) {
        deleteImageIds.stream()
            .sorted((a, b) -> Long.compare(b, a))
            .forEach(idxLong -> {
                int idx = idxLong.intValue();
                if (idx >= 0 &&& idx < currentImages.size()) {
                    String urlToDelete = currentImages.get(idx);
                    cloudinaryService.deleteMediaByUrl(urlToDelete);
                    currentImages.remove(idx);
                }
            });
    }

    post.setImageUrls(currentImages);

    if (imageFiles != null &&& !imageFiles.isEmpty()) {
        for (MultipartFile imageFile : imageFiles) {
            if (imageFile == null || imageFile.isEmpty()) continue;
            if (imageFile.getSize() > 20L * 1024 * 1024) {
                throw new ResponseStatusException(HttpStatus.PAYLOAD_TOO_LARGE,
                    "Image file is too large. Max 20MB per image.");
            }
            String contentType = imageFile.getContentType();
            if (contentType == null || !contentType.toLowerCase().startsWith("image/")) {
                throw new ResponseStatusException(HttpStatus.BAD_REQUEST,

```

```

        "Only image files are allowed for images");
    }
    String imageUrl = cloudinaryService.uploadMedia(imageFile);
    post.getImageUrls().add(imageUrl);
}

String effectiveVideoUrl = existing.getVideoUrl();
String effectiveVideoPublicId = existing.getVideoPublicId();

if (Boolean.TRUE.equals(deleteVideo) && effectiveVideoUrl != null) {
    if (effectiveVideoPublicId != null && !effectiveVideoPublicId.isBlank()) {
        cloudinaryService.deleteVideoByPublicId(effectiveVideoPublicId);
    } else {
        cloudinaryService.deleteMediaByUrl(effectiveVideoUrl);
    }
    effectiveVideoUrl = null;
    effectiveVideoPublicId = null;
}

if (videoFile != null && !videoFile.isEmpty()) {
    if (videoFile.getSize() > 100L * 1024 * 1024) {
        throw new ResponseStatusException(HttpStatus.PAYLOAD_TOO_LARGE,
            "Video file is too large. Max 100MB.");
    }
    if (effectiveVideoUrl != null && !Boolean.TRUE.equals(deleteVideo)) {
        if (effectiveVideoPublicId != null && !effectiveVideoPublicId.isBlank()) {
            cloudinaryService.deleteVideoByPublicId(effectiveVideoPublicId);
        } else {
            cloudinaryService.deleteMediaByUrl(effectiveVideoUrl);
        }
    }
    String videoUrl = cloudinaryService.uploadVideo(videoFile, "posts/videos");
    post.setVideoUrl(videoUrl);
} else {
    post.setVideoUrl(effectiveVideoUrl);
    post.setVideoPublicId(effectiveVideoPublicId);
}

String userEmail = principal.getName();
Post saved = postService.savePost(post, userEmail);
return ResponseEntity.ok(saved);
}

// ■ Like Post
@PostMapping("/posts/{id}/like")
public ResponseEntity<Map<String, Object>> likePost(@PathVariable Long id, Principal principal) {
    if (principal == null) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
    }
    String userEmail = principal.getName();
    boolean nowLiked = postService.toggleLike(id, userEmail);

    Post updated = postService.findPostById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Post not found with Id: " + id));

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("liked", nowLiked);
    body.put("likes", updated.getLikes());
    body.put("id", updated.getId());

    // Realtime: notify all subscribers that this post's counts changed.
    postRealtimePublisher.publishPostStats(id);

    return ResponseEntity.ok(body);
}

// ■ My Posts (Author/Admin)
@GetMapping("/posts/mine")
@PreAuthorize("hasAnyAuthority('ROLE_AUTHOR', 'ROLE_ADMIN')")
public ResponseEntity<Page<Post>> getMyPosts(Principal principal,

```

```

        @RequestParam(value = "keyword", required = false) String keyword,
        @RequestParam(value = "sortBy", required = false, defaultValue = "newest")
        @PageableDefault(size = 10) Pageable pageable) {

    if (principal == null) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Not authenticated");
    }

    String email = principal.getName();
    User currentUser = userService.findByEmail(email)
        .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + email));

    if (sortBy == null || sortBy.isBlank()) {
        sortBy = "newest";
    }
    Sort sort;
    switch (sortBy) {
        case "oldest" -> sort = Sort.by(Sort.Direction.ASC, "createdAt");
        case "mostLiked" -> sort = Sort.by(Sort.Direction.DISC, "likes");
        case "mostViewed" -> sort = Sort.by(Sort.Direction.DISC, "views");
        default -> {
            sortBy = "newest";
            sort = Sort.by(Sort.Direction.DISC, "createdAt");
        }
    }

    Pageable sortedPageable = PageRequest.of(pageable.getPageNumber(), pageable.getPageSize(), sort);

    Page<Post> postPage = postService.getAuthorPosts(currentUser.getId(), keyword, sortBy, sortedPageable);
    return ResponseEntity.ok(postPage);
}

// ■■ Delete Post (owner/admin)
@DeleteMapping("/posts/{id}")
@PreAuthorize("hasAnyAuthority('ROLE_AUTHOR', 'ROLE_ADMIN')")
public ResponseEntity<?> deletePost(@PathVariable Long id, Principal principal) {
    Post existing = postService.findPostById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Post not found with Id: " + id));
    if (!canModify(existing, principal)) {
        throw new ResponseStatusException(HttpStatus.FORBIDDEN, "You are not allowed to delete this post");
    }
    postService.deletePostById(id);
    return ResponseEntity.noContent().build();
}

// Admin-only: Delete comment
@DeleteMapping("/admin/comments/{commentId}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<?> deleteCommentAsAdmin(@PathVariable Long commentId) {
    Long postId = commentService.deleteCommentByAdmin(commentId);
    if (postId == null) {
        return ResponseEntity.noContent().build();
    }
    return ResponseEntity.noContent().build();
}

// Delete a single image from a post (owner/admin)
@DeleteMapping("/posts/{postId}/images")
@PreAuthorize("hasAnyAuthority('ROLE_AUTHOR', 'ROLE_ADMIN')")
public ResponseEntity<Post> deleteImage(@PathVariable Long postId,
        @RequestParam("imageUrl") String imageUrl,
        Principal principal) {
    Post post = postService.findPostById(postId)
        .orElseThrow(() -> new ResourceNotFoundException("Post not found with Id: " + postId));
    assertCanModifyPost(post, principal);

    List<String> urls = post.getImageUrls();
    if (urls != null && urls.removeIf(u -> u.equals(imageUrl))) {
        post.setImageUrls(urls);
        postService.savePost(post, principal.getName());
        cloudinaryService.deleteMediaByUrl(imageUrl);
    }
}

```

```

        return ResponseEntity.ok(post);
    }

    // Delete a single image by index (handy for UI; keeps original behavior)
    @DeleteMapping("/posts/{postId}/images/{index}")
    @PreAuthorize("hasAnyAuthority('ROLE_AUTHOR', 'ROLE_ADMIN')")
    public ResponseEntity<Post> deleteImageByIndex(@PathVariable Long postId,
                                                    @PathVariable int index,
                                                    Principal principal) {
        Post post = postService.findPostById(postId)
            .orElseThrow(() -> new ResourceNotFoundException("Post not found with Id: " + postId));
        assertCanModifyPost(post, principal);

        List<String> urls = post.getImageUrls();
        if (urls != null && index >= 0 && index < urls.size()) {
            String urlToDelete = urls.get(index);
            urls.remove(index);
            post.setImageUrls(urls);
            postService.savePost(post, principal.getName());
            cloudinaryService.deleteMediaByUrl(urlToDelete);
        }

        return ResponseEntity.ok(post);
    }

    // Delete video from a post (owner/admin)
    @DeleteMapping("/posts/{postId}/video")
    @PreAuthorize("hasAnyAuthority('ROLE_AUTHOR', 'ROLE_ADMIN')")
    public ResponseEntity<Post> deleteVideo(@PathVariable Long postId, Principal principal) {
        Post post = postService.findPostById(postId)
            .orElseThrow(() -> new ResourceNotFoundException("Post not found with Id: " + postId));
        assertCanModifyPost(post, principal);

        String existingVideo = post.getVideoUrl();
        if (existingVideo != null) {
            post.setVideoUrl(null);
            postService.savePost(post, principal.getName());
            cloudinaryService.deleteMediaByUrl(existingVideo);
        }

        return ResponseEntity.ok(post);
    }

    private boolean canModify(Post post, Principal principal) {
        if (principal == null) return false;
        String email = principal.getName();
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        boolean isAdmin = auth != null && auth.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .anyMatch("ROLE_ADMIN"::equals);
        if (isAdmin) return true;
        User author = post.getUser();
        return author != null && author.getEmail() != null && author.getEmail().equalsIgnoreCase(email);
    }

    private void assertCanModifyPost(Post post, Principal principal) {
        if (principal == null) {
            throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Not authenticated");
        }
        String email = principal.getName();
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        boolean isAdmin = auth != null && auth.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .anyMatch("ROLE_ADMIN"::equals);
        boolean isOwner = post.getUser() != null && post.getUser().getEmail() != null
            && post.getUser().getEmail().equalsIgnoreCase(email);
        if (!isAdmin && !isOwner) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "You are not allowed to modify this post");
        }
    }

    private String makeLinksClickable(String content) {

```

```

        if (content == null) return "";
        String urlRegex = "(https?://[^\s]+)";
        if (content.toLowerCase().contains("<a ")) {
            return content;
        }
        return content.replaceAll(urlRegex, "<a href=\"$1\" target=\"_blank\" rel=\"noopener noreferrer\">$1</a>");
    }

    private String getBaseUrl(HttpServletRequest request) {
        String scheme = request.getScheme();
        String host = request.getServerName();
        int port = request.getServerPort();
        boolean defaultPort = (port == 80 || port == 443);
        return scheme + "://" + host + (defaultPort ? "" : ":" + port);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/controller/ProfileController.java

```

package org.AE.alliededge.controller;

import jakarta.servlet.http.HttpServletResponse;
import org.AE.alliededge.dto.ProfileResponseDto;
import org.AE.alliededge.dto.PublicProfileUserDto;
import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.User;
import org.AE.alliededge.service.*;
import org.AE.alliededge.service.ProfileService.AdminPostStats;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.net.URI;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api")
public class ProfileController {

    private final ProfileService profileService;
    private final UserService userService;
    private final PostService postService;
    private final ProjectService projectService;
    private final UserFollowService userFollowService;
    private final ResumeService resumeService;
    private final CloudinaryService cloudinaryService;

    public ProfileController(ProfileService profileService, UserService userService, PostService postService,
                             ProjectService projectService, UserFollowService userFollowService, ResumeService resumeService,
                             CloudinaryService cloudinaryService) {
        this.profileService = profileService;
        this.userService = userService;
        this.postService = postService;
        this.projectService = projectService;
        this.userFollowService = userFollowService;
        this.resumeService = resumeService;
        this.cloudinaryService = cloudinaryService;
    }

    @GetMapping("/profile")
    public ResponseEntity<Map<String, Object>> getMyProfile(Authentication authentication) {
        User user = profileService.getCurrentUser();
        if (user == null || authentication == null || !authentication.isAuthenticated()) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
        }
    }
}

```

```

Map<String, Object> body = new LinkedHashMap<>();
body.put("user", user);
body.put("followerCount", userFollowService.getFollowerCount(user.getId()));
body.put("followingCount", userFollowService.getFollowingCount(user.getId()));

List<org.AE.alliededge.model.Project> projects = projectService.getProjectsByUser(user.getId());
body.put("projects", projects);

List<String> skills = user.getSkills();
body.put("skills", skills != null ? skills : new java.util.ArrayList<>());

boolean canBecomeAuthor = !user.isAuthor() && !user.isAdmin();
body.put("canBecomeAuthor", canBecomeAuthor);

if (user.isAdmin()) {
    AdminPostStats stats = profileService.calculateAdminPostStats(user);
    body.put("adminStats", stats);
} else {
    long commentCount = userService.countCommentsByUser(user);
    long likeCount = userService.countLikesByUser(user);
    body.put("userCommentCount", commentCount);
    body.put("userLikeCount", likeCount);
}

return ResponseEntity.ok(body);
}

@PutMapping("/profile")
public ResponseEntity<Map<String, Object>> updateProfile(@RequestParam(required = false) String username,
    @RequestParam(required = false) String displayName,
    @RequestParam(required = false) String bio,
    @RequestParam(required = false) String linkedin,
    @RequestParam(required = false) String github,
    @RequestParam(required = false) String website,
    @RequestParam(required = false) String location,
    @RequestParam(required = false) String twitter,
    @RequestParam(required = false) String skillsJson,
    @RequestParam(required = false) String projectsJson,
    @RequestParam(required = false) String experienceJson,
    @RequestParam(required = false) String languagesJson,
    @RequestParam(required = false) String educationJson,
    @RequestParam(required = false) String availabilityJson,
    @RequestParam(required = false) Boolean showEmailOnProfile,
    @RequestParam(required = false) Boolean showResumeOnProfile,
    @RequestParam(name = "profileImageFile", required = false) MultipartFile profileImageFile,
    @RequestParam(name = "removeProfileImage", required = false) Boolean removeProfileImage,
    @RequestParam(name = "bannerImageFile", required = false) MultipartFile bannerImageFile,
    @RequestParam(name = "removeBannerImage", required = false) Boolean removeBannerImage,
    Authentication authentication) {

    User user = profileService.getCurrentUser();
    if (user == null || authentication == null || !authentication.isAuthenticated()) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
    }

    // Handle username change (for both admins and normal users)
    if (username != null) {
        String error = userService.validateAndChangeUsername(user, username);
        if (error != null) {
            return ResponseEntity.badRequest().body(Map.of("message", error, "field", "username"));
        }
    }

    // Handle profile image removal ONLY when explicitly requested.
    // Frontend always sends removeProfileImage as "true"/"false".
    if (Boolean.parseBoolean(removeProfileImage)) {
        profileService.removeProfileImage(user);
    }

    // Handle new profile image upload
    if (profileImageFile != null && !profileImageFile.isEmpty()) {
        try {

```

```

        profileService.updateProfileImage(user, profileImageFile);
    } catch (IllegalArgumentException | IllegalStateException ex) {
        return ResponseEntity.badRequest().body(Map.of("message", ex.getMessage(), "field", "profileImageFile"));
    }
}

// Handle banner image removal ONLY when explicitly requested.
if (Boolean.parseBoolean(removeBannerImage)) {
    profileService.removeBannerImage(user);
}

// Handle new banner image upload
if (bannerImageFile != null && !bannerImageFile.isEmpty()) {
    try {
        profileService.updateBannerImage(user, bannerImageFile);
    } catch (IllegalArgumentException | IllegalStateException ex) {
        return ResponseEntity.badRequest().body(Map.of("message", ex.getMessage(), "field", "bannerImageFile"));
    }
}

profileService.updateProfile(user, displayName, bio, linkedin, github, website,
    location, twitter, skillsJson, projectsJson, experienceJson, languagesJson, educationJson, availability,
    showEmailOnProfile, showResumeOnProfile);

// Re-fetch current user to ensure response matches DB state after any saves.
User refreshed = profileService.getCurrentUser();

return ResponseEntity.ok(Map.of(
    "message", "Profile updated",
    "user", ProfileResponseDto.from(refreshed != null ? refreshed : user)
));
}

@PostMapping("/profile/become-author")
public ResponseEntity<Map<String, Object>> becomeAuthor(Authentication authentication) {
    if (authentication == null || !authentication.isAuthenticated()) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
    }
    userService.upgradeCurrentUserToAuthor();
    return ResponseEntity.ok(Map.of("message", "Upgraded to author"));
}

@GetMapping("/profile/users/{userId}")
public ResponseEntity<Map<String, Object>> getPublicProfile(@PathVariable Long userId, Authentication authentication) {
    User user = userService.getUserById(userId);
    if (user == null) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("message", "User not found"));
    }

    List<Post> posts = postService.findPostsByUserId(userId);

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("user", PublicProfileUserDto.from(user));
    body.put("posts", posts);
    body.put("projects", projectService.getProjectsByUser(userId));
    body.put("skills", user.getSkills() != null ? user.getSkills() : new java.util.ArrayList<>());
    body.put("experience", user.getExperienceJson());
    body.put("languages", user.getLanguagesJson());
    body.put("education", user.getEducationJson());
    body.put("availability", user.getAvailabilityJson());
    body.put("followerCount", userFollowService.getFollowerCount(userId));
    body.put("followingCount", userFollowService.getFollowingCount(userId));

    boolean isOwner = false;
    boolean isFollowing = false;
    boolean canMessage = false;

    if (authentication != null && authentication.isAuthenticated()) {
        String principal = authentication.getName();
        isFollowing = userFollowService.isFollowing(principal, userId);
        canMessage = userFollowService.isMutualFollowing(principal, userId);
    }
}

```



```

        User currentUser = userService.getUserByPrincipal(principal);
        isOwner = currentUser != null && currentUser.getId().equals(userId);
    }

    body.put("isFollowing", isFollowing);
    body.put("canMessage", canMessage);
    body.put("isOwnProfile", isOwner);
    body.put("isOwner", isOwner);

    return ResponseEntity.ok(body);
}

/**
 * Public profile by username.
 *
 * This is the preferred endpoint for shareable profile links because usernames are user-facing.
 */
@GetMapping("/profile/u/{username}")
public ResponseEntity<Map<String, Object>> getPublicProfileByUsername(@PathVariable String username,
                                                                    Authentication authentication) {
    if (username == null || username.isBlank()) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of("message", "Username is required"));
    }

    User user = userService.findByUsername(username).orElse(null);
    if (user == null) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("message", "User not found"));
    }

    Long userId = user.getId();
    List<Post> posts = postService.findPostsByUserId(userId);

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("user", PublicProfileUserDto.from(user));
    body.put("posts", posts);
    body.put("projects", projectService.getProjectsByUser(userId));
    body.put("skills", user.getSkills() != null ? user.getSkills() : new java.util.ArrayList<>());
    body.put("experience", user.getExperienceJson());
    body.put("languages", user.getLanguagesJson());
    body.put("education", user.getEducationJson());
    body.put("availability", user.getAvailabilityJson());
    body.put("followerCount", userFollowService.getFollowerCount(userId));
    body.put("followingCount", userFollowService.getFollowingCount(userId));

    boolean isOwner = false;
    boolean isFollowing = false;
    boolean canMessage = false;

    if (authentication != null && authentication.isAuthenticated()) {
        String principal = authentication.getName();
        isFollowing = userFollowService.isFollowing(principal, userId);
        canMessage = userFollowService.isMutualFollowing(principal, userId);

        User currentUser = userService.getUserByPrincipal(principal);
        isOwner = currentUser != null && currentUser.getId().equals(userId);
    }

    body.put("isFollowing", isFollowing);
    body.put("canMessage", canMessage);
    body.put("isOwnProfile", isOwner);
    body.put("isOwner", isOwner);

    return ResponseEntity.ok(body);
}

@PostMapping("/profile/resume")
public ResponseEntity<Map<String, Object>> uploadResume(@RequestParam("resumeFile") MultipartFile resume,
                                                         Authentication authentication) {
    User user = profileService.getCurrentUser();
    if (user == null || authentication == null || !authentication.isAuthenticated()) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
    }
}

```

```

    }

    try {
        resumeService.uploadResume(user, resumeFile);
        return ResponseEntity.ok(Map.of("message", "Resume uploaded"));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(Map.of("message", e.getMessage()));
    }
}

@DeleteMapping("/profile/resume")
public ResponseEntity<Map<String, Object>> deleteResume(Authentication authentication) {
    User user = profileService.getCurrentUser();
    if (user == null || authentication == null || !authentication.isAuthenticated()) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
    }

    resumeService.deleteResume(user);
    return ResponseEntity.ok(Map.of("message", "Resume deleted"));
}

/**
 * Download own resume - streams PDF through server with proper headers
 * Only accessible to the profile owner
 */
@GetMapping("/profile/resume/download")
public void downloadOwnResume(HttpServletResponse response, Authentication authentication) {
    if (authentication == null || !authentication.isAuthenticated()) {
        response.setStatus(HttpServletResponse.SC_FORBIDDEN);
        return;
    }

    User user = profileService.getCurrentUser();
    if (user == null) {
        response.setStatus(HttpServletResponse.SC_FORBIDDEN);
        return;
    }

    if (user.getResumeUrl() == null || user.getResumeUrl().isEmpty()) {
        response.setStatus(HttpServletResponse.SC_NOT_FOUND);
        return;
    }

    String filename = user.getDisplayName() != null
        ? user.getDisplayName().replaceAll("\\s+", "_") + "_Resume.pdf"
        : "Resume.pdf";

    try {
        cloundinaryService.streamResumeToResponse(user.getResumeUrl(), filename, response);
    } catch (Exception e) {
        response.setStatus(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

/**
 * Resume link for a user (return URL rather than redirect/view)
 */
@GetMapping("/users/{userId}/resume")
public ResponseEntity<Map<String, Object>> getResumeLink(@PathVariable Long userId) {
    User user = userService.getUserById(userId);
    if (user == null || user.getResumeUrl() == null || user.getResumeUrl().isBlank()) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("message", "Resume not found"));
    }
    return ResponseEntity.ok(Map.of("resumeUrl", user.getResumeUrl()));
}

/**
 * Download resume by user id (HTTP 302 Location)
 */
@GetMapping("/users/{userId}/resume/download")
public ResponseEntity<Void> downloadResumeByUserId(@PathVariable Long userId) {

```

```

        User user = userService.getUserById(userId);
        if (user == null || user.getResumeUrl() == null) {
            return ResponseEntity.notFound().build();
        }

        String resumeUrl = user.getResumeUrl();
        if (resumeUrl == null || resumeUrl.isEmpty()) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity.status(HttpStatus.FOUND)
            .location(URI.create(resumeUrl))
            .build();
    }
}

```

■ backend/src/main/java/org/AE/alliededge/controller/SetupUsernameController.java

```

package org.AE.alliededge.controller;

import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.web.bind.annotation.*;

import java.util.LinkedHashMap;
import java.util.Map;

@RestController
@RequestMapping("/api")
public class SetupUsernameController {

    private final UserRepository userRepository;

    public SetupUsernameController(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    private User getCurrentUser() {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
        if (authentication == null) {
            return null;
        }

        Object principal = authentication.getPrincipal();

        // For OAuth2 and form login we treat authentication.getName() as the canonical email
        if (principal instanceof OAuth2User) {
            String email = authentication.getName();
            return userRepository.findByEmail(email).orElse(null);
        }

        // Fallback for non-OAuth principals (e.g. form login): also treat name as email
        String email = authentication.getName();
        return userRepository.findByEmail(email).orElse(null);
    }

    private boolean needsUsernameSetup(User user) {
        if (user == null) return false;
        // Admin users should never be forced through username setup
        if (user.isAdmin()) return false;
        String username = user.getUsername();
        return username == null || username.isBlank() || username.contains("@");
    }

    @GetMapping("/setup-username")

```

```

public ResponseEntity<Map<String, Object>> getSetupStatus() {
    User user = getCurrentUser();
    if (user == null) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
    }

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("needsSetup", needsUsernameSetup(user));
    body.put("currentDisplayName", user.getDisplayName());

    String suggested = null;
    if (user.getDisplayName() != null && !user.getDisplayName().isBlank()) {
        suggested = user.getDisplayName().trim().toLowerCase().replaceAll("[^a-z0-9]+", "-");
    } else if (user.getUsername() != null && user.getUsername().contains("@")) {
        suggested = user.getUsername().substring(0, user.getUsername().indexOf('@'));
    }
    body.put("suggestedUsername", suggested);

    return ResponseEntity.ok(body);
}

@PostMapping("/setup-username")
public ResponseEntity<Map<String, Object>> handleSetup(@RequestParam("username") String username) {
    User user = getCurrentUser();
    if (user == null) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("message", "Not authenticated"));
    }

    // If user is admin, or no longer needs setup, reject (frontend should just proceed)
    if (user.isAdmin() || !needsUsernameSetup(user)) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(Map.of("message", "Username setup not required"));
    }

    String trimmed = username == null ? "" : username.trim();
    String error = null;

    if (trimmed.isEmpty()) {
        error = "Username is required";
    } else if (trimmed.contains("@")) {
        error = "Username cannot be an email address";
    } else if (!trimmed.matches("[a-zA-Z0-9._-]{3,30}")) {
        error = "Username must be 3-30 characters and contain only letters, numbers, dots, underscores, or hyphens";
    } else if (userRepository.findByUsername(trimmed).isPresent()) {
        error = "Username is already taken";
    }

    if (error != null) {
        return ResponseEntity.badRequest().body(Map.of("message", error));
    }

    user.setUsername(trimmed);
    userRepository.save(user);

    return ResponseEntity.ok(Map.of("message", "Username set"));
}
}

```

■ backend/src/main/java/org/AE/alliededge/controller/UserController.java

```

package org.AE.alliededge.controller;

import jakarta.servlet.http.HttpServletResponse;
import org.AE.alliededge.model.Project;
import org.AE.alliededge.model.User;
import org.AE.alliededge.service.CloudinaryService;
import org.AE.alliededge.service.ProjectService;
import org.AE.alliededge.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;

```

```

import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api")
public class UserController {
    private final UserService userService;
    private final CloudinaryService cloudinaryService;

    @Autowired
    private ProjectService projectService;

    public UserController(UserService userService, CloudinaryService cloudinaryService) {
        this.userService = userService;
        this.cloudinaryService = cloudinaryService;
    }

    @PreAuthorize("isAuthenticated()")
    @GetMapping("/users/search")
    public ResponseEntity<Map<String, Object>> searchUsers(@RequestParam String keyword, Authentication authentication) {
        User currentUser = authentication == null ? null : userService.getUserByPrincipal(authentication.getName());
        Long excludeId = currentUser == null ? null : currentUser.getId();

        List<User> users = userService.searchUsersExcludingUserId(keyword, excludeId);
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("users", users);
        body.put("keyword", keyword);
        return ResponseEntity.ok(body);
    }

    @PreAuthorize("isAuthenticated()")
    @GetMapping("/resume/view/{id}")
    public ResponseEntity<Map<String, Object>> viewResume(@PathVariable Long id) {
        User user = userService.getUserById(id);
        String resumeUrl = user == null ? null : user.getResumeUrl();
        if (resumeUrl == null || resumeUrl.isBlank()) {
            if (user != null && user.getResumePublicId() != null) {
                resumeUrl = cloudinaryService.getRawUrlFromPublicId(user.getResumePublicId());
            }
        }
        if (resumeUrl == null || resumeUrl.isBlank()) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(Map.of("message", "Resume not found"));
        }
        return ResponseEntity.ok(Map.of("resumeUrl", resumeUrl));
    }

    @PreAuthorize("isAuthenticated()")
    @GetMapping("/resume/download/{id}")
    public void downloadResume(@PathVariable Long id, HttpServletResponse response, Authentication authentication) {
        if (authentication == null || !authentication.isAuthenticated()) {
            response.setStatus(HttpServletResponse.SC_FORBIDDEN);
            return;
        }

        User user = userService.getUserById(id);
        if (user == null) {
            response.setStatus(HttpServletResponse.SC_NOT_FOUND);
            return;
        }

        String resumeUrl = user.getResumeUrl();

        if (resumeUrl == null || resumeUrl.isEmpty()) {
            if (user.getResumePublicId() != null) {

```

```

        resumeUrl = cloudinaryService.getRawUrlFromPublicId(user.getResumePublicId());
    }
}

if (resumeUrl == null || resumeUrl.isEmpty()) {
    response.setStatus(HttpServletResponse.SC_NOT_FOUND);
    return;
}

String filename = user.getDisplayName() != null
    ? user.getDisplayName().replaceAll("\\s+", "_") + "_Resume.pdf"
    : user.getUsername() + "_Resume.pdf";

try {
    cloudinaryService.streamResumeToResponse(resumeUrl, filename, response);
} catch (Exception e) {
    response.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
}
}

// ===== SKILLS ENDPOINTS =====

@PreAuthorize("isAuthenticated()")
@PostMapping("/profile/skills")
public ResponseEntity<Map<String, Object>> addSkill(@RequestParam String skill, Authentication auth) {
    User currentUser = userService.getUserByEmail(auth.getName());
    if (currentUser != null && skill != null && !skill.trim().isEmpty()) {
        userService.addSkill(currentUser.getId(), skill.trim());
        return ResponseEntity.ok(Map.of("message", "Skill added successfully!"));
    }
    return ResponseEntity.badRequest().body(Map.of("message", "Invalid skill"));
}

@PreAuthorize("isAuthenticated()")
@DeleteMapping("/profile/skills")
public ResponseEntity<Map<String, Object>> removeSkill(@RequestParam String skill, Authentication auth) {
    User currentUser = userService.getUserByEmail(auth.getName());
    if (currentUser != null && skill != null) {
        userService.removeSkill(currentUser.getId(), skill);
        return ResponseEntity.ok(Map.of("message", "Skill removed successfully!"));
    }
    return ResponseEntity.badRequest().body(Map.of("message", "Invalid skill"));
}

// ===== PROJECTS ENDPOINTS =====

@PreAuthorize("isAuthenticated()")
@PostMapping("/profile/projects")
public ResponseEntity<Map<String, Object>> addProject(@RequestParam String title,
                                                    @RequestParam String description,
                                                    @RequestParam(required = false) String projectUrl,
                                                    Authentication auth) {
    User currentUser = userService.getUserByEmail(auth.getName());
    if (currentUser != null && title != null && !title.trim().isEmpty()) {
        Project project = new Project();
        project.setTitle(title.trim());
        project.setDescription(description != null ? description.trim() : "");
        project.setProjectUrl(projectUrl != null && !projectUrl.trim().isEmpty() ? projectUrl.trim() : null);
        project.setUser(currentUser);

        projectService.saveProject(project);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(Map.of("message", "Project added successfully!", "project", project));
    }
    return ResponseEntity.badRequest().body(Map.of("message", "Invalid project"));
}

@PreAuthorize("isAuthenticated()")
@PutMapping("/profile/projects/{id}")
public ResponseEntity<Map<String, Object>> editProject(@PathVariable Long id,
                                                    @RequestParam String title,

```

```

        @RequestParam String description,
        @RequestParam(required = false) String projectUrl,
        Authentication auth) {
    User currentUser = userService.getUserByEmail(auth.getName());
    Project project = projectService.findById(id).orElse(null);

    if (project != null && currentUser != null && project.getUser().getId().equals(currentUser.getId())) {
        project.setTitle(title.trim());
        project.setDescription(description != null ? description.trim() : "");
        project.setProjectUrl(projectUrl != null && !projectUrl.trim().isEmpty() ? projectUrl.trim() : null);

        projectService.saveProject(project);
        return ResponseEntity.ok(Map.of("message", "Project updated successfully!", "project", project));
    }

    return ResponseEntity.status(HttpStatus.FORBIDDEN)
        .body(Map.of("message", "Unauthorized or project not found"));
}

@PreAuthorize("isAuthenticated()")
@DeleteMapping("/profile/projects/{id}")
public ResponseEntity<Map<String, Object>> deleteProject(@PathVariable Long id, Authentication auth) {
    User currentUser = userService.getUserByEmail(auth.getName());
    Project project = projectService.findById(id).orElse(null);

    if (project != null && currentUser != null && project.getUser().getId().equals(currentUser.getId())) {
        projectService.deleteProject(id);
        return ResponseEntity.ok(Map.of("message", "Project deleted successfully!"));
    }

    return ResponseEntity.status(HttpStatus.FORBIDDEN)
        .body(Map.of("message", "Unauthorized or project not found"));
}
}

```

■ backend/src/main/java/org/AE/alliededge/controller/WsDebugController.java

```

package org.AE.alliededge.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

/**
 * Temporary diagnostics endpoint to verify if /ws/** isn't being accidentally blocked.
 */
@RestController
@RequestMapping("/ws")
public class WsDebugController {

    @GetMapping("/debug")
    public ResponseEntity<?> debug() {
        return ResponseEntity.ok(Map.of("ok", true));
    }
}

```

■ backend/src/main/java/org/AE/alliededge/dto/AvailabilityDto.java

```

package org.AE.alliededge.dto;

import com.fasterxml.jackson.annotation.JsonInclude;

@JsonInclude(JsonInclude.Include.NON_NULL)
public record AvailabilityDto(
    Boolean openToCollaboration,

```

```

        Boolean openToInternships,
        Boolean openToFreelance,
        Boolean justBuilding
    ) {
    }
}

```

■ backend/src/main/java/org/AE/alliededge/dto/ChatMessageDTO.java

```

package org.AE.alliededge.dto;

import java.time.LocalDateTime;

public class ChatMessageDTO {
    private Long roomId;
    private String senderUsername;
    private String senderDisplayName;
    private String content;
    private LocalDateTime timestamp;

    public ChatMessageDTO() {
    }

    public ChatMessageDTO(Long roomId, String senderUsername, String senderDisplayName, String content,
        LocalDateTime timestamp) {
        this.roomId = roomId;
        this.senderUsername = senderUsername;
        this.senderDisplayName = senderDisplayName;
        this.content = content;
        this.timestamp = timestamp;
    }

    // Getters and Setters

    public Long getRoomId() {
        return roomId;
    }

    public void setRoomId(Long roomId) {
        this.roomId = roomId;
    }

    public String getSenderUsername() {
        return senderUsername;
    }

    public void setSenderUsername(String senderUsername) {
        this.senderUsername = senderUsername;
    }

    public String getSenderDisplayName() {
        return senderDisplayName;
    }

    public void setSenderDisplayName(String senderDisplayName) {
        this.senderDisplayName = senderDisplayName;
    }

    public String getContent() {
        return content;
    }

    public void setContent(String content) {
        this.content = content;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(LocalDateTime timestamp) {
    }
}

```



```

        this.timestamp = timestamp;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/dto/ChatRoomDTO.java

```

package org.AE.alliededge.dto;

import java.time.LocalDateTime;

public class ChatRoomDTO {
    private Long roomId;
    private Long otherUserId;
    private String otherUserUsername;
    private String otherUserDisplayName;
    private String otherUserProfileImage;
    private String lastMessage;
    private LocalDateTime lastMessageTime;

    public ChatRoomDTO() {
    }

    public ChatRoomDTO(Long roomId, Long otherUserId, String otherUserUsername, String otherUserDisplayName,
        String otherUserProfileImage, String lastMessage, LocalDateTime lastMessageTime) {
        this.roomId = roomId;
        this.otherUserId = otherUserId;
        this.otherUserUsername = otherUserUsername;
        this.otherUserDisplayName = otherUserDisplayName;
        this.otherUserProfileImage = otherUserProfileImage;
        this.lastMessage = lastMessage;
        this.lastMessageTime = lastMessageTime;
    }

    // Getters and Setters

    public Long getRoomId() {
        return roomId;
    }

    public void setRoomId(Long roomId) {
        this.roomId = roomId;
    }

    public Long getOtherUserId() {
        return otherUserId;
    }

    public void setOtherUserId(Long otherUserId) {
        this.otherUserId = otherUserId;
    }

    public String getOtherUserUsername() {
        return otherUserUsername;
    }

    public void setOtherUserUsername(String otherUserUsername) {
        this.otherUserUsername = otherUserUsername;
    }

    public String getOtherUserDisplayName() {
        return otherUserDisplayName;
    }

    public void setOtherUserDisplayName(String otherUserDisplayName) {
        this.otherUserDisplayName = otherUserDisplayName;
    }

    public String getOtherUserProfileImage() {
        return otherUserProfileImage;
    }
}

```

```

    public void setOtherUserProfileImage(String otherUserProfileImage) {
        this.otherUserProfileImage = otherUserProfileImage;
    }

    public String getLastMessage() {
        return lastMessage;
    }

    public void setLastMessage(String lastMessage) {
        this.lastMessage = lastMessage;
    }

    public LocalDateTime getLastMessageTime() {
        return lastMessageTime;
    }

    public void setLastMessageTime(LocalDateTime lastMessageTime) {
        this.lastMessageTime = lastMessageTime;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/dto/CommentDto.java

```

package org.AE.alliededge.dto;

import jakarta.validation.constraints.NotEmpty;

public class CommentDto {
    @NotEmpty(message = "Content must not be empty")
    public String content;

    public String getContent() {
        return content;
    }

    public void setContent(String content) {
        this.content = content;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/dto/EducationDto.java

```

package org.AE.alliededge.dto;

import com.fasterxml.jackson.annotation.JsonInclude;

@JsonInclude(JsonInclude.Include.NON_NULL)
public record EducationDto(
    Long id,
    String school,
    String degree,
    String field,
    String startYear,
    String endYear,
    String cgpa
) {
}

```

■ backend/src/main/java/org/AE/alliededge/dto/ExperienceDto.java

```

package org.AE.alliededge.dto;

import com.fasterxml.jackson.annotation.JsonInclude;

@JsonInclude(JsonInclude.Include.NON_NULL)

```

```

public record ExperienceDto(
    Long id,
    String company,
    String position,
    String startDate,
    String endDate,
    String description
) {
}

```

■ backend/src/main/java/org/AE/alliededge/dto/LanguageDto.java

```

package org.AE.alliededge.dto;

import com.fasterxml.jackson.annotation.JsonInclude;

@JsonInclude(JsonInclude.Include.NON_NULL)
public record LanguageDto(
    Long id,
    String language,
    String proficiency
) {
}

```

■ backend/src/main/java/org/AE/alliededge/dto/PostDetailDto.java

```

package org.AE.alliededge.dto;

import java.time.LocalDateTime;
import java.util.List;

/**
 * Stable DTO for post detail view.
 * Avoids exposing JPA entities directly (prevents lazy proxy / serialization issues).
 */
public class PostDetailDto {
    private Long id;
    private String title;
    private String content;
    private LocalDateTime createdAt;
    private int likes;
    private int views;
    private List<String> imageUrls;
    private String videoUrl;

    private AuthorDto author;
    private List<CommentViewDto> comments;

    public static class AuthorDto {
        private Long id;
        private String username;
        private String displayName;
        private String profileImageUrl;
        private boolean admin;

        public Long getId() { return id; }
        public void setId(Long id) { this.id = id; }

        public String getUsername() { return username; }
        public void setUsername(String username) { this.username = username; }

        public String getDisplayName() { return displayName; }
        public void setDisplayName(String displayName) { this.displayName = displayName; }

        public String getProfileImageUrl() { return profileImageUrl; }
        public void setProfileImageUrl(String profileImageUrl) { this.profileImageUrl = profileImageUrl; }

        public boolean isAdmin() { return admin; }
    }
}

```

```

        public void setAdmin(boolean admin) { this.admin = admin; }
    }

    public static class CommentReplyViewDto {
        private Long id;
        private String content;
        private java.time.LocalDateTime createdAt;
        private AuthorDto author;
        private int likes;
        private boolean likedByMe;

        public Long getId() { return id; }
        public void setId(Long id) { this.id = id; }

        public String getContent() { return content; }
        public void setContent(String content) { this.content = content; }

        public java.time.LocalDateTime getCreatedAt() { return createdAt; }
        public void setCreatedAt(java.time.LocalDateTime createdAt) { this.createdAt = createdAt; }

        public AuthorDto getAuthor() { return author; }
        public void setAuthor(AuthorDto author) { this.author = author; }

        public int getLikes() { return likes; }
        public void setLikes(int likes) { this.likes = likes; }

        public boolean isLikedByMe() { return likedByMe; }
        public void setLikedByMe(boolean likedByMe) { this.likedByMe = likedByMe; }
    }

    public static class CommentViewDto {
        private Long id;
        private String content;
        private LocalDateTime createdAt;
        private AuthorDto author;
        private List<CommentReplyViewDto> replies;
        private int likes;
        private boolean likedByMe;

        public Long getId() { return id; }
        public void setId(Long id) { this.id = id; }

        public String getContent() { return content; }
        public void setContent(String content) { this.content = content; }

        public LocalDateTime getCreatedAt() { return createdAt; }
        public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

        public AuthorDto getAuthor() { return author; }
        public void setAuthor(AuthorDto author) { this.author = author; }

        public List<CommentReplyViewDto> getReplies() { return replies; }
        public void setReplies(List<CommentReplyViewDto> replies) { this.replies = replies; }

        public int getLikes() { return likes; }
        public void setLikes(int likes) { this.likes = likes; }

        public boolean isLikedByMe() { return likedByMe; }
        public void setLikedByMe(boolean likedByMe) { this.likedByMe = likedByMe; }
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getContent() { return content; }
    public void setContent(String content) { this.content = content; }

    public LocalDateTime getCreatedAt() { return createdAt; }

```

```

    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

    public int getLikes() { return likes; }
    public void setLikes(int likes) { this.likes = likes; }

    public int getViews() { return views; }
    public void setViews(int views) { this.views = views; }

    public List<String> getImageUrls() { return imageUrls; }
    public void setImageUrls(List<String> imageUrls) { this.imageUrls = imageUrls; }

    public String getVideoUrl() { return videoUrl; }
    public void setVideoUrl(String videoUrl) { this.videoUrl = videoUrl; }

    public AuthorDto getAuthor() { return author; }
    public void setAuthor(AuthorDto author) { this.author = author; }

    public List<CommentViewDto> getComments() { return comments; }
    public void setComments(List<CommentViewDto> comments) { this.comments = comments; }
}

```

■ backend/src/main/java/org/AE/alliededge/dto/PostFeedItemDto.java

```

package org.AE.alliededge.dto;

import java.time.LocalDateTime;
import java.util.List;

/**
 * Minimal post representation safe for JSON serialization (no entity graphs).
 */
public class PostFeedItemDto {
    private Long id;
    private String title;
    private String content;
    private LocalDateTime createdAt;
    private int likes;
    private int views;

    /** Number of comments for this post (cheap aggregate for feeds/dashboards). */
    private int commentCount;

    /** Whether the current authenticated user has liked this post (null for anonymous). */
    private Boolean likedByMe;

    private String authorName;
    private Boolean authorIsAdmin;

    /** Optional author metadata (kept flat to avoid recursion) */
    private Long authorId;
    private String authorUsername;
    private String authorEmail;
    private String authorDisplayName;
    private String authorProfileImageUrl;

    private List<String> imageUrls;
    private String videoUrl;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }
}

```

```

public void setTitle(String title) {
    this.title = title;
}

public String getContent() {
    return content;
}

public void setContent(String content) {
    this.content = content;
}

public LocalDateTime getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDateTime createdAt) {
    this.createdAt = createdAt;
}

public int getLikes() {
    return likes;
}

public void setLikes(int likes) {
    this.likes = likes;
}

public int getViews() {
    return views;
}

public void setViews(int views) {
    this.views = views;
}

public int getCommentCount() {
    return commentCount;
}

public void setCommentCount(int commentCount) {
    this.commentCount = commentCount;
}

public Boolean getLikedByMe() {
    return likedByMe;
}

public void setLikedByMe(Boolean likedByMe) {
    this.likedByMe = likedByMe;
}

public String getAuthorName() {
    return authorName;
}

public void setAuthorName(String authorName) {
    this.authorName = authorName;
}

public Boolean getAuthorIsAdmin() {
    return authorIsAdmin;
}

public void setAuthorIsAdmin(Boolean authorIsAdmin) {
    this.authorIsAdmin = authorIsAdmin;
}

public Long getAuthorId() {
    return authorId;
}

```

```

    public void setAuthorId(Long authorId) {
        this.authorId = authorId;
    }

    public String getAuthorUsername() {
        return authorUsername;
    }

    public void setAuthorUsername(String authorUsername) {
        this.authorUsername = authorUsername;
    }

    public String getAuthorEmail() {
        return authorEmail;
    }

    public void setAuthorEmail(String authorEmail) {
        this.authorEmail = authorEmail;
    }

    public String getAuthorDisplayName() {
        return authorDisplayName;
    }

    public void setAuthorDisplayName(String authorDisplayName) {
        this.authorDisplayName = authorDisplayName;
    }

    public String getAuthorProfileImageUrl() {
        return authorProfileImageUrl;
    }

    public void setAuthorProfileImageUrl(String authorProfileImageUrl) {
        this.authorProfileImageUrl = authorProfileImageUrl;
    }

    public List<String> getImageUrls() {
        return imageUrls;
    }

    public void setImageUrls(List<String> imageUrls) {
        this.imageUrls = imageUrls;
    }

    public String getVideoUrl() {
        return videoUrl;
    }

    public void setVideoUrl(String videoUrl) {
        this.videoUrl = videoUrl;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/dto/PostPageDto.java

```

package org.AE.alliededge.dto;

import java.util.List;

/**
 * Page wrapper DTO to avoid serializing Spring Data Page internals.
 */
public class PostPageDto {

    private List<PostFeedItemDto> content;
    private int pageNumber;
    private int pageSize;
    private long totalElements;
    private int totalPages;
    private boolean first;
}

```

```

private boolean last;

public List<PostFeedItemDto> getContent() {
    return content;
}

public void setContent(List<PostFeedItemDto> content) {
    this.content = content;
}

public int getPageNumber() {
    return pageNumber;
}

public void setPageNumber(int pageNumber) {
    this.pageNumber = pageNumber;
}

public int getPageSize() {
    return pageSize;
}

public void setPageSize(int pageSize) {
    this.pageSize = pageSize;
}

public long getTotalElements() {
    return totalElements;
}

public void setTotalElements(long totalElements) {
    this.totalElements = totalElements;
}

public int getTotalPages() {
    return totalPages;
}

public void setTotalPages(int totalPages) {
    this.totalPages = totalPages;
}

public boolean isFirst() {
    return first;
}

public void setFirst(boolean first) {
    this.first = first;
}

public boolean isLast() {
    return last;
}

public void setLast(boolean last) {
    this.last = last;
}
}

```

■ backend/src/main/java/org/AE/alliededge/dto/PostStatsDto.java

```

package org.AE.alliededge.dto;

import java.time.Instant;

/**
 * Lightweight realtime payload for post counters.
 * Sent over STOMP to: /topic/posts/{postId}
 */
public class PostStatsDto {

```



```

private Long postId;
private Integer likes;
private Integer commentCount;
private Instant updatedAt;

public PostStatsDto() {
}

public PostStatsDto(Long postId, Integer likes, Integer commentCount, Instant updatedAt) {
    this.postId = postId;
    this.likes = likes;
    this.commentCount = commentCount;
    this.updatedAt = updatedAt;
}

public Long getPostId() {
    return postId;
}

public void setPostId(Long postId) {
    this.postId = postId;
}

public Integer getLikes() {
    return likes;
}

public void setLikes(Integer likes) {
    this.likes = likes;
}

public Integer getCommentCount() {
    return commentCount;
}

public void setCommentCount(Integer commentCount) {
    this.commentCount = commentCount;
}

public Instant getUpdatedAt() {
    return updatedAt;
}

public void setUpdatedAt(Instant updatedAt) {
    this.updatedAt = updatedAt;
}
}

```

■ backend/src/main/java/org/AE/alliededge/dto/ProfileResponseDto.java

```

package org.AE.alliededge.dto;

import org.AE.alliededge.model.User;

/**
 * Minimal, safe representation of a user profile for frontend consumption.
 * Keeps response shape stable and avoids serializing JPA relationships.
 */
public class ProfileResponseDto {
    private Long id;
    private String email;
    private String username;
    private String role;
    private boolean admin;

    private String displayName;
    private String bio;
    private String linkedin;
    private String github;
    private String website;
}

```

```

private String location;
private String twitter;

private String profileImageUrl;
private String bannerImageUrl;
private String resumeUrl;

// JSON sections (frontend parses them)
private String experienceJson;
private String languagesJson;
private String educationJson;
private String availabilityJson;

public static ProfileResponseDto from(User u) {
    ProfileResponseDto dto = new ProfileResponseDto();
    dto.id = u.getId();
    dto.email = u.getEmail();
    dto.username = u.getUsername();
    dto.role = u.getRole();
    dto.admin = u.isAdmin();
    dto.displayName = u.getDisplayName();
    dto.bio = u.getBio();
    dto.linkedin = u.getLinkedin();
    dto.github = u.getGithub();
    dto.website = u.getWebsite();
    dto.location = u.getLocation();
    dto.twitter = u.getTwitter();
    dto.profileImageUrl = u.getProfileImageUrl();
    dto.bannerImageUrl = u.getBannerImageUrl();
    dto.resumeUrl = u.getResumeUrl();
    dto.experienceJson = u.getExperienceJson();
    dto.languagesJson = u.getLanguagesJson();
    dto.educationJson = u.getEducationJson();
    dto.availabilityJson = u.getAvailabilityJson();
    return dto;
}

public Long getId() {
    return id;
}

public String getEmail() {
    return email;
}

public String getUsername() {
    return username;
}

public String getRole() {
    return role;
}

public boolean isAdmin() {
    return admin;
}

public String getDisplayName() {
    return displayName;
}

public String getBio() {
    return bio;
}

public String getLinkedin() {
    return linkedin;
}

public String getGithub() {
    return github;
}

```

```

    }

    public String getWebsite() {
        return website;
    }

    public String getLocation() {
        return location;
    }

    public String getTwitter() {
        return twitter;
    }

    public String getProfileImageUrl() {
        return profileImageUrl;
    }

    public String getBannerImageUrl() {
        return bannerImageUrl;
    }

    public String getResumeUrl() {
        return resumeUrl;
    }

    public String getExperienceJson() {
        return experienceJson;
    }

    public String getLanguagesJson() {
        return languagesJson;
    }

    public String getEducationJson() {
        return educationJson;
    }

    public String getAvailabilityJson() {
        return availabilityJson;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/dto/ProfileSectionsDto.java

```

package org.AE.alliededge.dto;

import com.fasterxml.jackson.annotation.JsonInclude;

import java.util.List;

/**
 * JSON sections edited on the profile page.
 * Stored in User as JSON strings.
 */
@JsonInclude(JsonInclude.Include.NON_NULL)
public record ProfileSectionsDto(
    List<ExperienceDto> experience,
    List<LanguageDto> languages,
    List<EducationDto> education,
    AvailabilityDto availability
) {
}

```

■ backend/src/main/java/org/AE/alliededge/dto/ProjectDto.java

```

package org.AE.alliededge.dto;

import org.AE.alliededge.model.Project;

```

```

import java.util.List;
import java.util.Map;

/**
 * DTO for the frontend's project editor.
 */
public record ProjectDto(
    Long id,
    String title,
    String summary,
    String status,
    String problem,
    String built,
    String role,
    List<String> techStack,
    Map<String, Object> proofLinks
) {
    public static ProjectDto from(Project p) {
        if (p == null) return null;
        return new ProjectDto(
            p.getId(),
            p.getTitle(),
            p.getSummary(),
            p.getStatus(),
            p.getProblem(),
            p.getBuilt(),
            p.getRole(),
            p.getTechStack(),
            p.getProofLinks()
        );
    }
}

```

■ backend/src/main/java/org/AE/alliededge/dto/PublicProfileUserDto.java

```

package org.AE.alliededge.dto;

import org.AE.alliededge.model.User;

/**
 * A sanitized view of {@link User} suitable for public profile pages.
 *
 * IMPORTANT: Do not add sensitive fields here (email, resumeUrl, etc.).
 */
public class PublicProfileUserDto {
    private Long id;
    private String username;
    private String displayName;

    private String bio;
    private String linkedin;
    private String github;
    private String website;
    private String location;
    private String twitter;

    private String experienceJson;
    private String languagesJson;
    private String educationJson;
    private String availabilityJson;

    private String profileImageUrl;
    private String bannerImageUrl;

    // Visibility flags (so the frontend can render conditional sections)
    private boolean showEmailOnProfile;
    private boolean showResumeOnProfile;

    // Public role signal used for the verified badge.
    // This is safe to expose and does not grant permissions.
}

```

```

private boolean admin;

// Optional public fields (only set when corresponding show* flags are true)
private String email;
private String resumeUrl;

public static PublicProfileUserDto from(User u) {
    if (u == null) return null;
    PublicProfileUserDto dto = new PublicProfileUserDto();
    dto.id = u.getId();
    dto.username = u.getUsername();
    dto.displayName = u.getDisplayName();
    dto.bio = u.getBio();
    dto.linkedin = u.getLinkedin();
    dto.github = u.getGithub();
    dto.website = u.getWebsite();
    dto.location = u.getLocation();
    dto.twitter = u.getTwitter();
    dto.experienceJson = u.getExperienceJson();
    dto.languagesJson = u.getLanguagesJson();
    dto.educationJson = u.getEducationJson();
    dto.availabilityJson = u.getAvailabilityJson();
    dto.profileImageUrl = u.getProfileImageUrl();
    dto.bannerImageUrl = u.getBannerImageUrl();
    dto.showEmailOnProfile = u.isShowEmailOnProfile();
    dto.showResumeOnProfile = u.isShowResumeOnProfile();
    dto.admin = u.isAdmin();

    dto.email = dto.showEmailOnProfile ? u.getEmail() : null;
    dto.resumeUrl = dto.showResumeOnProfile ? u.getResumeUrl() : null;
    return dto;
}

public Long getId() {
    return id;
}

public String getUsername() {
    return username;
}

public String getDisplayName() {
    return displayName;
}

public String getBio() {
    return bio;
}

public String getLinkedin() {
    return linkedin;
}

public String getGithub() {
    return github;
}

public String getWebsite() {
    return website;
}

public String getLocation() {
    return location;
}

public String getTwitter() {
    return twitter;
}

public String getExperienceJson() {
    return experienceJson;
}

```

```

    }

    public String getLanguagesJson() {
        return languagesJson;
    }

    public String getEducationJson() {
        return educationJson;
    }

    public String getAvailabilityJson() {
        return availabilityJson;
    }

    public String getProfileImageUrl() {
        return profileImageUrl;
    }

    public String getBannerImageUrl() {
        return bannerImageUrl;
    }

    public boolean isShowEmailOnProfile() {
        return showEmailOnProfile;
    }

    public boolean isShowResumeOnProfile() {
        return showResumeOnProfile;
    }

    public String getEmail() {
        return email;
    }

    public String getResumeUrl() {
        return resumeUrl;
    }

    public boolean isAdmin() {
        return admin;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/dto/SendChatMessageRequest.java

```

package org.AE.alliededge.dto;

public class SendChatMessageRequest {
    private String content;

    public SendChatMessageRequest() {
    }

    public SendChatMessageRequest(String content) {
        this.content = content;
    }

    public String getContent() {
        return content;
    }

    public void setContent(String content) {
        this.content = content;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/exception/ApiExceptionHandler.java

```

package org.AE.alliededge.exception;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.core.Ordered;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.server.ResponseStatusException;

import java.time.Instant;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Ensures API endpoints always return JSON on errors.
 *
 * The UI expects JSON responses for /api/** even on failures.
 * Without this, Spring may render an HTML error page for exceptions like ResponseStatusException,
 * which triggers the frontend "Unexpected response from server" message.
 */
@RestControllerAdvice
@Order(Ordered.HIGHEST_PRECEDENCE)
public class ApiExceptionHandler {

    @ExceptionHandler(ResponseStatusException.class)
    public ResponseEntity<Map<String, Object>>> handleResponseStatusException(ResponseStatusException ex,
                                                                              HttpServletRequest request) {

        String uri = request.getRequestURI();
        if (uri == null || !uri.startsWith("/api/")) {
            // Let MVC/other handlers deal with non-API requests.
            throw ex;
        }

        HttpStatus status = HttpStatus.resolve(ex.getStatusCode().value());
        if (status == null) status = HttpStatus.INTERNAL_SERVER_ERROR;

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("status", status.value());
        body.put("error", status.getReasonPhrase());
        body.put("message", ex.getReason() != null ? ex.getReason() : status.getReasonPhrase());
        body.put("path", uri);
        body.put("timestamp", Instant.now().toString());

        return ResponseEntity.status(status)
            .header(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE)
            .body(body);
    }

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<Map<String, Object>>> handleRuntimeException(RuntimeException ex,
                                                                        HttpServletRequest request) {

        String uri = request.getRequestURI();
        if (uri == null || !uri.startsWith("/api/")) {
            throw ex;
        }

        HttpStatus status = HttpStatus.INTERNAL_SERVER_ERROR;

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("status", status.value());
        body.put("error", status.getReasonPhrase());
        body.put("message", ex.getMessage() != null ? ex.getMessage() : "internal_server_error");
        body.put("path", uri);
        body.put("timestamp", Instant.now().toString());

        return ResponseEntity.status(status)
            .header(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE)

```

```

        .body(body);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/exception/GlobalExceptionHandler.java

```

package org.AE.alliededge.exception;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.multipart.MaxUploadSizeExceededException;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@ControllerAdvice
public class GlobalExceptionHandler {
    private static final Logger logger = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(ResourceNotFoundException.class)
    public String handleResourceNotFoundException(ResourceNotFoundException ex, Model model) {
        model.addAttribute("errorMessage", ex.getMessage());
        return "error/404";
    }

    // Handle client aborts from media streaming (e.g. large Cloudinary-hosted videos) gracefully
    @ExceptionHandler(org.apache.catalina.connector.ClientAbortException.class)
    public void handleClientAbort(org.apache.catalina.connector.ClientAbortException ex,
        HttpServletRequest request) {
        logger.debug("ClientAbortException: {}", ex.getMessage());
        // The client closed the connection (e.g., stopped video) - not really an application error.
        logger.info("Client aborted connection while handling URI={}", request.getRequestURI());
        // Do not attempt to render a view or write to the response; just swallow.
    }

    @ExceptionHandler(Exception.class)
    public Object handleGenericException(Exception ex, Model model,
        HttpServletRequest request,
        HttpServletResponse response) {
        final String uri = request.getRequestURI();
        logger.error("An unexpected error occurred at URI=" + uri, ex);

        // For API calls, never render Thymeleaf HTML.
        // Let RestControllerAdvice handlers (ApiExceptionHandler) produce a structured JSON body.
        if (uri != null && uri.startsWith("/api/")) {
            // Re-throw to let ApiExceptionHandler (or Spring) handle it.
            if (ex instanceof RuntimeException re) {
                throw re;
            }
            throw new RuntimeException(ex);
        }

        // If the response is already committed (e.g., while streaming a file),
        // we cannot render an error view. Just log and return null.
        if (response.isCommitted()) {
            logger.warn("Response already committed for URI={}, skipping error view rendering", uri);
            return null;
        }

        model.addAttribute("errorMessage", "An unexpected error occurred. Please try again later.");
        model.addAttribute("path", uri);
        return "error/generic-error";
    }

    @ExceptionHandler(MaxUploadSizeExceededException.class)
    public String handleMaxSizeException(MaxUploadSizeExceededException exc, RedirectAttributes redirectAttributes) {
        String detail = exc.getMessage(); // ensure parameter referenced
    }
}

```



```

        logger.warn("File upload exceeded max size: " + detail);
        redirectAttributes.addFlashAttribute("message", "\u26a0\u200f File too large! Please upload a smaller image or
        return "redirect:/admin/posts/new";
    }
}

```

■ backend/src/main/java/org/AE/alliededge/exception/ResourceNotFoundException.java

```

package org.AE.alliededge.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(value = HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String message) {
        super(message);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/model/Announcement.java

```

package org.AE.alliededge.model;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import jakarta.persistence.*;

@Entity
public class Announcement {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    @Column(length = 2000)
    private String message;

    private LocalDateTime createdAt;

    private boolean visible;

    @Column(name = "is_welcome_announcement")
    private boolean isWelcomeAnnouncement = false;

    @Column(name = "video_url")
    private String videoUrl;

    @Column(name = "video_public_id")
    private String videoPublicId;

    @ElementCollection
    @CollectionTable(name = "announcement_images", joinColumns = @JoinColumn(name = "announcement_id"))
    @Column(name = "image_url", length = 1000)
    private List<String> imageUrls = new ArrayList<>();

    // Constructors
    public Announcement() {
        this.visible = true;
        this.createdAt = LocalDateTime.now();
    }

    // Getters and Setters

```

```

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getMessage() {
    return message;
}

public void setMessage(String message) {
    this.message = message;
}

public LocalDateTime getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDateTime createdAt) {
    this.createdAt = createdAt;
}

public boolean isVisible() {
    return visible;
}

public void setVisible(boolean visible) {
    this.visible = visible;
}

public boolean isWelcomeAnnouncement() {
    return isWelcomeAnnouncement;
}

public void setWelcomeAnnouncement(boolean welcomeAnnouncement) {
    isWelcomeAnnouncement = welcomeAnnouncement;
}

public String getVideoUrl() {
    return videoUrl;
}

public void setVideoUrl(String videoUrl) {
    this.videoUrl = videoUrl;
}

public String getVideoPublicId() {
    return videoPublicId;
}

public void setVideoPublicId(String videoPublicId) {
    this.videoPublicId = videoPublicId;
}

public List<String> getImageUrls() {
    return imageUrls;
}

public void setImageUrls(List<String> imageUrls) {
    this.imageUrls = imageUrls;
}

```

```
}
```

■ backend/src/main/java/org/AE/alliededge/model/ChatMessage.java

```
package org.AE.alliededge.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "chat_message")
public class ChatMessage {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "chat_room_id", nullable = false)
    private ChatRoom chatRoom;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "sender_id", nullable = false)
    private User sender;

    @Column(columnDefinition = "TEXT", nullable = false)
    private String content;

    @Column(nullable = false)
    private LocalDateTime timestamp;

    @PrePersist
    protected void onCreate() {
        timestamp = LocalDateTime.now();
    }

    // Getters and Setters

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public ChatRoom getChatRoom() {
        return chatRoom;
    }

    public void setChatRoom(ChatRoom chatRoom) {
        this.chatRoom = chatRoom;
    }

    public User getSender() {
        return sender;
    }

    public void setSender(User sender) {
        this.sender = sender;
    }

    public String getContent() {
        return content;
    }

    public void setContent(String content) {
        this.content = content;
    }
}
```

```

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(LocalDateTime timestamp) {
        this.timestamp = timestamp;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/model/ChatRoom.java

```

package org.AE.alliededge.model;

import jakarta.persistence.*;

@Entity
@Table(name = "chat_room")
public class ChatRoom {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user1_id", nullable = false)
    private User user1;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user2_id", nullable = false)
    private User user2;

    // Helper method to check if a user is a participant in this room
    public boolean isParticipant(User user) {
        return (user1 != null && user1.getId().equals(user.getId())) ||
            (user2 != null && user2.getId().equals(user.getId()));
    }

    // Getters and Setters

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public User getUser1() {
        return user1;
    }

    public void setUser1(User user1) {
        this.user1 = user1;
    }

    public User getUser2() {
        return user2;
    }

    public void setUser2(User user2) {
        this.user2 = user2;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/model/Comment.java

```

package org.AE.alliededge.model;

import jakarta.persistence.*;

```

```

import jakarta.validation.constraints.NotEmpty;
import com.fasterxml.jackson.annotation.JsonIgnore;

import java.time.LocalDateTime;
import java.util.List;

@Entity
public class Comment {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotEmpty(message = "Content must not be empty")
    @Column(nullable = false, length = 2000)
    private String content;

    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "post_id", nullable = false)
    @JsonIgnore
    private Post post;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @OneToMany(mappedBy = "comment", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<CommentReply> replies;

    // Denormalized like count (synchronized with comment_likes table)
    @Column(nullable = false)
    private int likes = 0;

    public User getUser() {
        return user;
    }

    public void setUser(User user) {
        this.user = user;
    }

    public Post getPost() {
        return post;
    }

    public void setPost(Post post) {
        this.post = post;
    }

    public List<CommentReply> getReplies() {
        return replies;
    }

    public void setReplies(List<CommentReply> replies) {
        this.replies = replies;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getContent() {
        return content;
    }

```

```

    }

    public void setContent(String content) {
        this.content = content;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }

    public int getLikes() {
        return likes;
    }

    public void setLikes(int likes) {
        this.likes = likes;
    }

    @Override
    public String toString() {
        return "Comment{" +
            "id=" + id +
            ", content='" + content + '\'' +
            ", createdAt=" + createdAt +
            '}';
    }
}

```

■ backend/src/main/java/org/AE/alliededge/model/CommentLike.java

```

package org.AE.alliededge.model;

import jakarta.persistence.*;

@Entity
@Table(name = "comment_likes", uniqueConstraints = @UniqueConstraint(columnNames = {"comment_id", "user_id"}))
public class CommentLike {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "comment_id", nullable = false)
    private Comment comment;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public Comment getComment() {
        return comment;
    }

    public void setComment(Comment comment) {
        this.comment = comment;
    }
}

```

```

    public User getUser() {
        return user;
    }

    public void setUser(User user) {
        this.user = user;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/model/CommentReply.java

```

package org.AE.alliededge.model;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import jakarta.validation.constraints.NotEmpty;

import java.time.LocalDateTime;

@Entity
@Table(name = "comment_reply")
public class CommentReply {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotEmpty(message = "Content must not be empty")
    @Column(nullable = false, length = 2000)
    private String content;

    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "comment_id", nullable = false)
    @JsonIgnore
    private Comment comment;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    // Denormalized like count (synchronized with comment_reply_likes table)
    @Column(nullable = false)
    private int likes = 0;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getContent() { return content; }
    public void setContent(String content) { this.content = content; }

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

    public Comment getComment() { return comment; }
    public void setComment(Comment comment) { this.comment = comment; }

    public User getUser() { return user; }
    public void setUser(User user) { this.user = user; }

    public int getLikes() { return likes; }
    public void setLikes(int likes) { this.likes = likes; }
}

```

■ backend/src/main/java/org/AE/alliededge/model/CommentReplyLike.java

```

package org.AE.alliededge.model;

```

```

import jakarta.persistence.*;

@Entity
@Table(name = "comment_reply_likes", uniqueConstraints = @UniqueConstraint(columnNames = {"reply_id", "user_id"}))
public class CommentReplyLike {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "reply_id", nullable = false)
    private CommentReply reply;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public CommentReply getReply() { return reply; }
    public void setReply(CommentReply reply) { this.reply = reply; }

    public User getUser() { return user; }
    public void setUser(User user) { this.user = user; }
}

```

■ backend/src/main/java/org/AE/alliededge/model/Post.java

```

package org.AE.alliededge.model;
import jakarta.persistence.*;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.Size;
import com.fasterxml.jackson.annotation.JsonIgnore;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
public class Post {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotEmpty(message = "Title must not be empty")
    @Size(min = 5, max = 100, message = "Title must be between 5 and 100 characters")
    @Column(nullable = false)
    private String title;

    @NotEmpty(message = "Content must not be empty")
    @Column(nullable = false, length = 10000)
    private String content;

    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @ElementCollection
    @CollectionTable(name = "post_images", joinColumns = @JoinColumn(name = "post_id"))
    @Column(name = "image_url")
    private List<String> imageUrls = new ArrayList<>();

    @Column(name = "video_url")
    private String videoUrl;

    // New field to track Cloudinary video public_id so that we can delete it reliably
    @Column(name = "video_public_id")
    private String videoPublicId;

    // Transient flag used only for edit form binding to request video deletion
}

```



```

@Transient
private boolean deleteVideo;

@Column(nullable = false)
private int likes = 0; // synchronized with PostLike count

// New field to track views
@Column(nullable = false)
private int views = 0;

// Denormalized author metadata for UI
@Column(name = "author_name")
private String authorName;

@Column(name = "author_is_admin", nullable = false)
private boolean authorIsAdmin = false;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "user_id", nullable = false)
@JsonIgnore
private User user;

@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Comment> comments;

public Post() {
}

public int getLikes() { return likes; }
public void setLikes(int likes) { this.likes = likes; }
public User getUser() { return user; }
public void setUser(User user) { this.user = user; }
public List<Comment> getComments() { return comments; }
public void setComments(List<Comment> comments) { this.comments = comments; }
public String getTitle() { return title; }
public void setTitle(String title) { this.title = title; }
public LocalDateTime getCreatedAt() { return createdAt; }
public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
public String getContent() { return content; }
public void setContent(String content) { this.content = content; }
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }

// New getters/setters for multiple image URLs
public List<String> getImageUrls() {
    return imageUrls;
}

public void setImageUrls(List<String> imageUrls) {
    this.imageUrls = imageUrls;
}

public String getVideoUrl() { return videoUrl; }
public void setVideoUrl(String videoUrl) { this.videoUrl = videoUrl; }

// Getter/Setter for Cloudinary video public_id
public String getVideoPublicId() {
    return videoPublicId;
}

public void setVideoPublicId(String videoPublicId) {
    this.videoPublicId = videoPublicId;
}

// Getter/Setter for transient deleteVideo flag
public boolean isDeleteVideo() {
    return deleteVideo;
}

public void setDeleteVideo(boolean deleteVideo) {
    this.deleteVideo = deleteVideo;
}

```

```

    }

    // Getter/Setter for views
    public int getViews() { return views; }
    public void setViews(int views) { this.views = views; }

    public String getAuthorName() {
        return authorName;
    }

    public void setAuthorName(String authorName) {
        this.authorName = authorName;
    }

    public boolean isAuthorIsAdmin() {
        return authorIsAdmin;
    }

    public void setAuthorIsAdmin(boolean authorIsAdmin) {
        this.authorIsAdmin = authorIsAdmin;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/model/PostLike.java

```

package org.AE.alliededge.model;

import jakarta.persistence.*;

@Entity
@Table(name = "post_likes", uniqueConstraints = @UniqueConstraint(columnNames = {"post_id", "user_id"}))
public class PostLike {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "post_id", nullable = false)
    private Post post;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Post getPost() { return post; }
    public void setPost(Post post) { this.post = post; }
    public User getUser() { return user; }
    public void setUser(User user) { this.user = user; }
}

```

■ backend/src/main/java/org/AE/alliededge/model/PostView.java

```

package org.AE.alliededge.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "post_views", uniqueConstraints = @UniqueConstraint(columnNames = {"post_id", "user_id"}))
public class PostView {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "post_id", nullable = false)

```

```

private Post post;

@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "user_id", nullable = false)
private User user;

@Column(name = "viewed_at", nullable = false)
private LocalDateTime viewedAt;

public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
public Post getPost() { return post; }
public void setPost(Post post) { this.post = post; }
public User getUser() { return user; }
public void setUser(User user) { this.user = user; }
public LocalDateTime getViewedAt() { return viewedAt; }
public void setViewedAt(LocalDateTime viewedAt) { this.viewedAt = viewedAt; }
}

```

■ backend/src/main/java/org/AE/alliededge/model/Project.java

```

package org.AE.alliededge.model;

import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.persistence.*;
import org.hibernate.annotations.CreationTimestamp;

import java.time.LocalDateTime;
import java.util.Collections;
import java.util.List;
import java.util.Map;

@Entity
@Table(name = "project")
public class Project {

    @Transient
    private static final ObjectMapper JSON = new ObjectMapper().findAndRegisterModules();

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    // Frontend uses summary/description; keep legacy 'description' but persist the richer fields too.
    @Column(length = 1000)
    private String summary;

    @Column(length = 1000)
    private String description;

    @Column(length = 64)
    private String status;

    @Column(length = 2000)
    private String problem;

    @Column(length = 2000)
    private String built;

    @Column(length = 255)
    private String role;

    @Column(name = "tech_stack_json", columnDefinition = "TEXT")
    private String techStackJson;

    @Column(name = "proof_links_json", columnDefinition = "TEXT")

```

```

private String proofLinksJson;

// Legacy URL field (used by older parts of the app)
@Column(name = "project_url", length = 500)
private String projectUrl;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "user_id", nullable = false)
private User user;

@CreationTimestamp
@Column(name = "created_at", updatable = false)
private LocalDateTime createdAt;

// Constructors
public Project() {
}

public Project(String title, String description, String projectUrl, User user) {
    this.title = title;
    this.description = description;
    this.projectUrl = projectUrl;
    this.user = user;
}

// Getters and Setters
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getSummary() {
    return summary;
}

public void setSummary(String summary) {
    this.summary = summary;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}

public String getStatus() {
    return status;
}

public void setStatus(String status) {
    this.status = status;
}

public String getProblem() {
    return problem;
}

public void setProblem(String problem) {

```

```

        this.problem = problem;
    }

    public String getBuilt() {
        return built;
    }

    public void setBuilt(String built) {
        this.built = built;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
        this.role = role;
    }

    public List<String> getTechStack() {
        if (techStackJson == null || techStackJson.isBlank()) return Collections.emptyList();
        try {
            return JSON.readValue(techStackJson, new TypeReference<List<String>>>() {});
        } catch (Exception e) {
            return Collections.emptyList();
        }
    }

    public void setTechStack(List<String> techStack) {
        try {
            this.techStackJson = techStack == null ? null : JSON.writeValueAsString(techStack);
        } catch (Exception e) {
            this.techStackJson = null;
        }
    }

    public Map<String, Object> getProofLinks() {
        if (proofLinksJson == null || proofLinksJson.isBlank()) return Collections.emptyMap();
        try {
            return JSON.readValue(proofLinksJson, new TypeReference<Map<String, Object>>>() {});
        } catch (Exception e) {
            return Collections.emptyMap();
        }
    }

    public void setProofLinks(Map<String, Object> proofLinks) {
        try {
            this.proofLinksJson = proofLinks == null ? null : JSON.writeValueAsString(proofLinks);
        } catch (Exception e) {
            this.proofLinksJson = null;
        }
    }

    public String getProjectUrl() {
        return projectUrl;
    }

    public void setProjectUrl(String projectUrl) {
        this.projectUrl = projectUrl;
    }

    public User getUser() {
        return user;
    }

    public void setUser(User user) {
        this.user = user;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

```

```

    }

    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/model/User.java

```

package org.AE.alliededge.model;

import com.fasterxml.jackson.annotation.JsonIgnore;
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;

import java.util.List;

@Entity
@Table(name = "users")
@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // Canonical login identifier (Google email for OAuth logins)
    @Column(unique = true, nullable = false)
    private String email;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private String role; // ROLE_USER, ROLE_AUTHOR, ROLE_ADMIN

    // Human-readable display name for UI (e.g. "Rayan Mohammed Rafeeq")
    @Column(name = "display_name")
    private String displayName;

    // Admin flag for UI/behaviour (in addition to role)
    @Column(nullable = false)
    private boolean admin = false;

    // First login flag for showing welcome announcement
    @Column(nullable = false)
    private boolean firstLogin = true;

    // Optional status field for blocking/suspension (e.g. ACTIVE, BLOCKED)
    @Column(length = 32)
    private String status;

    // Admin-enforced ban: when true, user cannot log in.
    @Column(nullable = false)
    private boolean banned = false;

    // --- Profile fields ---
    @Column(length = 512)
    private String bio;

    @Column(length = 255)
    private String linkedin;

    @Column(length = 255)
    private String github;

    @Column(length = 255)
    private String website;
}

```

```

// --- Additional profile fields persisted for the profile page ---

@Column(length = 255)
private String location;

@Column(length = 255)
private String twitter;

/**
 * The following sections are edited in the UI but don't currently have relational tables.
 * Persist them as JSON strings (stored as TEXT) to keep the schema simple.
 */
@Column(name = "experience_json", columnDefinition = "TEXT")
private String experienceJson;

@Column(name = "languages_json", columnDefinition = "TEXT")
private String languagesJson;

@Column(name = "education_json", columnDefinition = "TEXT")
private String educationJson;

@Column(name = "availability_json", columnDefinition = "TEXT")
private String availabilityJson;

// Canonical profile image URL (Cloudinary secure_url or other external URL)
@Column(name = "profile_image_url", length = 512)
private String profileImageUrl;

// User-uploaded banner image URL (Cloudinary secure_url or other external URL)
@Column(name = "banner_image_url", length = 512)
private String bannerImageUrl;

// Legacy columns kept for schema compatibility but no longer used by the app
@Column(name = "profile_image_path", length = 512)
private String profileImagePath;

@Column(name = "profile_photo", length = 512)
private String profilePhoto;

// Resume upload fields
@Column(name = "resume_url", length = 512)
private String resumeUrl;

@Column(name = "resume_public_id", length = 255)
private String resumePublicId;

// Skills collection
@ElementCollection(fetch = FetchType.EAGER)
@CollectionTable(name = "user_skills", joinColumns = @JoinColumn(name = "user_id"))
@Column(name = "skills")
private List<String> skills = new java.util.ArrayList<>();

// Projects relationship
@OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)
private List<Project> projects = new java.util.ArrayList<>();

@OneToMany(mappedBy = "user")
private List<Post> posts;

@OneToMany(mappedBy = "user")
private List<Comment> comments;

// --- Public profile visibility flags ---
@Column(name = "show_email_on_profile", nullable = false)
private boolean showEmailOnProfile = false;

@Column(name = "show_resume_on_profile", nullable = false)
private boolean showResumeOnProfile = false;

public Long getId() {
    return id;
}

```

```

    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    @JsonIgnore
    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
        this.role = role;
    }

    public String getDisplayName() {
        return displayName;
    }

    public void setDisplayName(String displayName) {
        this.displayName = displayName;
    }

    public boolean isAdmin() {
        return admin;
    }

    public void setAdmin(boolean admin) {
        this.admin = admin;
    }

    public boolean isFirstLogin() {
        return firstLogin;
    }

    public void setFirstLogin(boolean firstLogin) {
        this.firstLogin = firstLogin;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }

    public boolean isBanned() {

```



```

        return banned;
    }

    public void setBanned(boolean banned) {
        this.banned = banned;
    }

    public String getBio() {
        return bio;
    }

    public void setBio(String bio) {
        this.bio = bio;
    }

    public String getLinkedin() {
        return linkedin;
    }

    public void setLinkedin(String linkedin) {
        this.linkedin = linkedin;
    }

    public String getGithub() {
        return github;
    }

    public void setGithub(String github) {
        this.github = github;
    }

    public String getWebsite() {
        return website;
    }

    public void setWebsite(String website) {
        this.website = website;
    }

    public String getLocation() {
        return location;
    }

    public void setLocation(String location) {
        this.location = location;
    }

    public String getTwitter() {
        return twitter;
    }

    public void setTwitter(String twitter) {
        this.twitter = twitter;
    }

    public String getExperienceJson() {
        return experienceJson;
    }

    public void setExperienceJson(String experienceJson) {
        this.experienceJson = experienceJson;
    }

    public String getLanguagesJson() {
        return languagesJson;
    }

    public void setLanguagesJson(String languagesJson) {
        this.languagesJson = languagesJson;
    }

    public String getEducationJson() {

```

```

        return educationJson;
    }

    public void setEducationJson(String educationJson) {
        this.educationJson = educationJson;
    }

    public String getAvailabilityJson() {
        return availabilityJson;
    }

    public void setAvailabilityJson(String availabilityJson) {
        this.availabilityJson = availabilityJson;
    }

    public String getProfileImageUrl() {
        return profileImageUrl;
    }

    public void setProfileImageUrl(String profileImageUrl) {
        this.profileImageUrl = profileImageUrl;
    }

    public String getBannerImageUrl() {
        return bannerImageUrl;
    }

    public void setBannerImageUrl(String bannerImageUrl) {
        this.bannerImageUrl = bannerImageUrl;
    }

    public String getProfileImagePath() {
        return profileImagePath;
    }

    public void setProfileImagePath(String profileImagePath) {
        this.profileImagePath = profileImagePath;
    }

    public String getProfilePhoto() {
        return profilePhoto;
    }

    public void setProfilePhoto(String profilePhoto) {
        this.profilePhoto = profilePhoto;
    }

    @JsonIgnore
    public List<Post> getPosts() {
        return posts;
    }

    public void setPosts(List<Post> posts) {
        this.posts = posts;
    }

    @JsonIgnore
    public List<Comment> getComments() {
        return comments;
    }

    public void setComments(List<Comment> comments) {
        this.comments = comments;
    }

    public String getResumeUrl() {
        return resumeUrl;
    }

    public void setResumeUrl(String resumeUrl) {
        this.resumeUrl = resumeUrl;
    }

```

```

    }

    public String getResumePublicId() {
        return resumePublicId;
    }

    public void setResumePublicId(String resumePublicId) {
        this.resumePublicId = resumePublicId;
    }

    public List<String> getSkills() {
        return skills;
    }

    public void setSkills(List<String> skills) {
        this.skills = skills;
    }

    @JsonIgnore
    public List<Project> getProjects() {
        return projects;
    }

    public void setProjects(List<Project> projects) {
        this.projects = projects;
    }

    public boolean isShowEmailOnProfile() {
        return showEmailOnProfile;
    }

    public void setShowEmailOnProfile(boolean showEmailOnProfile) {
        this.showEmailOnProfile = showEmailOnProfile;
    }

    public boolean isShowResumeOnProfile() {
        return showResumeOnProfile;
    }

    public void setShowResumeOnProfile(boolean showResumeOnProfile) {
        this.showResumeOnProfile = showResumeOnProfile;
    }

    // Convenience helpers
    @Transient
    public boolean isAuthor() {
        return "ROLE_AUTHOR".equals(this.role) || "ROLE_ADMIN".equals(this.role);
    }

    @Transient
    public boolean isActive() {
        return status == null || status.isBlank() || "ACTIVE".equalsIgnoreCase(status);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/model/UserFollow.java

```

package org.AE.alliededge.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "user_follow", uniqueConstraints = {
    @UniqueConstraint(columnNames = { "follower_id", "following_id" })
})
public class UserFollow {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)

```

```

private Long id;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "follower_id", nullable = false)
private User follower;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "following_id", nullable = false)
private User following;

@Column(name = "created_at", nullable = false)
private LocalDateTime createdAt;

@PrePersist
protected void onCreate() {
    createdAt = LocalDateTime.now();
}

// Getters and Setters

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public User getFollower() {
    return follower;
}

public void setFollower(User follower) {
    this.follower = follower;
}

public User getFollowing() {
    return following;
}

public void setFollowing(User following) {
    this.following = following;
}

public LocalDateTime getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDateTime createdAt) {
    this.createdAt = createdAt;
}
}

```

■ backend/src/main/java/org/AE/alliededge/repository/AnnouncementRepository.java

```

package org.AE.alliededge.repository;

import org.AE.alliededge.model.Announcement;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
import java.util.Optional;

public interface AnnouncementRepository extends JpaRepository<Announcement, Long> {
    List<Announcement> findByVisibleTrueOrderByCreatedAtDesc();

    List<Announcement> findAllByOrderByCreatedAtDesc();

    Optional<Announcement> findByIsWelcomeAnnouncementTrue();
}

```

■ backend/src/main/java/org/AE/alliededge/repository/ChatMessageRepository.java

```
package org.AE.alliededge.repository;

import org.AE.alliededge.model.ChatMessage;
import org.AE.alliededge.model.ChatRoom;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.time.LocalDateTime;
import java.util.List;

@Repository
public interface ChatMessageRepository extends JpaRepository<ChatMessage, Long> {

    /**
     * Find all messages in a chat room, ordered by timestamp
     */
    List<ChatMessage> findByChatRoomOrderByTimestampAsc(ChatRoom chatRoom);

    /**
     * Find the latest message in a chat room for dashboard preview
     */
    ChatMessage findFirstByChatRoomOrderByTimestampDesc(ChatRoom chatRoom);

    /**
     * Newest-first page.
     */
    List<ChatMessage> findByChatRoomOrderByTimestampDesc(ChatRoom chatRoom, Pageable pageable);

    /**
     * Newest-first page of messages strictly older than the cursor.
     */
    List<ChatMessage> findByChatRoomAndTimestampBeforeOrderByTimestampDesc(ChatRoom chatRoom, LocalDateTime beforeTimestamp);
}
```

■ backend/src/main/java/org/AE/alliededge/repository/ChatRoomRepository.java

```
package org.AE.alliededge.repository;

import org.AE.alliededge.model.ChatRoom;
import org.AE.alliededge.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface ChatRoomRepository extends JpaRepository<ChatRoom, Long> {

    /**
     * Find a chat room between two users (regardless of order)
     */
    @Query("SELECT cr FROM ChatRoom cr WHERE " +
        "(cr.user1 = :user1 AND cr.user2 = :user2) OR " +
        "(cr.user1 = :user2 AND cr.user2 = :user1)")
    Optional<ChatRoom> findByUsers(@Param("user1") User user1, @Param("user2") User user2);

    /**
     * Find all chat rooms for a user (where user is either user1 or user2)
     */
    @Query("SELECT cr FROM ChatRoom cr WHERE cr.user1 = :user OR cr.user2 = :user ORDER BY cr.id DESC")
    List<ChatRoom> findChatRoomsForUser(@Param("user") User user);
}
```

```
}
```

■ backend/src/main/java/org/AE/alliededge/repository/CommentLikeRepository.java

```
package org.AE.alliededge.repository;

import org.AE.alliededge.model.Comment;
import org.AE.alliededge.model.CommentLike;
import org.AE.alliededge.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CommentLikeRepository extends JpaRepository<CommentLike, Long> {
    boolean existsByCommentAndUser(Comment comment, User user);
    long countByComment(Comment comment);
    void deleteByCommentAndUser(Comment comment, User user);
    void deleteByComment(Comment comment);
}
```

■ backend/src/main/java/org/AE/alliededge/repository/CommentReplyLikeRepository.java

```
package org.AE.alliededge.repository;

import org.AE.alliededge.model.CommentReply;
import org.AE.alliededge.model.CommentReplyLike;
import org.AE.alliededge.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CommentReplyLikeRepository extends JpaRepository<CommentReplyLike, Long> {
    boolean existsByReplyAndUser(CommentReply reply, User user);

    long countByReply(CommentReply reply);

    void deleteByReplyAndUser(CommentReply reply, User user);

    void deleteByReply(CommentReply reply);
}
```

■ backend/src/main/java/org/AE/alliededge/repository/CommentReplyRepository.java

```
package org.AE.alliededge.repository;

import org.AE.alliededge.model.CommentReply;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface CommentReplyRepository extends JpaRepository<CommentReply, Long> {
    List<CommentReply> findByIdOrderByCreatedAtAsc(Long commentId);
}
```

■ backend/src/main/java/org/AE/alliededge/repository/CommentRepository.java

```
package org.AE.alliededge.repository;

import org.AE.alliededge.model.Comment;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.Collection;
import java.util.List;

public interface CommentRepository extends JpaRepository<Comment, Long> {
```

```

    long countByPostId(Long postId);

    /**
     * Batch comment counts for a set of posts.
     * Returns rows: [postId, count].
     */
    @Query("select c.post.id, count(c) from Comment c where c.post.id in :postIds group by c.post.id")
    List<Object[]> countByPostIds(@Param("postIds") Collection<Long> postIds);

    @Query("select count(c) from Comment c")
    long countAllComments();
}

```

■ backend/src/main/java/org/AE/alliededge/repository/PostLikeRepository.java

```

package org.AE.alliededge.repository;

import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.PostLike;
import org.AE.alliededge.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

public interface PostLikeRepository extends JpaRepository<PostLike, Long> {
    boolean existsByPostAndUser(Post post, User user);
    long countByPost(Post post);

    @Query("select count(pl) from PostLike pl")
    long countAllLikes();

    void deleteByPostAndUser(Post post, User user);
    void deleteByPost(Post post);
}

```

■ backend/src/main/java/org/AE/alliededge/repository/PostRepository.java

```

package org.AE.alliededge.repository;

import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.User;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface PostRepository extends JpaRepository<Post, Long> {

    Page<Post> findByTitleContainingIgnoreCase(String keyword, Pageable pageable);

    // Fetch all posts created by the given user (used for admin stats on profile page)
    List<Post> findByUser(User user);

    // Author dashboard: paginated view of posts by author
    Page<Post> findByUser(User user, Pageable pageable);

    // Convenience: lookup by author id directly
    Page<Post> findById(Long authorId, Pageable pageable);

    long countByUser(User user);

    @Query("SELECT p FROM Post p " +
        "WHERE (:keyword IS NULL OR :keyword = ' ' " +
        "OR LOWER(p.title) LIKE LOWER(CONCAT('%', :keyword, '%')) " +

```

```

        "OR LOWER(p.content) LIKE LOWER(CONCAT('%', :keyword, '%')) " +
        "OR LOWER(p.authorName) LIKE LOWER(CONCAT('%', :keyword, '%'))))"
    Pageable pageable);

    @Query("SELECT p FROM Post p " +
        "WHERE p.user = :author " +
        "AND (:keyword IS NULL OR :keyword = ' ' " +
        "OR LOWER(p.title) LIKE LOWER(CONCAT('%', :keyword, '%')) " +
        "OR LOWER(p.content) LIKE LOWER(CONCAT('%', :keyword, '%')) " +
        "OR LOWER(p.authorName) LIKE LOWER(CONCAT('%', :keyword, '%'))))"
    Pageable pageable); searchPostsByAuthor(@Param("author") User author,
        @Param("keyword") String keyword,
        Pageable pageable);

    // Variant using author id directly (optional, for simpler service wiring)
    @Query("SELECT p FROM Post p WHERE p.user.id = :authorId AND " +
        "(:keyword IS NULL OR :keyword = ' ' " +
        "OR LOWER(p.title) LIKE LOWER(CONCAT('%', :keyword, '%')) " +
        "OR LOWER(p.content) LIKE LOWER(CONCAT('%', :keyword, '%'))))"
    Pageable pageable); searchByAuthorId(@Param("authorId") Long authorId,
        @Param("keyword") String keyword,
        Pageable pageable);

    List<Post> findByUserOrderByCreatedAtDesc(User user);
}

```

■ backend/src/main/java/org/AE/alliededge/repository/PostViewRepository.java

```

package org.AE.alliededge.repository;

import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.PostView;
import org.AE.alliededge.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PostViewRepository extends JpaRepository<PostView, Long> {
    boolean existsByPostAndUser(Post post, User user);
    long countByPost(Post post);
    void deleteByPost(Post post);
}

```

■ backend/src/main/java/org/AE/alliededge/repository/ProjectRepository.java

```

package org.AE.alliededge.repository;

import org.AE.alliededge.model.Project;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface ProjectRepository extends JpaRepository<Project, Long> {

    List<Project> findByIdOrderByCreatedAtDesc(Long userId);

    List<Project> findById(Long userId);
}

```

■ backend/src/main/java/org/AE/alliededge/repository/UserFollowRepository.java

```

package org.AE.alliededge.repository;

import org.AE.alliededge.model.User;
import org.AE.alliededge.model.UserFollow;
import org.springframework.data.jpa.repository.JpaRepository;

```



```

import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Repository
public interface UserFollowRepository extends JpaRepository<UserFollow, Long> {

    /**
     * Check if follower is following the target user
     */
    boolean existsByFollowerAndFollowing(User follower, User following);

    /**
     * Delete follow relationship
     */
    @Transactional
    void deleteByFollowerAndFollowing(User follower, User following);

    /**
     * Count how many users the given user is following
     */
    long countByFollower(User user);

    /**
     * Count how many followers the given user has
     */
    long countByFollowing(User user);

    /**
     * Get list of users that the given user is following
     */
    List<UserFollow> findByFollower(User user);

    /**
     * Get list of followers for the given user
     */
    List<UserFollow> findByFollowing(User user);
}

```

■ backend/src/main/java/org/AE/alliededge/repository/UserRepository.java

```

package org.AE.alliededge.repository;

import org.AE.alliededge.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    /**
     * Finds a user by their username.
     *
     * @param username the username to search for
     * @return an Optional containing the User if found, or empty if not found
     */
    Optional<User> findByUsername(String username);

    /**
     * Check if a username is already taken.
     *
     * @param username the username to check
     * @return true if the username exists, false otherwise
     */
    boolean existsByUsername(String username);
}

```

```

/**
 * Canonical lookup by email (used for OAuth logins and identity).
 *
 * @param email the email to search for
 * @return an Optional containing the User if found, or empty if not found
 */
Optional<User> findByEmail(String email);

/**
 * Check if an email is already in use.
 *
 * @param email the email to check
 * @return true if the email exists, false otherwise
 */
boolean existsByEmail(String email);

/**
 * Finds users by their role.
 *
 * @param role the role to search for
 * @return a List of Users with the given role
 */
List<User> findByRole(String role);

/**
 * Count users by their role.
 *
 * @param role the role to count users for
 * @return the number of users with the given role
 */
long countByRole(String role);

/**
 * Finds users by their role and status.
 *
 * @param role the role to search for
 * @param status the status to search for
 * @return a List of Users with the given role and status
 */
List<User> findByRoleAndStatus(String role, String status);

/**
 * Search users by username or display name (case-insensitive, partial match).
 */
List<User> findByUsernameContainingIgnoreCaseOrDisplayNameContainingIgnoreCase(String username, String displayName);

/**
 * Search users by username, email, or display name (case-insensitive, partial match).
 * Used by admin dashboard for comprehensive user search.
 */
@org.springframework.data.jpa.repository.Query("SELECT u FROM User u WHERE " +
    "LOWER(u.username) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
    "LOWER(u.email) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
    "LOWER(u.displayName) LIKE LOWER(CONCAT('%', :keyword, '%'))")
List<User> searchUsers(@org.springframework.data.repository.query.Param("keyword") String keyword);

@org.springframework.data.jpa.repository.Query("SELECT u FROM User u ORDER BY u.id DESC")
org.springframework.data.domain.Page<User> findAllUsersPaged(org.springframework.data.domain.Pageable pageable);

@org.springframework.data.jpa.repository.Query("SELECT u FROM User u WHERE " +
    "LOWER(u.username) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
    "LOWER(u.email) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
    "LOWER(u.displayName) LIKE LOWER(CONCAT('%', :keyword, '%'))")
org.springframework.data.domain.Page<User> searchUsersPaged(
    @org.springframework.data.repository.query.Param("keyword") String keyword,
    org.springframework.data.domain.Pageable pageable);

/**
 * Search users by username or display name (case-insensitive, partial match), excluding a specific user id.
 */

```

```

        List<User> findByIdNotAndUsernameContainingIgnoreCaseOrIdNotAndDisplayNameContainingIgnoreCase(
            Long excludeId,
            String username,
            Long excludeId2,
            String displayName);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/AnnouncementService.java

```

package org.AE.alliededge.service;

import org.AE.alliededge.model.Announcement;
import org.AE.alliededge.repository.AnnouncementRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

@Service
public class AnnouncementService {
    @Autowired
    private AnnouncementRepository announcementRepository;

    @Autowired
    private CloudinaryService cloudinaryService;

    public List<Announcement> findAllVisible() {
        return announcementRepository.findByVisibleTrueOrderByCreatedAtDesc()
            .stream()
            .filter(a -> a != null && a.getCreatedAt() != null)
            .toList();
    }

    public List<Announcement> findAll() {
        return announcementRepository.findAllByOrderByCreatedAtDesc()
            .stream()
            .filter(a -> a != null && a.getCreatedAt() != null)
            .toList();
    }

    public Announcement create(Announcement a) {
        a.setCreatedAt(java.time.LocalDateTime.now());

        // If this is being set as welcome announcement, clear any existing welcome
        // announcement
        if (a.isWelcomeAnnouncement()) {
            announcementRepository.findByIsWelcomeAnnouncementTrue().ifPresent(existing -> {
                existing.setWelcomeAnnouncement(false);
                announcementRepository.save(existing);
            });
        }

        return announcementRepository.save(a);
    }

    // Overloaded create method for convenience
    public Announcement create(String title, String message, String adminEmail) {
        Announcement announcement = new Announcement();
        announcement.setTitle(title);
        announcement.setMessage(message);
        announcement.setCreatedAt(java.time.LocalDateTime.now());
        return announcementRepository.save(announcement);
    }

    public void delete(Long id) {
        announcementRepository.deleteById(id);
    }

    public void toggleVisibility(Long id) {
        Optional<Announcement> opt = announcementRepository.findById(id);
    }
}

```

```

        if (opt.isPresent()) {
            Announcement a = opt.get();
            a.setVisible(!a.isVisible());
            announcementRepository.save(a);
        }
    }

    public void toggleWelcomeAnnouncement(Long id) {
        Optional<Announcement> opt = announcementRepository.findById(id);
        if (opt.isPresent()) {
            Announcement a = opt.get();
            boolean newWelcomeStatus = !a.isWelcomeAnnouncement();

            // If setting as welcome, clear any existing welcome announcement
            if (newWelcomeStatus) {
                announcementRepository.findByIsWelcomeAnnouncementTrue().ifPresent(existing -> {
                    if (!existing.getId().equals(id)) {
                        existing.setWelcomeAnnouncement(false);
                        announcementRepository.save(existing);
                    }
                });
            }

            a.setWelcomeAnnouncement(newWelcomeStatus);
            announcementRepository.save(a);
        }
    }

    public Optional<Announcement> getWelcomeAnnouncement() {
        return announcementRepository.findByIsWelcomeAnnouncementTrue();
    }

    public Optional<Announcement> findById(Long id) {
        return announcementRepository.findById(id);
    }

    /**
     * Update an existing announcement.
     *
     * Contract:
     * - Does NOT change createdAt.
     * - If isWelcomeAnnouncement is set to true, clears any other welcome announcement.
     * - Supports optional video and image replacement/removal.
     */
    public Announcement update(Long id, AnnouncementUpdateRequest req) {
        Announcement existing = announcementRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Announcement not found"));

        if (req.getTitle() != null) existing.setTitle(req.getTitle());
        if (req.getMessage() != null) existing.setMessage(req.getMessage());
        if (req.getVisible() != null) existing.setVisible(req.getVisible());

        if (req.getIsWelcomeAnnouncement() != null) {
            boolean wantWelcome = Boolean.TRUE.equals(req.getIsWelcomeAnnouncement());
            if (wantWelcome) {
                announcementRepository.findByIsWelcomeAnnouncementTrue().ifPresent(other -> {
                    if (other.getId() != null && !other.getId().equals(existing.getId())) {
                        other.setWelcomeAnnouncement(false);
                        announcementRepository.save(other);
                    }
                });
            }
            existing.setWelcomeAnnouncement(wantWelcome);
        }

        // Video removal
        if (Boolean.TRUE.equals(req.getRemoveVideo()) && existing.getVideoPublicId() != null) {
            try {
                cloudinaryService.deleteVideo(existing.getVideoPublicId());
            } catch (Exception e) {
                throw new RuntimeException("Failed to delete existing video", e);
            }
        }
    }

```

```

    }
    existing.setVideoUrl(null);
    existing.setVideoPublicId(null);
}

// Video replacement
if (req.getVideoFile() != null && !req.getVideoFile().isEmpty()) {
    try {
        // If there was an existing video, delete it first.
        if (existing.getVideoPublicId() != null) {
            cloudinaryService.deleteVideo(existing.getVideoPublicId());
        }
        String videoUrl = cloudinaryService.uploadVideo(req.getVideoFile(), "announcements/videos");
        String publicId = cloudinaryService.extractPublicId(videoUrl);
        existing.setVideoUrl(videoUrl);
        existing.setVideoPublicId(publicId);
    } catch (Exception e) {
        throw new RuntimeException("Failed to upload new video", e);
    }
}

// Images removal (remove all)
if (Boolean.TRUE.equals(req.getRemoveImages()) && existing.getImageUrls() != null) {
    try {
        for (String url : existing.getImageUrls()) {
            if (url == null || url.isBlank()) continue;
            cloudinaryService.deleteMediaByUrl(url);
        }
    } catch (Exception e) {
        throw new RuntimeException("Failed to delete existing images", e);
    }
    existing.getImageUrls().clear();
}

// Images add
if (req.getImageFiles() != null && !req.getImageFiles().isEmpty()) {
    if (existing.getImageUrls() == null) {
        existing.setImageUrls(new java.util.ArrayList<>());
    }
    for (var imageFile : req.getImageFiles()) {
        if (imageFile == null || imageFile.isEmpty()) continue;
        String contentType = imageFile.getContentType();
        if (contentType == null || !contentType.toLowerCase().startsWith("image/")) {
            throw new RuntimeException("Only image files are allowed");
        }
        try {
            String imageUrl = cloudinaryService.uploadMedia(imageFile);
            existing.getImageUrls().add(imageUrl);
        } catch (Exception e) {
            throw new RuntimeException("Failed to upload announcement image", e);
        }
    }
}

return announcementRepository.save(existing);
}
}

```

■ backend/src/main/java/org/AE/alliededge/service/AnnouncementUpdateRequest.java

```

package org.AE.alliededge.service;

import java.util.List;
import org.springframework.web.multipart.MultipartFile;

/**
 * Contract for updating an Announcement from the REST API.
 * We keep this separate from the JPA entity to avoid accidentally resetting immutable fields
 * like createdAt.
 */

```

```

public class AnnouncementUpdateRequest {
    private String title;
    private String message;
    private Boolean visible;
    private Boolean isWelcomeAnnouncement;

    /** Optional: replace the existing video when provided. */
    private MultipartFile videoFile;

    /** Optional: set true to remove the existing video. */
    private Boolean removeVideo;

    /** Optional: add additional images. */
    private List<MultipartFile> imageFiles;

    /** Optional: set true to remove all existing images. */
    private Boolean removeImages;

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    public Boolean getVisible() {
        return visible;
    }

    public void setVisible(Boolean visible) {
        this.visible = visible;
    }

    public Boolean getIsWelcomeAnnouncement() {
        return isWelcomeAnnouncement;
    }

    public void setIsWelcomeAnnouncement(Boolean welcomeAnnouncement) {
        isWelcomeAnnouncement = welcomeAnnouncement;
    }

    public MultipartFile getVideoFile() {
        return videoFile;
    }

    public void setVideoFile(MultipartFile videoFile) {
        this.videoFile = videoFile;
    }

    public Boolean getRemoveVideo() {
        return removeVideo;
    }

    public void setRemoveVideo(Boolean removeVideo) {
        this.removeVideo = removeVideo;
    }

    public List<MultipartFile> getImageFiles() {
        return imageFiles;
    }

    public void setImageFiles(List<MultipartFile> imageFiles) {

```

```

        this.imageFiles = imageFiles;
    }

    public Boolean getRemoveImages() {
        return removeImages;
    }

    public void setRemoveImages(Boolean removeImages) {
        this.removeImages = removeImages;
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/ChatService.java

```

package org.AE.alliededge.service;

import org.AE.alliededge.model.ChatMessage;
import org.AE.alliededge.model.ChatRoom;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.ChatMessageRepository;
import org.AE.alliededge.repository.ChatRoomRepository;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.Collections;
import java.util.List;

@Service
public class ChatService {

    private final ChatRoomRepository chatRoomRepository;
    private final ChatMessageRepository chatMessageRepository;
    private final UserRepository userRepository;

    public ChatService(ChatRoomRepository chatRoomRepository,
                       ChatMessageRepository chatMessageRepository,
                       UserRepository userRepository) {
        this.chatRoomRepository = chatRoomRepository;
        this.chatMessageRepository = chatMessageRepository;
        this.userRepository = userRepository;
    }

    /**
     * Get or create a chat room between two users
     */
    @Transactional
    public ChatRoom getOrCreateChatRoom(User user1, User user2) {
        return chatRoomRepository.findByUsers(user1, user2)
            .orElseGet(() -> {
                ChatRoom newRoom = new ChatRoom();
                newRoom.setUser1(user1);
                newRoom.setUser2(user2);
                return chatRoomRepository.save(newRoom);
            });
    }

    /**
     * Send a message in a chat room
     */
    @Transactional
    public ChatMessage sendMessage(Long roomId, User sender, String content) {
        ChatRoom room = chatRoomRepository.findById(roomId)
            .orElseThrow(() -> new IllegalArgumentException("Chat room not found"));

        if (!room.isParticipant(sender)) {
            throw new IllegalArgumentException("User is not a participant in this chat room");
        }
    }
}

```

```

    }

    ChatMessage message = new ChatMessage();
    message.setChatRoom(room);
    message.setSender(sender);
    message.setContent(content);

    return chatMessageRepository.save(message);
}

/**
 * Get all messages in a chat room
 */
public List<ChatMessage> getMessages(Long roomId) {
    ChatRoom room = chatRoomRepository.findById(roomId)
        .orElseThrow(() -> new IllegalArgumentException("Chat room not found"));

    return chatMessageRepository.findByChatRoomOrderByTimestampAsc(room);
}

/**
 * Check if a user can access a chat room
 */
public boolean canUserAccessRoom(User user, Long roomId) {
    ChatRoom room = chatRoomRepository.findById(roomId).orElse(null);
    if (room == null) {
        return false;
    }
    return room.isParticipant(user);
}

/**
 * Get chat room by ID
 */
public ChatRoom getChatRoom(Long roomId) {
    return chatRoomRepository.findById(roomId).orElse(null);
}

/**
 * Get all chat rooms for a user
 */
public List<ChatRoom> getChatRoomsForUser(User user) {
    return chatRoomRepository.findChatRoomsForUser(user);
}

/**
 * Get the latest message in a chat room
 */
public ChatMessage getLatestMessage(ChatRoom room) {
    return chatMessageRepository.findFirstByChatRoomOrderByTimestampDesc(room);
}

/**
 * Get the other user in a chat room
 */
public User getOtherUser(ChatRoom room, User currentUser) {
    if (room.getUser1().getId().equals(currentUser.getId())) {
        return room.getUser2();
    } else {
        return room.getUser1();
    }
}

/**
 * Get newest messages in a chat room (page).
 */
public List<ChatMessage> getRecentMessages(Long roomId, int size) {
    ChatRoom room = chatRoomRepository.findById(roomId)
        .orElseThrow(() -> new IllegalArgumentException("Chat room not found"));

    int safeSize = Math.max(1, Math.min(size, 200));

```



```

        Pageable pageable = PageRequest.of(0, safeSize);
        return chatMessageRepository.findByChatRoomOrderByTimestampDesc(room, pageable);
    }

    /**
     * Get older messages strictly before the given timestamp (page), newest-first.
     */
    public List<ChatMessage> getOlderMessages(Long roomId, LocalDateTime before, int size) {
        if (before == null) {
            return Collections.emptyList();
        }

        ChatRoom room = chatRoomRepository.findById(roomId)
            .orElseThrow(() -> new IllegalArgumentException("Chat room not found"));

        int safeSize = Math.max(1, Math.min(size, 200));
        Pageable pageable = PageRequest.of(0, safeSize);
        return chatMessageRepository.findByChatRoomAndTimestampBeforeOrderByTimestampDesc(room, before, pageable);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/CloudinaryService.java

```

package org.AE.alliededge.service;

import org.springframework.web.multipart.MultipartFile;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.InputStream;
import java.util.Map;

public interface CloudinaryService {

    String uploadFile(MultipartFile file, String folder);

    String uploadMedia(MultipartFile file);

    Map<String, String> uploadRawFile(MultipartFile file, String folder);

    void deleteMediaByUrl(String url);

    default void deleteImageByPublicId(String publicId) {
        deleteByPublicId(publicId, "image");
    }

    default void deleteVideoByPublicId(String publicId) {
        deleteByPublicId(publicId, "video");
    }

    default void deleteRawFileByPublicId(String publicId) {
        deleteByPublicId(publicId, "raw");
    }

    void deleteByPublicId(String publicId, String resourceType);

    void streamResumeToResponse(String resumeUrl, String filename, HttpServletResponse response)
        throws IOException;

    String getRawUrlFromPublicId(String publicId);

    String extractPublicId(String url);

    // Stream resume PDF
    InputStream streamResume(String publicId) throws IOException;

    // Upload video to Cloudinary
    String uploadVideo(MultipartFile file, String folder) throws IOException;

    // Delete video from Cloudinary
    void deleteVideo(String publicId) throws IOException;
}

```

```
}
```

■ backend/src/main/java/org/AE/alliededge/service/CloudinaryServiceImpl.java

```
package org.AE.alliededge.service;

import com.cloudinary.Cloudinary;
import com.cloudinary.utils.ObjectUtils;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;
import java.io.InputStream;
import java.net.URL;
import java.util.HashMap;
import java.util.Map;
import java.util.logging.Logger;

@Service
@ConditionalOnProperty(name = "cloudinary.enabled", havingValue = "true")
public class CloudinaryServiceImpl implements CloudinaryService {

    private final Cloudinary cloudinary;
    private static final Logger LOGGER = Logger.getLogger(CloudinaryServiceImpl.class.getName());

    public CloudinaryServiceImpl(Cloudinary cloudinary) {
        this.cloudinary = cloudinary;
    }

    @Override
    public String uploadFile(MultipartFile file, String folder) {
        try {
            Map upload = cloudinary.uploader().upload(file.getBytes(),
                ObjectUtils.asMap("folder", folder, "resource_type", "auto"));
            return upload.get("secure_url").toString();
        } catch (Exception e) {
            throw new RuntimeException("Upload failed", e);
        }
    }

    @Override
    public String uploadMedia(MultipartFile file) {
        try {
            Map upload = cloudinary.uploader().upload(file.getBytes(),
                ObjectUtils.asMap("resource_type", "auto"));
            return upload.get("secure_url").toString();
        } catch (Exception e) {
            throw new RuntimeException("Media upload failed", e);
        }
    }

    @Override
    public Map<String, String> uploadRawFile(MultipartFile file, String folder) {
        java.io.File tempFile = null;
        try {
            String uuid = java.util.UUID.randomUUID().toString();
            String publicId = folder + "/" + uuid;

            tempFile = java.io.File.createTempFile("resume_upload_", ".pdf");
            try (java.io.InputStream inputStream = file.getInputStream();
                java.io.FileOutputStream fos = new java.io.FileOutputStream(tempFile)) {
                byte[] buffer = new byte[8192];
                int bytesRead;
                while ((bytesRead = inputStream.read(buffer)) != -1) {
                    fos.write(buffer, 0, bytesRead);
                }
            }
        }
    }
}
```

```

        Map upload = cloudinary.uploader().upload(tempFile,
            ObjectUtils.asMap(
                "resource_type", "raw",
                "folder", folder,
                "public_id", publicId,
                "use_filename", true,
                "unique_filename", true,
                "access_mode", "public",
                "format", "pdf"));

        String secureUrl = upload.get("secure_url").toString();
        String storedPublicId = upload.get("public_id").toString();

        Map<String, String> result = new HashMap<>();
        result.put("url", secureUrl);
        result.put("publicId", storedPublicId);
        result.put("resourceType", "raw");

        LOGGER.info("Uploaded PDF: " + secureUrl + " | publicId: " + storedPublicId);

        return result;
    } catch (Exception e) {
        LOGGER.severe("Raw file upload failed: " + e.getMessage());
        throw new RuntimeException("Raw file upload failed: " + e.getMessage(), e);
    } finally {
        if (tempFile != null && tempFile.exists()) {
            tempFile.delete();
        }
    }
}

@Override
public String getRawUrlFromPublicId(String publicId) {
    if (publicId == null || publicId.isEmpty()) {
        return null;
    }

    try {
        String cloudName = cloudinary.config.cloudName;
        return "https://res.cloudinary.com/" + cloudName + "/raw/upload/" + publicId + ".pdf";
    } catch (Exception e) {
        LOGGER.warning("Failed to build raw URL for publicId: " + publicId);
        return null;
    }
}

@Override
public void streamResumeToResponse(String resumeUrl, String filename, HttpServletResponse response)
    throws IOException {
    if (resumeUrl == null || resumeUrl.isEmpty()) {
        throw new IllegalArgumentException("Resume URL cannot be null or empty");
    }

    try {
        java.net.URL url = new java.net.URL(resumeUrl);
        java.net.HttpURLConnection connection = (java.net.HttpURLConnection) url.openConnection();
        connection.setRequestMethod("GET");
        connection.setConnectTimeout(10000);
        connection.setReadTimeout(30000);

        response.setContentType("application/pdf");
        response.setHeader("Content-Disposition", "attachment; filename=\"" + filename + "\"");

        int contentLength = connection.getContentLength();
        if (contentLength > 0) {
            response.setContentLength(contentLength);
        }

        try (java.io.InputStream in = new java.io.BufferedInputStream(connection.getInputStream());
            java.io.OutputStream out = new java.io.BufferedOutputStream(response.getOutputStream())) {
            byte[] buffer = new byte[8192];

```

```

        int bytesRead;
        while ((bytesRead = in.read(buffer)) != -1) {
            out.write(buffer, 0, bytesRead);
        }
        out.flush();
    }

    LOGGER.info("Streamed resume: " + filename);
} catch (IOException e) {
    LOGGER.severe("Failed to stream resume: " + e.getMessage());
    throw e;
}
}

@Override
public InputStream streamResume(String publicId) throws IOException {
    if (publicId == null || publicId.isEmpty()) {
        throw new IllegalArgumentException("Public ID cannot be null or empty");
    }

    try {
        String urlString = cloudinary.url()
            .resourceType("raw")
            .publicId(publicId)
            .generate();

        return new URL(urlString).openStream();
    } catch (Exception e) {
        throw new IOException("Failed to stream resume: " + e.getMessage(), e);
    }
}

public String extractPublicId(String url) {
    try {
        String noQuery = url.split("\\?")[0];
        String[] parts = noQuery.split("/");

        String fileName = parts[parts.length - 1];
        fileName = fileName.substring(0, fileName.lastIndexOf("."));

        String folder = parts[parts.length - 2];

        return folder + "/" + fileName;
    } catch (Exception e) {
        LOGGER.warning("Failed to extract publicId from url: " + url);
        return null;
    }
}

@Override
public void deleteMediaByUrl(String url) {
    String publicId = extractPublicId(url);

    if (url.contains("/video/")) {
        deleteVideoByPublicId(publicId);
    } else {
        deleteImageByPublicId(publicId);
    }
}

@Override
public void deleteByPublicId(String publicId, String resourceType) {
    if (publicId == null)
        return;

    try {
        cloudinary.uploader().destroy(publicId,
            ObjectUtils.asMap("resource_type", resourceType, "invalidate", true));

        LOGGER.info("Deleted " + resourceType + " with publicId=" + publicId);
    } catch (Exception e) {

```

```

        LOGGER.warning("Cloudinary delete failed for " + publicId + ": " + e.getMessage());
    }
}

@Override
public String uploadVideo(MultipartFile file, String folder) throws IOException {
    if (file == null || file.isEmpty()) {
        throw new IllegalArgumentException("Video file cannot be null or empty");
    }

    try (InputStream inputStream = file.getInputStream()) {
        Map<String, Object> uploadParams = ObjectUtils.asMap(
            "resource_type", "video",
            "folder", folder,
            "chunk_size", 6000000 // 6MB chunks
        );

        @SuppressWarnings("unchecked")
        Map<String, Object> uploadResult =
            cloudinary.uploader().uploadLarge(inputStream, uploadParams);

        String secureUrl = (String) uploadResult.get("secure_url");

        LOGGER.info("Uploaded video successfully: " + secureUrl);

        return secureUrl;
    } catch (Exception e) {
        LOGGER.severe("Video upload failed: " + e.getMessage());
        throw new IOException("Failed to upload video", e);
    }
}

@Override
public void deleteVideo(String publicId) throws IOException {
    if (publicId == null || publicId.isEmpty()) {
        return; // Nothing to delete
    }

    try {
        Map<String, Object> deleteParams = new HashMap<>();
        deleteParams.put("resource_type", "video");

        cloudinary.uploader().destroy(publicId, deleteParams);
        LOGGER.info("Deleted video with publicId=" + publicId);
    } catch (Exception e) {
        LOGGER.warning("Failed to delete video from Cloudinary: " + e.getMessage());
        throw new IOException("Failed to delete video from Cloudinary: " + e.getMessage(), e);
    }
}
}

```

■ backend/src/main/java/org/AE/alliededge/service/CommentLikeService.java

```

package org.AE.alliededge.service;

import jakarta.transaction.Transactional;
import org.AE.alliededge.exception.ResourceNotFoundException;
import org.AE.alliededge.model.Comment;
import org.AE.alliededge.model.CommentLike;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.CommentLikeRepository;
import org.AE.alliededge.repository.CommentRepository;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;

```

```

@Service
public class CommentLikeService {

    private final CommentRepository commentRepository;
    private final CommentLikeRepository commentLikeRepository;
    private final UserRepository userRepository;

    public CommentLikeService(CommentRepository commentRepository,
                              CommentLikeRepository commentLikeRepository,
                              UserRepository userRepository) {
        this.commentRepository = commentRepository;
        this.commentLikeRepository = commentLikeRepository;
        this.userRepository = userRepository;
    }

    /** Toggle like/unlike on a comment. Returns true if the comment is now liked by the user. */
    @Transactional
    public boolean toggleLike(Long commentId, String userEmail) {
        Comment comment = commentRepository.findById(commentId)
            .orElseThrow(() -> new ResourceNotFoundException("Comment not found: " + commentId));

        User user = userRepository.findByEmail(userEmail)
            .orElseThrow(() -> new ResourceNotFoundException("User not found: " + userEmail));

        // Prevent liking your own comment (keeps UX consistent with post likes)
        if (comment.getUser() != null && comment.getUser().getEmail() != null
            && comment.getUser().getEmail().equalsIgnoreCase(userEmail)) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "You cannot like your own comment.");
        }

        boolean alreadyLiked = commentLikeRepository.existsByCommentAndUser(comment, user);
        if (alreadyLiked) {
            commentLikeRepository.deleteByCommentAndUser(comment, user);
        } else {
            CommentLike like = new CommentLike();
            like.setComment(comment);
            like.setUser(user);
            commentLikeRepository.save(like);
        }

        long count = commentLikeRepository.countByComment(comment);
        comment.setLikes((int) count);
        commentRepository.save(comment);

        return !alreadyLiked;
    }

    public boolean hasUserLiked(Comment comment, String userEmail) {
        if (userEmail == null) return false;
        User user = userRepository.findByEmail(userEmail).orElse(null);
        return user != null && commentLikeRepository.existsByCommentAndUser(comment, user);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/CommentReplyLikeService.java

```

package org.AE.alliededge.service;

import jakarta.transaction.Transactional;
import org.AE.alliededge.exception.ResourceNotFoundException;
import org.AE.alliededge.model.CommentReply;
import org.AE.alliededge.model.CommentReplyLike;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.CommentReplyLikeRepository;
import org.AE.alliededge.repository.CommentReplyRepository;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;

```

```

@Service
public class CommentReplyLikeService {

    private final CommentReplyRepository replyRepository;
    private final CommentReplyLikeRepository replyLikeRepository;
    private final UserRepository userRepository;

    public CommentReplyLikeService(CommentReplyRepository replyRepository,
                                   CommentReplyLikeRepository replyLikeRepository,
                                   UserRepository userRepository) {
        this.replyRepository = replyRepository;
        this.replyLikeRepository = replyLikeRepository;
        this.userRepository = userRepository;
    }

    /** Toggle like/unlike on a reply. Returns true if the reply is now liked by the user. */
    @Transactional
    public boolean toggleLike(Long replyId, String userEmail) {
        CommentReply reply = replyRepository.findById(replyId)
            .orElseThrow(() -> new ResourceNotFoundException("Reply not found: " + replyId));

        User user = userRepository.findByEmail(userEmail)
            .orElseThrow(() -> new ResourceNotFoundException("User not found: " + userEmail));

        // Prevent liking your own reply
        if (reply.getUser() != null && reply.getUser().getEmail() != null
            && reply.getUser().getEmail().equalsIgnoreCase(userEmail)) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "You cannot like your own reply.");
        }

        boolean alreadyLiked = replyLikeRepository.existsByReplyAndUser(reply, user);
        if (alreadyLiked) {
            replyLikeRepository.deleteByReplyAndUser(reply, user);
        } else {
            CommentReplyLike like = new CommentReplyLike();
            like.setReply(reply);
            like.setUser(user);
            replyLikeRepository.save(like);
        }

        long count = replyLikeRepository.countByReply(reply);
        reply.setLikes((int) count);
        replyRepository.save(reply);

        return !alreadyLiked;
    }

    public boolean hasUserLiked(CommentReply reply, String userEmail) {
        if (userEmail == null) return false;
        User user = userRepository.findByEmail(userEmail).orElse(null);
        return user != null && replyLikeRepository.existsByReplyAndUser(reply, user);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/CommentReplyService.java

```

package org.AE.alliededge.service;

import jakarta.transaction.Transactional;
import org.AE.alliededge.dto.CommentDto;
import org.AE.alliededge.exception.ResourceNotFoundException;
import org.AE.alliededge.model.Comment;
import org.AE.alliededge.model.CommentReply;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.CommentReplyRepository;
import org.AE.alliededge.repository.CommentRepository;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.stereotype.Service;

@Service

```

```

public class CommentReplyService {

    private final CommentReplyRepository replyRepository;
    private final CommentRepository commentRepository;
    private final UserRepository userRepository;

    public CommentReplyService(CommentReplyRepository replyRepository,
                               CommentRepository commentRepository,
                               UserRepository userRepository) {
        this.replyRepository = replyRepository;
        this.commentRepository = commentRepository;
        this.userRepository = userRepository;
    }

    public CommentReply getById(Long id) {
        return replyRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Reply not found with ID: " + id));
    }

    @Transactional
    public CommentReply addReply(Long commentId, String userEmail, CommentDto dto) {
        Comment comment = commentRepository.findById(commentId)
            .orElseThrow(() -> new ResourceNotFoundException("Comment not found with ID: " + commentId));

        User user = userRepository.findByEmail(userEmail)
            .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + userEmail));

        CommentReply reply = new CommentReply();
        reply.setComment(comment);
        reply.setUser(user);
        reply.setContent(dto.getContent());
        reply.setCreatedAt(java.time.LocalDateTime.now());
        return replyRepository.save(reply);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/CommentService.java

```

package org.AE.alliededge.service;

import jakarta.transaction.Transactional;
import org.AE.alliededge.dto.CommentDto;
import org.AE.alliededge.exception.ResourceNotFoundException;
import org.AE.alliededge.model.Comment;
import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.CommentLikeRepository;
import org.AE.alliededge.repository.CommentRepository;
import org.AE.alliededge.repository.PostRepository;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;

@Service
public class CommentService {

    private final CommentRepository commentRepository;
    private final PostRepository postRepository;
    private final UserRepository userRepository;
    private final CommentLikeRepository commentLikeRepository;

    public CommentService(CommentRepository commentRepository,
                           PostRepository postRepository,
                           UserRepository userRepository,
                           CommentLikeRepository commentLikeRepository) {
        this.commentRepository = commentRepository;
        this.postRepository = postRepository;
        this.userRepository = userRepository;
    }
}

```



```

        this.commentLikeRepository = commentLikeRepository;
    }

    public Comment getById(Long id) {
        return commentRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Comment not found with ID: " + id));
    }

    @Transactional
    public Comment saveComment(Long postId, String userEmail, CommentDto commentDto) {
        Post post = postRepository.findById(postId)
            .orElseThrow(() -> new ResourceNotFoundException("Post not found with ID: " + postId));

        User user = userRepository.findByEmail(userEmail)
            .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + userEmail));

        Comment comment = new Comment();
        comment.setContent(commentDto.getContent());
        comment.setCreatedAt(java.time.LocalDateTime.now());
        comment.setPost(post);
        comment.setUser(user);
        return commentRepository.save(comment);
    }

    @Transactional
    public Long deleteCommentByAdmin(Long commentId) {
        Comment comment = getById(commentId);

        Post post = comment.getPost();
        Long postId = (post != null ? post.getId() : null);

        // Clean up likes explicitly (DB cascade also handles it, but keep consistent with PostService)
        commentLikeRepository.deleteByComment(comment);

        commentRepository.delete(comment);

        return postId;
    }

    // Update existing comment content with security validation
    @Transactional
    public Comment updateComment(Long id, String newContent, String userEmail) {
        Comment c = commentRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Comment not found"));

        boolean owner = c.getUser() != null && c.getUser().getEmail() != null &&
            c.getUser().getEmail().equalsIgnoreCase(userEmail);
        boolean admin = userRepository.findByEmail(userEmail)
            .map(User::isAdmin)
            .orElse(false);

        if (!owner && !admin) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN);
        }

        c.setContent(newContent);
        return commentRepository.save(c);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/ContentLinkifier.java

```

package org.AE.alliededge.service;

import java.util.ArrayList;
import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class ContentLinkifier {

```

```

// Detect full anchor tags so we don't modify their contents
private static final Pattern ANCHOR_PATTERN = Pattern.compile("<a\\b[^>]*>.*?</a>", Pattern.CASE_INSENSITIVE);
// URL pattern (basic) not preceded by quote or equals (to avoid matching inside attributes)
private static final Pattern URL_PATTERN = Pattern.compile("(?<!['=])(https?:/[\\w./?%&=+ #-]+)");

public static String linkify(String original) {
    if (original == null || original.isBlank()) return original;

    // Find existing anchor tag ranges to skip
    List<int[]> anchorRanges = new ArrayList<>();
    Matcher anchorMatcher = ANCHOR_PATTERN.matcher(original);
    while (anchorMatcher.find()) {
        anchorRanges.add(new int[]{anchorMatcher.start(), anchorMatcher.end()});
    }

    StringBuilder sb = new StringBuilder();
    Matcher urlMatcher = URL_PATTERN.matcher(original);
    int lastIndex = 0;

    while (urlMatcher.find()) {
        int start = urlMatcher.start();
        int end = urlMatcher.end();
        if (isInsideRanges(start, anchorRanges)) {
            // leave unchanged
            continue;
        }
        String url = urlMatcher.group(1);
        // Append text before URL (from lastIndex up to start) but only up to current match start if not already appended
        if (start >= lastIndex) {
            sb.append(original, lastIndex, start);
            // Append anchor
            sb.append("<a href=\"").append(url).append("\" target=\"_blank\" rel=\"noopener noreferrer\">").append(url);
            lastIndex = end;
        }
    }
    // Append remaining tail
    if (lastIndex < original.length()) {
        sb.append(original.substring(lastIndex));
    }

    // If we never appended anything (no changes), return original
    return sb.isEmpty() ? original : sb.toString();
}

private static boolean isInsideRanges(int pos, List<int[]> ranges) {
    for (int[] r : ranges) {
        if (pos >= r[0] & pos < r[1]) return true;
    }
    return false;
}
}

```

■ backend/src/main/java/org/AE/alliededge/service/CustomUserDetailsService.java

```

package org.AE.alliededge.service;

import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.Collection;
import java.util.Collections;
import java.util.List;

@Service

```

```

public class CustomUserDetailsService implements UserDetailsService {
    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found with username: " + username));

        String storedRole = user.getRole();
        // DB role values are stored like ROLE_AUTHOR / ROLE_ADMIN.
        // Avoid double-prefixing (ROLE_ROLE_AUTHOR), which breaks @PreAuthorize checks.
        String authority = (storedRole != null && storedRole.toUpperCase().startsWith("ROLE_"))
            ? storedRole.toUpperCase()
            : "ROLE_" + String.valueOf(storedRole).toUpperCase();

        List<GrantedAuthority> authorities = Collections.singletonList(new SimpleGrantedAuthority(authority));

        return new CustomUserDetails(user, authorities);
    }

    /**
     * Custom UserDetails implementation that exposes the underlying User entity
     * and the profileImageUrl field for use in Thymeleaf templates (e.g. navbar avatar).
     */
    public static class CustomUserDetails implements UserDetails {
        private final User user;
        private final List<GrantedAuthority> authorities;

        public CustomUserDetails(User user, List<GrantedAuthority> authorities) {
            this.user = user;
            this.authorities = authorities;
        }

        public User getUser() {
            return user;
        }

        public String getProfileImageUrl() {
            return user.getProfileImageUrl();
        }

        @Override
        public Collection<? extends GrantedAuthority> getAuthorities() {
            return authorities;
        }

        @Override
        public String getPassword() {
            return user.getPassword();
        }

        @Override
        public String getUsername() {
            return user.getUsername();
        }

        @Override
        public boolean isAccountNonExpired() {
            return true;
        }

        @Override
        public boolean isAccountNonLocked() {
            return true;
        }

        @Override

```

```

        public boolean isCredentialsNonExpired() {
            return true;
        }

        @Override
        public boolean isEnabled() {
            return !user.isBanned();
        }
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/JwtService.java

```

package org.AE.alliededge.service;

import com.nimbusds.jose.JWSAlgorithm;
import com.nimbusds.jose.JWSHeader;
import com.nimbusds.jose.crypto.MACSigner;
import com.nimbusds.jwt.JWTClaimsSet;
import com.nimbusds.jwt.SignedJWT;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.time.Instant;
import java.util.Date;
import java.util.Map;

@Service
public class JwtService {
    private final byte[] secret;
    private final long expirationSeconds;

    public JwtService(
        @Value("${app.jwt.secret}") String secret,
        @Value("${app.jwt.expirationSeconds:86400}") long expirationSeconds
    ) {
        this.secret = secret.getBytes(java.nio.charset.StandardCharsets.UTF_8);
        this.expirationSeconds = expirationSeconds;
    }

    public String sign(String subject, Map<String, Object> claims) {
        try {
            Instant now = Instant.now();
            Instant exp = now.plusSeconds(expirationSeconds);
            JWTClaimsSet.Builder cb = new JWTClaimsSet.Builder()
                .subject(subject)
                .issueTime(Date.from(now))
                .expirationTime(Date.from(exp));
            if (claims != null) {
                claims.forEach(cb::claim);
            }
            JWTClaimsSet claimSet = cb.build();
            SignedJWT signedJWT = new SignedJWT(new JWSHeader(JWSAlgorithm.HS256), claimSet);
            signedJWT.sign(new MACSigner(secret));
            return signedJWT.serialize();
        } catch (Exception e) {
            throw new RuntimeException("Failed to sign JWT", e);
        }
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/PostRealtimePublisher.java

```

package org.AE.alliededge.service;

import org.AE.alliededge.dto.PostStatsDto;
import org.AE.alliededge.model.Post;
import org.AE.alliededge.repository.CommentRepository;

```

```

import org.AE.alliededge.repository.PostRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.stereotype.Service;

import java.time.Instant;

@Service
public class PostRealtimePublisher {
    private static final Logger log = LoggerFactory.getLogger(PostRealtimePublisher.class);

    private final SimpMessagingTemplate messagingTemplate;
    private final PostRepository postRepository;
    private final CommentRepository commentRepository;

    public PostRealtimePublisher(SimpMessagingTemplate messagingTemplate,
                                PostRepository postRepository,
                                CommentRepository commentRepository) {
        this.messagingTemplate = messagingTemplate;
        this.postRepository = postRepository;
        this.commentRepository = commentRepository;
    }

    /**
     * Publish the latest post counters to /topic/posts/{postId}.
     * Safe to call even if websocket is unused.
     */
    public void publishPostStats(Long postId) {
        try {
            Post post = postRepository.findById(postId).orElse(null);
            if (post == null) return;

            int likes = post.getLikes();
            int commentCount = (int) commentRepository.countByPostId(postId);

            PostStatsDto dto = new PostStatsDto(postId, likes, commentCount, Instant.now());
            messagingTemplate.convertAndSend("/topic/posts/" + postId, dto);
        } catch (Exception e) {
            // Never break the request because realtime publish failed.
            log.warn("Failed to publish post stats for postId={}", postId, e);
        }
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/PostService.java

```

package org.AE.alliededge.service;

import jakarta.transaction.Transactional;
import org.AE.alliededge.exception.ResourceNotFoundException;
import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.PostLike;
import org.AE.alliededge.model.PostView;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.PostLikeRepository;
import org.AE.alliededge.repository.PostRepository;
import org.AE.alliededge.repository.PostViewRepository;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;
import org.springframework.http.HttpStatus;
import org.springframework.data.domain.PageRequest;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

```

```

import java.util.logging.Logger;

@Service
public class PostService {

    private final PostRepository postRepository;
    private final UserRepository userRepository;
    private final PostLikeRepository postLikeRepository;
    private final PostViewRepository postViewRepository;
    private final CloudinaryService cloudinaryService;

    private static final Logger LOGGER = Logger.getLogger(PostService.class.getName());

    public PostService(PostRepository postRepository,
                       UserRepository userRepository,
                       PostLikeRepository postLikeRepository,
                       PostViewRepository postViewRepository,
                       CloudinaryService cloudinaryService) {

        this.postRepository = postRepository;
        this.userRepository = userRepository;
        this.postLikeRepository = postLikeRepository;
        this.postViewRepository = postViewRepository;
        this.cloudinaryService = cloudinaryService;
    }

    public Page<Post> findAllPosts(Pageable pageable) {
        return postRepository.findAll(pageable);
    }

    public List<Post> findAllPosts() {
        return postRepository.findAll(Sort.by(Sort.Direction.DESC, "createdAt"));
    }

    public Page<Post> searchByTitle(String keyword, Pageable pageable) {
        return postRepository.findByTitleContainingIgnoreCase(keyword, pageable);
    }

    public Optional<Post> findPostById(Long id) {
        return postRepository.findById(id);
    }

    /** -----
     * Create OR Update Post
     * ----- */
    @Transactional
    public Post savePost(Post postFromForm, String userEmail) {

        if (postFromForm.getContent() != null) {
            postFromForm.setContent(ContentLinkifier.linkify(postFromForm.getContent()));
        }

        /** CREATE NEW POST */
        if (postFromForm.getId() == null) {

            User author = userRepository.findByEmail(userEmail)
                .orElseThrow(() -> new RuntimeException("User not found: " + userEmail));

            postFromForm.setUser(author);
            postFromForm.setCreatedAt(LocalDateTime.now());

            String displayName = (author.getDisplayName() != null && !author.getDisplayName().isBlank())
                ? author.getDisplayName()
                : author.getUsername();

            postFromForm.setAuthorName(displayName);
            postFromForm.setAuthorIsAdmin(author.isAdmin());

            LOGGER.info("Creating new post with videoUrl=" + postFromForm.getVideoUrl());
            return postRepository.save(postFromForm);
        }
    }
}

```

```

/** UPDATE EXISTING POST */
Post existingPost = postRepository.findById(postFromForm.getId())
    .orElseThrow(() -> new ResourceNotFoundException("Post not found: " + postFromForm.getId()));

// Only overwrite fields that were actually provided.
// The React UI often updates content only, so preserve title if not provided.
if (postFromForm.getTitle() != null && !postFromForm.getTitle().isBlank()) {
    existingPost.setTitle(postFromForm.getTitle());
}

if (postFromForm.getContent() != null && !postFromForm.getContent().isBlank()) {
    existingPost.setContent(postFromForm.getContent());
}

// UPDATE IMAGES
if (postFromForm.getImageUrls() != null) {
    existingPost.setImageUrls(postFromForm.getImageUrls());
}

// --- VIDEO DELETE / UPDATE LOGIC ---
if (postFromForm.isDeleteVideo()) {
    // User requested deletion via checkbox on edit form
    String publicId = existingPost.getVideoPublicId();
    String url = existingPost.getVideoUrl();

    if (publicId != null && !publicId.isBlank()) {
        cloundinaryService.deleteVideoByPublicId(publicId);
    } else if (url != null && !url.isBlank()) {
        // Fallback: derive publicId from URL and delete as video
        String derivedPublicId = cloundinaryService.extractPublicId(url);
        if (derivedPublicId != null && !derivedPublicId.isBlank()) {
            cloundinaryService.deleteVideoByPublicId(derivedPublicId);
        }
    }

    existingPost.setVideoUrl(null);
    existingPost.setVideoPublicId(null);
} else {
    // If a new video URL was set (from controller upload), replace the previous one
    if (postFromForm.getVideoUrl() != null && !postFromForm.getVideoUrl().isBlank()) {
        String oldPublicId = existingPost.getVideoPublicId();
        String oldUrl = existingPost.getVideoUrl();

        if (oldPublicId != null && !oldPublicId.isBlank()) {
            cloundinaryService.deleteVideoByPublicId(oldPublicId);
        } else if (oldUrl != null && !oldUrl.isBlank()) {
            String derivedPublicId = cloundinaryService.extractPublicId(oldUrl);
            if (derivedPublicId != null && !derivedPublicId.isBlank()) {
                cloundinaryService.deleteVideoByPublicId(derivedPublicId);
            }
        }

        existingPost.setVideoUrl(postFromForm.getVideoUrl());
        // Optionally, update videoPublicId based on new URL
        String newPublicId = cloundinaryService.extractPublicId(postFromForm.getVideoUrl());
        existingPost.setVideoPublicId(newPublicId);
    }
}

LOGGER.info("Updating post id=" + existingPost.getId());
return postRepository.save(existingPost);
}

/** -----
 * DELETE POST (and everything in Cloundinary)
 * ----- */
@Transactional
public void deletePostById(Long id) {
    Post post = postRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Post not found: " + id));
    // Delete video if present

```

```

        deleteVideo(post);

        // Likes + Views
        postLikeRepository.deleteByPost(post);
        postViewRepository.deleteByPost(post);

        postRepository.delete(post);
    }

    /** -----
     * DELETE VIDEO FROM CLOUDINARY
     * ----- */
    private void deleteVideo(Post post) {
        if (post.getVideoUrl() == null || post.getVideoUrl().isBlank()) {
            return;
        }

        String publicId = cloudinaryService.extractPublicId(post.getVideoUrl());

        try {
            cloudinaryService.deleteVideoByPublicId(publicId);
            LOGGER.info("Deleted Cloudinary video: " + publicId);
        } catch (Exception e) {
            LOGGER.warning("Failed to delete video: " + e.getMessage());
        }

        post.setVideoUrl(null);
        postRepository.save(post);
    }

    /** -----
     * LIKE / UNLIKE
     * ----- */
    @Transactional
    public boolean toggleLike(Long postId, String userEmail) {

        Post post = postRepository.findById(postId)
            .orElseThrow(() -> new ResourceNotFoundException("Post not found: " + postId));

        User user = userRepository.findById(userEmail)
            .orElseThrow(() -> new ResourceNotFoundException("User not found: " + userEmail));

        if (post.getUser().getEmail().equalsIgnoreCase(userEmail)) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "You cannot like your own post.");
        }

        boolean alreadyLiked = postLikeRepository.existsByPostAndUser(post, user);
        if (alreadyLiked) {
            postLikeRepository.deleteByPostAndUser(post, user);
        } else {
            PostLike like = new PostLike();
            like.setPost(post);
            like.setUser(user);
            postLikeRepository.save(like);
        }

        long count = postLikeRepository.countByPost(post);
        post.setLikes((int) count);
        postRepository.save(post);

        return !alreadyLiked;
    }

    public boolean hasUserLiked(Post post, String userEmail) {
        if (userEmail == null) return false;
        User user = userRepository.findById(userEmail).orElse(null);
        return user != null && postLikeRepository.existsByPostAndUser(post, user);
    }

    /** -----
     * VIEWS

```



```

* ----- */
@Transactional
public void incrementViews(Long postId) {
    Post post = postRepository.findById(postId)
        .orElseThrow(() -> new ResourceNotFoundException("Post not found: " + postId));
    post.setViews(post.getViews() + 1);
    postRepository.save(post);
}

@Transactional
public Post recordView(Long postId, String userEmail) {

    Post post = postRepository.findById(postId)
        .orElseThrow(() -> new ResourceNotFoundException("Post not found: " + postId));

    User user = userRepository.findByEmail(userEmail)
        .orElseThrow(() -> new ResourceNotFoundException("User not found: " + userEmail));

    if (!postViewRepository.existsByPostAndUser(post, user)) {
        PostView pv = new PostView();
        pv.setPost(post);
        pv.setUser(user);
        pv.setViewedAt(LocalDateTime.now());
        postViewRepository.save(pv);

        long views = postViewRepository.countByPost(post);
        post.setViews((int) views);
        postRepository.save(post);
    }

    return post;
}

/** -----
 * SORTING + FILTERS
 * ----- */
public Sort resolveSort(String sortBy) {
    if (sortBy == null || sortBy.isEmpty() || sortBy.equals("newest"))
        return Sort.by(Sort.Direction.DESC, "createdAt");

    switch (sortBy) {
        case "oldest": return Sort.by(Sort.Direction.ASC, "createdAt");
        case "mostLiked": return Sort.by(Sort.Direction.DESC, "likes");
        case "mostViewed": return Sort.by(Sort.Direction.DESC, "views");
        default: return Sort.by(Sort.Direction.DESC, "createdAt");
    }
}

public Page<Post> getAllPosts(String keyword, String sortBy, Pageable pageable) {
    Sort sort = resolveSort(sortBy);
    Pageable sorted = PageRequest.of(pageable.getPageNumber(), pageable.getPageSize(), sort);

    String kw = (keyword == null) ? null : keyword.trim();
    if (kw != null && !kw.isBlank()) {
        return postRepository.searchAllPosts(kw, sorted);
    }

    return postRepository.findAll(sorted);
}

public Page<Post> getAuthorPosts(Long authorId, String keyword, String sortBy, Pageable pageable) {
    User author = userRepository.findById(authorId)
        .orElseThrow(() -> new ResourceNotFoundException("User not found: " + authorId));

    Sort sort = resolveSort(sortBy);
    Pageable sorted = PageRequest.of(pageable.getPageNumber(), pageable.getPageSize(), sort);

    String kw = (keyword == null) ? null : keyword.trim();
    if (kw != null && !kw.isBlank()) {
        return postRepository.searchPostsByAuthor(author, kw, sorted);
    }
}

```

```

        return postRepository.findByUser(author, sorted);
    }

    public List<Post> findPostsByUserId(Long userId) {
        return userRepository.findById(userId)
            .map(postRepository::findByUserOrderByCreatedAtDesc)
            .orElse(List.of());
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/ProfileService.java

```

package org.AE.alliededge.service;

import org.AE.alliededge.model.User;
import org.springframework.web.multipart.MultipartFile;

public interface ProfileService {

    /**
     * Resolve the currently authenticated user based on the Spring Security principal (email).
     *
     * @return the current User
     * @throws IllegalStateException if the authentication is missing or the user cannot be found
     */
    User getCurrentUser();

    /**
     * Update editable profile fields for the given user.
     * Admin users cannot change username or email via this method.
     */
    void updateProfile(User user,
        String displayName,
        String bio,
        String linkedin,
        String github,
        String website,
        String location,
        String twitter,
        String skillsJson,
        String projectsJson,
        String experienceJson,
        String languagesJson,
        String educationJson,
        String availabilityJson,
        Boolean showEmailOnProfile,
        Boolean showResumeOnProfile);

    /**
     * Update the username for a regular (non-admin) user.
     * Validation (uniqueness, format, etc.) is delegated to UserService.
     */
    void updateUsernameForUser(User user, String newUsername);

    /**
     * Compute admin-specific post statistics for the given admin user.
     */
    AdminPostStats calculateAdminPostStats(User adminUser);

    /**
     * Simple DTO for aggregated admin post statistics.
     */
    class AdminPostStats {
        private long totalPosts;
        private long totalLikes;
        private long totalComments;
        private long totalViews;

        public long getTotalPosts() {
            return totalPosts;
        }
    }
}

```

```

    }

    public void setTotalPosts(long totalPosts) {
        this.totalPosts = totalPosts;
    }

    public long getTotalLikes() {
        return totalLikes;
    }

    public void setTotalLikes(long totalLikes) {
        this.totalLikes = totalLikes;
    }

    public long getTotalComments() {
        return totalComments;
    }

    public void setTotalComments(long totalComments) {
        this.totalComments = totalComments;
    }

    public long getTotalViews() {
        return totalViews;
    }

    public void setTotalViews(long totalViews) {
        this.totalViews = totalViews;
    }
}

// New API for profile image uploads via Cloudinary
/**
 * Uploads and sets a new profile image for the given user.
 * Returns the secure Cloudinary URL for the image.
 */
String updateProfileImage(User user, MultipartFile imageFile);

/**
 * Removes any stored profile image reference for the user and clears profileImageUrl/profileImagePath as needed.
 */
void removeProfileImage(User user);

/**
 * Upload (or replace) the user's banner image and return the stored URL.
 */
String updateBannerImage(User user, MultipartFile imageFile);

/**
 * Remove the user's banner image reference.
 */
void removeBannerImage(User user);
}

```

■ backend/src/main/java/org/AE/alliededge/service/ProfileServiceImpl.java

```

package org.AE.alliededge.service;

import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.AE.alliededge.model.Project;
import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.PostRepository;
import org.AE.alliededge.repository.ProjectRepository;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

```

```

import org.springframework.web.multipart.MultipartFile;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.Map;

@Service
@Transactional
public class ProfileServiceImpl implements ProfileService {

    private final UserRepository userRepository;
    private final PostRepository postRepository;
    private final ProjectRepository projectRepository;
    private final UserService userService;
    private final CloudinaryService cloudinaryService;
    private final ObjectMapper objectMapper;

    public ProfileServiceImpl(UserRepository userRepository,
                             PostRepository postRepository,
                             ProjectRepository projectRepository,
                             UserService userService,
                             CloudinaryService cloudinaryService,
                             ObjectMapper objectMapper) {
        this.userRepository = userRepository;
        this.postRepository = postRepository;
        this.projectRepository = projectRepository;
        this.userService = userService;
        this.cloudinaryService = cloudinaryService;
        this.objectMapper = objectMapper;
    }

    @Override
    public User getCurrentUser() {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
        if (authentication == null || authentication.getName() == null) {
            // No authenticated principal in context
            return null;
        }

        String principalName = authentication.getName();
        // Spring Security uses the literal string "anonymousUser" when no user is logged in
        if ("anonymousUser".equals(principalName)) {
            return null;
        }

        return userRepository.findByEmail(principalName).orElse(null);
    }

    @Override
    public void updateProfile(User user,
                             String displayName,
                             String bio,
                             String linkedin,
                             String github,
                             String website,
                             String location,
                             String twitter,
                             String skillsJson,
                             String projectsJson,
                             String experienceJson,
                             String languagesJson,
                             String educationJson,
                             String availabilityJson,
                             Boolean showEmailOnProfile,
                             Boolean showResumeOnProfile) {
        // IMPORTANT: Treat this endpoint as a partial update.
        // If a field is omitted (null), keep the existing DB value.
        // If a field is provided but blank, clear it (store null).

        if (displayName != null) {

```

```

        String v = displayName.trim();
        user.setDisplayName(v.isEmpty() ? null : v);
    }
    if (bio != null) {
        String v = bio.trim();
        user.setBio(v.isEmpty() ? null : v);
    }
    if (linkedin != null) {
        String v = linkedin.trim();
        user.setLinkedin(v.isEmpty() ? null : v);
    }
    if (github != null) {
        String v = github.trim();
        user.setGithub(v.isEmpty() ? null : v);
    }
    if (website != null) {
        String v = website.trim();
        user.setWebsite(v.isEmpty() ? null : v);
    }
    if (location != null) {
        String v = location.trim();
        user.setLocation(v.isEmpty() ? null : v);
    }
    if (twitter != null) {
        String v = twitter.trim();
        user.setTwitter(v.isEmpty() ? null : v);
    }

    if (experienceJson != null) {
        String v = experienceJson.trim();
        user.setExperienceJson(v.isEmpty() ? null : v);
    }
    if (languagesJson != null) {
        String v = languagesJson.trim();
        user.setLanguagesJson(v.isEmpty() ? null : v);
    }
    if (educationJson != null) {
        String v = educationJson.trim();
        user.setEducationJson(v.isEmpty() ? null : v);
    }
    if (availabilityJson != null) {
        String v = availabilityJson.trim();
        user.setAvailabilityJson(v.isEmpty() ? null : v);
    }

    // Public visibility flags: only update when explicitly provided.
    if (showEmailOnProfile != null) {
        user.setShowEmailOnProfile(showEmailOnProfile);
    }
    if (showResumeOnProfile != null) {
        user.setShowResumeOnProfile(showResumeOnProfile);
    }

    if (skillsJson != null) {
        // Skills are stored as a normalized list, not JSON.
        List<String> skills = parseStringListJson(skillsJson);
        user.setSkills(skills != null ? skills : new ArrayList<>());
    }

    // Persist scalar fields first.
    userRepository.save(user);

    if (projectsJson != null) {
        // Replace-all semantics for projects: frontend edits the whole list.
        // We delete existing projects and re-insert.
        List<Map<String, Object>> projects = parseListOfMapsJson(projectsJson);

        // remove existing
        List<Project> existing = projectRepository.findByUserId(user.getId());
        if (existing != null && !existing.isEmpty()) {
            projectRepository.deleteAll(existing);

```

```

    }

    if (projects != null) {
        for (Map<String, Object> p : projects) {
            String title = stringVal(p.get("title"));
            if (title == null || title.isBlank()) continue;

            Project proj = new Project();
            proj.setUser(user);
            proj.setTitle(title);
            proj.setSummary(stringVal(p.get("summary")));
            proj.setStatus(stringVal(p.get("status")));
            proj.setProblem(stringVal(p.get("problem")));
            proj.setBuilt(stringVal(p.get("built")));
            proj.setRole(stringVal(p.get("role")));

            // Optional legacy fields
            proj.setDescription(stringVal(p.get("description")));
            proj.setProjectUrl(stringVal(p.get("link")));

            proj.setTechStack(stringListVal(p.get("techStack")));
            proj.setProofLinks(mapVal(p.get("proofLinks")));

            projectRepository.save(proj);
        }
    }
}

private List<String> parseStringListJson(String json) {
    if (json == null) return null;
    String trimmed = json.trim();
    if (trimmed.isEmpty()) return new ArrayList<>();
    try {
        return objectMapper.readValue(trimmed, new TypeReference<>() {});
    } catch (Exception e) {
        return new ArrayList<>();
    }
}

private List<Map<String, Object>> parseListOfMapsJson(String json) {
    if (json == null) return null;
    String trimmed = json.trim();
    if (trimmed.isEmpty()) return new ArrayList<>();
    try {
        return objectMapper.readValue(trimmed, new TypeReference<>() {});
    } catch (Exception e) {
        return new ArrayList<>();
    }
}

private String stringVal(Object o) {
    if (o == null) return null;
    String s = String.valueOf(o);
    return s.isBlank() ? null : s;
}

@SuppressWarnings("unchecked")
private List<String> stringListVal(Object o) {
    if (o == null) return null;
    if (o instanceof List<?> list) {
        List<String> out = new ArrayList<>();
        for (Object item : list) {
            if (item == null) continue;
            String s = String.valueOf(item).trim();
            if (!s.isEmpty()) out.add(s);
        }
        return out;
    }
    return null;
}

```

```

@SuppressWarnings("unchecked")
private Map<String, Object> mapVal(Object o) {
    if (o instanceof Map<?, ?> m) {
        return (Map<String, Object>) m;
    }
    return null;
}

@Override
public void updateUsernameForUser(User user, String newUsername) {
    // Delegate to existing validation logic in UserService so we don't duplicate rules
    String error = userService.validateAndChangeUsername(user, newUsername);
    if (error != null) {
        throw new IllegalArgumentException(error);
    }
}

@Override
public AdminPostStats calculateAdminPostStats(User adminUser) {
    List<Post> posts = postRepository.findByUser(adminUser);
    AdminPostStats stats = new AdminPostStats();
    stats.setTotalPosts(posts.size());

    long totalLikes = 0L;
    long totalComments = 0L;
    long totalViews = 0L;
    for (Post post : posts) {
        totalLikes += post.getLikes();
        totalViews += post.getViews();
        if (post.getComments() != null) {
            totalComments += post.getComments().size();
        }
    }
    stats.setTotalLikes(totalLikes);
    stats.setTotalComments(totalComments);
    stats.setTotalViews(totalViews);
    return stats;
}

@Override
public String updateProfileImage(User user, MultipartFile imageFile) {
    if (imageFile == null || imageFile.isEmpty()) {
        throw new IllegalArgumentException("Please select an image to upload.");
    }

    String contentType = imageFile.getContentType();
    if (contentType == null || !isSupportedImageContentType(contentType)) {
        throw new IllegalArgumentException("Only JPG, JPEG, PNG and WEBP images are allowed.");
    }

    // Upload to Cloudinary under profile_pics/ folder. The implementation of this
    // method already returns the secure_url string for the uploaded resource.
    String secureUrl = cloudinaryService.uploadFile(imageFile, "profile_pics");

    // Store canonical Cloudinary URL for all future usage. Legacy fields are kept for schema compatibility only.
    user.setProfileImageUrl(secureUrl);
    user.setProfilePhoto(null);
    user.setProfileImagePath(null);
    userRepository.save(user);

    return secureUrl;
}

@Override
public void removeProfileImage(User user) {
    // Just null-out stored references. We intentionally do not delete from Cloudinary here.
    user.setProfileImageUrl(null);
    user.setProfilePhoto(null);
    user.setProfileImagePath(null);
    userRepository.save(user);
}

```

```

@Override
public String updateBannerImage(User user, MultipartFile imageFile) {
    if (imageFile == null || imageFile.isEmpty()) {
        throw new IllegalArgumentException("Please select an image to upload.");
    }

    String contentType = imageFile.getContentType();
    if (contentType == null || !isSupportedImageContentType(contentType)) {
        throw new IllegalArgumentException("Only JPG, JPEG, PNG and WEBP images are allowed.");
    }

    // Upload to Cloundinary under profile_banners/ folder.
    String secureUrl = cloundinaryService.uploadFile(imageFile, "profile_banners");

    user.setBannerImageUrl(secureUrl);
    userRepository.save(user);

    return secureUrl;
}

@Override
public void removeBannerImage(User user) {
    user.setBannerImageUrl(null);
    userRepository.save(user);
}

private boolean isSupportedImageContentType(String contentType) {
    String type = contentType.toLowerCase(Locale.ROOT);
    return type.equals("image/jpeg") || type.equals("image/jpg") || type.equals("image/png") || type.equals("image/webp");
}

// Local file deletion is no longer needed once migrated to Cloundinary.
}

```

■ backend/src/main/java/org/AE/alliededge/service/ProjectService.java

```

package org.AE.alliededge.service;

import org.AE.alliededge.model.Project;
import org.AE.alliededge.repository.ProjectRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Optional;

@Service
@Transactional
public class ProjectService {

    @Autowired
    private ProjectRepository projectRepository;

    public List<Project> getProjectsByUser(Long userId) {
        return projectRepository.findByUserIdOrderByCreatedAtDesc(userId);
    }

    public Project saveProject(Project project) {
        return projectRepository.save(project);
    }

    public Optional<Project> findById(Long id) {
        return projectRepository.findById(id);
    }

    public void deleteProject(Long projectId) {
        projectRepository.deleteById(projectId);
    }
}

```



```

    public boolean existsById(Long id) {
        return projectRepository.existsById(id);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/ResumeService.java

```

package org.AE.alliededge.service;

import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.multipart.MultipartFile;

import java.util.Map;

@Service
public class ResumeService {

    private final CloudinaryService cloudinaryService;
    private final UserRepository userRepository;

    public ResumeService(CloudinaryService cloudinaryService, UserRepository userRepository) {
        this.cloudinaryService = cloudinaryService;
        this.userRepository = userRepository;
    }

    @Transactional
    public void uploadResume(User user, MultipartFile file) {
        if (file.isEmpty()) {
            throw new IllegalArgumentException("Resume file cannot be empty");
        }

        String contentType = file.getContentType();
        if (contentType == null || !contentType.equals("application/pdf")) {
            throw new IllegalArgumentException("Only PDF files are allowed");
        }

        if (user.getResumePublicId() != null) {
            cloudinaryService.deleteRawFileByPublicId(user.getResumePublicId());
        }

        Map<String, String> result = cloudinaryService.uploadRawFile(file, "resumes");

        user.setResumeUrl(result.get("url"));
        user.setResumePublicId(result.get("publicId"));

        userRepository.save(user);
    }

    @Transactional
    public void deleteResume(User user) {
        if (user.getResumePublicId() != null) {
            cloudinaryService.deleteRawFileByPublicId(user.getResumePublicId());
        }

        user.setResumeUrl(null);
        user.setResumePublicId(null);

        userRepository.save(user);
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/UserFollowService.java

```

package org.AE.alliededge.service;

import org.AE.alliededge.model.User;

```

```

import org.AE.alliededge.model.UserFollow;
import org.AE.alliededge.repository.UserFollowRepository;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class UserFollowService {

    private final UserFollowRepository userFollowRepository;
    private final UserRepository userRepository;

    public UserFollowService(UserFollowRepository userFollowRepository, UserRepository userRepository) {
        this.userFollowRepository = userFollowRepository;
        this.userRepository = userRepository;
    }

    private User resolvePrincipalUser(String principal) {
        if (principal == null || principal.isBlank()) {
            return null;
        }
        User byEmail = userRepository.findByEmail(principal).orElse(null);
        if (byEmail != null) return byEmail;
        return userRepository.findByUsername(principal).orElse(null);
    }

    /**
     * Follow a user
     */
    @Transactional
    public void followUser(String followerEmail, Long targetUserId) {
        User follower = resolvePrincipalUser(followerEmail);
        if (follower == null) {
            throw new IllegalArgumentException("Follower not found");
        }

        User target = userRepository.findById(targetUserId)
            .orElseThrow(() -> new IllegalArgumentException("Target user not found"));

        // Prevent self-follow
        if (follower.getId().equals(target.getId())) {
            throw new IllegalArgumentException("Cannot follow yourself");
        }

        // Check if already following
        if (userFollowRepository.existsByFollowerAndFollowing(follower, target)) {
            return; // Already following, do nothing
        }

        UserFollow userFollow = new UserFollow();
        userFollow.setFollower(follower);
        userFollow.setFollowing(target);
        userFollowRepository.save(userFollow);
    }

    /**
     * Unfollow a user
     */
    @Transactional
    public void unfollowUser(String followerEmail, Long targetUserId) {
        User follower = resolvePrincipalUser(followerEmail);
        if (follower == null) {
            throw new IllegalArgumentException("Follower not found");
        }

        User target = userRepository.findById(targetUserId)
            .orElseThrow(() -> new IllegalArgumentException("Target user not found"));

        userFollowRepository.deleteByFollowerAndFollowing(follower, target);
    }
}

```

```

}

/**
 * Check if user is following another user
 */
public boolean isFollowing(String followerEmail, Long targetUserId) {
    User follower = resolvePrincipalUser(followerEmail);
    User target = userRepository.findById(targetUserId).orElse(null);

    if (follower == null || target == null) {
        return false;
    }

    return userFollowRepository.existsByFollowerAndFollowing(follower, target);
}

/**
 * Check if two users follow each other (mutual follow / "connected").
 */
public boolean isMutualFollowing(String currentUserEmail, Long targetUserId) {
    User currentUser = resolvePrincipalUser(currentUserEmail);
    User targetUser = userRepository.findById(targetUserId).orElse(null);

    if (currentUser == null || targetUser == null) {
        return false;
    }

    boolean aFollowsB = userFollowRepository.existsByFollowerAndFollowing(currentUser, targetUser);
    boolean bFollowsA = userFollowRepository.existsByFollowerAndFollowing(targetUser, currentUser);
    return aFollowsB && bFollowsA;
}

/**
 * Get follower count for a user
 */
public long getFollowerCount(Long userId) {
    User user = userRepository.findById(userId).orElse(null);
    if (user == null) {
        return 0;
    }
    return userFollowRepository.countByFollowing(user);
}

/**
 * Get following count for a user
 */
public long getFollowingCount(Long userId) {
    User user = userRepository.findById(userId).orElse(null);
    if (user == null) {
        return 0;
    }
    return userFollowRepository.countByFollower(user);
}

/**
 * Get list of followers
 */
public List<User> getFollowers(Long userId) {
    User user = userRepository.findById(userId).orElse(null);
    if (user == null) {
        return List.of();
    }
    return userFollowRepository.findByFollowing(user).stream()
        .map(UserFollow::getFollower)
        .collect(Collectors.toList());
}

/**
 * Get list of users being followed
 */
public List<User> getFollowing(Long userId) {

```

```

        User user = userRepository.findById(userId).orElse(null);
        if (user == null) {
            return List.of();
        }
        return userFollowRepository.findByFollower(user).stream()
            .map(UserFollow::getFollowing)
            .collect(Collectors.toList());
    }
}

```

■ backend/src/main/java/org/AE/alliededge/service/UserService.java

```

package org.AE.alliededge.service;

import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class UserService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public UserService(UserRepository userRepository, PasswordEncoder passwordEncoder) {
        this.passwordEncoder = passwordEncoder;
        this.userRepository = userRepository;
    }

    public User registerUser(User user) {

        if (userRepository.findByUsername(user.getUsername()).isPresent()) {
            throw new IllegalArgumentException("Username already exists");
        }
        String hashedPassword = passwordEncoder.encode(user.getPassword());
        user.setPassword(hashedPassword);

        // Default role for local registrations (but don't override if a caller explicitly set one)
        if (user.getRole() == null || user.getRole().isBlank()) {
            user.setRole("ROLE_AUTHOR");
        }

        // Normal registrations are never admin unless explicitly set elsewhere
        if (user.getRole() == null || !user.getRole().equals("ROLE_ADMIN")) {
            user.setAdmin(false);
        }

        // Default status
        if (user.getStatus() == null || user.getStatus().isBlank()) {
            user.setStatus("ACTIVE");
        }

        // If no displayName provided, fall back to username
        if (user.getDisplayName() == null || user.getDisplayName().isBlank()) {
            user.setDisplayName(user.getUsername());
        }

        return userRepository.save(user);
    }

    /**
     * Validate and change the username for the given user.
     * Returns null on success, or an error message if validation fails.
     */
    public String validateAndChangeUsername(User user, String newUsername) {
        if (user == null) {

```

```

        return "User not found";
    }

    String trimmed = newUsername == null ? "" : newUsername.trim();
    if (trimmed.isEmpty()) {
        return "Username is required";
    }
    if (trimmed.contains("@")) {
        return "Username cannot be an email address";
    }
    if (!trimmed.matches("[a-zA-Z0-9._-]{3,30}")) {
        return "Username must be 3-30 characters and contain only letters, numbers, dots, underscores, or hyphens";
    }

    // If username is unchanged, treat as success (no-op)
    if (trimmed.equals(user.getUsername())) {
        return null;
    }

    if (userRepository.existsByUsername(trimmed)) {
        return "Username is already taken";
    }

    String oldUsername = user.getUsername();
    user.setUsername(trimmed);
    // Keep displayName in sync if it previously matched the old username or was
    // blank
    if (user.getDisplayName() == null || user.getDisplayName().isBlank()
        || user.getDisplayName().equals(oldUsername)) {
        user.setDisplayName(trimmed);
    }
    userRepository.save(user);
    return null;
}

// --- Helpers used by profile-related controllers and global model attributes
// ---

public Optional<User> findByEmail(String email) {
    return userRepository.findByEmail(email);
}

public Optional<User> findByUsername(String username) {
    return userRepository.findByUsername(username);
}

/**
 * Returns the currently authenticated User entity, if any.
 */
public Optional<User> getCurrentUserOptional() {
    Authentication auth = SecurityContextHolder.getContext().getAuthentication();
    if (auth == null || !auth.isAuthenticated() || "anonymousUser".equals(auth.getName())) {
        return Optional.empty();
    }
    String email = auth.getName();
    return userRepository.findByEmail(email);
}

public User getCurrentUserOrThrow() {
    return getCurrentUserOptional()
        .orElseThrow(() -> new IllegalStateException("Current user not found"));
}

public boolean isCurrentUserAuthor() {
    return getCurrentUserOptional().map(User::isAuthor).orElse(false);
}

public boolean isCurrentUserAdmin() {
    return getCurrentUserOptional().map(User::isAdmin).orElse(false);
}
/**

```

```

    * Upgrade the currently logged-in plain user to AUTHOR.
    * Does nothing if already AUTHOR or ADMIN.
    */
public void upgradeCurrentUserToAuthor() {
    User user = getCurrentUserOrThrow();
    if (user.isAuthor() || user.isAdmin()) {
        return; // already has authoring privileges
    }
    user.setRole("ROLE_AUTHOR");
    if (user.getStatus() == null || user.getStatus().isBlank()) {
        user.setStatus("ACTIVE");
    }
    userRepository.save(user);
}

/**
 * Count comments created by the given user. If Comment repository is not
 * available,
 * fall back to the size of the mapped collection.
 */
public long countCommentsByUser(User user) {
    if (user.getComments() == null) {
        return 0L;
    }
    return user.getComments().size();
}

/**
 * Count likes given by the user. This implementation returns 0 unless a
 * dedicated
 * likes repository exists; you can wire it later without changing callers.
 */
public long countLikesByUser(User user) {
    return 0L;
}

/**
 * Search users by username or display name (case-insensitive, partial match).
 */
public List<User> searchUsers(String keyword) {
    return userRepository.findByUsernameContainingIgnoreCaseOrDisplayNameContainingIgnoreCase(keyword, keyword);
}

/**
 * Search users by username or display name (case-insensitive, partial match), excluding a specific user id.
 */
public List<User> searchUsersExcludingUserId(String keyword, Long excludeUserId) {
    if (excludeUserId == null) {
        return searchUsers(keyword);
    }
    return userRepository.findByIdNotAndUsernameContainingIgnoreCaseOrIdNotAndDisplayNameContainingIgnoreCase(
        excludeUserId,
        keyword,
        excludeUserId,
        keyword
    );
}

public User getUserById(Long id) {
    return userRepository.findById(id).orElse(null);
}

public User getUserByEmail(String email) {
    return userRepository.findByEmail(email).orElse(null);
}

/**
 * Resolve an authenticated principal string to a User.
 *
 * In this app, {@code Authentication#getName()} can be either:
 * - email (OAuth2 / Google)

```

```

    * - username (form login / tests)
    */
    public User getUserByPrincipal(String principal) {
        if (principal == null || principal.isBlank() || "anonymousUser".equals(principal)) {
            return null;
        }
        User byEmail = userRepository.findByEmail(principal).orElse(null);
        if (byEmail != null) {
            return byEmail;
        }
        return userRepository.findByUsername(principal).orElse(null);
    }

    // Skills management
    public void addSkill(Long userId, String skill) {
        User user = userRepository.findById(userId)
            .orElseThrow(() -> new IllegalArgumentException("User not found"));
        if (user.getSkills() == null) {
            user.setSkills(new java.util.ArrayList<>());
        }
        if (!user.getSkills().contains(skill)) {
            user.getSkills().add(skill);
            userRepository.save(user);
        }
    }

    public void removeSkill(Long userId, String skill) {
        User user = userRepository.findById(userId)
            .orElseThrow(() -> new IllegalArgumentException("User not found"));
        if (user.getSkills() != null) {
            user.getSkills().remove(skill);
            userRepository.save(user);
        }
    }

    public List<String> getSkills(Long userId) {
        User user = userRepository.findById(userId).orElse(null);
        return user != null && user.getSkills() != null ? user.getSkills() : new java.util.ArrayList<>();
    }
}

```

■ backend/src/main/resources/application.properties

```

spring.application.name=AlliedEdge

# =====
# Server (Render/Cloudflare)
# =====
# Render sets $PORT for web services. If we don't bind to it, the service may be
# considered down and Cloudflare can return 521.
server.port=${PORT:8080}

# =====
# Database (env-first, localhost fallback for local dev)
# =====
spring.datasource.url=${SPRING_DATASOURCE_URL}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver

# =====
# Hibernate & JPA
# =====
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.sql.init.mode=never

# =====
# Flyway
# =====

```

```

spring.flyway.enabled=true
spring.flyway.baseline-on-migrate=true
flyway.repair.enabled=${FLYWAY_REPAIR_ENABLED:false}

spring.datasource.hikari.maximum-pool-size=3
spring.datasource.hikari.minimum-idle=1
spring.datasource.hikari.idle-timeout=10000
spring.datasource.hikari.connection-timeout=30000
spring.datasource.hikari.max-lifetime=30000

# =====
# H2 (disabled)
# =====
spring.h2.console.enabled=false

# =====
# Multipart / Upload limits
# =====
spring.servlet.multipart.enabled=true
spring.servlet.multipart.max-file-size=120MB
spring.servlet.multipart.max-request-size=130MB

# =====
# Google OAuth2 (env-based)
# =====
spring.security.oauth2.client.registration.google.client-id=${VITE_GOOGLE_CLIENT_ID}
spring.security.oauth2.client.registration.google.client-secret=${VITE_GOOGLE_CLIENT_SECRET}
spring.security.oauth2.client.registration.google.scope=email,profile
spring.security.oauth2.client.registration.google.redirect-uri={baseUrl}/login/oauth2/code/google

spring.security.oauth2.client.provider.google.authorization-uri=https://accounts.google.com/o/oauth2/v2/auth
spring.security.oauth2.client.provider.google.token-uri=https://oauth2.googleapis.com/token
spring.security.oauth2.client.provider.google.user-info-uri=https://www.googleapis.com/oauth2/v3/userinfo
spring.security.oauth2.client.provider.google.user-name-attribute=sub

# =====
# Error handling
# =====
server.error.include-message=never

# =====
# Cloudinary (REQUIRED in Docker/Prod)
# =====
cloudinary.enabled=${CLOUDINARY_ENABLED:true}
cloudinary.cloud-name=${CLOUDINARY_CLOUD_NAME}
cloudinary.api-key=${CLOUDINARY_API_KEY}
cloudinary.api-secret=${CLOUDINARY_API_SECRET}

# =====
# JWT
# =====
app.jwt.secret=${APP_JWT_SECRET:change-this-in-prod}
app.jwt.expirationSeconds=${APP_JWT_EXPIRATION:86400}

# =====
# CORS
# =====
app.cors.allowed-origins=${APP_CORS_ALLOWED_ORIGINS}

# =====
# Frontend redirect (OAuth2 success)
# =====
app.frontend.redirect-url=${APP_FRONTEND_REDIRECT_URL}

# =====
# Cookies / proxy (needed for cross-site session auth in production)
# =====
# When frontend (Vercel) and backend (Render) are on different sites, browsers will
# drop the session cookie unless it is marked SameSite=None; Secure.

```



```

server.servlet.session.cookie.same-site=none
server.servlet.session.cookie.secure=true
# Ensure Spring correctly detects https behind Render/Cloudflare proxies.
server.forward-headers-strategy=framework

# =====
# Timezone (keep timestamps consistent across environments)
# =====
# Entities currently persist LocalDateTime, so we must pin the JVM/Jackson/Hibernate timezone
# to avoid seeing times shifted after deployment.
spring.jackson.time-zone=UTC
spring.jpa.properties.hibernate.jdbc.time_zone=UTC

```

■ backend/src/main/resources/templates/error/404.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>404 - Not Found</title>
    <style>
        body {
            font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif;
            background-color: #f8f9fa;
            color: #343a40;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
            text-align: center;
        }
        .container {
            max-width: 600px;
        }
        h1 {
            font-size: 5rem;
            font-weight: 300;
            margin: 0;
        }
        h2 {
            font-size: 1.5rem;
            margin-top: 0;
            margin-bottom: 20px;
            color: #6c757d;
        }
        p {
            margin-bottom: 20px;
        }
        .error-details {
            font-family: "Courier New", Courier, monospace;
            background-color: #e9ecef;
            padding: 10px;
            border-radius: 5px;
            color: #495057;
            font-size: 0.9em;
            margin-bottom: 30px;
        }
        a {
            color: #007bff;
            text-decoration: none;
            font-weight: bold;
        }
        a:hover {
            text-decoration: underline;
        }
    </style>
</head>
<body>
    <div class="container">

```

```

<h1>404</h1>
<h2>Oops! Page Not Found.</h2>
<p>The page you are looking for might have been removed, had its name changed, or is temporarily unavailable.

<div class="error-details" th:if="${errorMessage}" th:text="${errorMessage}">
    Specific error details will go here.
</div>

<p><a th:href="@{/}">Return to Homepage</a></p>
</div>
</body>
</html>

```

■ backend/src/main/resources/templates/error/generic-error.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Error</title>
    <!-- We can reuse the same styling from our 404 page -->
    <style>
        body {
            font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif;
            background-color: #f8f9fa;
            color: #343a40;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
            text-align: center;
        }
        .container {
            max-width: 600px;
        }
        h1 {
            font-size: 5rem;
            font-weight: 300;
            margin: 0;
            color: #dc3545; /* Red for error */
        }
        h2 {
            font-size: 1.5rem;
            margin-top: 0;
            margin-bottom: 20px;
            color: #6c757d;
        }
        p {
            margin-bottom: 20px;
        }
        .error-details {
            font-family: "Courier New", Courier, monospace;
            background-color: #e9ecef;
            padding: 10px;
            border-radius: 5px;
            color: #495057;
            font-size: 0.9em;
            margin-bottom: 30px;
        }
        a {
            color: #007bff;
            text-decoration: none;
            font-weight: bold;
        }
        a:hover {
            text-decoration: underline;
        }
    </style>

```

```

</head>;
<body>;
<div class="container">;
    <h1>;Error</h1>;
    <h2>;Something Went Wrong</h2>;

    <div class="error-details" th:if="${errorMessage}" th:text="${errorMessage}">;
        An unexpected error occurred. Please try again later.
    </div>;
    <div class="error-details" th:if="${path}" >;Path: <span th:text="${path}">;</span>;</div>;
    <div class="error-details" th:if="${status}" >;Status: <span th:text="${status}">;</span>;</div>;

    <p>;We are sorry for the inconvenience. Our technical team has been notified.</p>;
    <p>;<a th:href="@{/}">;Return to Homepage</a>;</p>;
</div>;
</body>;
</html>;

```

■ backend/src/test/java/org/AE/alliededge/AlliedEdgeApplicationTests.java

```

package org.AE.alliededge;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class AlliedEdgeApplicationTests {

    @Test
    void contextLoads() {
    }

}

```

■ backend/src/test/java/org/AE/alliededge/config/TestCloudinaryConfig.java

```

package org.AE.alliededge.config;

import org.AE.alliededge.service.CloudinaryService;
import org.AE.alliededge.service.FakeCloudinaryService;
import org.springframework.boot.test.context.TestConfiguration;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Primary;

@TestConfiguration
public class TestCloudinaryConfig {

    @Bean
    @Primary
    public CloudinaryService cloudinaryService() {
        return new FakeCloudinaryService();
    }

}

```

■ backend/src/test/java/org/AE/alliededge/controller/AnnouncementsPublicWebMvcTest.java

```

package org.AE.alliededge.controller;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

```

```

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
class AnnouncementsPublicWebMvcTest {

    @Autowired MockMvc mvc;

    @Test
    void getAnnouncements_isPublic() throws Exception {
        mvc.perform(get("/api/announcements")
            .accept(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk());
    }
}

```

■ backend/src/test/java/org/AE/alliededge/controller/AuthControllerWebMvcTest.java

```

package org.AE.alliededge.controller;

import org.AE.alliededge.repository.UserRepository;
import org.AE.alliededge.service.UserService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.web.servlet.MockMvc;

import org.springframework.test.context.bean.override.mockito.MockitoBean;

@WebMvcTest(AuthController.class)
class AuthControllerWebMvcTest {

    @Autowired
    MockMvc mvc;

    @MockitoBean
    UserService userService;

    @MockitoBean
    UserRepository userRepository;

    @Test
    void statusEndpoint_isReachable_withoutServerError() throws Exception {
        mvc.perform(org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get("/api/auth/status"))
            .andExpect(result -> {
                int s = result.getResponse().getStatus();
                if (!(s == 200 || (s >= 300 && s < 400))) {
                    throw new AssertionError("Expected 200 or 3xx redirect, got " + s);
                }
            });
    }
}

```

■ backend/src/test/java/org/AE/alliededge/controller/ChatMutualFollowSecurityTest.java

```

package org.AE.alliededge.controller;

import org.AE.alliededge.config.TestCloudinaryConfig;
import org.AE.alliededge.model.User;
import org.AE.alliededge.model.UserFollow;
import org.AE.alliededge.repository.UserFollowRepository;
import org.AE.alliededge.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;

```

```

import org.springframework.test.web.servlet.MockMvc;

import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.user;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
@Import(TestCloudinaryConfig.class)
class ChatMutualFollowSecurityTest {

    @Autowired
    MockMvc mockMvc;

    @Autowired
    UserRepository userRepository;

    @Autowired
    UserFollowRepository userFollowRepository;

    private User newUser(String email, String username) {
        User u = new User();
        u.setEmail(email);
        u.setUsername(username);
        u.setPassword("password");
        u.setRole("ROLE_USER");
        u.setAdmin(false);
        u.setDisplayName(username);
        return userRepository.save(u);
    }

    @Test
    void openChat_requiresMutualFollow() throws Exception {
        long n = System.nanoTime();
        String aEmail = "a+" + n + "@example.com";
        String bEmail = "b+" + (n + 1) + "@example.com";

        User a = newUser(aEmail, "a" + n);
        User b = newUser(bEmail, "b" + (n + 1));

        // A follows B only (one-way)
        UserFollow aFollowsB = new UserFollow();
        aFollowsB.setFollower(a);
        aFollowsB.setFollowing(b);
        userFollowRepository.save(aFollowsB);

        mockMvc.perform(get("/api/chats/with/{id}", b.getId()).with(user(aEmail)))
            .andExpect(status().isForbidden())
            .andExpect(jsonPath("$.canMessage").value(false))
            .andExpect(jsonPath("$.reason").value("FOLLOW_BACK_REQUIRED"));

        // B follows A -> mutual
        UserFollow bFollowsA = new UserFollow();
        bFollowsA.setFollower(b);
        bFollowsA.setFollowing(a);
        userFollowRepository.save(bFollowsA);

        mockMvc.perform(get("/api/chats/with/{id}", b.getId()).with(user(aEmail)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.canMessage").value(true))
            .andExpect(jsonPath("$.chatRoom.id").exists());
    }

    @Test
    void openChat_allowsMutualFollow_whenPrincipalIsUsername() throws Exception {
        long n = System.nanoTime();
        String aEmail = "a2+" + n + "@example.com";
        String bEmail = "b2+" + (n + 1) + "@example.com";

        User a = newUser(aEmail, "a2" + n);

```

```

        User b = newUser(bEmail, "b2" + (n + 1));

        UserFollow aFollowsB = new UserFollow();
        aFollowsB.setFollower(a);
        aFollowsB.setFollowing(b);
        userFollowRepository.save(aFollowsB);

        UserFollow bFollowsA = new UserFollow();
        bFollowsA.setFollower(b);
        bFollowsA.setFollowing(a);
        userFollowRepository.save(bFollowsA);

        // Authenticate as username (typical form-login principal)
        mockMvc.perform(get("/api/chats/with/{id}", b.getId()).with(user(a.getUsername())))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.canMessage").value(true));
    }
}

```

■ backend/src/test/java/org/AE/alliededge/controller/ChatPaginationWebMvcTest.java

```

package org.AE.alliededge.controller;

import org.AE.alliededge.model.ChatRoom;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.AE.alliededge.service.ChatService;
import org.AE.alliededge.service.UserFollowService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.web.servlet.MockMvc;

import java.util.UUID;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
public class ChatPaginationWebMvcTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private UserRepository userRepository;
    @Autowired private ChatService chatService;
    @Autowired private UserFollowService userFollowService;

    @Test
    @WithMockUser(username = "alice@example.com")
    void listMessagesPaged_returns200_andMessagesArray() throws Exception {
        final String suffix = UUID.randomUUID().toString().substring(0, 8);
        final String aliceEmail = "alice+" + suffix + "@example.com";
        final String bobEmail = "bob+" + suffix + "@example.com";
        final String aliceUsername = "alice_" + suffix;
        final String bobUsername = "bob_" + suffix;

        // Create two users
        User alice = new User();
        alice.setEmail(aliceEmail);
        alice.setUsername(aliceUsername);
        alice.setPassword("x");
        alice.setRole("ROLE_USER");
        alice.setAdmin(false);
        alice.setFirstLogin(false);
        userRepository.save(alice);
    }
}

```

```

        User bob = new User();
        bob.setEmail(bobEmail);
        bob.setUsername(bobUsername);
        bob.setPassword("x");
        bob.setRole("ROLE_USER");
        bob.setAdmin(false);
        bob.setFirstLogin(false);
        userRepository.save(bob);

        // Mutual follow
        userFollowService.followUser(alice.getEmail(), bob.getId());
        userFollowService.followUser(bob.getEmail(), alice.getId());

        // Create room and send a message
        ChatRoom room = chatService.getOrCreateChatRoom(alice, bob);
        chatService.sendMessage(room.getId(), alice, "hello");

        mockMvc.perform(get("/api/chats/{roomId}/messages", room.getId())
            .with(org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors
                .accept(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.messages").isArray())
            .andExpect(jsonPath("$.hasMore").exists()));
    }
}

```

■ backend/src/test/java/org/AE/alliededge/controller/PostDetailJsonTest.java

```

package org.AE.alliededge.controller;

import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.PostRepository;
import org.AE.alliededge.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import java.time.LocalDateTime;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
class PostDetailJsonTest {

    @Autowired MockMvc mockMvc;
    @Autowired UserRepository userRepository;
    @Autowired PostRepository postRepository;

    @Test
    void postDetail_returnsValidJson() throws Exception {
        String unique = String.valueOf(System.currentTimeMillis());

        User u = new User();
        u.setEmail("postdetail-" + unique + "@test.local");
        u.setUsername("postdetail_" + unique);
        u.setPassword("{noop}pw");
        u.setRole("ROLE_USER");
        u.setDisplayName("Post Detail");
        u.setAdmin(false);
        u.setFirstLogin(false);
        u = userRepository.save(u);

        Post p = new Post();
    }
}

```

```

        p.setTitle("Hello World");
        p.setContent("Content");
        p.setCreatedAt(LocalDate.now());
        p.setUser(u);
        p.setAuthorName(u.getDisplayName());
        p.setAuthorIsAdmin(false);
        p = postRepository.save(p);

        mockMvc.perform(get("/api/posts/{id}", p.getId()).accept(MediaType.APPLICATION_JSON))
                .andExpect(status().isOk())
                .andExpect(content().contentTypeCompatibleWith(MediaType.APPLICATION_JSON));
    }
}

```

■ backend/src/test/java/org/AE/alliededge/controller/PostFeedCommentCountWebMvcTest.java

```

package org.AE.alliededge.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import org.AE.alliededge.model.Comment;
import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.CommentRepository;
import org.AE.alliededge.repository.PostRepository;
import org.AE.alliededge.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import java.time.LocalDate;

import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
class PostFeedCommentCountWebMvcTest {

    @Autowired MockMvc mockMvc;
    @Autowired ObjectMapper objectMapper;

    @Autowired UserRepository userRepository;
    @Autowired PostRepository postRepository;
    @Autowired CommentRepository commentRepository;

    @Test
    void postsFeed_includesCommentCount() throws Exception {
        // Arrange: create a user + post + 2 comments
        String unique = String.valueOf(System.currentTimeMillis());
        User u = new User();
        u.setEmail("feedcount-" + unique + "@test.local");
        u.setUsername("feedcount_user_" + unique);
        u.setPassword("x");
        u.setRole("ROLE_USER");
        u.setAdmin(false);
        u.setFirstLogin(false);
        u = userRepository.save(u);

        Post p = new Post();
        p.setUser(u);
        p.setAuthorName("Feed Count");
        p.setAuthorIsAdmin(false);
        p.setTitle("title1");
        p.setContent("hello");
    }
}

```



```

        p.setCreatedAt(LocalDateTime.now());
        p.setLikes(0);
        p.setViews(0);
        p = postRepository.save(p);

        Comment c1 = new Comment();
        c1.setPost(p);
        c1.setUser(u);
        c1.setContent("c1");
        c1.setCreatedAt(LocalDateTime.now());
        commentRepository.save(c1);

        Comment c2 = new Comment();
        c2.setPost(p);
        c2.setUser(u);
        c2.setContent("c2");
        c2.setCreatedAt(LocalDateTime.now());
        commentRepository.save(c2);

        // Act
        String json = mvc.perform(get("/api/posts")
                                .param("page", "0")
                                .param("size", "5")
                                .accept(MediaType.APPLICATION_JSON))
                        .andExpect(status().isOk())
                        .andReturn()
                        .getResponse()
                        .getContentAsString();

        // Assert: ensure commentCount field exists and equals 2 for our post
        var root = objectMapper.readTree(json);
        var content = root.path("postPage").path("content");
        assertThat(content.isArray(), isTrue());

        boolean found = false;
        for (var item : content) {
            if (item.path("id").asLong() == p.getId()) {
                found = true;
                assertThat(item.has("commentCount"), isTrue());
                assertThat(item.path("commentCount").asInt(), isEqualTo(2));
            }
        }
        assertThat(found, isTrue());
    }
}

```

■ backend/src/test/java/org/AE/alliededge/controller/ProfileControllerTest.java

```

package org.AE.alliededge.controller;

import org.AE.alliededge.config.TestCloudinaryConfig;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.test.web.servlet.MockMvc;

import java.util.UUID;

import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.multipart;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.csrf;

@SpringBootTest

```

```

@AutoConfigureMockMvc
@Import(TestCloudinaryConfig.class)
class ProfileControllerTest {

    @Autowired
    MockMvc mockMvc;

    @Autowired
    UserRepository userRepository;

    private String testEmail;

    @BeforeEach
    void setup() {
        // Don't wipe the DB here: other tests may have inserted rows referencing users (FK constraints).
        // Instead, create a unique user for each test run.
        String suffix = UUID.randomUUID().toString().replace("-", "");
        testEmail = "me+" + suffix + "@example.com";
        User u = new User();
        u.setEmail(testEmail);
        u.setUsername("me_" + suffix);
        u.setPassword("password");
        u.setRole("ROLE_USER");
        u.setAdmin(false);
        u.setDisplayName("Me");
        u.setBio("Old bio");
        userRepository.save(u);
    }

    @Test
    void updateProfile_persistsChanges_andDoesNotNullOutMissingFields() throws Exception {
        mockMvc.perform(
            multipart("/api/profile")
                .with(req -> {
                    req.setMethod("PUT");
                    return req;
                })
                .with(org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.csrf())
                .param("displayName", "New Name")
                // bio intentionally omitted; should stay "Old bio"
                .param("removeProfileImage", "false")
        )
        .andExpect(status().isOk());

        User saved = userRepository.findByEmail(testEmail).orElseThrow();
        assertThat(saved.getDisplayName()).isEqualTo("New Name");
        assertThat(saved.getBio()).isEqualTo("Old bio");
    }

    @Test
    void updateProfile_invalidUsername_returns400() throws Exception {
        mockMvc.perform(
            multipart("/api/profile")
                .with(req -> {
                    req.setMethod("PUT");
                    return req;
                })
                .with(org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.csrf())
                .param("username", "bad@name")
                .param("removeProfileImage", "false")
        )
        .andExpect(status().isBadRequest());
    }

    @Test
    void updateProfile_canSetAndRemoveBannerImageUrl() throws Exception {
        // Set banner (simulate with a real image part; service will validate content-type and non-empty)
        mockMvc.perform(
            multipart("/api/profile")

```

```

        .file(new org.springframework.mock.web.MockMultipartFile(
            "bannerImageFile",
            "banner.png",
            "image/png",
            new byte[]{1, 2, 3}
        ))
        .with(req -> {
            req.setMethod("PUT");
            return req;
        })
        .with(org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessor.DEFAULT)
        .with(csrf())
        .param("removeBannerImage", "false")
        .param("removeProfileImage", "false")
    )
    .andExpect(status().isOk());

User afterUpload = userRepository.findByEmail(testEmail).orElseThrow();
// We can't assert the exact Clouinary URL without mocking; just assert it's non-blank.
assertThat(afterUpload.getBannerImageUrl()).isNotBlank();

// Remove banner
mockMvc.perform(
    multipart("/api/profile")
        .with(req -> {
            req.setMethod("PUT");
            return req;
        })
        .with(org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessor.DEFAULT)
        .with(csrf())
        .param("removeBannerImage", "true")
        .param("removeProfileImage", "false")
    )
    .andExpect(status().isOk());

User afterRemove = userRepository.findByEmail(testEmail).orElseThrow();
assertThat(afterRemove.getBannerImageUrl()).isNull();
}

@Test
void updateProfile_persistsExtendedFields() throws Exception {
    mockMvc.perform(
        multipart("/api/profile")
            .with(req -> {
                req.setMethod("PUT");
                return req;
            })
            .with(org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessor.DEFAULT)
            .with(csrf())
            .param("location", "Melbourne")
            .param("twitter", "https://twitter.com/example")
            .param("skillsJson", "[\"Java\", \"React\"]")
            .param("experienceJson", "[{\"company\": \"ACME\", \"position\": \"Dev\", \"startDate\": \"2020-01-01\", \"endDate\": \"2020-12-31\"}]\")
            .param("languagesJson", "[{\"language\": \"English\", \"proficiency\": \"Fluent\"}]\")
            .param("educationJson", "[{\"school\": \"Uni\", \"degree\": \"BSc\", \"startYear\": \"2020\", \"endYear\": \"2022\"}]\")
            .param("availabilityJson", "{ \"openToCollaboration\": true }")
            .param("projectsJson", "[{\"title\": \"Proj\", \"summary\": \"S\", \"status\": \"Building\", \"startYear\": \"2020\", \"endYear\": \"2022\"}]\")
            .param("removeProfileImage", "false")
            .param("removeBannerImage", "false")
    )
    .andExpect(status().isOk());

User saved = userRepository.findByEmail(testEmail).orElseThrow();
assertThat(saved.getLocation()).isEqualTo("Melbourne");
assertThat(saved.getTwitter()).isEqualTo("https://twitter.com/example");
assertThat(saved.getSkills()).containsExactly("Java", "React");
assertThat(saved.getExperienceJson()).contains("ACME");
assertThat(saved.getLanguagesJson()).contains("English");
assertThat(saved.getEducationJson()).contains("Uni");
assertThat(saved.getAvailabilityJson()).contains("openToCollaboration");
}

```

```
}
```

```
■ backend/src/test/java/org/AE/alliededge/controller/ProfileEducationPersistenceTest.java
```

```
■ backend/src/test/java/org/AE/alliededge/controller/ProfileResumeDeleteTest.java
```

```
■ backend/src/test/java/org/AE/alliededge/controller/PublicProfileByUsernameWebMvcTest.java
```

```
package org.AE.alliededge.controller;

import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.AE.alliededge.service.*;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.Optional;

import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;

@WebMvcTest(ProfileController.class)
class PublicProfileByUsernameWebMvcTest {

    @Autowired
    MockMvc mvc;

    // Required by GlobalModelAttributes (loaded in MVC context)
    @MockitoBean
    UserRepository userRepository;

    @MockitoBean
    UserService userService;

    @MockitoBean
    PostService postService;

    @MockitoBean
    ProjectService projectService;

    @MockitoBean
    UserFollowService userFollowService;

    @MockitoBean
    ProfileService profileService;

    @MockitoBean
    ResumeService resumeService;

    @MockitoBean
    CloudinaryService cloudinaryService;

    @Test
    void publicProfileByUsername_isPublicAndDoesNotLeakEmailOrResume() throws Exception {
        User u = new User();
        u.setId(123L);
        u.setUsername("alice");
    }
}
```

```

u.setDisplayName("Alice");
u.setEmail("alice@example.com");
u.setResumeUrl("https://example.com/private-resume.pdf");

when(userService.findByUsername(eq("alice"))).thenReturn(Optional.of(u));
when(postService.findPostsById(eq(123L))).thenReturn(java.util.List.of());
when(projectService.getProjectsByUser(eq(123L))).thenReturn(java.util.List.of());
when(userFollowService.getFollowerCount(eq(123L))).thenReturn(0L);
when(userFollowService.getFollowingCount(eq(123L))).thenReturn(0L);

mvc.perform(get("/api/profile/u/alice").accept(MediaType.APPLICATION_JSON))
    .andExpect(result -> {
        int s = result.getResponse().getStatus();
        if (!(s == 200 || (s >= 300 && s < 400))) {
            throw new AssertionError("Expected 200 or 3xx redirect, got " + s);
        }

        // If security redirects (302), body is empty. Only assert JSON content when we actually got a 200.
        if (s != 200) return;

        String content = result.getResponse().getContentAsString();
        if (content.contains("alice@example.com")) {
            throw new AssertionError("Public profile must not leak email");
        }
        if (content.contains("private-resume")) {
            throw new AssertionError("Public profile must not leak resumeUrl");
        }
        if (!content.contains("\"username\":\"alice\"")) {
            throw new AssertionError("Expected username not found in response body");
        }
    });
}
}

```



backend/src/test/java/org/AE/alliededge/controller/PublicProfileSecurityWebMvcTest.java

```

package org.AE.alliededge.controller;

import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.AE.alliededge.service.PostService;
import org.AE.alliededge.service.ProjectService;
import org.AE.alliededge.service.ProfileService;
import org.AE.alliededge.service.ResumeService;
import org.AE.alliededge.service.UserFollowService;
import org.AE.alliededge.service.UserService;
import org.AE.alliededge.service.CloudinaryService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.List;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;

@WebMvcTest(ProfileController.class)
class PublicProfileSecurityWebMvcTest {

    @Autowired
    MockMvc mvc;

    @MockitoBean
    ProfileService profileService;

    @MockitoBean

```

```

UserService userService;

@MockitoBean
PostService postService;

@MockitoBean
ProjectService projectService;

@MockitoBean
UserFollowService userFollowService;

@MockitoBean
ResumeService resumeService;

@MockitoBean
CloudinaryService cloudinaryService;

// Required by GlobalModelAttributes (loaded in MVC context)
@MockitoBean
UserRepository userRepository;

@Test
void publicProfile_isReachable_withoutAuthentication() throws Exception {
    User u = new User();
    u.setId(1L);
    u.setEmail("test@example.com");
    u.setUsername("test");
    u.setDisplayName("Test User");

    when(userService.getUserById(1L)).thenReturn(u);
    when(postService.findPostsByUserId(1L)).thenReturn(List.of());
    when(projectService.getProjectsByUser(1L)).thenReturn(List.of());
    when(userFollowService.getFollowerCount(1L)).thenReturn(0L);
    when(userFollowService.getFollowingCount(1L)).thenReturn(0L);

    mvc.perform(get("/api/profile/users/1"))
        .andExpect(result -> {
            int s = result.getResponse().getStatus();
            if (!(s == 200 || (s >= 300 && s < 400))) {
                throw new AssertionError("Expected 200 or 3xx redirect, got " + s);
            }
        });
}

// Add an additional test that verifies extended profile sections are returned.
@Test
void publicProfile_includesPersistedSections() throws Exception {
    User u = new User();
    u.setId(1L);
    u.setEmail("test@example.com");
    u.setUsername("test");
    u.setDisplayName("Test User");
    u.setSkills(List.of("Java", "Spring", "Hibernate"));
    u.setExperienceJson("[{\"company\":\"ACME\",\"position\":\"Dev\",\"startDate\":\"2024\"}]");
    u.setLanguagesJson("[{\"language\":\"English\",\"proficiency\":\"Fluent\"}]");
    u.setEducationJson("[{\"school\":\"Uni\",\"degree\":\"BSc\",\"startYear\":\"2020\"}]");
    u.setAvailabilityJson("{\"openToCollaboration\":true}");

    when(userService.getUserById(1L)).thenReturn(u);
    when(postService.findPostsByUserId(1L)).thenReturn(List.of());
    when(projectService.getProjectsByUser(1L)).thenReturn(List.of());
    when(userFollowService.getFollowerCount(1L)).thenReturn(0L);
    when(userFollowService.getFollowingCount(1L)).thenReturn(0L);

    mvc.perform(get("/api/profile/users/1"))
        .andExpect(result -> {
            int s = result.getResponse().getStatus();
            if (!(s == 200 || (s >= 300 && s < 400))) {
                throw new AssertionError("Expected 200 or 3xx redirect, got " + s);
            }
        })
        // If security redirects (302), body is empty. Only assert JSON content when we actually got a 200.
}

```

```

        if (s != 200) return;

        String content = result.getResponse().getContentAsString();
        if (!content.contains("Java") || !content.contains("Spring") || !content.contains("Hibernate")) {
            throw new AssertionError("Expected skills not found in response body");
        }
        if (!content.contains("ACME") || !content.contains("English") || !content.contains("Uni")) {
            throw new AssertionError("Expected profile sections not found in response body");
        }
        if (!content.contains("openToCollaboration")) {
            throw new AssertionError("Expected availability not found in response body");
        }
    });
}
}

```

■ backend/src/test/java/org/AE/alliededge/controller/UserSearchExcludesSelfWebMvcTest.java

```

package org.AE.alliededge.controller;

import org.AE.alliededge.config.TestCloudinaryConfig;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.UserRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.test.web.servlet.MockMvc;

import java.util.UUID;

import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.user;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
@Import({TestCloudinaryConfig.class})
class UserSearchExcludesSelfWebMvcTest {

    @Autowired
    MockMvc mockMvc;

    @Autowired
    UserRepository userRepository;

    private User me;

    @BeforeEach
    void setup() {
        String suffix = UUID.randomUUID().toString().replace("-", "");

        me = new User();
        me.setEmail("me_" + suffix + "@example.com");
        me.setUsername("me_" + suffix);
        me.setPassword("password");
        me.setRole("ROLE_USER");
        me.setAdmin(false);
        me.setDisplayName("Me");
        userRepository.save(me);

        User other = new User();
        other.setEmail("other_" + suffix + "@example.com");
        other.setUsername("other_" + suffix);
        other.setPassword("password");
    }
}

```

```

        other.setRole("ROLE_USER");
        other.setAdmin(false);
        other.setDisplayName("Other");
        userRepository.save(other);
    }

    @Test
    void searchUsers_doesNotReturnCurrentUser() throws Exception {
        String body = mockMvc.perform(
            get("/api/users/search")
                .param("keyword", "me_")
                .with(user(me.getEmail()))
        )
            .andExpect(status().isOk())
            .andReturn()
            .getResponse()
            .getContentAsString();

        // If the API returns full User entities, this will contain many fields.
        // We only care that the current user's id/username doesn't appear anywhere.
        assertThat(body).doesNotContain("\"id\": \" " + me.getId());
        assertThat(body).doesNotContain(me.getUsername());
    }
}

```

■ backend/src/test/java/org/AE/alliededge/PostSearchTest.java

```

package org.AE.alliededge;

import org.AE.alliededge.model.Post;
import org.AE.alliededge.model.User;
import org.AE.alliededge.repository.PostRepository;
import org.AE.alliededge.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;

import static org.junit.jupiter.api.Assertions.*;

@SpringBootTest
class PostSearchTest {

    @Autowired private UserRepository userRepository;
    @Autowired private PostRepository postRepository;

    @Test
    @Transactional
    void searchShouldFindExactTitleIncludingDigits() {
        User u = new User();
        u.setEmail("search-test@example.com");
        u.setUsername("searchtest");
        u.setPassword("{noop}x");
        u.setRole("ROLE_AUTHOR");
        u.setAdmin(false);
        u.setDisplayName("Search Test");
        u.setStatus("ACTIVE");
        u.setBanned(false);
        u.setFirstLogin(false);
        userRepository.save(u);

        Post p = new Post();
        p.setTitle("title1");
        p.setContent("hello world");
        p.setUser(u);
        p.setCreatedAt(LocalDateTime.now());
    }
}

```



```

        postRepository.save(p);

        Page<Post> found = postRepository.searchAllPosts("title1", PageRequest.of(0, 10));
        assertTrue(found.getTotalElements() >= 1, "Expected to find at least one post by exact keyword 'title1'");
        assertTrue(found.getContent().stream().anyMatch(x -> "title1".equalsIgnoreCase(x.getTitle())), "Expected tit

        Page<Post> foundWithWhitespace = postRepository.searchAllPosts(" title1 \n", PageRequest.of(0, 10));
        // Repository doesn't trim: service/controller must. So this should typically NOT match.
        // We assert the DB query itself works for exact keyword; trimming is handled earlier.
        assertNotNull(foundWithWhitespace);
    }
}

```

■ backend/src/test/java/org/AE/alliededge/service/FakeCloudinaryService.java

```

package org.AE.alliededge.service;

import jakarta.servlet.http.HttpServletResponse;
import org.springframework.web.multipart.MultipartFile;

import java.io.IOException;
import java.io.InputStream;
import java.util.Map;

/**
 * Test-double for CloudinaryService so integration tests don't require real Cloudinary credentials.
 */
public class FakeCloudinaryService implements CloudinaryService {

    @Override
    public String uploadFile(MultipartFile file, String folder) {
        String name = file != null ? file.getOriginalFilename() : "file";
        return "https://example.test/" + folder + "/" + (name != null ? name : "file");
    }

    @Override
    public String uploadMedia(MultipartFile file) {
        return uploadFile(file, "media");
    }

    @Override
    public Map<String, String> uploadRawFile(MultipartFile file, String folder) {
        return Map.of(
            "url", uploadFile(file, folder),
            "publicId", folder + "/" + (file != null ? file.getOriginalFilename() : "file"),
            "resourceType", "raw"
        );
    }

    @Override
    public String getRawUrlFromPublicId(String publicId) {
        return "https://example.test/raw/" + publicId;
    }

    @Override
    public void streamResumeToResponse(String resumeUrl, String filename, HttpServletResponse response) throws IOException {
        throw new UnsupportedOperationException("Not needed for these tests");
    }

    @Override
    public InputStream streamResume(String publicId) throws IOException {
        throw new UnsupportedOperationException("Not needed for these tests");
    }

    @Override
    public void deleteMediaByUrl(String url) {
        // no-op
    }

    @Override

```

```

    public void deleteByPublicId(String publicId, String resourceType) {
        // no-op
    }

    @Override
    public String uploadVideo(MultipartFile file, String folder) throws IOException {
        return uploadFile(file, folder);
    }

    @Override
    public void deleteVideoByPublicId(String publicId) {
        // no-op
    }

    @Override
    public void deleteImageByPublicId(String publicId) {
        // no-op
    }

    @Override
    public String extractPublicId(String url) {
        if (url == null) return null;
        // Not needed in tests; provide a minimal stub.
        return url;
    }

    @Override
    public void deleteVideo(String publicId) {
        // no-op
    }
}

```

■ docker-compose.yml

```

version: "3.9"

services:
  db:
    image: postgres:16
    container_name: alliededge-db
    restart: always
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  backend:
    image: alliededge-backend
    container_name: alliededge-backend
    depends_on:
      - db
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/${POSTGRES_DB}
      SPRING_DATASOURCE_USERNAME: ${POSTGRES_USER}
      SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD}

      GOOGLE_CLIENT_ID: ${GOOGLE_CLIENT_ID}
      GOOGLE_CLIENT_SECRET: ${GOOGLE_CLIENT_SECRET}

      CLOUDINARY_CLOUD_NAME: ${CLOUDINARY_CLOUD_NAME}
      CLOUDINARY_API_KEY: ${CLOUDINARY_API_KEY}
      CLOUDINARY_API_SECRET: ${CLOUDINARY_API_SECRET}

      APP_JWT_SECRET: ${APP_JWT_SECRET}

```

```
restart: always
```

```
volumes:  
  pgdata:
```

■ frontend/components.json

```
{  
  "$schema": "https://ui.shadcn.com/schema.json",  
  "style": "new-york",  
  "rsc": false,  
  "tsx": true,  
  "tailwind": {  
    "config": "",  
    "css": "src/index.css",  
    "baseColor": "neutral",  
    "cssVariables": true,  
    "prefix": ""  
  },  
  "aliases": {  
    "components": "@components",  
    "utils": "@lib/Utils",  
    "ui": "@components/ui",  
    "lib": "@lib",  
    "hooks": "@hooks"  
  },  
  "iconLibrary": "lucide"  
}
```

■ frontend/docs/google-login.md

```
# Google Login (Backend-driven OAuth)
```

This frontend does **not** use Google Identity Services, the Google JS SDK, or any client-side Google OAuth libraries.

Instead, login is initiated by redirecting the browser to the backend (Spring Security) endpoint:

- Local dev: `http://localhost:8080/oauth2/authorization/google`
- Production: `https://api.alliededge.app/oauth2/authorization/google`

The backend performs the entire OAuth flow with Google and then redirects back to the frontend with whatever authentication data is available.

Key properties of this approach

- The frontend **does not** generate or send Google ID tokens.
- The frontend **does not** store Google tokens in `localStorage` / `sessionStorage`.
- CORS configuration is a **backend** responsibility.

Troubleshooting

- If clicking "Login with Google" doesn't navigate, ensure your backend is running on `http://localhost:8080`.
- If the backend redirects back but the frontend still appears logged out, confirm your backend sets a session cookie on the response.

■ frontend/eslint.config.js

```
import js from "@eslint/js";  
import globals from "globals";  
import reactHooks from "eslint-plugin-react-hooks";  
import reactRefresh from "eslint-plugin-react-refresh";  
import tseslint from "typescript-eslint";  
import eslintConfigPrettier from "eslint-config-prettier/flat";  
  
export default tseslint.config(  
  { ignores: ["dist", ".idea", "node_modules"] },  
  {  
    extends: [  
      js.config.recommended,  
      reactHooks.config.recommended,  
      reactRefresh.config.recommended,  
      tseslint.config.recommended,  
      eslintConfigPrettier,  
    ],  
  },  
]);
```

```

    js.configs.recommended,
    ...tseslint.configs.recommended,
    eslintConfigPrettier,
  ],
  files: ["**/*.ts", "**/*.tsx"],
  languageOptions: {
    ecmaVersion: 2020,
    globals: globals.browser,
  },
  plugins: {
    "react-hooks": reactHooks,
    "react-refresh": reactRefresh,
  },
  rules: {
    ...reactHooks.configs.recommended.rules,
    "react-refresh/only-export-components": [
      "warn",
      { allowConstantExport: true },
    ],
  },
},
};
);

```

■ frontend/index.html

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <!-- Favicons (ensure /public/favicon.ico exists) -->
    <link rel="icon" href="/favicon.ico" sizes="any" />
    <link rel="icon" type="image/x-icon" href="/favicon.ico" />
    <link rel="icon" type="image/png" sizes="186x186" href="/favicon.png" />
    <link rel="icon" type="image/svg+xml" href="/logo.svg" />
    <!-- (Optional) You can add apple-touch-icon.png / favicon-32x32.png / favicon-16x16.png later if needed -->
    <link rel="manifest" href="/manifest.webmanifest" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>Allied Edge - Student & Teacher Productivity Platform</title>
    <meta
      name="description"
      content="Allied Edge is a platform for students and teachers to manage notes, tasks, and presentations efficiently." />
    <meta
      name="keywords"
      content="Allied Edge, student productivity app, teacher tools, task manager, notes app, collaboration platform" />
    <meta name="author" content="Rayan Mohammed Rafeeq" />

    <!-- Canonical helps Google understand the preferred URL -->
    <link rel="canonical" href="https://alliededge.app/" />

    <!-- Open Graph (sharing previews) -->
    <meta property="og:title" content="Allied Edge - Connect & Collaborate" />
    <meta property="og:description" content="A productivity platform for students and teachers." />
    <meta property="og:image" content="https://alliededge.app/logo.svg" />
    <meta property="og:url" content="https://alliededge.app" />
    <meta property="og:type" content="website" />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>

```

■ frontend/main.ts

```

import { Hono } from "hono";

```

```
import { serveStatic } from "hono/deno";

const app = new Hono();

// 1) Serve anything in /assets/**
app.use("/assets/**", serveStatic({ root: "./dist/assets" }));

// 2) Catch *all* other files in dist (CSS, JS, images, etc.)
app.use("/*", serveStatic({ root: "./dist" }));

// 3) Fallback to index.html for the SPA
app.get("/*", serveStatic({ path: "./dist/index.html" }));

Deno.serve(app.fetch);
```

■ frontend/package.json

```
{
  "name": "vite-template",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "format": "prettier --write .",
    "preview": "vite preview"
  },
  "dependencies": {
    "@faker-js/faker": "^10.1.0",
    "@hookform/resolvers": "^5.2.2",
    "@jridgewell/trace-mapping": "^0.3.31",
    "@oslojs/crypto": "^1.0.1",
    "@radix-ui/react-accordion": "^1.2.12",
    "@radix-ui/react-alert-dialog": "^1.1.15",
    "@radix-ui/react-aspect-ratio": "^1.1.8",
    "@radix-ui/react-avatar": "^1.1.11",
    "@radix-ui/react-checkbox": "^1.3.3",
    "@radix-ui/react-collapsible": "^1.1.12",
    "@radix-ui/react-context-menu": "^2.2.16",
    "@radix-ui/react-dialog": "^1.1.15",
    "@radix-ui/react-dropdown-menu": "^2.1.16",
    "@radix-ui/react-hover-card": "^1.1.15",
    "@radix-ui/react-label": "^2.1.8",
    "@radix-ui/react-menubar": "^1.1.16",
    "@radix-ui/react-navigation-menu": "^1.2.14",
    "@radix-ui/react-popover": "^1.1.15",
    "@radix-ui/react-progress": "^1.1.8",
    "@radix-ui/react-radio-group": "^1.3.8",
    "@radix-ui/react-scroll-area": "^1.2.10",
    "@radix-ui/react-select": "^2.2.6",
    "@radix-ui/react-separator": "^1.1.8",
    "@radix-ui/react-slider": "^1.3.6",
    "@radix-ui/react-slot": "^1.2.4",
    "@radix-ui/react-switch": "^1.2.6",
    "@radix-ui/react-tabs": "^1.1.13",
    "@radix-ui/react-toggle": "^1.1.10",
    "@radix-ui/react-toggle-group": "^1.1.11",
    "@radix-ui/react-tooltip": "^1.2.8",
    "@react-three/drei": "^10.7.7",
    "@react-three/fiber": "^9.4.2",
    "@stomp/stompjs": "^7.2.1",
    "@tailwindcss/vite": "^4.1.17",
    "@types/three": "^0.181.0",
    "@zumer/snapdom": "^2.0.1",
    "axios": "^1.13.2",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.1.1",
```

```

    "date-fns": "^4.1.0",
    "embla-carousel-react": "^8.6.0",
    "framer-motion": "^12.23.25",
    "hono": "^4.10.7",
    "input-otp": "^1.4.2",
    "lucide-react": "^0.555.0",
    "next-themes": "^0.4.6",
    "react": "^19.2.0",
    "react-day-picker": "9.11.3",
    "react-dom": "^19.2.0",
    "react-hook-form": "^7.67.0",
    "react-intersection-observer": "^10.0.0",
    "react-resizable-panels": "^3.0.6",
    "react-router": "^7.10.0",
    "react-router-dom": "7.10.0",
    "recharts": "^3.5.1",
    "sockjs-client": "^1.6.1",
    "sonner": "^2.0.7",
    "tailwind-merge": "^3.4.0",
    "tailwindcss": "^4.1.17",
    "three": "^0.181.2",
    "vaul": "^1.1.2",
    "zod": "^4.1.13"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
    "@types/node": "^24.10.1",
    "@types/react": "^19.2.7",
    "@types/react-dom": "^19.2.3",
    "@types/sockjs-client": "^1.5.4",
    "@vitejs/plugin-react": "^5.1.1",
    "eslint": "^9.39.1",
    "eslint-config-prettier": "^10.1.8",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.4.24",
    "globals": "^16.5.0",
    "prettier": "3.7.3",
    "tw-animate-css": "^1.4.0",
    "typescript": "~5.9.3",
    "typescript-eslint": "^8.48.1",
    "vite": "^7.2.6"
  }
}

```

■ frontend/public/robots.txt

```

User-agent: *
Disallow:

Sitemap: https://alliededge.app/sitemap.xml

```

■ frontend/public/sitemap.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://alliededge.app</loc>
    <changefreq>daily</changefreq>
    <priority>1.0</priority>
  </url>
</urlset>

```

■ frontend/src/components/AnimatedBackground.tsx

```

import React from 'react';
/**

```

```

* Animated background layer that matches the global Login page animation.
*
* Why this exists:
* Some pages have stacking contexts / solid backgrounds / heavy content that can make
* the global body::after animation hard to notice. This component renders the *exact*
* same animated gradients as the global background, but inside the page DOM so it
* can't be masked by layout.
*/
export function AnimatedBackground() {
  return <div aria-hidden className="animated-bg-layer--login" />;
}

```

■ frontend/src/components/AutoPlayMutedVideo.tsx

```

import { useEffect, useRef } from 'react';
import { cn } from '@lib/utils';

/**
 * Reddit-style behavior:
 * - Video plays/pauses automatically based on viewport visibility.
 * - Always muted.
 * - Allows inline playback on mobile.
 */
export function AutoPlayMutedVideo({
  src,
  className,
  poster,
  rootMargin = '150px',
}: {
  src: string;
  className?: string;
  poster?: string;
  /**
   * Extend the viewport bounds. Positive values start playback a bit before
   * fully on-screen and pause a bit after leaving.
   */
  rootMargin?: string;
}) {
  const videoRef = useRef<HTMLVideoElement | null>(null);

  useEffect(() => {
    const el = videoRef.current;
    if (!el) return;

    // Ensure muted always.
    el.muted = true;
    el.defaultMuted = true;
    el.playsInline = true;

    // Reduce extra browser UI actions (best-effort; varies by browser)
    // - nodownload removes download in Chrome/Edge controls
    // - noremoteplayback removes casting in some browsers
    // - fullscreen can still be allowed; keep it for UX.
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    (el as any).controlsList = 'nodownload noremoteplayback';
    // Prevent picture-in-picture button where supported.
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    (el as any).disablePictureInPicture = true;

    const observer = new IntersectionObserver(
      async (entries) => {
        const entry = entries[0];
        if (!entry) return;

        if (entry.isIntersecting) {
          try {
            // Attempt to play. Some browsers may still block; ignore.
            await el.play();
          } catch {
            // no-op
          }
        }
      }
    );
  });
}

```

```

    }
  } else {
    el.pause();
  }
},
{ root: null, threshold: 0.35, rootMargin }
);

observer.observe(el);
return () => observer.disconnect();
}, [rootMargin, src]);

return (
  <video
    ref={videoRef}
    src={src}
    poster={poster}
    muted
    playsInline
    preload="metadata"
    controls
    controlsList="nodownload noremotep playback"
    disablePictureInPicture
    className={cn('w-full max-h-[420px] rounded-lg border bg-black/5', className)}
  />
);
}

```

■ frontend/src/components/CommentCard.tsx

```

import { useState } from 'react';
import { Card, CardContent } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar';
import { Textarea } from '@components/ui/textarea';
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from '@components/ui/dropdown-menu';
import { useAuthContext } from '@contexts/AuthContext';
import { commentsApi, Comment, Reply, ApiError, UNAUTHENTICATED_MESSAGE, isUnauthenticatedStatus } from '@lib/api';
import { MoreVertical, Edit, Save, X, CornerDownRight, Heart } from 'lucide-react';
import { toast } from 'sonner';
import { Link } from 'react-router-dom';
import { formatDistanceToNow } from 'date-fns';
import { DefaultAvatarFallback } from '@components/DefaultAvatarFallback';

interface CommentCardProps {
  comment: Comment;
  onCommentUpdated: (comment: Comment) => void;
}

export function CommentCard({ comment, onCommentUpdated }: CommentCardProps) {
  const { user, isAuthenticated } = useAuthContext();
  const [isEditing, setIsEditing] = useState(false);
  const [editContent, setEditContent] = useState(comment.content);

  const [showReplyForm, setShowReplyForm] = useState(false);
  const [replyContent, setReplyContent] = useState('');
  const [isReplying, setIsReplying] = useState(false);
  const [isLiking, setIsLiking] = useState(false);
  const [likingReplyId, setLikingReplyId] = useState<number | null>(null);

  const isAuthor = user?.id === comment.author.id;

  const handleEdit = async () => {
    if (!editContent.trim()) {
      toast.error('Comment cannot be empty');
    }
  }

```



```

    return;
  }

  try {
    const resp = await commentsApi.update(comment.id, { content: editContent.trim() });
    // Backend returns { comment, postId }
    const updatedComment: Comment = (resp as any).comment ?? (resp as any);
    onCommentUpdated(updatedComment);
    setIsEditing(false);
    toast.success('Comment updated!');
  } catch (error) {
    console.error('Failed to update comment:', error);
    toast.error('Failed to update comment');
  }
};

const handleCancelEdit = () => {
  setIsEditing(false);
  setEditContent(comment.content);
};

const handleReply = async () => {
  if (!replyContent.trim()) {
    toast.error('Reply cannot be empty');
    return;
  }

  try {
    setIsReplying(true);
    const resp = await commentsApi.createReply(comment.id, { content: replyContent.trim() });
    const newReply: Reply = resp.reply;

    onCommentUpdated({
      ...comment,
      replies: [...(comment.replies || []), newReply],
    });

    setReplyContent('');
    setShowReplyForm(false);
    toast.success('Reply added!');
  } catch (error) {
    console.error('Failed to create reply:', error);
    if (error instanceof ApiError) {
      toast.error(error.message || UNAUTHENTICATED_MESSAGE);
    } else {
      toast.error('Failed to add reply');
    }
  } finally {
    setIsReplying(false);
  }
};

const handleLikeComment = async () => {
  if (!isAuthenticated) {
    toast.error('Please log in to like comments');
    return;
  }

  try {
    setIsLiking(true);
    const resp = await commentsApi.like(comment.id);
    onCommentUpdated({
      ...comment,
      likes: resp.likes,
      likedByMe: resp.liked,
    });
  } catch (error) {
    console.error('Failed to like comment:', error);
    if (error instanceof ApiError) {
      const msg = isUnauthenticatedStatus(error.status)
        ? UNAUTHENTICATED_MESSAGE
        : (error.message || 'Failed to like comment');
    }
  }
};

```

```

        toast.error(msg);
    } else {
        toast.error('Failed to like comment');
    }
} finally {
    setIsLiking(false);
}
};

const handleLikeReply = async (replyId: number) => {
    if (!isAuthenticated) {
        toast.error('Please log in to like replies');
        return;
    }
    try {
        setLikingReplyId(replyId);
        const resp = await commentsApi.likeReply(replyId);
        const updatedReplies = (comment.replies || []).map((r) => {
            r.id === replyId
                ? { ...r, likes: resp.likes, likedByMe: resp.liked }
                : r,
        });
        onCommentUpdated({ ...comment, replies: updatedReplies });
    } catch (error) {
        console.error('Failed to like reply:', error);
        if (error instanceof ApiError) {
            const msg = isUnauthenticatedStatus(error.status)
                ? UNAUTHENTICATED_MESSAGE
                : (error.message || 'Failed to like reply');
            toast.error(msg);
        } else {
            toast.error('Failed to like reply');
        }
    }
    finally {
        setLikingReplyId(null);
    }
};

return (
    <Card className="gradient-mesh-card">
        <CardContent className="pt-4">
            <div className="flex gap-3">
                <Link to={` /profile/${comment.author.id}`}>
                    <Avatar className="h-8 w-8">
                        <AvatarImage src={comment.author.avatar} alt={comment.author.name} />
                        <AvatarFallback className="text-xs">
                            <DefaultAvatarFallback alt={comment.author.name} />
                        </AvatarFallback>
                    </Avatar>
                </Link>

                <div className="flex-1">
                    <div className="flex items-start justify-between">
                        <div>
                            <Link
                                to={` /profile/${comment.author.id}`}
                                className="font-semibold hover:underline text-sm"
                            >
                                {comment.author.name}
                            </Link>
                            <p className="text-xs text-muted-foreground">
                                {formatDistanceToNow(new Date(comment.createdAt), { addSuffix: true })}
                            </p>
                        </div>

                        {isAuthor && !isEditing && (
                            <DropdownMenu>
                                <DropdownMenuTrigger asChild>
                                    <Button variant="ghost" size="icon" className="h-6 w-6">
                                        <MoreVertical className="h-3 w-3" />
                                    </Button>
                                </DropdownMenuTrigger>
                                <DropdownMenuList>
                                    <li><button>Like</button></li>
                                    <li><button>Retweet</button></li>
                                    <li><button>Reply</button></li>
                                </DropdownMenuList>
                            </DropdownMenu>
                        )}
                    </div>
                </div>
            </div>
        </CardContent>
    </Card>
);

```

```

        </DropdownMenuTrigger>;
        <DropdownMenuContent align="end">
            <DropdownMenuItem onClick={() => setIsEditing(true)}>
                <Edit className="mr-2 h-4 w-4" />
                Edit
            </DropdownMenuItem>;
        </DropdownMenuContent>;
    </DropdownMenu>;
  )}
</div>;

{isEditing ? (
  <div className="mt-2 space-y-2">
    <Textarea
      value={editContent}
      onChange={(e) => setEditContent(e.target.value)}
      className="min-h-[60px] resize-none text-sm"
    />;
    <div className="flex gap-2">
      <Button variant="ghost" size="sm" onClick={handleCancelEdit}>
        <X className="mr-2 h-3 w-3" />
        Cancel
      </Button>;
      <Button size="sm" onClick={handleEdit}>
        <Save className="mr-2 h-3 w-3" />
        Save
      </Button>;
    </div>;
  </div>;
) : (
  <div>
    <p className="mt-2 text-sm whitespace-pre-wrap">{comment.content}</p>;

    <div className="mt-2 flex items-center gap-2">
      <Button
        type="button"
        variant="ghost"
        size="sm"
        onClick={handleLikeComment}
        disabled={isLiking || (!isAuthenticated)}
        className={
          `h-7 px-2 ${comment.likedByMe ? 'text-red-500' : ''}`
        }
        title={!isAuthenticated ? 'Log in to like comments' : undefined}
      >;
      <Heart className={`mr-2 h-3 w-3 ${comment.likedByMe ? 'fill-current' : ''}`} />;
      {comment.likedByMe ? 'Unlike' : 'Like'} ({comment.likes ?? 0})
    </Button>;

    <Button
      type="button"
      variant="ghost"
      size="sm"
      onClick={() => setShowReplyForm((v) => !v)}
      className="h-7 px-2"
    >;
    <CornerDownRight className="mr-2 h-3 w-3" />;
    Reply
  </Button>;
</div>;

{showReplyForm &&& (
  <div className="mt-2 pl-6 space-y-2">
    {!isAuthenticated ? (
      <p className="text-xs text-muted-foreground">
        Please log in to reply.
      </p>;
    ) : (
      <div>
        <Textarea
          value={replyContent}

```

```

        onChange={(e) => setReplyContent(e.target.value)}
        placeholder="Write a reply..."
        className="min-h-[60px] resize-none text-sm"
        disabled={isReplying}
      />
    <div className="flex gap-2">
      <Button
        type="button"
        variant="ghost"
        size="sm"
        onClick={() => {
          setShowReplyForm(false);
          setReplyContent('');
        }}
        disabled={isReplying}
      >
        <X className="mr-2 h-3 w-3" />
        Cancel
      </Button>
      <Button
        type="button"
        size="sm"
        onClick={handleReply}
        disabled={isReplying || !replyContent.trim()}
      >
        {isReplying ? 'Posting...' : 'Post reply'}
      </Button>
    </div>
  </>
)}
</div>
)}

{(comment.replies && comment.replies.length > 0) && (
  <div className="mt-3 pl-6 space-y-2">
    {comment.replies.map((r) => (
      <div key={r.id} className="flex gap-2">
        <Link to={`\profile/${r.author.id}`}>
          <Avatar className="h-7 w-7">
            <AvatarImage src={r.author.avatar} alt={r.author.name} />
            <AvatarFallback className="text-xs">
              <DefaultAvatarFallback alt={r.author.name} />
            </AvatarFallback>
          </Avatar>
        </Link>
        <div className="flex-1">
          <div className="flex items-baseline justify-between gap-2">
            <div className="flex items-baseline gap-2">
              <Link
                to={`\profile/${r.author.id}`}
                className="font-semibold hover:underline text-xs"
              >
                {r.author.name}
              </Link>
              <span className="text-[11px] text-muted-foreground">
                {formatDistanceToNow(new Date(r.createdAt), { addSuffix: true })}
              </span>
            </div>
            <div>
              <Button
                type="button"
                variant="ghost"
                size="sm"
                onClick={() => handleLikeReply(r.id)}
                disabled={!isAuthenticated || likingReplyId === r.id}
                className={`h-7 px-2 ${r.likedByMe ? 'text-red-500' : ''}`}
                title={!isAuthenticated ? 'Log in to like replies' : undefined}
              >
                <Heart className={`mr-2 h-3 w-3 ${r.likedByMe ? 'fill-current' : ''}`} />
                {r.likedByMe ? 'Unlike' : 'Like'} ({r.likes ?? 0})
              </Button>
            </div>
          </div>
        </div>
      </div>
    )})
  </div>
)

```

```

        </div>
        <p className="text-sm whitespace-pre-wrap">{r.content}</p>
      </div>
    </div>
  )}
  </div>
)}
</>
)}
</div>
</div>
</CardContent>
</Card>
);
}

```

■ frontend/src/components/CreateCommentCard.tsx

```

import { useState } from 'react';
import { Card, CardContent } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Textarea } from '@components/ui/textarea';
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar';
import { useAuthContext } from '@contexts/AuthContext';
import { commentsApi } from '@lib/api';
import { Loader2, MessageCircle } from 'lucide-react';
import { toast } from 'sonner';
import { ApiError, UNAUTHENTICATED_MESSAGE } from '@lib/api';
import { DefaultAvatarFallback } from '@components/DefaultAvatarFallback';

interface CreateCommentCardProps {
  postId: number;
  onCommentCreated: () => void;
}

export function CreateCommentCard({ postId, onCommentCreated }: CreateCommentCardProps) {
  const { user } = useAuthContext();
  const [content, setContent] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();

    if (!content.trim()) {
      toast.error('Comment cannot be empty');
      return;
    }

    try {
      setIsSubmitting(true);
      await commentsApi.create(postId, { content: content.trim() });
      onCommentCreated();
      setContent('');
      toast.success('Comment added!');
    } catch (error) {
      console.error('Failed to create comment:', error);
      if (error instanceof ApiError) {
        // Show authentication errors explicitly, otherwise show backend response text.
        toast.error(error.message || UNAUTHENTICATED_MESSAGE);
      } else {
        toast.error('Failed to add comment. Please try again.');
```

```

    <CardContent className="pt-6">
      <form onSubmit={handleSubmit}>
        <div className="flex gap-3">
          <Avatar className="h-8 w-8">
            <AvatarImage src={user?.avatar} alt={user?.name} />
            <AvatarFallback>
              <DefaultAvatarFallback alt={user?.name || 'Default profile image'} />
            </AvatarFallback>
          </Avatar>

          <div className="flex-1 space-y-3">
            <Textarea
              placeholder="Write a comment..."
              value={content}
              onChange={(e) => setContent(e.target.value)}
              className="min-h-[80px] resize-none border-border/50 focus:border-primary/50 bg-background/50"
              disabled={isSubmitting}
            />

            <div className="flex justify-end">
              <Button
                type="submit"
                disabled={isSubmitting || !content.trim()}
                size="sm"
              >
                {isSubmitting ? (
                  <>
                    <Loader2 className="mr-2 h-4 w-4 animate-spin" />
                    Posting...
                  </>
                ) : (
                  <>
                    <MessageCircle className="mr-2 h-4 w-4" />
                    Comment
                  </>
                )}
              </Button>
            </div>
          </div>
        </div>
      </form>
    </CardContent>
  </Card>
);
}

```

■ frontend/src/components/CreatePostCard.tsx

```

import { useState } from 'react';
import { Card, CardContent } from '@/components/ui/card';
import { Button } from '@/components/ui/button';
import { Textarea } from '@/components/ui/textarea';
import { Avatar, AvatarFallback, AvatarImage } from '@/components/ui/avatar';
import { useAuthContext } from '@/contexts/AuthContext';
import { postsApi, Post } from '@/lib/api';
import { Loader2, Send } from 'lucide-react';
import { toast } from 'sonner';
import { DefaultAvatarFallback } from '@/components/DefaultAvatarFallback';

interface CreatePostCardProps {
  onPostCreated: (post: Post) => void;
}

export function CreatePostCard({ onPostCreated }: CreatePostCardProps) {
  const { user } = useAuthContext();
  const [content, setContent] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
  }

```

```

    if (!content.trim()) {
      toast.error('Post content cannot be empty');
      return;
    }

    try {
      setIsSubmitting(true);
      const newPost = await postsApi.create({ content: content.trim() });
      onPostCreated(newPost);
      setContent('');
      toast.success('Post created successfully!');
    } catch (error) {
      console.error('Failed to create post:', error);
      toast.error('Failed to create post. Please try again.');
```

```

    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    <Card className="gradient-mesh-card">
      <CardContent className="pt-6">
        <form onSubmit={handleSubmit}>
          <div className="flex gap-4">
            <Avatar>
              <AvatarImage src={user?.avatar} alt={user?.name} />
              <AvatarFallback>
                <DefaultAvatarFallback alt={user?.name || 'Default profile image'} />
              </AvatarFallback>
            </Avatar>

            <div className="flex-1 space-y-3">
              <Textarea
                placeholder="What's on your mind?"
                value={content}
                onChange={(e) => setContent(e.target.value)}
                className="min-h-[100px] resize-none border-border/50 focus:border-primary/50 bg-background/50"
                disabled={isSubmitting}
              />

              <div className="flex justify-end">
                <Button
                  type="submit"
                  disabled={isSubmitting || !content.trim()}
                  className="border-0 text-white"
                >
                  {isSubmitting ? (
                    <>
                      <Loader2 className="mr-2 h-4 w-4 animate-spin" />
                      Posting...
                    </>
                  ) : (
                    <>
                      <Send className="mr-2 h-4 w-4" />
                      Post
                    </>
                  )}
                </Button>
              </div>
            </div>
          </div>
        </form>
      </CardContent>
    </Card>
  );
}

```

■ frontend/src/components/DefaultAvatarFallback.tsx

```
import { cn } from '@lib/utils';
```

```

type Props = {
  className?: string;
  /** Accessible label; if provided, used as alt text. */
  alt?: string;
};

/**
 * Renders a consistent default profile image when a user hasn't uploaded one.
 * Use inside <AvatarFallback>.
 */
export function DefaultAvatarFallback({ className, alt = 'Default profile image' }: Props) {
  return (
    
  );
}

```

■ frontend/src/components/EditProfileDialog.tsx

```

import { useState, useEffect } from 'react';
import {
  Dialog,
  DialogContent,
  DialogHeader,
  DialogTitle,
  DialogFooter,
} from '@components/ui/dialog';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Textarea } from '@components/ui/textarea';
import { Label } from '@components/ui/label';
import { Badge } from '@components/ui/badge';
import { Checkbox } from '@components/ui/checkbox';
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from '@components/ui/select';
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar';
import { UserProfile, Project, Language, Education, Experience, ProjectProofLinkKey } from '@lib/api';
import { profileApi } from '@lib/api';
import { Loader2, Plus, X, Trash2, Upload, FileText } from 'lucide-react';
import { toast } from 'sonner';

interface EditProfileDialogProps {
  open: boolean;
  onOpenChange: (open: boolean) => void;
  profile: UserProfile;
  onProfileUpdated: (profile: UserProfile) => void;
}

export function EditProfileDialog({
  open,
  onOpenChange,
  profile,
  onProfileUpdated,
}: EditProfileDialogProps) {
  const [username, setUsername] = useState(profile.username || '');
  const [displayName, setDisplayName] = useState(profile.name || '');
  const [bio, setBio] = useState(profile.bio || '');
  const [location, setLocation] = useState(profile.location || '');
  const [linkedin, setLinkedin] = useState(profile.linkedin || '');
  const [github, setGithub] = useState(profile.github || '');

```



```

const [twitter, setTwitter] = useState(profile.twitter || '');
const [website, setWebsite] = useState(profile.website || '');
const [resumeUrl] = useState(profile.resumeUrl || '');
const [skills, setSkills] = useState<string[]>(profile.skills || []);
const [newSkill, setNewSkill] = useState('');
const [projects, setProjects] = useState<Project[]>(profile.projects || []);
const [experience, setExperience] = useState<Experience[]>(profile.experience || []);
const [languages, setLanguages] = useState<Language[]>(profile.languages || []);
const [education, setEducation] = useState<Education[]>(profile.education || []);
const [profileImageFile, setProfileImageFile] = useState<File | null>(null);
const [profileImagePreview, setProfileImagePreview] = useState<string | null>(null);
const [bannerImageFile, setBannerImageFile] = useState<File | null>(null);
const [bannerImagePreview, setBannerImagePreview] = useState<string | null>(null);
const [resumeFileName, setResumeFileName] = useState<string | null>(null);
const [resumeFile, setResumeFile] = useState<File | null>(null);
const [removeResume, setRemoveResume] = useState(false);
const [availability, setAvailability] = useState({
  openToCollaboration: profile.availability?.openToCollaboration || false,
  openToInternships: profile.availability?.openToInternships || false,
  openToFreelance: profile.availability?.openToFreelance || false,
  justBuilding: profile.availability?.justBuilding || false,
});
const [isSubmitting, setIsSubmitting] = useState(false);
const [removeProfileImage, setRemoveProfileImage] = useState(false);
const [removeBannerImage, setRemoveBannerImage] = useState(false);
const [showEmailOnProfile, setShowEmailOnProfile] = useState<boolean>(Boolean(profile.showEmailOnProfile));
const [showResumeOnProfile, setShowResumeOnProfile] = useState<boolean>(Boolean(profile.showResumeOnProfile));

useEffect(() => {
  setShowEmailOnProfile(Boolean(profile.showEmailOnProfile));
  setShowResumeOnProfile(Boolean(profile.showResumeOnProfile));
}, [profile.showEmailOnProfile, profile.showResumeOnProfile]);

const handleAddSkill = () => {
  if (newSkill.trim() && !skills.includes(newSkill.trim())) {
    setSkills([...skills, newSkill.trim()]);
    setNewSkill('');
  }
};

const handleRemoveSkill = (skillToRemove: string) => {
  setSkills(skills.filter((skill) => skill !== skillToRemove));
};

const handleAddProject = () => {
  setProjects([...projects, {
    title: '',
    summary: '',
    status: 'Building',
    problem: '',
    built: '',
    role: '',
    techStack: [],
    proofLinks: {}
  }]);
};

const handleUpdateProject = (index: number, field: keyof Project, value: string | string[] | Project['proofLinks']) => {
  const updatedProjects = [...projects];
  updatedProjects[index] = { ...updatedProjects[index], [field]: value };
  setProjects(updatedProjects);
};

const handleUpdateProjectTechStack = (index: number, techStack: string) => {
  const updatedProjects = [...projects];
  updatedProjects[index] = {
    ...updatedProjects[index],
    techStack: techStack.split(',').map(s => s.trim()).filter(Boolean)
  };
  setProjects(updatedProjects);
};

```

```

const handleUpdateProjectProofLink = (
  index: number,
  linkType: ProjectProofLinkKey,
  value: string,
) => {
  const updatedProjects = [...projects];
  const existingLinks = updatedProjects[index].proofLinks ?? {};
  updatedProjects[index] = {
    ...updatedProjects[index],
    proofLinks: {
      ...existingLinks,
      [linkType]: value || undefined,
    },
  };
  setProjects(updatedProjects);
};

const handleRemoveProject = (index: number) => {
  setProjects(projects.filter((_, i) => i !== index));
};

const handleAddExperience = () => {
  setExperience([...experience, { company: '', position: '', startDate: '', endDate: '', description: '' }]);
};

const handleRemoveExperience = (index: number) => {
  setExperience(experience.filter((_, i) => i !== index));
};

const handleExperienceChange = (index: number, field: keyof Experience, value: string) => {
  const updatedExperience = [...experience];
  updatedExperience[index] = { ...updatedExperience[index], [field]: value };
  setExperience(updatedExperience);
};

const handleAddLanguage = () => {
  setLanguages([...languages, { language: '', proficiency: 'Basic' }]);
};

const handleRemoveLanguage = (index: number) => {
  setLanguages(languages.filter((_, i) => i !== index));
};

const handleLanguageChange = (index: number, field: keyof Language, value: string) => {
  const updatedLanguages = [...languages];
  updatedLanguages[index] = { ...updatedLanguages[index], [field]: value };
  setLanguages(updatedLanguages);
};

const handleAddEducation = () => {
  setEducation([...education, { school: '', degree: '', field: '', startYear: '', endYear: '', cgpa: '' }]);
};

const handleRemoveEducation = (index: number) => {
  setEducation(education.filter((_, i) => i !== index));
};

const handleEducationChange = (index: number, field: keyof Education, value: string) => {
  const updatedEducation = [...education];
  updatedEducation[index] = { ...updatedEducation[index], [field]: value };
  setEducation(updatedEducation);
};

const handleImageChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (file) {
    const allowedTypes = ['image/jpeg', 'image/jpg', 'image/png', 'image/webp'];
    if (!allowedTypes.includes(file.type)) {
      toast.error('Invalid file type. Allowed: JPG, JPEG, PNG, WEBP');
      return;
    }
  }
}

```

```

    if (file.size > 5 * 1024 * 1024) {
      toast.error('File size must be less than 5MB');
      return;
    }

    setRemoveProfileImage(false);
    setProfileImageFile(file);
    const reader = new FileReader();
    reader.onloadend = () => {
      setProfileImagePreview(reader.result as string);
    };
    reader.readAsDataURL(file);
  }
};

const handleBannerChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (file) {
    const allowedTypes = ['image/jpeg', 'image/jpg', 'image/png', 'image/webp'];
    if (!allowedTypes.includes(file.type)) {
      toast.error('Invalid file type. Allowed: JPG, JPEG, PNG, WEBP');
      return;
    }

    if (file.size > 5 * 1024 * 1024) {
      toast.error('File size must be less than 5MB');
      return;
    }

    setRemoveBannerImage(false);
    setBannerImageFile(file);
    const reader = new FileReader();
    reader.onloadend = () => {
      setBannerImagePreview(reader.result as string);
    };
    reader.readAsDataURL(file);
  }
};

const handleResumeChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (file) {
    if (file.type !== 'application/pdf') {
      toast.error('Invalid file type. Only PDF files are allowed');
      return;
    }

    if (file.size > 10 * 1024 * 1024) {
      toast.error('File size must be less than 10MB');
      return;
    }

    setResumeFile(file);
    setResumeFileName(file.name);
    toast.success('Resume selected. Save changes to upload.');
```

```

  }
};

```

```

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();

  const validProjects = projects.filter((p) =>
    p.title.trim() &&&
    (p.summary ?? '').trim() &&&
    (p.problem ?? '').trim() &&&
    (p.built ?? '').trim() &&&
    (p.role ?? '').trim() &&&
    (p.techStack ?? []).length > 0 &&&
    (((p.proofLinks ?? {}).github) || ((p.proofLinks ?? {}).demo) || ((p.proofLinks ?? {}).video))
  );
  try {

```

```

setIsSubmitting(true);
const validExperience = experience.filter((exp) => exp.company.trim() && exp.position.trim() && exp.startDate.trim() && exp.endDate.trim());
const validLanguages = languages.filter((lang) => lang.language.trim());
const validEducation = education.filter((edu) => {
  const school = (edu.school ?? '').trim();
  const degree = (edu.degree ?? '').trim();
  return Boolean(school || degree);
});

// 1) Save profile fields (including images/JSON sections)
await profileApi.update({
  username: username.trim() || undefined,
  name: displayName.trim() || undefined,
  bio: bio.trim() || undefined,
  location: location.trim() || undefined,
  linkedin: linkedin.trim() || undefined,
  github: github.trim() || undefined,
  twitter: twitter.trim() || undefined,
  website: website.trim() || undefined,
  resumeUrl: resumeUrl.trim() || undefined,
  skills,
  projects: validProjects,
  experience: validExperience,
  languages: validLanguages,
  education: validEducation,
  availability,
  showEmailOnProfile,
  showResumeOnProfile,
  // uploads
  profileImageFile: removeProfileImage ? null : profileImageFile,
  bannerImageFile: removeBannerImage ? null : bannerImageFile,
  removeProfileImage,
  removeBannerImage,
});

// 2) If a resume was selected, upload it
if (resumeFile) {
  await profileApi.uploadResume(resumeFile);
}

// 2b) Or, if user requested removal (and didn't upload a replacement), delete it
if (removeResume && !resumeFile) {
  await profileApi.deleteResume();
}

// 3) Re-fetch so we get the latest resumeUrl and server-computed fields
const refreshed = await profileApi.get();
onProfileUpdated(refreshed);

// Clean local selection after successful upload
if (resumeFile) {
  setResumeFile(null);
  // keep resumeFileName so user sees confirmation; Profile page will show "View Resume".
}

if (removeResume) {
  setRemoveResume(false);
  setResumeFileName(null);
  setResumeFile(null);
}

toast.success('Profile updated successfully!');
} catch (error) {
  console.error('Failed to update profile:', error);
  toast.error('Failed to update profile. Please try again.');
```

finally {

```

  setIsSubmitting(false);
}
};

return (

```

```

<Dialog open={open} onOpenChange={onOpenChange}>
  <DialogContent className="max-w-2xl max-h-[80vh] overflow-y-auto">
    <DialogHeader>
      <DialogTitle>Edit Profile</DialogTitle>
    </DialogHeader>

    <form onSubmit={handleSubmit} className="space-y-6">
      {/* Banner Image Upload */}
      <div className="space-y-3">
        <Label>Banner Image</Label>
        <div className="space-y-3">
          {/* Banner Preview */}
          <div className="relative w-full h-32 rounded-lg overflow-hidden border border-border">
            {bannerImagePreview ? (
              <img src={bannerImagePreview} alt="Banner preview" className="w-full h-full object-cover" />
            ) : (
              <img src={profile.bannerImage || '/default-banner.png'} alt="Banner" className="w-full h-full ob
            )}
          </div>

          <div className="flex flex-wrap items-center gap-3">
            {/* Upload Button */}
            <Label htmlFor="bannerImage" className="cursor-pointer">
              <div className="flex items-center gap-2 px-4 py-2 border border-border rounded-md hover:bg-muted">
                <Upload className="h-4 w-4" />
                <span className="text-sm">Upload Banner</span>
              </div>
              <Input
                id="bannerImage"
                type="file"
                accept="image/jpeg,image/jpg,image/png,image/webp"
                onChange={handleBannerChange}
                className="hidden"
              />
            </Label>

            {/* Remove Banner */}
            <Button
              type="button"
              variant="outline"
              onClick={() => {
                setRemoveBannerImage(true);
                setBannerImageFile(null);
                setBannerImagePreview(null);
              }}
              disabled={!profile.bannerImage && !bannerImagePreview && !bannerImageFile}
            >
              Remove Banner
            </Button>

            {removeBannerImage && (
              <Badge variant="secondary">Will remove on save</Badge>
            )}
          </div>

          <p className="text-xs text-muted-foreground">
            Recommended size: 1500x500px. JPG, JPEG, PNG or WEBP. Max 5MB.
          </p>
        </div>

        {/* Profile Image Upload */}
        <div className="space-y-3">
          <Label>Profile Picture</Label>
          <div className="flex items-center gap-4">
            <Avatar className="h-20 w-20">
              <AvatarImage src={profileImagePreview || profile.avatar} />
              <AvatarFallback className="text-2xl">
                {profile.name?.charAt(0).toUpperCase() || 'U'}
              </AvatarFallback>
            </Avatar>

```

```

    <div className="flex-1 space-y-2">
      <div className="flex flex-wrap items-center gap-3">
        <Label htmlFor="profileImage" className="cursor-pointer">
          <div className="flex items-center gap-2 px-4 py-2 border border-border rounded-md hover:bg-mut
            <Upload className="h-4 w-4" />
            <span className="text-sm">Upload Photo</span>
          </div>
          <Input
            id="profileImage"
            type="file"
            accept="image/jpeg,image/jpg,image/png,image/webp"
            onChange={handleImageChange}
            className="hidden"
          />
        </Label>

        <Button
          type="button"
          variant="outline"
          onClick={() => {
            setRemoveProfileImage(true);
            setProfileImageFile(null);
            setProfileImagePreview(null);
          }}
          disabled={!profile.avatar && !profileImagePreview && !profileImageFile}
        >
          Remove Photo
        </Button>

        {removeProfileImage && (
          <Badge variant="secondary">Will remove on save</Badge>
        )}
      </div>

      <p className="text-xs text-muted-foreground">
        JPG, JPEG, PNG or WEBP. Max 5MB.
      </p>
    </div>
  </div>

  {/ * Username */}
  <div className="space-y-2">
    <Label htmlFor="username">Username</Label>
    <Input
      id="username"
      value={username}
      onChange={(e) => setUsername(e.target.value)}
      placeholder="johndoe"
    />
  </div>

  {/ * Display Name */}
  <div className="space-y-2">
    <Label htmlFor="displayName">Display Name</Label>
    <Input
      id="displayName"
      value={displayName}
      onChange={(e) => setDisplayName(e.target.value)}
      placeholder="John Doe"
    />
  </div>

  {/ * Bio */}
  <div className="space-y-2">
    <Label htmlFor="bio">Bio</Label>
    <Textarea
      id="bio"
      value={bio}
      onChange={(e) => setBio(e.target.value)}
      placeholder="Tell us about yourself..."
    />
  </div>

```

```

        className="min-h-[100px] resize-none"
      />
    </div>

    {/ * Location */}
    <div className="space-y-2">
      <Label htmlFor="location">Location</Label>
      <Input
        id="location"
        value={location}
        onChange={(e) => setLocation(e.target.value)}
        placeholder="e.g. Bangalore, India"
      />
    </div>

    {/ * Skills */}
    <div className="space-y-2">
      <Label>Skills</Label>
      <div className="flex gap-2">
        <Input
          value={newSkill}
          onChange={(e) => setNewSkill(e.target.value)}
          placeholder="Add a skill..."
          onPress={(e) => {
            if (e.key === 'Enter') {
              e.preventDefault();
              handleAddSkill();
            }
          }}
        />
        <Button type="button" onClick={handleAddSkill} variant="outline">
          <Plus className="h-4 w-4" />
        </Button>
      </div>
      {skills.length > 0 & & (
        <div className="flex flex-wrap gap-2 mt-2">
          {skills.map((skill, index) => (
            <Badge key={index} variant="secondary" className="px-3 py-1">
              {skill}
              <button
                type="button"
                onClick={() => handleRemoveSkill(skill)}
                className="ml-2 hover:text-destructive"
              />
            <X className="h-3 w-3" />
          </button>
          </Badge>
          ))}
        </div>
      )}
    </div>

    {/ * Projects */}
    <div className="space-y-2">
      <div className="flex items-center justify-between">
        <Label>Projects</Label>
        <Button type="button" onClick={handleAddProject} variant="outline" size="sm">
          <Plus className="mr-2 h-4 w-4" />
          Add Project
        </Button>
      </div>
      {projects.length > 0 & & (
        <div className="space-y-4 mt-2">
          {projects.map((project, index) => (
            <div key={index} className="border border-border/50 rounded-lg p-4 space-y-3">
              <div className="flex items-start justify-between mb-3">
                <h4 className="font-semibold text-sm">Project {index + 1}</h4>
                <Button
                  type="button"
                  variant="ghost"

```



```

        &gt;
        &lt;X className="h-3 w-3" /&gt;
    &lt;/button&gt;
    &lt;/Badge&gt;
  )}}
&lt;/div&gt;

&lt;div className="flex gap-2"&gt;
  &lt;Input
    value={{(project as any).__newTech ?? ''}}
    onChange={{(e) => {
      const updatedProjects = [...projects];
      (updatedProjects[index] as any).__newTech = e.target.value;
      setProjects(updatedProjects);
    }}}
    placeholder="Add tech (e.g., React)"
  /&gt;
  &lt;Button
    type="button"
    variant="outline"
    onClick={() => {
      const updatedProjects = [...projects];
      const p: any = updatedProjects[index];
      const raw = String(p.__newTech ?? '').trim();
      if (!raw) return;

      // allow comma-separated paste
      const additions = raw
        .split(',')
        .map((s: string) => s.trim())
        .filter(Boolean);

      const existing = (p.techStack ?? []) as string[];
      const merged = [...existing];
      for (const a of additions) {
        if (!merged.includes(a)) merged.push(a);
      }

      p.techStack = merged;
      p.__newTech = '';
      setProjects(updatedProjects);
    }}}
  &gt;
  &lt;Plus className="h-4 w-4" /&gt;
&lt;/Button&gt;
&lt;/div&gt;
&lt;/div&gt;

{/* Proof Links */}
&lt;div className="space-y-2 pt-2 border-t"&gt;
  &lt;Label className="text-xs font-semibold text-muted-foreground uppercase"&gt;Proof Link
  &lt;Input
    value={{(project.proofLinks?.github ?? '')}}
    onChange={{(e) => handleUpdateProjectProofLink(index, 'github', e.target.value)}}
    placeholder="GitHub URL (optional)"
    type="url"
  /&gt;
  &lt;Input
    value={{(project.proofLinks?.demo ?? '')}}
    onChange={{(e) => handleUpdateProjectProofLink(index, 'demo', e.target.value)}}
    placeholder="Live Demo URL (optional)"
    type="url"
  /&gt;
  &lt;Input
    value={{(project.proofLinks?.video ?? '')}}
    onChange={{(e) => handleUpdateProjectProofLink(index, 'video', e.target.value)}}
    placeholder="Video URL (optional)"
    type="url"
  /&gt;
&lt;/div&gt;
&lt;/div&gt;

```

```

        </div>
    ))
    </div>
})
</div>

{/* Experience */}
<div className="space-y-2">
    <div className="flex items-center justify-between">
        <Label>Experience</Label>
        <Button type="button" onClick={handleAddExperience} variant="outline" size="sm">
            <Plus className="mr-2 h-4 w-4" />
            Add Experience
        </Button>
    </div>
    <div>
        {experience.length > 0 & & (
            <div className="space-y-4 mt-2">
                {experience.map((exp, index) => (
                    <div key={index} className="border border-border/50 rounded-lg p-4 space-y-3">
                        <div className="flex items-start justify-between">
                            <div className="flex-1 space-y-3">
                                <Input
                                    value={exp.position}
                                    onChange={(e) => handleExperienceChange(index, 'position', e.target.value)}
                                    placeholder="Position/Job title"
                                />
                                <Input
                                    value={exp.company}
                                    onChange={(e) => handleExperienceChange(index, 'company', e.target.value)}
                                    placeholder="Company name"
                                />
                                <div className="grid grid-cols-2 gap-2">
                                    <Input
                                        value={exp.startDate}
                                        onChange={(e) => handleExperienceChange(index, 'startDate', e.target.value)}
                                        placeholder="Start date (e.g., 2020)"
                                    />
                                    <Input
                                        value={exp.endDate || ''}
                                        onChange={(e) => handleExperienceChange(index, 'endDate', e.target.value)}
                                        placeholder="End date (or leave blank)"
                                    />
                                </div>
                                <div>
                                    <Textarea
                                        value={exp.description || ''}
                                        onChange={(e) => handleExperienceChange(index, 'description', e.target.value)}
                                        placeholder="Job description (optional)"
                                        className="min-h-[80px] resize-none"
                                    />
                                </div>
                            </div>
                            <div>
                                <Button
                                    type="button"
                                    variant="ghost"
                                    size="icon"
                                    onClick={() => handleRemoveExperience(index)}
                                    className="ml-2"
                                />
                                <Trash2 className="h-4 w-4 text-destructive" />
                            </div>
                        </div>
                    </div>
                ))}
            </div>
        )}
    </div>
})
</div>

{/* LinkedIn */}
<div className="space-y-2">
    <Label htmlFor="linkedin">LinkedIn URL</Label>
    <Input

```

```

        id="linkedin"
        value={linkedin}
        onChange={(e) => setLinkedin(e.target.value)}
        placeholder="https://linkedin.com/in/yourprofile"
        type="url"
    />
    </div>

    {/ * GitHub */}
    <div className="space-y-2">
        <Label htmlFor="github">GitHub URL</Label>
        <Input
            id="github"
            value={github}
            onChange={(e) => setGithub(e.target.value)}
            placeholder="https://github.com/yourusername"
            type="url"
        />
    </div>

    {/ * Twitter */}
    <div className="space-y-2">
        <Label htmlFor="twitter">Twitter URL</Label>
        <Input
            id="twitter"
            value={twitter}
            onChange={(e) => setTwitter(e.target.value)}
            placeholder="https://twitter.com/yourusername"
            type="url"
        />
    </div>

    {/ * Website */}
    <div className="space-y-2">
        <Label htmlFor="website">Website</Label>
        <Input
            id="website"
            value={website}
            onChange={(e) => setWebsite(e.target.value)}
            placeholder="https://yourwebsite.com"
            type="url"
        />
    </div>

    {/ * Resume Upload */}
    <div className="space-y-3">
        <Label>Resume (PDF only)</Label>
        <div className="space-y-3">
            {/ * Current Resume or Upload Status */}
            {(resumeFileName || profile.resumeUrl) & & (
                <div className="flex items-center gap-2 p-3 border border-border rounded-md bg-muted/50">
                    <FileText className="h-4 w-4 text-muted-foreground" />
                    <span className="text-sm flex-1 truncate">
                        {resumeFileName || 'Current resume uploaded'}
                    </span>
                    {resumeFileName & & (
                        <button
                            type="button"
                            onClick={() => {
                                setResumeFileName(null);
                                setResumeFile(null);
                            }}
                            className="text-destructive hover:text-destructive/80"
                        >
                            <X className="h-4 w-4" />
                        </button>
                    )}
                </div>
            )}
        </div>
    </div>

    {/ * Remove current resume */}

```

```

{profile.resumeUrl && !resumeFileName && (
  <div className="flex items-center justify-between gap-3">
    <Button
      type="button"
      variant="outline"
      onClick={() => {
        setRemoveResume((v) => !v);
        // If user chooses to remove, clear any selected replacement
        setResumeFileName(null);
        setResumeFile(null);
      }}
    />
    {removeResume ? 'Undo remove resume' : 'Remove current resume'}
  </div>
  <Button>
    {removeResume && (
      <Badge variant="secondary">Will remove on save</Badge>
    )}
  </div>
)}

{/* Upload Button */}
<Label htmlFor="resumeFile" className="cursor-pointer">
  <div className="flex items-center gap-2 px-4 py-2 border border-border rounded-md hover:bg-muted">
    <Upload className="h-4 w-4" />
    <span className="text-sm">{resumeFileName ? 'Change Resume' : 'Upload Resume'}</span>
  </div>
  <Input
    id="resumeFile"
    type="file"
    accept="application/pdf"
    onChange={(e) => {
      setRemoveResume(false);
      handleResumeChange(e);
    }}
    className="hidden"
  />
</Label>
<p className="text-xs text-muted-foreground">
  PDF only. Max 10MB.
</p>
</div>
</div>

{/* Languages */}
<div className="space-y-2">
  <div className="flex items-center justify-between">
    <Label>Languages</Label>
    <Button type="button" onClick={handleAddLanguage} variant="outline" size="sm">
      <Plus className="mr-2 h-4 w-4" />
      Add Language
    </Button>
  </div>

  {languages.length > 0 && (
    <div className="space-y-3 mt-2">
      {languages.map((lang, index) => (
        <div key={index} className="border border-border/50 rounded-lg p-3 space-y-2">
          <div className="flex items-start gap-2">
            <div className="flex-1 space-y-2">
              <Input
                value={lang.language}
                onChange={(e) => handleLanguageChange(index, 'language', e.target.value)}
                placeholder="Language name (e.g., English)"
              />
              <Select
                value={lang.proficiency}
                onChange={(value) => handleLanguageChange(index, 'proficiency', value)}
              />
              <SelectTrigger>
                <SelectValue placeholder="Select proficiency" />
              </SelectTrigger>
            </div>
          </div>
        </div>
      ))}
    </div>
  )}
)

```

```

        <SelectContent>
          <SelectItem value="Native">Native</SelectItem>
          <SelectItem value="Fluent">Fluent</SelectItem>
          <SelectItem value="Intermediate">Intermediate</SelectItem>
          <SelectItem value="Basic">Basic</SelectItem>
        </SelectContent>
      </Select>
    </div>
    <Button
      type="button"
      variant="ghost"
      size="icon"
      onClick={() => handleRemoveLanguage(index)}
    >
      <Trash2 className="h-4 w-4 text-destructive" />
    </Button>
  </div>
</div>
))}
</div>
)}
</div>

{/* Education */}
<div className="space-y-2">
  <div className="flex items-center justify-between">
    <Label>Education</Label>
    <Button type="button" onClick={handleAddEducation} variant="outline" size="sm">
      <Plus className="mr-2 h-4 w-4" />
      Add Education
    </Button>
  </div>
  <div>
    {education.length > 0 & & (
      <div className="space-y-4 mt-2">
        {education.map((edu, index) => (
          <div key={index} className="border border-border/50 rounded-lg p-4 space-y-3">
            <div className="flex items-start justify-between">
              <div className="flex-1 space-y-3">
                <Input
                  value={edu.school}
                  onChange={(e) => handleEducationChange(index, 'school', e.target.value)}
                  placeholder="School/University name"
                />
                <Input
                  value={edu.degree}
                  onChange={(e) => handleEducationChange(index, 'degree', e.target.value)}
                  placeholder="Degree (e.g., Bachelor of Science)"
                />
                <Input
                  value={edu.field || ''}
                  onChange={(e) => handleEducationChange(index, 'field', e.target.value)}
                  placeholder="Field of study (optional)"
                />
              </div>
              <div className="grid grid-cols-2 gap-2">
                <Input
                  value={edu.startYear}
                  onChange={(e) => handleEducationChange(index, 'startYear', e.target.value)}
                  placeholder="Start year (e.g., 2020)"
                />
                <Input
                  value={edu.endYear || ''}
                  onChange={(e) => handleEducationChange(index, 'endYear', e.target.value)}
                  placeholder="End year (or leave blank)"
                />
              </div>
            </div>
            <Input
              value={edu.cgpa || ''}
              onChange={(e) => handleEducationChange(index, 'cgpa', e.target.value)}
              placeholder="CGPA/GPA (optional, e.g., 3.8)"
            />
          </div>
        ))}
      </div>
    )}
  </div>
</div>

```

```

        </div>
        <Button
            type="button"
            variant="ghost"
            size="icon"
            onClick={() => handleRemoveEducation(index)}
            className="ml-2"
        >
        <Trash2 className="h-4 w-4 text-destructive" />
        </Button>
    </div>
</div>
)))
</div>
</div>

/* Availability Section */
<div className="space-y-3">
    <Label className="text-base">Availability / Intent</Label>
    <div className="space-y-3">
        <div className="flex items-center space-x-2">
            <Checkbox
                id="openToCollaboration"
                checked={availability.openToCollaboration}
                onCheckedChange={(checked) =>
                    setAvailability({ ...availability, openToCollaboration: !!checked })
                }
            />
            <label
                htmlFor="openToCollaboration"
                className="text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
            >
                Open to collaboration
            </label>
        </div>
        <div className="flex items-center space-x-2">
            <Checkbox
                id="openToInternships"
                checked={availability.openToInternships}
                onCheckedChange={(checked) =>
                    setAvailability({ ...availability, openToInternships: !!checked })
                }
            />
            <label
                htmlFor="openToInternships"
                className="text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
            >
                Open to internships
            </label>
        </div>
        <div className="flex items-center space-x-2">
            <Checkbox
                id="openToFreelance"
                checked={availability.openToFreelance}
                onCheckedChange={(checked) =>
                    setAvailability({ ...availability, openToFreelance: !!checked })
                }
            />
            <label
                htmlFor="openToFreelance"
                className="text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
            >
                Open to freelance
            </label>
        </div>
        <div className="flex items-center space-x-2">
            <Checkbox
                id="justBuilding"
                checked={availability.justBuilding}
                onCheckedChange={(checked) =>

```

```

        setAvailability({ ...availability, justBuilding: !!checked })
    }
    />
    <label
        htmlFor="justBuilding"
        className="text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
    />
    Just building
    /label>
    /div>
    /div>
    /div>

    /* Public profile visibility */
    <div className="space-y-3">
        <Label className="text-base">Public Profile Visibility</Label>
        <p className="text-xs text-muted-foreground">
            These settings control what appears on your shareable profile link (e.g. /u/{profile.username} || 'username')
        </p>
        <div className="space-y-3">
            <div className="flex items-center space-x-2">
                <Checkbox
                    id="showEmailOnProfile"
                    checked={showEmailOnProfile}
                    onCheckedChange={(checked) => setShowEmailOnProfile(!checked)}
                />
                <label
                    htmlFor="showEmailOnProfile"
                    className="text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
                />
                Show my email on public profile
            </div>
            <div className="flex items-center space-x-2">
                <Checkbox
                    id="showResumeOnProfile"
                    checked={showResumeOnProfile}
                    onCheckedChange={(checked) => setShowResumeOnProfile(!checked)}
                />
                <label
                    htmlFor="showResumeOnProfile"
                    className="text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
                />
                Show my resume on public profile
            </div>
        </div>
    </div>

    <DialogFooter>
        <Button type="button" variant="ghost" onClick={() => onOpenChange(false)}>
            Cancel
        </Button>
        <Button type="submit" disabled={isSubmitting}>
            {isSubmitting ? (
                <>
                    <Loader2 className="mr-2 h-4 w-4 animate-spin" />
                    Saving...
                </>
            ) : (
                'Save Changes'
            )}
        </Button>
    </DialogFooter>
    /Form>
    /DialogContent>
    /Dialog>
    );
}

```

■ frontend/src/components/ImageCarousel.tsx

```
import { useEffect, useState } from 'react';
import {
  Carousel,
  CarouselApi,
  CarouselContent,
  CarouselItem,
  CarouselNext,
  CarouselPrevious,
} from '@components/ui/carousel';
import { cn } from '@lib/utils';

export function ImageCarousel({
  imageUrls,
  className,
  imgClassName,
  heightClassName = 'h-64',
  autoSlideIntervalMs = 4000,
  slideGapPx = 12,
}: {
  imageUrls: string[];
  className?: string;
  imgClassName?: string;
  /** Tailwind height class for the viewport. */
  heightClassName?: string;
  /** Auto-advance interval for multi-image carousels. Set to 0 to disable. */
  autoSlideIntervalMs?: number;
  /** Visual gap between slides (in px). */
  slideGapPx?: number;
}) {
  const [api, setApi] = useState<CarouselApi | null>(null);
  const [activeIndex, setActiveIndex] = useState(0);

  useEffect(() => {
    if (!api) return;
    const update = () => { setActiveIndex(api.selectedScrollSnap());
    update();
    api.on('select', update);
    api.on('reInit', update);
    return () => {
      api.off('select', update);
      api.off('reInit', update);
    };
  }, [api]);

  useEffect(() => {
    if (!api) return;
    if (!imageUrls || imageUrls.length <= 1) return;
    if (!autoSlideIntervalMs || autoSlideIntervalMs <= 0) return;

    const id = window.setInterval(() => {
      const next = api.selectedScrollSnap() + 1;
      if (next >= imageUrls.length) {
        api.scrollTo(0);
      } else {
        api.scrollTo(next);
      }
    }, autoSlideIntervalMs);

    return () => { window.clearInterval(id);
  }, [api, autoSlideIntervalMs, imageUrls]);

  if (!imageUrls || imageUrls.length === 0) return null;

  // Single image: still use same layout + object-contain for full visibility.
  if (imageUrls.length === 1) {
    const url = imageUrls[0];
    return (
      <a
```



```

        href={url}
        target="_blank"
        rel="noopener noreferrer"
        className={cn('block overflow-hidden rounded-lg border bg-black/5', heightClassName, className)}
        title="Open image"
      >
      <img
        src={url}
        alt="Post image"
        loading="lazy"
        className={cn('h-full w-full object-contain', imgClassName)}
      />
    </a>
  );
}

return (
  <div className={cn('relative', className)}>
    <Carousel
      className="w-full"
      opts={{ loop: false }}
      setApi={(a) => setApi(a)}
    >
      <CarouselContent className="ml-0" style={{ gap: `${slideGapPx}px` }}>
        {imageUrls.map((url, idx) => (
          <CarouselItem key={` ${url}-${idx}`} className="pl-0">
            <a
              href={url}
              target="_blank"
              rel="noopener noreferrer"
              className={cn('block overflow-hidden rounded-lg border bg-black/5', heightClassName)}
              title="Open image"
            >
              <img
                src={url}
                alt={`Post image ${idx + 1}`}
                loading="lazy"
                className={cn('h-full w-full object-contain', imgClassName)}
              />
            </a>
          </CarouselItem>
        ))}
      </CarouselContent>

      {/* Keep nav buttons inside the card bounds */}
      <CarouselPrevious className="left-3 top-1/2 -translate-y-1/2" />
      <CarouselNext className="right-3 top-1/2 -translate-y-1/2" />

      {/* Dots */}
      <div className="mt-2 flex items-center justify-center gap-1">
        {imageUrls.map((_, i) => (
          <button
            key={i}
            type="button"
            aria-label={`Go to image ${i + 1}`}
            aria-current={i === activeIndex}
            onClick={() => api?.scrollTo(i)}
            className={cn(
              'h-1.5 w-4 rounded-full transition-colors',
              i === activeIndex ? 'bg-foreground' : 'bg-muted-foreground/40'
            )}
          />
        ))}
      </div>
    </Carousel>
  </div>
);
}

```

■ frontend/src/components/LogoDropdown.tsx

// simple logo dropdown component that can be used to go to the landing page or sign out for the user

```
import { Button } from "@components/ui/button";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from "@components/ui/dropdown-menu";
import { useAuthContext } from "@contexts/AuthContext";
import { Home, LogOut } from "lucide-react";
import { useNavigate } from "react-router";

export function LogoDropdown() {
  const { isAuthenticated, logout } = useAuthContext();
  const navigate = useNavigate();

  const handleSignOut = async () => {
    try {
      await logout();
    } catch (error) {
      console.error("Sign out error:", error);
    } finally {
      navigate("/");
    }
  };

  const handleGoHome = () => {
    navigate("/");
  };

  return (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
        <Button variant="ghost" size="icon" className="h-10 w-10">
          
        </Button>
      </DropdownMenuTrigger>
      <DropdownMenuContent align="start" className="w-48">
        <DropdownMenuItem onClick={handleGoHome} className="cursor-pointer">
          <Home className="mr-2 h-4 w-4" />
          Landing Page
        </DropdownMenuItem>
        {isAuthenticated && (
          <>
            <DropdownMenuSeparator />
            <DropdownMenuItem
              onClick={handleSignOut}
              className="cursor-pointer text-destructive focus:text-destructive"
            >
              <LogOut className="mr-2 h-4 w-4" />
              Sign Out
            </DropdownMenuItem>
          </>
        )}
      </DropdownMenuContent>
    </DropdownMenu>
  );
}
```

■ frontend/src/components/Navigation.tsx

```
import { Link, useNavigate, useLocation } from 'react-router-dom';
import { useAuthContext } from '@/contexts/AuthContext';
import { useTheme } from '@/contexts/ThemeContext';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Avatar, AvatarFallback, AvatarImage } from '@/components/ui/avatar';
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from '@/components/ui/dropdown-menu';
import { Home, User, Shield, Logout, Menu, Login, Bell, MessageCircle, Moon, Sun, Search, X, FileText } from 'lucide-react';
import { useEffect, useState, useMemo } from 'react';
import { DefaultAvatarFallback } from '@/components/DefaultAvatarFallback';

export function Navigation() {
  const { user, logout, isAuthenticated } = useAuthContext();
  const { theme, toggleTheme } = useTheme();
  const navigate = useNavigate();
  const location = useLocation();
  const [searchQuery, setSearchQuery] = useState('');

  // Keep the search box in sync with the current URL (?search=...)
  // so users can clear it after searching.
  useEffect(() => {
    const params = new URLSearchParams(location.search);
    const q = params.get('search') || '';
    setSearchQuery(q);
  }, [location.search]);

  const resolvedAvatarSrc = useMemo(() => {
    const raw = user?.avatar;
    if (!raw) return undefined;
    const trimmed = raw.trim();
    if (!trimmed) return undefined;

    // Stable cache-bust: only changes when the avatar URL itself changes.
    const stamp = encodeURIComponent(trimmed);
    const hasQuery = trimmed.includes('?');
    return `${trimmed}${hasQuery ? '&' : '?'}v=${stamp}`;
  }, [user?.avatar]);

  const handleSearch = (e: React.FormEvent) => {
    e.preventDefault();
    const q = searchQuery.trim();
    if (q) {
      navigate(`/?search=${encodeURIComponent(q)}`);
    }
  };

  const handleClearSearch = () => {
    setSearchQuery('');
    navigate('/');
  };

  const handleLogout = async () => {
    try {
      await logout();
    } finally {
      navigate('/');
    }
  };

  const isAdmin = user?.role === 'ROLE_ADMIN';

  return (
    <nav className="sticky top-0 z-50 w-full border-b border-border/40 backdrop-blur-lg bg-background/80">
```

```

<div className="container mx-auto px-4 h-16 flex items-center justify-between">
  <div className="flex items-center gap-6">
    <Link to="/" className="flex items-center gap-2">
      
    </Link>

    <div className="hidden md:flex items-center gap-2">
      <Button variant="ghost" asChild>
        <Link to="/" className="flex items-center gap-2">
          <Home className="h-4 w-4" />
          Feed
        </Link>
      </Button>
      <Button variant="ghost" asChild>
        <Link to="/announcements" className="flex items-center gap-2">
          <Bell className="h-4 w-4" />
          Announcements
        </Link>
      </Button>
      {isAuthenticated && (
        <Button variant="ghost" asChild>
          <Link to="/chat" className="flex items-center gap-2">
            <MessageCircle className="h-4 w-4" />
            Chat
          </Link>
        </Button>
      )}
    </div>
  </div>

  <div className="flex items-center gap-4">
    {isAuthenticated && (
      <form onSubmit={handleSearch} className="flex items-center gap-2">
        <div className="relative">
          <input
            type="text"
            placeholder="Search posts..."
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            className="w-72 pr-9"
          />
          {searchQuery && (
            <button
              type="button"
              variant="ghost"
              size="icon"
              onClick={handleClearSearch}
              className="absolute right-1 top-1/2 -translate-y-1/2 h-7 w-7"
              aria-label="Clear search"
            >
              <X className="h-4 w-4" />
            </button>
          )}
        </div>
        <button type="submit" size="icon" variant="ghost" className="search-button">
          <Search className="h-4 w-4" />
        </button>
      </form>
    )}

    <button
      variant="ghost"
      size="icon"
      onClick={toggleTheme}
      aria-label="Toggle theme"
    >
      <Moon className="h-4 w-4" />
    </button>
  </div>
</div>

```



```

        <Link className="mr-2 h-4 w-4" />
        Feed
      </Link>
    </DropdownMenuItem>
    <DropdownMenuItem asChild>
      <Link to="/announcements" className="cursor-pointer">
        <Bell className="mr-2 h-4 w-4" />
        Announcements
      </Link>
    </DropdownMenuItem>
    <DropdownMenuItem asChild>
      <Link to="/chat" className="cursor-pointer">
        <MessageCircle className="mr-2 h-4 w-4" />
        Chat
      </Link>
    </DropdownMenuItem>
  </DropdownMenuContent>
</DropdownMenu>
</>
) : (
  <Button asChild>
    <Link to="/login">
      <LogIn className="mr-2 h-4 w-4" />
      Login
    </Link>
    <Button>
  )
</div>
</div>
</nav>
);
}

```

■ frontend/src/components/PostCard.tsx

```

import { useState, useEffect } from 'react';
import { Card, CardContent, CardFooter, CardHeader } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar';
import { Textarea } from '@components/ui/textarea';
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from '@components/ui/dropdown-menu';
import { useAuthContext } from '@contexts/AuthContext';
import { postsApi, ApiError, UNAUTHENTICATED_MESSAGE, isUnauthenticatedStatus, type Post } from '@lib/api';
import { Heart, MessageCircle, MoreVertical, Edit, Trash, Save, X, Share2 } from 'lucide-react';
import { toast } from 'sonner';
import { Link, useNavigate } from 'react-router-dom';
import { formatDistanceToNow } from 'date-fns';
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from '@components/ui/alert-dialog';
import { ImageCarousel } from '@components/ImageCarousel';
import { AutoPlayMutedVideo } from '@components/AutoPlayMutedVideo';
import { DefaultAvatarFallback } from '@components/DefaultAvatarFallback';
import { formatCount } from '@lib/formatCount';
import { VerifiedBadge } from '@components/VerifiedBadge';

interface PostCardProps {
  post: Post;
}

```

```

    onPostUpdated: (post: Post) => void;
    onPostDeleted: (postId: number) => void;
}

export function PostCard({ post, onPostUpdated, onPostDeleted }: PostCardProps) {
  const { user, isAuthenticated } = useAuthContext();
  const [isEditing, setIsEditing] = useState(false);
  const [editContent, setEditContent] = useState(post?.content ?? '');
  const [isLiking, setIsLiking] = useState(false);
  const [showDeleteDialog, setShowDeleteDialog] = useState(false);
  const navigate = useNavigate();

  const authorId = post?.author?.id;
  const authorName = post?.author?.name ?? 'Unknown';
  const authorAvatar = post?.author?.avatar;

  const isAuthor = user?.id !== null && authorId !== null && user.id === authorId;
  const isAdmin = user?.role === 'ROLE_ADMIN';
  const canDelete = isAuthor || isAdmin;

  const isAuthorAdmin = post?.author?.role === 'ROLE_ADMIN';

  // Keep local edit field synced when parent state updates the post.
  // This prevents stale content and avoids showing incorrect author info after an update.
  useEffect(() => {
    setEditContent(post?.content ?? '');
  }, [post?.content]);

  const handleLike = async () => {
    if (!isAuthenticated) {
      // Handled by rendering a login link button when unauthenticated.
      return;
    }

    // Friendly UX: don't even call the API if it's your own post.
    if (isAuthor) {
      toast.info("You can't like your own post.");
      return;
    }

    try {
      setIsLiking(true);
      const updated = await postsApi.like(post.id);
      const likedByMe = (updated as unknown as { liked?: boolean }).liked;
      onPostUpdated({ ...post, likes: updated.likes, likedByMe });
    } catch (error) {
      console.error('Failed to like post:', error);
      if (error instanceof ApiError) {
        // Backend sends: "You cannot like your own post." (403)
        if (error.status === 403 && /like your own post/i.test(error.message || '')) {
          toast.info("You can't like your own post.");
          return;
        }
      }

      const msg = isUnauthenticatedStatus(error.status)
        ? UNAUTHENTICATED_MESSAGE
        : (error.message || 'Failed to like post');
      toast.error(msg);
    } else {
      toast.error('Failed to like post');
    }
  } finally {
    setIsLiking(false);
  }
};

  const handleEdit = async () => {
    if (!editContent.trim()) {
      toast.error('Post content cannot be empty');
      return;
    }
  }
}

```

```

try {
  const updatedPost = await postsApi.update(post.id, { content: editContent.trim() });
  // Some update responses may not include author info; preserve existing author details.
  const merged: Post = {
    ...post,
    ...updatedPost,
    author: updatedPost?.author?.id ? updatedPost.author : post.author,
    imageUrls: updatedPost?.imageUrls ?? post.imageUrls,
    videoUrl: updatedPost?.videoUrl ?? post.videoUrl,
  };
  onPostUpdated(merged);
  setIsEditing(false);
  toast.success('Post updated successfully!');
} catch (error) {
  console.error('Failed to update post:', error);
  if (error instanceof ApiError) {
    const msg = isUnauthenticatedStatus(error.status)
      ? UNAUTHENTICATED_MESSAGE
      : (error.message || 'Failed to update post');
    toast.error(msg);
  } else {
    toast.error('Failed to update post');
  }
}
};

const handleDelete = async () => {
  try {
    await postsApi.delete(post.id);
    onPostDeleted(post.id);
    toast.success('Post deleted successfully!');
  } catch (error) {
    console.error('Failed to delete post:', error);
    toast.error('Failed to delete post');
  }
};

const handleCancelEdit = () => {
  setIsEditing(false);
  setEditContent(post.content);
};

const handleShare = async () => {
  const url = `${window.location.origin}/post/${post.id}`;
  try {
    await navigator.clipboard.writeText(url);
    toast.success('Link copied to clipboard!');
  } catch {
    // Fallback for environments where clipboard permission isn't granted.
    try {
      window.prompt('Copy this link:', url);
    } finally {
      toast.success('Link ready to copy');
    }
  }
};

return (
  <>
  <Card className="gradient-mesh-card relative">
    {/* Bottom-right views (plain text like the feed) */}
    <div className="absolute bottom-4 right-6 text-xs text-muted-foreground">
      {formatCount(post.views ?? 0)} views
    </div>
  </Card>
  <CardHeader>
    <div className="flex items-start justify-between">
      <Link
        to={`/profile/${authorId ?? ''}`}
        className="flex items-center gap-3 hover:opacity-80 transition-opacity"
      >

```



```

<Avatar>
  <AvatarImage src={authorAvatar} alt={authorName} />
  <AvatarFallback>
    <DefaultAvatarFallback alt={authorName} />
  </AvatarFallback>
</Avatar>
</div>

<p className="font-semibold inline-flex items-center gap-1">
  {authorName}
</p>
<p className="text-xs text-muted-foreground inline-flex items-center gap-1">
  <VerifiedBadge show={isAuthorAdmin} size={12} className="mr-1" title="Admin" />
  {post.createdAt ? formatDistanceToNow(new Date(post.createdAt), { addSuffix: true }) : ''}
</p>
</div>
</Link>

{(isAuthor || isAdmin) && (
  <DropdownMenu>
    <DropdownMenuTrigger asChild>
      <Button variant="ghost" size="icon" className="h-8 w-8">
        <MoreVertical className="h-4 w-4" />
      </Button>
    </DropdownMenuTrigger>
    <DropdownMenuContent align="end">
      {isAuthor && (
        <DropdownMenuItem onClick={() => navigate(`/posts/edit/${post.id}`)}>
          <Edit className="mr-2 h-4 w-4" />
          Edit
        </DropdownMenuItem>
      )}
      {canDelete && (
        <DropdownMenuItem
          onClick={() => setShowDeleteDialog(true)}
          className="text-destructive">
          <Trash className="mr-2 h-4 w-4" />
          Delete
        </DropdownMenuItem>
      )}
    </DropdownMenuContent>
  </DropdownMenu>
)}
</div>
</CardHeader>

<CardContent>
  /* Media (Cloudinary) */
  {(post.imageUrls?.length || post.videoUrl) && (
    <div className="mb-4 space-y-3">
      {post.videoUrl && (
        <AutoPlayMutedVideo src={post.videoUrl} />
      )}

      {post.imageUrls && post.imageUrls.length > 0 && (
        <ImageCarousel imageUrls={post.imageUrls} heightClassName="h-72" />
      )}
    </div>
  )}

  /* Title */
  {post.title && post.title.trim() && (
    <h3 className="mb-2 text-lg font-semibold leading-snug">
      {post.title}
    </h3>
  )}

  {isEditing ? (
    <div className="space-y-3">
      <Textarea
        value={editContent}

```

```

        onChange={(e) => setEditContent(e.target.value)}
        className="min-h-[100px] resize-none"
      />
    <div className="flex gap-2 justify-end">
      <Button variant="ghost" size="sm" onClick={handleCancelEdit}>
        <X className="mr-2 h-4 w-4" />
        Cancel
      </Button>
      <Button size="sm" onClick={handleEdit}>
        <Save className="mr-2 h-4 w-4" />
        Save
      </Button>
    </div>
  </div>
) : (
  <p className="whitespace-pre-wrap">{post.content}</p>
)
}
</CardContent>

<CardFooter className="flex gap-4 pt-0">
  {isAuthenticated ? (
    <Button
      variant={post.likedByMe ? "ghost" : "outline"}
      size="sm"
      onClick={handleLike}
      disabled={isLiking}
      className={
        post.likedByMe
          ? "rounded-full border border-transparent text-white bg-linear-to-r from-[rgb(75,85,99)] via-[rgb(55,71,87)] to-[rgb(42,57,72)]"
          : ""
      }
    >
      <Heart className={`mr-2 h-4 w-4 ${post.likedByMe ? 'fill-current' : ''}`} />
      {formatCount(post.likes)}
    </Button>
  ) : (
    <Button variant="outline" size="sm" asChild>
      <Link to={` /login?redirect=${encodeURIComponent(`/post/${post.id}`)} `}>
        <Heart className="mr-2 h-4 w-4" />
        {formatCount(post.likes)}
      </Link>
    </Button>
  )}
  <Button variant="ghost" size="sm" asChild>
    <Link to={` /post/${post.id}`}>
      <MessageCircle className="mr-2 h-4 w-4" />
      {formatCount(post.commentCount ?? post.comments?.length ?? 0)}
    </Link>
  </Button>

  <Button variant="outline" size="sm" onClick={handleShare}>
    <Share2 className="mr-2 h-4 w-4" />
    Share
  </Button>
</CardFooter>
</Card>

<AlertDialog open={showDeleteDialog} onOpenChange={setShowDeleteDialog}>
  <AlertDialogContent>
    <AlertDialogHeader>
      <AlertDialogTitle>Delete Post</AlertDialogTitle>
      <AlertDialogDescription>
        Are you sure you want to delete this post? This action cannot be undone.
      </AlertDialogDescription>
    </AlertDialogHeader>
    <AlertDialogFooter>
      <AlertDialogCancel>Cancel</AlertDialogCancel>
      <AlertDialogAction onClick={handleDelete} className="bg-destructive text-destructive-foreground">Delete</AlertDialogAction>
    </AlertDialogFooter>
  </AlertDialogContent>
</AlertDialog>

```

```

        </AlertDialogFooter>;
        </AlertDialogContent>;
    </AlertDialog>;
    </>;
  );
}

```

■ frontend/src/components/ProtectedRoute.tsx

```

import { ReactNode } from 'react';
import { Navigate } from 'react-router-dom';
import { useAuthContext } from '@/contexts/AuthContext';
import { Loader2 } from 'lucide-react';

interface ProtectedRouteProps {
  children: ReactNode;
}

export function ProtectedRoute({ children }: ProtectedRouteProps) {
  const { isAuthenticated, isLoading } = useAuthContext();

  if (isLoading) {
    return (
      <div className="min-h-screen flex items-center justify-center">
        <Loader2 className="h-8 w-8 animate-spin text-primary" />
      </div>;
    );
  }

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  return <>{children}</>;
}

```

■ frontend/src/components/RightSidebar.tsx

```

import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Link, useNavigate } from 'react-router-dom';
import { Info, Search } from 'lucide-react';
import { useState } from 'react';
import { toast } from 'sonner';
import { useAuthContext } from '@/contexts/AuthContext';

export function RightSidebar() {
  const [searchQuery, setSearchQuery] = useState('');
  const navigate = useNavigate();
  const { isAuthenticated } = useAuthContext();

  const handleSearch = (e: React.FormEvent) => {
    e.preventDefault();

    if (!isAuthenticated) {
      toast.error('Please log in to search users');
      navigate('/login');
      return;
    }

    const q = searchQuery.trim();
    if (!q) return;

    const params = new URLSearchParams(window.location.search);
    params.set('userSearch', q);
    // Reset posts search if present, because we are switching modes.
    params.delete('search');
  }
}

```

```

// Reset paging if any
params.delete('page');
window.location.href = `/?${params.toString()}`;
};

return (
  <aside className="hidden lg:block w-72 xl:w-80 fixed right-6 top-20 max-h-[calc(100vh-5rem)] overflow-y-auto h
  <div className="space-y-4">
    {/ * Search Users */}
    <Card className="gradient-mesh-card">
      <CardHeader>
        <CardTitle className="text-lg flex items-center gap-2">
          <Search className="h-5 w-5" />
          Search Users
        </CardTitle>
      </CardHeader>
      <CardContent>
        <form onSubmit={handleSearch} className="flex gap-2">
          <Input
            type="text"
            placeholder="Search users..."
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            className="flex-1"
          />
          <Button type="submit" size="icon" className="search-button">
            <Search className="h-4 w-4" />
          </Button>
        </form>
      </CardContent>
    </Card>

    {/ * Community Guidelines */}
    <Card className="gradient-mesh-card">
      <CardHeader className="pb-3">
        <CardTitle className="text-lg flex items-center gap-2">
          <Info className="h-5 w-5" />
          Community Guidelines
        </CardTitle>
      </CardHeader>
      <CardContent>
        <ul className="space-y-2 text-sm">
          <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-0.5">•</span>
            <span>Be respectful to all users</span>
          </li>
          <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-0.5">•</span>
            <span>Share genuine, tech-related content</span>
          </li>
          <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-0.5">•</span>
            <span>Protect privacy and personal data</span>
          </li>
          <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-0.5">•</span>
            <span>No fake profiles, spam, or harmful activity</span>
          </li>
        </ul>
      </CardContent>
    </Card>

    {/ * About AlliedEdge */}
    <Card className="gradient-mesh-card">
      <CardHeader>
        <CardTitle className="text-lg brand-name">
          About <span className="allied">Allied</span> <span className="edge">Edge</span>
        </CardTitle>
      </CardHeader>
      <CardContent className="space-y-3">
        <p className="text-sm">

```

```

    A social platform for students and developers to connect, share knowledge, and grow together.
    </p>
    <div className="flex flex-wrap gap-2 text-xs">
      <Link to="/help" className="hover:underline">Help</Link>
      <span>•</span>
      <Link to="/terms" className="hover:underline">Terms</Link>
      <span>•</span>
      <Link to="/privacy" className="hover:underline">Privacy</Link>
    </div>
    <div className="flex flex-wrap gap-2 text-xs">
      <span>Admin</span>
      <span>Contacts</span>
      <a
        href="https://www.linkedin.com/in/ryan-mohammed-rafeeq"
        target="_blank"
        rel="noreferrer noopener"
        className="hover:underline"
      >
        Rayan
      </a>
      <span>•</span>
      <a
        href="https://www.linkedin.com/in/rakinmohammedrafeeq"
        target="_blank"
        rel="noreferrer noopener"
        className="hover:underline"
      >
        Rakin
      </a>
    </div>
    <p className="text-xs text-muted-foreground brand-name">
      © 2026 <span className="allied">Allied</span> <span className="edge">Edge</span>
    </p>
  </CardContent>
</Card>
</div>
</aside>
);
}

```

■ frontend/src/components/ui/accordion.tsx

```

import * as React from "react"
import * as AccordionPrimitive from "@radix-ui/react-accordion"
import { ChevronDownIcon } from "lucide-react"

import { cn } from "@lib/Utils"

function Accordion({
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Root>) {
  return <AccordionPrimitive.Root data-slot="accordion" {...props} />
}

function AccordionItem({
  className,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Item>) {
  return (
    <AccordionPrimitive.Item
      data-slot="accordion-item"
      className={cn("border-b last:border-b-0", className)}
      {...props}
    />
  )
}

function AccordionTrigger({
  className,
  children,
  ...props

```

```

}: React.ComponentProps<typeof AccordionPrimitive.Trigger>); {
  return (
    <AccordionPrimitive.Header className="flex">
      <AccordionPrimitive.Trigger
        data-slot="accordion-trigger"
        className={cn(
          "focus-visible:border-ring focus-visible:ring-ring/50 flex flex-1 items-start justify-between gap-4 rounded-m
          className
        )}
        {...props}
      >
      {children}
      <ChevronDownIcon className="text-muted-foreground pointer-events-none size-4 shrink-0 translate-y-0.5 transi
      </AccordionPrimitive.Trigger>
    </AccordionPrimitive.Header>
  )
}

function AccordionContent({
  className,
  children,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Content>); {
  return (
    <AccordionPrimitive.Content
      data-slot="accordion-content"
      className="data-[state=closed]:animate-accordion-up data-[state=open]:animate-accordion-down overflow-hidden text
      {...props}
    >
    <div className={cn("pt-0 pb-4", className)}>{children}</div>
    </AccordionPrimitive.Content>
  )
}

export { Accordion, AccordionItem, AccordionTrigger, AccordionContent }

```

■ frontend/src/components/ui/alert-dialog.tsx

```

"use client"

import * as React from "react"
import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog"

import { cn } from "@lib/utls"
import { buttonVariants } from "@components/ui/button"

function AlertDialog({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Root>); {
  return <AlertDialogPrimitive.Root data-slot="alert-dialog" {...props} />
}

function AlertDialogTrigger({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Trigger>); {
  return (
    <AlertDialogPrimitive.Trigger data-slot="alert-dialog-trigger" {...props} />
  )
}

function AlertDialogPortal({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Portal>); {
  return (
    <AlertDialogPrimitive.Portal data-slot="alert-dialog-portal" {...props} />
  )
}

function AlertDialogOverlay({
  className,

```

```

    ...props
  }: React.ComponentProps<typeof AlertDialogPrimitive.Overlay> ) {
    return (
      <AlertDialogPrimitive.Overlay
        data-slot="alert-dialog-overlay"
        className={cn(
          "data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:",
          className
        )}
        {...props}
      />
    )
  }
}

function AlertDialogContent({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Content>) {
  return (
    <AlertDialogPortal>
      <AlertDialogOverlay />
      <AlertDialogPrimitive.Content
        data-slot="alert-dialog-content"
        className={cn(
          "bg-background data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 da",
          className
        )}
        {...props}
      />
    </AlertDialogPortal>
  )
}

function AlertDialogHeader({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="alert-dialog-header"
      className={cn("flex flex-col gap-2 text-center sm:text-left", className)}
      {...props}
    />
  )
}

function AlertDialogFooter({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="alert-dialog-footer"
      className={cn(
        "flex flex-col-reverse gap-2 sm:flex-row sm:justify-end",
        className
      )}
      {...props}
    />
  )
}

function AlertDialogTitle({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Title>) {
  return (
    <AlertDialogPrimitive.Title
      data-slot="alert-dialog-title"
      className={cn("text-lg font-semibold", className)}
      {...props}
    />
  )
}

```

```

    /&gt;
  )
}

function AlertDialogDescription({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Description>) {
  return (
    <AlertDialogPrimitive.Description
      data-slot="alert-dialog-description"
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    /&gt;
  )
}

function AlertDialogAction({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Action>) {
  return (
    <AlertDialogPrimitive.Action
      className={cn(buttonVariants(), className)}
      {...props}
    /&gt;
  )
}

function AlertDialogCancel({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Cancel>) {
  return (
    <AlertDialogPrimitive.Cancel
      className={cn(buttonVariants({ variant: "outline" }), className)}
      {...props}
    /&gt;
  )
}

export {
  AlertDialog,
  AlertDialogPortal,
  AlertDialogOverlay,
  AlertDialogTrigger,
  AlertDialogContent,
  AlertDialogHeader,
  AlertDialogFooter,
  AlertDialogTitle,
  AlertDialogDescription,
  AlertDialogAction,
  AlertDialogCancel,
}

```

■ frontend/src/components/ui/alert.tsx

```

import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@/lib/utils"

const alertVariants = cva(
  "relative w-full rounded-lg border px-4 py-3 text-sm grid has-[:&svg]:grid-cols-[calc(var(--spacing)*4)_1fr] grid-c
  {
    variants: {
      variant: {
        default: "bg-card text-card-foreground",
        destructive:
          "text-destructive bg-card [&:svg]:text-current *:data-[slot=alert-description]:text-destructive/90",

```



```

    },
  },
  defaultVariants: {
    variant: "default",
  },
}
)

function Alert({
  className,
  variant,
  ...props
}: React.ComponentProps<"div"> & VariantProps<typeof alertVariants> & {
  return (
    <div
      data-slot="alert"
      role="alert"
      className={cn(alertVariants({ variant }), className)}
      {...props}
    />
  )
}

function AlertTitle({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="alert-title"
      className={cn(
        "col-start-2 line-clamp-1 min-h-4 font-medium tracking-tight",
        className
      )}
      {...props}
    />
  )
}

function AlertDescription({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="alert-description"
      className={cn(
        "text-muted-foreground col-start-2 grid justify-items-start gap-1 text-sm [&_p]:leading-relaxed",
        className
      )}
      {...props}
    />
  )
}

export { Alert, AlertTitle, AlertDescription }

```

■ frontend/src/components/ui/aspect-ratio.tsx

```

import * as AspectRatioPrimitive from "@radix-ui/react-aspect-ratio"

function AspectRatio({
  ...props
}: React.ComponentProps<typeof AspectRatioPrimitive.Root>) {
  return <AspectRatioPrimitive.Root data-slot="aspect-ratio" {...props} />
}

export { AspectRatio }

```

■ frontend/src/components/ui/avatar.tsx

```

"use client"

import * as React from "react"
import * as AvatarPrimitive from "@radix-ui/react-avatar"

import { cn } from "@lib/Utils"

function Avatar({
  className,
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Root> & {
  return (
    <AvatarPrimitive.Root
      data-slot="avatar"
      className={cn(
        "relative flex size-8 shrink-0 overflow-hidden rounded-full",
        className
      )}
      {...props}
    /&gt;
  )
}

function AvatarImage({
  className,
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Image> & {
  return (
    <AvatarPrimitive.Image
      data-slot="avatar-image"
      className={cn("aspect-square size-full", className)}
      {...props}
    /&gt;
  )
}

function AvatarFallback({
  className,
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Fallback> & {
  return (
    <AvatarPrimitive.Fallback
      data-slot="avatar-fallback"
      className={cn(
        "bg-muted flex size-full items-center justify-center rounded-full",
        className
      )}
      {...props}
    /&gt;
  )
}

export { Avatar, AvatarImage, AvatarFallback }

```

■ frontend/src/components/ui/badge.tsx

```

import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@lib/Utils"

const badgeVariants = cva(
  "inline-flex items-center justify-center rounded-md border px-2 py-0.5 text-xs font-medium w-fit whitespace-nowrap shrink-0",
  {
    variants: {
      variant: {
        default:
          "border-transparent bg-primary text-primary-foreground [&]:hover:bg-primary/90",
        secondary:

```

```

        "border-transparent bg-secondary text-secondary-foreground [a&]:hover:bg-secondary/90",
        destructive:
        "border-transparent bg-destructive text-white [a&]:hover:bg-destructive/90 focus-visible:ring-destructive",
        outline:
        "text-foreground [a&]:hover:bg-accent [a&]:hover:text-accent-foreground",
    },
  },
  defaultVariants: {
    variant: "default",
  },
},
)

function Badge({
  className,
  variant,
  asChild = false,
  ...props
}: React.ComponentProps<"span"> &
VariantProps<typeof badgeVariants> & { asChild?: boolean }) {
  const Comp = asChild ? Slot : "span"

  return (
    <Comp
      data-slot="badge"
      className={cn(badgeVariants({ variant }), className)}
      {...props}
    />
  )
}

export { Badge, badgeVariants }

```

■ frontend/src/components/ui/breadcrumb.tsx

```

import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { ChevronRight, MoreHorizontal } from "lucide-react"

import { cn } from "@/lib/utils"

function Breadcrumb({ ...props }: React.ComponentProps<"nav">) {
  return <nav aria-label="breadcrumb" data-slot="breadcrumb" {...props} />
}

function BreadcrumbList({ className, ...props }: React.ComponentProps<"ol">) {
  return (
    <ol
      data-slot="breadcrumb-list"
      className={cn(
        "text-muted-foreground flex flex-wrap items-center gap-1.5 text-sm break-words sm:gap-2.5",
        className
      )}
      {...props}
    />
  )
}

function BreadcrumbItem({ className, ...props }: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="breadcrumb-item"
      className={cn("inline-flex items-center gap-1.5", className)}
      {...props}
    />
  )
}

function BreadcrumbLink({
  asChild,

```

```

    className,
    ...props
  }: React.ComponentProps<"a"> & {
    asChild?: boolean
  }) {
    const Comp = asChild ? Slot : "a"

    return (
      <Comp
        data-slot="breadcrumb-link"
        className={cn("hover:text-foreground transition-colors", className)}
        {...props}
      />
    )
  }

function BreadcrumbPage({ className, ...props }: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="breadcrumb-page"
      role="link"
      aria-disabled="true"
      aria-current="page"
      className={cn("text-foreground font-normal", className)}
      {...props}
    />
  )
}

function BreadcrumbSeparator({
  children,
  className,
  ...props
}: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="breadcrumb-separator"
      role="presentation"
      aria-hidden="true"
      className={cn("[&svg]:size-3.5", className)}
      {...props}
    >
    {children ?? <ChevronRight />}
    </li>
  )
}

function BreadcrumbEllipsis({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="breadcrumb-ellipsis"
      role="presentation"
      aria-hidden="true"
      className={cn("flex size-9 items-center justify-center", className)}
      {...props}
    >
    <MoreHorizontal className="size-4" />
    <span className="sr-only">More</span>
    </span>
  )
}

export {
  Breadcrumb,
  BreadcrumbList,
  BreadcrumbItem,
  BreadcrumbLink,
  BreadcrumbPage,

```

```

    BreadcrumbSeparator,
    BreadcrumbEllipsis,
  }

```

■ frontend/src/components/ui/button.tsx

```

import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@lib/utils"

const buttonVariants = cva(
  "cursor-pointer inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium transi
  {
    variants: {
      variant: {
        default:
          "bg-primary text-primary-foreground shadow-xs",
        destructive:
          "bg-destructive text-white shadow-xs focus-visible:ring-destructive/20 dark:focus-visible:ring-destructive/40",
        outline:
          "border bg-background text-foreground shadow-xs dark:bg-input/30 dark:border-input",
        secondary:
          "bg-secondary text-secondary-foreground shadow-xs",
        ghost:
          "text-foreground",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default: "h-9 px-4 py-2 has-[:svg]:px-3",
        sm: "h-8 rounded-md gap-1.5 px-3 has-[:svg]:px-2.5",
        lg: "h-10 rounded-md px-6 has-[:svg]:px-4",
        icon: "size-9",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

function Button({
  className,
  variant,
  size,
  asChild = false,
  ...props
}: React.ComponentProps<"button"> & VariantProps<typeof buttonVariants> & {
  asChild?: boolean
}) {
  const Comp = asChild ? Slot : "button"

  return (
    <Comp
      data-slot="button"
      className={cn(buttonVariants({ variant, size, className }))}
      {...props}
    />
  )
}

export { Button, buttonVariants }

```

■ frontend/src/components/ui/calendar.tsx

```

import * as React from "react"
import {
  ChevronDownIcon,
  ChevronLeftIcon,
  ChevronRightIcon,
} from "lucide-react"
import { DayButton, DayPicker, getDefaultClassNames } from "react-day-picker"

import { cn } from "@/lib/Utils"
import { Button, buttonVariants } from "@/components/ui/button"

function Calendar({
  className,
  classNames,
  showOutsideDays = true,
  captionLayout = "label",
  buttonVariant = "ghost",
  formatters,
  components,
  ...props
}: React.ComponentProps<typeof DayPicker> & {
  buttonVariant?: React.ComponentProps<typeof Button>["variant"]
}) {
  const defaultClassNames = getDefaultClassNames()

  return (
    <DayPicker
      showOutsideDays={showOutsideDays}
      className={cn(
        "bg-background group/calendar p-3 [--cell-size:--spacing(8)] [[data-slot=card-content]]&:bg-transparent",
        String.raw`rtl:*.rdp-button\_next&svg:rotate-180`,
        String.raw`rtl:*.rdp-button\_previous&svg:rotate-180`,
        className
      )}
      captionLayout={captionLayout}
      formatters={{
        formatMonthDropdown: (date) => {
          date.toLocaleString("default", { month: "short" }),
          ...formatters,
        }
      }}
      classNames={{
        root: cn("w-fit", defaultClassNames.root),
        months: cn(
          "flex gap-4 flex-col md:flex-row relative",
          defaultClassNames.months
        ),
        month: cn("flex flex-col w-full gap-4", defaultClassNames.month),
        nav: cn(
          "flex items-center gap-1 w-full absolute top-0 inset-x-0 justify-between",
          defaultClassNames.nav
        ),
        button_previous: cn(
          buttonVariants({ variant: buttonVariant }),
          "size-(--cell-size) aria-disabled:opacity-50 p-0 select-none",
          defaultClassNames.button_previous
        ),
        button_next: cn(
          buttonVariants({ variant: buttonVariant }),
          "size-(--cell-size) aria-disabled:opacity-50 p-0 select-none",
          defaultClassNames.button_next
        ),
        month_caption: cn(
          "flex items-center justify-center h-(--cell-size) w-full px-(--cell-size)",
          defaultClassNames.month_caption
        ),
        dropdowns: cn(
          "w-full flex items-center text-sm font-medium justify-center h-(--cell-size) gap-1.5",
          defaultClassNames.dropdowns
        ),
        dropdown_root: cn(
          "relative has-focus:border-ring border border-input shadow-xs has-focus:ring-ring/50 has-focus:ring-[3px] rounded"
        )
      }}
    />
  )
}

```



```

        return (
          <ChevronRightIcon
            className={cn("size-4", className)}
            {...props}
          />
        )
      }

      return (
        <ChevronDownIcon className={cn("size-4", className)} {...props} />
      )
    },
    DayButton: CalendarDayButton,
    weekNumber: ({ children, ...props }) => {
      return (
        <td {...props}>
          <div className="flex size-(--cell-size) items-center justify-center text-center">
            {children}
          </div>
        </td>
      )
    },
    ...components,
  })
  {...props}
/>
)
}

function CalendarDayButton({
  className,
  day,
  modifiers,
  ...props
}: React.ComponentProps<typeof DayButton>) {
  const defaultClassNames = getDefaultClassNames()

  const ref = React.useRef<HTMLElement>(null)
  React.useEffect(() => {
    if (modifiers.focused) ref.current?.focus()
  }, [modifiers.focused])

  return (
    <Button
      ref={ref}
      variant="ghost"
      size="icon"
      data-day={day.date.toLocaleDateString()}
      data-selected-single={
        modifiers.selected &&
        !modifiers.range_start &&
        !modifiers.range_end &&
        !modifiers.range_middle
      }
      data-range-start={modifiers.range_start}
      data-range-end={modifiers.range_end}
      data-range-middle={modifiers.range_middle}
      className={cn(
        "data-[selected-single=true]:bg-primary data-[selected-single=true]:text-primary-foreground data-[range-middle=
        defaultClassNames.day,
        className
      )}
      {...props}
    />
  )
}

export { Calendar, CalendarDayButton }

```


■ frontend/src/components/ui/card.tsx

```
import * as React from "react"

import { cn } from "@/lib/utils"

function Card({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card"
      className={cn(
        "bg-card text-card-foreground flex flex-col gap-6 rounded-xl border py-6 shadow-sm",
        className
      )}
      {...props}
    />
  )
}

function CardHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-header"
      className={cn(
        "@container/card-header grid auto-rows-min grid-rows-[auto_auto] items-start gap-1.5 px-6 has-data-[slot=card-a",
        className
      )}
      {...props}
    />
  )
}

function CardTitle({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-title"
      className={cn("leading-none font-semibold", className)}
      {...props}
    />
  )
}

function CardDescription({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-description"
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    />
  )
}

function CardAction({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-action"
      className={cn(
        "col-start-2 row-span-2 row-start-1 self-start justify-self-end",
        className
      )}
      {...props}
    />
  )
}

function CardContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-content"
      className={cn("px-6", className)}
      {...props}
    />
  )
}
```

```

    )
  }

function CardFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-footer"
      className={cn("flex items-center px-6 [.border-t]:pt-6", className)}
      {...props}
    />
  )
}

export {
  Card,
  CardHeader,
  CardFooter,
  CardTitle,
  CardAction,
  CardDescription,
  CardContent,
}

```

■ frontend/src/components/ui/carousel.tsx

```

"use client"

import * as React from "react"
import useEmblaCarousel, {
  type UseEmblaCarouselType,
} from "embla-carousel-react"
import { ArrowLeft, ArrowRight } from "lucide-react"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"

type CarouselApi = UseEmblaCarouselType[1]
type UseCarouselParameters = Parameters<typeof useEmblaCarousel>
type CarouselOptions = UseCarouselParameters[0]
type CarouselPlugin = UseCarouselParameters[1]

type CarouselProps = {
  opts?: CarouselOptions
  plugins?: CarouselPlugin
  orientation?: "horizontal" | "vertical"
  setApi?: (api: CarouselApi) => void
}

type CarouselContextProps = {
  carouselRef: ReturnType<typeof useEmblaCarousel>[0]
  api: ReturnType<typeof useEmblaCarousel>[1]
  scrollPrev: () => void
  scrollNext: () => void
  canScrollPrev: boolean
  canScrollNext: boolean
} & CarouselProps

const CarouselContext = React.createContext<CarouselContextProps | null>(null)

function useCarousel() {
  const context = React.useContext(CarouselContext)

  if (!context) {
    throw new Error("useCarousel must be used within a <Carousel />")
  }

  return context
}

function Carousel({

```

```

orientation = "horizontal",
opts,
setApi,
plugins,
className,
children,
...props
}: React.ComponentProps<"div"> & CarouselProps) {
  const [carouselRef, api] = useEmblaCarousel(
    {
      ...opts,
      axis: orientation === "horizontal" ? "x" : "y",
    },
    plugins
  )
  const [canScrollPrev, setCanScrollPrev] = React.useState(false)
  const [canScrollNext, setCanScrollNext] = React.useState(false)

  const onSelect = React.useCallback((api: CarouselApi) => {
    if (!api) return
    setCanScrollPrev(api.canScrollPrev())
    setCanScrollNext(api.canScrollNext())
  }, [])

  const scrollPrev = React.useCallback(() => {
    api?.scrollPrev()
  }, [api])

  const scrollNext = React.useCallback(() => {
    api?.scrollNext()
  }, [api])

  const handleKeyDown = React.useCallback(
    (event: React.KeyboardEvent<HTMLDivElement>) => {
      if (event.key === "ArrowLeft") {
        event.preventDefault()
        scrollPrev()
      } else if (event.key === "ArrowRight") {
        event.preventDefault()
        scrollNext()
      }
    },
    [scrollPrev, scrollNext]
  )

  React.useEffect(() => {
    if (!api || !setApi) return
    setApi(api)
  }, [api, setApi])

  React.useEffect(() => {
    if (!api) return
    onSelect(api)
    api.on("reInit", onSelect)
    api.on("select", onSelect)

    return () => {
      api?.off("select", onSelect)
    }
  }, [api, onSelect])

  return (
    <CarouselContext.Provider
      value={{
        carouselRef,
        api: api,
        opts,
        orientation:
          orientation || (opts?.axis === "y" ? "vertical" : "horizontal"),
        scrollPrev,
        scrollNext,

```

```

        canScrollPrev,
        canScrollNext,
      }}
    &gt;
    &lt;div
      onKeyDownCapture={handleKeyDown}
      className={cn("relative", className)}
      role="region"
      aria-roledescription="carousel"
      data-slot="carousel"
      {...props}
    &gt;
      {children}
    &lt;/div&gt;
    &lt;/CarouselContext.Provider&gt;
  )
}

function CarouselContent({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  const { carouselRef, orientation } = useCarousel()

  return (
    &lt;div
      ref={carouselRef}
      className="overflow-hidden"
      data-slot="carousel-content"
    &gt;
      &lt;div
        className={cn(
          "flex",
          orientation === "horizontal" ? "-ml-4" : "-mt-4 flex-col",
          className
        )}
        {...props}
      /&gt;
    &lt;/div&gt;
  )
}

function CarouselItem({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  const { orientation } = useCarousel()

  return (
    &lt;div
      role="group"
      aria-roledescription="slide"
      data-slot="carousel-item"
      className={cn(
        "min-w-0 shrink-0 grow-0 basis-full",
        orientation === "horizontal" ? "pl-4" : "pt-4",
        className
      )}
      {...props}
    /&gt;
  )
}

function CarouselPrevious({
  className,
  variant = "outline",
  size = "icon",
  ...props
}: React.ComponentProps&lt;typeof Button&gt;) {
  const { orientation, scrollPrev, canScrollPrev } = useCarousel()

  return (
    &lt;Button
      data-slot="carousel-previous"
      variant={variant}
      size={size}
      className={cn(

```

```

        "absolute size-8 rounded-full",
        orientation === "horizontal"
        ? "top-1/2 -left-12 -translate-y-1/2"
        : "-top-12 left-1/2 -translate-x-1/2 rotate-90",
        className
    )}
    disabled={!canScrollPrev}
    onClick={scrollPrev}
    {...props}
  </Button>
  <ArrowLeft />
  <span className="sr-only">Previous slide</span>
  </Button>
)
}

function CarouselNext({
  className,
  variant = "outline",
  size = "icon",
  ...props
}: React.ComponentProps<typeof Button>) {
  const { orientation, scrollNext, canScrollNext } = useCarousel()

  return (
    <Button
      data-slot="carousel-next"
      variant={variant}
      size={size}
      className={cn(
        "absolute size-8 rounded-full",
        orientation === "horizontal"
        ? "top-1/2 -right-12 -translate-y-1/2"
        : "-bottom-12 left-1/2 -translate-x-1/2 rotate-90",
        className
      )}
      disabled={!canScrollNext}
      onClick={scrollNext}
      {...props}
    >
      <ArrowRight />
      <span className="sr-only">Next slide</span>
    </Button>
  )
}

export {
  type CarouselApi,
  Carousel,
  CarouselContent,
  CarouselItem,
  CarouselPrevious,
  CarouselNext,
}

```

■ frontend/src/components/ui/chart.tsx

```

import * as React from "react"
import * as RechartsPrimitive from "recharts"
import type { Payload } from "recharts/types/component/DefaultTooltipContent"

import { cn } from "@lib/Utils"

// Format: { THEME_NAME: CSS_SELECTOR }
const THEMES = { light: "", dark: ".dark" } as const

export type ChartConfig = {
  [k in string]: {
    label?: React.ReactNode
    icon?: React.ComponentType
  }
}

```

```

    } & (
      | { color?: string; theme?: never }
      | { color?: never; theme: Record<keyof typeof THEMES, string> }
    )
  }

type ChartContextProps = {
  config: ChartConfig
}

const ChartContext = React.createContext<ChartContextProps | null>(null)

function useChart() {
  const context = React.useContext(ChartContext)

  if (!context) {
    throw new Error("useChart must be used within a <ChartContainer />")
  }

  return context
}

function ChartContainer({
  id,
  className,
  children,
  config,
  ...props
}: React.ComponentProps<"div"> & {
  config: ChartConfig
  children: React.ComponentProps<
    typeof RechartsPrimitive.ResponsiveContainer
  >["children"]
}) {
  const uniqueId = React.useId()
  const chartId = `chart-${id || uniqueId.replace(/:/g, "")}`

  return (
    <ChartContext.Provider value={{ config }}>
      <div
        data-slot="chart"
        data-chart={chartId}
        className={cn(
          "[&__recharts-cartesian-axis-tick_text]:fill-muted-foreground [&__recharts-cartesian-grid_line[stroke
          className
        )}
        {...props}
      >
        <ChartStyle id={chartId} config={config} />
        <RechartsPrimitive.ResponsiveContainer>
          {children}
        </RechartsPrimitive.ResponsiveContainer>
      </div>
    </ChartContext.Provider>
  )
}

const ChartStyle = ({ id, config }: { id: string; config: ChartConfig }) => {
  const colorConfig = Object.entries(config).filter(
    ([, config]) => config.theme || config.color
  )

  if (!colorConfig.length) {
    return null
  }

  return (
    <style
      dangerouslySetInnerHTML={{
        __html: Object.entries(THEMES)
          .map(

```

```

        ([theme, prefix]) => `
    ${prefix} [data-chart=${id}] {
    ${colorConfig
      .map(([, itemConfig]) => {
        const color =
          itemConfig.theme?.[theme as keyof typeof itemConfig.theme] ||
          itemConfig.color
        return color ? `--color-${key}: ${color};` : null
      })
      .join("\n")}
    }
    `
    )
    .join("\n"),
  }}
  />
)
}

const ChartTooltip = RechartsPrimitive.Tooltip

function ChartTooltipContent({
  active,
  payload,
  className,
  indicator = "dot",
  hideLabel = false,
  hideIndicator = false,
  label,
  labelFormatter,
  labelClassName,
  formatter,
  color,
  nameKey,
  labelKey,
}: React.ComponentProps<typeof RechartsPrimitive.Tooltip> &
React.ComponentProps<"div"> & {
  active?: boolean
  payload?: Array<Payload>number | string, string>;>
  label?: string | number
  hideLabel?: boolean
  hideIndicator?: boolean
  indicator?: "line" | "dot" | "dashed"
  labelFormatter?: (value: unknown, payload: Array<Payload>number | string, string>;>) => React.ReactNode
  formatter?: (
    value: number | string,
    name: string,
    item: Payload<number | string, string>;,
    index: number,
    payload: Array<Payload>number | string, string>;>
  ) => React.ReactNode
  color?: string
  nameKey?: string
  labelKey?: string
}) {
  const { config } = useChart()

  const tooltipLabel = React.useMemo(() => {
    if (hideLabel || !payload?.length) {
      return null
    }

    const [item] = payload
    const key = `${labelKey || item?.dataKey || item?.name || "value"}`
    const itemConfig = getPayloadConfigFromPayload(config, item, key)
    const value: React.ReactNode =
      !labelKey && typeof label === "string"
        ? config[label as keyof typeof config]?.label || label
        : itemConfig?.label

    if (labelFormatter) {

```

```

        return (
            <div className={cn("font-medium", labelClassName)}>
                {labelFormatter(value, payload)}
            </div>
        )
    }

    if (!value) {
        return null
    }

    return <div className={cn("font-medium", labelClassName)}>{value}</div>
}, [
    label,
    labelFormatter,
    payload,
    hideLabel,
    labelClassName,
    config,
    labelKey,
])

if (!active || !payload?.length) {
    return null
}

const nestLabel = payload.length === 1 && indicator !== "dot"

return (
    <div
        className={cn(
            "border-border/50 bg-background grid min-w-[8rem] items-start gap-1.5 rounded-lg border px-2.5 py-1.5 text-xs s
            className
        )}
    >
        {!nestLabel ? tooltipLabel : null}
        <div className="grid gap-1.5">
            {payload.map((item, index) => {
                const key = `${nameKey || item.name || item.dataKey || "value"}`
                const itemConfig = getPayloadConfigFromPayload(config, item, key)
                const indicatorColor = color || item.payload?.fill || item.color

                return (
                    <div
                        key={`item-${index}`}
                        className={cn(
                            "[&&svg]:text-muted-foreground flex w-full flex-wrap items-stretch gap-2 [&&svg]:h-2.5 [&
                            indicator === "dot" && "items-center"
                        )}
                    >
                        {formatter && item?.value !== undefined && item?.value !== null && item.name ? (
                            formatter(item.value, item.name, item, index, payload)
                        ) : (
                            <&&
                            {itemConfig?.icon ? (
                                <itemConfig.icon />
                            ) : (
                                !hideIndicator && (
                                    <div
                                        className={cn(
                                            "shrink-0 rounded-[2px] border(--color-border) bg(--color-bg)",
                                            {
                                                "h-2.5 w-2.5": indicator === "dot",
                                                "w-1": indicator === "line",
                                                "w-0 border-[1.5px] border-dashed bg-transparent":
                                                    indicator === "dashed",
                                                "my-0.5": nestLabel && indicator === "dashed",
                                            }
                                        )}
                                    <style={
                                        {

```



```

        "--color-bg": indicatorColor,
        "--color-border": indicatorColor,
    } as React.CSSProperties
    }
    /&gt;
  )
  })
  &lt;div
    className={cn(
      "flex flex-1 justify-between leading-none",
      nestLabel ? "items-end" : "items-center"
    )}
    &gt;
    &lt;div className="grid gap-1.5"&gt;
      {nestLabel ? tooltipLabel : null}
      &lt;span className="text-muted-foreground"&gt;
        {itemConfig?.label || item.name}
      &lt;/span&gt;
    &lt;/div&gt;
    {item.value &amp;&amp; (
      &lt;span className="text-foreground font-mono font-medium tabular-nums"&gt;
        {typeof item.value === 'number' ? item.value.toLocaleString() : item.value}
      &lt;/span&gt;
    )}
    &lt;/div&gt;
  &lt;/div&gt;
  )}
  &lt;/div&gt;
  &lt;/div&gt;
)
}

```

```
const ChartLegend = RechartsPrimitive.Legend
```

```

function ChartLegendContent({
  className,
  hideIcon = false,
  payload,
  verticalAlign = "bottom",
  nameKey,
}: React.ComponentProps<div> & & {
  payload?: Array<Payload<number | string, string>>
  verticalAlign?: "top" | "bottom"
  hideIcon?: boolean
  nameKey?: string
}) {
  const { config } = useChart()

  if (!payload?.length) {
    return null
  }

  return (
    &lt;div
      className={cn(
        "flex items-center justify-center gap-4",
        verticalAlign === "top" ? "pb-3" : "pt-3",
        className
      )}
    &gt;
      {payload.map((item, index) => {
        const key = `${nameKey || item.dataKey || "value"}`
        const itemConfig = getPayloadConfigFromPayload(config, item, key)

        return (
          &lt;div
            key={`legend-${index}`}
            className={cn(

```

```

        "[&svg]:text-muted-foreground flex items-center gap-1.5 [&svg]:h-3 [&svg]:w-3"
    )}
    &gt;
    {itemConfig?.icon && !hideIcon ? (
        <itemConfig.icon />
    ) : (
        <div
            className="h-2 w-2 shrink-0 rounded-[2px]"
            style={{
                backgroundColor: item.color,
            }}
        />
    )}
    {itemConfig?.label}
    </div>
)
}}
</div>
)
}

// Helper to extract item config from a payload.
function getPayloadConfigFromPayload(
    config: ChartConfig,
    payload: unknown,
    key: string
) {
    if (typeof payload !== "object" || payload === null) {
        return undefined
    }

    const payloadPayload =
        "payload" in payload &&
        typeof payload.payload === "object" &&
        payload.payload !== null
        ? payload.payload
        : undefined

    let configLabelKey: string = key

    if (
        key in payload &&
        typeof payload[key as keyof typeof payload] === "string"
    ) {
        configLabelKey = payload[key as keyof typeof payload] as string
    } else if (
        payloadPayload &&
        key in payloadPayload &&
        typeof payloadPayload[key as keyof typeof payloadPayload] === "string"
    ) {
        configLabelKey = payloadPayload[
            key as keyof typeof payloadPayload
        ] as string
    }

    return configLabelKey in config
        ? config[configLabelKey]
        : config[key as keyof typeof config]
}

export {
    ChartContainer,
    ChartTooltip,
    ChartTooltipContent,
    ChartLegend,
    ChartLegendContent,
    ChartStyle,
}

```

■ frontend/src/components/ui/checkbox.tsx

```
"use client"

import * as React from "react"
import * as CheckboxPrimitive from "@radix-ui/react-checkbox"
import { CheckIcon } from "lucide-react"

import { cn } from "@/lib/utils"

function Checkbox({
  className,
  ...props
}: React.ComponentProps<typeof CheckboxPrimitive.Root> & {
  return (
    <CheckboxPrimitive.Root
      data-slot="checkbox"
      className={cn(
        "peer border-input dark:bg-input/30 data-[state=checked]:bg-primary data-[state=checked]:text-primary-foreground"
        className
      )}
      {...props}
    &gt;
      <CheckboxPrimitive.Indicator
        data-slot="checkbox-indicator"
        className="flex items-center justify-center text-current transition-none"
      &gt;
        <CheckIcon className="size-3.5" /&gt;
      </CheckboxPrimitive.Indicator>
    </CheckboxPrimitive.Root>
  )
}

export { Checkbox }
```

■ frontend/src/components/ui/collapsible.tsx

```
import * as CollapsiblePrimitive from "@radix-ui/react-collapsible"

function Collapsible({
  ...props
}: React.ComponentProps<typeof CollapsiblePrimitive.Root> & {
  return <CollapsiblePrimitive.Root data-slot="collapsible" {...props} /&gt;
}

function CollapsibleTrigger({
  ...props
}: React.ComponentProps<typeof CollapsiblePrimitive.CollapsibleTrigger> & {
  return (
    <CollapsiblePrimitive.CollapsibleTrigger
      data-slot="collapsible-trigger"
      {...props}
    /&gt;
  )
}

function CollapsibleContent({
  ...props
}: React.ComponentProps<typeof CollapsiblePrimitive.CollapsibleContent> & {
  return (
    <CollapsiblePrimitive.CollapsibleContent
      data-slot="collapsible-content"
      {...props}
    /&gt;
  )
}

export { Collapsible, CollapsibleTrigger, CollapsibleContent }
```

■ frontend/src/components/ui/command.tsx

```
"use client"

import * as React from "react"
import { Command as CommandPrimitive } from "cmdk"
import { SearchIcon } from "lucide-react"

import { cn } from "@/lib/utils"
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog"

function Command({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive> & {
  return (
    <CommandPrimitive
      data-slot="command"
      className={cn(
        "bg-popover text-popover-foreground flex h-full w-full flex-col overflow-hidden rounded-md",
        className
      )}
      {...props}
    />
  )
}) {

function CommandDialog({
  title = "Command Palette",
  description = "Search for a command to run...",
  children,
  className,
  showCloseButton = true,
  ...props
}: React.ComponentProps<typeof Dialog> & {
  title?: string
  description?: string
  className?: string
  showCloseButton?: boolean
}) {
  return (
    <Dialog {...props}>
      <DialogHeader className="sr-only">
        <DialogTitle>{title}</DialogTitle>
        <DialogDescription>{description}</DialogDescription>
      </DialogHeader>
      <DialogContent
        className={cn("overflow-hidden p-0", className)}
        showCloseButton={showCloseButton}
      >
        <Command className="[&_[:cmdk-group-heading]]:text-muted-foreground **:data-[slot=command-input-wrapper]:"
          {children}
        </Command>
      </DialogContent>
    </Dialog>
  )
}

function CommandInput({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Input> & {
  return (
    <div
```

```

        data-slot="command-input-wrapper"
        className="flex h-9 items-center gap-2 border-b px-3"
      >
        <SearchIcon className="size-4 shrink-0 opacity-50" />
        <CommandPrimitive.Input
          data-slot="command-input"
          className={cn(
            "placeholder:text-muted-foreground flex h-10 w-full rounded-md bg-transparent py-3 text-sm outline-hidden dis
            className
          )}
          {...props}
        />
      </div>
    )
  }

function CommandList({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.List>) {
  return (
    <CommandPrimitive.List
      data-slot="command-list"
      className={cn(
        "max-h-[300px] scroll-py-1 overflow-x-hidden overflow-y-auto",
        className
      )}
      {...props}
    />
  )
}

function CommandEmpty({
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Empty>) {
  return (
    <CommandPrimitive.Empty
      data-slot="command-empty"
      className="py-6 text-center text-sm"
      {...props}
    />
  )
}

function CommandGroup({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Group>) {
  return (
    <CommandPrimitive.Group
      data-slot="command-group"
      className={cn(
        "text-foreground [&_<cmdk-group-heading>]:text-muted-foreground overflow-hidden p-1 [&_<cmdk-group-head
        className
      )}
      {...props}
    />
  )
}

function CommandSeparator({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Separator>) {
  return (
    <CommandPrimitive.Separator
      data-slot="command-separator"
      className={cn("bg-border -mx-1 h-px", className)}
      {...props}
    />
  )
}

```

```

}

function CommandItem({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Item> & {
  return (
    <CommandPrimitive.Item
      data-slot="command-item"
      className={cn(
        "data-[selected=true]:bg-accent data-[selected=true]:text-accent-foreground [&_svg:not([class*='text-'])]:t
        className
      )}
      {...props}
    /&gt;
  )
}

function CommandShortcut({
  className,
  ...props
}: React.ComponentProps<"span"> & {
  return (
    <span
      data-slot="command-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className
      )}
      {...props}
    /&gt;
  )
}

export {
  Command,
  CommandDialog,
  CommandInput,
  CommandList,
  CommandEmpty,
  CommandGroup,
  CommandItem,
  CommandShortcut,
  CommandSeparator,
}

```

■ frontend/src/components/ui/context-menu.tsx

```

"use client"

import * as React from "react"
import * as ContextMenuPrimitive from "@radix-ui/react-context-menu"
import { CheckIcon, ChevronRightIcon, CircleIcon } from "lucide-react"

import { cn } from "@/lib/utls"

function ContextMenu({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Root> & {
  return <ContextMenuPrimitive.Root data-slot="context-menu" {...props} /&gt;
}

function ContextMenuTrigger({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Trigger> & {
  return (
    <ContextMenuPrimitive.Trigger data-slot="context-menu-trigger" {...props} /&gt;
  )
}

```

```

function ContextMenuGroup({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Group> >) {
  return (
    <ContextMenuPrimitive.Group data-slot="context-menu-group" {...props} />
  )
}

function ContextMenuPortal({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Portal> >) {
  return (
    <ContextMenuPrimitive.Portal data-slot="context-menu-portal" {...props} />
  )
}

function ContextMenuSub({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Sub> >) {
  return <ContextMenuPrimitive.Sub data-slot="context-menu-sub" {...props} />
}

function ContextMenuRadioGroup({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.RadioGroup> >) {
  return (
    <ContextMenuPrimitive.RadioGroup
      data-slot="context-menu-radio-group"
      {...props}
    />
  )
}

function ContextMenuSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.SubTrigger> & {
  inset?: boolean
}) {
  return (
    <ContextMenuPrimitive.SubTrigger
      data-slot="context-menu-sub-trigger"
      data-inset={inset}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
        className
      )}
      {...props}
    >
      {children}
      <ChevronRightIcon className="ml-auto" />
    </ContextMenuPrimitive.SubTrigger>
  )
}

function ContextMenuSubContent({
  className,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.SubContent> >) {
  return (
    <ContextMenuPrimitive.SubContent
      data-slot="context-menu-sub-content"
      className={cn(
        "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
        className
      )}
      {...props}
    />
  )
}

```

```

}

function ContextMenuContent({
  className,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Content> & {
  return (
    <ContextMenuPrimitive.Portal>
      <ContextMenuPrimitive.Content
        data-slot="context-menu-content"
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=
          className
        )}
        {...props}
      />
    </ContextMenuPrimitive.Portal>
  )
}

function ContextMenuItem({
  className,
  inset,
  variant = "default",
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Item> & {
  inset?: boolean
  variant?: "default" | "destructive"
}) {
  return (
    <ContextMenuPrimitive.Item
      data-slot="context-menu-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[variant=destructive]:text-destructive data-[variant=destructive]:text-destructive"
        className
      )}
      {...props}
    />
  )
}

function ContextMenuCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.CheckboxItem> & {
  return (
    <ContextMenuPrimitive.CheckboxItem
      data-slot="context-menu-checkbox-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2 rounded-sm py-1.5"
        className
      )}
      checked={checked}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
        <ContextMenuPrimitive.ItemIndicator>
          <CheckIcon className="size-4" />
        </ContextMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </ContextMenuPrimitive.CheckboxItem>
  )
}

function ContextMenuRadioItem({
  className,

```



```

    children,
    ...props
  }: React.ComponentProps<typeof ContextMenuPrimitive.RadioItem> {
    return (
      <ContextMenuPrimitive.RadioItem
        data-slot="context-menu-radio-item"
        className={cn(
          "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2 rounded-sm py-1.5",
          className
        )}
        {...props}
      >
        <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
          <ContextMenuPrimitive.ItemIndicator>
            <CircleIcon className="size-2 fill-current" />
          </ContextMenuPrimitive.ItemIndicator>
        </span>
        {children}
      </ContextMenuPrimitive.RadioItem>
    )
  }
}

function ContextMenuLabel({
  className,
  inset,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Label> & {
  inset?: boolean
}) {
  return (
    <ContextMenuPrimitive.Label
      data-slot="context-menu-label"
      data-inset={inset}
      className={cn(
        "text-foreground px-2 py-1.5 text-sm font-medium data-[inset]:pl-8",
        className
      )}
      {...props}
    >
    </>
  )
}

function ContextMenuSeparator({
  className,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Separator> {
  return (
    <ContextMenuPrimitive.Separator
      data-slot="context-menu-separator"
      className={cn("bg-border -mx-1 my-1 h-px", className)}
      {...props}
    >
    </>
  )
}

function ContextMenuShortcut({
  className,
  ...props
}: React.ComponentProps<"span"> {
  return (
    <span
      data-slot="context-menu-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className
      )}
      {...props}
    >
    </>
  )
}

export {

```

```

ContextMenu,
ContextMenuTrigger,
ContextMenuContent,
ContextMenuItem,
ContextMenuCheckboxItem,
ContextMenuRadioItem,
ContextMenuLabel,
ContextMenuSeparator,
ContextMenuShortcut,
ContextMenuGroup,
ContextMenuPortal,
ContextMenuSub,
ContextMenuSubContent,
ContextMenuSubTrigger,
ContextMenuRadioGroup,
}

```

■ frontend/src/components/ui/dialog.tsx

```

import * as React from "react"
import * as DialogPrimitive from "@radix-ui/react-dialog"
import { XIcon } from "lucide-react"

import { cn } from "@/lib/utils"

function Dialog({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Root> & {
  return <<DialogPrimitive.Root data-slot="dialog" {...props} /&gt;
}) {

function DialogTrigger({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Trigger> & {
  return <<DialogPrimitive.Trigger data-slot="dialog-trigger" {...props} /&gt;
}) {

function DialogPortal({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Portal> & {
  return <<DialogPrimitive.Portal data-slot="dialog-portal" {...props} /&gt;
}) {

function DialogClose({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Close> & {
  return <<DialogPrimitive.Close data-slot="dialog-close" {...props} /&gt;
}) {

function DialogOverlay({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Overlay> & {
  return (
    <<DialogPrimitive.Overlay
      data-slot="dialog-overlay"
      className={cn(
        "data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:",
        className
      )}
      {...props}
    /&gt;
  )
}

function DialogContent({
  className,
  children,
  showCloseButton = true,
  ...props

```

```

}: React.ComponentProps<typeof DialogPrimitive.Content> & {
  showCloseButton?: boolean
}) {
  return (
    <DialogPortal data-slot="dialog-portal">
      <DialogOverlay />
      <DialogPrimitive.Content
        data-slot="dialog-content"
        className={cn(
          "bg-background data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 da
          className
        )}
        {...props}
      >
        {children}
        {showCloseButton & & (
          <DialogPrimitive.Close
            data-slot="dialog-close"
            className="ring-offset-background focus:ring-ring data-[state=open]:bg-accent data-[state=open]:text-muted-
            >
            <XIcon />
            <span className="sr-only">Close</span>
          </DialogPrimitive.Close>
        )}
      </DialogPrimitive.Content>
    </DialogPortal>
  )
}

function DialogHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="dialog-header"
      className={cn("flex flex-col gap-2 text-center sm:text-left", className)}
      {...props}
    />
  )
}

function DialogFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="dialog-footer"
      className={cn(
        "flex flex-col-reverse gap-2 sm:flex-row sm:justify-end",
        className
      )}
      {...props}
    />
  )
}

function DialogTitle({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Title>) {
  return (
    <DialogPrimitive.Title
      data-slot="dialog-title"
      className={cn("text-lg leading-none font-semibold", className)}
      {...props}
    />
  )
}

function DialogDescription({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Description>) {
  return (
    <DialogPrimitive.Description

```

```

        data-slot="dialog-description"
        className={cn("text-muted-foreground text-sm", className)}
        {...props}
      />
    )
  }

export {
  Dialog,
  DialogClose,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogOverlay,
  DialogPortal,
  DialogTitle,
  DialogTrigger,
}

```

■ frontend/src/components/ui/drawer.tsx

```

import * as React from "react"
import { Drawer as DrawerPrimitive } from "vaul"

import { cn } from "@/lib/utils"

function Drawer({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Root> & {
  return <DrawerPrimitive.Root data-slot="drawer" {...props} />
}) {

function DrawerTrigger({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Trigger> & {
  return <DrawerPrimitive.Trigger data-slot="drawer-trigger" {...props} />
}) {

function DrawerPortal({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Portal> & {
  return <DrawerPrimitive.Portal data-slot="drawer-portal" {...props} />
}) {

function DrawerClose({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Close> & {
  return <DrawerPrimitive.Close data-slot="drawer-close" {...props} />
}) {

function DrawerOverlay({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Overlay> & {
  return (
    <DrawerPrimitive.Overlay
      data-slot="drawer-overlay"
      className={cn(
        "data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:
        className
      )}
      {...props}
    />
  )
}

function DrawerContent({
  className,
  children,

```

```

    ...props
  }: React.ComponentProps<typeof DrawerPrimitive.Content>); {
    return (
      <DrawerPortal data-slot="drawer-portal">
        <DrawerOverlay />
        <DrawerPrimitive.Content
          data-slot="drawer-content"
          className={cn(
            "group/drawer-content bg-background fixed z-50 flex h-auto flex-col",
            "data-[vaul-drawer-direction=top]:inset-x-0 data-[vaul-drawer-direction=top]:top-0 data-[vaul-drawer-direction=bottom]:inset-x-0 data-[vaul-drawer-direction=bottom]:bottom-0 data-[vaul-drawer-direction=right]:inset-y-0 data-[vaul-drawer-direction=right]:right-0 data-[vaul-drawer-direction=left]:inset-y-0 data-[vaul-drawer-direction=left]:left-0 data-[vaul-drawer-direction=left]:left-0 data-[vaul-drawer-direction=right]:right-0",
            className
          )}
        >
          {...props}
        </DrawerPrimitive.Content>
      </DrawerPortal>
    )
  }
}

function DrawerHeader({ className, ...props }: React.ComponentProps<"div">); {
  return (
    <div
      data-slot="drawer-header"
      className={cn(
        "flex flex-col gap-0.5 p-4 group-data-[vaul-drawer-direction=bottom]/drawer-content:text-center group-data-[vaul-drawer-direction=bottom]/drawer-content:font-italic",
        className
      )}
    >
      {...props}
    </div>
  )
}

function DrawerFooter({ className, ...props }: React.ComponentProps<"div">); {
  return (
    <div
      data-slot="drawer-footer"
      className={cn("mt-auto flex flex-col gap-2 p-4", className)}
    >
      {...props}
    </div>
  )
}

function DrawerTitle({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Title>); {
  return (
    <DrawerPrimitive.Title
      data-slot="drawer-title"
      className={cn("text-foreground font-semibold", className)}
    >
      {...props}
    </DrawerPrimitive.Title>
  )
}

function DrawerDescription({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Description>); {
  return (
    <DrawerPrimitive.Description
      data-slot="drawer-description"
      className={cn("text-muted-foreground text-sm", className)}
    >
      {...props}
    </DrawerPrimitive.Description>
  )
}

```

```

}

export {
  Drawer,
  DrawerPortal,
  DrawerOverlay,
  DrawerTrigger,
  DrawerClose,
  DrawerContent,
  DrawerHeader,
  DrawerFooter,
  DrawerTitle,
  DrawerDescription,
}

```

■ frontend/src/components/ui/dropdown-menu.tsx

"use client"

```

import * as React from "react"
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu"
import { CheckIcon, ChevronRightIcon, CircleIcon } from "lucide-react"

import { cn } from "@/lib/utils"

function DropdownMenu({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Root> >) {
  return <DropdownMenuPrimitive.Root data-slot="dropdown-menu" {...props} />
}

function DropdownMenuPortal({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Portal> >) {
  return (
    <DropdownMenuPrimitive.Portal data-slot="dropdown-menu-portal" {...props} />
  )
}

function DropdownMenuTrigger({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Trigger> >) {
  return (
    <DropdownMenuPrimitive.Trigger
      data-slot="dropdown-menu-trigger"
      {...props}
    />
  )
}

function DropdownMenuContent({
  className,
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Content> >) {
  return (
    <DropdownMenuPrimitive.Portal>
      <DropdownMenuPrimitive.Content
        data-slot="dropdown-menu-content"
        sideOffset={sideOffset}
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
          className
        )}
        {...props}
      />
    </DropdownMenuPrimitive.Portal>
  )
}

```

```

function DropdownMenuGroup({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Group> & {
  return (
    <DropdownMenuPrimitive.Group data-slot="dropdown-menu-group" {...props} />
  )
}

function DropdownMenuItem({
  className,
  inset,
  variant = "default",
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Item> & {
  inset?: boolean
  variant?: "default" | "destructive"
}) {
  return (
    <DropdownMenuPrimitive.Item
      data-slot="dropdown-menu-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[variant=destructive]:text-destructive data-[variant=destructive]:text-destructive",
        className
      )}
      {...props}
    />
  )
}

function DropdownMenuCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.CheckboxItem> & {
  return (
    <DropdownMenuPrimitive.CheckboxItem
      data-slot="dropdown-menu-checkbox-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2 rounded-sm py-1.5",
        className
      )}
      checked={checked}
      {...props}
    >
    <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
      <CheckIcon className="size-4" />
    </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.CheckboxItem>
  )
}

function DropdownMenuRadioGroup({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.RadioGroup> & {
  return (
    <DropdownMenuPrimitive.RadioGroup
      data-slot="dropdown-menu-radio-group"
      {...props}
    />
  )
}

function DropdownMenuRadioItem({
  className,
  children,

```

```

    ...props
  }: React.ComponentProps<typeof DropdownMenuPrimitive.RadioItem>); {
    return (
      <DropdownMenuPrimitive.RadioItem
        data-slot="dropdown-menu-radio-item"
        className={cn(
          "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2 rounded-sm py-1.5",
          className
        )}
        {...props}
      />
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
        <DropdownMenuPrimitive.ItemIndicator>
          <CircleIcon className="size-2 fill-current" />
        </DropdownMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </DropdownMenuPrimitive.RadioItem>
  )
}

function DropdownMenuLabel({
  className,
  inset,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Label> & {
  inset?: boolean
}) {
  return (
    <DropdownMenuPrimitive.Label
      data-slot="dropdown-menu-label"
      data-inset={inset}
      className={cn(
        "px-2 py-1.5 text-sm font-medium data-[inset]:pl-8",
        className
      )}
      {...props}
    />
  )
}

function DropdownMenuSeparator({
  className,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Separator>); {
  return (
    <DropdownMenuPrimitive.Separator
      data-slot="dropdown-menu-separator"
      className={cn("bg-border -mx-1 my-1 h-px", className)}
      {...props}
    />
  )
}

function DropdownMenuShortcut({
  className,
  ...props
}: React.ComponentProps<"span">); {
  return (
    <span
      data-slot="dropdown-menu-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className
      )}
      {...props}
    />
  )
}

function DropdownMenuSub({

```



```

    ...props
  }: React.ComponentProps<typeof DropdownMenuPrimitive.Sub>) {
    return <<DropdownMenuPrimitive.Sub data-slot="dropdown-menu-sub" {...props} />>
  }

function DropdownMenuSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.SubTrigger> & {
  inset?: boolean
}) {
  return (
    <<DropdownMenuPrimitive.SubTrigger
      data-slot="dropdown-menu-sub-trigger"
      data-inset={inset}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground"
        className
      )}
      {...props}
    >>
    {children}
    <<ChevronRightIcon className="ml-auto size-4" />>
    <<DropdownMenuPrimitive.SubTrigger>>
  )
}

function DropdownMenuSubContent({
  className,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.SubContent>) {
  return (
    <<DropdownMenuPrimitive.SubContent
      data-slot="dropdown-menu-sub-content"
      className={cn(
        "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0"
        className
      )}
      {...props}
    />>
  )
}

export {
  DropdownMenu,
  DropdownMenuPortal,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuGroup,
  DropdownMenuLabel,
  DropdownMenuItem,
  DropdownMenuCheckboxItem,
  DropdownMenuRadioGroup,
  DropdownMenuRadioItem,
  DropdownMenuSeparator,
  DropdownMenuShortcut,
  DropdownMenuSub,
  DropdownMenuSubTrigger,
  DropdownMenuSubContent,
}

```

■ frontend/src/components/ui/form.tsx

```

import * as React from "react"
import * as LabelPrimitive from "@radix-ui/react-label"
import { Slot } from "@radix-ui/react-slot"
import {
  Controller,

```



```

    <FormItemContext.Provider value={{ id }}>
      <div
        data-slot="form-item"
        className={cn("grid gap-2", className)}
        {...props}
      />
    </FormItemContext.Provider>
  )
}

function FormLabel({
  className,
  ...props
}: React.ComponentProps<typeof LabelPrimitive.Root> & {
  const { error, formItemId } = useFormField()

  return (
    <Label
      data-slot="form-label"
      data-error={!!error}
      className={cn("data-[error=true]:text-destructive", className)}
      htmlFor={formItemId}
      {...props}
    />
  )
}

function FormControl({ ...props }: React.ComponentProps<typeof Slot>) {
  const { error, formItemId, formDescriptionId, formMessageId } = useFormField()

  return (
    <Slot
      data-slot="form-control"
      id={formItemId}
      aria-describedby={
        !error
          ? `${formDescriptionId}`
          : `${formDescriptionId} ${formMessageId}`
      }
      aria-invalid={!!error}
      {...props}
    />
  )
}

function FormDescription({ className, ...props }: React.ComponentProps<"p">) {
  const { formDescriptionId } = useFormField()

  return (
    <p
      data-slot="form-description"
      id={formDescriptionId}
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    />
  )
}

function FormMessage({ className, ...props }: React.ComponentProps<"p">) {
  const { error, formMessageId } = useFormField()
  const body = error ? String(error?.message ?? "") : props.children

  if (!body) {
    return null
  }

  return (
    <p
      data-slot="form-message"
      id={formMessageId}
      className={cn("text-destructive text-sm", className)}
    />
  )
}

```

```

    {...props}
    &gt;
    {body}
    &lt;/p&gt;
  )
}

export {
  useFormField,
  Form,
  FormItem,
  FormLabel,
  FormControl,
  FormDescription,
  FormMessage,
  FormField,
}

```

■ frontend/src/components/ui/hover-card.tsx

```

import * as React from "react"
import * as HoverCardPrimitive from "@radix-ui/react-hover-card"

import { cn } from "@/lib/utils"

function HoverCard({
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Root>) {
  return &lt;HoverCardPrimitive.Root data-slot="hover-card" {...props} /&gt;
}

function HoverCardTrigger({
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Trigger>) {
  return (
    &lt;HoverCardPrimitive.Trigger data-slot="hover-card-trigger" {...props} /&gt;
  )
}

function HoverCardContent({
  className,
  align = "center",
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Content>) {
  return (
    &lt;HoverCardPrimitive.Portal data-slot="hover-card-portal"&gt;
      &lt;HoverCardPrimitive.Content
        data-slot="hover-card-content"
        align={align}
        sideOffset={sideOffset}
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0"
          className
        )}
        {...props}
      /&gt;
    &lt;/HoverCardPrimitive.Portal&gt;
  )
}

export { HoverCard, HoverCardTrigger, HoverCardContent }

```

■ frontend/src/components/ui/index.ts

```

/**
 * UI Component Registry
 */

```

```

* This file provides organized exports of all UI components to help AI agents
* and developers find and import only the components they need.
*
* Components are grouped by category and include bundle size hints:
* - Small (<5KB): Basic components with minimal dependencies
* - Medium (5-15KB): Components with moderate complexity
* - Large (>15KB): Complex components with heavy dependencies
*/

// =====
// BASIC COMPONENTS (Small bundle size)
// =====

// Typography & Layout
export { Badge } from "../badge"; // ~2KB - Status badges
export { Label } from "../label"; // ~1KB - Form labels
export { Separator } from "../separator"; // ~1KB - Visual dividers
export { Skeleton } from "../skeleton"; // ~1KB - Loading placeholders

// Form Inputs
export { Button } from "../button"; // ~3KB - Primary button component
export { Input } from "../input"; // ~2KB - Text input
export { Textarea } from "../textarea"; // ~2KB - Multi-line text input
export { Checkbox } from "../checkbox"; // ~3KB - Checkbox input
export { Switch } from "../switch"; // ~3KB - Toggle switch
export { Slider } from "../slider"; // ~4KB - Range slider
export { RadioGroup } from "../radio-group"; // ~3KB - Radio button group

// Display Components
export { Avatar } from "../avatar"; // ~3KB - User avatars
export { Card, CardHeader, CardFooter, CardTitle, CardDescription, CardContent } from "../card"; // ~2KB - Content cards
export { Progress } from "../progress"; // ~2KB - Progress bars
export { AspectRatio } from "../aspect-ratio"; // ~1KB - Maintain aspect ratios

// =====
// INTERACTIVE COMPONENTS (Medium bundle size)
// =====

// Navigation
export { Breadcrumb, BreadcrumbList, BreadcrumbItem, BreadcrumbLink, BreadcrumbPage, BreadcrumbSeparator, BreadcrumbEllipsis } from "../breadcrumb"; // ~1KB - Breadcrumb navigation
export { Tabs, TabsList, TabsTrigger, TabsContent } from "../tabs"; // ~5KB - Tab navigation
export { NavigationMenu, NavigationMenuList, NavigationMenuItem, NavigationMenuTrigger, NavigationMenuContent, NavigationMenuSeparator } from "../navigation-menu"; // ~3KB - Navigation menu

// Overlays & Dialogs
export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider } from "../tooltip"; // ~4KB - Hover tooltips
export { Popover, PopoverTrigger, PopoverContent } from "../popover"; // ~5KB - Floating popovers
export { HoverCard, HoverCardTrigger, HoverCardContent } from "../hover-card"; // ~5KB - Hover cards
export { Dialog, DialogTrigger, DialogContent, DialogHeader, DialogFooter, DialogTitle, DialogDescription } from "../dialog"; // ~3KB - Modal dialogs
export { Sheet, SheetTrigger, SheetContent, SheetHeader, SheetFooter, SheetTitle, SheetDescription } from "../sheet"; // ~3KB - Slide-over panels
export { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, AlertDialogDescription, AlertDialogFooter, AlertDialogHeader, AlertDialogTitle } from "../alert-dialog"; // ~3KB - Confirmation dialogs

// Menus & Dropdowns
export { DropdownMenu, DropdownMenuTrigger, DropdownMenuContent, DropdownMenuItem, DropdownMenuCheckboxItem, DropdownMenuRadioItem, DropdownMenuSeparator } from "../dropdown-menu"; // ~3KB - Context menus
export { ContextMenu, ContextMenuTrigger, ContextMenuContent, ContextMenuItem, ContextMenuCheckboxItem, ContextMenuRadioItem, ContextMenuSeparator } from "../context-menu"; // ~3KB - Context menus
export { Menubar, MenubarMenu, MenubarTrigger, MenubarContent, MenubarItem, MenubarSeparator, MenubarLabel, MenubarCheckboxItem, MenubarRadioItem, MenubarShortcutItem } from "../menubar"; // ~3KB - Top-level navigation

// Form Components
export { Select, SelectGroup, SelectValue, SelectTrigger, SelectContent, SelectLabel, SelectItem, SelectSeparator, SelectScrollDownButton, SelectScrollUpButton } from "../select"; // ~3KB - Select dropdowns
export { Form, FormItem, FormLabel, FormControl, FormDescription, FormMessage, FormField } from "../form"; // ~4KB - Form structure
export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator } from "../input-otp"; // ~5KB - OTP input

// Content Display
export { Accordion, AccordionItem, AccordionTrigger, AccordionContent } from "../accordion"; // ~5KB - Collapsible sections
export { Collapsible, CollapsibleTrigger, CollapsibleContent } from "../collapsible"; // ~3KB - Simple collapse
export { Toggle, toggleVariants } from "../toggle"; // ~3KB - Toggle button
export { ToggleGroup, ToggleGroupItem } from "../toggle-group"; // ~4KB - Button group toggle
export { Alert, AlertTitle, AlertDescription } from "../alert"; // ~3KB - Alert messages
export { Table, TableHeader, TableBody, TableFooter, TableHead, TableRow, TableCell, TableCaption } from "../table"; // ~3KB - Data tables

// =====
// ADVANCED COMPONENTS (Large bundle size - use sparingly)
// =====

```

```
// =====

// Complex Interactions
export { ScrollArea, ScrollBar } from "./scroll-area"; // ~6KB - Custom scrollbars
export { Drawer, DrawerPortal, DrawerOverlay, DrawerTrigger, DrawerClose, DrawerContent, DrawerHeader, DrawerFooter, DrawerHeaderTrigger } from "./drawer"; // ~10KB - Drawer components
export { Carousel, CarouselContent, CarouselItem, CarouselPrevious, CarouselNext } from "./carousel"; // ~15KB - Image carousel
export { Calendar } from "./calendar"; // ~12KB - Date picker calendar (uses react-day-picker)
export { Command, CommandDialog, CommandInput, CommandList, CommandEmpty, CommandGroup, CommandItem, CommandShortcut, CommandSeparator } from "./command"; // ~10KB - Command palette
export { Pagination, PaginationContent, PaginationEllipsis, PaginationItem, PaginationLink, PaginationNext, PaginationPrevious } from "./pagination"; // ~8KB - Pagination components

// Layout Components
export { ResizablePanelGroup, ResizablePanel, ResizableHandle } from "./resizable"; // ~8KB - Resizable panels
export { Sidebar, SidebarProvider, SidebarTrigger, SidebarInset, SidebarHeader, SidebarFooter, SidebarContent, SidebarMenu, SidebarMenuList, SidebarMenuItem, SidebarMenuGroup } from "./sidebar"; // ~10KB - Sidebar components

// Data Visualization
export { ChartContainer, ChartTooltip, ChartTooltipContent, ChartLegend, ChartLegendContent, ChartStyle } from "./chart"; // ~10KB - Chart components

// Notifications
export { Toaster } from "./sonner"; // ~8KB - Toast notifications

// =====
// USAGE EXAMPLES FOR AI AGENTS
// =====

/**
 * Example: Building a simple form
 *
 * import { Card, CardHeader, CardTitle, CardContent } from "@components/ui";
 * import { Form, FormField, FormItem, FormLabel, FormControl } from "@components/ui";
 * import { Input, Button } from "@components/ui";
 *
 * Small bundle impact: ~12KB total
 */

/**
 * Example: Building a complex dashboard
 *
 * import { Card, Tabs, Chart } from "@components/ui";
 * import { Select, Button } from "@components/ui";
 *
 * Large bundle impact: ~35KB+ (due to Chart/recharts)
 * Consider lazy loading: const Chart = lazy(() => import("@components/ui/chart"));
 */

/**
 * Example: Building a navigation menu
 *
 * import { NavigationMenu, DropdownMenu } from "@components/ui";
 * import { Button, Avatar } from "@components/ui";
 *
 * Medium bundle impact: ~15KB total
 */

// =====
// OPTIMIZATION TIPS FOR AI AGENTS
// =====

/**
 * 1. Import only what you need:
 *   ■ import { Button, Input } from "@components/ui";
 *   ■ import * as UI from "@components/ui";
 *
 * 2. Lazy load heavy components:
 *   const Chart = lazy(() => import("@components/ui").then(m => ({ default: m.Chart })));
 *   const Calendar = lazy(() => import("@components/ui/calendar").then(m => ({ default: m.Calendar })));
 *
 * 3. Group imports by route:
 *   - Landing page: Button, Card, Badge (small components only)
 *   - Dashboard: + Chart, Tabs, Select (add medium components)
 *   - Admin: + Command, Carousel (add large components as needed)
 */

```

```

* 4. Prefer native HTML when possible:
*   - Use <input> instead of Input for simple cases
*   - Use <button> instead of Button for basic buttons
*   - Use these components when you need the styling/accessibility features
*/

```

■ frontend/src/components/ui/input-otp.tsx

```

"use client"

import * as React from "react"
import { OTPInput, OTPInputContext } from "input-otp"
import { MinusIcon } from "lucide-react"

import { cn } from "@lib/Utils"

function InputOTP({
  className,
  containerClassName,
  ...props
}: React.ComponentProps<typeof OTPInput> & {
  containerClassName?: string
}) {
  return (
    <OTPInput
      data-slot="input-otp"
      containerClassName={cn(
        "flex items-center gap-2 has-disabled:opacity-50",
        containerClassName
      )}
      className={cn("disabled:cursor-not-allowed", className)}
      {...props}
    />
  )
}

function InputOTPGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="input-otp-group"
      className={cn("flex items-center", className)}
      {...props}
    />
  )
}

function InputOTPSlot({
  index,
  className,
  ...props
}: React.ComponentProps<"div"> & {
  index: number
}) {
  const inputOTPContext = React.useContext(OTPInputContext)
  const { char, hasFakeCaret, isActive } = inputOTPContext?.slots[index] ?? {}

  return (
    <div
      data-slot="input-otp-slot"
      data-active={isActive}
      className={cn(
        "data-[active=true]:border-ring data-[active=true]:ring-ring/50 data-[active=true]:aria-invalid:ring-destructive",
        className
      )}
      {...props}
    >
      {char}
      {hasFakeCaret & & (
        <div className="pointer-events-none absolute inset-0 flex items-center justify-center">
          <div className="animate-caret-blink bg-foreground h-4 w-px duration-1000" />
        )}
    )}

```

```

        </div>
      )
    }
  }
}

function InputOTPSeparator({ ...props }: React.ComponentProps<"div">) {
  return (
    <div data-slot="input-otp-separator" role="separator" {...props}>
      <MinusIcon />
    </div>
  )
}

export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator }

```

■ frontend/src/components/ui/input.tsx

```

import * as React from "react"

import { cn } from "@lib/Utils"

function Input({ className, type, ...props }: React.ComponentProps<"input">) {
  return (
    <input
      type={type}
      data-slot="input"
      className={cn(
        "file:text-foreground placeholder:text-muted-foreground selection:bg-primary selection:text-primary-foreground",
        "focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px]",
        "aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive",
        className
      )}
      {...props}
    />
  )
}

export { Input }

```

■ frontend/src/components/ui/label.tsx

```

"use client"

import * as React from "react"
import * as LabelPrimitive from "@radix-ui/react-label"

import { cn } from "@lib/Utils"

function Label({
  className,
  ...props
}: React.ComponentProps<typeof LabelPrimitive.Root>) {
  return (
    <LabelPrimitive.Root
      data-slot="label"
      className={cn(
        "flex items-center gap-2 text-sm leading-none font-medium select-none group-data-[disabled=true]:pointer-events-none",
        className
      )}
      {...props}
    />
  )
}

export { Label }

```


■ frontend/src/components/ui/menubar.tsx

```
import * as React from "react"
import * as MenubarPrimitive from "@radix-ui/react-menubar"
import { CheckIcon, ChevronRightIcon, CircleIcon } from "lucide-react"

import { cn } from "@lib/utils"

function Menubar({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Root> & {
  return (
    <MenubarPrimitive.Root
      data-slot="menubar"
      className={cn(
        "bg-background flex h-9 items-center gap-1 rounded-md border p-1 shadow-xs",
        className
      )}
      {...props}
    />
  )
}

function MenubarMenu({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Menu> & {
  return <MenubarPrimitive.Menu data-slot="menubar-menu" {...props} />
}

function MenubarGroup({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Group> & {
  return <MenubarPrimitive.Group data-slot="menubar-group" {...props} />
}

function MenubarPortal({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Portal> & {
  return <MenubarPrimitive.Portal data-slot="menubar-portal" {...props} />
}

function MenubarRadioGroup({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.RadioGroup> & {
  return (
    <MenubarPrimitive.RadioGroup data-slot="menubar-radio-group" {...props} />
  )
}

function MenubarTrigger({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Trigger> & {
  return (
    <MenubarPrimitive.Trigger
      data-slot="menubar-trigger"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
        className
      )}
      {...props}
    />
  )
}

function MenubarContent({
  className,
  align = "start",
  alignOffset = -4,
```

```

    sideOffset = 8,
    ...props
  }: React.ComponentProps<typeof MenubarPrimitive.Content>); {
    return (
      <MenubarPortal>
        <MenubarPrimitive.Content
          data-slot="menubar-content"
          align={align}
          alignOffset={alignOffset}
          sideOffset={sideOffset}
          className={cn(
            "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
            className
          )}
          {...props}
        />
      </MenubarPortal>
    )
  }

function MenubarItem({
  className,
  inset,
  variant = "default",
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Item> & {
  inset?: boolean
  variant?: "default" | "destructive"
}) {
  return (
    <MenubarPrimitive.Item
      data-slot="menubar-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[variant=destructive]:text-destructive data-[variant=destructive]:bg-destructive",
        className
      )}
      {...props}
    />
  )
}

function MenubarCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.CheckboxItem> & {
  checked?: boolean
}) {
  return (
    <MenubarPrimitive.CheckboxItem
      data-slot="menubar-checkbox-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2 rounded-xs py-1.5",
        className
      )}
      checked={checked}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
        <MenubarPrimitive.ItemIndicator>
          <CheckIcon className="size-4" />
        </MenubarPrimitive.ItemIndicator>
      </span>
      {children}
    </MenubarPrimitive.CheckboxItem>
  )
}

function MenubarRadioItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.RadioItem> & {
  checked?: boolean
}) {
  return (
    <MenubarPrimitive.RadioItem
      data-slot="menubar-radio-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2 rounded-xs py-1.5",
        className
      )}
      checked={checked}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
        <MenubarPrimitive.ItemIndicator>
          <RadioIcon className="size-4" />
        </MenubarPrimitive.ItemIndicator>
      </span>
      {children}
    </MenubarPrimitive.RadioItem>
  )
}

```

```

    children,
    ...props
  }: React.ComponentProps<typeof MenubarPrimitive.RadioItem> >) {
    return (
      <MenubarPrimitive.RadioItem
        data-slot="menubar-radio-item"
        className={cn(
          "focus:bg-accent focus:text-accent-foreground relative flex cursor-default items-center gap-2 rounded-xs py-1.5",
          className
        )}
        {...props}
      >
        <span className="pointer-events-none absolute left-2 flex size-3.5 items-center justify-center">
          <MenubarPrimitive.ItemIndicator>
            <CircleIcon className="size-2 fill-current" />
          </MenubarPrimitive.ItemIndicator>
        </span>
        {children}
      </MenubarPrimitive.RadioItem>
    )
  }
}

function MenubarLabel({
  className,
  inset,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Label> & {
  inset?: boolean
}) {
  return (
    <MenubarPrimitive.Label
      data-slot="menubar-label"
      data-inset={inset}
      className={cn(
        "px-2 py-1.5 text-sm font-medium data-[inset]:pl-8",
        className
      )}
      {...props}
    />
  )
}

function MenubarSeparator({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Separator> >) {
  return (
    <MenubarPrimitive.Separator
      data-slot="menubar-separator"
      className={cn("bg-border -mx-1 my-1 h-px", className)}
      {...props}
    />
  )
}

function MenubarShortcut({
  className,
  ...props
}: React.ComponentProps<"span"> >) {
  return (
    <span
      data-slot="menubar-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className
      )}
      {...props}
    />
  )
}

function MenubarSub({

```

```

    ...props
  }: React.ComponentProps<typeof MenubarPrimitive.Sub>); {
    return <MenubarPrimitive.Sub data-slot="menubar-sub" {...props} />;
  }

function MenubarSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.SubTrigger> & {
  inset?: boolean
}) {
  return (
    <MenubarPrimitive.SubTrigger
      data-slot="menubar-sub-trigger"
      data-inset={inset}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
        className
      )}
      {...props}
    >
      {children}
      <ChevronRightIcon className="ml-auto h-4 w-4" />
    </MenubarPrimitive.SubTrigger>
  )
}

function MenubarSubContent({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.SubContent>); {
  return (
    <MenubarPrimitive.SubContent
      data-slot="menubar-sub-content"
      className={cn(
        "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
        className
      )}
      {...props}
    />
  )
}

export {
  Menubar,
  MenubarPortal,
  MenubarMenu,
  MenubarTrigger,
  MenubarContent,
  MenubarGroup,
  MenubarSeparator,
  MenubarLabel,
  MenubarItem,
  MenubarShortcut,
  MenubarCheckboxItem,
  MenubarRadioGroup,
  MenubarRadioItem,
  MenubarSub,
  MenubarSubTrigger,
  MenubarSubContent,
}

```

■ frontend/src/components/ui/navigation-menu.tsx

```

import * as React from "react"
import * as NavigationMenuPrimitive from "@radix-ui/react-navigation-menu"
import { cva } from "class-variance-authority"
import { ChevronDownIcon } from "lucide-react"

```

```

import { cn } from "@lib/utils"

function NavigationMenu({
  className,
  children,
  viewport = true,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Root> & {
  viewport?: boolean
}) {
  return (
    <NavigationMenuPrimitive.Root
      data-slot="navigation-menu"
      data-viewport={viewport}
      className={cn(
        "group/navigation-menu relative flex max-w-max flex-1 items-center justify-center",
        className
      )}
      {...props}
    >
      {children}
      {viewport & & <NavigationMenuViewport />}
    </NavigationMenuPrimitive.Root>
  )
}

function NavigationMenuList({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.List>) {
  return (
    <NavigationMenuPrimitive.List
      data-slot="navigation-menu-list"
      className={cn(
        "group flex flex-1 list-none items-center justify-center gap-1",
        className
      )}
      {...props}
    />
  )
}

function NavigationMenuItem({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Item>) {
  return (
    <NavigationMenuPrimitive.Item
      data-slot="navigation-menu-item"
      className={cn("relative", className)}
      {...props}
    />
  )
}

const navigationMenuTriggerStyle = cva(
  "group inline-flex h-9 w-max items-center justify-center rounded-md bg-background px-4 py-2 text-sm font-medium hover:"
)

function NavigationMenuTrigger({
  className,
  children,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Trigger>) {
  return (
    <NavigationMenuPrimitive.Trigger
      data-slot="navigation-menu-trigger"
      className={cn(navigationMenuTriggerStyle(), "group", className)}
      {...props}
    >
      {children}{" "}
    </>
  )
}

```

```

        <ChevronDownIcon
          className="relative top-[1px] ml-1 size-3 transition duration-300 group-data-[state=open]:rotate-180"
          aria-hidden="true"
        />
      </NavigationMenuPrimitive.Trigger>
    )
  }

function NavigationMenuContent({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Content>) {
  return (
    <NavigationMenuPrimitive.Content
      data-slot="navigation-menu-content"
      className={cn(
        "data-[motion^=from-]:animate-in data-[motion^=to-]:animate-out data-[motion^=from-]:fade-in data-[motion^=to-]:fade-out"
        "group-data-[viewport=false]/navigation-menu:bg-popover group-data-[viewport=false]/navigation-menu:text-popover"
        className
      )}
      {...props}
    />
  )
}

function NavigationMenuViewport({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Viewport>) {
  return (
    <div
      className={cn(
        "absolute top-full left-0 isolate z-50 flex justify-center"
      )}
    >
      <NavigationMenuPrimitive.Viewport
        data-slot="navigation-menu-viewport"
        className={cn(
          "origin-top-center bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out"
          className
        )}
        {...props}
      />
    </div>
  )
}

function NavigationMenuLink({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Link>) {
  return (
    <NavigationMenuPrimitive.Link
      data-slot="navigation-menu-link"
      className={cn(
        "data-[active=true]:focus:bg-accent data-[active=true]:hover:bg-accent data-[active=true]:bg-accent/50 data-[active=true]:text-accent"
        className
      )}
      {...props}
    />
  )
}

function NavigationMenuIndicator({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Indicator>) {
  return (
    <NavigationMenuPrimitive.Indicator
      data-slot="navigation-menu-indicator"
      className={cn(

```

```

        "data-[state=visible]:animate-in data-[state=hidden]:animate-out data-[state=hidden]:fade-out data-[state=visible]:fade-in"
        className
      )}
      {...props}
    >
    <div className="bg-border relative top-[60%] h-2 w-2 rotate-45 rounded-tl-sm shadow-md" />
    </NavigationMenuPrimitive.Indicator>
  )
}

export {
  NavigationMenu,
  NavigationMenuList,
  NavigationMenuItem,
  NavigationMenuContent,
  NavigationMenuTrigger,
  NavigationMenuLink,
  NavigationMenuIndicator,
  NavigationMenuViewport,
  navigationMenuTriggerStyle,
}

```

■ frontend/src/components/ui/pagination.tsx

```

import * as React from "react"
import {
  ChevronLeftIcon,
  ChevronRightIcon,
  MoreHorizontalIcon,
} from "lucide-react"

import { cn } from "@lib/utlils"
import { Button, buttonVariants } from "@components/ui/button"

function Pagination({ className, ...props }: React.ComponentProps<"nav">) {
  return (
    <nav
      role="navigation"
      aria-label="pagination"
      data-slot="pagination"
      className={cn("mx-auto flex w-full justify-center", className)}
      {...props}
    />
  )
}

function PaginationContent({
  className,
  ...props
}: React.ComponentProps<"ul">) {
  return (
    <ul
      data-slot="pagination-content"
      className={cn("flex flex-row items-center gap-1", className)}
      {...props}
    />
  )
}

function PaginationItem({ ...props }: React.ComponentProps<"li">) {
  return <li data-slot="pagination-item" {...props} />
}

type PaginationLinkProps = {
  isActive?: boolean
  disabled?: boolean
} & Pick<React.ComponentProps<typeof Button>, "size"> &
  React.ComponentProps<"a">

function PaginationLink({

```

```

className,
isActive,
disabled,
size = "icon",
onClick,
...props
}: PaginationLinkProps) {
  return (
    <a
      aria-current={isActive ? "page" : undefined}
      aria-disabled={disabled ? true : undefined}
      tabIndex={disabled ? -1 : props.tabIndex}
      data-slot="pagination-link"
      data-active={isActive}
      className={cn(
        buttonVariants({
          variant: isActive ? "outline" : "ghost",
          size,
        }),
        disabled ? "pointer-events-none opacity-50" : undefined,
        className
      )}
      onClick={{(e) => {
        if (disabled) {
          e.preventDefault()
          return
        }
        onClick?.(e)
      }}}
      {...props}
    />
  )
}

function PaginationPrevious({
  className,
  ...props
}: React.ComponentProps<typeof PaginationLink> & {
  return (
    <PaginationLink
      aria-label="Go to previous page"
      size="default"
      className={cn("gap-1 px-2.5 sm:pl-2.5", className)}
      {...props}
    >
      <ChevronLeftIcon />
      <span className="hidden sm:block">Previous</span>
    </PaginationLink>
  )
}

function PaginationNext({
  className,
  ...props
}: React.ComponentProps<typeof PaginationLink> & {
  return (
    <PaginationLink
      aria-label="Go to next page"
      size="default"
      className={cn("gap-1 px-2.5 sm:pr-2.5", className)}
      {...props}
    >
      <span className="hidden sm:block">Next</span>
      <ChevronRightIcon />
    </PaginationLink>
  )
}

function PaginationEllipsis({
  className,
  ...props

```



```

}; React.ComponentProps<span>); {
  return (
    <span
      aria-hidden
      data-slot="pagination-ellipsis"
      className={cn("flex size-9 items-center justify-center", className)}
      {...props}
    />
    <MoreHorizontalIcon className="size-4" />
    <span className="sr-only">More pages</span>
  );
}

export {
  Pagination,
  PaginationContent,
  PaginationLink,
  PaginationItem,
  PaginationPrevious,
  PaginationNext,
  PaginationEllipsis,
}

```

■ frontend/src/components/ui/popover.tsx

```

"use client"

import * as React from "react"
import * as PopoverPrimitive from "@radix-ui/react-popover"

import { cn } from "@lib/Utils"

function Popover({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Root>): {
  return <PopoverPrimitive.Root data-slot="popover" {...props} />
}

function PopoverTrigger({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Trigger>): {
  return <PopoverPrimitive.Trigger data-slot="popover-trigger" {...props} />
}

function PopoverContent({
  className,
  align = "center",
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Content>): {
  return (
    <PopoverPrimitive.Portal>
      <PopoverPrimitive.Content
        data-slot="popover-content"
        align={align}
        sideOffset={sideOffset}
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
          className
        )}
        {...props}
      />
    </PopoverPrimitive.Portal>
  )
}

function PopoverAnchor({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Anchor>): {

```

```

    return <PopoverPrimitive.Anchor data-slot="popover-anchor" {...props} />;
  }

export { Popover, PopoverTrigger, PopoverContent, PopoverAnchor }

```

■ frontend/src/components/ui/progress.tsx

```

import * as React from "react"
import * as ProgressPrimitive from "@radix-ui/react-progress"

import { cn } from "@lib/Utils"

function Progress({
  className,
  value,
  ...props
}: React.ComponentProps<typeof ProgressPrimitive.Root>) {
  return (
    <ProgressPrimitive.Root
      data-slot="progress"
      className={cn(
        "bg-primary/20 relative h-2 w-full overflow-hidden rounded-full",
        className
      )}
      {...props}
    >
      <ProgressPrimitive.Indicator
        data-slot="progress-indicator"
        className="bg-primary h-full w-full flex-1 transition-all"
        style={{ transform: `translateX(-${100 - (value || 0)}%)` }}
      />
    </ProgressPrimitive.Root>
  )
}

export { Progress }

```

■ frontend/src/components/ui/radio-group.tsx

```

"use client"

import * as React from "react"
import * as RadioGroupPrimitive from "@radix-ui/react-radio-group"
import { CircleIcon } from "lucide-react"

import { cn } from "@lib/Utils"

function RadioGroup({
  className,
  ...props
}: React.ComponentProps<typeof RadioGroupPrimitive.Root>) {
  return (
    <RadioGroupPrimitive.Root
      data-slot="radio-group"
      className={cn("grid gap-3", className)}
      {...props}
    />
  )
}

function RadioGroupItem({
  className,
  ...props
}: React.ComponentProps<typeof RadioGroupPrimitive.Item>) {
  return (
    <RadioGroupPrimitive.Item
      data-slot="radio-group-item"
      className={cn(

```

```

        "border-input text-primary focus-visible:border-ring focus-visible:ring-ring/50 aria-invalid:ring-destructive/2"
        className
      )}
      {...props}
    >
    <RadioGroupPrimitive.Indicator
      data-slot="radio-group-indicator"
      className="relative flex items-center justify-center"
    >
      <CircleIcon className="fill-primary absolute top-1/2 left-1/2 size-2 -translate-x-1/2 -translate-y-1/2" />
    </RadioGroupPrimitive.Indicator>
  </RadioGroupPrimitive.Item>
)
}

export { RadioGroup, RadioGroupItem }

```

■ frontend/src/components/ui/resizable.tsx

```

import * as React from "react"
import { GripVerticalIcon } from "lucide-react"
import * as ResizablePrimitive from "react-resizable-panels"

import { cn } from "@/lib/utils"

function ResizablePanelGroup({
  className,
  ...props
}: React.ComponentProps<typeof ResizablePrimitive.PanelGroup>) {
  return (
    <ResizablePrimitive.PanelGroup
      data-slot="resizable-panel-group"
      className={cn(
        "flex h-full w-full data-[panel-group-direction=vertical]:flex-col",
        className
      )}
      {...props}
    />
  )
}

function ResizablePanel({
  ...props
}: React.ComponentProps<typeof ResizablePrimitive.Panel>) {
  return <ResizablePrimitive.Panel data-slot="resizable-panel" {...props} />
}

function ResizableHandle({
  withHandle,
  className,
  ...props
}: React.ComponentProps<typeof ResizablePrimitive.PanelResizeHandle> & {
  withHandle?: boolean
}) {
  return (
    <ResizablePrimitive.PanelResizeHandle
      data-slot="resizable-handle"
      className={cn(
        "bg-border focus-visible:ring-ring relative flex w-px items-center justify-center after:absolute after:inset-y-0",
        className
      )}
      {...props}
    >
    {withHandle & & (
      <div className="bg-border z-10 flex h-4 w-3 items-center justify-center rounded-xs border">
        <GripVerticalIcon className="size-2.5" />
      </div>
    )}
    </ResizablePrimitive.PanelResizeHandle>
  )
}

```

```

}

export { ResizablePanelGroup, ResizablePanel, ResizableHandle }

```

■ frontend/src/components/ui/scroll-area.tsx

```

"use client"

import * as React from "react"
import * as ScrollAreaPrimitive from "@radix-ui/react-scroll-area"

import { cn } from "@lib/Utils"

function ScrollArea({
  className,
  children,
  ...props
}: React.ComponentProps<typeof ScrollAreaPrimitive.Root> & {
  return (
    <ScrollAreaPrimitive.Root
      data-slot="scroll-area"
      className={cn("relative", className)}
      {...props}
    >
      <ScrollAreaPrimitive.Viewport
        data-slot="scroll-area-viewport"
        className="focus-visible:ring-ring/50 size-full rounded-[inherit] transition-[color,box-shadow] outline-none fo
      >
        {children}
      </ScrollAreaPrimitive.Viewport>
      <ScrollBar />
      <ScrollAreaPrimitive.Corner />
    </ScrollAreaPrimitive.Root>
  )
}

function ScrollBar({
  className,
  orientation = "vertical",
  ...props
}: React.ComponentProps<typeof ScrollAreaPrimitive.ScrollAreaScrollbar> & {
  return (
    <ScrollAreaPrimitive.ScrollAreaScrollbar
      data-slot="scroll-area-scrollbar"
      orientation={orientation}
      className={cn(
        "flex touch-none p-px transition-colors select-none",
        orientation === "vertical" &&
          "h-full w-2.5 border-l border-l-transparent",
        orientation === "horizontal" &&
          "h-2.5 flex-col border-t border-t-transparent",
        className
      )}
      {...props}
    >
      <ScrollAreaPrimitive.ScrollAreaThumb
        data-slot="scroll-area-thumb"
        className="bg-border relative flex-1 rounded-full"
      />
    </ScrollAreaPrimitive.ScrollAreaScrollbar>
  )
}

export { ScrollArea, ScrollBar }

```

■ frontend/src/components/ui/select.tsx

```

import * as React from "react"

```

```

import * as SelectPrimitive from "@radix-ui/react-select"
import { CheckIcon, ChevronDownIcon, ChevronUpIcon } from "lucide-react"

import { cn } from "@lib/utls"

function Select({
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Root> & {
  return <SelectPrimitive.Root data-slot="select" {...props} /&gt;
}) {

function SelectGroup({
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Group> & {
  return <SelectPrimitive.Group data-slot="select-group" {...props} /&gt;
}) {

function SelectValue({
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Value> & {
  return <SelectPrimitive.Value data-slot="select-value" {...props} /&gt;
}) {

function SelectTrigger({
  className,
  size = "default",
  children,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Trigger> & {
  size?: "sm" | "default"
}) {
  return (
    <SelectPrimitive.Trigger
      data-slot="select-trigger"
      data-size={size}
      className={cn(
        "border-input data-[placeholder]:text-muted-foreground [&_svg:not([class*='text-']):text-muted-foreground"
        className
      )}
      {...props}
    >
    {children}
    <SelectPrimitive.Icon asChild>
      <ChevronDownIcon className="size-4 opacity-50" /&gt;
    </SelectPrimitive.Icon>
    </SelectPrimitive.Trigger>
  )
}

function SelectContent({
  className,
  children,
  position = "popper",
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Content> & {
  return (
    <SelectPrimitive.Portal>
    <SelectPrimitive.Content
      data-slot="select-content"
      className={cn(
        "bg-popover text-popover-foreground data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0"
        "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1 data-[side=right]:translate-x-1 data-[side=top]:-translate-y-1"
        className
      )}
      position={position}
      {...props}
    >
    <SelectScrollUpButton /&gt;
    <SelectPrimitive.Viewport
      className={cn(

```



```

        "flex cursor-default items-center justify-center py-1",
        className
    )}
    {...props}
    &gt;
    &lt;ChevronUpIcon className="size-4" /&gt;
    &lt;/SelectPrimitive.ScrollUpButton&gt;
)
}

function SelectScrollDownButton({
  className,
  ...props
}): React.ComponentProps&lt;typeof SelectPrimitive.ScrollDownButton&gt; {
  return (
    &lt;SelectPrimitive.ScrollDownButton
      data-slot="select-scroll-down-button"
      className={cn(
        "flex cursor-default items-center justify-center py-1",
        className
      )}
      {...props}
    &gt;
    &lt;ChevronDownIcon className="size-4" /&gt;
    &lt;/SelectPrimitive.ScrollDownButton&gt;
  )
}

export {
  Select,
  SelectContent,
  SelectGroup,
  SelectItem,
  SelectLabel,
  SelectScrollDownButton,
  SelectScrollUpButton,
  SelectSeparator,
  SelectTrigger,
  SelectValue,
}

```

■ frontend/src/components/ui/separator.tsx

```

"use client"

import * as React from "react"
import * as SeparatorPrimitive from "@radix-ui/react-separator"

import { cn } from "@/lib/utils"

function Separator({
  className,
  orientation = "horizontal",
  decorative = true,
  ...props
}): React.ComponentProps&lt;typeof SeparatorPrimitive.Root&gt; {
  return (
    &lt;SeparatorPrimitive.Root
      data-slot="separator"
      decorative={decorative}
      orientation={orientation}
      className={cn(
        "bg-border shrink-0 data-[orientation=horizontal]:h-px data-[orientation=horizontal]:w-full data-[orientation=vertical]:h-full data-[orientation=vertical]:w-px",
        className
      )}
      {...props}
    /&gt;
  )
}

```

```
export { Separator }
```

■ frontend/src/components/ui/sheet.tsx

```
import * as React from "react"
import * as SheetPrimitive from "@radix-ui/react-dialog"
import { XIcon } from "lucide-react"

import { cn } from "@lib/utils"

function Sheet({ ...props }: React.ComponentProps<typeof SheetPrimitive.Root> ) {
  return <SheetPrimitive.Root data-slot="sheet" {...props} />
}

function SheetTrigger({
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Trigger>) {
  return <SheetPrimitive.Trigger data-slot="sheet-trigger" {...props} />
}

function SheetClose({
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Close>) {
  return <SheetPrimitive.Close data-slot="sheet-close" {...props} />
}

function SheetPortal({
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Portal> ) {
  return <SheetPrimitive.Portal data-slot="sheet-portal" {...props} />
}

function SheetOverlay({
  className,
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Overlay>) {
  return (
    <SheetPrimitive.Overlay
      data-slot="sheet-overlay"
      className={cn(
        "data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:",
        className
      )}
      {...props}
    />
  )
}

function SheetContent({
  className,
  children,
  side = "right",
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Content> & {
  side?: "top" | "right" | "bottom" | "left"
}) {
  return (
    <SheetPortal>
      <SheetOverlay />
      <SheetPrimitive.Content
        data-slot="sheet-content"
        className={cn(
          "bg-background data-[state=open]:animate-in data-[state=closed]:animate-out fixed z-50 flex flex-col gap-4 sh",
          side === "right" &&
            "data-[state=closed]:slide-out-to-right data-[state=open]:slide-in-from-right inset-y-0 right-0 h-full w-3/4",
          side === "left" &&
            "data-[state=closed]:slide-out-to-left data-[state=open]:slide-in-from-left inset-y-0 left-0 h-full w-3/4 b",
          side === "top" &&
            "data-[state=closed]:slide-out-to-top data-[state=open]:slide-in-from-top inset-x-0 top-0 h-auto border-b",
          side === "bottom" &&

```



```

        "data-[state=closed]:slide-out-to-bottom data-[state=open]:slide-in-from-bottom inset-x-0 bottom-0 h-auto b
        className
    })
    {...props}
    &gt;
    {children}
    &lt;SheetPrimitive.Close className="ring-offset-background focus:ring-ring data-[state=open]:bg-secondary absol
        &lt;XIcon className="size-4" /&gt;
        &lt;span className="sr-only"&gt;Close&lt;/span&gt;
    &lt;/SheetPrimitive.Close&gt;
    &lt;/SheetPrimitive.Content&gt;
    &lt;/SheetPortal&gt;
)
}

function SheetHeader({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
    return (
        &lt;div
            data-slot="sheet-header"
            className={cn("flex flex-col gap-1.5 p-4", className)}
            {...props}
        /&gt;
    )
}

function SheetFooter({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
    return (
        &lt;div
            data-slot="sheet-footer"
            className={cn("mt-auto flex flex-col gap-2 p-4", className)}
            {...props}
        /&gt;
    )
}

function SheetTitle({
    className,
    ...props
}: React.ComponentProps&lt;typeof SheetPrimitive.Title&gt;) {
    return (
        &lt;SheetPrimitive.Title
            data-slot="sheet-title"
            className={cn("text-foreground font-semibold", className)}
            {...props}
        /&gt;
    )
}

function SheetDescription({
    className,
    ...props
}: React.ComponentProps&lt;typeof SheetPrimitive.Description&gt;) {
    return (
        &lt;SheetPrimitive.Description
            data-slot="sheet-description"
            className={cn("text-muted-foreground text-sm", className)}
            {...props}
        /&gt;
    )
}

export {
    Sheet,
    SheetTrigger,
    SheetClose,
    SheetContent,
    SheetHeader,
    SheetFooter,
    SheetTitle,
    SheetDescription,
}

```

■ frontend/src/components/ui/sidebar.tsx

```
"use client"

import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva, VariantProps } from "class-variance-authority"
import { PanelLeftIcon } from "lucide-react"

import { useIsMobile } from "@hooks/use-mobile"
import { cn } from "@lib/utils"
import { Button } from "@components/ui/button"
import { Input } from "@components/ui/input"
import { Separator } from "@components/ui/separator"
import {
  Sheet,
  SheetContent,
  SheetDescription,
  SheetHeader,
  SheetTitle,
} from "@components/ui/sheet"
import { Skeleton } from "@components/ui/skeleton"
import {
  Tooltip,
  TooltipContent,
  TooltipProvider,
  TooltipTrigger,
} from "@components/ui/tooltip"

const SIDEBAR_COOKIE_NAME = "sidebar_state"
const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7
const SIDEBAR_WIDTH = "16rem"
const SIDEBAR_WIDTH_MOBILE = "18rem"
const SIDEBAR_WIDTH_ICON = "3rem"
const SIDEBAR_KEYBOARD_SHORTCUT = "b"

type SidebarContextProps = {
  state: "expanded" | "collapsed"
  open: boolean
  setOpen: (open: boolean) => void
  openMobile: boolean
  setOpenMobile: (open: boolean) => void
  isMobile: boolean
  toggleSidebar: () => void
}

const SidebarContext = React.createContext<SidebarContextProps | null>(null)

function useSidebar() {
  const context = React.useContext(SidebarContext)
  if (!context) {
    throw new Error("useSidebar must be used within a SidebarProvider.")
  }

  return context
}

function SidebarProvider({
  defaultOpen = true,
  open: openProp,
  onOpenChange: setOpenProp,
  className,
  style,
  children,
  ...props
}: React.ComponentProps<"div"> & {
  defaultOpen?: boolean
  open?: boolean
  onOpenChange?: (open: boolean) => void
```

```

}) {
  const isMobile = useIsMobile()
  const [openMobile, setOpenMobile] = React.useState(false)

  // This is the internal state of the sidebar.
  // We use openProp and setOpenProp for control from outside the component.
  const [_open, _setOpen] = React.useState(defaultOpen)
  const open = openProp ?? _open
  const setOpen = React.useCallback(
    (value: boolean | ((value: boolean) => boolean)) => {
      const openState = typeof value === "function" ? value(open) : value
      if (setOpenProp) {
        setOpenProp(openState)
      } else {
        _setOpen(openState)
      }
    },

    // This sets the cookie to keep the sidebar state.
    document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/; max-age=${SIDEBAR_COOKIE_MAX_AGE}`
  ),
  [setOpenProp, open]
)

// Helper to toggle the sidebar.
const toggleSidebar = React.useCallback(() => {
  return isMobile ? setOpenMobile((open) => !open) : setOpen((open) => !open)
}, [isMobile, setOpen, setOpenMobile])

// Adds a keyboard shortcut to toggle the sidebar.
React.useEffect(() => {
  const handleKeyDown = (event: KeyboardEvent) => {
    if (
      event.key === SIDEBAR_KEYBOARD_SHORTCUT &&
      (event.metaKey || event.ctrlKey)
    ) {
      event.preventDefault()
      toggleSidebar()
    }
  }

  window.addEventListener("keydown", handleKeyDown)
  return () => window.removeEventListener("keydown", handleKeyDown)
}, [toggleSidebar])

// We add a state so that we can do data-state="expanded" or "collapsed".
// This makes it easier to style the sidebar with Tailwind classes.
const state = open ? "expanded" : "collapsed"

const contextValue = React.useMemo<SidebarContextProps>(() => ({
  state,
  open,
  setOpen,
  isMobile,
  openMobile,
  setOpenMobile,
  toggleSidebar,
}),
[state, open, setOpen, isMobile, openMobile, setOpenMobile, toggleSidebar])

return (
  <SidebarContext.Provider value={contextValue}>
    <TooltipProvider delayDuration={0}>
      <div
        data-slot="sidebar-wrapper"
        style={
          {
            "--sidebar-width": SIDEBAR_WIDTH,
            "--sidebar-width-icon": SIDEBAR_WIDTH_ICON,
            ...style,

```

```

        } as React.CSSProperties
      }
      className={cn(
        "group/sidebar-wrapper has-data-[variant=inset]:bg-sidebar flex min-h-svh w-full",
        className
      )}
      {...props}
    >
      {children}
    </div>
  </TooltipProvider>
  </SidebarContext.Provider>
)
}

function Sidebar({
  side = "left",
  variant = "sidebar",
  collapsible = "offcanvas",
  className,
  children,
  ...props
}: React.ComponentProps<"div"> & {
  side?: "left" | "right"
  variant?: "sidebar" | "floating" | "inset"
  collapsible?: "offcanvas" | "icon" | "none"
}) {
  const { isMobile, state, openMobile, setOpenMobile } = useSidebar()

  if (collapsible === "none") {
    return (
      <div
        data-slot="sidebar"
        className={cn(
          "bg-sidebar text-sidebar-foreground flex h-full w-(--sidebar-width) flex-col",
          className
        )}
        {...props}
      >
        {children}
      </div>
    )
  }

  if (isMobile) {
    return (
      <Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}>
        <SheetContent
          data-sidebar="sidebar"
          data-slot="sidebar"
          data-mobile="true"
          className="bg-sidebar text-sidebar-foreground w-(--sidebar-width) p-0 [&button]:hidden"
          style={
            {
              "--sidebar-width": SIDEBAR_WIDTH_MOBILE,
            } as React.CSSProperties
          }
          side={side}
        >
          <SheetHeader className="sr-only">
            <SheetTitle>Sidebar</SheetTitle>
            <SheetDescription>Displays the mobile sidebar.</SheetDescription>
          </SheetHeader>
          <div className="flex h-full w-full flex-col">{children}</div>
        </SheetContent>
      </Sheet>
    )
  }

  return (
    <div

```

```

    className="group peer text-sidebar-foreground hidden md:block"
    data-state={state}
    data-collapsible={state === "collapsed" ? collapsible : ""}
    data-variant={variant}
    data-side={side}
    data-slot="sidebar"
  >
    /* This is what handles the sidebar gap on desktop */
    <div
      data-slot="sidebar-gap"
      className={cn(
        "relative w-(--sidebar-width) bg-transparent transition-[width] duration-200 ease-linear",
        "group-data-[collapsible=offcanvas]:w-0",
        "group-data-[side=right]:rotate-180",
        variant === "floating" || variant === "inset"
          ? "group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)+(--spacing(4)))]"
          : "group-data-[collapsible=icon]:w-(--sidebar-width-icon)"
      )}
    />
    <div
      data-slot="sidebar-container"
      className={cn(
        "fixed inset-y-0 z-10 hidden h-svh w-(--sidebar-width) transition-[left,right,width] duration-200 ease-linear",
        side === "left"
          ? "left-0 group-data-[collapsible=offcanvas]:left-[calc(var(--sidebar-width)*-1)]"
          : "right-0 group-data-[collapsible=offcanvas]:right-[calc(var(--sidebar-width)*-1)]",
        // Adjust the padding for floating and inset variants.
        variant === "floating" || variant === "inset"
          ? "p-2 group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)+(--spacing(4))+2px)]"
          : "group-data-[collapsible=icon]:w-(--sidebar-width-icon) group-data-[side=left]:border-r group-data-[side=right]:border-l",
        className
      )}
      {...props}
    />
    <div
      data-sidebar="sidebar"
      data-slot="sidebar-inner"
      className="bg-sidebar group-data-[variant=floating]:border-sidebar-border flex h-full w-full flex-col group-data-[variant=floating]:rounded-lg"
    />
    {children}
  />
</div>
</div>
</div>
)
}

function SidebarTrigger({
  className,
  onClick,
  ...props
}: React.ComponentProps<typeof Button>) {
  const { toggleSidebar } = useSidebar()

  return (
    <Button
      data-sidebar="trigger"
      data-slot="sidebar-trigger"
      variant="ghost"
      size="icon"
      className={cn("size-7", className)}
      onClick={(event) => {
        onClick?.(event)
        toggleSidebar()
      }}
      {...props}
    />
    <PanelLeftIcon />
    <span className="sr-only">Toggle Sidebar</span>
    </Button>
  )
}

```

```

function SidebarRail({ className, ...props }: React.ComponentProps<"button"> ) {
  const { toggleSidebar } = useSidebar()

  return (
    <button
      data-sidebar="rail"
      data-slot="sidebar-rail"
      aria-label="Toggle Sidebar"
      tabIndex=-1
      onClick={toggleSidebar}
      title="Toggle Sidebar"
      className={cn(
        "hover:after:bg-sidebar-border absolute inset-y-0 z-20 hidden w-4 -translate-x-1/2 transition-all ease-linear",
        "in-data-[side=left]:cursor-w-resize in-data-[side=right]:cursor-e-resize",
        "[[data-side=left][data-state=collapsed]&]:cursor-e-resize [[data-side=right][data-state=collapsed]&]:",
        "hover:group-data-[collapsible=offcanvas]:bg-sidebar group-data-[collapsible=offcanvas]:translate-x-0 group-dat",
        "[[data-side=left][data-collapsible=offcanvas]&]:-right-2",
        "[[data-side=right][data-collapsible=offcanvas]&]:-left-2",
        className
      )}
      {...props}
    />
  )
}

function SidebarInset({ className, ...props }: React.ComponentProps<"main"> ) {
  return (
    <main
      data-slot="sidebar-inset"
      className={cn(
        "bg-background relative flex w-full flex-1 flex-col",
        "md:peer-data-[variant=inset]:m-2 md:peer-data-[variant=inset]:ml-0 md:peer-data-[variant=inset]:rounded-xl md:",
        className
      )}
      {...props}
    />
  )
}

function SidebarInput({
  className,
  ...props
}: React.ComponentProps<typeof Input> ) {
  return (
    <Input
      data-slot="sidebar-input"
      data-sidebar="input"
      className={cn("bg-background h-8 w-full shadow-none", className)}
      {...props}
    />
  )
}

function SidebarHeader({ className, ...props }: React.ComponentProps<"div"> ) {
  return (
    <div
      data-slot="sidebar-header"
      data-sidebar="header"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props}
    />
  )
}

function SidebarFooter({ className, ...props }: React.ComponentProps<"div"> ) {
  return (
    <div
      data-slot="sidebar-footer"
      data-sidebar="footer"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props}
    />
  )
}

```

```

    /&gt;
  )
}

function SidebarSeparator({
  className,
  ...props
}: React.ComponentProps<Separator>) {
  return (
    <Separator
      data-slot="sidebar-separator"
      data-sidebar="separator"
      className={cn("bg-sidebar-border mx-2 w-auto", className)}
      {...props}
    /&gt;
  )
}

function SidebarContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-content"
      data-sidebar="content"
      className={cn(
        "flex min-h-0 flex-1 flex-col gap-2 overflow-auto group-data-[collapsible=icon]:overflow-hidden",
        className
      )}
      {...props}
    /&gt;
  )
}

function SidebarGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-group"
      data-sidebar="group"
      className={cn("relative flex w-full min-w-0 flex-col p-2", className)}
      {...props}
    /&gt;
  )
}

function SidebarGroupLabel({
  className,
  asChild = false,
  ...props
}: React.ComponentProps<"div"> & { asChild?: boolean }) {
  const Comp = asChild ? Slot : "div"

  return (
    <Comp
      data-slot="sidebar-group-label"
      data-sidebar="group-label"
      className={cn(
        "text-sidebar-foreground/70 ring-sidebar-ring flex h-8 shrink-0 items-center rounded-md px-2 text-xs font-medium",
        "group-data-[collapsible=icon]:-mt-8 group-data-[collapsible=icon]:opacity-0",
        className
      )}
      {...props}
    /&gt;
  )
}

function SidebarGroupAction({
  className,
  asChild = false,
  ...props
}: React.ComponentProps<"button"> & { asChild?: boolean }) {
  const Comp = asChild ? Slot : "button"
  return (

```

```

    <Comp
      data-slot="sidebar-group-action"
      data-sidebar="group-action"
      className={cn(
        "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent hover:text-sidebar-accent-foreground absolute
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 md:after:hidden",
        "group-data-[collapsible=icon]:hidden",
        className
      )}
      {...props}
    />
  )
}

function SidebarGroupContent({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-group-content"
      data-sidebar="group-content"
      className={cn("w-full text-sm", className)}
      {...props}
    />
  )
}

function SidebarMenu({ className, ...props }: React.ComponentProps<"ul">) {
  return (
    <ul
      data-slot="sidebar-menu"
      data-sidebar="menu"
      className={cn("flex w-full min-w-0 flex-col gap-1", className)}
      {...props}
    />
  )
}

function SidebarMenuItem({ className, ...props }: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="sidebar-menu-item"
      data-sidebar="menu-item"
      className={cn("group/menu-item relative", className)}
      {...props}
    />
  )
}

const sidebarMenuButtonVariants = cva(
  "peer/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md p-2 text-left text-sm outline-hidden ring
  {
    variants: {
      variant: {
        default: "hover:bg-sidebar-accent hover:text-sidebar-accent-foreground",
        outline:
          "bg-background shadow-[0_0_0_1px_hsl(var(--sidebar-border))] hover:bg-sidebar-accent hover:text-sidebar-accent
      },
      size: {
        default: "h-8 text-sm",
        sm: "h-7 text-xs",
        lg: "h-12 text-sm group-data-[collapsible=icon]:p-0!",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
)

```



```

)

function SidebarMenuButton({
  asChild = false,
  isActive = false,
  variant = "default",
  size = "default",
  tooltip,
  className,
  ...props
}: React.ComponentProps<"button"> & {
  asChild?: boolean
  isActive?: boolean
  tooltip?: string | React.ComponentProps<typeof TooltipContent>
} & VariantProps<typeof sidebarMenuButtonVariants> & {
  const Comp = asChild ? Slot : "button"
  const { isMobile, state } = useSidebar()

  const button = (
    <Comp
      data-slot="sidebar-menu-button"
      data-sidebar="menu-button"
      data-size={size}
      data-active={isActive}
      className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
      {...props}
    />
  )

  if (!tooltip) {
    return button
  }

  if (typeof tooltip === "string") {
    tooltip = {
      children: tooltip,
    }
  }

  return (
    <Tooltip>
      <TooltipTrigger asChild>{button}</TooltipTrigger>
      <TooltipContent
        side="right"
        align="center"
        hidden={state !== "collapsed" || isMobile}
        {...tooltip}
      />
    </Tooltip>
  )
}

function SidebarMenuAction({
  className,
  asChild = false,
  showOnHover = false,
  ...props
}: React.ComponentProps<"button"> & {
  asChild?: boolean
  showOnHover?: boolean
}) {
  const Comp = asChild ? Slot : "button"

  return (
    <Comp
      data-slot="sidebar-menu-action"
      data-sidebar="menu-action"
      className={cn(
        "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent hover:text-sidebar-accent-foreground peer-ho
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 md:after:hidden",

```

```

        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
        showOnHover &&&
        "peer-data-[active=true]/menu-button:text-sidebar-accent-foreground group-focus-within/menu-item:opacity-100
        className
    )}
    {...props}
  />
)
}

function SidebarMenuBadge({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-menu-badge"
      data-sidebar="menu-badge"
      className={cn(
        "text-sidebar-foreground pointer-events-none absolute right-1 flex h-5 min-w-5 items-center justify-center rounded",
        "peer-hover/menu-button:text-sidebar-accent-foreground peer-data-[active=true]/menu-button:text-sidebar-accent-foreground",
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
        className
      )}
      {...props}
    />
  )
}

function SidebarMenuSkeleton({
  className,
  showIcon = false,
  ...props
}: React.ComponentProps<"div"> & {
  showIcon?: boolean
}) {
  // Random width between 50 to 90%.
  const [width] = React.useState(() => `${Math.floor(Math.random() * 40) + 50}%`)

  return (
    <div
      data-slot="sidebar-menu-skeleton"
      data-sidebar="menu-skeleton"
      className={cn("flex h-8 items-center gap-2 rounded-md px-2", className)}
      {...props}
    >
      {showIcon &&& (
        <Skeleton
          className="size-4 rounded-md"
          data-sidebar="menu-skeleton-icon"
        />
      )}
      <Skeleton
        className="h-4 max-w-(--skeleton-width) flex-1"
        data-sidebar="menu-skeleton-text"
        style={
          {
            "--skeleton-width": width,
          } as React.CSSProperties
        }
      />
    </div>
  )
}

function SidebarMenuSub({ className, ...props }: React.ComponentProps<"ul">) {

```

```

return (
  <ul
    data-slot="sidebar-menu-sub"
    data-sidebar="menu-sub"
    className={cn(
      "border-sidebar-border mx-3.5 flex min-w-0 translate-x-py flex-col gap-1 border-l px-2.5 py-0.5",
      "group-data-[collapsible=icon]:hidden",
      className
    )}
    {...props}
  />
)
}

function SidebarMenuSubItem({
  className,
  ...props
}: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="sidebar-menu-sub-item"
      data-sidebar="menu-sub-item"
      className={cn("group/menu-sub-item relative", className)}
      {...props}
    />
  )
}

function SidebarMenuSubButton({
  asChild = false,
  size = "md",
  isActive = false,
  className,
  ...props
}: React.ComponentProps<"a"> & {
  asChild?: boolean
  size?: "sm" | "md"
  isActive?: boolean
}) {
  const Comp = asChild ? Slot : "a"

  return (
    <Comp
      data-slot="sidebar-menu-sub-button"
      data-sidebar="menu-sub-button"
      data-size={size}
      data-active={isActive}
      className={cn(
        "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent hover:text-sidebar-accent-foreground active:
        data-[active=true]:bg-sidebar-accent data-[active=true]:text-sidebar-accent-foreground",
        size === "sm" & & "text-xs",
        size === "md" & & "text-sm",
        "group-data-[collapsible=icon]:hidden",
        className
      )}
      {...props}
    />
  )
}

export {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupAction,
  SidebarGroupContent,
  SidebarGroupLabel,
  SidebarHeader,
  SidebarInput,
  SidebarInset,

```

```

SidebarMenu,
SidebarMenuAction,
SidebarMenuBadge,
SidebarMenuButton,
SidebarMenuItem,
SidebarMenuSkeleton,
SidebarMenuSub,
SidebarMenuSubButton,
SidebarMenuSubItem,
SidebarProvider,
SidebarRail,
SidebarSeparator,
SidebarTrigger,
useSidebar,
}

```

■ frontend/src/components/ui/skeleton.tsx

```

import { cn } from "@lib/utils"

function Skeleton({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="skeleton"
      className={cn("bg-accent animate-pulse rounded-md", className)}
      {...props}
    />
  )
}

export { Skeleton }

```

■ frontend/src/components/ui/slider.tsx

```

"use client"

import * as React from "react"
import * as SliderPrimitive from "@radix-ui/react-slider"

import { cn } from "@lib/utils"

function Slider({
  className,
  defaultValue,
  value,
  min = 0,
  max = 100,
  ...props
}: React.ComponentProps<typeof SliderPrimitive.Root>) {
  const _values = React.useMemo(
    () => {
      Array.isArray(value)
        ? value
        : Array.isArray(defaultValue)
          ? defaultValue
          : [min, max],
      [value, defaultValue, min, max]
    )

  return (
    <SliderPrimitive.Root
      data-slot="slider"
      defaultValue={defaultValue}
      value={value}
      min={min}
      max={max}
      className={cn(
        "relative flex w-full touch-none items-center select-none data-[disabled]:opacity-50 data-[orientation=vertical]

```

```

        className
      )}
      {...props}
    >
    <SliderPrimitive.Track
      data-slot="slider-track"
      className={cn(
        "bg-muted relative grow overflow-hidden rounded-full data-[orientation=horizontal]:h-1.5 data-[orientation=ho
      )}
    >
    <SliderPrimitive.Range
      data-slot="slider-range"
      className={cn(
        "bg-primary absolute data-[orientation=horizontal]:h-full data-[orientation=vertical]:w-full"
      )}
    />
    </SliderPrimitive.Track>
    {Array.from({ length: _values.length }, (_, index) => {
      <SliderPrimitive.Thumb
        data-slot="slider-thumb"
        key={index}
        className="border-primary bg-background ring-ring/50 block size-4 shrink-0 rounded-full border shadow-sm tran
      />
    })}
    </SliderPrimitive.Root>
  )
}

export { Slider }

```

■ frontend/src/components/ui/sonner.tsx

```

import { useTheme } from "next-themes"
import { Toaster as Sonner, ToasterProps } from "sonner"

const Toaster = ({ ...props }: ToasterProps) => {
  const { theme = "system" } = useTheme()

  return (
    <Sonner
      theme={theme as ToasterProps["theme"]}
      className="toaster group"
      style={
        {
          "--normal-bg": "var(--popover)",
          "--normal-text": "var(--popover-foreground)",
          "--normal-border": "var(--border)",
        } as React.CSSProperties
      }
      {...props}
    />
  )
}

export { Toaster }

```

■ frontend/src/components/ui/switch.tsx

```

"use client"

import * as React from "react"
import * as SwitchPrimitive from "@radix-ui/react-switch"

import { cn } from "@lib/utils"

function Switch({
  className,
  ...props

```

```

}; React.ComponentProps<typeof SwitchPrimitive.Root>); {
  return (
    <SwitchPrimitive.Root
      data-slot="switch"
      className={cn(
        "peer data-[state=checked]:bg-primary data-[state=unchecked]:bg-input focus-visible:border-ring focus-visible:ring"
        className
      )}
      {...props}
    >
    <SwitchPrimitive.Thumb
      data-slot="switch-thumb"
      className={cn(
        "bg-background dark:data-[state=unchecked]:bg-foreground dark:data-[state=checked]:bg-primary-foreground pointer-events-none"
      )}
    />
    </SwitchPrimitive.Root>
  )
}

export { Switch }

```

■ frontend/src/components/ui/table.tsx

```

import * as React from "react"

import { cn } from "@lib/utls"

function Table({ className, ...props }: React.ComponentProps<"table">): {
  return (
    <div
      data-slot="table-container"
      className="relative w-full overflow-x-auto"
    >
    <table
      data-slot="table"
      className={cn("w-full caption-bottom text-sm", className)}
      {...props}
    />
    </div>
  )
}

function TableHeader({ className, ...props }: React.ComponentProps<"thead">): {
  return (
    <thead
      data-slot="table-header"
      className={cn("[&_tr]:border-b", className)}
      {...props}
    />
  )
}

function TableBody({ className, ...props }: React.ComponentProps<"tbody">): {
  return (
    <tbody
      data-slot="table-body"
      className={cn("[&_tr:last-child]:border-0", className)}
      {...props}
    />
  )
}

function TableFooter({ className, ...props }: React.ComponentProps<"tfoot">): {
  return (
    <tfoot
      data-slot="table-footer"
      className={cn(
        "bg-muted/50 border-t font-medium [&_tr]:last:border-b-0",
        className
      )}
    >
    </tfoot>
  )
}

```

```

    })
    {...props}
  /&gt;
)
}

function TableRow({ className, ...props }: React.ComponentProps<"tr"&gt;) {
  return (
    &lt;tr
      data-slot="table-row"
      className={cn(
        "hover:bg-muted/50 data-[state=selected]:bg-muted border-b transition-colors",
        className
      )}
      {...props}
    /&gt;
  )
}

function TableHead({ className, ...props }: React.ComponentProps<"th"&gt;) {
  return (
    &lt;th
      data-slot="table-head"
      className={cn(
        "text-foreground h-10 px-2 text-left align-middle font-medium whitespace-nowrap [&:has([role=checkbox]):pr-0.5]",
        className
      )}
      {...props}
    /&gt;
  )
}

function TableCell({ className, ...props }: React.ComponentProps<"td"&gt;) {
  return (
    &lt;td
      data-slot="table-cell"
      className={cn(
        "p-2 align-middle whitespace-nowrap [&:has([role=checkbox]):pr-0.5] [&:has([role=checkbox]):translate-y-[-50%]]",
        className
      )}
      {...props}
    /&gt;
  )
}

function TableCaption({
  className,
  ...props
}: React.ComponentProps<"caption"&gt;) {
  return (
    &lt;caption
      data-slot="table-caption"
      className={cn("text-muted-foreground mt-4 text-sm", className)}
      {...props}
    /&gt;
  )
}

export {
  Table,
  TableHeader,
  TableBody,
  TableFooter,
  TableHead,
  TableRow,
  TableCell,
  TableCaption,
}

```

■ frontend/src/components/ui/tabs.tsx

```
"use client"

import * as React from "react"
import * as TabsPrimitive from "@radix-ui/react-tabs"

import { cn } from "@/lib/utils"

function Tabs({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.Root>) {
  return (
    <TabsPrimitive.Root
      data-slot="tabs"
      className={cn("flex flex-col gap-2", className)}
      {...props}
    />
  )
}

function TabsList({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.List>) {
  return (
    <TabsPrimitive.List
      data-slot="tabs-list"
      className={cn(
        "bg-muted text-muted-foreground inline-flex h-9 w-fit items-center justify-center rounded-lg p-[3px]",
        className
      )}
      {...props}
    />
  )
}

function TabsTrigger({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.Trigger>) {
  return (
    <TabsPrimitive.Trigger
      data-slot="tabs-trigger"
      className={cn(
        "data-[state=active]:bg-background dark:data-[state=active]:text-foreground focus-visible:border-ring focus-vis",
        className
      )}
      {...props}
    />
  )
}

function TabsContent({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.Content>) {
  return (
    <TabsPrimitive.Content
      data-slot="tabs-content"
      className={cn("flex-1 outline-none", className)}
      {...props}
    />
  )
}

export { Tabs, TabsList, TabsTrigger, TabsContent }
```


■ frontend/src/components/ui/textarea.tsx

```
import * as React from "react"

import { cn } from "@lib/utils"

function Textarea({ className, ...props }: React.ComponentProps<"textarea">) {
  return (
    <textarea
      data-slot="textarea"
      className={cn(
        "border-input placeholder:text-muted-foreground focus-visible:border-ring focus-visible:ring-ring/50 aria-inval
        className
      )}
      {...props}
    />
  )
}

export { Textarea }
```

■ frontend/src/components/ui/toggle-group.tsx

```
"use client"

import * as React from "react"
import * as ToggleGroupPrimitive from "@radix-ui/react-toggle-group"
import { type VariantProps } from "class-variance-authority"

import { cn } from "@lib/utils"
import { toggleVariants } from "@components/ui/toggle"

const ToggleGroupContext = React.createContext<
  VariantProps<typeof toggleVariants>
>({
  size: "default",
  variant: "default",
})

function ToggleGroup({
  className,
  variant,
  size,
  children,
  ...props
}: React.ComponentProps<typeof ToggleGroupPrimitive.Root> &
  VariantProps<typeof toggleVariants>
) {
  return (
    <ToggleGroupPrimitive.Root
      data-slot="toggle-group"
      data-variant={variant}
      data-size={size}
      className={cn(
        "group/toggle-group flex w-fit items-center rounded-md data-[variant=outline]:shadow-xs",
        className
      )}
      {...props}
    >
      <ToggleGroupContext.Provider value={{ variant, size }}>
        {children}
      </ToggleGroupContext.Provider>
    </ToggleGroupPrimitive.Root>
  )
}

function ToggleGroupItem({
  className,
  children,
  variant,
  size,
  ...props
}: React.ComponentProps<typeof ToggleGroupPrimitive.Item> &
  VariantProps<typeof toggleVariants>
) {
  return (
    <ToggleGroupPrimitive.Item
      data-slot="toggle-group-item"
      data-variant={variant}
      data-size={size}
      className={cn(
        "toggle-group-item",
        toggleVariants({ variant, size }),
        className
      )}
      {...props}
    />
  )
}
```

```

    ...props
  }: React.ComponentProps<ToggleGroupPrimitive.Item> &
  VariantProps<ToggleVariants>) {
    const context = React.useContext(ToggleGroupContext)

    return (
      <ToggleGroupPrimitive.Item
        data-slot="toggle-group-item"
        data-variant={context.variant || variant}
        data-size={context.size || size}
        className={cn(
          toggleVariants({
            variant: context.variant || variant,
            size: context.size || size,
          }),
          "min-w-0 flex-1 shrink-0 rounded-none shadow-none first:rounded-l-md last:rounded-r-md focus:z-10 focus-visible",
          className
        )}
        {...props}
      >
        {children}
      </ToggleGroupPrimitive.Item>
    )
  }
}

export { ToggleGroup, ToggleGroupItem }

```

■ frontend/src/components/ui/toggle.tsx

```

import * as React from "react"
import * as TogglePrimitive from "@radix-ui/react-toggle"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@lib/Utils"

const toggleVariants = cva(
  "inline-flex items-center justify-center gap-2 rounded-md text-sm font-medium hover:bg-muted hover:text-muted-foreground",
  {
    variants: {
      variant: {
        default: "bg-transparent",
        outline:
          "border border-input bg-transparent shadow-xs hover:bg-accent hover:text-accent-foreground",
      },
      size: {
        default: "h-9 px-2 min-w-9",
        sm: "h-8 px-1.5 min-w-8",
        lg: "h-10 px-2.5 min-w-10",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
)

function Toggle({
  className,
  variant,
  size,
  ...props
}: React.ComponentProps<TogglePrimitive.Root> &
  VariantProps<ToggleVariants>) {
  return (
    <TogglePrimitive.Root
      data-slot="toggle"
      className={cn(toggleVariants({ variant, size, className }))}
      {...props}
    />
  )
}

```

```

    )
  }

export { Toggle, toggleVariants }

```

■ frontend/src/components/ui/tooltip.tsx

```

import * as React from "react"
import * as TooltipPrimitive from "@radix-ui/react-tooltip"

import { cn } from "@/lib/utils"

function TooltipProvider({
  delayDuration = 0,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Provider>) {
  return (
    <TooltipPrimitive.Provider
      data-slot="tooltip-provider"
      delayDuration={delayDuration}
      {...props}
    />
  )
}

function Tooltip({
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Root>) {
  return (
    <TooltipProvider>
      <TooltipPrimitive.Root data-slot="tooltip" {...props} />
    </TooltipProvider>
  )
}

function TooltipTrigger({
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Trigger>) {
  return <TooltipPrimitive.Trigger data-slot="tooltip-trigger" {...props} />
}

function TooltipContent({
  className,
  sideOffset = 0,
  children,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Content>) {
  return (
    <TooltipPrimitive.Portal>
      <TooltipPrimitive.Content
        data-slot="tooltip-content"
        sideOffset={sideOffset}
        className={cn(
          "bg-primary text-primary-foreground animate-in fade-in-0 zoom-in-95 data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=closed]:zoom-out-95",
          className
        )}
        {...props}
      >
        {children}
      </TooltipPrimitive.Content>
    </TooltipPrimitive.Portal>
  )
}

export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider }

```

■ frontend/src/components/VerifiedBadge.tsx

```
import * as React from 'react';

export type VerifiedBadgeProps = {
  /**
   * Pass `true` for admin users (or any user you want to flag as verified).
   */
  show?: boolean;
  /**
   * Pixel size of the icon.
   */
  size?: number;
  className?: string;
  title?: string;
};

/**
 * Small app-logo badge meant to sit next to a user's display name.
 * Uses the public `logo.svg` so it works in dev/prod without bundler imports.
 */
export function VerifiedBadge({ show, size = 16, className, title = 'Verified' }: VerifiedBadgeProps) {
  if (!show) return null;

  // Starburst background container size (gives the logo some breathing room)
  const badgeSize = Math.max(size + 10, 18);

  // A simple starburst polygon (CSS clip-path). Works in modern browsers.
  // If clip-path isn't supported, it gracefully falls back to a square.
  const starburstClipPath =
    'polygon(50% 0%, 58% 10%, 71% 4%, 74% 18%, 88% 12%, 82% 26%, 96% 29%, 84% 38%, 100% 50%, 84% 62%, 96% 71%, 82%';

  // Theme-aware colors:
  // - light mode: black background
  // - dark mode: #cdd3ff background
  //
  // NOTE: We intentionally use explicit colors here rather than CSS variables,
  // because some pages may not have the theme variables applied.
  const baseClass =
    'inline-flex items-center justify-center bg-black dark:bg-[#e3ffd7] ring-4 ring-blue-500';

  const combinedClassName = [baseClass, className].filter(Boolean).join(' ');

  return (
    <span
      title={title}
      className={combinedClassName}
      style={{
        width: badgeSize,
        height: badgeSize,
        padding: 3,
        lineHeight: 0,
        clipPath: starburstClipPath,
        WebkitClipPath: starburstClipPath,
      }}
    >
      
    </span>
  );
}
```

■ frontend/src/contexts/AuthContext.tsx

```

import React, { createContext, useContext, useState, useEffect, ReactNode } from 'react';
import { authApi, User, ApiError, isUnauthenticatedStatus } from '@lib/api';

interface AuthContextType {
  user: User | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  logout: () => Promise<void>;
  checkAuth: () => Promise<void>;
  /**
   * Allows pages/components (like Edit Profile) to immediately reflect user changes
   * (e.g., avatar) across the whole app.
   */
  setUser: (user: User | null) => void;
  updateUser: (patch: Partial<User>) => void;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

const STORAGE_KEY = 'ae.auth.user';

function safeReadStoredUser(): User | null {
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (!raw) return null;
    const parsed = JSON.parse(raw) as unknown;
    if (parsed && typeof parsed === 'object' && 'id' in (parsed as any)) {
      return normalizeUser(parsed as User);
    }
    return null;
  } catch {
    return null;
  }
}

function normalizeUser(input: User): User {
  const anyUser = input as unknown as { profileImageUrl?: string; avatar?: string };
  const avatar = anyUser.profileImageUrl || anyUser.avatar;
  return {
    ...input,
    avatar: avatar || undefined,
  };
}

export const AuthProvider: React.FC<{ children: ReactNode }> = ({ children }) => {
  const [user, setUserState] = useState<User | null>(() => safeReadStoredUser());
  const [isLoading, setIsLoading] = useState(true);

  const persistUser = (next: User | null) => {
    try {
      if (next) localStorage.setItem(STORAGE_KEY, JSON.stringify(next));
      else localStorage.removeItem(STORAGE_KEY);
    } catch {
      // Ignore storage failures (private mode, quota, etc.).
    }
  };

  const setUser = (next: User | null) => {
    const normalized = next ? normalizeUser(next) : null;
    setUserState(normalized);
    persistUser(normalized);
  };

  const updateUser = (patch: Partial<User>) => {
    setUserState((prev: User | null) => {
      if (!prev) return prev;
      const next = normalizeUser({ ...prev, ...patch } as User);
      persistUser(next);
      return next;
    });
  };
};

```

```

const checkAuth = async () => {
  try {
    const response = await authApi.status();
    if (response.authenticated && response.user) {
      setUser(response.user as User);
    } else {
      setUser(null);
    }
  } catch (error) {
    const status = error instanceof ApiError ? error.status : undefined;
    if (!isUnauthenticatedStatus(status)) {
      console.error('Auth check failed:', error);
    }
    setUser(null);
  } finally {
    setIsLoading(false);
  }
};

useEffect(() => {
  checkAuth();
}, []);

useEffect(() => {
  // Keep auth state fresh across OAuth redirects, tab switches, and long-lived sessions.
  const onFocus = () => {
    // Fire-and-forget; errors are handled inside checkAuth.
    checkAuth();
  };

  window.addEventListener('focus', onFocus);

  // Periodic refresh helps when cookies/session change server-side while SPA stays open.
  const id = window.setInterval(() => {
    checkAuth();
  }, 60_000);

  return () => {
    window.removeEventListener('focus', onFocus);
    window.clearInterval(id);
  };
}, []);

const logout = async () => {
  // Optimistically update UI, but also await the backend logout so we don't leave
  // an unhandled rejection that triggers the global runtime-error overlay.
  setUser(null);
  try {
    await authApi.logout();
  } catch (error) {
    // If the session is already gone, backend may return 401/403; treat as success.
    const status = error instanceof ApiError ? error.status : undefined;
    if (!isUnauthenticatedStatus(status)) {
      console.error('Logout failed:', error);
    }
  }
};

return (
  <AuthContext.Provider
    value={{
      user,
      isAuthenticated: !!user,
      isLoading,
      logout,
      checkAuth,
      setUser,
      updateUser,
    }}
  >
    {children}
  </AuthContext.Provider>
);

```

```

    </AuthContext.Provider>;
  );
};

export const useAuthContext = () => {
  const context = useContext(AuthContext);
  if (context === undefined) {
    throw new Error('useAuthContext must be used within an AuthProvider');
  }
  return context;
};

```

■ frontend/src/context/ThemeContext.tsx

```

import { createContext, useContext, useEffect, useState, ReactNode } from 'react';

type Theme = 'light' | 'dark';

interface ThemeContextType {
  theme: Theme;
  toggleTheme: () => void;
  setTheme: (theme: Theme) => void;
}

const ThemeContext = createContext<ThemeContextType | undefined>(undefined);

export function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setThemeState] = useState<Theme>(() => {
    // Default to system preference.
    if (window.matchMedia('(prefers-color-scheme: dark)').matches) {
      return 'dark';
    }
    return 'light';
  });

  useEffect(() => {
    const root = window.document.documentElement;
    root.classList.remove('light', 'dark');
    root.classList.add(theme);
  }, [theme]);

  const toggleTheme = () => {
    setThemeState((prev) => (prev === 'light' ? 'dark' : 'light'));
  };

  const setTheme = (newTheme: Theme) => {
    setThemeState(newTheme);
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const context = useContext(ThemeContext);
  if (context === undefined) {
    throw new Error('useTheme must be used within a ThemeProvider');
  }
  return context;
}

```

■ frontend/src/hooks/use-auth.ts

■ frontend/src/hooks/use-mobile.ts

```
import * as React from "react";

const MOBILE_BREAKPOINT = 768;

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState<boolean | undefined>(<
    undefined,
  >);

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`);
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    };
    mql.addEventListener("change", onChange);
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    return () => mql.removeEventListener("change", onChange);
  }, []);

  return !!isMobile;
}
```

■ frontend/src/hooks/usePostRealtime.ts

```
import { useEffect, useMemo, useRef } from 'react';
import { PostRealtimeClient, type RealtimePostStats } from '@lib/postRealtime';

/**
 * Shared singleton client per page session.
 * Reconnects automatically.
 */
let singleton: PostRealtimeClient | null = null;

function getClient(): PostRealtimeClient {
  if (!singleton) singleton = new PostRealtimeClient('/ws/posts');
  return singleton;
}

export function usePostRealtime(postIds: number[], onStats: (stats: RealtimePostStats) => void) {
  const idsKey = useMemo(() => {
    const uniq = Array.from(new Set(postIds.filter((n) => Number.isFinite(n))));
    uniq.sort((a, b) => a - b);
    return uniq.join(',');
  }, [postIds]);

  const onStatsRef = useRef(onStats);
  onStatsRef.current = onStats;

  useEffect(() => {
    const ids = idsKey ? idsKey.split(',').map((s) => Number(s)).filter((n) => Number.isFinite(n)) : [];
    if (ids.length === 0) return;

    const client = getClient();
    let cancelled = false;
    let subs: { unsubscribe: () => void }[] = [];

    (async () => {
      try {
        await client.connect();
        if (cancelled) return;

        subs = ids.map((id) => client.subscribe(id, (stats: RealtimePostStats) => onStatsRef.current(stats)));
      } catch {
        // If WS fails, silently ignore (polling could be added as fallback).
      }
    })();
  });
}
```



```

    return () => {
      cancelled = true;
      subs.forEach((s) => s.unsubscribe());
    };
  }, [idsKey]);
}

```

■ frontend/src/index.css

```

/* DO NOT CHANGE */
@import "tailwindcss";
@import "tw-animate-css";
@custom-variant dark (&is(.dark *));

/* DO NOT CHANGE */
@theme inline {
  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --color-card: var(--card);
  --color-card-foreground: var(--card-foreground);
  --color-popover: var(--popover);
  --color-popover-foreground: var(--popover-foreground);
  --color-primary: var(--primary);
  --color-primary-foreground: var(--primary-foreground);
  --color-secondary: var(--secondary);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-muted: var(--muted);
  --color-muted-foreground: var(--muted-foreground);
  --color-accent: var(--accent);
  --color-accent-foreground: var(--accent-foreground);
  --color-destructive: var(--destructive);
  --color-border: var(--border);
  --color-input: var(--input);
  --color-ring: var(--ring);
  --color-chart-1: var(--chart-1);
  --color-chart-2: var(--chart-2);
  --color-chart-3: var(--chart-3);
  --color-chart-4: var(--chart-4);
  --color-chart-5: var(--chart-5);
  --color-sidebar: var(--sidebar);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
  --color-sidebar-accent: var(--sidebar-accent);
  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
  --color-sidebar-border: var(--sidebar-border);
  --color-sidebar-ring: var(--sidebar-ring);
}

/* CHANGE VALUES FOR THEME - Gradient Mesh Theme */
:root {
  --radius: 1rem;
  --background: oklch(0.98 0.01 280);
  --foreground: oklch(0.12 0.02 270);
  --card: oklch(1 0 0 / 0.9);
  --card-foreground: oklch(0.12 0.02 270);
  --popover: oklch(1 0 0 / 0.98);
  --popover-foreground: oklch(0.12 0.02 270);
  --primary: oklch(0.6 0.25 280);
  --primary-foreground: oklch(0.99 0 0);
  --secondary: oklch(0.68 0.18 220);
  --secondary-foreground: oklch(0.99 0 0);
  --muted: oklch(0.92 0.02 280);
  --muted-foreground: oklch(0.42 0.02 270);
  --accent: oklch(0.72 0.22 320);
  --accent-foreground: oklch(0.99 0 0);
}

```

```

--destructive: oklch(0.577 0.245 27.325);
--border: oklch(0.85 0.03 280 / 0.4);
--input: oklch(0.9 0.02 280 / 0.6);
--ring: oklch(0.6 0.25 280 / 0.5);
--chart-1: oklch(0.68 0.24 320);
--chart-2: oklch(0.7 0.22 280);
--chart-3: oklch(0.72 0.2 240);
--chart-4: oklch(0.75 0.18 200);
--chart-5: oklch(0.78 0.16 160);
--sidebar: oklch(0.99 0 0 / 0.95);
--sidebar-foreground: oklch(0.12 0.02 270);
--sidebar-primary: oklch(0.6 0.25 280);
--sidebar-primary-foreground: oklch(0.99 0 0);
--sidebar-accent: oklch(0.92 0.02 280);
--sidebar-accent-foreground: oklch(0.12 0.02 270);
--sidebar-border: oklch(0.85 0.03 280 / 0.4);
--sidebar-ring: oklch(0.6 0.25 280 / 0.5);
}

```

```

.dark {
  --background: oklch(0.13 0 0);
  --foreground: oklch(0.98 0 0);
  --card: oklch(0.18 0 0 / 0.9);
  --card-foreground: oklch(0.98 0 0);
  --popover: oklch(0.18 0 0 / 0.98);
  --popover-foreground: oklch(0.98 0 0);
  --primary: oklch(0.88 0 0);
  --primary-foreground: oklch(0.15 0 0);
  --secondary: oklch(0.25 0 0);
  --secondary-foreground: oklch(0.98 0 0);
  --muted: oklch(0.22 0 0);
  --muted-foreground: oklch(0.72 0 0);
  --accent: oklch(0.28 0 0);
  --accent-foreground: oklch(0.98 0 0);
  --destructive: oklch(0.704 0.191 22.216);
  --border: oklch(1 0 0 / 15%);
  --input: oklch(1 0 0 / 20%);
  --ring: oklch(0.6 0 0);
  --chart-1: oklch(0.488 0.243 264.376);
  --chart-2: oklch(0.696 0.17 162.48);
  --chart-3: oklch(0.769 0.188 70.08);
  --chart-4: oklch(0.627 0.265 303.9);
  --chart-5: oklch(0.645 0.246 16.439);
  --sidebar: oklch(0.18 0 0 / 0.95);
  --sidebar-foreground: oklch(0.98 0 0);
  --sidebar-primary: oklch(0.488 0.243 264.376);
  --sidebar-primary-foreground: oklch(0.98 0 0);
  --sidebar-accent: oklch(0.25 0 0);
  --sidebar-accent-foreground: oklch(0.98 0 0);
  --sidebar-border: oklch(1 0 0 / 15%);
  --sidebar-ring: oklch(0.6 0 0);
}

```

```

@layer base {
  * {
    @apply border-border outline-ring/50;
  }
  body {
    @apply bg-background text-foreground;
    position: relative;
  }
  body::before {
    content: '';
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    z-index: -2;
    background:
      radial-gradient(ellipse 80% 50% at 20% 40%, oklch(0.85 0.15 320 / 0.3), transparent),

```

```

        radial-gradient(ellipse 60% 50% at 80% 60%, oklch(0.82 0.18 280 / 0.25), transparent),
        radial-gradient(ellipse 70% 60% at 50% 80%, oklch(0.8 0.2 240 / 0.2), transparent),
        radial-gradient(ellipse 50% 40% at 30% 20%, oklch(0.88 0.12 200 / 0.15), transparent);
background-attachment: fixed;
}
.dark body::before {
    background:
        radial-gradient(ellipse 80% 50% at 20% 40%, oklch(0.35 0.15 320 / 0.2), transparent),
        radial-gradient(ellipse 60% 50% at 80% 60%, oklch(0.32 0.18 280 / 0.15), transparent),
        radial-gradient(ellipse 70% 60% at 50% 80%, oklch(0.3 0.2 240 / 0.12), transparent),
        radial-gradient(ellipse 50% 40% at 30% 20%, oklch(0.38 0.12 200 / 0.1), transparent);
}
body::after {
    content: '';
    position: fixed;
    top: -10%;
    left: -10%;
    width: 120%;
    height: 120%;
    z-index: -1;
    pointer-events: none;
    background:
        radial-gradient(circle 800px at 20% 30%, rgba(59, 130, 246, 0.55), transparent),
        radial-gradient(circle 700px at 80% 70%, rgba(139, 92, 246, 0.45), transparent),
        radial-gradient(circle 750px at 50% 50%, rgba(37, 99, 235, 0.52), transparent);
    animation: futuristicFloat 10s ease-in-out infinite;
}
.dark body::after {
    background:
        radial-gradient(circle 800px at 20% 30%, rgba(59, 130, 246, 0.55), transparent),
        radial-gradient(circle 700px at 80% 70%, rgba(139, 92, 246, 0.45), transparent),
        radial-gradient(circle 750px at 50% 50%, rgba(37, 99, 235, 0.52), transparent);
}
button:not([disabled]),
[role="button"]:not([disabled]) {
    cursor: pointer;
}

/* Override button default colors - Light mode: theme colors, Dark mode: theme colors */
button:not([disabled]),
[role="button"]:not([disabled]),
a[data-slot="button"] {
    background: hsl(var(--muted)) !important;
    color: hsl(var(--foreground)) !important;
    border-color: hsl(var(--border)) !important;
}

.dark button:not([disabled]),
.dark [role="button"]:not([disabled]),
.dark a[data-slot="button"] {
    background: hsl(var(--muted)) !important;
    color: hsl(var(--foreground)) !important;
    border-color: hsl(var(--border)) !important;
}

/* Button hover effects for light mode */
button:not([disabled]):hover,
[role="button"]:not([disabled]):hover,
a[role="button"]:not([disabled]):hover,
a[data-slot="button"]:hover,
button:not([disabled]) a:hover,
nav button:not([disabled]):hover,
nav a[role="button"]:not([disabled]):hover {
    background: linear-gradient(135deg, rgb(156, 163, 175) 0%, rgb(107, 114, 128) 50%, rgb(75, 85, 99) 100%) !important;
    border-color: rgb(75, 85, 99) !important;
    transition: all 0.3s ease !important;
}

button:not([disabled]):hover *,
[role="button"]:not([disabled]):hover *,
a[role="button"]:not([disabled]):hover *,

```

```

a[data-slot="button"]:hover *,
button:not([disabled]) a:hover *,
nav button:not([disabled]):hover *,
nav a[role="button"]:not([disabled]):hover * {
  color: white !important;
}

/* Button hover effects for dark mode */
.dark button:not([disabled]):hover,
.dark [role="button"]:not([disabled]):hover,
.dark a[role="button"]:not([disabled]):hover,
.dark a[data-slot="button"]:hover,
.dark button:not([disabled]) a:hover,
.dark nav button:not([disabled]):hover,
.dark nav a[role="button"]:not([disabled]):hover {
  background: linear-gradient(135deg, rgb(34, 211, 238) 0%, rgb(6, 182, 212) 50%, rgb(8, 145, 178) 100%) !important;
  border-color: rgb(8, 145, 178) !important;
  box-shadow: 0 0 25px rgba(34, 211, 238, 0.6) !important;
  transition: all 0.3s ease !important;
}

.dark button:not([disabled]):hover *,
.dark [role="button"]:not([disabled]):hover *,
.dark a[role="button"]:not([disabled]):hover *,
.dark a[data-slot="button"]:hover *,
.dark button:not([disabled]) a:hover *,
.dark nav button:not([disabled]):hover *,
.dark nav a[role="button"]:not([disabled]):hover * {
  color: white !important;
}
}

.gradient-mesh-card {
  background: linear-gradient(135deg,
    oklch(1 0 0 / 0.95) 0%,
    oklch(0.98 0.02 280 / 0.9) 100%);
  backdrop-filter: blur(10px);
  border: 1px solid oklch(0.9 0.05 280 / 0.3);
}

.dark .gradient-mesh-card {
  background: linear-gradient(135deg,
    oklch(0.20 0 0 / 0.95) 0%,
    oklch(0.18 0.02 280 / 0.9) 100%);
  border: 1px solid oklch(0.3 0.05 280 / 0.3);
}

.gradient-button {
  transition: all 0.3s ease;
}

/* Search button styling - Light mode: grey, Dark mode: cyan/blue */
.search-button {
  background: linear-gradient(135deg, rgb(75, 85, 99) 0%, rgb(55, 65, 81) 50%, rgb(31, 41, 55) 100%) !important;
  color: white !important;
}

.search-button:hover {
  box-shadow: 0 0 25px rgba(75, 85, 99, 0.6) !important;
}

.dark .search-button {
  background: linear-gradient(135deg, rgb(34, 211, 238) 0%, rgb(6, 182, 212) 50%, rgb(8, 145, 178) 100%) !important;
  color: white !important;
}

.dark .search-button:hover {
  box-shadow: 0 0 25px rgba(34, 211, 238, 0.6) !important;
}

/* AlliedEdge brand styling - Allied = cyan/blue, Edge = light grey */

```

```

.brand-name .allied {
  color: rgb(34, 211, 238);
}

.brand-name .edge {
  color: rgb(156, 163, 175);
}

/* Dropdown menu item hover - Light mode: grey, Dark mode: cyan */
[data-slot="dropdown-menu-item"]:hover,
[data-slot="dropdown-menu-item"]:focus,
[data-slot="dropdown-menu-item"][data-state="open"] {
  background: linear-gradient(135deg, rgb(156, 163, 175) 0%, rgb(107, 114, 128) 50%, rgb(75, 85, 99) 100%) !important;
  color: white !important;
}

[data-slot="dropdown-menu-item"]:hover *,
[data-slot="dropdown-menu-item"]:focus *,
[data-slot="dropdown-menu-item"][data-state="open"] * {
  color: white !important;
}

.dark [data-slot="dropdown-menu-item"]:hover,
.dark [data-slot="dropdown-menu-item"]:focus,
.dark [data-slot="dropdown-menu-item"][data-state="open"] {
  background: linear-gradient(135deg, rgb(34, 211, 238) 0%, rgb(6, 182, 212) 50%, rgb(8, 145, 178) 100%) !important;
  color: white !important;
}

.dark [data-slot="dropdown-menu-item"]:hover *,
.dark [data-slot="dropdown-menu-item"]:focus *,
.dark [data-slot="dropdown-menu-item"][data-state="open"] * {
  color: white !important;
}

/* Admin card button hover effects */
.admin-card-button:hover {
  background: linear-gradient(135deg, rgb(156, 163, 175) 0%, rgb(107, 114, 128) 50%, rgb(75, 85, 99) 100%) !important;
  color: white !important;
}

.dark .admin-card-button:hover {
  background: linear-gradient(135deg, rgb(34, 211, 238) 0%, rgb(6, 182, 212) 50%, rgb(8, 145, 178) 100%) !important;
  color: white !important;
}

/* Futuristic background animation */
@keyframes futuristicFloat {
  0% {
    transform: translate(0, 0) rotate(0deg) scale(1);
    opacity: 1;
  }
  25% {
    transform: translate(8%, -8%) rotate(5deg) scale(1.1);
    opacity: 0.8;
  }
  50% {
    transform: translate(-8%, 8%) rotate(-5deg) scale(1.15);
    opacity: 1;
  }
  75% {
    transform: translate(5%, -5%) rotate(3deg) scale(1.08);
    opacity: 0.9;
  }
  100% {
    transform: translate(0, 0) rotate(0deg) scale(1);
    opacity: 1;
  }
}

/* Feed page specific animation */

```

```

.feed-animated-bg {
  position: relative;
}

/* Keep the legacy pseudo-element version for any existing pages */
.feed-animated-bg::before {
  content: '';
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  /* Put it above body::before (-2) and behind app content (0+) */
  z-index: -1;
  pointer-events: none;
  background:
    radial-gradient(circle 900px at 25% 20%, rgba(59, 130, 246, 0.35), transparent),
    radial-gradient(circle 800px at 75% 80%, rgba(100, 116, 139, 0.3), transparent),
    radial-gradient(circle 700px at 50% 50%, rgba(37, 99, 235, 0.25), transparent);
  animation: feedFloat 12s ease-in-out infinite;
}

.dark .feed-animated-bg::before {
  background:
    radial-gradient(circle 900px at 25% 20%, rgba(59, 130, 246, 0.2), transparent),
    radial-gradient(circle 800px at 75% 80%, rgba(71, 85, 105, 0.18), transparent),
    radial-gradient(circle 700px at 50% 50%, rgba(37, 99, 235, 0.15), transparent);
}

@keyframes feedFloat {
  0% {
    transform: translate(0, 0) scale(1);
    opacity: 1;
  }
  33% {
    transform: translate(-5%, 5%) scale(1.08);
    opacity: 0.85;
  }
  66% {
    transform: translate(5%, -3%) scale(1.05);
    opacity: 0.9;
  }
  100% {
    transform: translate(0, 0) scale(1);
    opacity: 1;
  }
}

/* --- Profile label + link gradients (requested palette) --- */
.gradient-label {
  background: linear-gradient(135deg, rgb(75, 85, 99) 0%, rgb(55, 65, 81) 50%, rgb(31, 41, 55) 100%);
  color: white !important;
  border: 1px solid rgba(31, 41, 55, 0.35);
}

.dark .gradient-label {
  background: linear-gradient(135deg, rgb(34, 211, 238) 0%, rgb(6, 182, 212) 50%, rgb(8, 145, 178) 100%);
  color: white !important;
  border: 1px solid rgba(34, 211, 238, 0.35);
}

.gradient-link-text {
  background: linear-gradient(135deg, rgb(75, 85, 99) 0%, rgb(55, 65, 81) 50%, rgb(31, 41, 55) 100%);
  -webkit-background-clip: text;
  background-clip: text;
  -webkit-text-fill-color: transparent;
}

.dark .gradient-link-text {
  background: linear-gradient(135deg, rgb(34, 211, 238) 0%, rgb(6, 182, 212) 50%, rgb(8, 145, 178) 100%);
  -webkit-background-clip: text;

```

```

background-clip: text;
-webkit-text-fill-color: transparent;
}

@layer utilities {
  /* Hide scrollbar visuals but keep scrolling enabled */
  .no-scrollbar {
    -ms-overflow-style: none; /* IE and legacy Edge */
    scrollbar-width: none; /* Firefox */
  }
  .no-scrollbar::-webkit-scrollbar {
    display: none; /* Chrome, Safari, new Edge */
  }

  /* Hide scrollbars but keep scrolling enabled */
  .hide-scrollbar {
    -ms-overflow-style: none; /* IE and Edge */
    scrollbar-width: none; /* Firefox */
  }
  .hide-scrollbar::-webkit-scrollbar {
    display: none; /* Chrome, Safari, Opera */
  }

  /* Hard-wrap long tokens (URLs, long error strings) to prevent horizontal overflow */
  .break-anywhere {
    overflow-wrap: anywhere;
    word-break: break-word;
  }
}

/*
  Page-inserted background layer that matches the global Login animation (body::after).
  Use with <AnimatedBackground />.
*/
.animated-bg-layer--login {
  position: fixed;
  top: -10%;
  left: -10%;
  width: 120%;
  height: 120%;
  /* Above the global body layers (negative z-index) and below page content (z-10 wrapper) */
  z-index: 1;
  pointer-events: none;
  /* In light mode, blend-mode can effectively wash out into the very bright background. */
  mix-blend-mode: normal;
  opacity: 0.75;
  background:
    radial-gradient(circle 800px at 20% 30%, rgba(59, 130, 246, 0.95), transparent 60%),
    radial-gradient(circle 700px at 80% 70%, rgba(139, 92, 246, 0.75), transparent 60%),
    radial-gradient(circle 750px at 50% 50%, rgba(37, 99, 235, 0.90), transparent 62%);
  animation: futuristicFloat 10s ease-in-out infinite;
}

.dark .animated-bg-layer--login {
  opacity: 1;
  mix-blend-mode: screen;
  background:
    radial-gradient(circle 800px at 20% 30%, rgba(59, 130, 246, 0.55), transparent 60%),
    radial-gradient(circle 700px at 80% 70%, rgba(139, 92, 246, 0.45), transparent 60%),
    radial-gradient(circle 750px at 50% 50%, rgba(37, 99, 235, 0.52), transparent 62%);
}

```

■ frontend/src/instrumentation.tsx

```

import { Button } from "@components/ui/button";
import {
  Collapsible,
  CollapsibleContent,
  CollapsibleTrigger,

```

```

} from "@components/ui/collapsible";
import {
  DialogContent,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@components/ui/dialog";
import { Dialog } from "@radix-ui/react-dialog";
import { ChevronDown } from "lucide-react";
import React, { useEffect, useState } from "react";

type SyncError = {
  error: string;
  stack: string;
  filename: string;
  lineno: number;
  colno: number;
};

type AsyncError = {
  error: string;
  stack: string;
};

type GenericError = SyncError | AsyncError;

function ErrorDialog({
  error,
  setError,
}: {
  error: GenericError;
  setError: (error: GenericError | null) => void;
}) {
  return (
    <Dialog
      defaultOpen={true}
      onOpenChange={() => {
        setError(null);
      }}
    >
      <DialogContent className="bg-red-700 text-white max-w-4xl">
        <DialogHeader>
          <DialogTitle>Runtime Error</DialogTitle>
        </DialogHeader>
        A runtime error occurred.
        <div className="mt-4">
          <Collapsible>
            <CollapsibleTrigger>
              <div className="flex items-center font-bold cursor-pointer">
                See error details <ChevronDown />
              </div>
            </CollapsibleTrigger>
            <CollapsibleContent className="max-w-[460px]">
              <div className="mt-2 p-3 bg-neutral-800 rounded text-white text-sm overflow-x-auto max-h-60 max-w-full">
                <pre className="whitespace-pre">{error.stack}</pre>
              </div>
            </CollapsibleContent>
          </Collapsible>
        </div>
        <DialogFooter>
          <Button
            variant="secondary"
            onClick={() => {
              setError(null);
            }}
          >
            Close
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  );
}

```



```

    });
  }

type ErrorBoundaryState = {
  hasError: boolean;
  error: GenericError | null;
};

class ErrorBoundary extends React.Component<
  {
    children: React.ReactNode;
  },
  ErrorBoundaryState
> {
  constructor(props: { children: React.ReactNode }) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error: Error, info: React.ErrorInfo) {
    console.error("ErrorBoundary caught error", error, info);

    this.setState({
      hasError: true,
      error: {
        error: error.message,
        stack: info.componentStack ?? error.stack ?? "",
      },
    });
  }

  render() {
    if (this.state.hasError) {
      return (
        <ErrorDialog
          error={
            this.state.error ?? {
              error: "An error occurred",
              stack: "",
            }
          }
          setError={() => {
            // No-op: error boundaries can't really recover without remount.
          }}
        />
      );
    }

    return this.props.children;
  }
}

export function InstrumentationProvider({
  children,
}: {
  children: React.ReactNode;
}) {
  const [error, setError] = useState<GenericError | null>(null);

  useEffect(() => {
    const handleError = async (event: ErrorEvent) => {
      try {
        // Ignore React internal DOM errors (StrictMode dev errors)
        if (
          event.message?.includes("removeChild") ||
          event.message?.includes("Failed to execute") ||
          event.error?.message?.includes("removeChild")
        )

```

```

    ) {
      console.warn("Ignoring React internal DOM error (development only)");
      return;
    }

    event.preventDefault();
    setError({
      error: event.message,
      stack: event.error?.stack || "",
      filename: event.filename || "",
      lineno: event.lineno,
      colno: event.colno,
    });

    console.error("Window error", event.error ?? event.message);
  } catch (error) {
    console.error("Error in handleError:", error);
  }
};

const handleRejection = async (event: PromiseRejectionEvent) => {
  try {
    console.error("Unhandled rejection", event.reason);

    const reason = event.reason as any;
    const message =
      typeof reason === "string"
        ? reason
        : (reason?.message ?? "Unhandled promise rejection");

    const stack =
      typeof reason === "object" && reason
        ? (reason.stack ?? JSON.stringify(reason, null, 2) ?? "")
        : "";

    setError({
      error: message,
      stack,
    });
  } catch (error) {
    console.error("Error in handleRejection:", error);
  }
};

window.addEventListener("error", handleError);
window.addEventListener("unhandledrejection", handleRejection);

return () => {
  window.removeEventListener("error", handleError);
  window.removeEventListener("unhandledrejection", handleRejection);
};
}, []);

return (
  <&&
  <ErrorBoundary>{Children}</ErrorBoundary>
  {error && <ErrorDialog error={error} setError={setError} />}
  </&&
);
}

```

■ frontend/src/lib/api.ts

```

// Centralized API client for a Spring Security session (JSESSIONID cookie).
//
// Cross-origin local dev (Vite :5173 -& Spring Boot :8080):
// - Every request MUST use `credentials: 'include'` so the browser sends JSESSIONID.
// - Backend must set proper CORS headers (allow credentials + explicit origin).
// - Cross-site cookies commonly require `SameSite=None; Secure`.
// Allow overriding in local dev: set VITE_API_BASE_URL in frontend/.env

```

```

// Example: VITE_API_BASE_URL=http://localhost:8080/api
// Prefer the Vite dev proxy by default (same-origin) to avoid CORS/cookie issues.
// In production, we MUST call the real backend origin (Render) unless the app is
// deployed behind a same-origin reverse proxy. If you deploy a proxy, set
// VITE_API_BASE_URL=/api in that environment.
const API_BASE_URL =
  import.meta.env.VITE_API_BASE_URL ||
  (import.meta.env.DEV ? '/api' : 'https://api.alliededge.app/api');

// Dev-time guard: if we're using the relative proxy base but the dev server isn't proxying,
// requests can incorrectly hit :5173 and fail with 403/404.
if (import.meta.env.DEV) {
  const isRelativeApi = !/^https?:\/\/\//i.test(API_BASE_URL);
  if (isRelativeApi && location.port === '5173') {
    // eslint-disable-next-line no-console
    console.debug('[api] Using Vite proxy via API_BASE_URL=', API_BASE_URL);
  }
}

// Required UI message for any 401/403 from the backend.
export const UNAUTHENTICATED_MESSAGE = 'Please log in to continue.';
export const FORBIDDEN_MESSAGE = 'You do not have permission to perform this action.';

export function isUnauthenticatedStatus(status: number | undefined): boolean {
  return status === 401 || status === 403;
}

export class ApiError extends Error {
  status: number;
  constructor(status: number, message: string) {
    super(message);
    this.name = 'ApiError';
    this.status = status;
  }
}

function escapeRegexLiteral(value: string): string {
  // Escape characters that have special meaning in RegExp source.
  // Reference: used widely in cookie/header parsing.
  return value.replace(/[\.*+?^${}()|[\]\\"/g, "\\$&");
}

function getCookie(name: string): string | undefined {
  const escapedName = escapeRegexLiteral(name);
  const match = document.cookie.match(
    new RegExp(`(?:^|; )${escapedName}=(\\[\\]*?)`),
  );
  return match ? decodeURIComponent(match[1]) : undefined;
}

function isFormData(body: unknown): body is FormData {
  return typeof FormData !== "undefined" && body instanceof FormData;
}

// In a cross-site deployment (Vercel & Render), some browsers partition or hide
// backend cookies from `document.cookie`. We still can obtain the CSRF token
// via the JSON body returned by GET /api/csrf, so cache it here.
let cachedCsrfToken: string | undefined;

/**
 * Single API client for the entire app.
 * - Always sends cookies (session-based auth).
 * - Throws ApiError on non-2xx.
 */
export async function apiClient<T>(path: string, init: RequestInit = {}): Promise<T> {
  if (isUnsafeMethod(init.method)) {
    await ensureCsrfToken();
  }

  const csrf = cachedCsrfToken || getCookie('XSRF-TOKEN');

```

```

const headers: Record<string, string> = {};
// Only set JSON content-type when sending JSON. Never for FormData.
if (init.body && !isFormData(init.body)) {
  headers['Content-Type'] = 'application/json';
}
if (csrf) {
  // Spring may advertise either X-XSRF-TOKEN (common default) or X-CSRF-TOKEN.
  // Send both for maximum compatibility.
  headers['X-XSRF-TOKEN'] = csrf;
  headers['X-CSRF-TOKEN'] = csrf;
}

const res = await fetch(`${API_BASE_URL}${path}`, {
  ...init,
  credentials: 'include',
  headers: {
    ...headers,
    ...(init.headers || {}),
  },
});

if (!res.ok) {
  const ct = res.headers.get('content-type') || '';

  // Prefer structured JSON error messages when provided by the API.
  if (ct.includes('application/json')) {
    try {
      const data = (await res.json()) as any;
      const msgFromJson =
        (typeof data?.message === 'string' && data.message) ||
        (typeof data?.errorMessage === 'string' && data.errorMessage) ||
        (typeof data?.error === 'string' && data.error) ||
        '';

      const message = msgFromJson
        ? msgFromJson
        : (isUnauthenticatedStatus(res.status)
            ? UNAUTHENTICATED_MESSAGE
            : `Request failed (HTTP ${res.status})`);

      throw new ApiError(res.status, message);
    } catch {
      // fall through to text parsing
    }
  }

  const text = await res.text().catch(() => '');
  // Never mention providers or any non-session auth in UI errors.
  const message = isUnauthenticatedStatus(res.status)
    ? UNAUTHENTICATED_MESSAGE
    : text || `Request failed (HTTP ${res.status})`;
  throw new ApiError(res.status, message);
}

if (res.status === 204) return undefined as unknown as T;

const ct = res.headers.get('content-type') || '';

// Some Spring handlers can return 200/201 with an empty body.
const contentLength = res.headers.get('content-length');
const isEmptyBody = contentLength === '0' || res.status === 205;

if (ct.includes('application/json')) {
  return (await res.json()) as T;
}

// If it's a successful response but not JSON, try to consume text.
// If body is empty, return undefined (callers that don't need the body will be fine).
if (ct.startsWith('text/') || ct === '' || isEmptyBody) {
  const text = await res.text().catch(() => '');
  if (!text) return undefined as unknown as T;
}

```

```

    // If caller expects a string, this will work; otherwise it's still safer than throwing.
    return text as unknown as T;
}

// Keep this generic; backend might return HTML on success in some edge cases.
// Don't hard-fail the UI for a successful status.
const fallbackText = await res.text().catch(() => '');
return (fallbackText || undefined) as unknown as T;
}

async function ensureCsrfToken(): Promise<void> {
    if (cachedCsrfToken) return;

    // Always hit the backend CSRF bootstrap endpoint. It returns JSON:
    // { token, headerName, parameterName }
    const res = await fetch(`${API_BASE_URL}/csrf`, { credentials: 'include' });
    if (!res.ok) {
        throw new ApiError(res.status, UNAUTHENTICATED_MESSAGE);
    }

    try {
        const data = (await res.json()) as any;
        if (data && typeof data.token === 'string' && data.token) {
            cachedCsrfToken = data.token;
            return;
        }
    } catch {
        // ignore; fall back to cookie-based token
    }

    // Fallback: try cookie (works in same-site environments / local dev).
    const cookieToken = getCookie('XSRF-TOKEN');
    if (cookieToken) {
        cachedCsrfToken = cookieToken;
        return;
    }

    throw new ApiError(
        403,
        'Security token not set (CSRF). Please refresh and try again. If the problem persists, allow third-party cookies for this site.'
    );
}

// Backward-compatible alias
async function ensureCsrfCookie(): Promise<void> {
    return ensureCsrfToken();
}

function isUnsafeMethod(method: string | undefined): boolean {
    const m = (method || 'GET').toUpperCase();
    return m === 'POST' || m === 'PUT' || m === 'PATCH' || m === 'DELETE';
}

// ---- API wrappers (thin helpers) ----

export const authApi = {
    status: () => apiClient<{ authenticated: boolean; name?: string; user?: User; message?: string }>('/auth/status'),
    logout: () => apiClient<void>('/auth/logout', { method: 'POST' }),
};

export const postsApi = {
    fetchPosts: async (
        params: { page?: number; size?: number; sortBy?: string; keyword?: string; signal?: AbortSignal } = {},
    ) => {
        const { items } = await postsApi.fetchPostsPaged(params);
        return items;
    },

    fetchPostsPaged: async (
        params: { page?: number; size?: number; sortBy?: string; keyword?: string; signal?: AbortSignal } = {},
    ): Promise<{ items: Post[]; pageInfo?: { pageNumber: number; pageSize: number; totalPages: number; totalElements: number } }> {

```

```

const q = new URLSearchParams();
if (params.page != null) q.set('page', String(params.page));
if (params.size != null) q.set('size', String(params.size));
if (params.sortBy) q.set('sortBy', params.sortBy);
if (params.keyword) q.set('keyword', params.keyword);
const suffix = q.toString() ? `?${q.toString()}` : '';

const raw = await apiClient<unknown>({
  '/posts${suffix}',
  params.signal ? { signal: params.signal } : undefined,
});

const rawObj = raw as Record<string, unknown> | unknown[] | null;
const postPageRaw =
  rawObj &&& !Array.isArray(rawObj)
    ? ((rawObj as Record<string, unknown>)?.postPage as Record<string, unknown> | undefined)
    : undefined;

// Backend may provide either per-item likedByMe or a set of likedPostIds.
const likedPostIdsRaw =
  postPageRaw &&& rawObj &&& !Array.isArray(rawObj)
    ? (rawObj as Record<string, unknown>).likedPostIds
    : undefined;
const likedPostIds = new Set<number>((
  Array.isArray(likedPostIdsRaw)
    ? likedPostIdsRaw.map((v) => Number(v)).filter((n) => Number.isFinite(n))
    : likedPostIdsRaw &&& typeof likedPostIdsRaw === 'object'
    ? Object.keys(likedPostIdsRaw as Record<string, unknown>).map((k) => Number(k)).filter((n) => Number.isFinite(n))
    : []
));

const maybeContent = postPageRaw?.content;
const itemsRaw: unknown[] = Array.isArray(maybeContent)
  ? maybeContent
  : Array.isArray(raw)
    ? (raw as unknown[])
    : [];

const pageInfo = postPageRaw
  ? {
    pageNumber: Number(postPageRaw.pageNumber ?? 0),
    pageSize: Number(postPageRaw.pageSize ?? params.size ?? 0),
    totalPages: Number(postPageRaw.totalPages ?? 0),
    totalElements: Number(postPageRaw.totalElements ?? 0),
    last: Boolean(postPageRaw.last ?? false),
  }
  : undefined;

const items = itemsRaw
  .map((pUnknown: unknown) => {
    const p = (pUnknown ?? {}) as Record<string, unknown>;

    const authorName =
      (p.authorDisplayName as string | undefined) ||
      (p.authorName as string | undefined) ||
      (p.authorUsername as string | undefined) ||
      'Unknown';

    const authorIsAdmin =
      typeof p.authorIsAdmin === 'boolean'
        ? (p.authorIsAdmin as boolean)
        : typeof (p as any).author_is_admin === 'boolean'
          ? ((p as any).author_is_admin as boolean)
          : undefined;

    const imageUrlsRaw = p.imageUrls;
    const imageUrls = Array.isArray(imageUrlsRaw)
      ? imageUrlsRaw.filter((u): u is string => typeof u === 'string')
      : undefined;

    const videoUrl = typeof p.videoUrl === 'string' ? p.videoUrl : undefined;
  })

```

```

    return {
      id: Number(p.id ?? 0),
      title: typeof p.title === 'string' ? p.title : undefined,
      content: String(p.content ?? ''),
      createdAt: String(p.createdAt ?? ''),
      likes: Number(p.likes ?? 0),
      views: Number(p.views ?? 0),
      likedByMe:
        typeof p.likedByMe === 'boolean'
          ? (p.likedByMe as boolean)
          : (likedPostIds.has(Number(p.id)) ? true : undefined),
      imageUrls,
      videoUrl,
      updatedAt: p.updatedAt != null ? String(p.updatedAt as unknown) : undefined,
      commentCount: Number(p.commentCount ?? 0),
      comments: undefined,
      author: {
        id: Number(p.authorId ?? 0),
        name: String(authorName),
        avatar:
          p.authorProfileImageUrl != null
            ? String(p.authorProfileImageUrl)
            : undefined,
        role: authorIsAdmin ? 'ROLE_ADMIN' : undefined,
      },
    } as Post;
  })
  .filter((p) => p.id);

  return { items, pageInfo };
},
// Convenience for legacy callers that expect an "all posts" list.
getAll: () => postsApi.fetchPosts({ page: 0, size: 100 }),

/** Admin paged listing (scalable). Mirrors backend GET /api/admin/posts. */
fetchAdminPaged: async (
  params: { page?: number; size?: number; sortBy?: string; keyword?: string; signal?: AbortSignal } = {},
): Promise<{ items: Post[]; pageInfo?: { pageNumber: number; pageSize: number; totalPages: number; totalElements: number }}> {
  const q = new URLSearchParams();
  if (params.page != null) q.set('page', String(params.page));
  if (params.size != null) q.set('size', String(params.size));
  if (params.sortBy) q.set('sortBy', params.sortBy);
  if (params.keyword) q.set('keyword', params.keyword);
  const suffix = q.toString() ? `?${q.toString()}` : '';

  const raw = await apiClient<unknown>({
    path: `/admin/posts${suffix}`,
    params: { signal: params.signal } : undefined,
  });

  const rawObj = (raw ?? {}) as Record<string, unknown>;
  const postPageRaw = (rawObj.postPage ?? rawObj.posts ?? rawObj.page) as any;

  const contentRaw: unknown[] = Array.isArray(postPageRaw?.content)
    ? (postPageRaw.content as unknown[])
    : [];

  // Minimal admin mapping: accept entity-like Post from backend and coerce into our Post type.
  const adminItems: Post[] = contentRaw
    .map((pUnknown: unknown) => {
      const p = (pUnknown ?? {}) as Record<string, unknown>;
      const authorRaw = (p.user ?? p.author ?? {}) as Record<string, unknown>;
      const authorName =
        (authorRaw.displayName as string | undefined) ||
        (authorRaw.username as string | undefined) ||
        (p.authorDisplayName as string | undefined) ||
        (p.authorName as string | undefined) ||
        'Unknown';

      const imageUrlsRaw = p.imageUrls;
      const imageUrls = Array.isArray(imageUrlsRaw)

```

```

    ? imageUrlRaw.filter((u): u is string => typeof u === 'string')
    : undefined;

const videoUrl = typeof p.videoUrl === 'string' ? p.videoUrl : undefined;

return {
  id: Number(p.id ?? 0),
  title: typeof p.title === 'string' ? p.title : undefined,
  content: String(p.content ?? ''),
  createdAt: String(p.createdAt ?? ''),
  updatedAt: p.updatedAt != null ? String(p.updatedAt as unknown) : undefined,
  likes: Number(p.likes ?? 0),
  views: Number(p.views ?? 0),
  likedByMe: typeof p.likedByMe === 'boolean' ? (p.likedByMe as boolean) : undefined,
  imageUrl,
  videoUrl,
  comments: undefined,
  author: {
    id: Number(authorRaw.id ?? p.authorId ?? 0),
    name: String(authorName),
    avatar:
      authorRaw.profileImageUrl != null
        ? String(authorRaw.profileImageUrl)
        : (p.authorProfileImageUrl != null ? String(p.authorProfileImageUrl) : undefined),
  },
} as Post;
})
.filter((p) => p.id);

const adminPageInfo = postPageRaw
  ? {
    pageNumber: Number(postPageRaw.number ?? params.page ?? 0),
    pageSize: Number(postPageRaw.size ?? params.size ?? 0),
    totalPages: Number(postPageRaw.totalPages ?? 0),
    totalElements: Number(postPageRaw.totalElements ?? 0),
    last: Boolean(postPageRaw.last ?? false),
  }
  : undefined;

return { items: adminItems, pageInfo: adminPageInfo };
},

getId: async (id: number, opts: { sortComments?: 'newest' | 'oldest' | 'mostReplied' } = {}) => {
  const q = new URLSearchParams();
  if (opts.sortComments) q.set('sortComments', opts.sortComments);
  const suffix = q.toString() ? `?${q.toString()}` : '';

  const raw = await apiClient<unknown>(`/posts/${id}${suffix}`);
  const rawObj = (raw ?? {}) as Record<string, unknown>;

  // Backend returns { post: <PostDetailDto>, liked: boolean, ... }
  const postRaw = (rawObj.post ?? rawObj) as Record<string, unknown>;

  const imageUrlRaw = postRaw.imageUrl;
  const imageUrl = Array.isArray(imageUrlRaw)
    ? imageUrlRaw.filter((u): u is string => typeof u === 'string')
    : undefined;

  const videoUrl = typeof postRaw.videoUrl === 'string' ? postRaw.videoUrl : undefined;

  const likedByMe = typeof rawObj.liked === 'boolean' ? (rawObj.liked as boolean) : undefined;

  const authorRaw = (postRaw.author ?? {}) as Record<string, unknown>;
  const authorName =
    (authorRaw.displayName as string | undefined) ||
    (authorRaw.username as string | undefined) ||
    'Unknown';

  const authorIsAdmin = typeof (authorRaw as any).admin === 'boolean' ? ((authorRaw as any).admin as boolean) : undefined;

  const commentsRaw = Array.isArray(postRaw.comments) ? (postRaw.comments as unknown[]) : [];

```



```

const comments: Comment[] = commentsRaw
  .map((cUnknown) => {
    const c = (cUnknown ?? {}) as Record<string, unknown>;
    const ca = (c.author ?? {}) as Record<string, unknown>;

    const caName =
      (ca.displayName as string | undefined) ||
      (ca.username as string | undefined) ||
      'Unknown';

    const repliesRaw = Array.isArray(c.replies) ? (c.replies as unknown[]) : [];
    const replies: Reply[] = repliesRaw
      .map((rUnknown) => {
        const r = (rUnknown ?? {}) as Record<string, unknown>;
        const ra = (r.author ?? {}) as Record<string, unknown>;

        const raName =
          (ra.displayName as string | undefined) ||
          (ra.username as string | undefined) ||
          'Unknown';

        return {
          id: Number(r.id ?? 0),
          content: String(r.content ?? ''),
          createdAt: String(r.createdAt ?? ''),
          likes: Number(r.likes ?? 0),
          likedByMe: typeof r.likedByMe === 'boolean' ? (r.likedByMe as boolean) : undefined,
          author: {
            id: Number(ra.id ?? 0),
            name: String(raName),
            avatar: ra.profileImageUrl != null ? String(ra.profileImageUrl) : undefined,
          },
        } as Reply;
      })
      .filter((r) => r.id);

    return {
      id: Number(c.id ?? 0),
      content: String(c.content ?? ''),
      createdAt: String(c.createdAt ?? ''),
      updatedAt: undefined,
      likes: Number(c.likes ?? 0),
      likedByMe: typeof c.likedByMe === 'boolean' ? (c.likedByMe as boolean) : undefined,
      author: {
        id: Number(ca.id ?? 0),
        name: String(caName),
        avatar:
          ca.profileImageUrl != null
            ? String(ca.profileImageUrl)
            : undefined,
      },
      replies,
    } as Comment;
  })
  .filter((c) => c.id);

return {
  id: Number(postRaw.id ?? id),
  title: typeof postRaw.title === 'string' ? (postRaw.title as string) : undefined,
  content: String(postRaw.content ?? ''),
  createdAt: String(postRaw.createdAt ?? ''),
  updatedAt: postRaw.updatedAt != null ? String(postRaw.updatedAt as unknown) : undefined,
  likes: Number(postRaw.likes ?? 0),
  views: Number(postRaw.views ?? 0),
  likedByMe,
  imageUrls,
  videoUrl,
  comments,
  author: {
    id: Number(authorRaw.id ?? 0),
    name: String(authorName),
  },
};

```

```

        avatar:
          authorRaw.profileImageUrl != null
            ? String(authorRaw.profileImageUrl)
            : undefined,
      },
    } as Post;
  },
  // Prefer multipart endpoints because backend expects @ModelAttribute for /posts POST/PUT.
  create: (data: { content: string; title?: string }) => {
    const form = new FormData();
    if (data.title != null) form.append('title', data.title);
    form.append('content', data.content);
    return postsApi.createMultipart(form);
  },
  update: (id: number, data: { content: string; title?: string }) => {
    const form = new FormData();
    if (data.title != null) form.append('title', data.title);
    form.append('content', data.content);
    return postsApi.updateMultipart(id, form);
  },
  delete: (id: number) => apiClient<void>(`/posts/${id}`, { method: 'DELETE' }),
  like: (id: number) => apiClient<{ id: number; likes: number; liked: boolean }>(`/posts/${id}/like`, { method: 'POST' }),
  getPublic: () => postsApi.fetchPosts({ page: 0, size: 20 }),
  createMultipart: (form: FormData) => apiClient<Post>(`/posts`, { method: 'POST', body: form }),
  updateMultipart: (id: number, form: FormData) => apiClient<Post>(`/posts/${id}`, { method: 'PUT', body: form }),
  mine: () => apiClient<unknown>(`/posts/mine?page=0&size=50`),
};

export const commentsApi = {
  create: (postId: number, data: { content: string }) =>
    apiClient<void>(`/posts/${postId}/comments`, { method: 'POST', body: JSON.stringify(data) }),
  update: (id: number, data: { content: string }) =>
    apiClient<{ comment: Comment; postId: number | null }>(`/comments/${id}`, { method: 'PUT', body: JSON.stringify(data) }),
  createReply: (commentId: number, data: { content: string }) =>
    apiClient<{ reply: Reply; commentId: number }>(`/comments/${commentId}/replies`, {
      method: 'POST',
      body: JSON.stringify(data),
    }),
  like: (commentId: number) =>
    apiClient<{ id: number; likes: number; liked: boolean }>(`/comments/${commentId}/like`, { method: 'POST' }),
  likeReply: (replyId: number) =>
    apiClient<{ id: number; likes: number; liked: boolean }>(`/replies/${replyId}/like`, { method: 'POST' }),
};

export const announcementsApi = {
  listPublic: () => apiClient<Announcement[]>('/announcements'),
  listAdmin: () => apiClient<Announcement[]>('/admin/announcements'),
  getAdmin: (id: number) => apiClient<Announcement>(`/admin/announcements/${id}`),

  /** Admin paged listing (scalable). Backend must support ?page=&size= (Spring Data). */
  listAdminPaged: async (
    params: { page?: number; size?: number; signal?: AbortSignal } = {},
  ): Promise<{ items: Announcement[]; pageInfo?: { pageNumber: number; pageSize: number; totalPages: number; totalElements: number } }> {
    const q = new URLSearchParams();
    if (params.page != null) q.set('page', String(params.page));
    if (params.size != null) q.set('size', String(params.size));
    const suffix = q.toString() ? `?${q.toString()}` : '';

    const raw = await apiClient<unknown>(`/admin/announcements${suffix}`, {
      params: { signal: params.signal } : undefined,
    });

    // Support both old (array) and new (paged) backend responses.
    if (Array.isArray(raw)) {
      return {
        items: raw as Announcement[],
        pageInfo: {
          pageNumber: params.page ?? 0,
          pageSize: params.size ?? (raw as unknown[]).length,
          totalPages: 1,
        },
      };
    }
  },
};

```

```

        totalElements: (raw as unknown[]).length,
        last: true,
    },
};
}

const obj = (raw ?? {}) as Record<string, unknown>;
const page = (obj.announcementsPage ?? obj.page ?? obj.announcementPage ?? obj) as any;
const content: Announcement[] = Array.isArray(page?.content) ? (page.content as Announcement[]) : [];

const pageInfo = page
? {
    pageNumber: Number(page.number ?? page.pageNumber ?? params.page ?? 0),
    pageSize: Number(page.size ?? page.pageSize ?? params.size ?? 0),
    totalPages: Number(page.totalPages ?? 0),
    totalElements: Number(page.totalElements ?? 0),
    last: Boolean(page.last ?? false),
}
: undefined;

return { items: content, pageInfo };
},

create: (data: {
    title: string;
    message: string;
    visible?: boolean;
    imageFiles?: File[];
    videoFile?: File | null;
}) => {
    const form = new FormData();
    form.append('title', data.title);
    form.append('message', data.message);
    if (data.visible != null) form.append('visible', String(data.visible));
    if (data.imageFiles && data.imageFiles.length > 0) {
        data.imageFiles.forEach((f) => {
            // Some servers bind repeated fields better when using [] notation.
            form.append('imageFiles', f);
            form.append('imageFiles[]', f);
        });
    }
    if (data.videoFile) form.append('videoFile', data.videoFile);

    return apiClient<Announcement>('/admin/announcements', { method: 'POST', body: form });
},

update: (id: number, data: {
    title?: string;
    message?: string;
    visible?: boolean;
    removeImages?: boolean;
    imageFiles?: File[];
    removeVideo?: boolean;
    videoFile?: File | null;
}) => {
    const form = new FormData();
    if (data.title != null) form.append('title', data.title);
    if (data.message != null) form.append('message', data.message);
    if (data.visible != null) form.append('visible', String(data.visible));
    if (data.removeImages != null) form.append('removeImages', String(data.removeImages));
    if (data.imageFiles && data.imageFiles.length > 0) {
        data.imageFiles.forEach((f) => {
            form.append('imageFiles', f);
            form.append('imageFiles[]', f);
        });
    }
    if (data.removeVideo != null) form.append('removeVideo', String(data.removeVideo));
    if (data.videoFile) form.append('videoFile', data.videoFile);

    return apiClient<Announcement>(`/admin/announcements/${id}`, { method: 'PUT', body: form });
},

```

```

delete: (id: number) => apiClient<void>(`/admin/announcements/${id}`, { method: 'DELETE' }),
toggleVisible: (id: number) => apiClient<{ message: string }>(`/admin/announcements/${id}/toggle`, { method: 'POST' }),
deleteVideo: (id: number) => apiClient<Announcement>(`/admin/announcements/${id}/delete-video`, { method: 'DELETE' });
};

export const profileApi = {
  get: async () => {
    const raw = await apiClient<unknown>("/profile");
    const rawObj = (raw ?? {}) as Record<string, unknown>;
    const u = (rawObj.user as Record<string, unknown> | undefined) ?? rawObj;

    const parseJSONArray = <T>(value: unknown): T[] | undefined => {
      if (value == null) return undefined;
      if (Array.isArray(value)) return value as T[];
      if (typeof value === 'string' && value.trim()) {
        try {
          const parsed = JSON.parse(value);
          return Array.isArray(parsed) ? (parsed as T[]) : undefined;
        } catch {
          return undefined;
        }
      }
      return undefined;
    };

    const parseJSONObject = <T extends object>(value: unknown): T | undefined => {
      if (value == null) return undefined;
      if (typeof value === 'object') return value as T;
      if (typeof value === 'string' && value.trim()) {
        try {
          const parsed = JSON.parse(value);
          return (parsed && typeof parsed === 'object') ? (parsed as T) : undefined;
        } catch {
          return undefined;
        }
      }
      return undefined;
    };

    const experience =
      parseJSONArray<Experience>(rawObj?.experience ?? u?.experience ?? (u as any)?.experienceJson) ??
      parseJSONArray<Experience>((u as any)?.experienceJson);

    const languages =
      parseJSONArray<Language>(rawObj?.languages ?? u?.languages ?? (u as any)?.languagesJson) ??
      parseJSONArray<Language>((u as any)?.languagesJson);

    const education =
      parseJSONArray<Education>(rawObj?.education ?? u?.education ?? (u as any)?.educationJson) ??
      parseJSONArray<Education>((u as any)?.educationJson);

    const availability =
      (rawObj?.availability as Availability | undefined) ||
      (u?.availability as Availability | undefined) ||
      parseJSONObject<Availability>((u as any)?.availabilityJson);

    // NOTE: /profile is the owner-only endpoint. We should show email & resume regardless of the
    // public visibility flags; those flags are for public/shareable profile pages.

    // Owner endpoint still returns the flags; surface them so the edit dialog can stay in sync.
    const showEmailOnProfile = Boolean((u as any)?.showEmailOnProfile);
    const showResumeOnProfile = Boolean((u as any)?.showResumeOnProfile);

    return {
      id: Number(u?.id),
      name: String(u?.displayName || u?.name || u?.username || ""),
      email: String((u as any)?.email || ""),
      role: ((u as any)?.role as any) | undefined,
      username: (u?.username as string | undefined) | undefined,
      avatar: (u?.profileImageUrl as string | undefined) || (u?.avatar as string | undefined) | undefined,
      bannerImage: (u?.bannerImageUrl as string | undefined) || (u?.bannerImage as string | undefined) | undefined,
    };
  }
};

```

```

resumeUrl: ((u as any)?.resumeUrl as string | undefined) || undefined,
location: (u?.location as string | undefined) || undefined,
website: (u?.website as string | undefined) || undefined,
github: (u?.github as string | undefined) || undefined,
linkedin: (u?.linkedin as string | undefined) || undefined,
twitter: (u?.twitter as string | undefined) || undefined,
followersCount:
  (rawObj?.followersCount as number | undefined) ??
  (rawObj?.followerCount as number | undefined) ??
  (u?.followersCount as number | undefined),
followingCount: (rawObj?.followingCount as number | undefined) ?? (u?.followingCount as number | undefined),
bio: (u?.bio as string | undefined) || undefined,
skills: (rawObj?.skills ?? u?.skills ?? []) as string[],
projects: (rawObj?.projects ?? u?.projects ?? []) as Project[],
experience,
languages,
education,
availability,
showEmailOnProfile,
showResumeOnProfile,
} as UserProfile;
},
getId: async (userId: number) => {
  const raw = await apiClient<unknown>(`/profile/users/${userId}`);
  const rawObj = (raw ?? {}) as Record<string, unknown>;

  // /profile/users/{id} returns { user, posts, projects, followerCount, ... }
  const u = (rawObj.user as Record<string, unknown> | undefined) ?? rawObj;
  const roleFromPublic = (u as any)?.admin === true ? ('ROLE_ADMIN' as const) : undefined;

  const parseJsonArray = <T>(value: unknown): T[] | undefined => {
    if (value == null) return undefined;
    if (Array.isArray(value)) return value as T[];
    if (typeof value === 'string' && value.trim()) {
      try {
        const parsed = JSON.parse(value);
        return Array.isArray(parsed) ? (parsed as T[]) : undefined;
      } catch {
        return undefined;
      }
    }
    return undefined;
  };

  const parseJsonObject = <T extends object>(value: unknown): T | undefined => {
    if (value == null) return undefined;
    if (typeof value === 'object') return value as T;
    if (typeof value === 'string' && value.trim()) {
      try {
        const parsed = JSON.parse(value);
        return parsed && typeof parsed === 'object' ? (parsed as T) : undefined;
      } catch {
        return undefined;
      }
    }
    return undefined;
  };

  const experience =
    parseJsonArray<Experience>(rawObj?.experience ?? u?.experience ?? (u as any)?.experienceJson) ??
    parseJsonArray<Experience>((u as any)?.experienceJson);

  const languages =
    parseJsonArray<Language>(rawObj?.languages ?? u?.languages ?? (u as any)?.languagesJson) ??
    parseJsonArray<Language>((u as any)?.languagesJson);

  const education =
    parseJsonArray<Education>(rawObj?.education ?? u?.education ?? (u as any)?.educationJson) ??
    parseJsonArray<Education>((u as any)?.educationJson);

  const availability =

```

```

    parseJsonObject<Availability>(rawObj?.availability) ||
    (u?.availability as Availability | undefined) ||
    parseJsonObject<Availability>((u as any)?.availabilityJson);

const showEmailOnProfile = Boolean((u as any)?.showEmailOnProfile);
const showResumeOnProfile = Boolean((u as any)?.showResumeOnProfile);

return {
    id: Number(u?.id),
    name: String(u?.displayName || u?.name || u?.username || ''),
    email: showEmailOnProfile ? String((u as any)?.email || '') : '',
    role: (((u as any)?.role as any) || roleFromPublic) || undefined,
    username: (u?.username as string | undefined) || undefined,
    avatar:
        (u?.profileImageUrl as string | undefined) ||
        (u?.avatar as string | undefined) ||
        undefined,
    bannerImage: (u?.bannerImageUrl as string | undefined) || (u?.bannerImage as string | undefined) || undefined,
    resumeUrl: showResumeOnProfile ? ((u as any)?.resumeUrl as string | undefined) : undefined,
    location: (u?.location as string | undefined) || undefined,
    website: (u?.website as string | undefined) || undefined,
    github: (u?.github as string | undefined) || undefined,
    linkedin: (u?.linkedin as string | undefined) || undefined,
    twitter: (u?.twitter as string | undefined) || undefined,
    followersCount:
        (rawObj?.followersCount as number | undefined) ??
        (rawObj?.followerCount as number | undefined) ??
        (u?.followersCount as number | undefined),
    followingCount: (rawObj?.followingCount as number | undefined) ?? (u?.followingCount as number | undefined),
    bio: (u?.bio as string | undefined) || undefined,
    showEmailOnProfile,
    showResumeOnProfile,
    isFollowing: (rawObj?.isFollowing as boolean | undefined) ?? undefined,
    canMessage: (rawObj?.canMessage as boolean | undefined) ?? undefined,
    skills: (rawObj?.skills ?? u?.skills ?? []) as string[],
    projects: (rawObj?.projects ?? u?.projects ?? []) as Project[],
    experience,
    languages,
    education,
    availability,
} as UserProfile;
},

getByUsername: async (username: string) => {
    const raw = await apiClient<unknown>(`/profile/u/${encodeURIComponent(username)}`);
    const rawObj = (raw ?? {}) as Record<string, unknown>;

    // /profile/u/{username} returns { user, posts, projects, followerCount, ... }
    const u = (rawObj.user as Record<string, unknown> | undefined) ?? rawObj;
    const roleFromPublic = (u as any)?.admin === true ? ('ROLE_ADMIN' as const) : undefined;

    const parseJsonArray = <T>(value: unknown): T[] | undefined => {
        if (value == null) return undefined;
        if (Array.isArray(value)) return value as T[];
        if (typeof value === 'string' && value.trim()) {
            try {
                const parsed = JSON.parse(value);
                return Array.isArray(parsed) ? (parsed as T[]) : undefined;
            } catch {
                return undefined;
            }
        }
        return undefined;
    };

    const parseJsonObject = <T extends object>(value: unknown): T | undefined => {
        if (value == null) return undefined;
        if (typeof value === 'object') return value as T;
        if (typeof value === 'string' && value.trim()) {
            try {
                const parsed = JSON.parse(value);

```

```

        return parsed && typeof parsed === 'object' ? (parsed as T) : undefined;
    } catch {
        return undefined;
    }
}
return undefined;
};

const experience =
    parseJsonArray<Experience>(rawObj?.experience ?? u?.experience ?? (u as any)?.experienceJson) ??
    parseJsonArray<Experience>((u as any)?.experienceJson);

const languages =
    parseJsonArray<Language>(rawObj?.languages ?? u?.languages ?? (u as any)?.languagesJson) ??
    parseJsonArray<Language>((u as any)?.languagesJson);

const education =
    parseJsonArray<Education>(rawObj?.education ?? u?.education ?? (u as any)?.educationJson) ??
    parseJsonArray<Education>((u as any)?.educationJson);

const availability =
    parseJsonObject<Availability>(rawObj?.availability) ||
    (u?.availability as Availability | undefined) ||
    parseJsonObject<Availability>((u as any)?.availabilityJson);

const showEmailOnProfile = Boolean((u as any)?.showEmailOnProfile);
const showResumeOnProfile = Boolean((u as any)?.showResumeOnProfile);

return {
    id: Number(u?.id),
    name: String(u?.displayName || u?.name || u?.username || ''),
    email: showEmailOnProfile ? String((u as any)?.email || '') : '',
    role: (((u as any)?.role as any) || roleFromPublic) || undefined,
    username: (u?.username as string | undefined) || undefined,
    avatar: (u?.profileImageUrl as string | undefined) || (u?.avatar as string | undefined) || undefined,
    bannerImage: (u?.bannerImageUrl as string | undefined) || (u?.bannerImage as string | undefined) || undefined,
    resumeUrl: showResumeOnProfile ? ((u as any)?.resumeUrl as string | undefined) : undefined,
    location: (u?.location as string | undefined) || undefined,
    website: (u?.website as string | undefined) || undefined,
    github: (u?.github as string | undefined) || undefined,
    linkedin: (u?.linkedin as string | undefined) || undefined,
    twitter: (u?.twitter as string | undefined) || undefined,
    followersCount:
        (rawObj?.followersCount as number | undefined) ??
        (rawObj?.followerCount as number | undefined) ??
        (u?.followersCount as number | undefined),
    followingCount: (rawObj?.followingCount as number | undefined) ?? (u?.followingCount as number | undefined),
    bio: (u?.bio as string | undefined) || undefined,
    showEmailOnProfile,
    showResumeOnProfile,
    isFollowing: (rawObj?.isFollowing as boolean | undefined) ?? undefined,
    canMessage: (rawObj?.canMessage as boolean | undefined) ?? undefined,
    skills: (rawObj?.skills ?? u?.skills ?? []) as string[],
    projects: (rawObj?.projects ?? u?.projects ?? []) as Project[],
    experience,
    languages,
    education,
    availability,
} as UserProfile;
},
uploadResume: async (file: File) => {
    const form = new FormData();
    form.append('resumeFile', file);
    return apiClient<{ message: string }>('/profile/resume', { method: 'POST', body: form });
},
deleteResume: async () => {
    return apiClient<{ message: string }>('/profile/resume', { method: 'DELETE' });
},
update: async (data: {
    username?: string;
    name?: string;

```

```

bio?: string;
linkedin?: string;
github?: string;
website?: string;
profileImageFile?: File | null;
removeProfileImage?: boolean;
bannerImageFile?: File | null;
removeBannerImage?: boolean;
location?: string;
twitter?: string;
resumeUrl?: string;
skills?: string[];
projects?: Project[];
experience?: Experience[];
languages?: Language[];
education?: Education[];
availability?: Availability;
showEmailOnProfile?: boolean;
showResumeOnProfile?: boolean;
}) => {
  const form = new FormData();

  if (data.username != null) form.append('username', data.username);
  if (data.name != null) form.append('displayName', data.name);
  if (data.bio != null) form.append('bio', data.bio);
  if (data.linkedin != null) form.append('linkedin', data.linkedin);
  if (data.github != null) form.append('github', data.github);
  if (data.website != null) form.append('website', data.website);

  if (data.location != null) form.append('location', data.location);
  if (data.twitter != null) form.append('twitter', data.twitter);

  if (data.skills != null) form.append('skillsJson', JSON.stringify(data.skills));
  if (data.projects != null) form.append('projectsJson', JSON.stringify(data.projects));
  if (data.experience != null) form.append('experienceJson', JSON.stringify(data.experience));
  if (data.languages != null) form.append('languagesJson', JSON.stringify(data.languages));
  if (data.education != null) form.append('educationJson', JSON.stringify(data.education));
  if (data.availability != null) form.append('availabilityJson', JSON.stringify(data.availability));

  if (data.showEmailOnProfile != null) form.append('showEmailOnProfile', String(Boolean(data.showEmailOnProfile)));
  if (data.showResumeOnProfile != null) form.append('showResumeOnProfile', String(Boolean(data.showResumeOnProfile)));

  form.append('removeProfileImage', String(Boolean(data.removeProfileImage)));
  form.append('removeBannerImage', String(Boolean(data.removeBannerImage)));

  if (data.profileImageFile) form.append('profileImageFile', data.profileImageFile);
  if (data.bannerImageFile) form.append('bannerImageFile', data.bannerImageFile);

  // Backend returns { message, user }
  const resp = await apiClient<unknown>('/profile', { method: 'PUT', body: form });
  const respObj = (resp ?? {}) as Record<string, unknown>;
  const u = (respObj.user as Record<string, unknown> | undefined) ?? (respObj as Record<string, unknown>);

  const parseJSONArray = <T>(value: unknown): T[] | undefined => {
    if (value == null) return undefined;
    if (Array.isArray(value)) return value as T[];
    if (typeof value === 'string' && value.trim()) {
      try {
        const parsed = JSON.parse(value);
        return Array.isArray(parsed) ? (parsed as T[]) : undefined;
      } catch {
        return undefined;
      }
    }
    return undefined;
  };

  const parseJSONObject = <T extends object>(value: unknown): T | undefined => {
    if (value == null) return undefined;
    if (typeof value === 'object') return value as T;
    if (typeof value === 'string' && value.trim()) {

```



```

    try {
      const parsed = JSON.parse(value);
      return (parsed && typeof parsed === 'object') ? (parsed as T) : undefined;
    } catch {
      return undefined;
    }
  }
  return undefined;
};

const experience = parseJsonArray<Experience>((u as any)?.experienceJson) ?? data.experience;
const languages = parseJsonArray<Language>((u as any)?.languagesJson) ?? data.languages;
const education = parseJsonArray<Education>((u as any)?.educationJson) ?? data.education;
const availability = parseJsonObject<Availability>((u as any)?.availabilityJson) ?? data.availability;

const showEmailOnProfile = Boolean((u as any)?.showEmailOnProfile);
const showResumeOnProfile = Boolean((u as any)?.showResumeOnProfile);

// Map backend user DTO to UI UserProfile shape.
return {
  id: Number(u?.id ?? 0),
  name: String(u?.displayName || u?.name || u?.username || data.name || ''),
  email: String(u?.email || ''),
  username: (u?.username as string | undefined) ?? data.username,
  avatar: (u?.profileImageUrl as string | undefined) || undefined,
  bannerImage: (u?.bannerImageUrl as string | undefined) || undefined,
  bio: (u?.bio as string | undefined) || data.bio,
  website: (u?.website as string | undefined) || data.website,
  github: (u?.github as string | undefined) || data.github,
  linkedin: (u?.linkedin as string | undefined) || data.linkedin,
  twitter: (u?.twitter as string | undefined) ?? data.twitter,
  location: (u?.location as string | undefined) || data.location,
  resumeUrl: (u?.resumeUrl as string | undefined) || data.resumeUrl,
  followersCount: undefined,
  followingCount: undefined,
  skills: (data.skills ?? []) as string[],
  projects: (data.projects ?? []) as Project[],
  experience,
  languages,
  education,
  availability,
  showEmailOnProfile,
  showResumeOnProfile,
} as UserProfile;
},
};

// --- Users / Follow ---

export const usersApi = {
  search: async (keyword: string) => {
    const resp = await apiClient<{ users: any[]; keyword: string }>(`/users/search?keyword=${encodeURIComponent(keyword)}`,
    );

    const users = (resp.users || []).map((u: any) => ({
      ...u,
      avatar: (u?.profileImageUrl as string | undefined) || (u?.avatar as string | undefined) || undefined,
    })) as User[];

    return { ...resp, users } as { users: User[]; keyword: string };
  },
};

export const followApi = {
  follow: (userId: number) => apiClient<string>(`/users/${userId}/follow`, { method: 'POST' }),
  unfollow: (userId: number) => apiClient<string>(`/users/${userId}/unfollow`, { method: 'POST' }),
};

// ---- Chat ---
export type ChatRoomSummary = {

```

```

    id: number;
    otherUserId: number;
    otherUserUsername?: string;
    otherUserDisplayName: string;
    otherUserProfileImage?: string;
    lastMessage?: string;
    lastMessageTime?: string;
  };

export type ChatMessage = {
  roomId: number;
  senderUsername: string;
  senderDisplayName: string;
  content: string;
  timestamp: string;
};

export type ChatMessagesPage = {
  messages: ChatMessage[];
  hasMore?: boolean;
};

export type ChatTargetUser = {
  id: number;
  username: string;
  displayName?: string;
  profileImageUrl?: string;
};

export type OpenChatResponse = {
  canMessage: boolean;
  reason?: 'FOLLOW_REQUIRED' | 'FOLLOW_BACK_REQUIRED' | string;
  message?: string;
  chatRoom?: { id: number };
  targetUser: ChatTargetUser;
  currentUser?: any;
  messages?: ChatMessage[];
};

export const chatApi = {
  list: () => apiClient.get({ chatRooms: ChatRoomSummary[]; currentUser: any } & `/${chats}`),

  /**
   * Opens a chat thread with a given user.
   * Backend may respond with 403 for locked messaging along with a JSON body.
   */
  openWith: async (userId: number): Promise<OpenChatResponse> => {
    try {
      return await apiClient.get<OpenChatResponse>(`/${chats}/with/${userId}`);
    } catch (e) {
      // Special-case locked messaging: backend returns 403 with a useful JSON body.
      if (e instanceof ApiError && e.status === 403) {
        // Re-try without throwing so UI can show the locked state.
        // Use fetch directly to preserve the response body.
        const res = await fetch(`${API_BASE_URL}/${chats}/with/${userId}`, { credentials: 'include' });
        const data = (await res.json()).catch(() => ({})) as OpenChatResponse;
        return {
          ...data,
          canMessage: Boolean((data as any)?.canMessage),
        };
      }
      throw e;
    }
  },

  /**
   * Infinite scroll page (newest-first from backend).
   */
  listMessages: (roomId: number, opts: { size?: number; before?: string } = {}) => {
    const q = new URLSearchParams();
    if (opts.size != null) q.set('size', String(opts.size));
  }
};

```

```

    if (opts.before) q.set('before', opts.before);
    const suffix = q.toString() ? `?${q.toString()}` : '';
    return apiClient<&ChatMessagesPage>(`/chats/${roomId}/messages${suffix}`);
  },

  /**
   * Persisted send fallback (REST). Use when WS is unavailable.
   */
  send: (roomId: number, content: string) => {
    apiClient<&ChatMessage>(`/chats/${roomId}/messages`, {
      method: 'POST',
      body: JSON.stringify({ content }),
    }),

    /**
     * Fallback refresh: re-open the thread to get updated messages.
     */
    refreshThread: (userId: number) => chatApi.openWith(userId),
  };

// Admin: user management
export const adminUsersApi = {
  list: (params: { keyword?: string; page?: number; size?: number } = {}) => {
    const q = new URLSearchParams();
    if (params.keyword) q.set('keyword', params.keyword);
    if (params.page != null) q.set('page', String(params.page));
    if (params.size != null) q.set('size', String(params.size));
    const suffix = q.toString() ? `?${q.toString()}` : '';
    return apiClient<&any>(`/admin/users${suffix}`);
  },

  userPosts: (userId: number) => apiClient<&any>(`/admin/users/${userId}/posts`),

  ban: (userId: number) => apiClient<&any>(`/admin/users/${userId}/ban`, { method: 'POST' }),
  unban: (userId: number) => apiClient<&any>(`/admin/users/${userId}/unban`, { method: 'POST' }),
};

export const adminStatsApi = {
  postStats: () => apiClient<&{ totalUsers: number; activeUsers: number; totalComments: number; totalLikes: number }>,
};

// ---- Types used by the UI ----

export interface User {
  id: number;
  name: string;
  email: string;
  avatar?: string;
  role?: 'ROLE_USER' | 'ROLE_AUTHOR' | 'ROLE_ADMIN';
  username?: string;
  bio?: string;
  skills?: string[];
  github?: string;
  linkedin?: string;
  twitter?: string;
}

export interface Reply {
  id: number;
  author: {
    id: number;
    name: string;
    avatar?: string;
  };
  content: string;
  createdAt: string;
  likes?: number;
  likedByMe?: boolean;
}

export type Comment = {

```

```

    id: number;
    content: string;
    createdAt: string;
    author: {
        id: number;
        name: string;
        avatar?: string;
    };
    replies?: Reply[];
    likes?: number;
    likedByMe?: boolean;
}

export interface Post {
    id: number;
    author: {
        id: number;
        name: string;
        avatar?: string;
        role?: 'ROLE_USER' | 'ROLE_AUTHOR' | 'ROLE_ADMIN';
    };
    title?: string;
    content: string;
    imageUrls?: string[];
    videoUrl?: string;
    likes: number;
    /** Total view count for this post. */
    views?: number;
    likedByMe?: boolean;
    createdAt: string;
    updatedAt?: string;
    /** Present on feed items; avoids having to load the full comments array. */
    commentCount?: number;
    comments?: Comment[];
}

export interface Project {
    id?: number;
    title: string;
    // Backward-compat: older UI used `description`, newer profile UI uses richer fields.
    description?: string;
    link?: string;

    summary?: string;
    status?: 'Building' | 'Shipped' | string;
    problem?: string;
    built?: string;
    role?: string;
    techStack?: string[];
    proofLinks?: {
        github?: string;
        demo?: string;
        video?: string;
        screenshots?: string[];
    };
}

export type ProjectProofLinkKey = keyof NonNullable<Project['proofLinks']>;

export interface Language {
    id?: number;
    language: string;
    proficiency: 'Basic' | 'Conversational' | 'Fluent' | 'Native' | string;
}

export interface Education {
    id?: number;
    school: string;
    degree: string;
    field?: string;
    startYear: string;
}

```

```

    endYear?: string;
    cgpa?: string;
  }

export interface Experience {
  id?: number;
  company: string;
  position: string;
  startDate: string;
  endDate?: string;
  description?: string;
}

export interface Availability {
  openToCollaboration?: boolean;
  openToInternships?: boolean;
  openToFreelance?: boolean;
  justBuilding?: boolean;
}

export interface UserProfile {
  id: number;
  name: string;
  email: string;
  avatar?: string;
  role?: 'ROLE_USER' | 'ROLE_AUTHOR' | 'ROLE_ADMIN';

  // Newer profile fields
  username?: string;
  bio?: string;
  location?: string;
  website?: string;
  github?: string;
  linkedin?: string;
  twitter?: string;
  bannerImage?: string;
  resumeUrl?: string;
  followersCount?: number;
  followingCount?: number;

  // Public visibility preferences (used for shareable profile pages)
  showEmailOnProfile?: boolean;
  showResumeOnProfile?: boolean;

  // Relationship flags for viewing someone else's profile
  isFollowing?: boolean;
  canMessage?: boolean;

  skills: string[];
  projects: Project[];
  experience?: Experience[];
  languages?: Language[];
  education?: Education[];
  availability?: Availability;
}

export interface Announcement {
  id: number;
  title: string;
  message: string;
  visible?: boolean;
  videoUrl?: string;
  imageUrls?: string[];
  createdAt?: string;
}

```

■ frontend/src/lib/chatRealtime.ts

```
import { Client, type IMessage } from '@stomp/stompjs';
```

```

// Some builds of sockjs-client reference `global` (Node) even in the browser.
// Ensure it's defined before we dynamically import sockjs-client.
// eslint-disable-next-line @typescript-eslint/no-explicit-any
const _global = globalThis as any;
if (typeof _global.global === 'undefined') {
  _global.global = _global;
}

export type RealtimeChatMessage = {
  roomId: number;
  senderUsername: string;
  senderDisplayName: string;
  content: string;
  timestamp: string;
};

type SubscriptionHandle = { unsubscribe: () => void };

/**
 * Very small STOMP client wrapper for the chat page.
 *
 * Contract:
 * - Connects to backend WS endpoint: /ws/chat (SockJS)
 * - Publishes to: /app/chat.send
 * - Subscribes to: /topic/chat/{roomId}
 */
export class ChatRealtimeClient {
  private client: Client;
  private connected = false;
  private sockJsPath: string;

  constructor(sockJsPath: string = '/ws/chat') {
    this.sockJsPath = sockJsPath;
    // Defer SockJS creation until connect() so we don't crash on module import.
    this.client = new Client({
      debug: () => {
        // Keep silent in prod. Uncomment for WS debugging.
      },
      reconnectDelay: 2000,
    });

    // Avoid runtime errors if broker disconnects.
    this.client.onStompError = () => {
      this.connected = false;
    };
    this.client.onWebSocketClose = () => {
      this.connected = false;
    };
    this.client.onConnect = () => {
      this.connected = true;
    };
  }

  async connect(): Promise<void> {
    if (typeof window === 'undefined') {
      throw new Error('ChatRealtimeClient can only connect in the browser');
    }

    if (this.connected) return;
    if (this.client.active) return;

    // Dynamic import so sockjs-client is evaluated only when we actually connect.
    const mod = await import('sockjs-client');
    const SockJS = (mod as any).default ?? mod;
    const socket = new SockJS(this.sockJsPath);
    this.client.webSocketFactory = () => socket as any;

    await new Promise<void>((resolve, reject) => {
      const timeout = setTimeout(() => reject(new Error('WebSocket connect timeout')), 8000);
      this.client.onConnect = () => {
        clearTimeout(timeout);
      };
    });
  }
}

```

```

        this.connected = true;
        resolve();
    };
    this.client.onWebSocketError = () => {
        clearTimeout(timeout);
        this.connected = false;
        reject(new Error('WebSocket connection error'));
    };
    this.client.activate();
    });
}

disconnect(): void {
    try {
        this.client.deactivate();
    } finally {
        this.connected = false;
    }
}

subscribe(roomId: number, onMessage: (msg: RealtimeChatMessage) => void): SubscriptionHandle {
    const sub = this.client.subscribe(`/topic/chat/${roomId}`, (frame: IMessage) => {
        try {
            const parsed = JSON.parse(frame.body) as RealtimeChatMessage;
            onMessage(parsed);
        } catch {
            // ignore invalid messages
        }
    });
    return { unsubscribe: () => sub.unsubscribe() };
}

send(roomId: number, content: string): void {
    this.client.publish({
        destination: '/app/chat.send',
        body: JSON.stringify({ roomId, content }),
    });
}
}

```

■ frontend/src/lib/formatCount.ts

```

export function formatCount(value: number | null | undefined): string {
    const n = Number(value ?? 0);
    if (!Number.isFinite(n)) return '0';

    const abs = Math.abs(n);

    // Below 1000: show full number.
    if (abs < 1000) return String(Math.trunc(n));

    type Unit = { threshold: number; suffix: string };
    const units: Unit[] = [
        { threshold: 1_000_000_000, suffix: 'B' },
        { threshold: 1_000_000, suffix: 'M' },
        { threshold: 1_000, suffix: 'K' },
    ];

    const unit = units.find((u) => abs >= u.threshold) ?? units[units.length - 1];
    const scaled = n / unit.threshold;

    // Show one decimal only when needed and only for values < 10 (e.g., 1.2K, 2.5M).
    const useDecimal = Math.abs(scaled) < 10;
    const raw = useDecimal ? scaled.toFixed(1) : String(Math.round(scaled));

    // Remove trailing .0 (e.g., 1.0K -> 1K)
    const trimmed = raw.replace(/\.0$/, '');

    return `${trimmed}${unit.suffix}`;
}

```

■ frontend/src/lib/postRealtime.ts

```
import { Client, type IMessage } from '@stomp/stompjs';

// Ensure `global` exists for sockjs-client (some builds reference it).
// eslint-disable-next-line @typescript-eslint/no-explicit-any
const _global = globalThis as any;
if (typeof _global.global === 'undefined') {
  _global.global = _global;
}

export type RealtimePostStats = {
  postId: number;
  likes: number;
  commentCount: number;
  updatedAt?: string;
};

type SubscriptionHandle = { unsubscribe: () => void };

/**
 * Minimal STOMP client for realtime post stats.
 * - Connects to SockJS endpoint: /ws/posts
 * - Subscribes to: /topic/posts/{postId}
 */
export class PostRealtimeClient {
  private client: Client;
  private connected = false;
  private sockJsPath: string;

  constructor(sockJsPath: string = '/ws/posts') {
    this.sockJsPath = sockJsPath;
    this.client = new Client({
      reconnectDelay: 2000,
      debug: () => {
        // Keep silent by default.
      },
    });
  };

  this.client.onStompError = (frame) => {
    this.connected = false;
    // eslint-disable-next-line no-console
    console.warn('PostRealtime STOMP error', frame.headers?.message, frame.body);
  };
  this.client.onWebSocketClose = () => {
    this.connected = false;
    // eslint-disable-next-line no-console
    console.warn('PostRealtime WS closed');
  };
  this.client.onWebSocketError = () => {
    this.connected = false;
    // eslint-disable-next-line no-console
    console.warn('PostRealtime WS error');
  };
  this.client.onConnect = () => {
    this.connected = true;
    // eslint-disable-next-line no-console
    console.info('PostRealtime connected');
  };
}

async connect(): Promise<void> {
  if (typeof window === 'undefined') return;
  if (this.connected) return;
  if (this.client.active) return;

  const mod = await import('sockjs-client');
  const SockJS = (mod as any).default ?? mod;
  // SockJS options: ensure cookies (JSESSIONID) are sent.
```



```

const socket = new SockJS(this.sockJsPath, undefined, {
  transports: ['websocket', 'xhr-streaming', 'xhr-polling'],
  withCredentials: true,
} as any);

this.client.websocketFactory = () => socket as any;

await new Promise<void>((resolve, reject) => {
  const timeout = setTimeout(() => reject(new Error('WebSocket connect timeout')), 8000);
  this.client.onConnect = () => {
    clearTimeout(timeout);
    this.connected = true;
    resolve();
  };
  this.client.onWebSocketError = () => {
    clearTimeout(timeout);
    this.connected = false;
    reject(new Error('WebSocket connection error'));
  };
  this.client.activate();
});

disconnect(): void {
  try {
    this.client.deactivate();
  } finally {
    this.connected = false;
  }
}

subscribe(postId: number, onStats: (stats: RealtimePostStats) => void): SubscriptionHandle {
  const sub = this.client.subscribe(`/topic/posts/${postId}`, (frame: IMessage) => {
    try {
      const parsed = JSON.parse(frame.body) as RealtimePostStats;
      if (typeof parsed?.postId !== 'number') return;
      onStats(parsed);
    } catch {
      // ignore
    }
  });

  return { unsubscribe: () => sub.unsubscribe() };
}

```

■ frontend/src/lib/utils.ts

```

import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

```

■ frontend/src/main.tsx

```

import { Toaster } from "@components/ui/sonner";
import { AuthProvider } from "@contexts/AuthContext";
import { ThemeProvider } from "@contexts/ThemeContext";
import { StrictMode, useEffect, lazy, Suspense } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter, Route, Routes, useLocation } from "react-router-dom";
import { ProtectedRoute } from "@components/ProtectedRoute";
import { InstrumentationProvider } from "@instrumentation";
import "./index.css";
import "./types/global.d.ts";
// Vite + some CJS deps (e.g. sockjs-client) still expect a Node-like `global`.

```

```

// Define it early to prevent `ReferenceError: global is not defined` at runtime.
// eslint-disable-next-line @typescript-eslint/no-explicit-any
const _global = globalThis as any;
if (typeof _global.global === "undefined") {
  _global.global = _global;
}

// Lazy load route components for better code splitting
const Feed = lazy(() => import("./pages/Feed.tsx"));
const Login = lazy(() => import("./pages/Login.tsx"));
const Register = lazy(() => import("./pages/Register.tsx"));
const SetupUsername = lazy(() => import("./pages/SetupUsername.tsx"));
const Profile = lazy(() => import("./pages/Profile.tsx"));
const PostDetail = lazy(() => import("./pages/PostDetail.tsx"));
const PostForm = lazy(() => import("./pages/PostForm.tsx"));
const MyPosts = lazy(() => import("./pages/MyPosts.tsx"));
const Admin = lazy(() => import("./pages/Admin.tsx"));
const AdminDashboard = lazy(() => import("./pages/AdminDashboard.tsx"));
const AdminUsers = lazy(() => import("./pages/AdminUsers.tsx"));
const AdminUserPosts = lazy(() => import("./pages/AdminUserPosts.tsx"));
const AdminAnnouncements = lazy(() => import("./pages/AdminAnnouncements.tsx"));
const Announcements = lazy(() => import("./pages/Announcements.tsx"));
const AnnouncementCreate = lazy(() => import("./pages/AnnouncementCreate.tsx"));
const AnnouncementEdit = lazy(() => import("./pages/AnnouncementEdit.tsx"));
const Chat = lazy(() => import("./pages/Chat.tsx"));
const ChatDashboard = lazy(() => import("./pages/ChatDashboard.tsx"));
const ChangeUsername = lazy(() => import("./pages/ChangeUsername.tsx"));
const ChangeUsernameError = lazy(() => import("./pages/ChangeUsernameError.tsx"));
const UserSearch = lazy(() => import("./pages/UserSearch.tsx"));
const Help = lazy(() => import("./pages/Help.tsx"));
const Terms = lazy(() => import("./pages/Terms.tsx"));
const Privacy = lazy(() => import("./pages/Privacy.tsx"));
const ErrorPage = lazy(() => import("./pages/ErrorPage.tsx"));
const NotFound = lazy(() => import("./pages/NotFound.tsx"));

// Simple loading fallback for route transitions
function RouteLoading() {
  return (
    <div className="min-h-screen flex items-center justify-center">
      <div className="animate-pulse text-muted-foreground">Loading...</div>
    </div>
  );
}

function RouteSyncer() {
  const location = useLocation();
  useEffect(() => {
    window.parent.postMessage(
      { type: "iframe-route-change", path: location.pathname },
      "*"
    );
  }, [location.pathname]);

  useEffect(() => {
    function handleMessage(event: MessageEvent) {
      if (event.data?.type === "navigate") {
        if (event.data.direction === "back") window.history.back();
        if (event.data.direction === "forward") window.history.forward();
      }
    }
    window.addEventListener("message", handleMessage);
    return () => window.removeEventListener("message", handleMessage);
  }, []);

  return null;
}

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <InstrumentationProvider>

```

```

<ThemeProvider>
<AuthProvider>
  { /* Session-based auth: the backend sets/validates the JSESSIONID cookie; the frontend just sends cookies (cr
<BrowserRouter>
  <RouteSyncer />
  <Suspense fallback={<RouteLoading />}>
    <Routes>
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />
      <Route path="/setup-username" element={<SetupUsername />} />
      <Route path="/" element={<Feed />} />
      <Route path="/post/:id" element={<PostDetail />} />
      { /* Shareable public profile route */
      <Route path="/u/:username" element={<Profile />} />
      <Route path="/profile/:userId" element={<Profile />} />
      <Route path="/announcements" element={<Announcements />} />
      <Route path="/users/search" element={<UserSearch />} />
      <Route path="/help" element={<Help />} />
      <Route path="/terms" element={<Terms />} />
      <Route path="/privacy" element={<Privacy />} />
      <Route
        path="/profile"
        element={
          <ProtectedRoute>
            <Profile />
          </ProtectedRoute>
        }
      />
      <Route
        path="/change-username"
        element={
          <ProtectedRoute>
            <ChangeUsername />
          </ProtectedRoute>
        }
      />
      <Route
        path="/change-username-error"
        element={
          <ProtectedRoute>
            <ChangeUsernameError />
          </ProtectedRoute>
        }
      />
      <Route
        path="/chat"
        element={
          <ProtectedRoute>
            <Chat />
          </ProtectedRoute>
        }
      />
      <Route
        path="/chat/:userId"
        element={
          <ProtectedRoute>
            <Chat />
          </ProtectedRoute>
        }
      />
      <Route
        path="/chats"
        element={
          <ProtectedRoute>
            <ChatDashboard />
          </ProtectedRoute>
        }
      />
      <Route
        path="/posts/new"
        element={

```

```

        <ProtectedRoute>
            <PostForm />
        </ProtectedRoute>
    }
/>
<Route
    path="/posts/edit/:id"
    element={
        <ProtectedRoute>
            <PostForm />
        </ProtectedRoute>
    }
/>
<Route
    path="/announcements/create"
    element={
        <ProtectedRoute>
            <AnnouncementCreate />
        </ProtectedRoute>
    }
/>
<Route
    path="/myposts"
    element={
        <ProtectedRoute>
            <MyPosts />
        </ProtectedRoute>
    }
/>
<Route
    path="/admin"
    element={
        <ProtectedRoute>
            <AdminDashboard />
        </ProtectedRoute>
    }
/>
<Route
    path="/admin/posts"
    element={
        <ProtectedRoute>
            <Admin />
        </ProtectedRoute>
    }
/>
<Route
    path="/admin/users"
    element={
        <ProtectedRoute>
            <AdminUsers />
        </ProtectedRoute>
    }
/>
<Route
    path="/admin/users/:userId/posts"
    element={
        <ProtectedRoute>
            <AdminUserPosts />
        </ProtectedRoute>
    }
/>
<Route
    path="/admin/announcements"
    element={
        <ProtectedRoute>
            <AdminAnnouncements />
        </ProtectedRoute>
    }
/>
<Route
    path="/admin/announcements/edit/:id"

```

```

        element={
          <ProtectedRoute>
            <AnnouncementEdit />
          </ProtectedRoute>
        }
      />
      <Route path="/error" element={<ErrorPage />} />
      <Route path="*" element={<NotFound />} />
    </Routes>
    </Suspense>
    </BrowserRouter>
    <Toaster />
    </AuthProvider>
    </ThemeProvider>
    </InstrumentationProvider>
    </StrictMode>,
  );

```

■ frontend/src/pages/Admin.tsx

```

import { useEffect, useState } from 'react';
import { Navigation } from '@components/Navigation';
import { postsApi, Post, adminStatsApi } from '@lib/api';
import { useAuthContext } from '@contexts/AuthContext';
import { PostCard } from '@components/PostCard';
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import { Loader2, Shield, Users, FileText, TrendingUp } from 'lucide-react';
import { useNavigate } from 'react-router-dom';

export default function Admin() {
  const { user } = useAuthContext();
  const navigate = useNavigate();
  const [posts, setPosts] = useState<Post[]>([]);
  const [isLoading, setIsLoading] = useState(true);
  const [isPageLoading, setIsPageLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [page, setPage] = useState(0);
  const [size] = useState(10);
  const [totalPages, setTotalPages] = useState(1);
  const [totalElements, setTotalElements] = useState(0);

  const [stats, setStats] = useState<{ activeUsers: number; totalLikes: number; totalComments: number } | null>(null);

  useEffect(() => {
    // Check if user is admin
    if (user && user.role !== 'ROLE_ADMIN') {
      navigate('/');
    }
  }, [user, navigate]);

  const fetchPostsPage = async (nextPage: number, opts: { initial?: boolean } = {}) => {
    try {
      if (opts.initial) setIsLoading(true);
      else setIsPageLoading(true);

      const { items, pageInfo } = await postsApi.fetchAdminPaged({ page: nextPage, size });
      setPosts(items);
      setPage(pageInfo?.pageNumber ?? nextPage);
      setTotalPages(pageInfo?.totalPages ?? 1);
      setTotalElements(pageInfo?.totalElements ?? items.length);
      setError(null);
    } catch (err) {
      console.error('Failed to fetch posts:', err);
      setError('Failed to load posts. Please try again later.');
```

```

useEffect(() => {
  if (user?.role === 'ROLE_ADMIN') {
    fetchPostsPage(0, { initial: true });
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [user]);

// Fetch accurate admin aggregates (users/likes/comments)
useEffect(() => {
  let cancelled = false;

  const loadStats = async () => {
    if (user?.role !== 'ROLE_ADMIN') return;
    try {
      const s = await adminStatsApi.postStats();
      if (!cancelled) setStats(s);
    } catch (err) {
      console.warn('Failed to load admin stats:', err);
      if (!cancelled) setStats(null);
    }
  };

  void loadStats();
  return () => {
    cancelled = true;
  };
}, [user?.role]);

const handlePostUpdated = (updatedPost: Post) => {
  setPosts(posts.map((post) => (post.id === updatedPost.id ? updatedPost : post)));
};

const handlePostDeleted = (postId: number) => {
  // If you delete the last item on a page, refresh to keep pagination consistent.
  const next = posts.filter((post) => post.id !== postId);
  setPosts(next);
  const isEmptyPage = next.length === 0 && page > 0;
  void fetchPostsPage(isEmptyPage ? page - 1 : page);
};

if (user?.role !== 'ROLE_ADMIN') {
  return null;
}

const totalLikes = stats?.totalLikes ?? posts.reduce((sum, post) => sum + post.likes, 0);
const totalComments =
  stats?.totalComments ??
  posts.reduce((sum, post) => {
    const anyPost = post as unknown as { commentCount?: number };
    return sum + (anyPost.commentCount ?? post.comments?.length ?? 0);
  }, 0);
const activeUsers =
  stats?.activeUsers ??
  new Set(posts.map((p) => p.author.id)).size;

return (
  <div className="min-h-screen">
    <Navigation />

    <main className="container mx-auto px-4 py-8 max-w-6xl">
      <div className="flex items-center gap-3 mb-8">
        <Shield className="h-8 w-8 text-primary" />
        <h1 className="text-3xl font-bold">Admin Dashboard</h1>
      </div>

      {/* Stats Cards */}
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4 mb-8">
        <Card className="gradient-mesh-card">
          <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
            <CardTitle className="text-sm font-medium">Total Posts</CardTitle>
            <FileText className="h-4 w-4 text-muted-foreground" />

```

```

    </CardHeader>
    <CardContent>
      <div className="text-2xl font-bold">{totalElements}</div>
    </CardContent>
  </Card>

  <Card className="gradient-mesh-card">
    <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
      <CardTitle className="text-sm font-medium">Active Users</CardTitle>
      <Users className="h-4 w-4 text-muted-foreground" />
    </CardHeader>
    <CardContent>
      <div className="text-2xl font-bold">{activeUsers}</div>
    </CardContent>
  </Card>

  <Card className="gradient-mesh-card">
    <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
      <CardTitle className="text-sm font-medium">Total Likes</CardTitle>
      <TrendingUp className="h-4 w-4 text-muted-foreground" />
    </CardHeader>
    <CardContent>
      <div className="text-2xl font-bold">{totalLikes}</div>
    </CardContent>
  </Card>

  <Card className="gradient-mesh-card">
    <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
      <CardTitle className="text-sm font-medium">Total Comments</CardTitle>
      <FileText className="h-4 w-4 text-muted-foreground" />
    </CardHeader>
    <CardContent>
      <div className="text-2xl font-bold">{totalComments}</div>
    </CardContent>
  </Card>
</div>

{/* All Posts */}
<div>
  <div className="flex items-center justify-between gap-4 mb-6">
    <h2 className="text-2xl font-bold">All Posts</h2>
    <div className="flex items-center gap-2 text-sm text-muted-foreground">
      {isLoading & & (
        <span className="inline-flex items-center gap-2">
          <Loader2 className="h-4 w-4 animate-spin" />
          Loading page...
        </span>
      )}
    </div>
  </div>
  <div>
    Page {page + 1} of {Math.max(totalPages, 1)}
  </div>
</div>

<div className="flex items-center justify-between gap-2 mb-6">
  <button
    className="inline-flex items-center justify-center rounded-md text-sm font-medium transition-colors focus
    disabled={isLoading || isLoading || page <= 0}
    onClick={() => fetchPostsPage(page - 1)}
  >
    Previous
  </button>
  <button
    className="inline-flex items-center justify-center rounded-md text-sm font-medium transition-colors focus
    disabled={isLoading || isLoading || page >= totalPages - 1}
    onClick={() => fetchPostsPage(page + 1)}
  >
    Next
  </button>
</div>
<div className="space-y-6">

```

```

    {isLoading ? (
      <div className="flex justify-center items-center py-12">
        <Loader2 className="h-8 w-8 animate-spin text-primary" />
      </div>
    ) : error ? (
      <div className="text-center py-12">
        <p className="text-destructive">{error}</p>
      </div>
    ) : posts.length === 0 ? (
      <div className="text-center py-12">
        <p className="text-muted-foreground">No posts available.</p>
      </div>
    ) : (
      posts.map((post) => (
        <PostCard
          key={post.id}
          post={post}
          onPostUpdated={handlePostUpdated}
          onPostDeleted={handlePostDeleted}
        />
      ))
    )
  </div>
</div>
</main>
</div>
);
}

```

■ frontend/src/pages/AdminAnnouncements.tsx

```

import { useState, useEffect } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card, CardContent } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Badge } from '@components/ui/badge';
import { Alert, AlertDescription } from '@components/ui/alert';
import { Bell, Plus, Eye, EyeOff, Edit, Trash2, Video, ArrowLeft, Info, Loader2 } from 'lucide-react';
import { useAuthContext } from '@contexts/AuthContext';
import { format } from 'date-fns';
import { toast } from 'sonner';
import { announcementsApi, ApiError, type Announcement } from '@lib/api';

export default function AdminAnnouncements() {
  const navigate = useNavigate();
  const { user } = useAuthContext();

  const [announcements, setAnnouncements] = useState<Announcement[]>([]);
  const [isLoading, setIsLoading] = useState(false);
  const [isPageLoading, setIsPageLoading] = useState(false);

  const [page, setPage] = useState(0);
  const [size] = useState(10);
  const [totalPages, setTotalPages] = useState(1);
  const [totalElements, setTotalElements] = useState(0);

  const load = async (nextPage: number, opts: { initial?: boolean } = {}) => {
    if (opts.initial) setIsLoading(true);
    else setIsPageLoading(true);

    try {
      const { items, pageInfo } = await announcementsApi.listAdminPaged({ page: nextPage, size });
      setAnnouncements(items);
      setPage(pageInfo?.pageNumber ?? nextPage);
      setTotalPages(pageInfo?.totalPages ?? 1);
      setTotalElements(pageInfo?.totalElements ?? items.length);
    } catch (e) {
      const msg = e instanceof ApiError ? e.message : 'Failed to load announcements';
      toast.error(msg);
    }
  };

```



```

    } finally {
      setIsLoading(false);
      setIsPageLoading(false);
    }
  };

  useEffect(() => {
    if (user?.role !== 'ROLE_ADMIN') {
      navigate('/');
      return;
    }

    void load(0, { initial: true });
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [user, navigate]);

  const handleToggleVisible = async (id: number) => {
    try {
      await announcementsApi.toggleVisible(id);
      await load(page, { initial: false });
      toast.success('Visibility updated');
    } catch (e) {
      const msg = e instanceof ApiError ? e.message : 'Failed to update visibility';
      toast.error(msg);
    }
  };

  const handleDeleteVideo = async (id: number) => {
    if (!confirm('Delete this video?')) return;
    try {
      await announcementsApi.deleteVideo(id);
      await load(page, { initial: false });
      toast.success('Video deleted');
    } catch (e) {
      const msg = e instanceof ApiError ? e.message : 'Failed to delete video';
      toast.error(msg);
    }
  };

  const handleDelete = async (id: number) => {
    if (!confirm('Are you sure you want to delete this announcement?')) return;
    try {
      await announcementsApi.delete(id);
      // If we deleted the last row on the page, go back one page.
      const next = announcements.filter((a) => a.id !== id);
      const isEmptyPage = next.length === 0 && page > 0;
      await load(isEmptyPage ? page - 1 : page, { initial: false });
      toast.success('Announcement deleted');
    } catch (e) {
      const msg = e instanceof ApiError ? e.message : 'Failed to delete announcement';
      toast.error(msg);
    }
  };

  if (user?.role !== 'ROLE_ADMIN') {
    return null;
  }

  return (
    <div className="min-h-screen bg-background">
      <Navigation />

      <main className="container mx-auto px-4 py-6 max-w-6xl">
        </* Header */>
        <div className="flex justify-between items-center mb-6">
          <h1 className="text-3xl font-bold flex items-center gap-2">
            <Bell className="h-8 w-8 text-primary" />
            Manage Announcements
          </h1>
          <div className="flex gap-2">
            <Button asChild>

```

```

        <Link to="/announcements/create">
        <Plus className="h-4 w-4 mr-2" />
        Create Announcement
        </Link>
    </Button>
</div>
</div>

<Button variant="outline" asChild className="mb-4">
    <Link to="/admin">
        <ArrowLeft className="h-4 w-4 mr-2" />
        Back to Admin Dashboard
    </Link>
</Button>

{isLoading ? (
    <div className="flex justify-center items-center py-12">
        <Loader2 className="h-8 w-8 animate-spin text-primary" />
    </div>
) : (
    <>
        <div className="flex items-center justify-between gap-4 mb-4">
            <div className="text-sm text-muted-foreground">
                {isLoading && (
                    <span className="inline-flex items-center gap-2">
                        <Loader2 className="h-4 w-4 animate-spin" />
                        Loading page...
                    </span>
                )}
            </div>
            <div className="flex items-center gap-2 text-sm text-muted-foreground">
                <span>
                    Page {page + 1} of {Math.max(totalPages, 1)}
                </span>
                <span>({totalElements} total)</span>
            </div>
        </div>

        <div className="flex items-center justify-between gap-2 mb-4">
            <Button
                variant="outline"
                disabled={isLoading || page <= 0}
                onClick={() => void load(page - 1)}
            >
                Previous
            </Button>
            <Button
                variant="outline"
                disabled={isLoading || page >= totalPages - 1}
                onClick={() => void load(page + 1)}
            >
                Next
            </Button>
        </div>

        {/* Empty State */}
        {announcements.length === 0 && (
            <Alert>
                <Info className="h-4 w-4" />
                <AlertDescription>
                    No announcements yet. Click "Create Announcement" to add one.
                </AlertDescription>
            </Alert>
        )}

        {/* Announcements Table */}
        {announcements.length > 0 && (
            <Card className="gradient-mesh-card">
                <CardContent className="p-0">
                    <div className="overflow-x-auto">
                        <table className="w-full">

```

```

<thead className="bg-muted/50 border-b">
  <tr>
    <th className="px-4 py-3 text-left font-semibold">Title</th>
    <th className="px-4 py-3 text-left font-semibold">Message</th>
    <th className="px-4 py-3 text-left font-semibold">Date</th>
    <th className="px-4 py-3 text-center font-semibold">Video</th>
    <th className="px-4 py-3 text-center font-semibold">Visible</th>
    <th className="px-4 py-3 text-center font-semibold">Actions</th>
  </tr>
</thead>
<tbody>
  {announcements.map((announcement) => (
    <tr key={announcement.id} className="border-b last:border-b-0 hover:bg-muted/30">
      <td className="px-4 py-3">
        <strong>{announcement.title}</strong>
      </td>

      <td className="px-4 py-3 max-w-xs">
        <div className="truncate text-muted-foreground text-sm">
          {announcement.message}
        </div>
      </td>

      <td className="px-4 py-3">
        <Badge variant="secondary">
          {format(new Date(announcement.createdAt ?? ''), 'dd-MMM-yyyy HH:mm')}
        </Badge>
      </td>

      <td className="px-4 py-3 text-center">
        {announcement.videoUrl ? (
          <div className="flex flex-col items-center gap-1">
            <Badge variant="default">
              <Video className="h-3 w-3 mr-1" />
              Video
            </Badge>
            <div className="flex gap-1">
              <Button
                variant="outline"
                size="sm"
                asChild
                className="h-6 px-2 text-xs"
              >
                <a href={announcement.videoUrl} target="_blank" rel="noopener noreferrer">
                  <Eye className="h-3 w-3" />
                </a>
              </Button>
              <Button
                variant="outline"
                size="sm"
                className="h-6 px-2 text-xs text-destructive hover:text-destructive"
                onClick={() => handleDeleteVideo(announcement.id)}
              >
                <Trash2 className="h-3 w-3" />
              </Button>
            </div>
          </div>
        ) : (
          <span className="text-muted-foreground text-sm">-</span>
        )}
      </td>

      <td className="px-4 py-3 text-center">
        <Button
          variant={announcement.visible ? 'default' : 'outline'}
          size="sm"
        >

```

```

        onClick={() => handleToggleVisible(announcement.id)}
      </>
      {announcement.visible ? (
        <Eye className="h-4 w-4" />
      ) : (
        <EyeOff className="h-4 w-4" />
      )}
    </Button>
  </td>
</tr>

<tr>
  <td>
    <div className="flex gap-1 justify-center">
      <Button variant="default" size="sm" asChild>
        <Link to={`/admin/announcements/edit/${announcement.id}`}>
          <Edit className="h-4 w-4 mr-1" />
          Edit
        </Link>
      </Button>
      <Button
        variant="destructive"
        size="sm"
        onClick={() => handleDelete(announcement.id)}
      >
        <Trash2 className="h-4 w-4 mr-1" />
        Delete
      </Button>
    </div>
  </td>
</tr>
</tbody>
</table>
</div>
</CardContent>
</Card>
  )}
</>
)}
</main>
</div>
);
}

```

■ frontend/src/pages/AdminDashboard.tsx

```

import { useEffect } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card, CardContent } from '@components/ui/card';
import { Shield, Users, Newspaper, Bell, ArrowRight, Home } from 'lucide-react';
import { useAuthContext } from '@contexts/AuthContext';

export default function AdminDashboard() {
  const navigate = useNavigate();
  const { user } = useAuthContext();

  useEffect(() => {if (user?.role !== 'ROLE_ADMIN') {
    navigate('/');
  }
}, [user, navigate]);

if (user?.role !== 'ROLE_ADMIN') {
  return null;
}

const dashboardCards = [
  {
    title: 'User Management',
    description: 'View, search, and manage all registered users. Monitor user activity and access user profiles.',
  },

```

```

        icon: Users,
        link: '/admin/users',
    },
    {
        title: 'Post Management',
        description: 'Create, edit, and moderate posts. Review all content published on the platform.',
        icon: Newspaper,
        link: '/admin/posts',
    },
    {
        title: 'Manage Announcements',
        description: 'Create and publish announcements to notify all users about important updates and news.',
        icon: Bell,
        link: '/admin/announcements',
    },
];

return (
    <div className="min-h-screen bg-background">
        <Navigation />

        <main className="container mx-auto px-4 py-12 max-w-6xl">
            { /* Header */ }
            <div className="text-center mb-12">
                <h1 className="text-5xl font-bold mb-4 flex items-center justify-center gap-3">
                    <Shield className="h-12 w-12 text-primary" />
                    Admin Dashboard
                </h1>
                <p className="text-xl text-muted-foreground">
                    Manage your platform from one central location
                </p>
            </div>

            { /* Dashboard Cards */ }
            <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8 mb-8">
                {dashboardCards.map((card) => (
                    <Link key={card.link} to={card.link} className="group">
                        <Card className="gradient-mesh-card h-full transition-all duration-300 hover:scale-105 hover:shadow-2xl">
                            <CardContent className="pt-6 text-center">
                                { /* Icon Container */ }
                                <div className="w-20 h-20 mx-auto mb-6 rounded-2xl bg-muted flex items-center justify-center transition">
                                    <card.icon className="h-10 w-10" />
                                </div>

                                { /* Title */ }
                                <h3 className="text-2xl font-bold mb-3">
                                    {card.title}
                                </h3>

                                { /* Description */ }
                                <p className="text-muted-foreground mb-6 min-h-[4rem]">
                                    {card.description}
                                </p>

                                { /* Button */ }
                                <div className="inline-flex items-center gap-2 px-6 py-3 rounded-xl bg-muted font-semibold transition">
                                    <span>Manage</span>
                                    <ArrowRight className="h-5 w-5" />
                                </div>
                            </CardContent>
                        </Card>
                    </Link>
                ))}
            </div>

            { /* Back to Home */ }
            <div className="text-center">
                <Link
                    to="/"
                    className="inline-flex items-center gap-2 text-lg hover:opacity-80 transition-opacity">
                    >

```

```

        <Home className="h-5 w-5" />
        <span>Back to Home</span>
      </Link>
    </div>
  </main>
</div>
);
}

```

■ frontend/src/pages/AdminUserPosts.tsx

```

import { useState, useEffect } from 'react';
import { useNavigate, useParams, Link } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card, CardContent } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar';
import { ArrowLeft, Newspaper, Calendar, Trash2, Loader2 } from 'lucide-react';
import { useAuthContext } from '@contexts/AuthContext';
import { format } from 'date-fns';
import { toast } from 'sonner';
import { postsApi, usersApi, ApiError, isUnauthenticatedStatus, UNAUTHENTICATED_MESSAGE, type Post } from '@lib/api';
import { DefaultAvatarFallback } from '@components/DefaultAvatarFallback';

interface UserPost {
  id: number;
  title: string;
  content: string;
  createdAt: string;
}

interface UserProfile {
  id: number;
  username: string;
  displayName?: string;
  email: string;
  profileImageUrl?: string;
}

export default function AdminUserPosts() {
  const navigate = useNavigate();
  const { userId } = useParams<{ userId: string }>();
  const { user } = useAuthContext();

  const [userProfile, setUserProfile] = useState<UserProfile | null>(null);
  const [posts, setPosts] = useState<UserPost[]>([]);
  const [isLoading, setIsLoading] = useState(true);

  const [totalPosts, setTotalPosts] = useState(0);

  useEffect(() => {
    if (user?.role !== 'ROLE_ADMIN') {
      navigate('/');
      return;
    }

    if (!userId) {
      navigate('/admin/users');
      return;
    }
  }, [userId, user, navigate]);

  useEffect(() => {
    const load = async () => {
      if (user?.role !== 'ROLE_ADMIN') return;
      if (!userId) return;

      try {
        setIsLoading(true);

```

```

// Fetch user info via search endpoint (best available in current API surface)
const search = await usersApi.search('');
const match = (search.users || []).find((u) => String(u.id) === String(userId));
if (match) {
  setUserProfile({
    id: match.id,
    username: match.username || match.email || String(match.id),
    displayName: match.name,
    email: match.email,
    profileImageUrl: match.avatar,
  });
} else {
  // Fallback to minimal profile
  setUserProfile({
    id: Number(userId),
    username: String(userId),
    displayName: 'User',
    email: '',
  });
}

// Fetch posts and filter by author
const allPosts = await postsApi.getAll();
const userPosts = allPosts.filter((p: Post) => String(p.author?.id) === String(userId));

setPosts(
  userPosts.map((p) => ({
    id: p.id,
    title: (p.title & p.title.trim()) ? p.title : `Post #${p.id}`,
    content: p.content,
    createdAt: p.createdAt,
  }))
);
setTotalPosts(userPosts.length);
} catch (error) {
  console.error('Failed to load admin user posts:', error);
  if (error instanceof ApiError) {
    const msg = isUnauthenticatedStatus(error.status)
      ? UNAUTHENTICATED_MESSAGE
      : (error.message || 'Failed to load posts');
    toast.error(msg);
  } else {
    toast.error('Failed to load posts');
  }
} finally {
  setIsLoading(false);
}
};

load();
}, [user?.role, userId]);

const handleDeletePost = async (postId: number) => {
  if (!confirm('Are you sure you want to delete this post? This action cannot be undone.')) {
    return;
  }

  try {
    await postsApi.delete(postId);
    setPosts((prev) => prev.filter((p) => p.id !== postId));
    setTotalPosts((prev) => Math.max(0, prev - 1));
    toast.success('Post deleted successfully');
  } catch (error) {
    console.error('Failed to delete post:', error);
    toast.error('Failed to delete post');
  }
};

const stripHtml = (html: string): string => {
  const tmp = document.createElement('DIV');
  tmp.innerHTML = html;

```

```

    return tmp.textContent || tmp.innerText || '';
};

if (user?.role !== 'ROLE_ADMIN') {
    return null;
}

return (
    <div className="min-h-screen bg-background">
        <Navigation />

        <main className="container mx-auto px-4 py-6 max-w-7xl">
            { /* Header */ }
            <div className="flex justify-between items-center mb-6">
                <h1 className="text-3xl font-bold flex items-center gap-2">
                    <Newspaper className="h-8 w-8 text-primary" />
                    User Posts Management
                </h1>
                <Button variant="outline" asChild>
                    <Link to="/admin/users">
                        <ArrowLeft className="h-4 w-4 mr-2" />
                        Back to Users
                    </Link>
                </Button>
            </div>

            {isLoading ? (
                <div className="flex justify-center items-center py-12">
                    <Loader2 className="h-8 w-8 animate-spin text-primary" />
                </div>
            ) : !userProfile ? (
                <div className="text-center py-12">
                    <p className="text-muted-foreground">User not found.</p>
                </div>
            ) : (
                <>
                    { /* User Info Card */ }
                    <Card className="gradient-mesh-card mb-6">
                        <CardContent className="py-6">
                            <div className="flex items-center justify-between">
                                <div className="flex items-center gap-4">
                                    <Avatar className="h-20 w-20 border-2 border-border">
                                        <AvatarImage src={userProfile.profileImageUrl} />
                                        <AvatarFallback>
                                            <DefaultAvatarFallback alt={userProfile.displayName || userProfile.username} />
                                        </AvatarFallback>
                                    </Avatar>
                                </div>
                                <h2 className="text-2xl font-bold mb-1">
                                    {userProfile.displayName || userProfile.username}
                                </h2>
                                {userProfile.email & & (
                                    <p className="text-muted-foreground mb-1">
                                        <span className="inline-block mr-4">
                                            ✉ {userProfile.email}
                                        </span>
                                    </p>
                                )}
                                <p className="text-muted-foreground">@{userProfile.username}</p>
                            </div>
                        </CardContent>
                    </Card>

                    { /* Posts Table */ }
                </>
            )}
        </main>
    </div>

```



```

<Card className="gradient-mesh-card">
  <CardContent className="p-0">
    <div className="p-6 border-b">
      <h5 className="text-lg font-semibold flex items-center gap-2">
        <Newspaper className="h-5 w-5" />
        Posts by {userProfile.displayName || userProfile.username}
      </h5>
    </div>
    {posts.length === 0 ? (
      <div className="text-center py-12">
        <Newspaper className="h-16 w-16 mx-auto mb-4 text-muted-foreground opacity-50" />
        <h5 className="text-lg font-semibold mb-2">No posts found</h5>
        <p className="text-muted-foreground">This user hasn't created any posts yet.
      </p>
    </div>
    ) : (
      <div className="overflow-x-auto">
        <table className="w-full">
          <thead className="bg-muted/50 border-b">
            <tr>
              <th className="px-4 py-3 text-left font-semibold" style={{ width: '30%' }}>
                Title
              </th>
              <th className="px-4 py-3 text-left font-semibold" style={{ width: '35%' }}>
                Preview
              </th>
              <th className="px-4 py-3 text-left font-semibold" style={{ width: '20%' }}>
                Created At
              </th>
              <th className="px-4 py-3 text-left font-semibold" style={{ width: '15%' }}>
                Actions
              </th>
            </tr>
          </thead>
          <tbody>
            {posts.map((post) => (
              <tr key={post.id} className="border-b last:border-b-0 hover:bg-muted/30">
                <td className="px-4 py-3">
                  <a
                    href={` /post/${post.id} `}
                    target="_blank"
                    rel="noopener noreferrer"
                    className="text-primary hover:underline font-semibold"
                  >
                    {post.title}
                  </a>
                </td>
                <td className="px-4 py-3">
                  <div className="text-sm text-muted-foreground truncate max-w-md">
                    {stripHtml(post.content).substring(0, 50)}...
                  </div>
                </td>
                <td className="px-4 py-3">
                  <div className="text-sm">
                    <Calendar className="h-4 w-4 inline mr-1" />
                    {format(new Date(post.createdAt), 'dd MMM yyyy')}
                    <br />
                    <span className="text-muted-foreground">
                      {format(new Date(post.createdAt), 'HH:mm')}
                    </span>
                  </div>
                </td>
                <td className="px-4 py-3">
                  <div>
                </div>
              </tr>
            )
          </tbody>
        </table>
      </div>
    )
  </CardContent>
</Card>

```

```

        <td className="px-4 py-3">
          <Button
            variant="destructive"
            size="sm"
            onClick={() => handleDeletePost(post.id)}
          />
          <Trash2 className="h-4 w-4 mr-1" />
          Delete
        </td>
      </tr>
    </tbody>
  </table>
</div>
)}
</CardContent>
</Card>
</>
)}
</main>
</div>
);
}

```

■ frontend/src/pages/AdminUsers.tsx

```

import { useState, useEffect } from 'react';
import { useNavigate, useSearchParams, Link } from 'react-router-dom';
import { Navigation } from '@/components/Navigation';
import { Card, CardContent } from '@/components/ui/card';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Avatar, AvatarFallback, AvatarImage } from '@/components/ui/avatar';
import { Badge } from '@/components/ui/badge';
import { Users, Search, Newspaper, Eye, Home } from 'lucide-react';
import { useAuthContext } from '@/contexts/AuthContext';
import { adminUsersApi } from '@/lib/api';
import { toast } from 'sonner';
import {
  Pagination,
  PaginationContent,
  PaginationItem,
  PaginationLink,
  PaginationNext,
  PaginationPrevious,
} from '@/components/ui/pagination';
import { DefaultAvatarFallback } from '@/components/DefaultAvatarFallback';

interface AdminUser {
  id: number;
  username: string;
  displayName?: string;
  email: string;
  profileImageUrl?: string;
  role?: string;
  admin: boolean;
  banned?: boolean;
}

export default function AdminUsers() {
  const navigate = useNavigate();
  const { user } = useAuthContext();
  const [searchParams, setSearchParams] = useSearchParams();

  const keyword = searchParams.get('keyword') || '';
  const pageFromUrl = Number(searchParams.get('page') || '0');
  const safePageFromUrl = Number.isFinite(pageFromUrl) && pageFromUrl >= 0 ? pageFromUrl : 0;

  const [searchInput, setSearchInput] = useState(keyword);

```

```

// URL-driven pagination
const [page, setPage] = useState(safePageFromUrl);
const [size] = useState(10);

const [users, setUsers] = useState<AdminUser[]>([]);
const [isLoading, setIsLoading] = useState(false);
const [loadError, setLoadError] = useState<string | null>(null);

const [totalUsers, setTotalUsers] = useState(0);
const [totalPages, setTotalPages] = useState(0);

useEffect(() => {
  if (user?.role !== 'ROLE_ADMIN') {
    navigate('/');
    return;
  }
}, [user, navigate]);

useEffect(() => {
  if (user?.role !== 'ROLE_ADMIN') return;

  const run = async () => {
    try {
      setIsLoading(true);
      setLoadError(null);

      const resp = await adminUsersApi.list({ keyword, page, size });
      const mapped: AdminUser[] = (resp.users || []).map((u: any) => ({
        id: Number(u.id),
        username: String(u.username || ''),
        displayName: (u.displayName as string | undefined) || undefined,
        email: String(u.email || ''),
        profileImageUrl: (u.profileImageUrl as string | undefined) || undefined,
        role: (u.role as string | undefined) || undefined,
        admin: Boolean(u.admin),
        banned: Boolean(u.banned),
      }));

      setUsers(mapped);
      setTotalUsers(Number(resp.totalUsers ?? 0));
      setTotalPages(Number(resp.totalPages ?? 0));

      // If we requested a page beyond the server range (can happen after search),
      // redirect to last page by updating URL (single source of truth).
      const tp = Number(resp.totalPages ?? 0);
      if (tp > 0 && page >= tp) {
        updateQuery({ page: Math.max(0, tp - 1) });
        return;
      }
    } catch (e) {
      console.error('Failed to load users:', e);
      setUsers([]);
      setTotalUsers(0);
      setTotalPages(0);
      setLoadError('Failed to load users. Please try again.');
```

toast.error('Failed to load users');

```

    } finally {
      setIsLoading(false);
    }
  }
};

run();
}, [user?.role, keyword, page, size]);

useEffect(() => {
  // Keep local page in sync with URL changes (back/forward/refresh)
  setPage(safePageFromUrl);
}, [safePageFromUrl]);

// Keep the search box synced with URL changes (back/forward, links)
useEffect(() => {

```

```

    setSearchInput(keyword);
  }, [keyword]));

const updateQuery = (next: { keyword?: string; page?: number }) => {
  const nextParams: Record<string, string> = {};
  const k = (next.keyword ?? keyword).trim();
  if (k) nextParams.keyword = k;

  const p = next.page ?? page;
  if (p > 0) nextParams.page = String(p);

  setSearchParams(nextParams);
};

const buildHref = (next: { keyword?: string; page?: number }) => {
  const q = new URLSearchParams();
  const k = (next.keyword ?? keyword).trim();
  if (k) q.set('keyword', k);
  const p = next.page ?? page;
  if (p > 0) q.set('page', String(p));
  const s = q.toString();
  return s ? `?${s}` : '';
};

const handleSearch = (e: React.FormEvent) => {
  e.preventDefault();
  const k = searchInput.trim();
  // Always go to first page when searching
  updateQuery({ keyword: k, page: 0 });
};

const clearSearch = () => {
  setSearchInput('');
  updateQuery({ keyword: '', page: 0 });
};

const toggleBan = async (target: AdminUser) => {
  if (!target?.id) return;

  if (target.admin) {
    toast.error('You cannot ban an admin user');
    return;
  }

  const nextBanned = !Boolean(target.banned);
  const actionLabel = nextBanned ? 'Ban' : 'Unban';

  const ok = window.confirm(
    `${actionLabel} @${target.username}? ${nextBanned ? 'They will not be able to log in.' : 'They will be able to log in.'}
  );
  if (!ok) return;

  try {
    // Optimistic UI update
    setUsers((prev) => prev.map((u) => (u.id === target.id ? { ...u, banned: nextBanned } : u)));

    if (nextBanned) {
      await adminUsersApi.ban(target.id);
      toast.success(`Banned @${target.username}`);
    } else {
      await adminUsersApi.unban(target.id);
      toast.success(`Unbanned @${target.username}`);
    }
  } catch (e) {
    // Revert optimistic update
    setUsers((prev) => prev.map((u) => (u.id === target.id ? { ...u, banned: !nextBanned } : u)));
    console.error('Failed to toggle ban:', e);
    toast.error(`Failed to ${actionLabel.toLowerCase()} user`);
  }
};

if (user?.role !== 'ROLE_ADMIN') {

```

```

    return null;
}

return (
    <div className="min-h-screen bg-background">
        <Navigation />

        <main className="container mx-auto px-4 py-6 max-w-7xl">
            { /* Header */ }
            <div className="flex justify-between items-center mb-6">
                <h1 className="text-3xl font-bold flex items-center gap-2">
                    <Users className="h-8 w-8 text-primary" />
                    User Management
                </h1>
                <div className="flex gap-2">
                    <Button variant="outline" asChild>
                        <Link to="/admin/posts">
                            <Newspaper className="h-4 w-4 mr-2" />
                            Manage Posts
                        </Link>
                    </Button>
                    <Button variant="outline" asChild>
                        <Link to="/">
                            <Home className="h-4 w-4 mr-2" />
                            Home
                        </Link>
                    </Button>
                </div>
            </div>

            { /* Statistics Card */ }
            <Card className="mb-6 gradient-mesh-card">
                <CardContent className="py-6">
                    <div className="flex items-center gap-4">
                        <Users className="h-12 w-12 text-primary" />
                        <div>
                            <h2 className="text-4xl font-bold mb-1">{totalUsers}</h2>
                            <p className="text-lg text-muted-foreground">Total Registered Users</p>
                        </div>
                    </div>
                </CardContent>
            </Card>

            { /* Search Section */ }
            <Card className="gradient-mesh-card mb-6">
                <CardContent className="py-6">
                    <form onSubmit={handleSearch} className="flex flex-col md:flex-row gap-3">
                        <div className="flex-1 relative">
                            <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 h-4 w-4 text-muted-foreground">
                                <Input
                                    type="text"
                                    value={searchInput}
                                    onChange={(e) => setSearchInput(e.target.value)}
                                    placeholder="Search by username, name, or email..."
                                    className="pl-10"
                                />
                            </div>
                            <Button type="submit">
                                <Search className="h-4 w-4 mr-2" />
                                Search
                            </Button>
                        </form>
                        {keyword && (
                            <div className="mt-3 text-sm text-muted-foreground">
                                Showing results for: <strong>{keyword}</strong>{' '}
                                <button onClick={clearSearch} className="text-primary hover:underline ml-2">
                                    Clear
                                </button>
                            </div>
                        )}
                    </CardContent>
                </Card>

```

```

</Card>;

{/* Users Table */}
<Card className="gradient-mesh-card">
  <CardContent className="p-0">
    {isLoading ? (
      <div className="text-center py-12">
        <Users className="h-16 w-16 mx-auto mb-4 text-muted-foreground opacity-50" />
        <h5 className="text-lg font-semibold mb-2">Loading users...</h5>
      </div>
    ) : loadError ? (
      <div className="text-center py-12">
        <Users className="h-16 w-16 mx-auto mb-4 text-muted-foreground opacity-50" />
        <h5 className="text-lg font-semibold mb-2">{loadError}</h5>
        <p className="text-muted-foreground">Try again or refresh the page.</p>
      </div>
    ) : users.length === 0 ? (
      <div className="text-center py-12">
        <Users className="h-16 w-16 mx-auto mb-4 text-muted-foreground opacity-50" />
        <h5 className="text-lg font-semibold mb-2">No users found</h5>
        {keyword &amp; (
          <p className="text-muted-foreground">
            Try a different search term or{' '}
            <button onClick={clearSearch} className="text-primary hover:underline">
              view all users
            </button>
          </p>
        )}
      </div>
    ) : (
      <div className="overflow-x-auto">
        <table className="w-full">
          <thead className="bg-muted/50 border-b">
            <tr>
              <th className="px-4 py-3 text-left font-semibold">Profile</th>
              <th className="px-4 py-3 text-left font-semibold">Full Name</th>
              <th className="px-4 py-3 text-left font-semibold">Email</th>
              <th className="px-4 py-3 text-left font-semibold">Role</th>
              <th className="px-4 py-3 text-left font-semibold">Actions</th>
            </tr>
          </thead>
          <tbody>
            {users.map((adminUser) => (
              <tr key={adminUser.id} className="border-b last:border-b-0 hover:bg-muted/30">
                {/* Profile Picture */}
                <td className="px-4 py-3">
                  <Avatar className="h-10 w-10">
                    <AvatarImage src={adminUser.profileImageUrl} />
                    <AvatarFallback>
                      <DefaultAvatarFallback alt={adminUser.displayName || adminUser.username} />
                    </AvatarFallback>
                  </Avatar>
                </td>

                {/* Full Name */}
                <td className="px-4 py-3">
                  <div>
                    <strong>
                      {adminUser.displayName || adminUser.username}
                    </strong>
                    <br />
                    <small className="text-muted-foreground">
                      @{adminUser.username}
                    </small>
                  </div>
                </td>

                {/* Email */}
                <td className="px-4 py-3">{adminUser.email}</td>

                {/* Role Badge */}

```

```

        <td className="px-4 py-3">
          <div className="flex flex-col gap-1 items-start">
            {adminUser.admin ? (
              <Badge variant="default">
                <Users className="h-3 w-3 mr-1" />
                Admin
              </Badge>
            ) : adminUser.role === 'ROLE_AUTHOR' ? (
              <Badge variant="default">
                <Newspaper className="h-3 w-3 mr-1" />
                Author
              </Badge>
            ) : (
              <Badge variant="secondary">
                <Users className="h-3 w-3 mr-1" />
                User
              </Badge>
            )}

            {adminUser.banned ? (
              <Badge variant="destructive">Banned</Badge>
            ) : null}
          </div>
        </td>

        { /* Actions */ }
        <td className="px-4 py-3">
          <div className="flex gap-2 flex-wrap">
            <Button variant="default" size="sm" asChild>
              <Link to={`/admin/users/${adminUser.id}/posts`} >
                <Newspaper className="h-4 w-4 mr-1" />
                View Posts
              </Link>
            </Button>
            <Button variant="outline" size="sm" asChild>
              <a
                href={`/profile/${adminUser.id}`}
                target="_blank"
                rel="noopener noreferrer"
              >
                <Eye className="h-4 w-4 mr-1" />
                Profile
              </a>
            </Button>

            <Button
              variant={adminUser.banned ? 'outline' : 'destructive'}
              size="sm"
              disabled={adminUser.admin}
              onClick={() => toggleBan(adminUser)}
              title={adminUser.admin ? 'Cannot ban admin users' : undefined}
            >
              {adminUser.banned ? 'Unban' : 'Ban'}
            </Button>
          </div>
        </td>
      </tr>
    </tbody>
  </table>
</div>
)}
</CardContent>
</Card>

{ /* Pagination */ }
{!isLoading && !loadError && totalPages > 1 && (
  <div className="mt-6 flex flex-col items-center gap-2">
    <p className="text-sm text-muted-foreground">
      Page <strong>{page + 1}</strong> of <strong>{totalPages}</strong>
      {totalUsers ? (

```

```

        &lt;&gt; - &lt;strong&gt;{totalUsers}&lt;/strong&gt; users&lt;/&gt;
    ) : null}
    &lt;/p&gt;

    &lt;Pagination&gt;
    &lt;PaginationContent&gt;
    &lt;PaginationItem&gt;
    &lt;PaginationPrevious
    href={buildHref({ page: Math.max(0, page - 1) })}
    disabled={page === 0}
    onClick={(e) => {
    e.preventDefault();
    updateQuery({ page: Math.max(0, page - 1) });
    }}
    /&gt;
    &lt;/PaginationItem&gt;

    {Array.from({ length: Math.min(5, totalPages) }, (_, i) => {
    const start = Math.max(0, Math.min(page - 2, totalPages - 5));
    const pageNumber = start + i;
    if (pageNumber &gt;= totalPages) return null;
    return (
    &lt;PaginationItem key={pageNumber}&gt;
    &lt;PaginationLink
    href={buildHref({ page: pageNumber })}
    isActive={pageNumber === page}
    onClick={(e) => {
    e.preventDefault();
    updateQuery({ page: pageNumber });
    }}
    &gt;
    {pageNumber + 1}
    &lt;/PaginationLink&gt;
    &lt;/PaginationItem&gt;
    );
    )}}

    &lt;PaginationItem&gt;
    &lt;PaginationNext
    href={buildHref({ page: Math.min(totalPages - 1, page + 1) })}
    disabled={page &gt;= totalPages - 1}
    onClick={(e) => {
    e.preventDefault();
    updateQuery({ page: Math.min(totalPages - 1, page + 1) });
    }}
    /&gt;
    &lt;/PaginationItem&gt;
    &lt;/PaginationContent&gt;
    &lt;/Pagination&gt;
    &lt;/div&gt;
  )}
  &lt;/main&gt;
  &lt;/div&gt;
);
}

```

■ frontend/src/pages/AnnouncementCreate.tsx

```

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Label } from '@components/ui/label';
import { Textarea } from '@components/ui/textarea';
import { Checkbox } from '@components/ui/checkbox';
import { PlusCircle, Save, X, Info, ArrowLeft } from 'lucide-react';
import { useAuthContext } from '@contexts/AuthContext';
import { announcementsApi, ApiError } from '@lib/api';

```



```

import { toast } from 'sonner';

export default function AnnouncementCreate() {
  const navigate = useNavigate();
  const { user } = useAuthContext();

  const [formData, setFormData] = useState({
    title: '',
    message: '',
    visible: true,
  });
  const [videoFile, setVideoFile] = useState<File | null>(null);
  const [videoPreview, setVideoPreview] = useState<string | null>(null);
  const [imageFiles, setImageFiles] = useState<File[]>([]);
  const [imagePreviews, setImagePreviews] = useState<string[]>([]);
  const [isSaving, setIsSaving] = useState(false);

  useEffect(() => {
    if (user?.role !== 'ROLE_ADMIN') {
      navigate('/');
    }
  }, [user, navigate]);

  const handleVideoChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (!file) return;

    if (!file.type.startsWith('video/')) {
      toast.error('Please select a valid video file');
      return;
    }

    if (file.size > 100 * 1024 * 1024) {
      toast.error('Video file must be less than 100MB');
      return;
    }

    setVideoFile(file);
    const url = URL.createObjectURL(file);
    setVideoPreview(url);
  };

  const handleImagesChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const files = Array.from(e.target.files ?? []);
    if (files.length === 0) return;

    const bad = files.find((f) => !f.type.startsWith('image/'));
    if (bad) {
      toast.error('Please select valid image files');
      return;
    }

    // Keep it reasonable for announcements
    const maxEach = 10 * 1024 * 1024;
    const tooLarge = files.find((f) => f.size > maxEach);
    if (tooLarge) {
      toast.error('Each image must be less than 10MB');
      return;
    }

    // Limit to 6 to keep UI snappy
    const next = files.slice(0, 6);
    setImageFiles(next);
    setImagePreviews(next.map((f) => URL.createObjectURL(f)));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setIsSaving(true);

    try {

```

```

    await announcementsApi.create({
      title: formData.title,
      message: formData.message,
      visible: formData.visible,
      imageFiles,
      videoFile,
    });

    toast.success('Announcement created');
    navigate('/announcements');
  } catch (err) {
    const msg = err instanceof ApiError ? err.message : 'Failed to create announcement';
    toast.error(msg);
  } finally {
    setIsSaving(false);
  }
};

const handleCancel = () => {
  navigate('/announcements');
};

if (user?.role !== 'ROLE_ADMIN') return null;

return (
  <>
    <Navigation />
    <div className="min-h-screen pb-8">
      <div className="max-w-4xl mx-auto px-4 py-8">
        <Card className="gradient-mesh-card p-8">
          <h1 className="text-3xl font-semibold mb-6 pb-4 border-b border-gray-200 flex items-center gap-3">
            <PlusCircle className="w-8 h-8 text-purple-600" />
            Create New Announcement
          </h1>

          <form onSubmit={handleSubmit} className="space-y-6">
            {/* Title */}
            <div>
              <Label htmlFor="title" className="text-base font-semibold text-gray-700 mb-2">
                Title
              </Label>
              <Input
                id="title"
                type="text"
                placeholder="Enter announcement title"
                value={formData.title}
                onChange={(e) => setFormData({ ...formData, title: e.target.value })}
                required
                className="mt-2"
              />
            </div>

            {/* Message */}
            <div>
              <Label htmlFor="message" className="text-base font-semibold text-gray-700 mb-2">
                Message
              </Label>
              <Textarea
                id="message"
                placeholder="Enter announcement message"
                value={formData.message}
                onChange={(e) => setFormData({ ...formData, message: e.target.value })}
                required
                rows={5}
                className="mt-2"
              />
            </div>

            {/* Video Upload */}
            <div>
              <Label htmlFor="videoFile" className="text-base font-semibold text-gray-700 mb-2">

```

```

        Video (Optional)
    </Label>
    <Input
        id="videoFile"
        type="file"
        accept="video/mp4,video/mov,video/avi"
        onChange={handleVideoChange}
        className="mt-2"
    />
    <div className="flex items-center gap-2 mt-2 text-sm text-gray-600">
        <Info className="w-4 h-4" />
        <span>Supported formats: MP4, MOV, AVI (Max file size: 100MB)</span>
    </div>

    {/ * Video Preview */}
    {videoPreview && (
        <div className="mt-4">
            <video
                controls
                src={videoPreview}
                className="max-w-full max-h-[300px] rounded-lg shadow-md"
            />
        </div>
    )}
    </div>

    {/ * Images Upload */}
    <div>
        <Label htmlFor="imageFiles" className="text-base font-semibold text-gray-700 mb-2">
            Images (Optional)
        </Label>
        <Input
            id="imageFiles"
            type="file"
            accept="image/*"
            multiple
            onChange={handleImagesChange}
            className="mt-2"
        />
        <div className="flex items-center gap-2 mt-2 text-sm text-gray-600">
            <Info className="w-4 h-4" />
            <span>Up to 6 images, each max 10MB</span>
        </div>

        {imagePreviews.length > 0 && (
            <div className="mt-4 grid grid-cols-2 sm:grid-cols-3 gap-3">
                {imagePreviews.map((src, idx) => (
                    <img
                        key={idx}
                        src={src}
                        alt={`Announcement image ${idx + 1}`}
                        className="w-full h-32 object-cover rounded-lg shadow-sm border"
                    />
                ))}
            </div>
        )}
    </div>

    {/ * Visible Checkbox */}
    <div className="flex items-center space-x-2">
        <Checkbox
            id="visible"
            checked={formData.visible}
            onChange={(checked) => {
                setFormData({ ...formData, visible: checked as boolean })
            }}
        />
        <Label htmlFor="visible" className="text-base font-medium cursor-pointer flex items-center gap-2">
            <span className="text-lg">
                ■■
            </span>
        </Label>
    </div>

```

```

        Visible to users
      </Label>
    </div>

    { /* Action Buttons */ }
    <div className="flex gap-3 pt-4">
      <Button type="submit" disabled={isSaving}>
        <Save className="w-4 h-4 mr-2" />
        {isSaving ? 'Creating...' : 'Create Announcement'}
      </Button>
      <Button type="button" variant="outline" onClick={handleCancel} disabled={isSaving}>
        <X className="w-4 h-4 mr-2" />
        Cancel
      </Button>
    </div>
  </form>

  <Button variant="link" onClick={handleCancel} className="mt-4 pl-0">
    <ArrowLeft className="w-4 h-4 mr-2" />
    Back to Announcements
  </Button>
</Card>
</div>
</div>
</>
);
}

```

■ frontend/src/pages/AnnouncementEdit.tsx

```

import { useState, useEffect } from 'react';
import { useNavigate, useParams, Link } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card, CardContent } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Textarea } from '@components/ui/textarea';
import { Label } from '@components/ui/label';
import { Checkbox } from '@components/ui/checkbox';
import { ArrowLeft, Save, X, Trash2, Upload, Loader2 } from 'lucide-react';
import { useAuthContext } from '@contexts/AuthContext';
import { toast } from 'sonner';
import { announcementsApi, ApiError } from '@lib/api';

export default function AnnouncementEdit() {
  const navigate = useNavigate();
  const { id } = useParams<{ id: string }>();
  const { user } = useAuthContext();

  const [loading, setLoading] = useState(false);
  const [saving, setSaving] = useState(false);

  const [formData, setFormData] = useState({
    title: '',
    message: '',
    visible: true,
    videoUrl: '',
    removeVideo: false,
    removeImages: false,
  });

  const [existingImages, setExistingImages] = useState<string[]>([]);
  const [videoFile, setVideoFile] = useState<File | null>(null);
  const [videoPreview, setVideoPreview] = useState<string | null>(null);
  const [imageFiles, setImageFiles] = useState<File[]>([]);
  const [imagePreviews, setImagePreviews] = useState<string[]>([]);

  useEffect(() => {
    if (user?.role !== 'ROLE_ADMIN') {
      navigate('/');
    }
  }, [user]);

```

```

    return;
  }

  const numericId = Number(id);
  if (!numericId) {
    toast.error('Invalid announcement id');
    navigate('/admin/announcements');
    return;
  }

  let cancelled = false;

  (async () => {
    setLoading(true);
    try {
      const a = await announcementsApi.getAdmin(numericId);
      if (cancelled) return;

      setFormData({
        title: a.title ?? '',
        message: a.message ?? '',
        visible: a.visible ?? true,
        videoUrl: a.videoUrl ?? '',
        removeVideo: false,
        removeImages: false,
      });
      setExistingImages(Array.isArray(a.imageUrls) ? a.imageUrls : []);
    } catch (e) {
      const msg = e instanceof ApiError ? e.message : 'Failed to load announcement';
      toast.error(msg);
      navigate('/admin/announcements');
    } finally {
      if (!cancelled) setLoading(false);
    }
  })();

  return () => {
    cancelled = true;
  };
}, [id, user, navigate]));

const handleVideoChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (!file) return;

  if (!file.type.startsWith('video/')) {
    toast.error('Please select a valid video file');
    return;
  }

  if (file.size > 100 * 1024 * 1024) {
    toast.error('Video file must be less than 100MB');
    return;
  }

  setVideoFile(file);
  const url = URL.createObjectURL(file);
  setVideoPreview(url);

  // If user picked a replacement, we should not remove the existing video.
  setFormData((prev) => ({ ...prev, removeVideo: false }));
};

const handleImagesChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const files = Array.from(e.target.files ?? []);
  if (files.length === 0) return;

  const bad = files.find((f) => !f.type.startsWith('image/'));
  if (bad) {
    toast.error('Please select valid image files');
    return;
  }

```

```

    }

    const maxEach = 10 * 1024 * 1024;
    const tooLarge = files.find((f) => f.size > maxEach);
    if (tooLarge) {
      toast.error('Each image must be less than 10MB');
      return;
    }

    const next = files.slice(0, 6);
    setImageFiles(next);
    setImagePreviews(next.map((f) => URL.createObjectURL(f)));

    // If user is adding images, they probably don't want to wipe existing ones.
    setFormData((prev) => ({ ...prev, removeImages: false }));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();

    const numericId = Number(id);
    if (!numericId) {
      toast.error('Invalid announcement id');
      return;
    }

    setSaving(true);
    try {
      await announcementsApi.update(numericId, {
        title: formData.title,
        message: formData.message,
        visible: formData.visible,
        removeImages: formData.removeImages,
        imageFiles,
        removeVideo: formData.removeVideo,
        videoFile,
      });

      toast.success('Announcement updated successfully!');
      navigate('/admin/announcements');
    } catch (error) {
      const msg = error instanceof ApiError ? error.message : 'Failed to update announcement';
      toast.error(msg);
    } finally {
      setSaving(false);
    }
  };

  if (user?.role !== 'ROLE_ADMIN') {
    return null;
  }

  return (
    <div className="min-h-screen bg-background">
      <Navigation />

      <main className="container mx-auto px-4 py-6 max-w-4xl">
        <div className="mb-6">
          <Button variant="outline" asChild className="mb-4">
            <Link to="/admin/announcements">
              <ArrowLeft className="h-4 w-4 mr-2" />
              Back to Announcements
            </Link>
          </Button>
          <h2 className="text-3xl font-bold">Edit Announcement</h2>
        </div>

        <Card className="gradient-mesh-card">
          <CardContent className="pt-6">
            {loading ? (
              <div className="flex justify-center items-center py-12">

```

```

        <Loader2 className="h-8 w-8 animate-spin text-primary" />
    </div>
) : (
    <form onSubmit={handleSubmit} className="space-y-6">
        { /* Title */
        <div className="space-y-2">
            <Label htmlFor="title">
                Title <span className="text-red-500">*</span>
            </Label>
            <Input
                id="title"
                type="text"
                value={formData.title}
                onChange={(e) => setFormData({ ...formData, title: e.target.value })}
                placeholder="Enter announcement title"
                required
            />
        </div>

        { /* Message */
        <div className="space-y-2">
            <Label htmlFor="message">
                Message <span className="text-red-500">*</span>
            </Label>
            <Textarea
                id="message"
                value={formData.message}
                onChange={(e) => setFormData({ ...formData, message: e.target.value })}
                placeholder="Enter announcement message"
                rows={5}
                required
            />
        </div>

        { /* Existing Video */
        {formData.videoUrl && !formData.removeVideo && (
            <div className="space-y-2">
                <Label>Current Video</Label>
                <video controls className="w-full max-h-64 rounded-lg" src={formData.videoUrl}>
                    Your browser does not support the video tag.
                </video>
                <div className="flex items-center space-x-2">
                    <Checkbox
                        id="removeVideo"
                        checked={formData.removeVideo}
                        onChange={(checked) =>
                            setFormData({ ...formData, removeVideo: checked as boolean })
                        }
                    />
                    <Label htmlFor="removeVideo" className="cursor-pointer">
                        <Trash2 className="h-4 w-4 inline mr-2" />
                        Remove this video
                    </Label>
                </div>
            </div>
        )}

        { /* New Video Upload */
        <div className="space-y-2">
            <Label htmlFor="videoFile">
                {formData.videoUrl && !formData.removeVideo
                    ? 'Replace Video (Optional)'
                    : 'Upload Video (Optional)'}
            </Label>
            <div className="flex items-center gap-3">
                <Input
                    id="videoFile"
                    type="file"
                    accept="video/mp4,video/mov,video/avi"
                    onChange={handleVideoChange}
                    className="flex-1"
                />
            </div>
        </div>
        }
    </form>
)

```

```

    /&gt;
    &lt;Upload className="h-5 w-5 text-muted-foreground" /&gt;
  &lt;/div&gt;
  &lt;p className="text-sm text-muted-foreground"&gt;
    Supported formats: MP4, MOV, AVI (Max file size: 100MB)
  &lt;/p&gt;
&lt;/div&gt;

{/* Video Preview */}
{videoPreview &amp; (
  &lt;div className="space-y-2"&gt;
    &lt;Label&gt;New Video Preview&lt;/Label&gt;
    &lt;video controls className="w-full max-h-64 rounded-lg" src={videoPreview}&gt;
      Your browser does not support the video tag.
    &lt;/video&gt;
  &lt;/div&gt;
)}

{/* Existing Images */}
{existingImages.length &gt; 0 &amp; (
  &lt;div className="space-y-2"&gt;
    &lt;Label&gt;Current Images&lt;/Label&gt;
    &lt;div className="grid grid-cols-2 sm:grid-cols-3 gap-3"&gt;
      {existingImages.map((src, idx) => (
        &lt;img
          key={idx}
          src={src}
          alt={`Announcement image ${idx + 1}`}
          className="w-full h-32 object-cover rounded-lg border"
        /&gt;
      ))}
    &lt;/div&gt;
    &lt;div className="flex items-center space-x-2"&gt;
      &lt;Checkbox
        id="removeImages"
        checked={formData.removeImages}
        onChange={() => {
          setFormData({ ...formData, removeImages: checked as boolean })
        }}
      /&gt;
      &lt;Label htmlFor="removeImages" className="cursor-pointer"&gt;
        &lt;Trash2 className="h-4 w-4 inline mr-2" /&gt;
        Remove all images
      &lt;/Label&gt;
    &lt;/div&gt;
  &lt;/div&gt;
)}

{/* New Images Upload */}
&lt;div className="space-y-2"&gt;
  &lt;Label htmlFor="imageFiles"&gt;Add Images (Optional)&lt;/Label&gt;
  &lt;div className="flex items-center gap-3"&gt;
    &lt;Input
      id="imageFiles"
      type="file"
      accept="image/*"
      multiple
      onChange={handleImagesChange}
      className="flex-1"
    /&gt;
    &lt;Upload className="h-5 w-5 text-muted-foreground" /&gt;
  &lt;/div&gt;
  &lt;p className="text-sm text-muted-foreground"&gt;Up to 6 images, each max 10MB&lt;/p&gt;
&lt;/div&gt;

{imagePreviews.length &gt; 0 &amp; (
  &lt;div className="space-y-2"&gt;
    &lt;Label&gt;New Images Preview&lt;/Label&gt;
    &lt;div className="grid grid-cols-2 sm:grid-cols-3 gap-3"&gt;
      {imagePreviews.map((src, idx) => (
        &lt;img

```



```

        key={idx}
        src={src}
        alt={`New image ${idx + 1}`}
        className="w-full h-32 object-cover rounded-lg border"
      />
    ))}
  </div>
</div>
)}

{/* Visible Checkbox */}
<div className="flex items-center space-x-2">
  <Checkbox
    id="visible"
    checked={formData.visible}
    onCheckedChange={(checked) => {
      setFormData({ ...formData, visible: checked as boolean })
    }}
  />
  <Label htmlFor="visible" className="cursor-pointer">
    Visible to users
  </Label>
</div>

{/* Submit Buttons */}
<div className="flex gap-3 pt-4">
  <Button type="submit" className="flex-1 disabled={saving}>
    {saving ? (
      'Saving...'
    ) : (
      <Save className="h-4 w-4 mr-2" />
      Save Changes
    )}
  </Button>
  <Button
    type="button"
    variant="outline"
    onClick={() => navigate('/admin/announcements')}
    disabled={saving}
  >
    <X className="h-4 w-4 mr-2" />
    Cancel
  </Button>
</div>
</form>
)}
</CardContent>
</Card>
</main>
</div>
);
}

```

■ frontend/src/pages/Announcements.tsx

```

import { useEffect, useState } from 'react';
import { Navigation } from '@components/Navigation';
import { Card, CardContent } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Badge } from '@components/ui/badge';
import { Alert, AlertDescription } from '@components/ui/alert';
import { Loader2, Plus, Bell } from 'lucide-react';
import { Link } from 'react-router-dom';
import { format } from 'date-fns';
import { useAuthContext } from '@contexts/AuthContext';
import { announcementsApi, type Announcement, ApiError } from '@lib/api';
import { AnimatedBackground } from '@components/AnimatedBackground';

```

```

export default function Announcements() {
  const { user } = useAuthContext();
  const [announcements, setAnnouncements] = useState<Announcement[]>([]);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [previewImage, setPreviewImage] = useState<string | null>(null);

  useEffect(() => {
    // Close preview on Escape
    const onKeyDown = (e: KeyboardEvent) => {
      if (e.key === 'Escape') setPreviewImage(null);
    };
    if (previewImage) {
      window.addEventListener('keydown', onKeyDown);
      return () => window.removeEventListener('keydown', onKeyDown);
    }
  }, [previewImage]);

  useEffect(() => {
    let cancelled = false;

    (async () => {
      setIsLoading(true);
      setError(null);
      try {
        const data = await announcementsApi.listPublic();
        if (!cancelled) setAnnouncements(data);
      } catch (e) {
        const message = e instanceof ApiError ? e.message : 'Failed to load announcements';
        if (!cancelled) setError(message);
      } finally {
        if (!cancelled) setIsLoading(false);
      }
    })();

    return () => {
      cancelled = true;
    };
  }, []);

  useEffect(() => {
    // Auto-play videos as they scroll into view
    const videos = document.querySelectorAll('.announcement-video');

    const observerOptions = {
      root: null,
      rootMargin: '0px',
      threshold: 0.5,
    };

    const videoObserver = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
        const video = entry.target as HTMLVideoElement;
        if (entry.isIntersecting) {
          video.play().catch((err) => console.log('Autoplay prevented:', err));
        } else {
          video.pause();
        }
      });
    });

    videos.forEach((video) => {
      videoObserver.observe(video);
    });

    return () => {
      videos.forEach((video) => {
        videoObserver.unobserve(video);
      });
    };
  });
}

```

```

}, [announcements]);

return (
  <div className="min-h-screen bg-background relative">
    {/ * Image preview modal */}
    {previewImage & & (
      <div
        className="fixed inset-0 z-60 bg-black/70 flex items-center justify-center p-4"
        role="dialog"
        aria-modal="true"
        onClick={() => setPreviewImage(null)}
      >
        <div
          className="relative max-w-5xl w-full"
          onClick={(e) => e.stopPropagation()}
        >
          <button
            type="button"
            aria-label="Close image preview"
            className="absolute -top-3 -right-3 bg-background text-foreground rounded-full border shadow px-3 py-1"
            onClick={() => setPreviewImage(null)}
          >
            X
          </button>
          <img
            src={previewImage}
            alt="Announcement image preview"
            className="w-full max-h-[85vh] object-contain rounded-xl bg-black"
          />
        </div>
      </div>
    )}

    <div className="z-0">
      <AnimatedBackground />
    </div>

    <div className="relative z-10">
      <Navigation />

      <main className="container mx-auto px-4 py-6 max-w-6xl">
        <div className="mb-6 flex justify-end items-center">
          {user?.role === 'ROLE_ADMIN' & & (
            <Button asChild>
              <Link to="/announcements/create">
                <Plus className="h-4 w-4 mr-2" />
                Create Announcement
              </Link>
            </Button>
          )}
        </div>

        {isLoading ? (
          <div className="flex justify-center items-center py-12">
            <Loader2 className="h-8 w-8 animate-spin text-primary" />
          </div>
        ) : error ? (
          <Alert variant="destructive">
            <AlertDescription>{error}</AlertDescription>
          </Alert>
        ) : announcements.length === 0 ? (
          <Alert>
            <AlertDescription>No announcements at this time.</AlertDescription>
          </Alert>
        ) : (
          <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
            {announcements.map((announcement) => (
              <Card key={announcement.id} className="gradient-mesh-card">
                <CardContent className="pt-6">
                  <h5 className="text-xl font-bold mb-3 flex items-center gap-2">
                    <Bell className="h-5 w-5 text-yellow-500" />

```

```

        {announcement.title}
    </h5>
    <p className="text-foreground/90 mb-4">{announcement.message}</p>

    /* Images Display */
    {announcement.imageUrls && announcement.imageUrls.length > 0 && (
        <div className="mt-3 mb-3 space-y-4">
            {announcement.imageUrls.map((src, idx) => (
                <div key={idx} className="w-full flex justify-center">
                    <button
                        type="button"
                        onClick={() => setPreviewImage(src)}
                        className="inline-block focus:outline-none focus-visible:ring-2 focus-visible:ring-primar
                        aria-label={`View announcement image ${idx + 1}`}
                    >
                <div className="w-[520px] max-w-full h-56 sm:h-72 rounded-xl border bg-black/5 overflow
                    <img
                        src={src}
                        alt={`Announcement image ${idx + 1}`}
                        className="w-full h-full object-contain cursor-zoom-in"
                        loading="lazy"
                    />
                </div>
            </button>
        </div>
    )})
    </div>
})

/* Video Display */
{announcement.videoUrl && (
    <div className="mt-3 mb-3">
        <video
            controls
            muted
            loop
            controlsList="nodownload noremotep playback"
            disablePictureInPicture
            onContextMenu={(e) => e.preventDefault()}
            className="announcement-video w-full rounded-lg"
        >
            <source src={announcement.videoUrl} type="video/mp4" />
            Your browser does not support the video tag.
        </video>
    </div>
})

    {announcement.createdAt && (
        <Badge variant="secondary" className="mt-2">
            {format(new Date(announcement.createdAt), 'yyyy-MM-dd HH:mm')}
        </Badge>
    )}
    </CardContent>
    </Card>
    )})
    </div>
})

    <div className="mt-6">
        <Button variant="outline" asChild>
            <Link to="/">Back to Home</Link>
        </Button>
    </div>
    </main>
    </div>
    </div>
);
}

```

■ frontend/src/pages/ChangeUsername.tsx

```

import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Label } from '@components/ui/label';
import { useAuthContext } from '@contexts/AuthContext';

export default function ChangeUsername() {
  const navigate = useNavigate();
  const { user } = useAuthContext();
  const [username, setUsername] = useState(user?.username || '');
  const [error, setError] = useState<string | null>(null);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);

    // Validation
    if (username.length < 3 || username.length > 30) {
      setError('Username must be between 3 and 30 characters');
      return;
    }

    const usernamePattern = /^[A-Za-z0-9._-]+$/;
    if (!usernamePattern.test(username)) {
      setError('Username can only contain letters, numbers, dots, underscores, and hyphens');
      return;
    }

    if (username.includes('@')) {
      setError('Username cannot be an email address');
      return;
    }

    // TODO: Replace with actual API call
    console.log('Updating username to:', username);

    // Navigate back to home or profile after successful update
    navigate('/');
  };

  const handleCancel = () => {
    navigate('/');
  };

  return (
    <>
    <Navigation />
    <div className="min-h-screen pb-8">
      <div className="max-w-2xl mx-auto px-4 py-8">
        <Card className="gradient-mesh-card p-8">
          <h1 className="text-2xl font-semibold mb-6">Change Username</h1>

          <div className="mb-6">
            <p className="text-sm text-gray-600 mb-1">Email</p>
            <p className="font-semibold text-lg">{user?.email || 'user@example.com'}</p>
          </div>

          {error & & (
            <div className="mb-4 p-4 bg-red-50 border border-red-200 rounded-lg text-red-800">
              {error}
            </div>
          )}
        </Card>
      </div>
    </div>
  );
}

```

```

    <form onSubmit={handleSubmit}>
      <div className="mb-6">
        <Label htmlFor="username" className="text-base font-semibold mb-2">
          New Username
        </Label>
        <Input
          id="username"
          type="text"
          value={username}
          onChange={(e) => setUsername(e.target.value)}
          required
          minLength={3}
          maxLength={30}
          pattern="^[A-Za-z0-9._-]+$"
          className="mt-2"
        />
        <div className="mt-2 text-sm text-gray-600">
          3-30 characters. Allowed: letters, numbers, dots, underscores, hyphens. Must not be an email.
        </div>
      </div>

      <div className="flex justify-end gap-3">
        <Button type="button" variant="outline" onClick={handleCancel}>
          Cancel
        </Button>
        <Button type="submit">
          Save Username
        </Button>
      </div>
    </form>
  </Card>
</div>
</div>
</>
);
}

```

■ frontend/src/pages/ChangeUsernameError.tsx

```

import { useNavigate, useSearchParams } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { AlertTriangle } from 'lucide-react';
import { useAuthContext } from '@contexts/AuthContext';

export default function ChangeUsernameError() {
  const navigate = useNavigate();
  const [searchParams] = useSearchParams();
  const { user } = useAuthContext();

  const error = searchParams.get('error') || 'You are not allowed to change your username.';

  return (
    <>
      <Navigation />
      <div className="min-h-screen pb-8">
        <div className="max-w-2xl mx-auto px-4 py-8">
          <Card className="gradient-mesh-card p-8">
            <h1 className="text-2xl font-semibold mb-6">Change Username</h1>

            <div className="mb-6 p-4 bg-yellow-50 border border-yellow-200 rounded-lg">
              <div className="flex items-start gap-3">
                <AlertTriangle className="w-5 h-5 text-yellow-600 mt-0.5" />
              </div>
              <p className="text-yellow-800 mb-2">{error}</p>
              <p className="text-sm text-yellow-700">
                Account: <strong>{user?.email || 'Unknown'}</strong>
              </p>
            </div>
          </Card>
        </div>
      </div>
    </>
  );
}

```

```

        </div>
    </div>

    <Button onClick={() => navigate('/')}>
        Back to Home
    </Button>
    </Card>
</div>
</div>
</>
);
}

```

■ frontend/src/pages/Chat.tsx

```

import { useState, useEffect, useRef, useCallback } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar';
import { Send, MessageCircle, Lock } from 'lucide-react';
import { useAuthContext } from '@contexts/AuthContext';
import { DefaultAvatarFallback } from '@components/DefaultAvatarFallback';
import { chatApi, followApi, OpenChatResponse, type ChatMessage as ApiChatMessage, type ChatRoomSummary } from '@lib/api';
import { ChatRealtimeClient } from '@lib/chatRealtime';

interface Message {
  id: number;
  roomId: number;
  senderUsername: string;
  senderDisplayName: string;
  content: string;
  timestamp: string;
}

interface ChatContact {
  id: number;
  username: string;
  displayName: string;
  profileImageUrl?: string;
  /** Optional compatibility field used elsewhere in UI */
  avatar?: string;
  lastMessage?: string;
  lastMessageTime?: string;
  unreadCount?: number;
}

interface TargetUser {
  id: number;
  username: string;
  displayName: string;
  profileImageUrl?: string;
  /** Optional compatibility field used elsewhere in UI */
  avatar?: string;
}

export default function Chat() {
  const { userId } = useParams<{ userId: string }>();
  const navigate = useNavigate();
  const { user: currentUser, isAuthenticated } = useAuthContext();
  const [messages, setMessages] = useState<Message[]>([]);
  const [messageInput, setMessageInput] = useState('');
  const [contacts, setContacts] = useState<ChatContact[]>([]);
  const [isLoadingContacts, setIsLoadingContacts] = useState(false);
  const [contactQuery, setContactQuery] = useState('');
  const messagesEndRef = useRef<HTMLDivElement>(null);
  const lastMessageCountRef = useRef<number>(0);
  const lastRoomIdRef = useRef<number | null>(null);

```

```

const [isLoadingOlder, setIsLoadingOlder] = useState(false);
const [hasMoreOlder, setHasMoreOlder] = useState(true);
const topSentinelRef = useRef<HTMLElement>(null);
const scrollContainerRef = useRef<HTMLElement>(null);
const oldestTimestampRef = useRef<string | null>(null);

const selectedContactId = userId ? Number(userId) : null;
const selectedContactIdSafe = selectedContactId != null && Number.isFinite(selectedContactId) ? selectedContactId : null;
const [targetUser, setTargetUser] = useState<TargetUser | null>(null);
const [roomId, setRoomId] = useState<number | null>(null);
const [canMessage, setCanMessage] = useState<boolean>(false);
const [lockedReason, setLockedReason] = useState<string | null>(null);
const [isLoadingThread, setIsLoadingThread] = useState(false);
const realtimeRef = useRef<ChatRealtimeClient | null>(null);
const subRef = useRef<{ unsubscribe: () => void } | null>(null);
const pollRef = useRef<number | null>(null);

// Used to ignore stale async responses when quickly switching contacts.
const threadRequestSeqRef = useRef(0);

// Reduce page size (pagination limit) for smoother UX.
const PAGE_SIZE = 15;

useEffect(() => {
  if (!isAuthenticated) {
    navigate('/login');
  }
}, [isAuthenticated, navigate]);

// Only auto-scroll when:
// - switching rooms, OR
// - new messages arrive AND the user is already near the bottom.
useEffect(() => {
  const currentCount = messages.length;
  const roomChanged = roomId !== lastRoomIdRef.current;

  const scroller = scrollContainerRef.current;
  const nearBottom = (() => {
    if (!scroller) return true;
    const thresholdPx = 120;
    const distanceFromBottom = scroller.scrollHeight - (scroller.scrollTop + scroller.clientHeight);
    return distanceFromBottom <= thresholdPx;
  })();

  const appended = currentCount > lastMessageCountRef.current;

  if (roomChanged || (appended && nearBottom)) {
    if (scroller) {
      // More reliable than scrollIntoView when we have nested overflow containers.
      scroller.scrollTop = scroller.scrollHeight;
    } else {
      messagesEndRef.current?.scrollIntoView({
        behavior: roomChanged ? 'auto' : 'smooth',
        block: 'end',
      });
    }
  }

  lastMessageCountRef.current = currentCount;
  lastRoomIdRef.current = roomId;
}, [messages.length, roomId]);

useEffect(() => {
  return () => {
    // Cleanup realtime + polling when leaving page
    try {
      subRef.current?.unsubscribe();
    } catch {
      // ignore
    }
    subRef.current = null;
  };
}, []);

```



```

    if (pollRef.current !== null) {
      window.clearInterval(pollRef.current);
      pollRef.current = null;
    }

    realtimeRef.current?.disconnect();
    realtimeRef.current = null;
  };
}, []);

const refreshContacts = useCallback(async () => {
  if (!isAuthenticated) return;
  try {
    setIsLoadingContacts(true);
    const resp = await chatApi.list();
    const rooms = (resp.chatRooms || []) as ChatRoomSummary[];

    const next: ChatContact[] = rooms
      // Only show real conversations (at least one message exchanged)
      .filter((r) => r.lastMessageTime !== null)
      .map((r) => ({
        id: Number(r.otherUserId),
        username: r.otherUserUsername || '',
        displayName: r.otherUserDisplayName,
        profileImageUrl: r.otherUserProfileImage,
        lastMessage: r.lastMessage || undefined,
        lastMessageTime: r.lastMessageTime || undefined,
        unreadCount: 0,
      }));

    // Sort most recent first
    next.sort((a, b) => {
      const ta = a.lastMessageTime ? new Date(a.lastMessageTime).getTime() : 0;
      const tb = b.lastMessageTime ? new Date(b.lastMessageTime).getTime() : 0;
      return tb - ta;
    });

    setContacts(next);
  } catch (e) {
    console.error('Failed to load conversations:', e);
    setContacts([]);
  } finally {
    setIsLoadingContacts(false);
  }
}, [isAuthenticated]);

// Load conversations on entry + whenever auth becomes available
useEffect(() => {
  if (!isAuthenticated) return;
  void refreshContacts();
}, [isAuthenticated, refreshContacts]);

// Helper: force scroll to bottom (used for initial load / thread switches)
const scrollToBottom = useCallback((behavior: ScrollBehavior = 'auto') => {
  const scroller = scrollContainerRef.current;
  if (!scroller) return;

  // Do it a few times to account for async layout (fonts/images) and React paint timing.
  const doScroll = () => {
    try {
      scroller.scrollTo({ top: scroller.scrollHeight, behavior });
    } catch {
      scroller.scrollTop = scroller.scrollHeight;
    }
  };

  requestAnimationFrame(() => {
    doScroll();
    requestAnimationFrame(() => {
      doScroll();
      // One last nudge after the browser has had a moment to finalize layout.

```

```

        window.setTimeout(doScroll, 0);
    });
});
}, []);

// After finishing loading a thread, always snap to the bottom.
// This covers cases where message heights change after initial render.
useEffect(() => {
    if (isLoadingThread) return;
    if (!targetUser) return;
    // When opening a thread, we prefer auto to avoid "jump" animation.
    scrollToBottom('auto');
}, [isLoadingThread, targetUser, roomId, scrollToBottom]);

useEffect(() => {
    let mounted = true;
    // Bump request sequence so older in-flight calls can't update state.
    const reqId = ++threadRequestSeqRef.current;

    // Mark thread as loading immediately to avoid flashing stale/locked UI.
    setIsLoadingThread(true);

    // Reset thread state immediately on contact switch (prevents flicker/glitch).
    setTargetUser(null);
    setMessages([]);
    setRoomId(null);
    setCanMessage(false);
    setLockedReason(null);
    setHasMoreOlder(true);
    setIsLoadingOlder(false);
    oldestTimestampRef.current = null;

    // Also clear any draft so switching contacts doesn't carry text over.
    setMessageInput('');

    // Clear realtime + polling for the previous room.
    try {
        subRef.current?.unsubscribe();
    } catch {
        // ignore
    }
    subRef.current = null;
    if (pollRef.current !== null) {
        window.clearInterval(pollRef.current);
        pollRef.current = null;
    }
    realtimeRef.current?.disconnect();
    realtimeRef.current = null;

    const loadThread = async () => {
        if (!selectedContactIdSafe || !isAuthenticated) {
            setTargetUser(null);
            return;
        }

        try {
            setIsLoadingThread(true);
            const resp: OpenChatResponse = await chatApi.openWith(selectedContactIdSafe);
            if (!mounted) return;
            if (reqId !== threadRequestSeqRef.current) return;

            // Ensure left panel stays up to date.
            void refreshContacts();

            const tu = resp.targetUser;
            setTargetUser({
                id: Number(tu.id),
                username: tu.username,
                displayName: tu.displayName || tu.username,
                profileImageUrl: tu.profileImageUrl,
            });
        }
    };

```

```

setCanMessage(Boolean(resp.canMessage));
setLockedReason(!resp.canMessage ? (resp.reason || resp.message || 'Messaging is locked') : null);

const rid = resp.chatRoom?.id != null ? Number(resp.chatRoom.id) : null;
setRoomId(rid);

// Reset pagination cursors
setHasMoreOlder(true);
oldestTimestampRef.current = null;

// Only load the most recent page (WhatsApp style)
if (rid && resp.canMessage) {
  const page = await chatApi.listMessages(rid, { size: PAGE_SIZE });
  if (!mounted) return;
  if (reqId !== threadRequestSeqRef.current) return;

  // backend returns newest-first
  const newestFirst = page.messages || [];
  const chron = [...newestFirst].reverse();

  const mapped = chron.map((m: ApiChatMessage, idx: number) => ({
    id: idx + 1,
    roomId: Number(m.roomId),
    senderUsername: m.senderUsername,
    senderDisplayName: m.senderDisplayName,
    content: m.content,
    timestamp: m.timestamp,
  }));
  setMessages(mapped);

  // Open the chat at the most recent message
  scrollToBottom('auto');

  const oldest = chron[0]?.timestamp;
  oldestTimestampRef.current = oldest || null;
  setHasMoreOlder(Boolean(page.hasMore) && newestFirst.length > 0);
} else {
  // Locked or no room: show whatever backend included (usually none).
  const loadedMessages = (resp.messages || []).map((m: ApiChatMessage, idx: number) => ({
    id: idx + 1,
    roomId: Number(m.roomId),
    senderUsername: m.senderUsername,
    senderDisplayName: m.senderDisplayName,
    content: m.content,
    timestamp: m.timestamp,
  }));
  setMessages(loadedMessages);

  // If backend returned a thread, still start at the bottom
  scrollToBottom('auto');

  oldestTimestampRef.current = loadedMessages[0]?.timestamp || null;
  setHasMoreOlder(false);
}

// Realtime wiring only after the thread is confirmed current.
const allowed = Boolean(resp.canMessage);
if (allowed && rid) {
  try {
    const rt = new ChatRealtimeClient();
    realtimeRef.current = rt;
    await rt.connect();
    if (!mounted) return;
    if (reqId !== threadRequestSeqRef.current) return;

    subRef.current = rt.subscribe(rid, (msg) => {
      setMessages((prev) => {
        const key = `${msg.timestamp}|${msg.senderUsername}|${msg.content}`;
        const seen = prev.some((p) => `${p.timestamp}|${p.senderUsername}|${p.content}` === key);
        if (seen) return prev;
        return [

```

```

        ...prev,
        {
            id: Date.now(),
            roomId: msg.roomId,
            senderUsername: msg.senderUsername,
            senderDisplayName: msg.senderDisplayName,
            content: msg.content,
            timestamp: msg.timestamp,
        },
    ];
    });
    void refreshContacts();
    });
} catch (e) {
    console.warn('Realtime chat unavailable; falling back to polling', e);
    pollRef.current = window.setInterval(async () => {
        if (!selectedContactIdSafe) return;
        try {
            const refreshed = await chatApi.refreshThread(selectedContactIdSafe);
            if (!mounted) return;
            if (reqId !== threadRequestSeqRef.current) return;

            const next = (refreshed.messages || []).map((m: ApiChatMessage, idx: number) => ({
                id: idx + 1,
                roomId: Number(m.roomId),
                senderUsername: m.senderUsername,
                senderDisplayName: m.senderDisplayName,
                content: m.content,
                timestamp: m.timestamp,
            }));
            setMessages(next);
        } catch {
            // ignore
        }
    }, 3000);
}
} catch (e) {
    console.error('Failed to open chat:', e);
    if (!mounted) return;
    if (reqId !== threadRequestSeqRef.current) return;
    setTargetUser(null);
    setMessages([]);
    setRoomId(null);
    setCanMessage(false);
    setLockedReason(null);
    setHasMoreOlder(true);
    oldestTimestampRef.current = null;
} finally {
    if (mounted && reqId === threadRequestSeqRef.current) setIsLoadingThread(false);
}
};

void loadThread();
return () => {
    mounted = false;
};
}, [selectedContactIdSafe, isAuthenticated, refreshContacts, scrollToBottom]);

const loadOlder = useCallback(async () => {
    if (!roomId) return;
    if (!canMessage) return;
    if (!hasMoreOlder) return;
    if (isLoadingOlder) return;
    const before = oldestTimestampRef.current;
    if (!before) return;

    const scroller = scrollContainerRef.current;
    const prevScrollHeight = scroller?.scrollHeight || 0;
    const prevScrollTop = scroller?.scrollTop || 0;
    try {

```

```

    setIsLoadingOlder(true);
    const page = await chatApi.listMessages(roomId, { size: PAGE_SIZE, before });
    const newestFirst = page.messages || [];
    // if none returned, we are done
    if (newestFirst.length === 0) {
        setHasMoreOlder(false);
        return;
    }

    const chron = [...newestFirst].reverse();
    const olderMapped: Message[] = chron.map((m: ApiChatMessage, idx: number) => ({
        id: Date.now() + idx,
        roomId: Number(m.roomId),
        senderUsername: m.senderUsername,
        senderDisplayName: m.senderDisplayName,
        content: m.content,
        timestamp: m.timestamp,
    }));

    setMessages((prev) => {
        const existingKeys = new Set(prev.map((p) => `${p.timestamp}|${p.senderUsername}|${p.content}`));
        const filtered = olderMapped.filter((m) => !existingKeys.has(`${m.timestamp}|${m.senderUsername}|${m.content}`));
        return [...filtered, ...prev];
    });

    oldestTimestampRef.current = chron[0]?.timestamp || before;
    setHasMoreOlder(Boolean(page.hasMore) && newestFirst.length > 0);

    // Preserve scroll position after prepending
    requestAnimationFrame(() => {
        const nextScrollHeight = scroller?.scrollHeight || 0;
        if (scroller) {
            scroller.scrollTop = prevScrollTop + (nextScrollHeight - prevScrollHeight);
        }
    });
} catch (e) {
    console.error('Failed to load older messages:', e);
} finally {
    setIsLoadingOlder(false);
}
}, [PAGE_SIZE, canMessage, hasMoreOlder, isLoadingOlder, roomId]);

// Trigger loadOlder when user scrolls near the top
useEffect(() => {
    const scroller = scrollContainerRef.current;
    if (!scroller) return;

    const onScroll = () => {
        if (scroller.scrollTop <= 80) {
            void loadOlder();
        }
    };

    scroller.addEventListener('scroll', onScroll, { passive: true });
    return () => {
        scroller.removeEventListener('scroll', onScroll);
    };
}, [loadOlder]);

const handleContactClick = (contactId: number) => {
    navigate(`/chat/${contactId}`);
};

const handleFollowAndRetry = async () => {
    if (!selectedContactId) return;
    try {
        await followApi.follow(selectedContactId);
        const resp = await chatApi.openWith(selectedContactId);
        const tu = resp.targetUser;
        setTargetUser({
            id: Number(tu.id),

```

```

        username: tu.username,
        displayName: tu.displayName || tu.username,
        profileImageUrl: tu.profileImageUrl,
    });
    setCanMessage(Boolean(resp.canMessage));
    setLockedReason(!resp.canMessage ? (resp.reason || resp.message || 'Messaging is locked') : null);
    setRoomId(resp.chatRoom?.id !== null ? Number(resp.chatRoom.id) : null);
    const loadedMessages = (resp.messages || []).map((m: ApiChatMessage, idx: number) => ({
        id: idx + 1,
        roomId: Number(m.roomId),
        senderUsername: m.senderUsername,
        senderDisplayName: m.senderDisplayName,
        content: m.content,
        timestamp: m.timestamp,
    }));
    setMessages(loadedMessages);
} catch (e) {
    console.error('Follow failed:', e);
}
};

const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();

    if (!canMessage || !roomId) return;

    const content = messageInput.trim();
    if (!content) return;

    setMessageInput('');

    // 1) Always persist via REST so the recipient can load it even if WS fails.
    try {
        const saved = await chatApi.send(roomId, content);
        setMessages((prev) => {
            const key = `${saved.timestamp}|${saved.senderUsername}|${saved.content}`;
            const seen = prev.some((p) => `${p.timestamp}|${p.senderUsername}|${p.content}` === key);
            if (seen) return prev;
            return [
                ...prev,
                {
                    id: Date.now(),
                    roomId: Number(saved.roomId),
                    senderUsername: saved.senderUsername,
                    senderDisplayName: saved.senderDisplayName,
                    content: saved.content,
                    timestamp: saved.timestamp,
                },
            ];
        });

        // Update conversation list (so left panel shows latest preview and ordering)
        void refreshContacts();

        // IMPORTANT: don't also send over WS on success.
        // The REST endpoint already broadcasts to `/topic/chat/{roomId}`.
        // Sending again would create a second DB row via the WS handler.
        return;
    } catch (err) {
        console.warn('REST send failed; falling back to realtime only', err);
    }

    // 2) If REST fails (rare), try WS send as a last resort.
    const rt = realtimeRef.current;
    if (rt) {
        try {
            rt.send(roomId, content);
            return;
        } catch (err) {
            console.warn('WebSocket send failed', err);
        }
    }
}

```

```

    }

    // 3) Last resort optimistic UI so the user sees something.
    const newMessage: Message = {
      id: Date.now(),
      roomId,
      senderUsername: currentUser?.username || 'you',
      senderDisplayName: currentUser?.name || 'You',
      content,
      timestamp: new Date().toISOString(),
    };

    setMessages((prev) => [...prev, newMessage]);
  };

  const formatTime = (timestamp: string) => {
    const date = new Date(timestamp);
    if (!Number.isFinite(date.getTime())) return '';
    return (
      date.toLocaleDateString('en-US', { month: 'short', day: 'numeric' }) +
      ', ' +
      date.toLocaleTimeString('en-US', {
        hour: 'numeric',
        minute: '2-digit',
        hour12: true,
      })
    );
  };

  const formatLastMessageTime = (timestamp: string) => {
    const date = new Date(timestamp);
    if (!Number.isFinite(date.getTime())) return '';

    const now = new Date();
    const diffMs = now.getTime() - date.getTime();
    const diffMins = Math.floor(diffMs / 60000);
    const diffHours = Math.floor(diffMs / 3600000);
    const diffDays = Math.floor(diffMs / 86400000);

    if (diffMins < 60) return `${diffMins}m ago`;
    if (diffHours < 24) return `${diffHours}h ago`;
    if (diffDays < 7) return `${diffDays}d ago`;
    return date.toLocaleDateString('en-US', { month: 'short', day: 'numeric' });
  };

  const filteredContacts = contacts.filter((c) => {
    const q = contactQuery.trim().toLowerCase();
    if (!q) return true;
    return (
      c.displayName.toLowerCase().includes(q) ||
      c.username.toLowerCase().includes(q)
    );
  });

  return (
    <>
    <Navigation />
    {/* Use the app-wide animated gradient background (same as Login). */}
    <div className="min-h-screen p-4">
      <div className="max-w-7xl mx-auto px-4 py-8">
        {/* Constrain the chat layout height and prevent page growth; panes scroll internally */}
        <div className="grid grid-cols-1 md:grid-cols-12 gap-4 h-[calc(100vh-180px)] overflow-hidden">
          {/* Contacts List */}
          <div className="md:col-span-4 lg:col-span-3 min-h-0">
            <Card className="h-full flex flex-col min-h-0 gradient-mesh-card">
              <div className="p-4 border-b border-border">
                <Input
                  value={contactQuery}
                  onChange={(e) => setContactQuery(e.target.value)}
                  placeholder="Search"
                  className="w-full"
                />
              </div>
            </Card>
          </div>
        </div>
      </div>
    </div>
  );

```

```

        autoComplete="off"
      />
    </div>
    <div className="flex-1 min-h-0 overflow-y-auto no-scrollbar">
      {isLoadingContacts ? (
        <div className="p-8 text-center text-muted-foreground">
          <p>Loading conversations...</p>
        </div>
      ) : filteredContacts.length === 0 ? (
        <div className="p-8 text-center text-muted-foreground">
          <MessageCircle className="h-12 w-12 mx-auto mb-3 opacity-50" />
          <p>No conversations found</p>
        </div>
      ) : (
        <>
        {filteredContacts.map((contact) => (
          <div
            key={contact.id}
            onClick={() => handleContactClick(contact.id)}
            className={
              `p-4 border-b border-border cursor-pointer transition-colors ` +
              `hover:text-white ` +
              `hover:bg-linear-to-r hover:from-[rgb(75,85,99)] hover:via-[rgb(55,65,81)] hover:to-[rgb(31,41,55)]` +
              `dark:hover:bg-linear-to-r dark:hover:from-[rgb(34,211,238)] dark:hover:via-[rgb(6,182,212)]` +
              `${selectedContactId === contact.id ? 'bg-muted' : ''}`
            }
          >
            <div className="flex items-center gap-3">
              <Avatar className="h-12 w-12">
                <AvatarImage src={contact.avatar || contact.profileImageUrl} />
                <AvatarFallback>
                  <DefaultAvatarFallback alt={contact.displayName} />
                </AvatarFallback>
              </Avatar>
              <div className="flex-1 min-w-0">
                <div className="flex items-center justify-between mb-1">
                  <h3 className="font-semibold text-sm truncate">{contact.displayName}</h3>
                  {contact.lastMessageTime ? (
                    <span className="text-xs text-muted-foreground">
                      {formatLastMessageTime(contact.lastMessageTime)}
                    </span>
                  ) : null}
                </div>
                <div className="flex items-center justify-between">
                  <p className="text-sm text-muted-foreground truncate">
                    {contact.lastMessage || 'No messages yet'}
                  </p>
                </div>
              </div>
            </div>
          </div>
        ))}
        </>
      </div>
    </div>
  </div>
</div>

{/* Chat Area */}
<div className="md:col-span-8 lg:col-span-9 min-h-0">
  {!targetUser ? (
    <Card className="h-full flex items-center justify-center gradient-mesh-card">
      <div className="text-center text-muted-foreground p-8">
        <MessageCircle className="h-16 w-16 mx-auto mb-4 opacity-30" />
        <h3 className="text-xl font-semibold mb-2">
          {isLoadingThread ? 'Loading conversation...' : 'Select a conversation'}
        </h3>
        <p>Choose a contact from the list to start chatting</p>
      </div>
    </Card>
  ) : (

```



```

<Card className="h-full flex flex-col min-h-0 gradient-mesh-card">
  <div className="p-4 bg-muted/60 border-b border-border flex items-center gap-4">
    <Avatar className="h-12 w-12 border-2 border-border">
      <AvatarImage src={targetUser.avatar || targetUser.imageUrl} />
      <AvatarFallback>
        <DefaultAvatarFallback alt={targetUser.displayName} />
      </AvatarFallback>
    </Avatar>
    <div>
      <h5 className="font-semibold text-lg">{targetUser.displayName}</h5>
      <small className="text-muted-foreground">@{targetUser.username}</small>
    </div>
  </div>

  <div
    className="flex-1 min-h-0 overflow-y-auto no-scrollbar overscroll-contain p-4 bg-background/30"
    ref={scrollContainerRef}
  >
    {isLoadingThread ? (
      <div className="h-full flex items-center justify-center text-sm text-muted-foreground">
        Loading...
      </div>
    ) : (
      <>
        <div ref={topSentinelRef} className="h-2" />
        <div className="h-8 flex items-center justify-center text-xs text-muted-foreground">
          {canMessage ? (
            isLoadingOlder ? (
              <span>Loading older messages...</span>
            ) : hasMoreOlder ? null : (
              <span>Start of conversation</span>
            )
          ) : null}
        </div>

        {messages.map((msg) => {
          const cuUsername = (currentUser?.username || '').toLowerCase();
          const cuName = (currentUser?.name || '').toLowerCase();
          const senderUsername = (msg.senderUsername || '').toLowerCase();
          const senderDisplayName = (msg.senderDisplayName || '').toLowerCase();
          const targetUsername = (targetUser?.username || '').toLowerCase();
          const targetDisplayName = (targetUser?.displayName || '').toLowerCase();

          // In a 1:1 thread, treat messages from the opened target user as RECEIVED (left/grey).
          // Everything else is SENT (right/green). This avoids ambiguous username/displayName mismatch
          const isFromTarget =
            (targetUsername && senderUsername && senderUsername === targetUsername) ||
            (!targetUsername && targetDisplayName && senderDisplayName && senderDisplayName === targetDisplayName)

          // If we can't determine (rare), fall back to matching current user.
          const isFromMe =
            (cuUsername && senderUsername && senderUsername === cuUsername) ||
            (!cuUsername && cuName && senderDisplayName && senderDisplayName === cuName)

          const isSent = isFromTarget ? false : isFromMe ? true : true;

          return (
            <div
              key={msg.id}
              className={'mb-4 flex ' + (isSent ? 'justify-end' : 'justify-start')}
            >
              <div className="flex flex-col min-w-0 max-w-[min(80%,44rem)]">
                <div
                  className={
                    'py-2.5 px-4 rounded-2xl whitespace-pre-wrap break-anywhere ' +
                    (isSent
                      ? 'bg-[#2e7d58] text-white rounded-br-md'
                      : 'bg-muted/60 text-foreground rounded-bl-md')
                  }
                >
                  {msg.content}
                </div>
              </div>
            </div>
          )
        })}
      </>
    )}
  </div>

```

```

        </div>
      </div>
      <div
        className={
          'text-[11px] mt-1 text-muted-foreground ' +
            (isSent ? 'text-right' : 'text-left')
        }
      >
        {formatTime(msg.timestamp)}
      </div>
    </div>
  </div>
);
}}

<div ref={messagesEndRef} />

{!canMessage ? (
  <div className="mt-6">
    <Card className="border border-dashed">
      <div className="p-6 text-center">
        <div className="mx-auto mb-3 inline-flex items-center justify-center w-12 h-12 rounded-full">
          <Lock className="h-5 w-5 text-muted-foreground" />
        </div>
        <h4 className="text-lg font-semibold mb-1">Messaging is locked</h4>
        <p className="text-sm text-muted-foreground mb-4 break-anywhere">
          {lockedReason === 'FOLLOW_REQUIRED'
            ? 'Only people who follow each other can message.'
            : lockedReason || 'They need to follow you back to unlock messaging.'}
        </p>
        {lockedReason === 'FOLLOW_REQUIRED' ? (
          <Button onClick={handleFollowAndRetry}>Follow to unlock messages</Button>
        ) : null}
      </div>
    </Card>
  </div>
) : null}
</div>

<div className="p-4 border-t border-border bg-card/60">
  <form onSubmit={handleSubmit} className="flex gap-3">
    <Input
      type="text"
      value={messageInput}
      onChange={(e) => setMessageInput(e.target.value)}
      placeholder={canMessage ? 'Type a message...' : isLoadingThread ? 'Loading...' : 'Messaging is locked'}
      className="flex-1 rounded-full px-5"
      autoComplete="off"
      disabled={!canMessage || isLoadingThread}
    />
    <Button
      type="submit"
      className="rounded-full w-12 h-12 p-0 flex items-center justify-center"
      disabled={!canMessage || isLoadingThread}
    >
      <Send className="w-5 h-5" />
    </Button>
  </form>
</div>
</Card>
)}
</div>
</div>
</div>
</div>
</div>
);
}

```

■ frontend/src/pages/ChatDashboard.tsx

```
import { useState, useEffect } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card, CardContent } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar';
import { MessageCircle, ArrowLeft, Home } from 'lucide-react';
import { useAuthContext } from '@contexts/AuthContext';
import { format } from 'date-fns';
import { DefaultAvatarFallback } from '@components/DefaultAvatarFallback';
import { chatApi, ChatRoomSummary } from '@lib/api';

export default function ChatDashboard() {
  const navigate = useNavigate();
  const { user } = useAuthContext();
  const [chatRooms, setChatRooms] = useState<ChatRoomSummary[]>([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    if (!user) {
      navigate('/login');
    }
  }, [user, navigate]);

  useEffect(() => {
    let mounted = true;

    const load = async () => {
      if (!user) return;
      try {
        setIsLoading(true);
        const resp = await chatApi.list();
        if (!mounted) return;
        setChatRooms(resp.chatRooms || []);
      } catch (e) {
        console.error('Failed to load chats:', e);
        if (!mounted) return;
        setChatRooms([]);
      } finally {
        if (mounted) setIsLoading(false);
      }
    };

    void load();
    return () => {
      mounted = false;
    };
  }, [user]);

  if (!user) {
    return null;
  }

  return (
    <div className="min-h-screen bg-background">
      <Navigation />

      <main className="container mx-auto px-4 py-6 max-w-4xl">
        <Card className="gradient-mesh-card overflow-hidden">
          {/ * Header */}
          <div className="bg-gradient-to-r from-purple-600 to-indigo-700 text-white p-6">
            <h2 className="text-3xl font-bold mb-2 flex items-center gap-2">
              <MessageCircle className="h-8 w-8" />
              My Chats
            </h2>
            <Link
              to="/"
              className="inline-flex items-center gap-2 text-white/90 hover:text-white transition-colors"
            >
              <ArrowLeft className="h-4 w-4" />

```

```

        Back to Home
        </Link>
    </div>

    { /* Chat List */
    <CardContent className="p-0">
        {isLoading ? (
            <div className="text-center py-16 px-4">
                <p className="text-muted-foreground">Loading conversations...</p>
            </div>
        ) : chatRooms.length === 0 ? (
            <div className="text-center py-16 px-4">
                <MessageCircle className="h-20 w-20 mx-auto mb-6 text-muted-foreground opacity-50" />
                <h4 className="text-xl font-semibold mb-2">No Conversations Yet</h4>
                <p className="text-muted-foreground mb-6">
                    Start following users and send them messages to begin chatting!
                </p>
                <Button asChild>
                    <Link to="/">
                        <Home className="h-4 w-4 mr-2" />
                        Explore Posts
                    </Link>
                </Button>
            </div>
        ) : (
            <div>
                {chatRooms.map((chat) => (
                    <Link
                        key={chat.id}
                        to={` /chat/${chat.otherUserId}`}
                        className="flex items-center gap-4 p-6 border-b last:border-b-0 hover:bg-muted/30 transition-colors">
                        <Avatar className="h-14 w-14 border-2 border-border">
                            <AvatarImage src={chat.otherUserProfileImage} />
                            <AvatarFallback>
                                <DefaultAvatarFallback alt={chat.otherUserDisplayName} />
                            </AvatarFallback>
                        </Avatar>

                        <div className="flex-1 min-w-0">
                            <div className="font-semibold text-lg mb-1">
                                {chat.otherUserDisplayName}
                            </div>
                            <div>
                                {chat.lastMessage & & (
                                    <div className="text-sm text-muted-foreground truncate">
                                        {chat.lastMessage}
                                    </div>
                                )}
                            </div>

                            {chat.lastMessageTime & & (
                                <div className="text-right text-sm text-muted-foreground flex-shrink-0">
                                    <div>{format(new Date(chat.lastMessageTime), 'MMM dd')}</div>
                                    <div className="text-xs">
                                        {format(new Date(chat.lastMessageTime), 'hh:mm a')}
                                    </div>
                                </div>
                            )}
                        </Link>
                    )})
                </div>
            </CardContent>
        </Card>
    </main>
    </div>
    );
}

```

■ frontend/src/pages/EditProfile.tsx

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Label } from '@components/ui/label';
import { Textarea } from '@components/ui/textarea';
import { Checkbox } from '@components/ui/checkbox';
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar';
import { ArrowLeft } from 'lucide-react';
import { useAuthContext } from '@contexts/AuthContext';
import { apiClient, ApiError } from '@lib/api';
import { toast } from 'sonner';
import { DefaultAvatarFallback } from '@components/DefaultAvatarFallback';

export default function EditProfile() {
  const navigate = useNavigate();
  const { user, checkAuth } = useAuthContext();

  const [formData, setFormData] = useState({
    username: user?.username || '',
    displayName: user?.name || '',
    bio: '',
    linkedin: '',
    github: '',
    website: '',
    profilePhoto: user?.avatar || '',
  });

  const [profileImageFile, setProfileImageFile] = useState<File | null>(null);
  const [removeProfileImage, setRemoveProfileImage] = useState(false);
  const [errors, setErrors] = useState({
    username: '',
    profileImage: '',
  });

  const handleImageChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (file) {
      const allowedTypes = ['image/jpeg', 'image/jpg', 'image/png', 'image/webp'];
      if (!allowedTypes.includes(file.type)) {
        setErrors({
          ...errors,
          profileImage: 'Invalid file type. Allowed: JPG, JPEG, PNG, WEBP',
        });
        return;
      }

      if (file.size > 2 * 1024 * 1024) {
        setErrors({
          ...errors,
          profileImage: 'File size must be less than 2MB',
        });
        return;
      }

      setProfileImageFile(file);
      setErrors({ ...errors, profileImage: '' });
    }
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setErrors({ username: '', profileImage: '' });

    const usernamePattern = /^[A-Za-z0-9._-]{3,30}$/;
    if (!usernamePattern.test(formData.username)) {
      setErrors({
        ...errors,

```

```

        username: 'Username must be 3-30 characters and contain only letters, numbers, dots, underscores, and hyphens',
    });
    return;
}

if (formData.username.includes('@')) {
    setErrors({
        ...errors,
        username: 'Username cannot be an email address',
    });
    return;
}

const data = new FormData();
data.append('username', formData.username);
data.append('displayName', formData.displayName);
data.append('bio', formData.bio);
data.append('linkedin', formData.linkedin);
data.append('github', formData.github);
data.append('website', formData.website);
data.append('removeProfileImage', String(removeProfileImage));

if (profileImageFile) {
    data.append('profileImageFile', profileImageFile);
}

try {
    // Backend expects multipart/form-data via @RequestParam
    await apiClient('/profile', { method: 'PUT', body: data });

    // Refresh the authenticated user from /auth/status so navbar gets the new avatar.
    await checkAuth();

    toast.success('Profile updated');
    navigate('/profile');
} catch (err) {
    console.error('Failed to update profile:', err);

    if (err instanceof ApiError) {
        // If backend returns field-specific error (e.g., username), respect it.
        if (err.message.toLowerCase().includes('username')) {
            setErrors({ ...errors, username: err.message });
        } else {
            toast.error(err.message || 'Failed to update profile');
        }
    } else {
        toast.error('Failed to update profile');
    }
}
};

return (
    <>
    <Navigation />
    <div className="min-h-screen pb-8">
        <div className="max-w-4xl mx-auto px-4 py-8">
            <div className="flex justify-between items-center mb-6">
                <h1 className="text-2xl font-semibold">Edit Profile</h1>
            </div>
            <Button variant="outline" size="sm" onClick={() => navigate('/profile')}>
                <ArrowLeft className="w-4 h-4 mr-2" />
                Back to Profile
            </Button>
        </div>
    </div>

    <Card className="gradient-mesh-card p-8">
        <form onSubmit={handleSubmit} className="space-y-6">
            <div className="text-center">
                <Label className="block mb-3 text-base font-semibold">Current Profile Image</Label>
                <div className="inline-block">
                    <Avatar className="w-24 h-24 border-2 border-gray-200">
                        <AvatarImage src={user?.avatar} />

```

```

        <AvatarFallback>
          <DefaultAvatarFallback alt={user?.name || 'Default profile image'} />
        </AvatarFallback>
      </Avatar>
    </div>
  </div>

  <div>
    <Label htmlFor="profileImageFile" className="text-base font-semibold mb-2">
      Upload New Profile Image
    </Label>
    <Input
      id="profileImageFile"
      type="file"
      accept="image/jpeg, image/jpg, image/png, image/webp"
      onChange={handleImageChange}
      className="mt-2"
    />
    <div className="text-sm text-gray-600 mt-2">
      Allowed types: JPG, JPEG, PNG, WEBP. Max size ~2MB.
    </div>
    {errors.profileImage && (
      <div className="mt-2 p-3 bg-red-50 border border-red-200 rounded text-red-800 text-sm">
        {errors.profileImage}
      </div>
    )}
  </div>

  <div className="flex items-center space-x-2">
    <Checkbox
      id="removeProfileImage"
      checked={removeProfileImage}
      onChange={(checked) => setRemoveProfileImage(checked as boolean)}
    />
    <Label htmlFor="removeProfileImage" className="cursor-pointer">
      Remove current profile image
    </Label>
  </div>

  <div>
    <Label className="text-base font-semibold mb-2">Email (readonly)</Label>
    <Input
      type="email"
      value={user?.email || ''}
      readOnly
      className="mt-2 bg-gray-50"
    />
  </div>

  <div>
    <Label htmlFor="username" className="text-base font-semibold mb-2">
      Username
    </Label>
    <Input
      id="username"
      type="text"
      value={formData.username}
      onChange={(e) => setFormData({ ...formData, username: e.target.value })}
      minLength={3}
      maxLength={30}
      pattern="^[A-Za-z0-9._-]{3,30}$"
      required
      className="mt-2"
    />
    <div className="text-sm text-gray-600 mt-2">
      3-30 characters. Allowed: letters, numbers, dots, underscores, hyphens. Cannot be an email address.
    </div>
    {errors.username && (
      <div className="mt-2 p-3 bg-red-50 border border-red-200 rounded text-red-800 text-sm">
        {errors.username}
      </div>
    )}
  </div>

```

```

    })
    </div>

    <div>
      <Label htmlFor="displayName" className="text-base font-semibold mb-2">
        Display Name
      </Label>
      <Input
        id="displayName"
        type="text"
        value={formData.displayName}
        onChange={(e) => setFormData({ ...formData, displayName: e.target.value })}
        maxLength={100}
        className="mt-2"
      />
    </div>

    <div>
      <Label htmlFor="bio" className="text-base font-semibold mb-2">
        Bio
      </Label>
      <Textarea
        id="bio"
        value={formData.bio}
        onChange={(e) => setFormData({ ...formData, bio: e.target.value })}
        maxLength={512}
        rows={3}
        className="mt-2"
      />
      <div className="text-sm text-gray-600 mt-2">Max 512 characters.</div>
    </div>

    <div>
      <Label htmlFor="linkedin" className="text-base font-semibold mb-2">
        LinkedIn URL
      </Label>
      <Input
        id="linkedin"
        type="url"
        value={formData.linkedin}
        onChange={(e) => setFormData({ ...formData, linkedin: e.target.value })}
        placeholder="https://linkedin.com/in/yourprofile"
        className="mt-2"
      />
    </div>

    <div>
      <Label htmlFor="github" className="text-base font-semibold mb-2">
        GitHub URL
      </Label>
      <Input
        id="github"
        type="url"
        value={formData.github}
        onChange={(e) => setFormData({ ...formData, github: e.target.value })}
        placeholder="https://github.com/yourusername"
        className="mt-2"
      />
    </div>

    <div>
      <Label htmlFor="website" className="text-base font-semibold mb-2">
        Website
      </Label>
      <Input
        id="website"
        type="url"
        value={formData.website}
        onChange={(e) => setFormData({ ...formData, website: e.target.value })}
        placeholder="https://yourwebsite.com"
        className="mt-2"
      />
    </div>

```



```

    /&gt;
    &lt;/div&gt;

    &lt;div&gt;
      &lt;Label htmlFor="profilePhoto" className="text-base font-semibold mb-2"&gt;
        Profile Image URL (optional)
      &lt;/Label&gt;
      &lt;Input
        id="profilePhoto"
        type="url"
        value={formData.profilePhoto}
        onChange={(e) =&gt; setFormData({ ...formData, profilePhoto: e.target.value })}
        placeholder="https://example.com/image.jpg"
        className="mt-2"
      /&gt;
      &lt;div className="text-sm text-gray-600 mt-2"&gt;
        Paste a direct image URL (e.g. Cloudinary). Uploaded image will be used in preference.
      &lt;/div&gt;
    &lt;/div&gt;

    &lt;div className="flex justify-end gap-3 pt-4"&gt;
      &lt;Button type="button" variant="outline" onClick={() =&gt; navigate('/profile')}&gt;
        Cancel
      &lt;/Button&gt;
      &lt;Button type="submit"&gt;
        Save Changes
      &lt;/Button&gt;
    &lt;/div&gt;
  &lt;/form&gt;
&lt;/Card&gt;
&lt;/div&gt;
&lt;/div&gt;
&lt;/&gt;
);
}

```

■ frontend/src/pages/ErrorPage.tsx

```

import { Link } from 'react-router-dom';
import { motion } from 'framer-motion';
import { Home, AlertCircle } from 'lucide-react';

interface ErrorPageProps {
  errorMessage?: string;
  path?: string;
  status?: number;
}

export default function ErrorPage({ errorMessage, path, status }: ErrorPageProps) {
  return (
    &lt;motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      transition={{ duration: 0.5 }}
      className="min-h-screen flex items-center justify-center bg-background p-4"
    &gt;
      &lt;div className="max-w-2xl text-center"&gt;
        &lt;div className="mb-6"&gt;
          &lt;AlertCircle className="h-24 w-24 mx-auto text-destructive" /&gt;
        &lt;/div&gt;
        &lt;h1 className="text-9xl font-light mb-4 text-destructive"&gt;Error&lt;/h1&gt;
        &lt;h2 className="text-2xl font-normal mb-6 text-muted-foreground"&gt;
          Something Went Wrong
        &lt;/h2&gt;

        {errorMessage &amp;&amp; (
          &lt;div className="bg-muted p-4 rounded-lg mb-4 font-mono text-sm text-left"&gt;
            {errorMessage}
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/motion.div&gt;
  );
}

```

```

    {path &&& (
      <div className="bg-muted p-4 rounded-lg mb-4 font-mono text-sm text-left">
        Path: {path}
      </div>
    )}

    {status &&& (
      <div className="bg-muted p-4 rounded-lg mb-4 font-mono text-sm text-left">
        Status: {status}
      </div>
    )}

    <p className="text-lg mb-4 text-muted-foreground">
      We are sorry for the inconvenience. Our technical team has been notified.
    </p>

    <Link
      to="/"
      className="inline-flex items-center gap-2 text-primary font-bold hover:underline text-lg"
    >
      <Home className="h-5 w-5" />
      Return to Homepage
    </Link>
  </div>
  </motion.div>
);
}

```

■ frontend/src/pages/Feed.tsx

```

import { useEffect, useState, useCallback, useRef } from 'react';
import { Navigation } from '@components/Navigation';
import { RightSidebar } from '@components/RightSidebar';
import { postsApi, usersApi, User, Post, ApiError, UNAUTHENTICATED_MESSAGE, isUnauthenticatedStatus } from '@lib/api';
import { useAuthContext } from '@contexts/AuthContext';
import { Card, CardContent } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar';
import { Loader2, Heart, MessageCircle, Calendar, Share2, Search } from 'lucide-react';
import { Link, useSearchParams } from 'react-router-dom';
import { formatDistanceToNow } from 'date-fns';
import { toast } from 'sonner';
import { Input } from '@components/ui/input';
import { ImageCarousel } from '@components/ImageCarousel';
import { AutoPlayMutedVideo } from '@components/AutoPlayMutedVideo';
import { DefaultAvatarFallback } from '@components/DefaultAvatarFallback';
import { usePostRealtime } from '@hooks/usePostRealtime';
import { formatCount } from '@lib/formatCount';
import { VerifiedBadge } from '@components/VerifiedBadge';

export default function Feed() {
  const { isAuthenticated, user } = useAuthContext();
  const [searchParams, setSearchParams] = useSearchParams();
  const [posts, setPosts] = useState<Post[]>([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const [page, setPage] = useState(0);
  const [hasMorePosts, setHasMorePosts] = useState(true);
  const [isLoadingMore, setIsLoadingMore] = useState(false);
  const loadMoreRef = useRef<HTMLDivElement | null>(null);
  const pageSize = 10;

  const postsSearchKeyword = searchParams.get('search') || '';
  const userSearchKeyword = searchParams.get('userSearch') || '';

  const [userResults, setUserResults] = useState<User[]>([]);
  const [userSearchInput, setUserSearchInput] = useState(userSearchKeyword);
  const [isUserLoading, setIsUserLoading] = useState(false);
  const [userError, setUserError] = useState<string | null>(null);

```

```

const [sortBy, setSortBy] = useState('newest');

const inFlightRef = useRef(false);
const abortRef = useRef<AbortController | null>(null);

useEffect(() => {
  setUserSearchInput(userSearchKeyword);
}, [userSearchKeyword]);

const fetchUsers = useCallback(async () => {
  if (!userSearchKeyword.trim()) return;
  try {
    setIsUserLoading(true);
    setUserError(null);
    const resp = await usersApi.search(userSearchKeyword.trim());
    setUserResults(resp.users || []);
  } catch (err) {
    console.error('Failed to search users:', err);
    setUserResults([]);
    setUserError('Failed to load users. Please try again later.');
```

toast.error('Failed to load users');

```

  } finally {
    setIsUserLoading(false);
  }
}, [userSearchKeyword]);

const fetchPostsPage = useCallback(async (opts: { page: number; append: boolean }) => {
  // Prevent overlapping requests (but don't block initial load forever)
  if (inFlightRef.current) return;
  inFlightRef.current = true;

  // Cancel any previous request if still running
  abortRef.current?.abort();
  const controller = new AbortController();
  abortRef.current = controller;

  try {
    if (opts.append) setIsLoadingMore(true);
    else setIsLoading(true);

    // Ask postsApi for both items + paging metadata in one shot.
    const { items, pageInfo } = await postsApi.fetchPostsPaged({
      page: opts.page,
      size: pageSize,
      sortBy,
      keyword: postsSearchKeyword || undefined,
      signal: controller.signal,
    });

    setPosts((prev) => (opts.append ? [...prev, ...items] : items));
    setError(null);

    if (pageInfo) {
      setHasMorePosts(!pageInfo.last);
      setPage(pageInfo.pageNumber);
    } else {
      setHasMorePosts(items.length === pageSize);
      setPage(opts.page);
    }
  } catch (err: unknown) {
    // Ignore aborts
    if (err instanceof DOMException && err.name === 'AbortError') {
      return;
    }

    const status = err instanceof ApiError ? err.status : undefined;
    console.error('Failed to fetch posts:', err);

    if (isUnauthenticatedStatus(status)) {
      // Logged-out users: load only the first page, then gate further loads.
      try {

```

```

        const data = await postsApi.getPublic();
        setPosts(data);
        setError(null);
        setHasMorePosts(false);
        setPage(0);
        return;
    } catch (fallbackErr) {
        console.error('Fallback to public posts failed:', fallbackErr);
    }
}

const message = isUnauthenticatedStatus(status)
    ? UNAUTHENTICATED_MESSAGE
    : 'Failed to load posts. Please try again later.';

setError(message);
toast.error(message);
} finally {
    inFlightRef.current = false;
    setIsLoading(false);
    setIsLoadingMore(false);
}
}, [pageSize, postsSearchKeyword, sortBy]);

// Reset pagination whenever sort/search changes.
useEffect(() => {
    if (userSearchKeyword.trim()) return; // user search mode uses different list
    setPosts([]);
    setPage(0);
    setHasMorePosts(true);
    fetchPostsPage({ page: 0, append: false });
}, [fetchPostsPage, sortBy, postsSearchKeyword, userSearchKeyword]);

// Infinite scroll: only for authenticated users and only for posts list mode.
useEffect(() => {
    if (!isAuthenticated) return;
    if (userSearchKeyword.trim()) return;
    if (!hasMorePosts) return;
    if (isLoadingMore || isLoading) return;

    const el = loadMoreRef.current;
    if (!el) return;

    const observer = new IntersectionObserver(
        (entries) => {
            const entry = entries[0];
            if (!entry?.isIntersecting) return;
            if (inFlightRef.current) return;
            fetchPostsPage({ page: page + 1, append: true });
        },
        { root: null, rootMargin: '300px', threshold: 0 }
    );

    observer.observe(el);
    return () => observer.disconnect();
}, [fetchPostsPage, hasMorePosts, isAuthenticated, isLoading, isLoadingMore, page, userSearchKeyword]);

const handleLike = async (postId: number) => {
    if (!isAuthenticated) {
        toast.error('Please login to like posts');
        return;
    }
    try {
        const updated = await postsApi.like(postId);

        // like endpoint returns { id, likes, liked }
        const nextLikedByMe = typeof (updated as any).liked === 'boolean'
            ? (updated as any).liked
            : undefined;

        setPosts(prev => prev.map(p => {

```

```

        if (p.id !== updated.id) return p;
        return {
            ...p,
            likes: updated.likes,
            likedByMe: nextLikedByMe ?? !p.likedByMe,
        };
    }));

    toast.success((nextLikedByMe ?? true) ? 'Post liked!' : 'Post unliked');
} catch (error) {
    console.error('Failed to like post:', error);
    if (error instanceof ApiError) {
        const msg = isUnauthenticatedStatus(error.status)
            ? UNAUTHENTICATED_MESSAGE
            : (error.message || 'Failed to like post');
        toast.error(msg);
    } else {
        toast.error('Failed to like post');
    }
}
};

const handleShare = (postId: number) => {
    const url = `${window.location.origin}/post/${postId}`;
    navigator.clipboard.writeText(url).then(() => {
        toast.success('Link copied to clipboard!');
    });
};

const clearUserSearch = () => {
    const next = new URLSearchParams(searchParams);
    next.delete('userSearch');
    setSearchParams(next);
};

const submitUserSearch = (e: React.FormEvent) => {
    e.preventDefault();
    const q = userSearchInput.trim();
    const next = new URLSearchParams(searchParams);
    if (q) {
        next.set('userSearch', q);
        next.delete('search');
    } else {
        next.delete('userSearch');
    }
    setSearchParams(next);
};

const filteredPosts = posts
    .filter((post) => {
        const q = postsSearchKeyword.trim().toLowerCase();
        if (!q) return true;

        const haystacks = [
            post.title,
            post.content,
            post.author?.name,
        ]
        .filter((v): v is string => typeof v === 'string' && v.trim().length > 0)
        .map((v) => v.toLowerCase());

        return haystacks.some((h) => h.includes(q));
    })
    .sort((a, b) => {
        switch (sortBy) {
            case 'oldest':
                return new Date(a.createdAt).getTime() - new Date(b.createdAt).getTime();
            case 'mostLiked':
                return b.likes - a.likes;
            case 'mostViewed':
                return (b.views ?? 0) - (a.views ?? 0);
        }
    });

```

```

        default: // newest
            return new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime();
        }
    });

useEffect(() => {
    if (userSearchKeyword.trim()) {
        fetchUsers();
    }
}, [fetchUsers, userSearchKeyword]);

const retryPosts = () => {
    fetchPostsPage({ page: 0, append: false });
};

// Realtime: patch likes/commentCount as they change for any user.
usePostRealtime(
    filteredPosts.map((p) => p.id),
    (stats) => {
        setPosts((prev) => {
            prev.map((p) => {
                p.id === stats.postId
                    ? {
                        ...p,
                        likes: typeof stats.likes === 'number' ? stats.likes : p.likes,
                        commentCount:
                            typeof stats.commentCount === 'number'
                                ? stats.commentCount
                                : (p.commentCount ?? p.comments?.length ?? 0),
                    }
                    : p,
            }),
        );
    },
);

return (
    <div className="min-h-screen bg-background feed-animated-bg">
        <Navigation />

        {/* 3-column layout to center the feed between left edge and right sidebar */}
        <div className="mx-auto max-w-[1400px] px-4 lg:px-6 py-6">
            <div className="grid grid-cols-1 lg:grid-cols-[1fr_minmax(0,48rem)_20rem] xl:grid-cols-[1fr_minmax(0,48rem)_20rem_20rem]">
                {/* Left spacer column (keeps the main column centered between edges + sidebar) */}
                <div className="hidden lg:block" />

                {/* Main Content */}
                <main className="min-w-0">
                    {userSearchKeyword.trim() ? (
                        <>
                            <div className="flex justify-between items-center mb-6 flex-wrap gap-4">
                                <div>
                                    <h1 className="text-2xl font-bold">User Results</h1>
                                    <p className="text-sm text-muted-foreground mt-1">
                                        Showing results for: <span className="font-medium">{userSearchKeyword}</span>
                                    </p>
                                </div>
                                <div className="flex items-center gap-2">
                                    <form onSubmit={submitUserSearch} className="flex items-center gap-2">
                                        <input
                                            value={userSearchInput}
                                            onChange={(e) => setUserSearchInput(e.target.value)}
                                            placeholder="Search users..."
                                            className="w-56"
                                        />
                                        <button type="submit" variant="outline" size="sm">Search</button>
                                    </form>
                                    <button variant="ghost" size="sm" onClick={clearUserSearch}>Clear</button>
                                </div>
                            </div>
                        </>
                    ) : (
                        <div>
                            <h1>All Users</h1>
                            <p>Showing results for: <span>All Users</span>
                        </div>
                    )}
                </main>
            </div>
        </div>
    );
);

```

```

        Clear
        </Button>
    </div>
</div>

{isUserLoading ? (
    <div className="flex justify-center items-center py-12">
        <Loader2 className="h-8 w-8 animate-spin text-primary" />
    </div>
) : userError ? (
    <div className="text-center py-12 space-y-4">
        <p className="text-destructive">{userError}</p>
        <Button variant="outline" onClick={clearUserSearch}>Back to posts</Button>
    </div>
) : userResults.length === 0 ? (
    <div className="text-center py-12">
        <p className="text-muted-foreground">No users found matching your search.</p>
    </div>
) : (
    <div className="space-y-3">
        {userResults.map((u) => (
            <Card key={u.id} className="gradient-mesh-card overflow-hidden">
                <CardContent className="p-4">
                    <div className="flex items-center gap-4">
                        <Link to={` /profile/${u.id}`} className="shrink-0">
                            <Avatar className="h-12 w-12 border">
                                <AvatarImage src={u.avatar} />
                                <AvatarFallback>
                                    <DefaultAvatarFallback alt={u.name || u.username || 'Default profile image'} />
                                </AvatarFallback>
                            </Avatar>
                        </Link>
                        <div className="min-w-0 flex-1">
                            <Link
                                to={` /profile/${u.id}`}
                                className="font-semibold hover:text-primary transition-colors"
                            >
                                {u.name}
                            </Link>
                            {u.username & & (
                                <p className="text-sm text-muted-foreground truncate">@{u.username}</p>
                            )}
                            <p className="text-sm text-muted-foreground truncate">{u.email}</p>
                        </div>
                        <Button variant="outline" size="sm" asChild>
                            <Link to={` /profile/${u.id}`}>View Profile</Link>
                        </Button>
                    </div>
                </CardContent>
            </Card>
        ))}
    </div>
)}
</div>
) : (
    <div>
        { /* Toolbar: Title + Sort */ }
        <div className="flex justify-between items-center mb-6 flex-wrap gap-4">
            <div>
                <h1 className="text-2xl font-bold">Latest Posts</h1>
                {postsSearchKeyword & & (
                    <p className="text-sm text-muted-foreground mt-1">
                        Showing results for: <span className="font-medium">{postsSearchKeyword}</span>
                    </p>
                )}
            </div>
            <div className="flex items-center gap-2">
                <label htmlFor="sortBy" className="text-sm text-muted-foreground">
                    Sort by:
                </label>
                <select

```

```

        id="sortBy"
        value={sortBy}
        onChange={(e) => setSortBy(e.target.value)}
        className="px-3 py-1.5 border border-border rounded-md text-sm bg-background"
      >
        <option value="newest">Newest</option>
        <option value="oldest">Oldest</option>
        <option value="mostLiked">Most Liked</option>
        <option value="mostViewed">Most Viewed</option>
      </select>
    </div>
  </div>

  { /* Posts */
  {isloading ? (
    <div className="flex justify-center items-center py-12">
      <Loader2 className="h-8 w-8 animate-spin text-primary" />
    </div>
  ) : error ? (
    <div className="text-center py-12 space-y-4">
      <p className="text-destructive">{error}</p>
      <div className="flex justify-center gap-2">
        <Button variant="outline" onClick={retryPosts}>Retry</Button>
        {error === UNAUTHENTICATED_MESSAGE & & (
          <Button asChild variant="default">
            <Link to="/login">Log in</Link>
          </Button>
        )}
      </div>
    </div>
  ) : filteredPosts.length === 0 ? (
    <div className="text-center py-12">
      <p className="text-muted-foreground">
        {postsSearchKeyword ? 'No posts found matching your search.' : 'No posts yet. Be the first to share'}
      </p>
    </div>
  ) : (
    <div className="space-y-6">
      {filteredPosts.map((post) => (
        <Card key={post.id} className="gradient-mesh-card overflow-hidden">
          <CardContent className="p-6 relative">
            { /* Post Header */
            <div className="mb-4">
              <h2 className="text-xl font-bold mb-2">
                <Link
                  to={` /post/${post.id}`}
                  className="hover:text-primary transition-colors"
                >
                  {(post.title & & post.title.trim())
                    ? post.title
                    : `${post.content.substring(0, 100)}${post.content.length > 100 ? '...' : ''}`}
                </Link>
              </h2>
              <div className="flex flex-wrap gap-3 text-sm text-muted-foreground items-center">
                <Link
                  to={`/profile/${post.author.id}`}
                  className="flex items-center gap-2 hover:opacity-90 transition-opacity"
                  aria-label={`View profile of ${post.author.name}`}
                >
                  <Avatar className="h-6 w-6">
                    <AvatarImage src={post.author.avatar} />
                    <AvatarFallback>
                      <DefaultAvatarFallback alt={post.author.name} />
                    </AvatarFallback>
                  </Avatar>
                  <span>
                    Posted by{' '}
                    <span className="font-medium text-foreground hover:text-primary transition-colors">
                      {post.author.name}
                    </span>
                  </span>
                </div>
            </div>
          </CardContent>
        </Card>
      )}
    </div>
  )}
  
```



```

        </Link>
        <span className="flex items-center gap-1">
            <VerifiedBadge show={post.author.role === 'ROLE_ADMIN'} size={12} title="Admin" />
            <Calendar className="h-4 w-4" />
            {formatDistanceToNow(new Date(post.createdAt), { addSuffix: true })}
        </span>
    </div>
</div>

{/* Post Content */}
<div className="mb-4">
    <p className="whitespace-pre-wrap">{post.content}</p>
</div>

{/* Media (Clouldinary) */}
{(post.videoUrl || (post.imageUrls && post.imageUrls.length > 0)) && (
    <div className="mb-4 space-y-3">
        {post.videoUrl && (
            <AutoPlayMutedVideo src={post.videoUrl} />
        )}

        {post.imageUrls && post.imageUrls.length > 0 && (
            <ImageCarousel imageUrls={post.imageUrls} heightClassName="h-72" />
        )}
    </div>
)}

{/* Bottom-right views */}
<div className="absolute bottom-4 right-6 text-xs text-muted-foreground">
    {formatCount(post.views ?? 0)} views
</div>

{/* Post Actions */}
<div className="flex flex-wrap gap-3 items-center">
    {/* Like Button */}
    {isAuthenticated ? (
        <Button
            variant={post.likedByMe ? "ghost" : "outline"}
            size="sm"
            onClick={() => {
                if (post.author.id === user?.id) {
                    toast.error('You cannot like your own post');
                    return;
                }
                handleLike(post.id);
            }}
            className={
                post.likedByMe
                    ? "rounded-full border border-transparent text-white bg-linear-to-r from-[rgb(75,85,99)] to-[rgb(255,255,255)]"
                    : ""
            }
        >
            <Heart className={`h-4 w-4 mr-2 ${post.likedByMe ? 'fill-current' : ''}`} />
            {formatCount(post.likes)}
        </Button>
    ) : (
        <Button variant="outline" size="sm" asChild>
            <Link to={`/login?redirect=${encodeURIComponent(`/post/${post.id}`)}`}>
                <Heart className="h-4 w-4 mr-2" />
                {formatCount(post.likes)}
            </Link>
        </Button>
    )}

    {/* Comment Button */}
    <Button variant="outline" size="sm" asChild>
        <Link to={`/post/${post.id}`}>
            <MessageCircle className="h-4 w-4 mr-2" />
            {formatCount(post.commentCount ?? post.comments?.length ?? 0)}
        </Link>
    </Button>

```

```

        {/* Share Button */}
        <Button
          variant="outline"
          size="sm"
          onClick={() => handleShare(post.id)}
        >
          <Share2 className="h-4 w-4 mr-2" />
          Share
        </Button>
      </div>
    </CardContent>
  </Card>
))}
</div>
})

{/* Load more */}
{isAuthenticated ? (
  hasMorePosts && filteredPosts.length > 0 && (
    <div ref={loadMoreRef} className="flex justify-center items-center py-4">
      {isLoadingMore ? (
        <Loader2 className="h-6 w-6 animate-spin text-primary" />
      ) : (
        <p className="text-sm text-muted-foreground">Scroll to load more posts...</p>
      )}
    </div>
  )
) : (
  // Only show the login gate after we successfully rendered at least one page
  posts.length > 0 && (
    <div className="mt-6 rounded-lg border bg-card/40 p-4 text-center">
      <p className="text-sm text-muted-foreground">
        Log in to load more posts.
      </p>
      <Button asChild variant="default" size="sm" className="mt-3">
        <Link to="/login">Log in</Link>
      </Button>
    </div>
  )
)}
</>
)}
</main>

{/* Right Sidebar */}
<RightSidebar />
</div>
</div>
</div>
);
}

```

■ frontend/src/pages/Help.tsx

```

import { Navigation } from '@/components/Navigation';
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';
import { Link } from 'react-router-dom';
import { HelpCircle, MessageCircle, Mail } from 'lucide-react';

export default function Help() {
  return (
    <div className="min-h-screen bg-background">
      <Navigation />

      <div className="container mx-auto px-4 py-8 max-w-4xl">
        <div className="mb-8">
          <h1 className="text-3xl font-bold mb-2">Help Center</h1>
          <p className="text-muted-foreground brand-name">
            Find answers to common questions and learn how to use <span className="allied">Allied</span>
          </p>

```

```

</div>

<div className="space-y-6">
  /* About AlliedEdge */
  <Card className="gradient-mesh-card">
    <CardHeader>
      <CardTitle className="flex items-center gap-2 brand-name">
        <HelpCircle className="h-5 w-5" />
        About <span className="allied">Allied</span><span className="edge">Edge</span>
      </CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
      <p className="brand-name">
        <span className="allied">Allied</span><span className="edge">Edge</span> is a s
        share knowledge, collaborate on projects, and grow together. Whether you're learning
        to code, working on a project, or looking to network with like-minded individuals,
        <span className="allied">Allied</span><span className="edge">Edge</span> provi
      </p>
    </CardContent>
  </Card>

  /* Account & Profile */
  <Card className="gradient-mesh-card">
    <CardHeader>
      <CardTitle>Account & Profile</CardTitle>
    </CardHeader>
    <CardContent className="space-y-4">
      <div>
        <h3 className="font-semibold mb-2">How do I create an account?</h3>
        <p className="text-sm text-muted-foreground">
          Click "Sign Up" in the navigation bar and provide your email address and password.
          You'll need to verify your email to activate your account.
        </p>
      </div>

      <div>
        <h3 className="font-semibold mb-2">How do I edit my profile?</h3>
        <p className="text-sm text-muted-foreground">
          Click on your profile picture in the top-right corner, then select "Edit Profile"
          from the dropdown menu. You can update your name, bio, avatar, and other details.
        </p>
      </div>

      <div>
        <h3 className="font-semibold mb-2">Can I change my username?</h3>
        <p className="text-sm text-muted-foreground">
          Yes! Go to your profile menu and select "Change Username". Your new username must
          be unique and follow our community guidelines.
        </p>
      </div>

      <div>
        <h3 className="font-semibold mb-2">How do I reset my password?</h3>
        <p className="text-sm text-muted-foreground">
          On the login page, click "Forgot Password" and enter your email. We'll send you
          instructions to reset your password.
        </p>
      </div>
    </CardContent>
  </Card>

  /* Posts & Content */
  <Card className="gradient-mesh-card">
    <CardHeader>
      <CardTitle>Posts & Content</CardTitle>
    </CardHeader>
    <CardContent className="space-y-4">
      <div>
        <h3 className="font-semibold mb-2">How do I create a post?</h3>
        <p className="text-sm text-muted-foreground">
          Click the "Create Post" button on your feed. Write your content, add any media

```

```

        if needed, and click "Post" to share with the community.
    </p>
</div>

<div>
    <h3 className="font-semibold mb-2">Can I edit or delete my posts?</h3>
    <p className="text-sm text-muted-foreground">
        Yes! Click on your post and select "Edit" or "Delete" from the options menu.
        Edited posts will show an "edited" indicator.
    </p>
</div>

<div>
    <h3 className="font-semibold mb-2">How do I search for posts?</h3>
    <p className="text-sm text-muted-foreground">
        Use the search bar in the navigation menu to find posts by keywords. You can
        also filter and sort posts on the main feed.
    </p>
</div>

<div>
    <h3 className="font-semibold mb-2">What content is allowed?</h3>
    <p className="text-sm text-muted-foreground">
        Share tech-related content, projects, questions, tutorials, and meaningful
        discussions. Avoid spam, harassment, or content that violates our community
        guidelines.
    </p>
</div>
</CardContent>
</Card>

{/* Community Features */}
<Card className="gradient-mesh-card">
    <CardHeader>
        <CardTitle>Community Features</CardTitle>
    </CardHeader>
    <CardContent className="space-y-4">
        <div>
            <h3 className="font-semibold mb-2">How do likes and comments work?</h3>
            <p className="text-sm text-muted-foreground">
                Click the heart icon to like a post. Click "View Comments" to read and add
                comments. Engage respectfully and constructively with others.
            </p>
        </div>

        <div>
            <h3 className="font-semibold mb-2">Can I message other users?</h3>
            <p className="text-sm text-muted-foreground">
                Yes! Use the Chat feature to send direct messages to other users. Access it
                from the navigation menu.
            </p>
        </div>

        <div>
            <h3 className="font-semibold mb-2">How do I find other users?</h3>
            <p className="text-sm text-muted-foreground">
                Use the "Search Users" feature in the sidebar to find people by name or username.
            </p>
        </div>
    </CardContent>
</Card>

{/* Reporting & Safety */}
<Card className="gradient-mesh-card">
    <CardHeader>
        <CardTitle>Reporting & Safety</CardTitle>
    </CardHeader>
    <CardContent className="space-y-4">
        <div>
            <h3 className="font-semibold mb-2">How do I report inappropriate content?</h3>
            <p className="text-sm text-muted-foreground">

```

```

        Click the options menu (three dots) on any post and select "Report". Choose
        the reason and provide details. Our team will review it promptly.
    </p>
</div>

<div>
    <h3 className="font-semibold mb-2">How do I block or mute someone?</h3>
    <p className="text-sm text-muted-foreground">
        Visit their profile and select "Block" or "Mute" from the options. Blocked
        users cannot see your content or interact with you.
    </p>
</div>
</CardContent>
</Card>

{/* Contact Support */}
<Card className="gradient-mesh-card">
    <CardHeader>
        <CardTitle className="flex items-center gap-2">
            <MessageCircle className="h-5 w-5" />
            Still Need Help?
        </CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
        <p className="text-sm">
            If you couldn't find the answer you're looking for, our support team is here to help.
        </p>
        <div className="flex items-center gap-2 text-sm">
            <Mail className="h-4 w-4" />
            <span className="text-muted-foreground">Contact us at:</span>
            <a href="mailto:support@alliededge.com" className="text-primary hover:underline">
                support@alliededge.com
            </a>
        </div>
        <p className="text-xs text-muted-foreground">
            We typically respond within 24-48 hours.
        </p>
    </CardContent>
</Card>
</div>

<div className="mt-8 text-center">
    <Link to="/" className="text-sm text-primary hover:underline">
        ← Back to Feed
    </Link>
</div>
</div>
</div>
);
}

```

■ frontend/src/pages/Landing.tsx

```

// TODO: REPLACE THIS LANDING PAGE WITH AN ELEGANT, THEMATIC, AND WELL-DESIGNED LANDING PAGE RELEVANT TO THE PROJECT
import { motion } from "framer-motion";

export default function Landing() {
    return (
        <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            transition={{ duration: 0.5 }}
            className="min-h-screen flex flex-col"
        >
            <div className="flex-1 flex flex-col items-center justify-center">
                <div className="max-w-5xl mx-auto relative px-4 text-center">
                    <div className="flex justify-center">
                        
                    </div>
                </div>
            </div>
        </motion.div>
    );
}

```

```

        width={64}
        height={64}
        className="rounded-lg mb-8 mt-24"
      />
    </div>
    <h1 className="text-3xl font-bold mb-3">Welcome to AlliedEdge</h1>
    <p className="text-muted-foreground max-w-xl mx-auto">
      Your community feed for posts, announcements, and chat.
    </p>
  </div>
</div>
</motion.div>
);
}

```

■ frontend/src/pages/Login.tsx

```

import { useEffect, useState } from "react";
import { useNavigate, useSearchParams } from "react-router-dom";
import { useAuthContext } from "@/contexts/AuthContext";
import { Card, CardContent } from "@/components/ui/card";
import { Alert, AlertDescription, AlertTitle } from "@/components/ui/alert";
import { authApi } from "@/lib/api";

// Use same-origin by default (works behind reverse proxies, avoids CORS/cookie issues).
// If VITE_API_BASE_URL is set (e.g., http://localhost:8080/api), derive backend origin from it.
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL as string | undefined;
const BACKEND_ORIGIN = API_BASE_URL
  ? API_BASE_URL.replace(/\/.*api\/?$/, "")
  : (import.meta.env.DEV ? "http://localhost:8080" : "");

function GoogleGIcon(props: React.SVGProps<SVGSVGElement> & {
  return (
    <svg viewBox="0 0 48 48" aria-hidden="true" focusable="false" {...props}>
      <path
        fill="#EA4335"
        d="M24 9.5c3.04 0 5.77 1.05 7.92 2.78l5.9-5.9c3.42 3.08 29.32 1 24 14.62 1 6.53 6.38 2.65 14.19l6.88 5.33c11.
      />
      <path
        fill="#4285F4"
        d="M46.5 24.5c0-1.57-.14-3.08-.4-4.54h24v9.09h12.63c-.54 2.9-2.17 5.36-4.62 7.02l7.06 5.47c43.4 37.6 46.5 31.6
      />
      <path
        fill="#FBBC05"
        d="M9.53 28.52a14.7 14.7 0 0 1 8.75 24c0-1.58.27-3.17.45-4.52l-6.88-5.33a23.93 23.93 0 0 0 1.5 24c0 3.86.92 7.5
      />
      <path
        fill="#34A853"
        d="M24 46.5c5.32 0 10.2-1.76 13.6-4.76l-7.06-5.47c-1.96 1.32-4.47 2.1-6.54 2.1-6.87 0-12.74-4.12-14.47-10.02l-7.
      />
      <path fill="none" d="M1.5 1h45v45h-45z" />
    </svg>
  );
}

export default function Login() {
  const navigate = useNavigate();
  const [searchParams] = useSearchParams();
  const { isAuthenticated, isLoading: authLoading } = useAuthContext();
  const [blockedMessage, setBlockedMessage] = useState<string | null>(null);

  useEffect(() => {
    const error = (searchParams.get('error') || '').toLowerCase();
    if (error === 'banned') {
      setBlockedMessage('Your account has been banned. Please contact support.');
```

```

useEffect(() => {
  if (!authLoading && isAuthenticated) {
    navigate("/");
  }
}, [isAuthenticated, authLoading, navigate]);

useEffect(() => {
  let cancelled = false;

  // If we're not authenticated, probe auth/status to see if backend is denying login
  // (e.g., banned user session that shouldn't be treated as logged-in).
  const run = async () => {
    if (authLoading || isAuthenticated) return;
    try {
      const s = await authApi.status();
      if (cancelled) return;
      const msg = typeof s?.message === "string" ? s.message.trim() : "";
      setBlockedMessage(msg || null);
    } catch {
      if (!cancelled) setBlockedMessage(null);
    }
  };

  run();
  return () => {
    cancelled = true;
  };
}, [authLoading, isAuthenticated]);

const handleLogin = () => {
  // Backend-owned login flow.
  // In production behind a reverse proxy, BACKEND_ORIGIN may be "" to use same-origin.
  // But oauth2 endpoint must still resolve correctly.
  const origin = BACKEND_ORIGIN || window.location.origin;
  window.location.href = `${origin}/oauth2/authorization/google`;
};

return (
  <div className="min-h-screen flex items-center justify-center p-4">
    <div className="w-full max-w-md space-y-4">
      {blockedMessage ? (
        <Alert variant="destructive">
          <AlertTitle>Login blocked</AlertTitle>
          <AlertDescription>{blockedMessage}</AlertDescription>
        </Alert>
      ) : null}

      <Card className="gradient-mesh-card">
        <CardContent className="pt-6 flex flex-col items-center gap-6">
          <h1 className="text-2xl font-bold text-center">Login</h1>

          <button
            type="button"
            onClick={handleLogin}
            className={[
              "w-full",
              "h-14",
              "rounded-full",
              "bg-white",
              "text-black",
              "border",
              "border-border/60",
              "shadow-sm",
              "hover:shadow-md",
              "transition-shadow",
              "relative",
              "flex",
              "items-center",
              "justify-center",
              "font-medium",
            ].join(" ")}
          >

```

```

        &gt;
        &lt;span className="absolute left-4 flex items-center justify-center"&gt;
            &lt;GoogleGIcon className="h-5 w-5" /&gt;
        &lt;/span&gt;
        &lt;span&gt;Continue with Google&lt;/span&gt;
    &lt;/button&gt;
    &lt;/CardContent&gt;
    &lt;/Card&gt;
    &lt;/div&gt;
    &lt;/div&gt;
);
}

```

■ frontend/src/pages/MyPosts.tsx

```

import { useEffect, useState, useCallback } from 'react';
import { Navigation } from '@components/Navigation';
import { postsApi, Post } from '@lib/api';
import { useAuthContext } from '@contexts/AuthContext';
import { PostCard } from '@components/PostCard';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Loader2, Plus, FileText } from 'lucide-react';
import { Link } from 'react-router-dom';

export default function MyPosts() {
    const { user } = useAuthContext();
    const [posts, setPosts] = useState<Post[]&gt;([]);
    const [filteredPosts, setFilteredPosts] = useState<Post[]&gt;([]);
    const [isLoading, setIsLoading] = useState(true);
    const [error, setError] = useState<string | null>(null);
    const [keyword, setKeyword] = useState('');
    const [sortBy, setSortBy] = useState('newest');

    const fetchMyPosts = useCallback(async () => {
        try {
            setIsLoading(true);
            const allPosts = await postsApi.getAll();
            // Filter posts by current user
            const myPosts = allPosts.filter((post: Post) => post.author.id === user?.id);
            setPosts(myPosts);
            setError(null);
        } catch (err) {
            console.error('Failed to fetch posts:', err);
            setError('Failed to load your posts. Please try again later.');
```



```

        result.sort((a, b) => new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime());
        break;
    case 'oldest':
        result.sort((a, b) => new Date(a.createdAt).getTime() - new Date(b.createdAt).getTime());
        break;
    case 'mostLiked':
        result.sort((a, b) => b.likes - a.likes);
        break;
    case 'mostViewed':
        // Assuming views property exists
        result.sort((a, b) => (b.likes || 0) - (a.likes || 0));
        break;
    }

    setFilteredPosts(result);
}, [posts, keyword, sortBy]);

const handlePostUpdated = (updatedPost: Post) => {
    setPosts(posts.map((post) => (post.id === updatedPost.id ? updatedPost : post)));
};

const handlePostDeleted = (postId: number) => {
    setPosts(posts.filter((post) => post.id !== postId));
};

return (
    <div className="min-h-screen bg-background">
        <Navigation />

        <main className="container mx-auto px-4 py-6 max-w-5xl">
            {/* Header with Create Button */}
            <div className="flex justify-between items-center mb-6">
                <h1 className="text-3xl font-bold flex items-center gap-2">
                    <FileText className="h-8 w-8 text-primary" />
                    My Posts
                </h1>
                <Button asChild>
                    <Link to="/posts/new">
                        <Plus className="h-4 w-4 mr-2" />
                        Create New Post
                    </Link>
                </Button>
            </div>

            {/* Search and Sort Bar */}
            <div className="flex flex-col sm:flex-row gap-3 mb-6">
                <Input
                    type="text"
                    placeholder="Search my posts..."
                    value={keyword}
                    onChange={(e) => setKeyword(e.target.value)}
                    className="flex-1"
                />
                <select
                    value={sortBy}
                    onChange={(e) => setSortBy(e.target.value)}
                    className="px-4 py-2 border rounded-md bg-background"
                >
                    <option value="newest">Latest</option>
                    <option value="oldest">Oldest</option>
                    <option value="mostLiked">Most Liked</option>
                    <option value="mostViewed">Most Viewed</option>
                </select>
            </div>

            {/* Posts List */}
            <div className="space-y-6">
                {isLoading ? (
                    <div className="flex justify-center items-center py-12">
                        <Loader2 className="h-8 w-8 animate-spin text-primary" />
                    </div>
                ) : (
                    <div>
                        {posts.map((post) => (
                            <PostCard
                                key={post.id}
                                post={post}
                                onDelete={handlePostDeleted}
                                onUpdate={handlePostUpdated}
                            />
                        ))}
                    </div>
                )}
            </div>
        </main>
    </div>
);

```

```

) : error ? (
  <div className="text-center py-12">
    <p className="text-destructive">{error}</p>
  </div>
) : posts.length === 0 ? (
  <div className="text-center py-12">
    <p className="text-muted-foreground mb-4">
      You haven't created any posts yet.
    </p>
    <Button asChild>
      <Link to="/posts/new">
        <Plus className="h-4 w-4 mr-2" />
        Create Your First Post
      </Link>
    </Button>
  </div>
) : filteredPosts.length === 0 ? (
  <div className="text-center py-12">
    <p className="text-muted-foreground">
      No posts match your search. Try a different keyword.
    </p>
  </div>
) : (
  filteredPosts.map((post) => (
    <PostCard
      key={post.id}
      post={post}
      onPostUpdated={handlePostUpdated}
      onPostDeleted={handlePostDeleted}
    />
  ))
)
</div>
</main>
</div>
);
}

```

■ frontend/src/pages/NotFound.tsx

```

import { Link } from 'react-router-dom';
import { motion } from 'framer-motion';
import { Home } from 'lucide-react';

export default function NotFound() {
  return (
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      transition={{ duration: 0.5 }}
      className="min-h-screen flex items-center justify-center bg-background p-4"
    >
      <div className="max-w-2xl text-center">
        <h1 className="text-9xl font-light mb-4">404</h1>
        <h2 className="text-2xl font-normal mb-6 text-muted-foreground">
          Oops! Page Not Found.
        </h2>
        <p className="text-lg mb-8 text-muted-foreground">
          The page you are looking for might have been removed, had its name changed, or is temporarily unavailable.
        </p>
        <Link
          to="/"
          className="inline-flex items-center gap-2 text-primary font-bold hover:underline text-lg"
        >
          <Home className="h-5 w-5" />
          Return to Homepage
        </Link>
      </div>
    </motion.div>
  );
}

```

```

    );
}

```

■ frontend/src/pages/PostDetail.tsx

```

import { useEffect, useState, useCallback } from 'react';
import { useParams, useNavigate, Link } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { postsApi, Post, Comment } from '@lib/api';
import { useAuthContext } from '@contexts/AuthContext';
import { PostCard } from '@components/PostCard';
import { CommentCard } from '@components/CommentCard';
import { CreateCommentCard } from '@components/CreateCommentCard';
import { Card, CardContent } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Loader2, ArrowLeft, LogIn } from 'lucide-react';
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from '@components/ui/select';
import { usePostRealtime } from '@hooks/usePostRealtime';

export default function PostDetail() {
  const { id } = useParams<{ id: string }>();
  const navigate = useNavigate();
  const { isAuthenticated } = useAuthContext();
  const [post, setPost] = useState<Post | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [commentSort, setCommentSort] = useState<'newest' | 'oldest' | 'mostReplied'>('newest');

  const fetchPost = useCallback(async () => {
    if (!id) return;

    try {
      setIsLoading(true);
      const data = await postsApi.getById(Number(id), { sortComments: commentSort });
      setPost(data);
      setError(null);
    } catch (err) {
      console.error('Failed to fetch post:', err);
      setError('Failed to load post. Please try again later.');
```

```

);

const handlePostUpdated = (updatedPost: Post) => {
  setPost(updatedPost);
};

const handlePostDeleted = () => {
  navigate('/');
};

const handleCommentCreated = () => {
  // Keep UI snappy: bump comment count immediately.
  setPost((prev) => {
    if (!prev) return prev;
    const current = prev.commentCount ?? prev.comments?.length ?? 0;
    return { ...prev, commentCount: current + 1 };
  });

  // Also re-fetch to get the server's canonical comment list + counts.
  fetchPost();
};

const handleCommentUpdated = (updatedComment: Comment) => {
  if (post && post.comments) {
    setPost({
      ...post,
      comments: post.comments.map((comment) => {
        comment.id === updatedComment.id ? updatedComment : comment
      }),
    });
  }
};

if (isLoading) {
  return (
    <div className="min-h-screen">
      <Navigation />
      <div className="flex justify-center items-center py-12">
        <Loader2 className="h-8 w-8 animate-spin text-primary" />
      </div>
    </div>
  );
}

if (error || !post) {
  return (
    <div className="min-h-screen">
      <Navigation />
      <div className="container mx-auto px-4 py-8 max-w-3xl">
        <div className="text-center py-12">
          <p className="text-destructive">{error || 'Post not found'}</p>
          <Button onClick={() => navigate('/')} variant="outline" className="mt-4">
            <ArrowLeft className="mr-2 h-4 w-4" />
            Back to Feed
          </Button>
        </div>
      </div>
    </div>
  );
}

return (
  <div className="min-h-screen">
    <Navigation />

    <main className="container mx-auto px-4 py-8 max-w-3xl">
      <Button onClick={() => navigate(-1)} variant="ghost" className="mb-4">
        <ArrowLeft className="mr-2 h-4 w-4" />
        Back
      </Button>
      <div className="space-y-6">

```

```

    <PostCard
      post={post}
      onPostUpdated={handlePostUpdated}
      onPostDeleted={handlePostDeleted}
    />

    <div className="space-y-4">
      <div className="flex items-center justify-between gap-4">
        <h2 className="text-2xl font-bold">Comments</h2>

        <div className="w-[200px]">
          <Select value={commentSort} onChange={(v) => setCommentSort(v as typeof commentSort)}>
            <SelectTrigger className="h-9">
              <SelectValue placeholder="Sort comments" />
            </SelectTrigger>
            <SelectContent>
              <SelectItem value="newest">Newest</SelectItem>
              <SelectItem value="oldest">Oldest</SelectItem>
              <SelectItem value="mostReplied">Most replied</SelectItem>
            </SelectContent>
          </Select>
        </div>
      </div>

      {isAuthenticated ? (
        <CreateCommentCard postId={post.id} onCommentCreated={handleCommentCreated} />
      ) : (
        <Card className="gradient-mesh-card">
          <CardContent className="pt-6 text-center">
            <p className="text-muted-foreground mb-4">
              Login to join the conversation
            </p>
            <Button asChild>
              <Link to="/login">
                <LogIn className="mr-2 h-4 w-4" />
                Login or Sign Up
              </Link>
            </Button>
          </CardContent>
        </Card>
      )}

      {post.comments && post.comments.length > 0 ? (
        <div className="space-y-3">
          {post.comments.map((comment) => (
            <CommentCard
              key={comment.id}
              comment={comment}
              onCommentUpdated={handleCommentUpdated}
            />
          ))}
        </div>
      ) : (
        <p className="text-center text-muted-foreground py-8">
          No comments yet. Be the first to comment!
        </p>
      )}
    </div>
  </div>
</main>
</div>
);
}

```

■ frontend/src/pages/PostForm.tsx

```

import { useState, useEffect } from 'react';
import { useNavigate, useParams, Link } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card, CardContent } from '@components/ui/card';

```

```

import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Textarea } from '@components/ui/textarea';
import { Label } from '@components/ui/label';
import { ArrowLeft, Save, X, Image as ImageIcon, Video, Trash2 } from 'lucide-react';
import { toast } from 'sonner';
import { postsApi, ApiError, UNAUTHENTICATED_MESSAGE, isUnauthenticatedStatus } from '@lib/api';

interface PostFormData {
  title: string;
  content: string;
  imageFiles: File[];
  imagePreviews: string[];
  videoFile: File | null;
  videoPreview: string | null;
  existingImages: string[];
  existingVideo: string | null;
  deletedExistingImageIndexes: number[];
  deleteExistingVideo: boolean;
}

export default function PostForm() {
  const navigate = useNavigate();
  const { id } = useParams<{ id: string }>();
  const isEditMode = Boolean(id);

  const [formData, setFormData] = useState<PostFormData>({
    title: '',
    content: '',
    imageFiles: [],
    imagePreviews: [],
    videoFile: null,
    videoPreview: null,
    existingImages: [],
    existingVideo: null,
    deletedExistingImageIndexes: [],
    deleteExistingVideo: false,
  });

  const [isLoading, setIsLoading] = useState(false);

  // Load existing post data if editing
  useEffect(() => {
    let cancelled = false;

    const load = async () => {
      if (!isEditMode || !id) return;

      try {
        setIsLoading(true);
        const post = await postsApi.getById(Number(id));
        if (cancelled) return;

        setFormData((prev) => ({
          ...prev,
          title: post.title ?? '',
          content: post.content ?? '',
          existingImages: post.imageUrls ?? [],
          existingVideo: post.videoUrl ?? null,
          deletedExistingImageIndexes: [],
          deleteExistingVideo: false,
          // clear any newly-selected files when switching to edit
          imageFiles: [],
          imagePreviews: [],
          videoFile: null,
          videoPreview: null,
        }));
      } catch (error) {
        console.error('Failed to load post for edit:', error);
        if (error instanceof ApiError) {
          const msg = isUnauthenticatedStatus(error.status)

```

```

        ? UNAUTHENTICATED_MESSAGE
        : (error.message || 'Failed to load post');
        toast.error(msg);
    } else {
        toast.error('Failed to load post');
    }
} finally {
    if (!cancelled) setIsLoading(false);
}
};

load();
return () => {
    cancelled = true;
};
}, [isEditMode, id]);

const handleImageChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const files = Array.from(e.target.files || []);
    if (files.length === 0) return;

    // Create preview URLs for new images
    const newPreviews = files.map((file) => URL.createObjectURL(file));

    setFormData((prev) => ({
        ...prev,
        imageFiles: [...prev.imageFiles, ...files],
        imagePreviews: [...prev.imagePreviews, ...newPreviews],
    }));
};

const handleVideoChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (!file) return;

    if (!file.type.startsWith('video/')) {
        toast.error('Please select a valid video file');
        return;
    }

    const preview = URL.createObjectURL(file);
    setFormData((prev) => ({
        ...prev,
        // If we're editing and a post already has a video, selecting a new one
        // should replace it. Mark the existing one for deletion so the backend
        // removes it and stores the new upload.
        deleteExistingVideo: prev.existingVideo ? true : prev.deleteExistingVideo,
        videoFile: file,
        videoPreview: preview,
    }));
};

const handleDeleteImage = (index: number) => {
    setFormData((prev) => {
        const newImageFiles = [...prev.imageFiles];
        const newPreviews = [...prev.imagePreviews];

        // Revoke the object URL to free memory
        URL.revokeObjectURL(newPreviews[index]);

        newImageFiles.splice(index, 1);
        newPreviews.splice(index, 1);

        return {
            ...prev,
            imageFiles: newImageFiles,
            imagePreviews: newPreviews,
        };
    });
};

const handleDeleteExistingImage = (index: number) => {

```

```

    setFormData((prev) => {
      if (index < 0 || index >= prev.existingImages.length) return prev;
      if (prev.deletedExistingImageIndexes.includes(index)) return prev;
      return {
        ...prev,
        deletedExistingImageIndexes: [...prev.deletedExistingImageIndexes, index],
      };
    });
  };

  const handleUndoDeleteExistingImage = (index: number) => {
    setFormData((prev) => ({
      ...prev,
      deletedExistingImageIndexes: prev.deletedExistingImageIndexes.filter((i) => i !== index),
    }));
  };

  const handleDeleteVideo = () => {
    if (formData.videoPreview) {
      URL.revokeObjectURL(formData.videoPreview);
    }
    setFormData((prev) => ({
      ...prev,
      videoFile: null,
      videoPreview: null,
    }));
  };

  const handleDeleteExistingVideo = () => {
    setFormData((prev) => ({
      ...prev,
      deleteExistingVideo: true,
    }));
  };

  const handleRestoreExistingVideo = () => {
    setFormData((prev) => ({
      ...prev,
      deleteExistingVideo: false,
    }));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setIsLoading(true);

    const titleTrimmed = formData.title.trim();
    const contentTrimmed = formData.content.trim();

    if (!titleTrimmed) {
      toast.error('Please provide a title');
      setIsLoading(false);
      return;
    }

    if (titleTrimmed.length < 5 || titleTrimmed.length > 100) {
      toast.error('Title must be between 5 and 100 characters');
      setIsLoading(false);
      return;
    }

    if (!contentTrimmed) {
      toast.error('Please provide content');
      setIsLoading(false);
      return;
    }

    try {
      const formDataToSend = new FormData();
      formDataToSend.append('title', titleTrimmed);
      formDataToSend.append('content', contentTrimmed);
    }
  };

```



```

    formData.imageFiles.forEach((file) => {
        formDataToSend.append('imageFiles', file);
    });

    if (formData.videoFile) {
        formDataToSend.append('videoFile', formData.videoFile);
    }

    if (isEditMode && id) {
        // Deletions are index-based in the backend (deleteImageId list)
        formData.deletedExistingImageIndexes.forEach((idx) => {
            formDataToSend.append('deleteImageId', String(idx));
        });
        if (formData.deleteExistingVideo) {
            formDataToSend.append('deleteVideo', 'true');
        }

        await postsApi.updateMultipart(Number(id), formDataToSend);
        toast.success('Post updated successfully!');
        navigate('/myposts');
    } else {
        await postsApi.createMultipart(formDataToSend);
        toast.success('Post created successfully!');
        navigate('/');
    }
} catch (error) {
    console.error('Failed to save post:', error);
    if (error instanceof ApiError) {
        const msg = isUnauthenticatedStatus(error.status)
            ? UNAUTHENTICATED_MESSAGE
            : (error.message || 'Failed to save post');
        toast.error(msg);
    } else {
        toast.error('Failed to save post');
    }
} finally {
    setIsLoading(false);
}
};

return (
    <div className="min-h-screen bg-background">
        <Navigation />

        <main className="container mx-auto px-4 py-6 max-w-4xl">
            <div className="mb-6">
                <Button variant="outline" asChild className="mb-4">
                    <Link to="/myposts">
                        <ArrowLeft className="h-4 w-4 mr-2" />
                        Back to My Posts
                    </Link>
                </Button>
                <h2 className="text-3xl font-bold">
                    {isEditMode ? 'Edit Post' : 'Create New Post'}
                </h2>
            </div>

            <Card className="gradient-mesh-card">
                <CardContent className="pt-6">
                    <form onSubmit={handleSubmit} className="space-y-6">
                        {/* Title */}
                        <div className="space-y-2">
                            <Label htmlFor="title">
                                Title <span className="text-red-500">*</span>
                            </Label>
                            <Input
                                id="title"
                                type="text"
                                value={formData.title}
                                onChange={(e) => setFormData({ ...formData, title: e.target.value })}
                                placeholder="Enter post title"

```

```

        required
      />
    </div>

    { /* Content */
    <div className="space-y-2">
      <Label htmlFor="content">
        Content <span className="text-red-500">*</span>
      </Label>
      <Textarea
        id="content"
        value={formData.content}
        onChange={e => setFormData({ ...formData, content: e.target.value })}
        placeholder="Write your post content here..."
        rows={12}
        required
      />
      <p className="text-sm text-muted-foreground">
        You can use rich text formatting in your content.
      </p>
    </div>

    { /* Image Upload */
    <div className="space-y-2">
      <Label htmlFor="images">Upload Images (Optional)</Label>
      <div className="flex items-center gap-3">
        <Input
          id="images"
          type="file"
          accept="image/*"
          multiple
          onChange={handleImageChange}
          className="flex-1"
        />
        <ImageIcon className="h-5 w-5 text-muted-foreground" />
      </div>
      <p className="text-sm text-muted-foreground">
        You can select multiple images
      </p>
    </div>

    { /* Existing Images Preview */
    {formData.existingImages.length > 0 && (
      <div className="space-y-2">
        <Label>Existing Images</Label>
        <div className="grid grid-cols-2 md:grid-cols-3 gap-4">
          {formData.existingImages.map((img, index) => {
            const marked = formData.deletedExistingImageIndexes.includes(index);
            return (
              <div key={index} className="relative group">
                <img
                  src={img}
                  alt={`Existing ${index + 1}`}
                  className={`w-full h-32 object-cover rounded-lg border ${marked ? 'opacity-40 grayscale' : ''}`}
                />
                {marked ? (
                  <Button
                    type="button"
                    variant="secondary"
                    size="sm"
                    className="absolute top-2 right-2 opacity-100"
                    onClick={() => handleUndoDeleteExistingImage(index)}
                  />
                  Undo
                </Button>
                : (
                  <Button
                    type="button"
                    variant="destructive"
                    size="sm"
                    className="absolute top-2 right-2 opacity-0 group-hover:opacity-100 transition-opacity"

```

```

        onClick={() => handleDeleteExistingImage(index)}
      </>
      <Trash2 className="h-4 w-4" />
    </Button>
  )}
  </div>
);
}}}
</div>
{formData.deletedExistingImageIndexes.length > 0 && (
  <p className="text-sm text-muted-foreground">
    {formData.deletedExistingImageIndexes.length} existing image(s) will be removed when you save.
  </p>
)}
</div>
)}

{/* New Images Preview */}
{formData.imagePreviews.length > 0 && (
  <div className="space-y-2">
    <Label>New Images to Upload</Label>
    <div className="grid grid-cols-2 md:grid-cols-3 gap-4">
      {formData.imagePreviews.map((preview, index) => (
        <div key={index} className="relative group">
          <img
            src={preview}
            alt={`Preview ${index + 1}`}
            className="w-full h-32 object-cover rounded-lg border"
          />
          <Button
            type="button"
            variant="destructive"
            size="sm"
            className="absolute top-2 right-2 opacity-0 group-hover:opacity-100 transition-opacity"
            onClick={() => handleDeleteImage(index)}
          >
            <Trash2 className="h-4 w-4" />
          </Button>
        </div>
      ))}
    </div>
  </div>
)}

{/* Video Upload */}
<div className="space-y-2">
  <Label htmlFor="video">Upload Video (Optional)</Label>
  <div className="flex items-center gap-3">
    <Input
      id="video"
      type="file"
      accept="video/*"
      onChange={handleVideoChange}
      className="flex-1"
    />
    <Video className="h-5 w-5 text-muted-foreground" />
  </div>
</div>

{/* Existing Video */}
{formData.existingVideo && !formData.videoFile && (
  <div className="space-y-2">
    <Label>Current Video</Label>
    <div className="relative">
      <video
        controls
        className={`w-full max-h-64 rounded-lg ${formData.deleteExistingVideo ? 'opacity-40' : ''}`}
        src={formData.existingVideo}
      >
        Your browser does not support the video tag.
      </video>
    </div>
  </div>
)}

```

```

        {!formData.deleteExistingVideo ? (
            <Button
                type="button"
                variant="destructive"
                size="sm"
                className="mt-2"
                onClick={handleDeleteExistingVideo}
            />
            <Trash2 className="h-4 w-4 mr-2" />
            Remove Existing Video
        ) : (
            <Button
                type="button"
                variant="secondary"
                size="sm"
                className="mt-2"
                onClick={handleRestoreExistingVideo}
            />
            Undo remove video
        )}
    </div>
</div>
)}

{/* Video Preview */}
{formData.videoPreview && (
    <div className="space-y-2">
        <Label>Video Preview</Label>
        <div className="relative">
            <video
                controls
                className="w-full max-h-64 rounded-lg"
                src={formData.videoPreview}
            />
            Your browser does not support the video tag.
        </div>
        <Button
            type="button"
            variant="destructive"
            size="sm"
            className="mt-2"
            onClick={handleDeleteVideo}
        />
        <Trash2 className="h-4 w-4 mr-2" />
        Remove Video
    </div>
</div>
)}

{/* Submit Buttons */}
<div className="flex gap-3 pt-4">
    <Button type="submit" className="flex-1" disabled={isLoading}>
        {isLoading ? (
            'Saving...'
        ) : (
            <Save className="h-4 w-4 mr-2" />
            {isEditMode ? 'Update Post' : 'Create Post'}
        )}
    </Button>
    <Button
        type="button"
        variant="outline"
        onClick={() => navigate('/myposts')}
        disabled={isLoading}
    />
    <X className="h-4 w-4 mr-2" />

```

```

        Cancel
      </Button>
    </div>
  </form>
</CardContent>
</Card>
</main>
</div>
);
}

```

■ frontend/src/pages/Privacy.tsx

```

import { Navigation } from '@components/Navigation';
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import { Link } from 'react-router-dom';
import { Shield, Eye, Lock, Database, UserCheck } from 'lucide-react';

export default function Privacy() {
  return (
    <div className="min-h-screen bg-background">
      <Navigation />

      <div className="container mx-auto px-4 py-8 max-w-4xl">
        <div className="mb-8">
          <h1 className="text-3xl font-bold mb-2">Privacy Policy</h1>
          <p className="text-muted-foreground">
            Last updated: January 6, 2026
          </p>
        </div>

        <div className="space-y-6">
          {/* Introduction */}
          <Card className="gradient-mesh-card">
            <CardHeader>
              <CardTitle className="flex items-center gap-2">
                <Shield className="h-5 w-5" />
                Your Privacy Matters
              </CardTitle>
            </CardHeader>
            <CardContent className="space-y-3">
              <p>
                At AlliedEdge, we take your privacy seriously. This policy explains what information we collect, how we use it, and how we protect it. We believe in transparency and giving you control over your personal data.
              </p>
              <p className="text-sm font-semibold text-primary">
                We will never sell your personal data to third parties.
              </p>
            </CardContent>
          </Card>

          {/* Data Collection */}
          <Card className="gradient-mesh-card">
            <CardHeader>
              <CardTitle className="flex items-center gap-2">
                <Database className="h-5 w-5" />
                1. What Data We Collect
              </CardTitle>
            </CardHeader>
            <CardContent className="space-y-4">
              <div>
                <h3 className="font-semibold mb-2">Account Information</h3>
                <p className="text-sm text-muted-foreground mb-2">
                  When you create an account, we collect:
                </p>
                <ul className="space-y-2 text-sm ml-4">
                  <li className="flex items-start gap-2">
                    <span className="text-muted-foreground mt-1">•</span>
                    <span>Email address</span>
                  </li>

```

```

        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Username</span>
        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Password (encrypted)</span>
        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Profile information (name, bio, avatar)</span>
        </li>
    </ul>
</div>

<div>
    <h3 className="font-semibold mb-2">Content You Create</h3>
    <p className="text-sm text-muted-foreground mb-2">
        We store content you voluntarily share:
    </p>
    <ul className="space-y-2 text-sm ml-4">
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Posts, comments, and messages</span>
        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Projects and code you share</span>
        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Likes and interactions with other content</span>
        </li>
    </ul>
</div>

<div>
    <h3 className="font-semibold mb-2">Usage Data</h3>
    <p className="text-sm text-muted-foreground mb-2">
        To improve our platform, we automatically collect:
    </p>
    <ul className="space-y-2 text-sm ml-4">
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Device type and browser information</span>
        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>IP address (for security purposes)</span>
        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Pages visited and features used</span>
        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Time and duration of your visits</span>
        </li>
    </ul>
</div>

<div>
    <h3 className="font-semibold mb-2">Cookies</h3>
    <p className="text-sm text-muted-foreground">
        We use cookies to keep you logged in and remember your preferences (like theme settings).
        These help improve your experience on AlliedEdge.
    </p>
</div>
</CardContent>
</Card>

```

```

{ /* How We Use Data */
<Card className="gradient-mesh-card">
  <CardHeader>
    <CardTitle className="flex items-center gap-2">
      <Eye className="h-5 w-5" />
      2. How We Use Your Data
    </CardTitle>
  </CardHeader>
  <CardContent className="space-y-3">
    <p>We use your information to:</p>
    <ul className="space-y-2 text-sm ml-4">
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Provide and maintain the AlliedEdge platform</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Personalize your experience (show relevant content)</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Send you important updates and notifications</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Improve our features and fix bugs</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Protect against spam, abuse, and security threats</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Comply with legal requirements</span>
      </li>
    </ul>
    <p className="text-sm text-muted-foreground mt-3">
      We only use your data for these purposes and don't use it in ways you wouldn't
      reasonably expect.
    </p>
  </CardContent>
</Card>

{ /* Data Sharing */
<Card className="gradient-mesh-card">
  <CardHeader>
    <CardTitle className="flex items-center gap-2">
      <UserCheck className="h-5 w-5" />
      3. Data Sharing Policy
    </CardTitle>
  </CardHeader>
  <CardContent className="space-y-4">
    <div className="bg-primary/10 border border-primary/20 rounded-lg p-4">
      <p className="font-semibold text-primary mb-2">Our Promise:</p>
      <p className="text-sm">
        We will NEVER sell your personal data to advertisers, data brokers, or any
        third parties for profit.
      </p>
    </div>

    <div>
      <h3 className="font-semibold mb-2">When We Might Share Data:</h3>
      <ul className="space-y-2 text-sm ml-4">
        <li className="flex items-start gap-2">
          <span className="text-muted-foreground mt-1">•</span>
          <span>
            <strong>Public Content:</strong> Posts and profiles you make public are
            visible to other users
          </span>
        </li>
        <li className="flex items-start gap-2">

```

```

        <span className="text-muted-foreground mt-1">•</span>
        <strong>Service Providers</strong> Trusted companies that help us run the
        platform (like hosting providers), under strict confidentiality agreements
    </span>
</li>
<li className="flex items-start gap-2">
    <span className="text-muted-foreground mt-1">•</span>
    <strong>Legal Obligations</strong> If required by law or to protect safety
    and rights
</li>
<li className="flex items-start gap-2">
    <span className="text-muted-foreground mt-1">•</span>
    <strong>With Your Consent</strong> Any other sharing will require your
    explicit permission
</li>
</ul>
</div>

<div>
    <h3 className="font-semibold mb-2">Anonymous Analytics</h3>
    <p className="text-sm text-muted-foreground">
        We may share anonymous, aggregated statistics about AlliedEdge usage (e.g.,
        "1,000 posts created this month") that don't identify individual users.
    </p>
</div>
</CardContent>
</Card>

{/* Data Security */}
<Card className="gradient-mesh-card">
    <CardHeader>
        <CardTitle className="flex items-center gap-2">
            <Lock className="h-5 w-5" />
            4. How We Protect Your Data
        </CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
        <p>We take security seriously and implement multiple layers of protection:</p>
        <ul className="space-y-2 text-sm ml-4">
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Passwords are encrypted using industry-standard methods</span>
            </li>
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Data is transmitted over secure HTTPS connections</span>
            </li>
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Regular security audits and monitoring for vulnerabilities</span>
            </li>
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Limited employee access to user data (only when necessary)</span>
            </li>
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Automated systems to detect and prevent unauthorized access</span>
            </li>
        </ul>
        <p className="text-sm text-muted-foreground mt-3">
            While we do our best to protect your data, no system is 100% secure. Please
            use a strong, unique password and keep it confidential.
        </p>
    </CardContent>
</Card>

```



```

{ /* User Control */}
<Card className="gradient-mesh-card">
  <CardHeader>
    <CardTitle>5. Your Control Over Your Data</CardTitle>
  </CardHeader>
  <CardContent className="space-y-4">
    <p>You have the right to:</p>

    <div>
      <h3 className="font-semibold mb-2">Access & Download</h3>
      <p className="text-sm text-muted-foreground">
        Request a copy of all the data we have about you at any time.
      </p>
    </div>

    <div>
      <h3 className="font-semibold mb-2">Update & Correct</h3>
      <p className="text-sm text-muted-foreground">
        Edit your profile, bio, and other information through your account settings.
      </p>
    </div>

    <div>
      <h3 className="font-semibold mb-2">Delete Content</h3>
      <p className="text-sm text-muted-foreground">
        Remove individual posts, comments, or messages at any time.
      </p>
    </div>

    <div>
      <h3 className="font-semibold mb-2">Delete Account</h3>
      <p className="text-sm text-muted-foreground">
        Permanently delete your account and associated data. Some information may be
        retained for legal or security purposes (like IP logs for abuse prevention).
      </p>
    </div>

    <div>
      <h3 className="font-semibold mb-2">Control Visibility</h3>
      <p className="text-sm text-muted-foreground">
        Choose what information is public and what stays private through privacy settings.
      </p>
    </div>

    <div>
      <h3 className="font-semibold mb-2">Opt-Out of Communications</h3>
      <p className="text-sm text-muted-foreground">
        Unsubscribe from promotional emails (you'll still receive important account updates).
      </p>
    </div>

    <p className="text-sm text-muted-foreground mt-4">
      To exercise any of these rights, contact us at{' '}
      <a href="mailto:privacy@alliededge.com" className="text-primary hover:underline">
        privacy@alliededge.com
      </a>
    </p>
  </CardContent>
</Card>

{ /* Data Retention */}
<Card className="gradient-mesh-card">
  <CardHeader>
    <CardTitle>6. How Long We Keep Your Data</CardTitle>
  </CardHeader>
  <CardContent className="space-y-3">
    <ul className="space-y-2 text-sm ml-4">
      <li>
        <span className="text-muted-foreground mt-1">•</span>
        <span>Account data is kept as long as your account is active</span>
      </li>
    </ul>
  </CardContent>
</Card>

```

```

        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Deleted content is permanently removed within 30 days</span>
        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Some data may be retained longer for legal compliance or security</span>
        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Anonymous analytics data may be kept indefinitely</span>
        </li>
    </ul>
</CardContent>
</Card>

{/* Children's Privacy */}
<Card className="gradient-mesh-card">
    <CardHeader>
        <CardTitle>7. Children's Privacy</CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
        <p className="text-sm">
            AlliedEdge is intended for users aged 13 and older. We don't knowingly collect
            data from children under 13. If you believe a child under 13 has created an
            account, please contact us immediately.
        </p>
    </CardContent>
</Card>

{/* International Users */}
<Card className="gradient-mesh-card">
    <CardHeader>
        <CardTitle>8. International Users</CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
        <p className="text-sm text-muted-foreground">
            If you're accessing AlliedEdge from outside the United States, your data may be
            transferred to and stored on servers in the U.S. By using our platform, you
            consent to this transfer and processing.
        </p>
    </CardContent>
</Card>

{/* Changes to Privacy Policy */}
<Card className="gradient-mesh-card">
    <CardHeader>
        <CardTitle>9. Changes to This Privacy Policy</CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
        <p className="text-sm">
            We may update this privacy policy from time to time. We'll notify you of
            significant changes through email or a notice on the platform. We encourage
            you to review this policy periodically.
        </p>
    </CardContent>
</Card>

{/* Contact */}
<Card className="gradient-mesh-card">
    <CardHeader>
        <CardTitle>Questions or Concerns?</CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
        <p className="text-sm">
            If you have any questions about this Privacy Policy or how we handle your data,
            we're here to help:
        </p>
        <div className="space-y-2 text-sm">
            <p>
                <strong>Email:</strong>{' '}
            </p>

```

```

        <a href="mailto:privacy@alliededge.com" className="text-primary hover:underline">
            privacy@alliededge.com
        </a>
    </p>
    <p>
        <strong>General Support:</strong>{' '}
        <a href="mailto:support@alliededge.com" className="text-primary hover:underline">
            support@alliededge.com
        </a>
    </p>
</div>
</CardContent>
</Card>
</div>

<div className="mt-8 text-center">
    <Link to="/" className="text-sm text-primary hover:underline">
        ← Back to Feed
    </Link>
</div>
</div>
</div>
);
}

```

```
import { useEffect, useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { profileApi, UserProfile, followApi } from '@lib/api';
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar';
import { Badge } from '@components/ui/badge';
import {
  Loader2,
  Edit,
  ExternalLink,
  Briefcase,
  MapPin,
  Globe,
  Mail,
  UserPlus,
  MessageCircle,
  Linkedin,
  Github,
  Twitter,
  Languages,
  GraduationCap,
  FileText,
  PlayCircle,
  Image,
  CheckCircle,
  Clock,
  Sparkles,
} from 'lucide-react';
import { EditProfileDialog } from '@components/EditProfileDialog';
import { DefaultAvatarFallback } from '@components/DefaultAvatarFallback';
import { toast } from 'sonner';
import { VerifiedBadge } from '@components/VerifiedBadge';

export default function Profile() {
  const { userId, username } = useParams<{ userId?: string; username?: string }>();
  const navigate = useNavigate();
  const [profile, setProfile] = useState<UserProfile | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [showEditDialog, setShowEditDialog] = useState(false);
```

```

const isOwnProfile = !userId && !username;

const fetchProfile = async () => {
  try {
    setIsLoading(true);
    const data = userId
      ? await profileApi.getById(Number(userId))
      : username
      ? await profileApi.getByUsername(username)
      : await profileApi.get();
    setProfile(data);
    setError(null);
  } catch (err) {
    console.error('Failed to fetch profile:', err);
    setError('Failed to load profile. Please try again later.');
```

finally {

```

    setIsLoading(false);
  }
};

useEffect(() => {
  fetchProfile();
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [userId, username]);

const handleProfileUpdated = (updatedProfile: UserProfile) => {
  // Optimistic UI update so the dialog closes immediately.
  setProfile(updatedProfile);
  setShowEditDialog(false);

  // Then re-fetch from the backend to ensure we display the persisted state.
  // This prevents fields that aren't persisted server-side from appearing to "stick" until refresh.
  void fetchProfile();
};

const handleFollowToggle = async () => {
  if (!profile) return;
  try {
    if (profile.isFollowing) {
      await followApi.unfollow(profile.id);
    } else {
      await followApi.follow(profile.id);
    }
    await fetchProfile();
  } catch (err) {
    console.error('Failed to toggle follow:', err);
  }
};

const handleMessageClick = () => {
  if (!profile) return;
  // Always navigate to the chat page; the chat UI will show a LinkedIn-style lock
  // when users aren't mutually following.
  navigate(`/chat/${profile.id}`);
};

const handleShareProfile = async () => {
  if (!profile) return;

  const sharePath = profile.username ? `/u/${profile.username}` : `/profile/${profile.id}`;
  const shareUrl = `${window.location.origin}${sharePath}`;

  try {
    if (navigator.clipboard?.writeText) {
      await navigator.clipboard.writeText(shareUrl);
    } else {
      // Fallback for older browsers / restricted contexts.
      const el = document.createElement('textarea');
      el.value = shareUrl;
      el.setAttribute('readonly', '');
      el.style.position = 'fixed';

```

```

        el.style.left = '-9999px';
        document.body.appendChild(el);
        el.select();
        document.execCommand('copy');
        document.body.removeChild(el);
    }
    toast.success('Profile link copied');
} catch (e) {
    console.error('Failed to copy profile link:', e);
    toast.error('Could not copy link. Please copy it manually from the address bar.');
```

```

    }
};
```

```

if (isLoading) {
    return (
        <div className="min-h-screen">
            <Navigation />
            <div className="flex justify-center items-center py-12">
                <Loader2 className="h-8 w-8 animate-spin text-primary" />
            </div>
        </div>
    );
}
```

```

if (error || !profile) {
    return (
        <div className="min-h-screen">
            <Navigation />
            <div className="container mx-auto px-4 py-8 max-w-4xl">
                <div className="text-center py-12">
                    <p className="text-destructive">{error || 'Profile not found'}</p>
                </div>
            </div>
        </div>
    );
}
```

```

return (
    <div className="min-h-screen">
        <Navigation />

        {/ * Banner - User uploaded or default image */}
        <div className="profile-banner h-48 w-full relative">
            <img
                src={profile.bannerImage || '/default-banner.png'}
                alt="Profile banner"
                className="absolute inset-0 w-full h-full object-cover"
            />
        </div>

        <main className="container mx-auto px-4 -mt-24 relative z-10">
            <div className="max-w-6xl mx-auto">
                {/ * Profile Header with Avatar */}
                <div className="text-center mb-8">
                    <Avatar className="h-40 w-40 sm:h-44 sm:w-44 md:h-48 md:w-48 mx-auto border-4 border-background shadow-xl">
                        <AvatarImage src={profile.avatar} alt={profile.name} />
                        <AvatarFallback className="text-4xl">
                            <DefaultAvatarFallback alt={profile.name} />
                        </AvatarFallback>
                    </Avatar>
                </div>

                <h1 className="text-3xl font-bold mt-4 inline-flex items-center justify-center gap-2">
                    {profile.name}
                    <VerifiedBadge show={profile.role === 'ROLE_ADMIN'} size={18} title="Admin" />
                </h1>
                <p className="text-muted-foreground">
                    @{profile.username || profile.email?.split('@')[0]}
                </p>

                {/ * Followers Count */}
                <div className="flex gap-4 justify-center mt-2 text-sm">
```

```

</div>
    <span className="font-semibold text-foreground">{profile.followersCount || 0}</span>
    <span className="text-muted-foreground ml-1">Followers</span>
</div>
</div>
    <span className="font-semibold text-foreground">{profile.followingCount || 0}</span>
    <span className="text-muted-foreground ml-1">Following</span>
</div>
</div>

/* Action Buttons */
<div className="flex gap-3 justify-center mt-4">
    <Button variant="outline" onClick={handleShareProfile}>
        <ExternalLink className="mr-2 h-4 w-4" />
        Share Profile
    </Button>
    {isOwnProfile ? (
        <Button variant="outline" onClick={() => setShowEditDialog(true)}>
            <Edit className="mr-2 h-4 w-4" />
            Edit Profile
        </Button>
    ) : (
        <>
            <Button variant="outline" onClick={handleFollowToggle}>
                <UserPlus className="mr-2 h-4 w-4" />
                {profile.isFollowing ? 'Unfollow' : 'Follow'}
            </Button>
            <Button onClick={handleMessageClick} disabled={!profile.canMessage}>
                <MessageCircle className="mr-2 h-4 w-4" />
                Message
            </Button>
        </>
    )}
</div>
</div>

/* Two Column Layout */
<div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
    /* Left Column - Main Content */
    <div className="lg:col-span-2 space-y-6">
        /* About me */
        {profile.bio && (
            <Card className="gradient-mesh-card">
                <CardHeader>
                    <CardTitle>About me</CardTitle>
                </CardHeader>
                <CardContent>
                    <p className="text-foreground/90 leading-relaxed">{profile.bio}</p>
                </CardContent>
            </Card>
        )}

        /* Skills */
        {profile.skills && profile.skills.length > 0 && (
            <Card className="gradient-mesh-card">
                <CardHeader>
                    <CardTitle>Skills</CardTitle>
                </CardHeader>
                <CardContent>
                    <div className="flex flex-wrap gap-2">
                        {profile.skills.map((skill, index) => (
                            <Badge key={index} className="gradient-label px-3 py-1">
                                {skill}
                            </Badge>
                        ))}
                    </div>
                </CardContent>
            </Card>
        )}

        /* Projects Section */

```

```

{profile.projects && profile.projects.length > 0 && (
  <Card className="gradient-mesh-card">
    <CardHeader>
      <CardTitle>Projects</CardTitle>
    </CardHeader>
    <CardContent>
      <div className="space-y-6">
        {profile.projects.map((project, index) => (
          <div
            key={project.id || index}
            className="border rounded-lg p-5 bg-card hover:shadow-lg transition-shadow"
          >
            {/* Header with Title and Status */}
            <div className="flex items-start justify-between mb-3">
              <div className="flex-1">
                <div className="flex items-center gap-2 mb-1">
                  <h3 className="font-bold text-lg">{project.title}</h3>
                  {project.status && (
                    <Badge className="gradient-label flex items-center gap-1">
                      {String(project.status).toLowerCase() === 'shipped' ? (
                        <CheckCircle className="h-3 w-3" />
                      ) : (
                        <Clock className="h-3 w-3" />
                      )}
                    {project.status}
                  </Badge>
                </div>
                <div>
                  {project.summary && (
                    <p className="text-sm text-muted-foreground italic">{project.summary}</p>
                  )}
                </div>
              </div>
              <div>
                {/* Problem and Solution */}
                <div className="space-y-3 mb-4">
                  {project.problem && (
                    <div>
                      <h4 className="text-xs font-semibold text-muted-foreground uppercase mb-1">Problem</h4>
                      <p className="text-sm text-foreground">{project.problem}</p>
                    </div>
                  )}
                  {project.built && (
                    <div>
                      <h4 className="text-xs font-semibold text-muted-foreground uppercase mb-1">What I Built</h4>
                      <p className="text-sm text-foreground">{project.built}</p>
                    </div>
                  )}
                  {project.role && (
                    <div>
                      <h4 className="text-xs font-semibold text-muted-foreground uppercase mb-1">Role</h4>
                      <p className="text-sm font-medium text-foreground">{project.role}</p>
                    </div>
                  )}
                <div>
                  {!project.problem && !project.built && project.description && (
                    <div>
                      <h4 className="text-xs font-semibold text-muted-foreground uppercase mb-1">Description</h4>
                      <p className="text-sm text-foreground">{project.description}</p>
                    </div>
                  )}
                </div>
              </div>
            </div>
          </div>
        )}
      </div>
      {/* Tech Stack */}
      {project.techStack && project.techStack.length > 0 && (
        <div className="mb-4">
          <h4 className="text-xs font-semibold text-muted-foreground uppercase mb-2">Tech Stack</h4>
          <div className="flex flex-wrap gap-1.5">
            {project.techStack
              .flatMap((t) =>
                String(t)
                  .split(',')

```

```

        .map((s) => s.trim())
        .filter(Boolean),
    )
    .map((tech, techIndex) => (
        <Badge key={`_${tech}-${techIndex}`} className="gradient-label text-xs px-2 py-0.5"
            {tech}
        </Badge>
    ))
    </div>
</div>
)}

{/* Proof Links */}
<div>
    <h4 className="text-xs font-semibold text-muted-foreground uppercase mb-2">Proof</h4>
    <div className="flex flex-wrap gap-2">
        {project.proofLinks?.github && (
            <Button asChild variant="outline" size="sm">
                <a
                    href={project.proofLinks.github}
                    target="_blank"
                    rel="noopener noreferrer"
                    className="flex items-center gap-1.5"
                >
                    <Github className="h-3.5 w-3.5" />
                    Github
                </a>
            </Button>
        )}
        {project.proofLinks?.demo && (
            <Button asChild variant="outline" size="sm">
                <a
                    href={project.proofLinks.demo}
                    target="_blank"
                    rel="noopener noreferrer"
                    className="flex items-center gap-1.5"
                >
                    <ExternalLink className="h-3.5 w-3.5" />
                    Live Demo
                </a>
            </Button>
        )}
        {project.proofLinks?.video && (
            <Button asChild variant="outline" size="sm">
                <a
                    href={project.proofLinks.video}
                    target="_blank"
                    rel="noopener noreferrer"
                    className="flex items-center gap-1.5"
                >
                    <PlayCircle className="h-3.5 w-3.5" />
                    Video
                </a>
            </Button>
        )}
        {!!project.proofLinks?.screenshots?.length && (
            <Button asChild variant="outline" size="sm">
                <a
                    href={project.proofLinks.screenshots[0]}
                    target="_blank"
                    rel="noopener noreferrer"
                    className="flex items-center gap-1.5"
                >
                    <Image className="h-3.5 w-3.5" />
                    Screenshots
                </a>
            </Button>
        )}
        {project.proofLinks?.github && !project.proofLinks?.demo && !project.proofLinks?.video && !project.proofLinks?.screenshots && (
            <Button asChild variant="outline" size="sm">
                <a

```



```

        href={project.link}
        target="_blank"
        rel="noopener noreferrer"
        className="flex items-center gap-1.5"
      >
        <ExternalLink className="h-3.5 w-3.5" />
        Link
      </a>
    </Button>
  )}
  </div>
</div>
</div>
))}
</div>
</CardContent>
</Card>
)}

{/* Experience Section */}
{profile.experience && profile.experience.length > 0 && (
  <Card className="gradient-mesh-card">
    <CardHeader>
      <CardTitle className="flex items-center gap-2">
        <Briefcase className="h-5 w-5" />
        Experience
      </CardTitle>
    </CardHeader>
    <CardContent>
      <div className="space-y-4">
        {profile.experience.map((exp, index) => (
          <div
            key={exp.id || index}
            className="border rounded-lg p-4 bg-card hover:shadow-md transition-shadow"
          >
            <div className="flex-1">
              <h3 className="font-semibold text-lg">{exp.position}</h3>
              <p className="text-sm text-muted-foreground">{exp.company}</p>
            </div>
            <p className="text-sm text-muted-foreground mb-2 mt-1">
              <span className="gradient-link-text">{exp.startDate} - {exp.endDate || 'Present'}</span>
            </p>
            {exp.description && (
              <p className="text-foreground/90">{exp.description}</p>
            )}
          </div>
        ))}
      </div>
    </CardContent>
  </Card>
)}
</div>

{/* Right Column - Info */}
<div className="space-y-6">
  {/* View Resume Section */}
  {profile.resumeUrl && (
    <Card className="gradient-mesh-card">
      <CardContent className="pt-6">
        <Button asChild className="w-full" variant="outline">
          <a
            href={profile.resumeUrl}
            target="_blank"
            rel="noopener noreferrer"
            className="flex items-center justify-center gap-2"
          >
            <FileText className="h-4 w-4" />
            View Resume
          </a>
        </Button>
      </CardContent>
    </Card>
  )}

```

```

    </Card>;
  })

  { /* Availability Section */
  {profile.availability &&
    (profile.availability.openToCollaboration ||
    profile.availability.openToInternships ||
    profile.availability.openToFreelance ||
    profile.availability.justBuilding) && (
    <Card className="gradient-mesh-card">
      <CardHeader>
        <CardTitle className="text-base flex items-center gap-2">
          <Sparkles className="h-4 w-4" />
          Availability
        </CardTitle>
      </CardHeader>
      <CardContent>
        <div className="space-y-2">
          {profile.availability.openToCollaboration && (
            <div className="flex items-center gap-2">
              <CheckCircle className="h-4 w-4 text-green-500" />
              <span className="text-sm">Open to collaboration</span>
            </div>
          )}
          {profile.availability.openToInternships && (
            <div className="flex items-center gap-2">
              <CheckCircle className="h-4 w-4 text-green-500" />
              <span className="text-sm">Open to internships</span>
            </div>
          )}
          {profile.availability.openToFreelance && (
            <div className="flex items-center gap-2">
              <CheckCircle className="h-4 w-4 text-green-500" />
              <span className="text-sm">Open to freelance</span>
            </div>
          )}
          {profile.availability.justBuilding && (
            <div className="flex items-center gap-2">
              <CheckCircle className="h-4 w-4 text-blue-500" />
              <span className="text-sm">Just building</span>
            </div>
          )}
        </div>
      </CardContent>
    </Card>
  })

  { /* Education Section */
  {profile.education && profile.education.length > 0 && (
    <Card className="gradient-mesh-card">
      <CardHeader>
        <CardTitle className="text-base flex items-center gap-2">
          <GraduationCap className="h-4 w-4" />
          Education
        </CardTitle>
      </CardHeader>
      <CardContent className="space-y-4">
        {profile.education.map((edu, index) => (
          <div key={edu.id || index} className="space-y-1">
            <h4 className="font-semibold text-sm">{edu.school}</h4>
            <p className="text-sm text-foreground/90">
              {edu.degree}{edu.field ? ` in ${edu.field}` : ''}
            </p>
            <div className="flex justify-between items-center text-xs">
              {edu.cgpa ? (
                <span className="gradient-link-text">CGPA: {edu.cgpa}</span>
              ) : (
                <span></span>
              )}
              <span className="gradient-link-text">{edu.startYear} - {edu.endYear || 'Present'}</span>
            </div>
          </div>
        )}
      </CardContent>
    </Card>
  )}
  )}

```

```

        </div>
    ))
    </CardContent>
</Card>
})

{/* Contact / basics */}
<Card className="gradient-mesh-card">
    <CardContent className="pt-6 space-y-4">
        {/* Location */}
        {!!profile.location && (
            <div>
                <h3 className="text-sm font-semibold text-muted-foreground mb-2">Location</h3>
                <div className="flex items-center gap-2 text-foreground">
                    <MapPin className="h-4 w-4" />
                    <span>{profile.location}</span>
                </div>
            </div>
        )}

        {/* Website */}
        {profile.website && (
            <div>
                <h3 className="text-sm font-semibold text-muted-foreground mb-2">Website</h3>
                <a
                    href={profile.website}
                    target="_blank"
                    rel="noopener noreferrer"
                    className="flex items-center gap-2 gradient-link-text hover:underline"
                >
                    <Globe className="h-4 w-4" />
                    <span className="truncate">{profile.website}</span>
                    <ExternalLink className="h-3 w-3" />
                </a>
            </div>
        )}

        {/* Portfolio (GitHub) */}
        {profile.github && (
            <div>
                <h3 className="text-sm font-semibold text-muted-foreground mb-2">Portfolio</h3>
                <a
                    href={profile.github}
                    target="_blank"
                    rel="noopener noreferrer"
                    className="flex items-center gap-2 gradient-link-text hover:underline"
                >
                    <Briefcase className="h-4 w-4" />
                    <span className="truncate">@{profile.github.split('/').pop()}</span>
                    <ExternalLink className="h-3 w-3" />
                </a>
            </div>
        )}

        {/* Email */}
        {!!profile.email && (
            <div>
                <h3 className="text-sm font-semibold text-muted-foreground mb-2">Email</h3>
                <p className="flex items-center gap-2 gradient-link-text">
                    <Mail className="h-4 w-4" />
                    <span className="truncate">{profile.email}</span>
                </p>
            </div>
        )}

    </CardContent>
</Card>

{/* Social Media Links */}
{(profile.linkedin || profile.github || profile.twitter) && (
    <Card className="gradient-mesh-card">

```

```

</CardHeader>
    <CardTitle className="text-base">Social Links</CardTitle>
</CardHeader>
<CardContent className="space-y-3">
    { /* LinkedIn */
    {profile.linkedin &&& (
        <a
            href={profile.linkedin}
            target="_blank"
            rel="noopener noreferrer"
            className="flex items-center gap-3 text-foreground hover:text-primary transition-colors group"
        >
            <div className="w-10 h-10 rounded-full bg-[#0A66C2] flex items-center justify-center">
                <Linkedin className="h-5 w-5 text-white" />
            </div>
            <div className="flex-1 min-w-0">
                <p className="text-sm font-semibold">LinkedIn</p>
                <p className="text-xs text-muted-foreground truncate">
                    {profile.linkedin.replace('https://', '').replace('http://', '')}
                </p>
                <ExternalLink className="h-4 w-4 opacity-0 group-hover:opacity-100 transition-opacity" />
            </a>
        </div>

    { /* GitHub */
    {profile.github &&& (
        <a
            href={profile.github}
            target="_blank"
            rel="noopener noreferrer"
            className="flex items-center gap-3 text-foreground hover:text-primary transition-colors group"
        >
            <div className="w-10 h-10 rounded-full bg-[#181717] dark:bg-white flex items-center justify-center">
                <Github className="h-5 w-5 text-white dark:text-black" />
            </div>
            <div className="flex-1 min-w-0">
                <p className="text-sm font-semibold">GitHub</p>
                <p className="text-xs text-muted-foreground truncate">
                    {profile.github.replace('https://', '').replace('http://', '')}
                </p>
                <ExternalLink className="h-4 w-4 opacity-0 group-hover:opacity-100 transition-opacity" />
            </a>
        </div>

    { /* Twitter */
    {profile.twitter &&& (
        <a
            href={profile.twitter}
            target="_blank"
            rel="noopener noreferrer"
            className="flex items-center gap-3 text-foreground hover:text-primary transition-colors group"
        >
            <div className="w-10 h-10 rounded-full bg-[#1DA1F2] flex items-center justify-center">
                <Twitter className="h-5 w-5 text-white" />
            </div>
            <div className="flex-1 min-w-0">
                <p className="text-sm font-semibold">Twitter</p>
                <p className="text-xs text-muted-foreground truncate">
                    {profile.twitter.replace('https://', '').replace('http://', '')}
                </p>
                <ExternalLink className="h-4 w-4 opacity-0 group-hover:opacity-100 transition-opacity" />
            </a>
        </div>
    </CardContent>
</Card>
)}

{ /* Languages Section */

```

```

    {profile.languages && profile.languages.length > 0 && (
      <Card className="gradient-mesh-card">
        <CardHeader>
          <CardTitle className="text-base flex items-center gap-2">
            <Languages className="h-4 w-4" />
            Languages
          </CardTitle>
        </CardHeader>
        <CardContent className="space-y-2">
          {profile.languages.map((lang, index) => (
            <div key={index} className="flex items-center justify-between">
              <span className="text-sm">{lang.language}</span>
              <Badge className="gradient-label">{lang.proficiency}</Badge>
            </div>
          ))}
        </CardContent>
      </Card>
    )}
  </div>
</div>

/* Empty State */
{isOwnProfile && !profile.skills?.length && !profile.projects?.length && !profile.bio
  <Card className="gradient-mesh-card mt-6">
    <CardContent className="pt-6 text-center">
      <p className="text-muted-foreground mb-4">Your profile is empty. Add some information about yourself</p>
      <Button onClick={() => setShowEditDialog(true)}>
        <Edit className="mr-2 h-4 w-4" />
        Edit Profile
      </Button>
    </CardContent>
  </Card>
)}
</div>
</main>

{isOwnProfile && (
  <EditProfileDialog
    open={showEditDialog}
    onOpenChange={setShowEditDialog}
    profile={profile}
    onProfileUpdated={handleProfileUpdated}
  />
)}
</div>
);
}

```

■ frontend/src/pages/Register.tsx

```

import { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { Card, CardContent } from '@/components/ui/card';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import { Alert, AlertDescription } from '@/components/ui/alert';
import { User, Mail, Lock, UserPlus } from 'lucide-react';
import { toast } from 'sonner';

export default function Register() {
  const navigate = useNavigate();
  const [formData, setFormData] = useState({
    displayName: '',
    email: '',
    password: '',
  });
  const [error, setError] = useState<string | null>(null);
  const [isLoading, setIsLoading] = useState(false);

```

```

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setError(null);
  setIsLoading(true);

  // Validation
  if (!formData.displayName.trim()) {
    setError('Please enter your full name');
    setIsLoading(false);
    return;
  }

  if (!formData.email.trim() || !formData.email.includes('@')) {
    setError('Please enter a valid email address');
    setIsLoading(false);
    return;
  }

  if (formData.password.length < 6) {
    setError('Password must be at least 6 characters long');
    setIsLoading(false);
    return;
  }

  try {
    // In real app, would call registration API
    console.log('Registering user:', formData);

    // Simulate API delay
    await new Promise((resolve) => { setTimeout(resolve, 1000) });

    toast.success('Account created successfully!');
    navigate('/setup-username');
  } catch (err) {
    console.error('Registration failed:', err);
    setError('Registration failed. Please try again.');
```

```

  } finally {
    setIsLoading(false);
  }
};
```

```

return (
  <div className="min-h-screen bg-background flex items-center justify-center p-4">
    <Card className="gradient-mesh-card w-full max-w-md">
      <CardContent className="pt-6">
        <div className="text-center mb-6">
          <h1 className="text-3xl font-bold mb-2">Create Account</h1>
          <p className="text-muted-foreground brand-name">
            Join the <span className="allied">Allied</span> <span className="edge">Edge</span>
          </p>
        </div>

        {error && (
          <Alert variant="destructive" className="mb-6">
            <AlertDescription>{error}</AlertDescription>
          </Alert>
        )}

        <form onSubmit={handleSubmit} className="space-y-4">
          <div className="space-y-2">
            <Label htmlFor="displayName">
              <User className="h-4 w-4 inline mr-2" />
              Full Name
            </Label>
            <Input
              id="displayName"
              type="text"
              value={formData.displayName}
              onChange={(e) => setFormData({ ...formData, displayName: e.target.value })}
              placeholder="John Doe"
            />
          </div>
        </form>
      </CardContent>
    </Card>
  </div>
);
```

```

        required
        disabled={isLoading}
      />
    </div>

    {/ * Email */}
    <div className="space-y-2">
      <Label htmlFor="email">
        <Mail className="h-4 w-4 inline mr-2" />
        Email
      </Label>
      <Input
        id="email"
        type="email"
        value={formData.email}
        onChange={(e) => setFormData({ ...formData, email: e.target.value })}
        placeholder="your.email@example.com"
        required
        autoFocus
        disabled={isLoading}
      />
    </div>

    {/ * Password */}
    <div className="space-y-2">
      <Label htmlFor="password">
        <Lock className="h-4 w-4 inline mr-2" />
        Password
      </Label>
      <Input
        id="password"
        type="password"
        value={formData.password}
        onChange={(e) => setFormData({ ...formData, password: e.target.value })}
        placeholder="Enter password (min 6 characters)"
        required
        minLength={6}
        disabled={isLoading}
      />
    </div>

    {/ * Submit Button */}
    <Button type="submit" className="w-full" disabled={isLoading}>
      {isLoading ? (
        'Creating Account...'
      ) : (
        <&gt;
          <UserPlus className="h-4 w-4 mr-2" />
          Register
        </>
      )}
    </Button>
  </form>

  <div className="mt-6 text-center text-sm">
    <p className="text-muted-foreground">
      Already have an account?{' '}
      <Link to="/login" className="text-primary hover:underline font-medium">
        Log in here
      </Link>
    </p>
  </div>
</CardContent>
</Card>
</div>
);
}

```

```

import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Card, CardContent } from '@components/ui/card';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Label } from '@components/ui/label';
import { Alert, AlertDescription } from '@components/ui/alert';
import { AtSign, Check } from 'lucide-react';
import { toast } from 'sonner';

export default function SetupUsername() {
  const navigate = useNavigate();
  const [username, setUsername] = useState('');
  const [error, setError] = useState<string | null>(null);
  const [isLoading, setIsLoading] = useState(false);

  // Mock user data - in real app, get from context/session
  const currentDisplayName = 'User';

  const validateUsername = (value: string): string | null => {
    if (value.length < 3 || value.length > 30) {
      return 'Username must be between 3 and 30 characters';
    }

    const usernamePattern = /^[A-Za-z0-9._-]+$/;
    if (!usernamePattern.test(value)) {
      return 'Username can only contain letters, numbers, dots (.), underscores (_), and hyphens (-)';
    }

    if (value.includes('@')) {
      return 'Username cannot be an email address';
    }

    return null;
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);

    const validationError = validateUsername(username);
    if (validationError) {
      setError(validationError);
      return;
    }

    setIsLoading(true);

    try {
      // In real app, would call API to save username
      console.log('Setting username:', username);

      // Simulate API delay
      await new Promise((resolve) => { setTimeout(resolve, 1000) });

      toast.success('Username saved successfully!');
      navigate('/');
    } catch (err) {
      console.error('Failed to save username:', err);
      setError('Failed to save username. Please try again.');
```



```

        <p className="text-muted-foreground">
          Hi <strong>{currentDisplayName}</strong>, please choose a username that will be visible
          on your posts and comments.
        </p>
      </div>

      {error && (
        <Alert variant="destructive" className="mb-6">
          <AlertDescription>{error}</AlertDescription>
        </Alert>
      )}

      <form onSubmit={handleSubmit} className="space-y-4">
        <div className="space-y-2">
          <Label htmlFor="username">
            <AtSign className="h-4 w-4 inline mr-2" />
            Username
          </Label>
          <Input
            id="username"
            type="text"
            value={username}
            onChange={(e) => {
              setUsername(e.target.value);
              setError(null);
            }}
            placeholder="Enter your username"
            required
            minLength={3}
            maxLength={30}
            disabled={isLoading}
            autoFocus
          />
          <p className="text-sm text-muted-foreground">
            3-30 characters, only letters, numbers, dots (.), underscores (_) and hyphens (-).
            Do not use your email.
          </p>
        </div>

        <Button type="submit" className="w-full" disabled={isLoading}>
          {isLoading ? (
            'Saving...'
          ) : (
            <>
              <Check className="h-4 w-4 mr-2" />
              Save Username
            </>
          )}
        </Button>
      </form>
    </CardContent>
  </Card>
</div>
);
}

```

■ frontend/src/pages/Terms.tsx

```

import { Navigation } from '@components/Navigation';
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import { Link } from 'react-router-dom';
import { FileText, AlertCircle } from 'lucide-react';

export default function Terms() {
  return (
    <div className="min-h-screen bg-background">
      <Navigation />

      <div className="container mx-auto px-4 py-8 max-w-4xl">
        <div className="mb-8">

```

```

    <h1 className="text-3xl font-bold mb-2">Terms & Conditions</h1>
    <p className="text-muted-foreground">
      Last updated: January 6, 2026
    </p>
  </div>

  <div className="space-y-6">
    { /* Introduction */ }
    <Card className="gradient-mesh-card">
      <CardHeader>
        <CardTitle className="flex items-center gap-2">
          <FileText className="h-5 w-5" />
          Welcome to AlliedEdge
        </CardTitle>
      </CardHeader>
      <CardContent className="space-y-3">
        <p>
          By creating an account and using AlliedEdge, you agree to follow these terms.
          These rules help us maintain a safe, respectful, and productive community for
          all students and developers.
        </p>
        <p className="text-sm text-muted-foreground">
          If you don't agree with these terms, please don't use our platform.
        </p>
      </CardContent>
    </Card>

    { /* Acceptance of Terms */ }
    <Card className="gradient-mesh-card">
      <CardHeader>
        <CardTitle>1. Acceptance of Terms</CardTitle>
      </CardHeader>
      <CardContent className="space-y-3">
        <p>
          When you sign up for AlliedEdge, you're agreeing to:
        </p>
        <ul className="space-y-2 text-sm ml-4">
          <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Follow all the rules outlined in these Terms & Conditions</span>
          </li>
          <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Respect our Community Guidelines and Privacy Policy</span>
          </li>
          <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Be at least 13 years old (or the minimum age in your country)</span>
          </li>
          <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Provide accurate information when creating your account</span>
          </li>
        </ul>
      </CardContent>
    </Card>

    { /* User Responsibilities */ }
    <Card className="gradient-mesh-card">
      <CardHeader>
        <CardTitle>2. Your Responsibilities</CardTitle>
      </CardHeader>
      <CardContent className="space-y-4">
        <div>
          <h3 className="font-semibold mb-2">Account Security</h3>
          <ul className="space-y-2 text-sm ml-4">
            <li className="flex items-start gap-2">
              <span className="text-muted-foreground mt-1">•</span>
              <span>Keep your password secure and don't share it with anyone</span>
            </li>
            <li className="flex items-start gap-2">

```

```

        <span className="text-muted-foreground mt-1">•</span>
        <span>You're responsible for all activity on your account</span>
    </li>
    <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Notify us immediately if your account is compromised</span>
    </li>
</ul>
</div>

<div>
    <h3 className="font-semibold mb-2">Respectful Behavior</h3>
    <ul className="space-y-2 text-sm ml-4">
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Treat everyone with respect and kindness</span>
        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>No harassment, bullying, hate speech, or discrimination</span>
        </li>
        <li className="flex items-start gap-2">
            <span className="text-muted-foreground mt-1">•</span>
            <span>Constructive criticism is welcome, but stay professional</span>
        </li>
    </ul>
</div>
</CardContent>
</Card>

{/* Acceptable Use */}
<Card className="gradient-mesh-card">
    <CardHeader>
        <CardTitle>3. Acceptable Use</CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
        <p className="font-semibold">You may use AlliedEdge to:</p>
        <ul className="space-y-2 text-sm ml-4">
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Share tech-related projects, tutorials, and learning resources</span>
            </li>
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Ask questions and help others with their coding problems</span>
            </li>
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Network with students and developers</span>
            </li>
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Collaborate on projects and ideas</span>
            </li>
        </ul>

        <p className="font-semibold mt-4">You may NOT:</p>
        <ul className="space-y-2 text-sm ml-4">
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Post spam, misleading content, or excessive self-promotion</span>
            </li>
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Share harmful, illegal, or inappropriate content</span>
            </li>
            <li className="flex items-start gap-2">
                <span className="text-muted-foreground mt-1">•</span>
                <span>Plagiarize or steal others' work without credit</span>
            </li>
            <li className="flex items-start gap-2">

```

```

        <span className="text-muted-foreground mt-1">•</span>
        <span>Attempt to hack, exploit, or damage the platform</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Use bots, scrapers, or automated tools without permission</span>
      </li>
    </ul>
  </CardContent>
</Card>

{/* Account Authenticity */}
<Card className="gradient-mesh-card">
  <CardHeader>
    <CardTitle>4. Account Authenticity</CardTitle>
  </CardHeader>
  <CardContent className="space-y-3">
    <p>To maintain trust in our community:</p>
    <ul className="space-y-2 text-sm ml-4">
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Create only one personal account</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Use your real name or a recognizable username</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Don't impersonate others or create fake profiles</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Don't create accounts for malicious purposes</span>
      </li>
    </ul>
  </CardContent>
</Card>

{/* Content Ownership */}
<Card className="gradient-mesh-card">
  <CardHeader>
    <CardTitle>5. Content Ownership & Rights</CardTitle>
  </CardHeader>
  <CardContent className="space-y-4">
    <div>
      <h3>Your Content</h3>
      <p>You own everything you post on AlliedEdge. We don't claim ownership of your posts, projects, or code. However, by posting on our platform, you give us permission to display, store, and share your content within the platform.</p>
    </div>

    <div>
      <h3>Platform Rights</h3>
      <p>AlliedEdge reserves the right to remove content that violates these terms or our community guidelines. We may also use anonymous, aggregated data to improve our services.</p>
    </div>

    <div>
      <h3>Respecting Others' Work</h3>
      <p>Always give credit when sharing someone else's work. Don't post copyrighted material without permission.</p>
    </div>
  </CardContent>
</Card>

```

```

</Card>

{/* Consequences */}
<Card className="gradient-mesh-card">
  <CardHeader>
    <CardTitle className="flex items-center gap-2">
      <AlertCircle className="h-5 w-5" />
      6. Consequences of Violations
    </CardTitle>
  </CardHeader>
  <CardContent className="space-y-3">
    <p>
      If you violate these terms or our community guidelines, we may:
    </p>
    <ul className="space-y-2 text-sm ml-4">
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Issue a warning</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Remove specific posts or content</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Temporarily suspend your account</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-muted-foreground mt-1">•</span>
        <span>Permanently ban your account for serious or repeated violations</span>
      </li>
    </ul>
    <p className="text-sm text-muted-foreground">
      We'll always try to be fair and give you a chance to improve, but we prioritize
      keeping our community safe.
    </p>
  </CardContent>
</Card>

{/* Changes to Terms */}
<Card className="gradient-mesh-card">
  <CardHeader>
    <CardTitle>7. Changes to These Terms</CardTitle>
  </CardHeader>
  <CardContent className="space-y-3">
    <p className="text-sm">
      We may update these terms from time to time. When we do, we'll notify you through
      the platform or via email. Continuing to use AlliedEdge after changes means you
      accept the new terms.
    </p>
  </CardContent>
</Card>

{/* Disclaimer */}
<Card className="gradient-mesh-card">
  <CardHeader>
    <CardTitle>8. Disclaimer</CardTitle>
  </CardHeader>
  <CardContent className="space-y-3">
    <p className="text-sm text-muted-foreground">
      AlliedEdge is provided "as is" without warranties. We strive to keep the platform
      running smoothly, but we can't guarantee it will always be available or error-free.
      We're not responsible for any loss or damage resulting from your use of the platform.
    </p>
  </CardContent>
</Card>

{/* Contact */}
<Card className="gradient-mesh-card">
  <CardHeader>
    <CardTitle>Questions About These Terms</CardTitle>

```

```

    </CardHeader>
    <CardContent className="space-y-3">
      <p className="text-sm">
        If you have questions or concerns about these Terms & Conditions, please contact us at:
      </p>
      <p className="text-sm">
        <a href="mailto:legal@alliededge.com" className="text-primary hover:underline">
          legal@alliededge.com
        </a>
      </p>
    </CardContent>
  </Card>
</div>

<div className="mt-8 text-center">
  <Link to="/" className="text-sm text-primary hover:underline">
    ← Back to Feed
  </Link>
</div>
</div>
</div>
);
}

```

■ frontend/src/pages/UserSearch.tsx

```

import { useState, useEffect } from 'react';
import { useSearchParams, Link } from 'react-router-dom';
import { Navigation } from '@components/Navigation';
import { Card, CardContent } from '@components/ui/card';
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Loader2, Search, FileText, Users } from 'lucide-react';
import { DefaultAvatarFallback } from '@components/DefaultAvatarFallback';
import { usersApi, type User as ApiUser } from '@lib/api';
import { useAuthContext } from '@contexts/AuthContext';

interface SearchUser {
  id: number;
  username: string;
  displayName?: string;
  profileImageUrl?: string;
  resumeUrl?: string;
  resumePublicId?: string;
}

export default function UserSearch() {
  const { user: currentUser } = useAuthContext();
  const currentUserId = currentUser?.id;

  const [searchParams, setSearchParams] = useSearchParams();
  const keyword = searchParams.get('keyword') || '';
  const [searchInput, setSearchInput] = useState(keyword);
  const [users, setUsers] = useState<SearchUser[]>([]);
  const [isLoading, setIsLoading] = useState(false);

  useEffect(() => {
    if (keyword) {
      searchUsers(keyword);
    }
  }, [keyword]);

  const searchUsers = async (query: string) => {
    if (!query.trim()) return;

    setIsLoading(true);
    try {
      const resp = await usersApi.search(query.trim());
      const rawUsers = (resp.users || []) as ApiUser[];
    }
  }

```

```

// Backend should already exclude current user, but filter client-side as defense-in-depth.
const filtered = (rawUsers as any[]).filter((u) => {
  const id = (u?.id as number | undefined) ?? undefined;
  return currentUserId == null || id == null || id !== currentUserId;
});

setUsers(filtered as SearchUser[]);
} catch (error) {
  console.error('Failed to search users:', error);
  setUsers([]);
} finally {
  setIsLoading(false);
}
};

const handleSearch = (e: React.FormEvent) => {
  e.preventDefault();
  if (searchInput.trim()) {
    setSearchParams({ keyword: searchInput.trim() });
  }
};

return (
  <div className="min-h-screen bg-background">
    <Navigation />

    <main className="container mx-auto px-4 py-6 max-w-3xl">
      <div className="mb-6">
        <h1 className="text-3xl font-bold mb-4 flex items-center gap-2">
          <Users className="h-8 w-8 text-primary" />
          User Search
        </h1>

        {/* Search Form */}
        <form onSubmit={handleSearch} className="flex gap-2">
          <input
            type="text"
            value={searchInput}
            onChange={(e) => setSearchInput(e.target.value)}
            placeholder="Search users by name or username..."
            className="flex-1"
          />
          <button type="submit" className="search-button">
            Search
          </button>
        </form>
      </div>

      {keyword && (
        <div className="mb-4">
          <p className="text-muted-foreground">
            Search results for: <strong>{keyword}</strong>
          </p>
        </div>
      )}

      {isLoading ? (
        <div className="flex justify-center items-center py-12">
          <Loader2 className="h-8 w-8 animate-spin text-primary" />
        </div>
      ) : users.length === 0 && keyword ? (
        <Card className="gradient-mesh-card">
          <CardContent className="py-12 text-center">
            <p className="text-muted-foreground">No users found matching your search.</p>
          </CardContent>
        </Card>
      ) : users.length === 0 ? (
        <Card className="gradient-mesh-card">
          <CardContent className="py-12 text-center">
            <Search className="h-12 w-12 mx-auto mb-4 text-muted-foreground" />

```

```

        <p className="text-muted-foreground">
            Enter a name or username to search for users
        </p>
    </CardContent>
</Card>
) : (
    <div className="space-y-3">
        {users.map((user) => (
            <Card key={user.id} className="gradient-mesh-card hover:shadow-lg transition-shadow">
                <CardContent className="py-4">
                    <div className="flex items-center gap-4">
                        <Link to={` /profile/${user.id}`}>
                            <Avatar className="h-12 w-12 border-2 border-border">
                                <AvatarImage src={user.profileImageUrl} />
                                <AvatarFallback>
                                    <DefaultAvatarFallback alt={user.displayName || user.username} />
                                </AvatarFallback>
                            </Avatar>
                            <Link>
                                <div className="flex-1 min-w-0">
                                    <Link
                                        to={` /profile/${user.id}`}
                                        className="text-lg font-semibold hover:text-primary transition-colors"
                                    >
                                        {user.displayName || user.username}
                                    </Link>
                                    <p className="text-sm text-muted-foreground">@{user.username}</p>
                                </div>
                                { /* Resume Button */ }
                            </Link>
                        </div>
                    </CardContent>
                </Card>
            </div>
        ))}
    </div>
</main>
</div>
);
}

```

■ frontend/src/types/global.d.ts

```

declare global {
    interface Window {
        /**
         * Navigate to the auth page with a custom redirect URL
         * @param redirectUrl - URL to redirect to after successful authentication
         */
        navigateToAuth: (redirectUrl: string) => void;
    }
}

export {};

```

■ frontend/src/types/sockjs-client.d.ts

```

declare module 'sockjs-client';

```

■ frontend/src/vite-env.d.ts

```
/// <reference types="vite/client" />
```

■ frontend/sst-env.d.ts

```
/* This file is auto-generated by SST. Do not edit. */
/* tslint:disable */
/* eslint-disable */
/* deno-fmt-ignore-file */

/// <reference path="../../sst-env.d.ts" />

import "sst"
export {}
```

■ frontend/tsconfig.app.json

```
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo",
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": false,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",

    /* Linting */
    "strict": true,
    "noUnusedLocals": false,
    "noUnusedParameters": false,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedSideEffectImports": true,
    "noPropertyAccessFromIndexSignature": false,
    "baseUrl": ".",
    "paths": {
      "@/*": ["/src/*"]
    }
  },
  "include": ["src"]
}
```

■ frontend/tsconfig.json

```
{
  "files": [],
  "references": [
    { "path": "./tsconfig.app.json" },
    { "path": "./tsconfig.node.json" }
  ],
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["/src/*"]
    }
  }
}
```

```
}
```

■ frontend/tsconfig.node.json

```
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.node.tsbuildinfo",
    "target": "ES2022",
    "lib": ["ES2023"],
    "module": "ESNext",
    "skipLibCheck": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": false,
    "moduleDetection": "force",
    "noEmit": true,

    /* Linting */
    "strict": true,
    "noUnusedLocals": false,
    "noUnusedParameters": false,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedSideEffectImports": true
  },
  "include": ["vite.config.ts"]
}
```

■ frontend/vercel.json

```
{
  "rewrites": [
    {
      "source": "/robots.txt",
      "destination": "/robots.txt"
    },
    {
      "source": "/sitemap.xml",
      "destination": "/sitemap.xml"
    },
    {
      "source": "/(.*)",
      "destination": "/"
    }
  ]
}
```

■ frontend/vite.config.ts

```
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import path from "path";
import { defineConfig } from "vite";

const backendTarget = process.env.VITE_BACKEND_TARGET || "http://localhost:8080";

export default defineConfig(({ mode }) => {
  return {
    plugins: [react(), tailwindcss()],
    resolve: {
      alias: {
        "@": path.resolve(__dirname, "./src"),
      },
    },
  },
});
```

```

server: {
  proxy: {
    '/api': {
      target: backendTarget,
      changeOrigin: true,
      secure: false,
    },
    '/ws': {
      target: backendTarget,
      ws: true,
      changeOrigin: true,
      secure: false,
    },
    // OAuth2 login flows are served by the backend, not the Vite dev server.
    "/oauth2": {
      target: backendTarget,
      changeOrigin: true,
    },
    "/login/oauth2": {
      target: backendTarget,
      changeOrigin: true,
    },
    // Some Spring Security endpoints (e.g., form login) live under /login
    "/login": {
      target: backendTarget,
      changeOrigin: true,
    },
  },
},
};
});

```

■ frontend/vly-toolbar-readonly.tsx

// DO NOT MODIFY THIS FILE. THIS FILE IS READ-ONLY. CONTAINS ALL KEY APP FUNCTIONALITY.

```

import React, { useState, useRef, useCallback } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { MousePointer, ExternalLink, AlertTriangle } from "lucide-react";

import { snapdom } from "@zumer/snapdom";

// FiberNode and ComponentInfo interfaces
export interface FiberNode {
  displayName?: string;
  name?: string;
  tag: number;
  type: unknown;
  return: FiberNode | null;
  _debugOwner?: {
    name: string;
    env: string;
  };
};

}

const FunctionComponent = 0;
const ClassComponent = 1;
const HostComponent = 5;

export interface ComponentInfo {
  name: string;
  type: "regular" | "rsc";
}

export function getReactComponentHierarchy(
  element: HTMLElement | null,
): ComponentInfo[] | null {
  if (!element) {
    return null;
  }
}

```

```

const components: ComponentInfo[] = [];
const maxComponents = 3;

// Find the internal React Fiber node key.
const fiberKey = Object.keys(element).find(
  (key) => {
    key.startsWith("__reactFiber$") ||
    key.startsWith("__reactInternalInstance$"),
  }
);

if (!fiberKey) {
  return null;
}

let currentFiber: FiberNode | null = (
  element as unknown as Record<string, unknown>
)[fiberKey] as FiberNode | null;

if (!currentFiber) {
  return null;
}

// Traverse up the Fiber tree.
while (currentFiber && components.length < maxComponents) {
  let componentData: ComponentInfo | null = null;

  if (
    currentFiber.tag === ClassComponent ||
    currentFiber.tag === FunctionComponent
  ) {
    const componentDefinition = currentFiber.type;
    if (componentDefinition) {
      const def = componentDefinition as {
        displayName?: string;
        name?: string;
      };
      const name =
        def.displayName ||
        def.name ||
        currentFiber._debugOwner?.name ||
        "AnonymousComponent";
      componentData = { name, type: "regular" };
    }
  } else if (
    currentFiber.tag === HostComponent &&
    currentFiber._debugOwner &&
    currentFiber._debugOwner.env?.toLowerCase().includes("server")
  ) {
    componentData = { name: currentFiber._debugOwner.name, type: "rsc" };
  }

  if (componentData) {
    const alreadyExists = components.some(
      (c) => c.name === componentData!.name && c.type === componentData!.type,
    );
    if (!alreadyExists) {
      components.push(componentData);
    }
  }
  currentFiber = currentFiber.return;
}

return components.length > 0 ? components : null;
}

export function formatReactComponentHierarchy(
  hierarchy: ComponentInfo[] | null,
): string {
  if (!hierarchy || hierarchy.length === 0) {
    return "No React components found for this element.";
  }
}

```

```

const parts = hierarchy.map(
  (info) => `{name: ${info.name}, type: ${info.type}}`,
);

let description = `React component tree (from closest to farthest, ${hierarchy.length} closest element${hierarchy.length > 1 ? 's' : ''}
description += parts.join(" child of ");

return description;
}

export function getSelectedElementAnnotation(element: HTMLElement) {
  const hierarchy = getReactComponentHierarchy(element);
  if (hierarchy?.[0]) {
    return {
      annotation: `${hierarchy[0].name}${hierarchy[0].type === "rsc" ? " (RSC)" : ""}`,
    };
  }
  return { annotation: null };
}

export function getSelectedElementsPrompt(elements: HTMLElement[]) {
  const selectedComponentHierarchies = elements.map((e) => {
    getReactComponentHierarchy(e),
  });

  if (selectedComponentHierarchies.some((h) => h && h.length > 0)) {
    const content = `This is additional information on the elements that the user selected. Use this information to find the element you want to edit.

    ${selectedComponentHierarchies.map((h, index) => {
      return `
      <element index=${index + 1}>
      ${h[0] || h.length === 0 ? "No React component as parent detected" : `React component tree (from closest to farthest, ${h[0].length} closest element${h[0].length > 1 ? 's' : ''}
      </element>
    `;
    })}
  `;

    return content;
  }

  return null;
}

const HIGHLIGHT_CLASS = "app-toolbar-highlight";

// Add highlight style to the document
const injectHighlightStyle = () => {
  if (document.getElementById("app-toolbar-style")) return;
  const style = document.createElement("style");
  style.id = "app-toolbar-style";
  style.innerHTML = `
    .${HIGHLIGHT_CLASS} {
      outline: 2px solid #0070f3;
      cursor: pointer;
      background: rgba(0,112,243,0.08);
    }
  `;
  document.head.appendChild(style);
};

function getDomSelector(el: HTMLElement): string {
  if (el.id) return `#${el.id}`;
  if (el.className && typeof el.className === "string") {
    return (
      el.tagName.toLowerCase() + "." + el.className.trim().replace(/\s+/g, ".")
    );
  }
  return el.tagName.toLowerCase();
}

const Overlay: React.FC<{

```

```

toolbarRef: React.RefObject<HTMLDivElement | null>;
onSelect: (el: HTMLElement) => void;
onHover: (el: HTMLElement) => void;
onUnhover: () => void;
selectMode: boolean;
}> = ({ toolbarRef, onSelect, onHover, onUnhover, selectMode }) => {
  const lastHovered = useRef<HTMLElement | null>(null);

  const handleMouseMove = useCallback(
    (e: React.MouseEvent<HTMLDivElement>) => {
      if (!selectMode) return;
      const overlay = e.currentTarget;
      overlay.style.pointerEvents = "none";
      const el = document.elementFromPoint(
        e.clientX,
        e.clientY,
      ) as HTMLElement | null;
      overlay.style.pointerEvents = "auto";

      // Build ignore list from toolbar ref
      const shouldIgnore = el && toolbarRef.current && (el === toolbarRef.current || toolbarRef.current
      if (!el || shouldIgnore) return;

      if (lastHovered.current !== el) {
        if (lastHovered.current)
          lastHovered.current.classList.remove(HIGHLIGHT_CLASS);
        lastHovered.current = el;
        el.classList.add(HIGHLIGHT_CLASS);
        onHover(el);
      }
    },
    [toolbarRef, onHover, selectMode],
  );

  const handleMouseLeave = useCallback(() => {
    if (!selectMode) return;
    if (lastHovered.current) {
      lastHovered.current.classList.remove(HIGHLIGHT_CLASS);
      lastHovered.current = null;
    }
    onUnhover();
  }, [onUnhover, selectMode]);

  const handleClick = useCallback(
    (e: React.MouseEvent<HTMLDivElement>) => {
      e.preventDefault();
      e.stopPropagation();
      const overlay = e.currentTarget;
      overlay.style.pointerEvents = "none";
      const el = document.elementFromPoint(
        e.clientX,
        e.clientY,
      ) as HTMLElement | null;
      overlay.style.pointerEvents = "auto";

      // Build ignore list from toolbar ref
      const shouldIgnore = el && toolbarRef.current && (el === toolbarRef.current || toolbarRef.current
      if (!el || shouldIgnore) return;

      if (lastHovered.current)
        lastHovered.current.classList.remove(HIGHLIGHT_CLASS);
      onSelect(el);
    },
    [toolbarRef, onSelect],
  );

  return (
    <div
      style={{
        position: "fixed",
        inset: 0,

```

```

        zIndex: 9998,
        cursor: "copy",
        pointerEvents: "auto",
    }}
    onMouseMove={handleMouseMove}
    onMouseLeave={handleMouseLeave}
    onClick={handleClick}
    tabIndex={0}
    role="button"
  />
);
};

export const VlyToolbar: React.FC = () => {
  const [selectMode, setSelectMode] = useState(false);
  const toolbarRef = useRef<HTMLDivElement | null>(null);
  const [showDevOverlay, setShowDevOverlay] = React.useState(true);

  const isDevDeployment =
    typeof window !== "undefined" &&
    window.location.hostname.endsWith(".vly.sh") &&
    window.self === window.top;

  React.useEffect(() => {
    injectHighlightStyle();
  }, []);

  // Listen for postMessage from parent to enable/disable select mode
  React.useEffect(() => {
    function handleMessage(event: MessageEvent) {
      if (event.data && event.data.type === "app-toolbar-enable-select") {
        setSelectMode(true);
      }
      if (event.data && event.data.type === "app-toolbar-disable-select") {
        setSelectMode(false);
      }
    }
    window.addEventListener("message", handleMessage);
    return () => window.removeEventListener("message", handleMessage);
  }, []);

  // Callbacks for overlay
  const handleSelect = useCallback(async (el: HTMLElement) => {
    setSelectMode(false);
    const selector = getDomSelector(el);
    const hierarchy = getReactComponentHierarchy(el);
    const formatted = formatReactComponentHierarchy(hierarchy);

    let imageUrl: string | undefined = undefined;
    try {
      // Optionally, preload resources for best results
      // await preCache(el);
      const canvas = await snapdom.toCanvas(el, { fast: true });
      imageUrl = canvas.toDataURL("image/png");
    } catch (e) {
      console.error("Failed to snapshot element", e);
    }

    window.parent.postMessage(
      {
        type: "app-toolbar-select",
        selector,
        reactHierarchy: hierarchy,
        reactHierarchyFormatted: formatted,
        image: imageUrl,
      },
      "*"
    );
  }, []);

  const handleHover = useCallback(() => {

```

```

    // Optionally do something on hover
  }, []);

  const handleUnhover = useCallback(() => {
    // Optionally do something on unhover
  }, []);

  // Handle escape key to close modal
  React.useEffect(() => {
    const handleKeyDown = (e: KeyboardEvent) => {
      if (e.key === 'Escape' && showDevOverlay) {
        setShowDevOverlay(false);
      }
    };

    if (showDevOverlay) {
      document.addEventListener('keydown', handleKeyDown);
      return () => document.removeEventListener('keydown', handleKeyDown);
    }
  }, [showDevOverlay]);

  const handleOverlayClick = (e: React.MouseEvent<HTMLDivElement>) => {
    // Only dismiss if clicking the overlay itself, not the modal content
    if (e.target === e.currentTarget) {
      setShowDevOverlay(false);
    }
  };

  const handleContinue = () => {
    setShowDevOverlay(false);
  };

  // Publish navigation removed.
  const handleGoToProject = () => {
    // Intentionally left blank.
  };

  return (
    <>
    { isDevDeployment && showDevOverlay && (
      <div
        onClick={handleOverlayClick}
        role="dialog"
        aria-modal="true"
        aria-labelledby="dev-env-label"
        style={{
          position: "fixed",
          inset: 0,
          zIndex: 99999,
          backdropFilter: "blur(8px)",
          WebkitBackdropFilter: "blur(8px)",
          backgroundColor: "rgba(0, 0, 0, 0.1)",
          display: "flex",
          alignItems: "center",
          justifyContent: "center",
          cursor: "pointer",
        }}
      >
        <AnimatePresence>
          <motion.div
            onClick={(e) => e.stopPropagation()}
            initial={{ opacity: 0, scale: 0.9 }}
            animate={{ opacity: 1, scale: 1 }}
            exit={{ opacity: 0, scale: 0.9 }}
            transition={{ duration: 0.3, ease: "easeInOut" }}
            style={{
              position: "relative",
              cursor: "default",
            }}
          >

```



```

    <div className="flex items-center gap-3 rounded-lg border border-white/30 bg-white/95 px-4 py-3 shadow
    <div className="flex items-center gap-2 rounded px-2 py-1 transition-colors hover:bg-white/95">
      <div className="flex items-center gap-2">
        <AlertTriangle className="h-4 w-4 text-orange-500" />
        <span
          id="dev-env-label"
          className="text-sm font-medium text-gray-700"
          title="Development deployment - should not be shared publicly. Use 'Go to project' to create a pr
        >
        This is a testing environment. Share only your published project.
      </span>
    </div>
  </div>
  <div className="h-6 w-px bg-gray-300"></div>
  <button
    onClick={handleContinue}
    className="flex items-center gap-2 rounded px-2 py-1 transition-colors hover:bg-white/95"
  >
    <MousePointer className="h-4 w-4 text-gray-600" />
    <span className="text-sm font-medium text-gray-700">
      Click to test
    </span>
  </button>
  <div className="h-6 w-px bg-gray-300"></div>
  <button
    onClick={(e) => {
      e.stopPropagation();
      handleGoToProject();
    }}
    className="flex items-center gap-2 rounded px-2 py-1 transition-colors hover:bg-white/95"
  >
    <ExternalLink className="h-4 w-4 text-gray-600" />
    <span className="text-sm font-medium text-gray-700">
      Publish Project
    </span>
  </button>
  </div>
  </motion.div>
  </AnimatePresence>
</div>
)}
{selectMode &amp; (
  <Overlay
    toolbarRef={toolbarRef}
    onSelect={handleSelect}
    onHover={handleHover}
    onUnhover={handleUnhover}
    selectMode={selectMode}
  />
)}
{/* Hide toolbar if there are no buttons or controls to show */}
{/* If toolbar has no children, render nothing */}
{/* <div ref={toolbarRef} ... /> is now hidden if empty */}
{/* If you add controls in the future, restore this div */}
</>
);
};

export default VlyToolbar;

```

■ LICENSE

Copyright (c) 2026 Rayan Mohammed Rafeeq

All rights reserved.

This software and associated source code are the exclusive property of the author. No permission is granted to use, copy, modify, merge, publish, distribute, sublicense, or sell copies of this software.

■ README.md

```
<p>
  
</p>

<h1>AlliedEdge</h1>

A full-stack social platform built with a Spring Boot (Java 17) API and a React (Vite + TypeScript) SPA, featuring:

- Live demo: https://alliededge.app

> Note: The backend is hosted on Render and may cold start after inactivity. The first request can take up to 30s.

## Overview

- This repo contains both the backend API (Spring Boot) and the frontend SPA (React/Vite).
- Focus areas: auth (Google OAuth), real-time WebSockets, data modeling with JPA/Hibernate, Flyway migrations.
- Deployment: Frontend on Vercel, Backend on Render, Postgres on Fly.io.

> If you'd like a guided walkthrough, start with the Highlights section and the Demo video below.

## Highlights

- Google OAuth2 login (backend-driven) via Spring Security
- Real-time chat using WebSockets (STOMP/SockJS on the client)
- Postgres + Flyway migrations (repeatable, production-friendly)
- Media uploads via Cloudinary
- SPA frontend with modern UI stack (Tailwind + Radix UI)
- Deployed: Frontend on Vercel, Backend on Render
- Dev-friendly: Vite proxying to backend (/api, /ws, OAuth routes)

## Tech stack

**Frontend**
- React + TypeScript (Vite)
- Tailwind CSS
- Radix UI
- Axios
- WebSockets: @stomp/stompjs + sockjs-client

**Backend**
- Java 17, Spring Boot
- Spring Security (OAuth2 Client)
- Spring Web, Validation
- Spring Data JPA (Hibernate)
- Flyway migrations
- WebSocket
- JWT support

**Infrastructure**
- Postgres 16 (Docker)
- Docker (multi-stage build for backend)

## Architecture (high level)

- Frontend (Vercel) serves the SPA.
- Backend (Render) runs the Spring Boot API and exposes:
  - REST endpoints (commonly under /api)
  - WebSocket endpoint(s) (proxied under /ws in dev)
  - OAuth endpoints (e.g. /oauth2/authorization/google)
- Database (Postgres) stores app data; schema is managed via Flyway.
- Cloudinary stores uploaded media.

## Repo layout

- backend/ - Spring Boot API + auth + realtime WebSocket endpoints
- frontend/ - Vite/React SPA
- docker-compose.yml - Postgres + backend containers
```

Quick links

- **Live site:** <https://alliededge.app>
- **Google OAuth flow details:** ``frontend/docs/google-login.md``

Usage & academic integrity

This repository is public **for recruiter/hiring-manager review**.

- **Do not copy, clone, fork, or reuse** this project (in whole or in part) for coursework, assignments, hackathons, or
- **Do not re-upload** the source or deploy it as your own (including "portfolio" clones).
- You may **read the code** to evaluate my work. If you want to reuse any portion, please **request permission first**.

In addition to the note above, the project is licensed as **proprietary / all rights reserved** (see **License**).

Local development (for evaluation)

Prerequisites

- **Java 17** (Temurin/OpenJDK recommended)
- **Maven** (or use ``backend/mvnw.cmd``)
- **Node.js** (LTS recommended)
- **pnpm** (repo uses ``pnpm-lock.yaml``)
- **Docker Desktop** (recommended; easiest way to run Postgres)

1) Start Postgres + Backend (Docker)

This starts **Postgres** and the **backend**.

1) Create a ``.env`` in the repo root (same folder as ``docker-compose.yml``).

Example:

```
```.env
Postgres
POSTGRES_DB=ae_blog_prod_db
POSTGRES_USER=allied_edge_user
POSTGRES_PASSWORD=change-me

Google OAuth2 (backend-driven login)
GOOGLE_CLIENT_ID=your-google-client-id
GOOGLE_CLIENT_SECRET=your-google-client-secret

Cloudinary (required for uploads)
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret

JWT
APP_JWT_SECRET=replace-with-a-long-random-secret
Optional
APP_JWT_EXPIRATION=86400

CORS / redirect targets (optional)
APP_CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
APP_FRONTEND_REDIRECT_URL=http://localhost:5173/
```.
```

2) Start services:

```
```.bash
docker compose up --build
```.

- Backend: http://localhost:8080
- Postgres: `localhost:5432`
```

2) Run the frontend (local dev)

In a second terminal:

```
```.bash
```

```
cd frontend
pnpm install
pnpm dev
````
```

Frontend: <http://localhost:5173>

Connecting the frontend to a different backend

Vite proxies API/WebSocket calls to a backend target.

- `VITE\_BACKEND\_TARGET` (defaults to `http://localhost:8080`)

Example (PowerShell):

```
```powershell
$env:VITE_BACKEND_TARGET="http://localhost:8080"
pnpm dev
````
```

Run the backend without Docker (optional)

If you want to run the backend on your machine, you can point it at:

- a local Postgres instance, or
- the Postgres container from `docker compose`.

From `backend/`:

```
```bash
mvnw.cmd spring-boot:run
````
```

The backend reads configuration from environment variables (see `.env` example above). The datasource also has a local

Authentication (Google OAuth2)

This app uses **backend-driven** Google OAuth (Spring Security).

- Login is initiated by navigating the browser to:
 - Local dev: `http://localhost:8080/oauth2/authorization/google`
 - Production: `https://api.alliededge.app/oauth2/authorization/google`
- The backend completes the OAuth flow and redirects back to the frontend.

More details:

- `frontend/docs/google-login.md`

Uploads (Cloudinary)

Cloudinary credentials are required when starting the backend.

Environment variables:

- `CLOUDINARY\_CLOUD\_NAME`
- `CLOUDINARY\_API\_KEY`
- `CLOUDINARY\_API\_SECRET`

If they're missing, the backend will fail fast at startup.

Scripts

Frontend

- `pnpm dev` – start Vite dev server
- `pnpm build` – typecheck + production build
- `pnpm lint` – eslint
- `pnpm preview` – preview production build

Backend

- `mvnw.cmd test` – run tests

- `mvnw.cmd spring-boot:run` – run locally
- `mvnw.cmd package` – build jar

Troubleshooting

- **CORS issues**: configure `APP\_CORS\_ALLOWED\_ORIGINS` to include your frontend URL.
- **Google login redirect mismatch**: ensure the Google OAuth client has the correct redirect URI:
 - `http://localhost:8080/login/oauth2/code/google`
- **Cookies not sticking in production**: backend is configured for `SameSite=None; Secure` and proxy headers; ensure y

Demo Video

<p>

</p>

License

All rights reserved. This project is proprietary.

- You are **not** granted permission to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies o
- For permission requests (e.g., recruiter evaluation, interview take-home, or other authorized review), contact me via

See [`LICENSE`](LICENSE).
